

Voltus IC Power Integrity Solution Text Command Reference

**Product Version 26.10
May 2026**

© 2026 Cadence Design Systems, Inc. All rights reserved.
Cadence Design Systems, Inc. All rights reserved.
Printed in the United States of America.

Cadence Design Systems, Inc., 2655 Seely Avenue, San Jose, CA 95134, USA

Open SystemC, Open SystemC Initiative, OSCI, SystemC, and SystemC Initiative are trademarks or registered trademarks of Open SystemC Initiative, Inc. in the United States and other countries and are used with permission.

Trademarks: Trademarks and service marks of Cadence Design Systems, Inc. contained in this document are attributed to Cadence with the appropriate symbol. For queries regarding Cadence's trademarks, contact the corporate legal department at the address shown above or call 800.862.4522. All other trademarks are the property of their respective holders.

All other trademarks are the property of their respective holders.

Restricted Permission: This publication is protected by copyright law and international treaties and contains trade secrets and proprietary information owned by Cadence. Unauthorized reproduction or distribution of this publication, or any portion of it, may result in civil and criminal penalties. Except as expressly permitted below, this publication may not be copied, reproduced, modified, published, uploaded, posted, transmitted, or distributed in any way, including automated processes, training or services involving a large language model, foundation model, deep machine learning, generative artificial intelligence, or any other similar process or technology, without prior written permission from Cadence. Unless otherwise agreed to by Cadence in writing, customers are granted permission to print one (1) hard copy of this publication subject to the following conditions:

1. the publication may be used only in accordance with a written agreement between Cadence and its customer;
2. the publication may not be modified in any way;
3. any authorized copy of the publication or portion thereof must include all original copyright, trademark, and other proprietary notices and this permission statement;
4. the information contained in this document cannot be used in the development of like products or software, whether for internal or external use, and shall not be used for the benefit of any other party, whether or not for consideration

Disclaimer: Information in this publication is subject to change without notice and does not represent a commitment on the part of Cadence. The information contained herein is the proprietary and confidential information of Cadence or its licensors, and is supplied subject to, and may be used only by Cadence's customer in accordance with, a written agreement between Cadence and its customer. Except as may be explicitly set forth in such agreement, Cadence does not make, and

expressly disclaims, any representations or warranties as to the completeness, accuracy or usefulness of the information contained in this document. Cadence does not warrant that use of such information will not infringe any third party rights, nor does Cadence assume any liability for damages or costs of any kind that may result from use of such information.

Cadence is committed to using respectful language in our code and communications. We are also active in the removal and replacement of inappropriate language from existing content. This product documentation may however contain material that is no longer considered appropriate but still reflects long-standing industry terminology. Such content will be addressed at a time when the related software can be updated without end-user impact.

Restricted Rights: Use, duplication, or disclosure by the Government is subject to restrictions as set forth in FAR52.227-14 and DFAR252.227-7013 et seq. or its successor.

Contents

About This Manual	6
Audience	6
Conventions Used in This Manual	6
Related Documents	8
1	9
Import Design Commands	9
check_design	10
checkTimingLibrary	13
create_hier_design	16
free_design	19
globalNetConnect	20
init_hier_design	25
read_def	27
read_design	31
read_hier_design	34
read_instance_voltage	35
read_lib	37
read_power_domain	41
read_rcdb	43
read_sdc	44
read_spef	45
read_stream	49
read_twf	53
read_verilog	55
reset_sdc	56
restore_oa_design	57
save_design	59
save_hier_design	64
set_cell_power_domain	65
set_dc_sources	67
set_import_mode	70
set_library_unit	72

set_license_check	73
set_top_module	77
specify_def	78
specify_lib	81
specify_oa	83
specify_spef	85
write_lef_abstract	86
write_rcdb	95
write_twf	96
2	99
Multiple-CPU Processing Commands	99
get_distribute_host	99
get_multi_cpu_usage	102
set_distribute_host	104
set_multi_cpu_usage	111
3	116
Verify Commands	116
verify_connectivity	116
verify_PG_short	121
verify_power_via	123
4	130
GUI Commands	130
add_gui_marker	131
add_gui_shape	133
createMarker	136
createPGPin	139
deselectAll	142
deselectInst	143
deselectInstByCellName	144
dumpToGIF	145
list_gui_marker	146
read_temperature_map_file	147
remove_gui_marker	147
reportSelect	148
selectInst	150

selectInstByCellName	151
selectInstOnNet	152
selectPGPin	153
set_power_rail_display	154
set_power_rail_layers_nets	166
zoomBox	172
zoomIn	173
zoomOut	174
zoomSelected	175
zoomTo	176
5	178
Power-Grid Library Commands	178
check_pg_library	179
generate_pg_library	192
generate_xpgv	194
generate_tech_pg_lib	198
merge_pg_library	202
rename_pg_library	205
set_advanced_pg_library_mode	207
set_pg_library_mode	245
trim_pg_library	281
validate_pg_library	283
6	293
Power Calculation Commands	293
analyze_thermal	294
create_thermal_model	295
dump_unannotated_nets	299
encrypt_thermal_file	301
extract_metal_density	303
get_default_switching_activity	304
get_dynamic_power_simulation	305
get_power_analysis_mode	306
map_activity_file	307
merge_vtm	310
propagate_activity	312
query_power_data	313

read_activity_file	316
report_annotated_parasitics	324
report_instance_power	328
report_power	330
reset_power_activity	351
restore_power_database	352
set_default_switching_activity	354
set_dynamic_power_simulation	359
set_inst_temperature_file	361
set_metal_density_options	362
set_power	366
set_power_analysis_mode	379
set_power_calc_temperature	438
set_power_include_file	439
set_power_output_dir	441
set_switching_activity	442
set_thermal_analysis_mode	447
set_twf_attribute	449
set_virtual_clock_network_parameters	453
write_power_constraints	455
write_tcf	456
7	459
Rail Analysis Commands	459
analyze_esd_network	460
analyze_esd_voltage	463
analyze_jitter	466
analyze_package	471
analyze_rail	474
analyze_resistance	475
analyze_signal_resistance	484
analyze_voltage_regulator	486
calculate_didt	487
calculate_differential_voltage	489
calculate_noise_margin	491
create_current_region	494
create_die_model	498

create_hier_view	509
create_power_pads	515
create_power_pads_regions	520
create_what_if_shape	522
debug_irdrop	526
extract_package	532
map_die_package	534
generate_esd_rlrp_report	536
generate_power_up_report	538
get_ir_insight	540
get_voltus_insight_mode	547
read_power_rail_results	549
report_power_rail_results	555
scale_what_if_capacitance	563
scale_what_if_current	566
scale_what_if_resistance	568
set_advanced_package_options	571
set_advanced_rail_options	572
set_die_model	573
set_dynamic_rail_simulation	575
set_multi_die_analysis_mode	577
set_net_group	580
set_offchip_package_trace	581
set_package	583
set_pg_nets	586
set_power_data	588
set_power_pads	593
set_rail_analysis_domain	598
set_rail_analysis_mode	600
set_voltus_insight_mode	675
set_voltage_regulator_module	682
view_dynamic_movie	684
view_dynamic_waveform	685
view_package_results	689
8	691
Advanced Timing Tcl Scripting Commands	691

get_activity	691
get_cells	697
get_lib_cells	701
get_nets	704
get_pins	706
get_ports	709
get_power	711
9	717
Signal ElectroMigration Commands	717
create_cell_signal_em_model	717
create_top_scope	721
get_signal_em_analysis_mode	722
report_design_rule	724
report_freq_violation	726
set_max_cap_per_freq	730
set_max_tran_per_freq	732
set_signal_em_analysis_mode	734
verify_AC_limit	750
10	752
Self-Heating Effect Analysis Commands	752
analyze_self_heat	752
analyze_joule_heat	757
set_self_heat_analysis_mode	760
11	766
Global Commands	766
exit	766
help	767
man	771
set_analysis_view	773
voltus	776
12	786
General Commands	786
get_metric	786
save_testcase	789
set_delay_cal_mode	792

set_message	802
set_message_limit	805
source	807
write_flow_template	808
13	810
General Global Variables	810
Using General Globals	810
load_netlist_ignore_undefined_cell	810
timing_library_infer_cap_range_from_ecsm_receiver_model	812
timing_library_read_ccs_power_data	813
enc_tcl_error_info	814
14	816
Legacy EPS Options	816
Extractor Options	816
Extractor Commands	818
black_macros	818
cell black	819
cell ignore	819
cell_ignore_file	820
cell map	820
cluster distance	821
density_check_rectangle	822
extraction_type	822
ignore_macros	825
ignore_pin	826
include	827
include_via_res_table_file	827
label net	828
label port	829
layer	830
layer bias	830
layer gds	831
layer map	833
pin ignore	834
promote_ports	835
Extractor Variables	836

density_check_method	836
dump_options	837
enable_erosion_effects	837
enable_metal_fill_effects	838
enable_poly_routing	839
enable_r_function_width	839
enable_toplevel_diff_extract	840
exclude_blockagewire	841
gds_unit	841
ignore_shorts	842
layout_scale	843
load_optimized_views	843
log_file	844
max_error_messages	845
max_resistor_length	845
message_detail_level	846
metal_fill_type	847
oa_analysis_point	847
output_cache_directory_name	848
output_directory_name	849
output_drawn_geometry	849
output_file_name	850
output_incomplete_nets	851
output_oa	851
remove_dangling_resistors	852
tech_file	853
top_level_names_only	853
use_def_over_library	854
use_layer_bias_in_cmd_and_process_bias_in_tech	854
use_manhattan_ports	855
wee_type	856
Library Generator Options	857
Library Generator Commands	860
cell_decap_area	860
cell_decap_area_file	861
cell_ground_names	862
cell_ignore_list	863

cell_ignore_list_file	864
cell_pin_net_map	864
cell_pin_net_map_file	865
cell_power_names	865
cell_supply_names_file	866
current_region	868
current_region_default	870
current_region_file	871
cell_decap	873
derived_layer_list	874
discard_detailed_powerview_cell_list	875
discard_detailed_powerview_cell_list_file	875
foreign_name_map	876
foreign_name_map_file	876
lef_layer_ignore_list	877
lef_layer_ignore_list_file	877
li	878
pgv_header_map	879
pgv_header_map_file	880
poly_command	881
port_power_gate_file	883
port_view_current_file	884
scale_current_region	886
scale_current_region_default	887
thunder_model_table_directory	887
well	888
well_cap_area	889
well_cap_area_file	890
well_cap	891
Library Generator Variables	892
assume_foreigns	892
assume_foreigns_mode	893
case_sensitivity_default	893
cell_accura_data_file	894
cmd_data_case_sensitive	894
collapsed_view_current_threshold	895
create_dummy_tech_file	896

database_precision	897
debug_skip_parasitic_extraction	898
debug_skip_thunder	898
debug_skip_xtc	898
default_ground_voltage	899
default_lef_min_spacing	899
default_lef_min_width	900
default_power_voltage	900
default_via_resistance	901
disable_lef_parser_escape_check	901
do_optimized_views	902
do_rextract_in_parallel	902
emulate_lef_parsing	903
gds_cell_extension	904
gds_extractor	904
gds_extractor_command_file	904
gds_port_explosion_depth	905
gds_to_lef_gds_extractor_command_file	906
generated_gds_file	906
halt_on_missing_foreign_data	907
halt_on_undefined_lef_layer	908
ignore_lef_use	908
ignore_missing_via_definition	909
keep_debug_files	909
keep_netlist_in_library	910
layout_scale_factor	911
libgen_executable	911
maxerrors	911
max_itfnode_dist	912
max_resistor_length	912
output_tool_performance_data	913
output_wlt	913
parasitic_extractor	914
parasitic_extractor_command_files_directory	915
port_file	915
prpgv_x_offset	916
prpgv_y_offset	917

std_cell_size_limit	918
stripe_step_size	919
suppress_regular_gds_layer	919
tech_file	920
temp_directory_name	921
temperature	921
thunder_command_file	921
thunder_executable	922
use_current_data_from_pgv	922
use_lef_only_itfnodes	923
use_lef_technology	923
use_old_lef_parser	924
use_pwrnet_ipeak	925
xtc_stripe_directory_name	925
Library Simulator Options	926
Library Simulator Commands	929
activity	929
activity cycle_time	929
activity default	930
activity filed	930
activity filen	931
activity report	932
activity reset	932
activity scale	933
activity scaled	933
activity scalen	934
activity setd	934
activity setn	935
activity vdd_range	935
check	936
check off, check on	936
check tgloff, check tglon	936
check time	938
devc	938
devc table	938
devc tran	940
pwrmeter	940

Format of the Clock File	943
Format of the Frequency-Activity File	943
Format of the Average-Power File	944
pwrmeter plot	944
pwrnet	945
pwrnet clear	946
pwrnet ipeak	946
pwrnet off, pwrnet on	947
pwrnet report	947
pwrnet specify	948
pwrnet tally	951
pwrnet tallyint	951
pwrnet tgloff, pwrnet tglon	953
pwrnet time	954
region	955
region params	955
region setd	956
region setn	957
setenv	958
stats	958
tglcnt	959
tglcnt check	960
tglcnt clear	961
tglcnt off, tglcnt on	961
tglcnt params	961
tglcnt report	962
tglcnt tally	963
vcd	964
VCD File	965
VCD Values	965
VCD Command Extensions	965
Signal Declaration File	968
Library Simulator Variables	970
allowMosAsSubckt	970
analysisReportSize	971
busNotation	971
caseSensitive	971

cmdBreakOnError	972
CoupledCapOnExtractedNetlist	972
DCInductorResistance	972
defaultTRiseFall	973
errorIfDevicesBelowMinW_L	973
forwardDiodeIpeakCurrent	974
gatePowerNetConnection	974
groundName	974
hierarchySeparator	975
isDetailedDynamicView	975
keepTopCell	975
logLimit	976
minimumCapacitance	976
minimumInductance	976
minimumNodeCapacitance	977
minimumResistance	977
netlistInlineComment	977
netlistTopCell	978
pwrnetSpecifyBjtRatio	978
pwrnetSpecifyCapCurrent	979
pwrnetSpecifyDiodeRatio	979
pwrnetSpecifyResistorRatio	979
quitOnCmdError	980
Power Analysis Options	980
calCellsToUseWaveEst	981
calCellsToUseWaveEstFile	981
calPinTransitionTime	982
calRailExclude	985
calSwitchingCells	986
rDefaultNetCap	987
rDefaultPinCap	987
rDynamicSwitchAllBuffers	988
rDynamicSwitchAllClockBuffers	989
rDynamicSwitchAllFlops	989
rEnhancedBlackboxMode	990
rEnhancedBlackboxAvg	992
rEnhancedBlackboxMax	992

rlgnoreBidirCap	993
rUseFastestClockDomain	993
rUseLibAdjustTransitionTime	994
rUseUserDefinedCurrent	995
Rail Analysis Options	995
Rail Analysis Commands	997
label	997
layer	998
layer_bias	999
oa_library	1000
oa_view_name	1001
print	1001
printvar	1008
reuse_state	1008
suppress_message	1009
viacurrent	1009
write	1010
Rail Analysis Variable	1017
auto_scale_adjacent_via_layers	1017
cell_area_capacitance	1018
create_label_from_vsc	1018
decap_exclude_static_violation_area	1019
decap_window_size	1019
DEF_merge_supply_shorts	1020
em_part_check_current	1021
em_part_current_threshold	1021
error_on_op_analysis	1022
extractor	1022
extractor_include	1023
halt_on_net_with_no_cell_connections	1023
hierarchy_char	1024
internal_nodes_visible	1025
internal_resistors_visible	1025
libgen_executable	1026
libgen_include	1026
lifetime	1027
max_error_messages	1027

max_warning_messages	1028
merge_util	1028
pin_char	1029
precision_digits	1030
prpgv_x_offset	1030
prpgv_y_offset	1031
reff_ignore_disconnected_pins	1032
use_average_power_for_op	1033
use_capacitor_adjustment	1033
use_current_data_from_pgv	1034
use_existing_rcdb	1035
use_gds_annotate_attributes	1035
use_port_voltage_for_iv	1036
write_bbv_pwl	1036
write_decap_filler_voltage	1037
15	1038
Voltus Utilities	1038
convert_gds_to_def	1038
map_dies	1049
16	1054
RTL-to-Gate Rule Mapping	1054

About This Manual

This manual describes the commands that you can use in Voltus IC Power Integrity Solution, also known as Voltus.

Within each chapter, the text commands appear alphabetically.

Audience

This manual is written for experienced designers of digital integrated circuits. Such designers must be familiar with design planning, placement and routing, block implementation, chip assembly, and design verification. Designers must also have a solid understanding of UNIX.

Conventions Used in This Manual

This section describes the typographic and syntax conventions used in this manual.

<i>text</i>	Indicates text that you must type exactly as shown. For example: <code>report_annotated_check -missing_setup</code>
<i>text</i>	Indicates information for which you must substitute a name or value. In the following example, you must substitute the name of a specific file for <i>worst_entries</i> : <code>report_clock_timing -type skew -nworst <i>worst_entries</i></code>
<i>text</i>	Indicates the following: • Text found in the graphical user interface (GUI), including form names, button labels, and field names • Terms that are new to the manual, are the subject of discussion, or need special emphasis • Titles of manuals

[]	Indicates optional arguments. In the following example, you can specify none, one, or both of the bracketed arguments: <pre>command [-arg1] [arg2 value]</pre>
[]	Indicates an optional choice from a mutually exclusive list. In the following example, you can specify any of the arguments or none of the arguments, but you cannot specify more than one: <pre>command [arg1 arg2 arg3 arg4]</pre>
{ }	Indicates a required choice from a mutually exclusive list. In the following example, you must specify one, and only one, of the arguments: <pre>command {arg1 arg2 arg3}</pre>
{ [] [] }	Indicates a required choice of one or more items in a list. In the following example, you must choose one argument from the list, but you can choose more than one: <pre>command {[arg1] [arg2] [arg3]}</pre>
{ }	Indicates curly braces that must be entered with the command syntax. In the following example, you must type the curly braces: <pre>command arg1 {x y}</pre>
...	Indicates that you can repeat the previous argument.
.	Indicates an omission in an example of computer output or input.

<i>Command - Subcommand</i>	Indicates a command sequence, which shows the order in which you choose commands and subcommands from the GUI menu. In the following example, you choose <i>Power & Rail</i> from the menu, then <i>Report</i> from the submenu, and then <i>Dynamic Movies</i> from the displayed list: <i>Power & Rail - Report - Dynamic Movies</i> This sequence opens the View Movies form.
-----------------------------	--

 Ensure that there are no extra whitespaces at the end of lines. The following syntax causes an error message in the Voltus software:

```
set_rail_analysis_mode \ $
    -method                static \ $
    -accuracy               xd \ $
    -analysis_view         AV_wc_on \ $
    -power_grid_library     { \ $
    ../data/pgv_dir/tech_pgv/techonly.cl \ $
    ../data/pgv_dir/stdcell_pgv/stdcells.cl \ $      <===== White space at EOL
    ../data/pgv_dir/macro_pgv/macros_pll.cl \ $
    }
```

Related Documents

For more information about the Voltus family of products, see the following documents. You can access these and other Cadence documents with the Cadence Help online documentation system.

- [Voltus Known Problems and Solutions](#)
Describes important Cadence Change Requests (CCRs) for Voltus, including solutions for working around known problems.
- [Voltus Menu Reference](#)
Provides information specific to the forms and commands available from the Voltus graphical user interface.
- [Voltus User Guide](#)
Provides information on using various Voltus features.

Import Design Commands

- [check_design](#)
- [checkTimingLibrary](#)
- [create_hier_design](#)
- [free_design](#)
- [globalNetConnect](#)
- [init_hier_design](#)
- [read_def](#)
- [read_design](#)
- [read_hier_design](#)
- [read_instance_voltage](#)
- [read_lib](#)
- [read_power_domain](#)
- [read_rcdb](#)
- [read_sdc](#)
- [read_spef](#)
- [read_stream](#)
- [read_twf](#)
- [read_verilog](#)
- [reset_sdc](#)
- [restore_oa_design](#)
- [save_design](#)
- [save_hier_design](#)
- [set_cell_power_domain](#)

- [set_dc_sources](#)
- [set_import_mode](#)
- [set_library_unit](#)
- [set_license_check](#)
- [set_top_module](#)
- [specify_def](#)
- [specify_lib](#)
- [specify_oa](#)
- [specify_spef](#)
- [write_lef_abstract](#)
- [write_rcdb](#)
- [write_twf](#)

check_design

```
check_design
[-help]
-format {table detail}
-id <string>
[-out_file fileName]
-type {power_intent timing hierarchical pin_assign budget assign_statements place opt cts
route signoff all}
-no_check
-type_3d {db tech_lib netlist power_intent timing place floorplan route all}
```

Checks that all prerequisites to run all or part of the hierarchical or block implementation flow are met.

The `check_design` command checks the preconditions for major flow steps before they are run. You can run these checks standalone using the `check_design` command. If there are error in the `check_design` run, the current script will stop.

Some examples of problems that will cause design checks to fail include:

- Cells used in the design that cannot be legally placed

- Tracks misaligned with some or all cell pins
- In advanced processes, power nets that cannot be colored in the expected pattern once cells are placed
- Cell rows that are unblocked for placement but where pin layers are obstructed
- Cell rows that are unusable due to orientation and power rail misalignment
- Cells instantiated in the netlist that have no timing model defined
- Tracks blocked by power grid spacing constraints
- Instances not covered by a true timing path in any view
- ILM model with no clock tree defined
- No hold fixing cells are specified and delay cells are not usable by default
- No clock inverters, buffers, clock gates or logic cells are specified and the library cells are not usable by default
- CTS cannot balance all skew groups without increasing insertion delay
- There are preserved nets and cells in the clock tree that will violate other constraints because they cannot be resized or buffered.

Parameters

<code>-help</code>	Prints a brief description that includes the type and default information for each <code>check_design</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <code>man check_design.</code>
<code>-format</code> <code>{table</code> <code>detail}</code>	Specifies the table format. • <code>table</code>: to print the results in table format. • <code>detail</code>: to print the results from <code>check_design</code> using the old style of log file. Basically, this is just a way to let user (if necessary) go back to the old style of log file for
<code>-id</code> <code><string></code>	Check violation list (e.g. 'CHKPLC-100').

<code>-type</code>	Specifies the category of the flow to be checked: • <code>assign_statements</code>: Performs checks for the netlist for assign statements. • <code>power_intent</code>: Performs checks related to MSV setup including power domain/fence checks. • <code>timing</code>: Performs checks related to timing setup integrity. • <code>feedthru</code>: Performs checks related to feedthru insertion for partitioning flows. • <code>pin_assign</code>: Performs checks related to pin assignment for partitioning flows. • <code>budget</code>: Performs checks related to time budgeting for partitioning flows. • <code>place</code>: Performs checks related to placement such as PG rail alignment and pin access issues. • <code>opt</code>: Performs checks related to optimization such as checks for available cells. • <code>cts</code>: Performs checks related to clock tree definition such as constraint targets and route types. • <code>route</code>: Performs checks related to routing such as congestion, placement, and pin access issues. • <code>signoff</code>: Checks related to the signoff activities including extraction and ECOs. • <code>all</code>: Runs all categories.
<code>-no_check</code>	Just echo out all the messages that would be checked, don't do any checks Do not run the checks but echo the checks that would run per the categories provided by the user
<code>-out_file fileName</code>	Specifies the output report file. Contents dumped to this file are in the Tcl format.
<code>-type_3d</code>	List of 3DIC design categories to check, accepts single & multiple values (one or more categories).

checkTimingLibrary

```
checkTimingLibrary
[-help]
[-cellname cellName]
[-checkpower]
[-library timingLibraryName]
[-outfile fileName]
[-reportMissingPowerOnly]
```

Checks the contents of the timing library and reports any inconsistencies in a log file.

You can use the `checkTimingLibrary` command after importing the design and timing libraries.

Parameters

-help	Prints a brief description that includes the type and default information for each <code>checkTimingLibrary</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <pre>man checkTimingLibrary</pre>
-cellname <i>cellName</i>	Checks if the timing is defined for all cells with the specified name, in all timing libraries. Note: In general, the <code>-cellName</code> parameter is optional. However, if you specify the <code>-library</code> parameter, then you <i>must</i> specify <code>-cellName</code>.
- checkpower	Checks if power constructs are defined for the cells, such as <code>internal_power</code>, <code>leakage_power</code>, or <code>ecsm_power</code>. Note: In general, the <code>-checkPower</code> parameter is optional. However, if you specify <code>-reportMissingPowerOnly</code>, then you <i>must</i> specify <code>-checkPower</code>. If you use the <code>-library</code> parameter, you do not have to specify the <code>-checkPower</code> parameter, because when a library-specific cell is given as input, the software automatically checks for power constructs. <i>Default:</i> Only checks if timing is defined for the cells.
-library <i>timingLibraryName</i>	

	Specifies the timing library in which to check a particular cell. You must specify the <code>-cellName</code> parameter with <code>-library</code>, to specify the particular library cell to check. Note: If you use the <code>-library</code> parameter, you do not have to specify the <code>-checkPower</code> parameter, because when a library-specific cell is given as input, the software automatically checks for power constructs.
<code>-outfile fileName</code>	
	Writes the report to the specified file.
<code>-reportMissingPowerOnly</code>	
	Reports only the cells that do not have <code>internal_power</code> (at pin level), or <code>leakage_power</code> defined. You must specify the <code>-checkPower</code> parameter with the <code>-reportMissingPowerOnly</code> parameter.

Examples

- The following command checks if timing is defined for all cells that have been read into the design, and writes a report to the log file:
`checkTimingLibrary`
- The following command checks if timing is defined for all cells that have been read into the design, and writes a report to a file named `CheckTiming1`:
`checkTimingLibrary -outfile CheckTiming1`
- The following command checks if timing and power constructs are defined for all cells that have been read into the design, and writes the report to the log file:
`checkTimingLibrary -checkPower`
- The following command checks if timing and power constructs are defined for all cell that have been read into the design, and writes the report to a file named `CheckTiming2`:
`checkTimingLibrary -outfile CheckTiming2 -checkPower`
- The following command checks if timing is defined for all cells with the name `A01` in all timing libraries, and writes the report to a file named `CheckTiming3`:
`checkTimingLibrary -outfile CheckTiming3 -cellName A01`
- The following command checks if timing and power constructs are defined for all cells with the

name A01, in all timing libraries, and writes the report to a file named AllA01Check:

```
checkTimingLibrary -outfile AllA01Check -cellName A01 -checkPower
```

- The following command checks if timing and power constructs are defined for the cell with the name A01 in the timing library minFile1.lib, and writes the report to a file named A01Check:

```
checkTimingLibrary -outfile A01Check -cellName A01 -library minFile1.lib
```

Note: When you use the `-library` parameter, you do not have to specify the `-checkPower` parameter, because when a library-specific cell is given as input, the software automatically checks for power constructs.

- The following command checks if the internal_power and/or leakage_power constructs are defined for all cells that have been read into the design, and reports only those cells that do not have the constructs defined. The software writes the report to a file named dma_mac.talib:

```
checkTimingLibrary -outfile dma_mac.talib -checkPower -reportMissingPowerOnly
```

create_hier_design

```
create_hier_design
[-help]
[-add_on_shape_def {{def1 x_loc y_loc orient} {def2 x_loc y_loc orient}...} ]
-def list_of_DEF_files
-hdb_dir dir_name
[-keep_blockages]
-lef list_of_LEF_files
[-rdl_def def_file]
[-rdl_orient { N|S|W|E|FN|FS|FE|FW }]
[-rdl_placement {X Y}]
[-skip_restore]
[-skip_signal]
[-top_cell cell_name]
```

Specifies to create and load a hierarchical database (HDB), that is, it constitutes the functions of the `init_hier_design`, `save_hier_design`, and `read_hier_design` commands. The command allows you to work with HDB-based GUI in the multi-CPU mode.

It can read multiple DEF files for the design with multiple levels of hierarchy and display a flattened database. When multiple DEFs are passed to `create_hier_design`, the software flattens all the levels of hierarchical DEFs and displays the design layout.

Parameters

-help	Outputs the command usage and a brief description about the command parameters.
-add_on_shape_def {{def1 x_loc y_loc orient} {def2 x_loc y_loc orient}...}	

	Specifies to merge physical shapes (such as pseudo what-if shapes) from additional side DEF files to the current top database. This parameter allows you to add multiple side DEF files together. For each DEF file, you need to specify the DEF file name (def1, def2), x and y or the physical shape coordinates, and the shape's orientation. A shape can have any of the following orientations: N (North), S (South), W (West), E (East), FN (Flipped North), FS (Flipped South), FW (Flipped West), or FE (Flipped East).
<code>-def list_of_DEF_files</code>	
	Specifies the name of the DEF files, starting with the top-level DEF.
<code>-hdb_dir dir_name</code>	
	Specifies the name of the directory that stores the design data.
<code>-keep_blockages</code>	
	Specifies to load all routing and placement blockages.
<code>-lef list_of_LEF_files</code>	
	Specifies the name of the LEF files for loading the hierarchy design.
<code>-rdl_def def_file</code>	
	Specifies to instantiate the RDL instance/DEF at the top level and merge it with the DEF of the top level block being analyzed.
<code>-rdl_orient { N S W E FN FS FE FW }</code>	
	Specifies the orientation of the RDL DEF file.
<code>-rdl_placement {X Y}</code>	
	Specifies to place the RDL at a given X,Y location (in micron).
<code>-skip_restore</code>	Specifies that the saved HDB design should not be restored.
<code>-skip_signal</code>	Reads PG nets with <code>USE POWER/GROUND</code> attributes in both <code>SPECIALNETS/NETS</code> sections in the input list of DEF files. Skips signal nets when reading <i>[list_of_DEFs]</i> .

<code>-top_cell cell_name</code>	
	Specifies the top cell name.

Examples

- The following command specifies to load an HDB in the distributed mode:

```
set_distribute_host \
-lsf \
    -queue ssvpe \
    -timeout 500000 \
    -lsf_args {-P VOLTUS} \
    -resource {rusage[mem=2800] select[ OSNAME==Linux &&
(OSREL==EE60||OSREL==EE50)] }

set_multi_cpu_usage -remoteHost 8 -cpuPerRemoteHost 16 -localCpu 16
create_hier_design -skip_signal -lef ../hdb.lef -def ../hier1.def ../hier2.def
../hier3.def ../hier4.def -top_cell MURKKU2 -hdb_dir saveDB
```

- The following command specifies to load an HDB with physical shape DEF files:

```
create_hier_design -lef $lefFiles \
-def $defFiles \
-skip_signal \
-add_on_shape_def {{/../../rd1.def -1000 -1000 FS} {/../../rd1.def -1000 -1000 N}
{/../../rd1.def -1000 -1000 FW} } \
-rdl_def ../../rd1.def \
-rdl_placement {-1000 -1000} \
-rdl_orient {W} \
-top_cell DMA_WRAP \
-hdb_dir saveDB
```


free_design

```
free_design  
-help
```

Removes libraries and design-specific data from the Voltus session. It can be used as a shortcut in place of exiting and re-starting Voltus.

Parameters

-help	Prints out the command usage.
-------	-------------------------------

Example

- The next command frees the design so another one can be loaded.

```
free_design
```

globalNetConnect

```
globalNetConnect
[-help]
globalNetName
{{-type pgpin -pin pinNamePattern |
-type tiehi [-pin pinNamePattern ] |
-type tielo [-pin pinNamePattern ]}
{{-singleInstance | -singleInst | -sinst} instName |
[{-instanceBasename | -instBasename | -inst} instBasenamePattern ]
[{-hierarchicalInstance | -hierInst | -module} hierInstName |
-region llx lly urx ury |
-powerDomain powerDomainName | -all]] |
-type net -net netBasenamePattern
[{-hierarchicalInstance | -hierInst | -module} hierInstName |
-powerDomain powerDomainName | -all]]
[-autoTie]
[-disconnect]
[-netlistOverride]
[-nonHierarchical]
[-override]
[-verbose]
```

Adds a new global net connection to the specified global net.

You can use the `globalNetCommand` syntax in three general design situations. The three design situations, and the suggested usage for the `globalNetCommand` syntax, appear below:

- Connecting pins in a single instance to a global net. In the example below, *instName* represents an instance's complete hierarchical name, without any wildcard characters:

```
globalNetConnect
globalNetName
{-type pgpin -pin pinNamePattern | -type tiehi [-pin pinNamePattern ]
| -type tielo [-pin pinNamePattern ]}
-singleInstance instName
[-override]
[-verbose]
```

- Connecting pins in multiple instances, or pins in a region, to a global net:

```
globalNetConnect
globalNetName
```

```
{-type pgpin -pin pinNamePattern | -type tiehi [-pin pinNamePattern ]
| -type tielo [-pin pinNamePattern ]}
[-instanceBasename instBasenamePattern ]
[-hierarchicalInstance hierInstName | -region llx lly urx ury | -all]
[-override]
[-verbose]
```

- **Connecting nets to a global net:**

```
globalNetConnect
globalNetName
-type net -net netBasenamePattern
[-hierarchicalInstance hierInstName | -all]
[-override]
[-verbose]
```

Parameters

-help	
	Outputs a brief description that includes type and default information for each globalNetConnect parameter. For a detailed description of the command and all of its parameters, use the man command: man globalNetConnect
-all	
	Applies the global net connection to all instances in the design.
-autoTie	
	Support supply/ground sensitivity. In a command for PG-pin connections, the option can only be specified together with - type pgpin. In addition to connecting power pin VDD to global net VDD for each instance, all the instance tie-high pins whose related power pin (as specified in CPF, .lib, or LEF) is VDD, will automatically be tied to global net VDD. For example, globalNetConnect VDD -type pgpin -pin VDD -autoTie

<code>-disconnect</code>	
	Disconnect p/g net connection. The option works with <code>-sinst</code>, <code>-type pgpin</code> and <code>-pin</code> options. Note: This option is mutually exclusive with <code>globalNetName</code> argument.
<i>globalNetName</i>	
	The name of the global net to which the specified pins, modules, or nets connect.
<code>-hierarchicalInstance</code> <i>hierInstName</i>	
	Applies the global net connection to instances under <i>hierInstName</i> . Only the pins of the instances or the local nets under the specified module or hierarchical instance are connected. This parameter is mutually exclusive with the <code>-region</code> parameter. Note: You can also use the following spelling variations for this argument: • <code>-hierInst</code> • <code>-module</code>
<code>-instanceBasename</code> <i>instBasenamePattern</i>	
	Specifies the names of leaf instances for which pins are to be connected to the global net. You can use the wildcard (*) character to specify a pattern of instance basenames. An instance basename cannot contain the "/" character. This parameter is used in conjunction with the <code>-type pgpin</code> parameter, or with the <code>-type tiehi</code> or <code>-type tielo</code> parameters. Note: You can also use the following spelling variations for this argument: • <code>-inst</code> • <code>-instBasename</code>
<code>-net</code> <i>netBaseName</i>	
	Specifies the basenames of nets to connect to the global net. You can use the wildcard (*) character to specify a pattern of net basenames. An net basename cannot contain the "/" character. This parameter is used in conjunction with the <code>-type net</code> parameter.

<code>-netlistOverride</code>	
	Overrides previous global net connections. The option is similar to the option <code>-override</code> . However, in addition to overriding previous global net connections, <code>-netlistOverride</code> also overrides netlist-specified P/G-pin connections if applicable, which the option <code>-override</code> can't do.
<code>-nonHierarchical</code>	
	Invokes the non hierarchical cell instance. The option is of type boolean and is optional.
<code>-override</code>	
	Specifies that the values that used with the <code>globalNetConnect</code> command override global net connection values that have been set previously.
<code>-pin <i>pinNamePattern</i></code>	
	Specifies the pins to connect to the global net. You can use the wildcard (*) character to specify a pattern of pin names. This parameter is used in conjunction with the <code>-type pgpin</code> parameter, or optionally with the <code>-type tiehi</code> or <code>-type tielo</code> parameters.
<code>-powerDomain <i>powerDomainName</i></code>	
	Applies the global net connection to instances within the specified power domain.
<code>-region <i>llx lly urx ury</i></code>	
	Applies the global net connection to instances inside the specified region. The region is specified by the x and y coordinates of the lower left and upper right boundary. This parameter is mutually exclusive with the <code>-module</code> parameter.
<code>-singleInstance <i>instName</i></code>	
	Connects pins in a single instance to a global net. <i>-instName</i> is the instance's complete hierarchical name, without any wildcard characters. Note: You can also use the following spelling variations for this argument: • <code>-sinst</code> • <code>-singleInst</code>

<code>-type {net pgpin tielo tiehi}</code>	
	Specifies the type of connection to be made to the global net. Use one of the following values: Specifies that the nets listed with the - net parameter are to be connected. Specifies that the power and ground pins listed with the - pin parameter are to be connected. Specifies that tie low pins be connected. Specifies that tie high pins be connected.
<code>-verbose</code>	
	Specifies that connection statistics and warning messages are displayed in the console. This parameter can be useful during troubleshooting.

Example

The following command connects the vdd! net with the VDD global net:

```
globalNetConnect VDD -type net -net vdd!
```

Note: You can also ignore polarity checking during globalNetConnect and CPF operations through the global variable `init_ignore_pgpin_polarity_check` that allows USE POWER pin to be connected to a USE GROUND net (or vice versa).

init_hier_design

```
init_hier_design
-defs list_of_DEF_files
[-add_on_shape_def {{def1x_locy_locorient} {def2x_locy_locorient}...} ]
[-help]
[-keep_blockages]
[-rdl_def def_file]
[-rdl_orient { N|S|W|E|FN|FS|FE|FW }]
[-rdl_placement {X Y}]
[-skip_signal]
[-top_cell cell_name]
[-top_cell_orient { N|S|W|E|FN|FS|FE|FW }]
[-top_cell_placement {X Y}]
```

The `init_hier_design` command can read multiple DEF files for the design with multiple levels of hierarchy and display a flattened database. When multiple DEFs are passed to `init_hier_design`, the software flattens all the levels of hierarchical DEFs and displays the design layout.

Parameters

-help	Outputs the command usage and a brief description about the command parameters.
-add_on_shape_def {{def1 x_loc y_loc orient} {def2 x_loc y_loc orient}...}	
	Specifies to merge physical shapes (such as pseudo what-if shapes) from additional side DEF files to the current top database. This parameter allows you to add multiple side DEF files together. For each DEF file, you need to specify the DEF file name (def1, def2), x and y or the physical shape coordinates, and the shape's orientation. A shape can have any of the following orientations: N (North), S (South), W (West), E (East), FN (Flipped North), FS (Flipped South), FW (Flipped West), or FE (Flipped East).
-defs list_of_DEF_files	
	Specifies the name of the DEF files, starting with the top-level DEF.

<code>-keep_blockages</code>	
	Specifies to load all routing and placement blockages.
<code>-rdl_def <i>def_file</i></code>	
	Specifies to instantiate the RDL instance/DEF at the top level and merge it with the DEF of the top level block being analyzed.
<code>-rdl_orient { N S W E FN FS FE FW }</code>	
	Specifies the orientation of the RDL DEF file.
<code>-rdl_placement {X Y}</code>	
	Specifies to place the RDL at a given X,Y location (in micron).
<code>-skip_signal</code>	Reads PG nets with <code>USE POWER/GROUND</code> attributes in both <code>SPECIALNETS/NETS</code> sections in the input list of DEF files. Skips signal nets when reading [<i>list_of_DEFs</i>].
<code>-top_cell <i>cell_name</i></code>	
	Specifies the top cell name.
<code>-top_cell_orient { N S W E FN FS FE FW }</code>	
	Specifies the orientation of the top DEF block.
<code>-top_cell_placement {X Y}</code>	
	Specifies to place the top DEF block at a given X,Y location (in micron) with respect to virtual top (0,0).

Examples

- The following command reads the hierarchical DEF files:

```
init_hier_design -defs $designDir/Data/def/design1.def $designDir/Data/def/Cpu.def
$designDir/Data/def/design2.def -skip_signal
```


read_def

```
read_def
[-help]
<defFile>
[-force_create_phy_inst]
[-keepPinGeometry]
[-preserve_shape]
[-rdl_def def_file]
[-rdl_orientation { N|S|W|E|FN|FS|FE|FW }]
[-rdl_placement {X Y}]
[-read_flat_data dir]
[-skip_pg]
[-skip_signal]
[-top]
[-top_scope]
[-top_scope_ignore_block_internal_nets_on_boundary_path]
[-topcell_orientation { N|S|W|E|FN|FS|FE|FW }]
[-topcell_placement {X Y}]
[-write_flat_data dir]
```

Reads the DEF files. This command can be used to supply hierarchical DEFs to generate a flattened physical database. `read_def` supports multi-level hierarchy flattening. The command also allows you to read and write floorplan, placement, and routing flat binary files from the specified directory.

The `read_def` command can read multiple DEF files for the design with multiple levels of hierarchy and display a flattened database as shown below:

```
read_def top.def block1.def block2.def
```

When multiple DEFs are passed to `read_def`, the software flattens all the levels of hierarchical DEFs and displays layout for the design.

 All hierarchical DEFs are passed to power and rail analysis without flattening and processed by Power Analysis and Rail Analysis engines separately. The flattened DEF is only used for display of layout in the GUI. The flattened DEF should not be used for analysis.

See *Innovus Text Command Reference* for information on the `defOut` command.

The flattened DEF can then be loaded in new session as follows:

```
read_def top.def block1.def block2.def block3_flat.def
```

Note: `read_lib -lef` is required in order to load DEFs. This command should never include LEF

definition of the block level partitions. It should only include LEF definition for primitive cells like standard cells, IOs and memories.

Parameters

<code>-force_create_phy_inst</code>	
	Specifies to force create physical cell instances. By default, the software skips physical instances (like fillers) without the “+ SOURCE DIST” attribute when reading DEF. This parameter considers instances without “+ SOURCE DIST” in DEF as physical instances, and adds them into the netlist during analysis.
<code>-help</code>	Outputs the command usage and a brief description about the command parameters.
<code><defFile></code>	Specifies the name of the DEF files, starting with the top-level DEF. It is mutually exclusive to the <code>-read_flat_data</code> parameter.
<code>-keepPinGeometry</code>	
	Specifies to keep pin shapes equivalent to the block pin shapes in the form of special route. This parameter ensures that the pin shapes of all layers in the hierarchical DEF are retained in routing after merge of the hierarchical DEF files.
<code>-preserve_shape</code>	
	Specifies to prevent creation of symbolic center-line connected routing data and preserve the routing data as-is.
<code>-rdl_def def_file</code>	
	Specifies to instantiate the RDL instance/DEF at the top level and merge it with the DEF of the top level block being analyzed.
<code>-rdl_orientation { N S W E FN FS FE FW }</code>	
	Specifies the orientation of the RDL DEF file.
<code>-rdl_placement {X Y}</code>	
	Specifies to place the RDL at a given X,Y location (in micron).

<code>-read_flat_data dir</code>	
	Specifies to read binary flat data files from the specified directory. The flat data directory is generated by the <code>-write_flat_data</code> parameter. This parameter is mutually exclusive to <code>[list_of_DEFs]</code> .
<code>-skip_pg</code>	Skips the power and ground nets with <code>USE POWER/GROUND</code> attributes when reading the input list of DEF files. This parameter can be used to reduce the memory footprint in the signal electromigration flow.
<code>-skip_signal</code>	Reads PG nets with <code>USE POWER/GROUND</code> attributes in both <code>SPECIALNETS/NETS</code> sections in the input list of DEF files. Skips signal nets when reading <code>[list_of_DEFs]</code>. This parameter is not available with <code>-read_flat_data</code> and <code>-write_flat_data</code> parameters.
<code>-top</code>	Specifies to merge two or more flattened DEF files with the same top cell name. The <code>-top</code> parameter does not support hierarchical DEF. The hierarchical DEF is read by specifying it directly with the <code>read_def</code> command, as shown below: <pre>read_def {chip_top_routed.def chip_top_floorplan.def block_hierachical_routed.def}</pre>
<code>-topcell_placement {X Y}</code>	
	Specifies to place the top DEF block at a given X,Y location (in micron) with respect to virtual top (0,0).
<code>-topcell_orientation { N S W E FN FS FE FW }</code>	
	Specifies the orientation of the top DEF block.
<code>-top_scope</code>	Reads the top-level nets only, and skips creation of the physical nets. By default (when <code>-top_scope</code> is not specified), the <code>read_def</code> command will create physical nets in the physical database if nets in the <code>SPECIALNETS</code> section do not exist in the database. This parameter must be used in the top scope signal electromigration flow to skip reading of these physical nets. The <code>read_def -top_scope</code> parameter can be used only after the running the <code>create_top_scope</code> command.

<code>-top_scope_ignore_block_internal_nets_on_boundary_path</code>	
	Specifies to ignore the physical layout of the internal nets from the first instance to the first flop on the boundary path for the signal EM checking. If this parameter is used with <code>read_def</code>, you must specify the same parameter with <code>verify_AC_limit</code>. The <code>read_def -top_scope_ignore_block_internal_nets_on_boundary_path</code> parameter can be used only after the running the <code>create_top_scope</code> command. The <code>read_def -top_scope_ignore_block_internal_nets_on_boundary_path</code> parameter is mutually exclusive with the <code>-top_scope</code> parameter.
<code>-write_flat_data dir</code>	
	Specifies the output directory in which the flat data is saved. This parameter is specified with <i>[list_of_DEFs]</i>.

Examples

- The following command reads the compressed DEF file `dma_mac_filled.def.gz`:

```
read_def dma_mac_filled.def.gz
```
- The following command reads the two flattened DEF files `top_level_flattened_floorplan.def` and `top_level_flattened_routing.def` at the top level:

```
read_def -top top_level_flattened_floorplan.def top_level_flattened_routing.def
```

read_design

```
read_design
[-help]
[dir dir topcell]
[-black_box_modules <blackBoxModules>]
[-cellview {lib cell view}]
[-common]
[-dynamic_view userDynamicView]
[-exclude_path_collection]
[-hold_views userHoldViews]
[-leakage_view userLeakageView]
[-mmmcFile userMmmcFile]
[-lef_files lefFiles]
[-module_dirs moduleDirs]
[-no_timing_graph]
[-oaRef oaRefLib]
[-physical_data]
[-setup_views userSetupViews]
```

Loads a saved design database. You can also read the physical data such as floorplan, placement and routing using the `-physical_data` parameter.

Note: You can also use the `read_def` command to read the DEF file for physical information.

Parameters

<code>-help</code>	Prints out the command usage.
<code>- black_box_modules <i>blackBoxModules</i></code>	Specifies modules for which black box models should be loaded.
<code>-cellview {<i>lib cell view</i>}</code>	Restores an OpenAccess design. It specifies the name of the library, top cell, and cell view for the OA design. This parameter works both for the physical and non-physical flow.
<code>-common</code>	Specifies to read the common-db design directory.
<code>dir <i>dir topcell</i></code>	Specifies the design data directory name and topcell.

<code>-dynamic_view userDynamicView</code>	Specifies the analysis view for the dynamic power analysis.
<code>-exclude_path_collection</code>	Restores design data without archived path collection.
<code>-hold_views userHoldViews</code>	Specifies the hold views for loading, rather than the hold views active during <code>save_design</code> .
<code>-leakage_view userLeakageView</code>	Specifies the analysis view for leakage power analysis.
<code>-lef_files lefFiles</code>	Specifies the LEF files for loading. You can use this parameter, if you need to override the existing LEF files in the design being restored and load different LEF files. • This option is intended to allow upgrading to new versions of LEF files that are compatible with the old versions. The list of all LEF files used by this design must be given. • If any pin locations or cell boundaries or OBS shapes are changed for used cells, new DRC violations may occur. • If pins are added or removed from used cells, or used cells are removed, fatal errors may occur when loading the netlist. • If the removed pins and cells are unused in the design, warnings or errors may still occur if any library-specific data was stored for those cells or pins during the design flow (for example, <code>dont_touch</code> data)
<code>-mmmcFile userMmmcFile</code>	Specifies the name of the path to a <code>viewDefinition.tcl</code> (MMMC) file.
<code>-module_dirs moduleDirs</code>	Specifies module-level sessions that should override the corresponding modules in the top-level hierarchical session.
<code>-no_timing_graph</code>	Restores the design data without a timing graph.
<code>-oaRef oaRefLib</code>	Reads the OA reference library.
<code>-physical_data</code>	Reads the physical data such as floorplan, placement, and routing that you generate.

<code>-setup_views userSetupViews</code>	Specifies the setup views for loading.
--	--

Examples

- The following command loads the saved database `dtmf_chip`:

```
read_design dtmf_chip.dat dtmf_chip
```

- The following command reads a design called `dtmf_chip` with physical data.

```
read_design -physical_data dtmf_chip.dat dtmf_chip
```

- The following command loads the 'layout' view of the 'design1' cell from the 'designLib' libraries of an OA database.

```
read_design -cellview {designLib design1 layout}
```

- The following command loads the 'layout' view of the 'dtmf_chip' design from the 'designLib' libraries of an OA database in physical mode.

```
read_design -cellview "designLib dtmf_chip layout" -physical_data
```

read_hier_design

```
read_hier_design
dir_name topcell_name
[-add_on_shape_def {{def1 x_loc y_loc orient} {def2 x_loc y_loc orient}...} ]
```

The `read_hier_design` command restores the hierarchical design information (hierarchical database) from the previous design session saved in the software. In order to restore a design in the hierarchical database format, you must have saved the design using the `save_hier_design` command in the software session.

Parameters

- help	Outputs the command usage and a brief description about the command parameters.
<i>dir_name topcell_name</i>	
	Specifies the name of the design data directory and top cell.
<i>-add_on_shape_def {{def1 x_loc y_loc orient} {def2 x_loc y_loc orient}...}</i>	
	Specifies to merge physical shapes (such as pseudo what-if shapes) from additional side DEF files to the current top database. This parameter allows you to add multiple side DEF files together. For each DEF file, you need to specify the DEF file name (def1, def2), x and y or the physical shape coordinates, and the shapes's orientation. A shape can have any of the following orientations: N (North), S (South), W (West), E (East), FN (Flipped North), FS (Flipped South), FW (Flipped West), or FE (Flipped East).

Examples

- The following command reads the design data from the specified directory and top cell:
`read_hier_design saveDB Chip`

read_instance_voltage

```
read_instance_voltage  
[-help]  
-ir_drop {irDropFile}  
[-early | -late]  
[-list]  
[-min | -max]  
[-reset]
```

Loads the instance-based IR-drop information that is output by power-rail analysis tools, such as Voltus, to account for the effects of IR drop when calculating the delays and slews. You can read in four different files for early-max, early-min, late-max, and late-min IR-drop analysis.

Parameters

-help	Prints out the command usage.
-early -late	
	Applies the effects of IR-drop to the early path (data hold/clock setup) or late path (data setup/clock hold) in your design. Use the <code>-early</code> or <code>-late</code> parameter in conjunction with the <code>-min</code> or <code>-max</code> parameter to account for the effects of early-max, early-min, late-max, or late-min IR-drop analysis.
-ir_drop {irDropFile}	
	Specifies the names of the IR-drop files to load.
-list	Provides a list of all specified IR-drop files.

<code>-min</code> <code>-max</code>	Applies the effects of IR-drop to the minimum operating condition or maximum operating condition. Use the <code>-min</code> or <code>-max</code> parameter in conjunction with the <code>-early</code> or <code>-late</code> parameter to account for the effects of early-max, early-min, late-max, or late-min IR-drop analysis. Note: If the <code>-min</code> or <code>-max</code> parameter is used without the <code>-early</code> or <code>-late</code> parameter, the IR drop is applied to both early and late analysis. To apply the IR drop for late analysis only, use the <code>-min</code> or <code>-max</code> parameter with the <code>-late</code> parameter. To apply the IR drop for early analysis only, use the <code>-min</code> or <code>-max</code> parameter with the <code>-early</code> parameter.
<code>-reset</code>	Resets all IR-drop files.

Examples

- The following commands apply the IR drop for early-min, late-min, early-max, and late-max IR-drop analysis:

```
read_instance_voltage -early -min -ir_drop { earlymin.ir }
read_instance_voltage -late -min -ir_drop { latemin.ir }
read_instance_voltage -early -max -ir_drop { earlymax.ir }
read_instance_voltage -late -max -ir_drop { latemax.ir }
```

- The following commands apply the IR drop for late-min and late-max IR-drop analysis:

```
read_instance_voltage -late -min -ir_drop { latemin.ir }
read_instance_voltage -late -max -ir_drop { latemax.ir }
```

- The following commands apply the IR drop to early and late IR-drop analysis:

```
read_instance_voltage -early -ir_drop { early.ir }
read_instance_voltage -late -ir_drop { late.ir }
```

read_lib

```
read_lib
-help
[-aocv aocv files]
[-cdb cdb_list]
[-lef file_names]
[-library_side_file <<side files>>]
[-max_aocv max_aocv files]
[-max_cdb max_cdb_list]
[-max max_lib_list]
[-min_aocv min_aocv files]
[-min_cdb min_cdb_list]
[-min min_lib_list]
[-oaRef OA_reference_libs]
[-pgv power_grid_list]
[-socv socv files]
```

Schedules reading of one or more technology library files. These files either contain timing-related or noise-related information for the primitive cells used in the design. When you use the `read_lib` command, the library information is scheduled for reading but is actually read in after you use the `set_top_module` command.

Timing libraries can be in the text format or Cadence binary library format (LDB). The LDB files generated from Innovus and Genus are read in Voltus with an error message (TECHLIB-1249). The Tempus generated LDB files can be used in Tempus, Innovus, Voltus, Genus, and Conformal software tools. The LDB files are not forward compatible, that is, an LDB generated from the 15.2 version cannot be used with the 15.1 or prior versions of the software.

Note: You can use wildcards with the `read_lib` command as follows:

```
read_lib [glob *.lib]
```

Parameters

-help	Prints out the command usage.
-aocv <i>aocv_file_list</i>	

	Reads the AOCV libraries with timing libraries. In this case, as compared to worst OCV, the tool annotates less pessimistic derate factors from the given AOCV library on to design elements (cells/nets) resulting in less pessimistic timing results. To enable the AOCV flow, you also need to set the following while reading the AOCV libraries: <pre>set_analysis_mode -aocv true</pre>
<code>-cdb cdb_list</code>	
	Specifies the name of the noise library file(s).
<code>-lef file_names</code>	
	Specifies the name of the LEF files to be loaded. <code>dotlib_file</code> and <code>-lef file_names</code> are mutually exclusive.
<code>-library_side_file <<side files>></code>	
	Specifies the name of the incremental side libraries.
<code>-max max_lib_list</code>	

	Specifies the libraries to be used for maximum timing analysis.
<code>-max_aocv max_aocv files</code>	
	Reads the maximum AOCV libraries.
<code>-max_cdb max_cdb_list</code>	
	Reads the maximum noise libraries.
<code>-min min_lib_list</code>	
	Specifies the libraries to be used for minimum timing analysis.
<code>-min_aocv min_aocv files</code>	
	Reads the minimum AOCV libraries.
<code>-min_cdb min_cdb_list</code>	
	Reads the minimum noise libraries.
<code>-oaRef OA_Reference_libs</code>	
	Loads OpenAccess reference libraries in the physical flow.
<code>-pgv power_grid_list</code>	

	Reads the power-grid library files.  This parameter is required when you run Voltus.
<code>-socv socv files</code>	
	Reads the SOCV libraries with the timing libraries. To enable the SOCV flow, you also need to set the following while reading the SOCV libraries: <pre>set_analysis_mode - socv true</pre>

Examples

- The below command reads the LEF files. Note that the first file is a technology LEF.

```
read_lib -lef tech.lef allcells.lef
```
- The next command reads the minimum libraries.

```
read_lib -min stdcells_min.lib memories_min.lib
```
- The next command reads the maximum libraries.

```
read_lib -max stdcells_min.lib memories_min.lib
```
- The following command reads a list of timing libraries to be used for minimum and maximum analysis.

```
read_lib -min { fast.lib mem1_min.lib macro2_min.lib } -max { slow.lib mem1_max.lib
macro2_max.lib }
```
- The following command reads a User Defined Noise model (UDN) and the noise library (cdB):

```
read_lib -cdb {std_cells.cdB cells.udn}
```

read_power_domain

```
read_power_domain
-help
<powerDomainFile>
[-1801 <IEEE1801File>]
[-cpf file_name ]
```

Reads the power domain file created in Innovus System, the MSMV file, or the Common Power Format (CPF) file. The power domain, MSMV, or CPF file is required to perform timing and signal integrity analysis for MSMV designs.

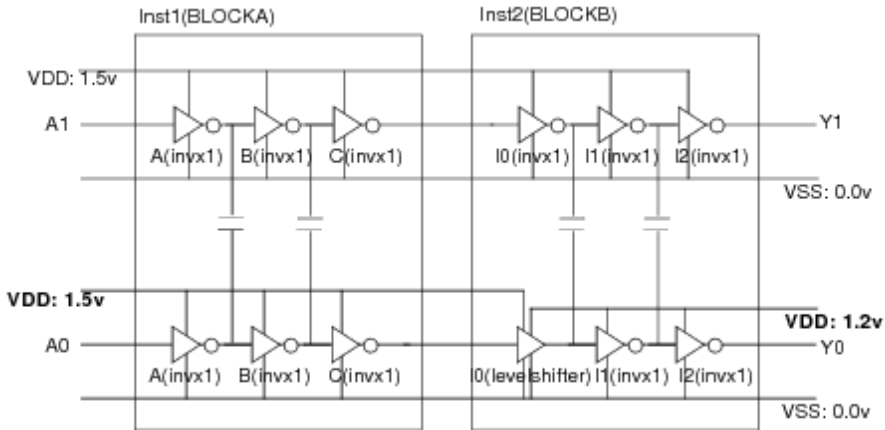
Note: If you read the CPF file with view and commit options, the software switches to Multi-Mode Multi-Corner analysis and resets the parasitics. Use the `read_spef` command to reload the parasitics.

Parameters

-help	Prints out the command usage.
<powerDomainFile>	Specifies the name of the power domain file created in Innovus System. This file contains the definition of power domain and membership information and uses a .pd extension.
-1801 <IEEE1801File>	Specifies the input file in the IEEE1801 format. While using IEEE 1801 support, note the following: - For Low Power (LP) cell definition: All the LP cells and their related power pin information must be defined in the Liberty file with the Liberty LP attributes. - For Timing Information: Timing information must be defined in Tempus MMMC viewDefinition.tcl.
-cpf file_name	Specifies the name of the Common Power Format (CPF) file, which contains the power domain definitions. This file uses a .cpf extension. The CPF file can be created in Innovus System.

Examples

- The following figure displays a circuit for which the power domain file is created in the tool.



The following power domain file is created:

```
createPowerDomain pd_1.2 -timingLibs {ss_1.2v 1vlshft}
createPowerDomain pd_1.5 -default -timingLibs {ss_1.5v}
modifyPowerDomainMember pd_1.5 -instances *
modifyPowerDomainMember pd_1.2 -instances Inst2/I0 Inst2/I1 Inst2/I2
```

- The following command reads the power domain file `dma_mac.cpf`:

```
read_power_domain -cpf dma_mac.cpf
```

- The following is the sample script for Tempus IEEE1801 LP support:

```
# read MMC include the library
read_view_definition ../INPUT/viewDefinition.tcl
```

```
# read in verilog
read_verilog ../INPUT/dma_mac_postroute.v
set_top_module dma_mac
read_power_domain -1801 <IEEE1801File>
```

```
# The rest of Tempus LP Flow (same as CPF low power flow)
```


read_rcdb

```
read_rcdb  
[-help]  
dirname
```

Restores the parasitic data that was previously saved with the `write_rcdb` command.

When you use the `read_rcdb` command, the design data must be the same as when you use the `write_rcdb` command. You can restore the parasitic data on systems that are same as those used for saving the data. For example, if the data was saved on Linux in 32bit mode, you can restore it on Linux 32 bit mode only.

 Restoring RCDB that you generated using previous major version(s) of the software is not supported and might result in an error. Use `write_rcdb` to generate a new RCDB.

Parameters

<code>-help</code>	Prints out the command usage.
<code>dirname</code>	
	Specifies the name of the parasitic file that you saved using the <code>write_rcdb</code> command. Note: The parasitic file must have the <code>.rcdb.d</code> extension.

Examples

- The following command reads an Innovus RC extracted database:

```
read_rcdb test.rcdb.d
```

read_sdc

```
read_sdc  
[-help]  
[-reset]  
fileName
```

Loads a timing constraints file. By default, the constraints files are loaded incrementally. Use this command after importing the design and loading the timing library.

A list of constraints files also can be specified in the configuration file.

Parameter

-help	Prints out the command usage.
<i>fileName</i>	Specifies the name of the constraint file.
-reset	Replaces all previously loaded constraints information with the information in the specified <i>fileName</i>. Note: Alternatively, you can use the <code>reset_sdc</code> command to reset the constraints files.

Examples

- The following command reads a SDC file called `dma_mac_m.sdc`:

```
read_sdc dma_mac_m.sdc
```

read_spef

```
read_spef
  [-help]
  list_of_fileNames
  [-decoupled]
  [-extended]
  [-rc_corner rcCornerName]
  [-rc_scale]
  [-scale_nets netListFile]
  [-spef_field integer_list]
  [-starN | -noStarN]
  [-via_variation_file viaVariationFileName]
```

Loads resistors and capacitors for the interconnects in SPEF into the software to calculate delays.

Note: The `read_spef` command supports { }, (), [], and < > bus delimiters while reading SPEF files.

Parameters

-help	Prints out the command usage.
-decoupled	Grounds the coupling capacitances in the SPEF file and merges the capacitances on the same node. This parameter can be used only for static timing analysis and cannot be used for SI analysis.
-extended	Enables the reading of “\$llx=<value1> \$lly=<value2> \$urx=<value3> \$ury=<value4> \$LAYER=<number>” to set the layer number for nodes of a resistance from Quantus extended SPEF file. It also reads in extended *LAYER_MAP section, if it is present. This parameter can also read in “\$l=<length> \$w=<width> \$si_w=<swidth> \$lvl=<number>” to set the width, silicon width, or layer for nodes of a resistance from StarRC extended SPEF file.
list_of_fileNames	Specifies the SPEF file. You can specify multiple files in a single command.

<code>-rc_corner</code> <code>rcCornerName</code>	Annotates the parasitics to the predefined RC corner for multi-corner analysis. If you want to use SPEF information when calculating delays in multi-mode multi-corner analysis mode, you must annotate the parasitics for each RC corner in the design. For example, if you have four active RC corners in the design, you must annotate the parasitics for each RC corner. <pre>read_spef -rc_corner corner1 rc1.spef read_spef -rc_corner corner2 rc2.spef read_spef -rc_corner corner3 rc3.spef read_spef -rc_corner corner4 rc4.spef</pre> Note: SPEF reading will not be triggered until SPEF files for all active RC corners are specified.
<code>-rc_scale</code>	Scales the SPEF read in for the specified corner as per the <code>postRoute_cap</code>, <code>postRoute_xcap</code>, and <code>postRoute_res</code> scale factors defined using the <code>create_rc_corner</code> and <code>update_rc_corner</code> commands.
<code>-scale_nets netListFile</code>	
	Applies scaling to the specified nets.
<code>-spef_field integer list</code>	
	Specifies the list of RC corner indices in SPEF to be loaded. This parameter provides the flexibility to perform order-independent and field-independent mapping to RC corners. For example, SPEF may contain data for 10 corner fields but you can choose to read data only for 6 corners. You can also choose to use data from one particular field to map to one or more RC corners. A few examples to illustrate different scenarios are given in the Examples section. Note: This parameter is relevant only when the <code>-rc_corner</code> parameter is specified. The <code>-spef_field</code> is an optional parameter, and if not specified, the software assumes the default fields of {1 2 3 ... N} where N is equal to the number of RC corners provided in the <code>-rc_corner</code> list.

<code>-starN -noStarN</code>	Specifies whether or not to read the *N statements in the SPEF file. The *N statements define the location of the RC nodes. The <code>-starN</code> parameter reads the *N statements and the <code>-noStarN</code> parameter ignores the *N statements. Default: <code>-noStarN</code> (By default, the software ignores the *N statements while reading SPEF.)
<code>-via_variation_file viaVariationFileName</code>	Specifies the name of the file for modeling via variation. The contents of the file include all the via layer names with the corresponding MEAN_SHIFT and STDDEV so that all the VIA resistance in SPEF can be mapped to their own variation multiplier. This option is used to specify the via_variation file only when the design has a single corner. For example, <pre>read_spef -rc_corner rcCornerName \ -via_variation_file viaVariationFileName</pre> The via_variation file format is provided below: <pre># ViaLayerName Area MEAN_SHIFT STDDEV : Format applied when three values are given AREA_TOLERANCE 0.005 # to define area tolerance. To be smaller than half the difference between 2 areas START_VIA VIA1 0.1 0.49 # default MEAN_SHIFT & STDDEV of VIA1, applied when no area defined VIA1 0.0040 0.11 0.50 # applied when area is equal to 0.0040, MEAN_SHIFT=0.11 & STDDEV=0.5 VIA1 0.0050 0.12 0.51 # applied when area is equal to 0.0050 VIA2 0.2 0.51 # default of VIA2, applied when no area defined VIA2 0.0060 0.13 0.61 # applied to VIA2, when area is equal to 0.0060 VIA2 END_VIA</pre>

Examples

- The following command loads the SPEF file `TOPCHIP_SP.spef`:

```
read_spef TOPCHIP_SP.spef
```

- The following command loads multiple SPEF files for hierarchical SPEF stitching:

```
read_spef { PTN/results_conv/results_conv.spef tdsp_core/tdsp_core/  
tdsp_core.spef PTN/TOPCHIP_SP/TOPCHIP_SP.spef }
```

- The following command loads a SPEF file `pnta-spef.gz` that has RC fields from 5 RC corners:

```
read_spef -rc_corner {rcCorner1 rcCorner2 rcCorner3 rcCorner4 rcCorner5} -spef_field  
{1 2 3 4 5} pnta-spef.gz
```

The above command will be interpreted as follows:

```
1st field in SPEF maps to RC corner rcCorner1  
2nd field in SPEF maps to RC corner rcCorner2  
3rd field in SPEF maps to RC corner rcCorner3  
4th field in SPEF maps to RC corner rcCorner4  
5th field in SPEF maps to RC corner rcCorner5
```

- The following command loads a SPEF file `pnta-spef.gz` that has RC fields from N RC corners:

```
read_spef -rc_corner {rc_corner1 rc_corner2} -spef_field {2 5} pnta-spef.gz
```

The above command will read SPEF data for 2 RC corners only and will be interpreted as follows:

```
2nd field in SPEF maps to RC corner rc_corner1  
5th field in SPEF maps to RC corner rc_corner2
```

- The following command loads a SPEF file `pnta-spef.gz` that has RC fields from N RC corners:

```
read_spef -rc_corner {rcCorner1 rcCorner2 rcCorner3} -spef_field {1 1 3} pnta-spef.gz
```

The above command will read SPEF data for 3 RC corners only and will be interpreted as follows:

```
1st field in SPEF maps to RC corner rcCorner1  
1st field in SPEF maps to RC corner rcCorner2 as well.  
3rd field in SPEF maps to RC corner rcCorner3
```

- The following command loads hierarchical design SPEF files. The software reads the top level multi-field SPEF file followed by the multi-field SPEF file from `block1` and `block2`:

```
read_spef -rc_corner {rcCorner1 rcCorner2 rcCorner3 rcCorner4 rcCorner5} -spef_field  
{1 2 3 4 5} {top.spef.gz block1.spef.gz block2.spef.gz}
```

read_stream

```
read_stream
[-help]
-layer_map layer_map_files
{[-pvs_fill ] | [[in_files] [-cells cell_list [-offset {x y}]]]}
```

Reads in-stream files or a layer map and outputs a summary of the cell names read in, and any layer/data types that have data with no map_file entry. The data is automatically scaled during reading to the correct DB units based on the stream units.

- It reads in the metal fill stream data (with top cell name and offset) to overlay on the design. The fill data is displayed to database fill shapes (equivalent to reading DEF FILL data) to allow editing or changing the fill data.
Reading in fill data is done once per session.
- It reads in-stream cells that have the same names as Voltus cell names to display the stream cell data (for example, overlay or toggle between stream shapes and LEF shapes for the cells).
If a stream cell name is repeated, it replaces any previous data for that cell.

Parameters

-help	Outputs a brief description that includes type and default information for each <code>read_stream</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <code>man read_stream</code> .
-cells cell_list	Reads only the specified cells from GDS files.
in_files	Lists GDS and OASIS files.
-layer_map layer_map_files	Specifies the layer map file to map GDS layer to design layer
-offset x y	Specifies, in microns, the offset for the top-level fill cell.
-pvs_fill	Specifies to process the PVS FILL data in the read process.

Layer Map File Format

The `-layer_map` file maps stream layer/dtype data to Voltus layer and objects. Stream data does not have net ids, so you can only map stream data for display, or to fill, fill_opc or fill_blockage shapes.

The format for the layer map file is one line for each stream layer and dtype:

layer data_type layer_name object_type

Where

- *layer* is an stream layer number. It is an integer.
- *data_type* is a stream data_type number. It must be either an integer, or a range (e.g. 0-255).
- *layer_name* is normally must be a LEF (or OA tech) layer name. It can only be a “custom” layer name if the obj_type is “display” (e.g. this data is not on any of the LEF layers, but is useful for display purposes). Any custom layer data will be displayed like data created with add_gui_shape or add_gui_text. The *layer_name* is case-sensitive.
- *object_type* is one of these keywords:
 - *fill*: Equivalent to DEF FILL shapes
 - *fill_opc*: Equivalent to DEF FILL with + OPC shapes
 - *fill_blockage*: Equivalent to DEF BLOCKAGE + FILLS shapes
 - *display*: Is only for display purposes, and cannot be flattened.

fill, *fill_opc*, and *display* can optionally be followed by

:mask:mask_num

where *mask_num* is the mask number for the shape for a multi-mask layer (equivalent of DEF + MASK *mask_num* for a shape).

fill_blockage can optionally be followed by

:name:blockage_name

where *blockage_name* is the name given to all the blockages from this layer/dtype.

Any text on the same layer/data_type will be mapped to the same DB layer as a text object on that layer. About the TEXT font and size, font 0 could be default (or other font decided by RD). Size could use the original size in GDS and scale it if dbu mismatch to Voltus.

The *object_type* keywords are case insensitive. The tool ignores blank lines or lines beginning with #.

A given layer/dtype combination can only appear once.

Example of Stream map-file for metal fill, via-fill, and fill blockages

```
#Stream map-file for metal fill, via-fill, and fill blockages

#only metall has two masks

#layer      dtype      layer_name      obj_type
1           1          metall         fill:mask:1
```


1	2	metal1	fill:mask:2
51	1	cut1	fill
2	1	metal2	fill
52	1	cut2	fill
3	1	metal3	fill
...			
101	1	metal1	fill_blockage:name:blk1
101	2	metal1	fill_blockage:name:blk2
101	3-255	metal1	fill_blockage

Example of Stream map-file for cell-data to display on top of LEF cell data

```
#Stream map-file for cell-data to display on top of LEF cell data
#Only routing layers and the cell-outline on custom layer "outline"
#are mapped. It does not show cut layers or layers below metal1.

#layer      dtype      layer_name  obj_type
1           0-255      metal1      display
2           0-255      metal2      display
...

#custom layer "outline" is created if it does not already exist
100         1          outline      display
```

Examples

Parse stream cells with specified layer map display stream cell data on top of LEF cell data. The report output is redirected to read_stream.rpt.

```
>read_stream {stdcells.gds memory.gds} -layer_map gds_layer.map > read_stream.rpt
```

Read in metal-fill data and overlay it on the design. The report will be in the log file.

```
>read_stream fill.gds -cells top_fill -layer_map gds_layer.map
```

Example of output report

```
#Cell names read in:
```

top_fill

fill_1

fill_2

...

#Layer/dtype data in the stream file with no map-file entry

1 10

4 1, 9

...

read_twf

```
read_twf
[-help]
[filenames]
[-cell master_cell_name]
[-prefix string]
[-quiet]
[-reset]
[-scope string]
[-skipconst]
[-skiptw]
[-strip_prefix string]
[-verbose]
[-view]
```

Reads the external Timing Window File (TWF) for noise and power calculation in CPE and Tempus SI.

Parameters

<i>filenames</i>	Specifies the names of the timing window files.
<i>-cell master_cell_name</i>	

	Specifies the master cell name for the timing window file. This parameter allows you to support master cell based TWF annotation. That is, you need to specify only the master cell name for the timing window file, instead of specifying all the instances that were instantiated from a master cell. <u>Example</u> Let us consider a design that has a cell named "cpu_core" with two instances "cpu_core_0" and "cpu_core_1": <u>Use Model for Instance-based TWF annotation</u> <pre>read_twf -scope cpu_core_0 cpu_core.twf.gz read_twf -scope cpu_core_1 cpu_core.twf.gz</pre> <u>Use Model for Master Cell based TWF annotation</u> <pre>read_twf -cell cpu_core cpu_core.twf.gz</pre>
-prefix <i>string</i>	Adds a prefix to the net or pin name.
-quiet	Skips information and warning messages.
-reset	Clears the previously read TWF files.
-scope <i>string</i>	Specifies the scope for the given hierarchical timing window file.
-skipconst	Skips constant processing.
-skiptw	Skips timing windows processing.
- strip_prefix	Removes hierarchical path name.
-verbose	Prints the command progress details.
-view	Specifies the view for the timing window file.

read_verilog

```
read_verilog
-disable_name_match
netlist
```

Schedules reading of a structural, gate-level verilog netlist. When you use the `read_lib` command, the verilog information is scheduled for reading but is actually read in after you use the `set_top_module` command. The verilog netlist must be fully mapped to the gate-level, and may not contain any verilog high-level constructs.

To schedule more than one verilog file for reading, you can either list all the files in one single command or use multiple `read_verilog` commands.

Parameter

<code>-disable_name_match</code>	
	Disables the file name matching feature. When the file name matching feature is enabled (default) and the specified Verilog file is not present in the working directory, the software searches for the <code>.gz</code> version of the file. For example, if the <code>sample.v</code> file is specified and is not found, the software searches for the <code>sample.v.gz</code> file and uses it if it is available. Similarly, if the <code>sample.v.gz</code> file is specified but is not available, the software searches for <code>sample.v</code> file and uses it if it is available.
<code>netlist</code>	Name of the verilog netlist.

Example

- The following command reads the verilog netlist, `dma_mac.v`:
`read_verilog dma_mac.v`
- The following command reads in the verilog netlists defined in `file1.v`, `file2.v` and `file3.v`:
`read_verilog file1.v file2.v file3.v`

reset_sdc

```
reset_sdc  
[-help]
```

Resets the timing constraints information. Alternatively, you can use the `-reset` parameter of the `read_sdc` command.

Parameter

<code>-help</code>	Prints out the command usage.
--------------------	-------------------------------

Examples

- The following command resets the timing constraint information:

```
reset_sdc
```

restore_oa_design

```
restore_oa_design  
[-help]  
<lib>  
<cell>  
<view>  
[-physical_data]
```

Restores all logical design information from the previous design session saved in the software, using the OpenAccess database format. The `restore_oa_design` command cannot restore physical data.

In order to restore a design in the OpenAccess format, you must have saved the design using the `saveOaDesign` command in the software session.

Note: The parameters of this command are order dependent and must be specified in the order shown in the syntax.

Parameters

<code>-help</code>	Prints out the command usage.
<code>cell</code>	Specifies the name of the top cell to restore.
<code>lib</code>	Specifies the name of the OpenAccess library where the design has been saved. When you save a design in OpenAccess format, the library name you specify (along with the path to the library) is saved in a file called <code>libs.def</code>. When you restore a design that was saved in OpenAccess format, the software checks the <code>libs.def</code> file for the library, to find the library's location.
<code>-physical_data</code>	Restores the physical data of the design along with other information.
<code>view</code>	Specifies the name of the cell view to restore.

Examples

- The following command restores the Open Access design called dma_mac. The library is called OAlibrary and the view is layout:

```
restore_oa_design OAlibrary dma_mac layout -physical_data
```


save_design

```
save_design
[-help]
<session>
[-common]
[-cellview {lib cell view}]
[-def]
[-keep_partial]
[-libs]
[-noRc]
[-no_timing_graph]
[-overwrite]
[-physical_data]
[-user_path]
```

Saves a snapshot of the design data, including the binary timing data, timing constraints file, netlist, and the timing analysis state. In addition, the `save_design` command saves the design in the portable mode by default. In this mode, the design and library data do not have any path dependency. As a result, the saved database can be restored from any directory within the same network. The saved database can also be moved to any directory within the same network without any change.

The software saves all library data (`.lib`, `.cdB`, and LEF files) as absolute paths in the configuration file, so that the saved design can be read from any directory.

The command also writes out a power constraints file called `topcellname_power_constraints.tcl` which can be sourced into other Voltus/Tempus sessions.

Note: By default, the `save_design` command cannot save any modified physical data (for example, data from reading the DEF files or the placement, routing, and floorplan information). If the software is in the physical mode, the `save_design` command copies the physical data from a previously read database in the session being saved, to make the saved session complete. If you intend to modify the physical data, and you want to save the changes, you must specify one of the possible Innovus licenses using the `voltus -lic_startup_extra` option when Voltus is started.

Parameters

<code>-help</code>	Prints out the command usage.
<code>-cellview</code> <code>{libcellview}</code>	Saves an OpenAccess design for the physical flow.

-common	Saves the common-db design data to the specified directory.
-def	Saves the design in the DEF format.
-keep_partial	Keeps the partially saved database for debugging purposes only, because restoring this database is not recommended. If the <code>save_design</code> command does not complete successfully, the session data will be removed by default.
-libs	Normally external file references, such as references to <code>.lib</code>, are saved using symbolic links to the original files inside a <code>libs</code> sub-directory inside the DB directory. This option forces copies of the files, rather than symbolic links, in the <code>libs</code> sub-directory so there are no external file references. This makes the DB directory much larger, but it is suitable for sending to a different network where the library and external data files are not available.
-no_timing_graph	Skips the saving of the timing graph by default. This saves time by preventing the timing update to occur in the case of designs where no timing graph is generated for the session.
-overwrite	Overwrites any previously saved session that has the same <code>session_name.dat</code> with the new session. By default, the software checks if the <code>session_name.dat</code> exists in the design. If it does, the software generates an error and does not execute the <code>save_design</code> command. If you specify the <code>-overwrite</code> parameter, the software removes the previously saved session, and then saves the new one.
-noRc	Does not allow RC parasitic information to be saved. You can use this option when input is already <code>rcdb</code> and you do not want to replicate it in the saved session.

<code>-physical_data</code>	Specifies to save the Innovus database after reading the design for faster restoration of the saved database (<code>read_design -physical_data</code>). Instead of reading multiple DEFs and merging them, you can use this parameter to save the database for full-chip designs with hierarchical DEFs. To use this parameter, you must launch Voltus with the extra Innovus license, as shown below: <pre>voltus -lic_startup_extra invs</pre> Voltus cannot save physical changes to the database, but if you check out the correct extra Innovus license at startup, you can save the physical changes. The allowed license names are: "invs invsb vdixl".
<code>session</code>	Specifies the user-defined name of the current Voltus session. The associated data directory is <code>session_name.dat</code>.
<code>-user_path</code>	By default all external file references (such as. library files) are stored as absolute paths without any symbolic links. So, the DB directory is portable anywhere on the same network. This option stores external file references exactly as you entered it For example, full paths like <code>/x/y</code> or relative paths like <code>../x/y</code>, <code>./x/y</code>, and keeps any symbolic links used inside the path names. Example: Consider that there are two library files <code>a.lib</code> and <code>b.lib</code> under: <pre>/global/project1/libs/mmmc</pre> And two lef files <code>c.lef</code> and <code>d.lef</code> under: <pre>/global/project1/libs/lef</pre> There are two symlinks to the libs: <pre>/usa/db1/libs1.0/mmmc/a.lib → /global/project1/libs/mmmc/a.lib /usa/libs/mmmc/b.lib → /global/project1/libs/mmmc/b.lib</pre> And two symlinks to the lefs: <pre>/usa/db1/libs1.0/lef/c.lef → /global/project1/libs/lef/c.lef /usa/libs/lef/d.lef → /global/project1/libs/lef/d.lef</pre> Your current work dir is <code>/usa/db1.1/</code>. You have an init script like:

```
set init_mmmc_files ./mmmc.tcl

set init_lef_file {/usa/db1/libs1.0/lef/c.lef ../libs/lef/d.lef}

set init_verilog design.v

init_design
```

Where mmmc.tcl has:

```
create_library_set -name test -timing {/usa/db1/libs1.0/mmmc/a.lib
../libs/mmmc/b.lib }
```

Then after saveDesign, **without -user_path**, the libs and lefs will save link to the real files:

```
> saveDesign db1.2
```

The ./mmmc.tcl file is always stored inside the DB dir, so no external reference is used.

By default, external file references are resolved to their fullpath name without any symlinks.

The .lib files inside the mmmc.tcl file will be referenced

as: {/global/project1/libs/mmmc/a.lib
/global/project1/libs/mmmc/b.lib}

The .lef files will be referenced as: {/global/project1/libs/lef/c.lef
/global/project1/libs/lef/d.lef}

This is portable and can be loaded by restoreDesign anywhere on the same network.

/usa/db1.2/libs1.2/mmmc/a.lib	→	/global/project1/libs/mmmc/a.lib
/usa/db1.2/libs1.2/mmmc/b.lib	→	/global/project1/libs/mmmc/b.lib
/usa/db1.2/libs1.2/lef/c.lef	→	/global/project1/libs/lef/c.lef
/usa/db1.2/libs1.2/lef/d.lef	→	/global/project1/libs/lef/d.lef

With -user_path, the libs and lefs keep the symlinks of the current db:

```
> saveDesign db1.2 -user_path
```

The .lib files will be referenced as entered by the

user: {/usa/db1/libs1.0/mmmc/a.lib ../libs/mmmc/b.lib}

The .lef files will be referenced as entered by the

user: {/usa/db1/libs1.0/lef/c.lef ../libs/lef/d.lef}

So `restoreDesign` must be run in the same directory as `saveDesign` was run, or in a directory with the same `./y` and `/x` directory structure.

```
/usa/db1.2/libs1.2/mmmc/a.lib → /usa/db1/libs1.0/mmmc/a.lib  
/usa/db1.2/libs1.2/mmmc/b.lib → ../../../../libs/mmmc/b.lib  
/usa/db1.2/libs1.2/lef/c.lef → /usa/db1/libs1.0/lef/c.lef  
/usa/db1.2/libs1.2/lef/d.lef → ../../../../libs/lef/d.lef
```

Examples

Here are some examples.

- The following command saves a snapshot of the complete timing session to a file called `myDesign`:

```
voltus > save_design myDesign
```

- The following command restores the complete timing session:

```
voltus > read_design myDesign.dat dtmf_chip
```

save_hier_design

```
save_hier_design  
[-help]  
directory_name
```

Saves a snapshot of the physical design data that includes data from reading the DEF files, and the placement, routing, and floorplan information. In addition, the `save_hier_design` command saves the design in the portable mode by default. In this mode, the design data does not have any path dependency. As a result, the saved database can be restored from any directory within the same network. The saved database can also be moved to any directory within the same network without any change.

To use this command, you must launch Voltus with the extra Innovus license, as shown below:

```
voltus -lic_startup_extra invs
```

Parameters

<code>-help</code>	Outputs the command usage and a brief description about the command parameters.
<code>directory_name</code>	Specifies the name of the directory that stores the design data.

Examples

- The following command saves a snapshot of the hierarchical design data to a directory called

```
saveDB:
```

```
save_hier_design saveDB
```

set_cell_power_domain

```
set_cell_power_domain  
[-file file_name]  
[-help]  
[-reset]
```

Defines domains for multi-VDD and multi-VSS cells. When reading an instance ASCII power file to perform power-rail analysis, if there are missing power values associated with ground pins, the software uses the information specified with the `set_cell_power_domain` command to derive domain association and distributes power to each ground net accordingly.

This domain information is stored in a side file that is used by power analysis to calculate power using the correct voltage for each domain. If you are specifying an external instance ASCII power file with pin information, this domain information is used to correctly distribute power to each pin.

Use this command before performing power and rail analysis.

Parameters

<code>-file <i>file_name</i></code>	
	Specifies the name of the file containing the cell names and corresponding power-ground pairs. The format of the file is: <pre>CELL: <cell name1> <power1> <gnd1> <power2> <gnd2> CELL: <cell name2> <power1> <gnd1> ...</pre>
<code>-help</code>	Outputs a brief description for the <code>set_cell_power_domain</code> parameter. For a detailed description of the command, use the <code>man</code> command: <code>man set_cell_power_domain</code>.
<code>-reset</code>	Resets all specified options back to default values.

Examples

- The following command defines two domains, `VDD:VSS` at 1v and `VDDm:VSSm` at 0.84v, of the cell

`mycell` in the file `domain_file`:

```
set_cell_power_domain -file domain_file
```

The following is a snippet from `domain_file`:

```
CELL: mycell
```

```
VDD_1v    VSS_1v
```

```
VDDm_0.84v  VSSm_0.84v
```


set_dc_sources

```
set_dc_sources
{ netNameList }
[-force]
[-global value]
[-power | -ground ]
[-tolgnd float]
[-tolvdd float]
[-voltage value]
```

Specifies the power and ground nets and sets the supply voltages. The command can also set power rail voltages.

You can use the command one or multiple times. If you specify a voltage value, the value applies to either power or ground, but not both. The specified voltages can be used for both, static and dynamic power analysis. In the presence of a trilib set, the internal and leakage power will be interpolated as per the given voltage specification.

Use this command after setting the name of the top module.

Parameters

-force	Creates power net if one does not exist and associates a voltage with it.
-global value	
	Sets the global power voltage value for signal integrity analysis. This option overrides the global power voltage value defined in the power domain or the -lib file. You can also set supply voltage for instances that are not hooked to any power net. This applies to both static and dynamic power analysis. <i>Default:</i> The software uses the global power voltage value defined in the default power domain or the first .lib file, if the power domain is not specified.
netNameList	Specifies the list of nets to apply the supply information to. Always specify the <i>netNameList</i> parameter as the first argument in the command.

<code>-power -ground</code>	
	Specifies whether the specified value is for nominal supply or ground voltage.
<code>-tolgnd</code> <i>float</i>	Specifies the expected voltage deviation in the ground voltage caused by power-supply noise such as IR drop that can occur between two connected cells. <i>Default:</i> 0.0 (units in volts) Note: The <code>-tolgnd</code> option will not be applied to nets that had an entry in the IR-drop file.
<code>-tolvdd</code> <i>float</i>	Specifies the expected voltage deviation in the nominal supply voltage caused by power-supply noise such as IR drop that can occur between two connected cells. <i>Default:</i> 0.0 (units in volts) Note: The <code>-tolvdd</code> option will not be applied to nets that had an entry in the IR-drop file.
<code>-voltage value</code>	
	Specifies the value of the voltage. <i>Default:</i> 0.0 (units in volts) when you use the <code>-ground</code> option. With the <code>-power</code> option, the software uses the voltage from the default power domain or from the first .lib file if power domain is not specified.

Examples

- The following commands sets the nominal power with voltage value:

```
set_dc_sources VDD1 -power -voltage 1.2
set_dc_sources VDD2 -power -voltage 1.5
```
- The following command sets normal power without voltage value. The voltage is used from the first .lib file:

```
set_dc_sources VDD
```
- The following command sets the ground supply:

```
set_dc_sources VSS -ground
```
- The following command sets the global power voltage value to 1.2:

```
set_dc_sources -global 1.2
```


set_import_mode

```
set_import_mode
[-help]
[-discardFloatingVNets {true|false}]
[-keepEmptyModule {true | false}]
[-reset]
[-syncRelativePath {true | false}]
```

Controls aspects of how the software reads in the Verilog netlist and other design files.

Parameters

-help	Outputs a brief description for the <code>set_import_mode</code> parameter. For a detailed description of the command, use the <code>man</code> command: <code>man set_import_mode</code>.
-discardFloatingVNets {true false}	
	Discards floating vnets. <i>Default:</i> false
-keepEmptyModule {true false}	
	Keeps empty modules. If set to <code>false</code>, the software converts the empty modules into special leaf cells that do not have a physical library associated with them. If the module is a hard macro, using a value of <code>true</code> or <code>false</code> produces the identical behavior: the software keeps the module. <i>Default:</i> false
-reset	Resets all or specified options to default value.
-syncRelativePath {true false}	

Synchronizes all relative paths in the configuration file to the current working directory. You can load this configuration file from any directory without first changing your current working directory to the previous working directory where the configuration file was saved.

Default: `true` (You do not need to change your working directory to the directory saved in the previous design session before you load the configuration file.)

Examples

- The following command does not synchronize the relative paths in the configuration file to the current working directory:

```
set_import_mode -syncRelativePath false
```

set_library_unit

```
set_library_unit
[-help]
[-cap capUnit]
[-time timeUnit]
```

Sets the time and capacitance unit values to be used for constraints and generation of reports.

The timing constraints (SDC) files do not contain any unit specification for capacitance and time values. Hence, the units for these values are considered to be same as those specified in the timing libraries. The commands, e.g., `report_timing`, require consistent units to generate reports. If an inconsistency occurs in the timing libraries then the default unit of time (as ns) and capacitance (as pf) is used.

You can use this command when you need to override the library units, or force the setting of units when libraries have different units.

Note: You must set the `set_library_unit` command before the design is read and the constraints are loaded.

Parameters

<code>-help</code>	Prints out the command usage.
<code>-cap capUnit</code>	Specifies the capacitance load unit in the timing library. The valid values are 1pf (pico farad), and 1ff (femto farad). <i>Default:</i> 1.0 pf
<code>-time timeUnit</code>	
	Specifies the time unit to be used for timing constraints. The valid values are 1ns, 1ps, 10ps, and 100ps. <i>Default:</i> 1.0 ns

Examples

- The following command sets the time unit for the design to 10ps:

```
set_library_unit -time 10ps
```

set_license_check

```
set_license_check
[-help]
[-checkout license]
[-default]
[-existOnServer]
[-optionList {license1 license2 ...}]
[-status]
[-wait time_in_minutes]
```

Manages licenses for product options. Specifies licenses to check out, license wait time, and the status of all product option licenses.

Parameters

-checkout <i>license</i>	Checks out the licenses for the specified product options when this command runs.
-default	Restores the default. values for this command.
- existOnServer	Outputs a table that provides the following information for each of the licenses that exist on the license server. • License name • Product number • Product name • License string • Version
-help	Outputs a brief description that includes type and default information for each <code>set_license_check</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <code>man set_license_check</code>.
-optionList { <i>license1 license2 ...</i> }	

	Specifies licenses to check out dynamically. Dynamic licenses are not checked out until they are needed. If you specify a license that is not allowed with your base license, or a license that is not available, the software does not check out that license and instead issues a warning message. If you specify more than one license, begin and end the option list with double quotation marks or braces.
<code>-status</code>	Outputs the following information: • The name of the base license that is checked out • The current wait time • A list of the allowed license options • A list of the available license options • A list of the currently checked-out license options.
<code>-wait <i>time_in_minutes</i></code>	
	Specifies the amount of time the system waits for a license to become available. If the license is available in less than the specified wait time, the system checks out the next needed license without waiting. <i>Default:</i> 0 (no wait time) <i>Value range:</i> 0 to 10,000

Examples

- The following command outputs a table that provides information about the licenses that exist on the license server:

```
set_license_check -existOnServer
```

The system outputs the following information:

Name Version	Prod #	Product Name	License string
vt saa 15.1	VT S201	Voltus Advanced Analysis GXL Option	Voltus_Power_Integrity_AA
vt sng 1.0000	VTSENG100	Voltus Next Generation Product 100	VT S_NG100
tp sxl 15.1	TPS200	Tempus Timing Signoff Solution XL	Tempus_Timing_Signoff_XL
tp sl 15.1	TPS100	Tempus Timing Signoff Solution L	Tempus_Timing_Signoff_L
inv s_ms 15.1	INVS30	Innovus Mixed Signal Option	Innovus_MS_Opt

- The following command specifies that licenses for Tempus Timing Signoff Solution-L and Tempus Timing Signoff Solution-XL may be checked out. Tempus Timing Signoff Solution-L licenses are checked out before Tempus Timing Signoff Solution-XL licenses.

```
set_license_check -optionList {tpsl tpsxl}
```

- The following command outputs license status and wait time and lists the optional licenses that can be checked out and the licenses that are checked out:

```
set_license_check -status
```

The system outputs the following information:

```
Current checked out base license : vtsxl (Voltus_Power_Integrity_XL 13.2).
```

```
When checking-out an optional license, wait for up to : 0 minutes.
```

```
The base license allows the following optional licenses available now on the  
server(s) : vtsaa vtsng tpsxl tpsl encsd
```

```
The automatic optional license check-out is limited by -optionList to : vtsaa vtsng  
tpsxl tpsl encsd
```

Current checked out optional licenses : tpsxl

Summary of all checked out licenses:

Voltus_Power_Integrity_XL:Tempus_Timing_Signoff_XL

- The following command sets the wait time to 5 minutes:

```
set_license_check -wait 5
```

Related Topics

- [Product and Licensing Information](#)
- [Getting Started](#)

set_top_module

```
set_top_module  
[-ignore_undefined_cell]  
[-ignore_timing_library_check]  
topCell
```

Sets the top module name, and checks for consistency between verilog netlist and timing libraries. Voltus uses the top module for timing analysis. You must read the libraries (`read_lib`) and verilog file (`read_verilog`) before using this command. Use this command before reading constraints, parasitic and delay data.

Parameters

<code>-ignore_timing_library_check</code>	
	Ignores the check that each cell have a timing library when reading the verilog file.
<code>-ignore_undefined_cell</code>	
	Creates a user-defined noise model (UDN) for a cell even if the corresponding cdB and .lib files are not available. When you set this option, the software creates a dummy UDN for cells that are not in the loaded cdB or .lib, and uses them during timing and signal integrity analysis.
<code><i>topCell</i></code>	The name of the top module.

Examples

- The following commands sets the top module to `dma_mac`.

```
set_top_module dma_mac
```

specify_def

```
specify_def  
-help  
fileNameList  
[-die_instance_name dieinstname]
```

Specify the name of the DEF file(s). This command can be used in lieu of `read_def` command for analysis purpose. When you use the `specify_def` command, the software passes specified DEF files to the power and rail analysis programs for internal processing without loading their physical data in memory. As a result, you will not be able to display and query layout of the design. These programs will only process the necessary data from DEF file(s). The usage of this command is recommended in dynamic power analysis, and static and dynamic rail analysis for large (>5M components) designs with hierarchical DEFs or in general to obtain better performance during analysis. You will be able to save memory and runtime that takes to load design during analysis. Once the analysis is complete, you can subsequently load DEFs in the same or new session using the `read_def -specialnets_components <DEFs>` command to view design layout and analysis results.

Note that for MSMV design, in order to use `specify_def`, static power analysis mode will require CPF (Common Power Format) that defines logical connectivity of the instances to power nets. If CPF is not available, you can optionally run dynamic power analysis which calculates dynamic as well as static power.

The DEF files must be specified in the top-down order of the design hierarchy, that is, top-level DEF, followed by block-level DEF, and then sub-block DEF. This is illustrated in the following example script:

Example flow script with `specify_def`:

```
read_lib -lef <LEFs>  
read_lib <timing libs>  
read_verilog <verilog file(s)>  
set_top_module <design name>  
read_sdc <timing constraints>  
read_spef -decoupled <signal parasitics file(s)>  
specify_def <top.def> <block.def> <sub_block.def>
```

#---- power calculation -----#

```
read_spef -decoupled ../DATA/spef/dma_mac.spef.gz

# Option (-dynamic_only true) available to turn off static power calculation during
dynamic power calculation
set_power_analysis_mode -reset
set_power_analysis_mode -method dynamic_vectorless -create_binary_db true -
write_static_currents true
set_dynamic_power_simulation -period 14ns -resolution 50ps
set_default_switching_activity -input_activity 0.2 -seq_activity 0.15 -clock_gates_output
2.0 -period 10.0
report_power
```

#---- rail analysis -----#

```
set_rail_analysis_mode -method dynamic -accuracy fast_accurate -power_grid_library {
stdcells.cl memories.cl }
set_power_data -reset
set_power_data -format current {dynamic_VDD.ptiavg dynamic_VSS.ptiavg}
set_pg_nets -net VDD -voltage 1.08 -threshold 0.972 -tolerance 0.3
set_pg_nets -net VSS -voltage 0.0 -threshold 0.108 -tolerance 0.3
set_rail_analysis_domain -name allDomains -pwrnets {VDD VDDm} -gndnets VSS
set_power_pads -net VDD -format xy -file ../DATA/vstorm/dma_mac-VDD.ppl
set_power_pads -net VSS -format xy -file ../DATA/vstorm/dma_mac-VSS.ppl
analyze_rail -type domain allDomains
```

#---- load DEF and view results -----#

```
read_def -specialnets_components <top.def> <block.def> <sub_block.def>
start_gui
view_analysis_results
```

-help	Prints out the command usage.
-die_instance_name <i>dieinstname</i>	
	Specifies the die instance name for multi-die analysis. This option is only available in multi-die analysis (<code>set_rail_analysis_mode -die_mode multi-die</code>).

fileNameList

Specifies the DEF filenames but not to load in (string, required).

specify_lib

```
specify_lib
[-help]
[<in_file <file1[file2] ...>>]
[-reset]
```

Specify the name of the timing library file(s) for dynamic power analysis. This command allows you to run the dynamic power analysis flow without reading a dotlib into design database to reduce background memory consumption. You must have an external TWF file to run this flow as the timing engine will not be invoked from within Voltus. This flow is recommended for large designs where front-end processing of timing libraries is high. Some of the database query commands will not work if `specify_lib` is used.

This command works in conjunction with the `specify_def` and `specify_spef` commands.

Parameters

-help	Outputs a brief description that includes type and default information for each <code>specify_lib</code> parameter.
<code>in_file <file1[file2] ...</code>	
	List of timing library files.
- reset	Resets all specified options back to default values.

Examples

- The following Tcl shows the use model of the `specify_lib` command that works in conjunction with the `specify_def/specify_spef` flow to perform dynamic power analysis:

```
read_lib -lef tech1.lef tech2.lef
specify_lib ../library1.lib ../library2.lib
specify_def adder_routed.def
specify_spef {test.spef }
read_twf test.twf2
read_verilog test2.v
set_top_module test
set_default_switching_activity -comb_clockgate_ratio 1 -period 2.01ns -input_activity
0.2 -icg_ratio 1
set_power_analysis_mode -default_supply 1.1 -method dynamic_vectorless
report_power
```


specify_oa

```
specify_oa
[-help]
-input {{ top_library top_cell top_view} {block1_library block1_cell block1_view} ...
{blockn_library blockn_cell blockn_view}}
```

Specify the names of the top-level and block-level OpenAccess (OA) databases for use in power and rail analysis. When you use the `specify_oa` command, the software passes the specified OA database files to the power and rail analysis programs for internal processing without loading their physical data in memory. As a result, you will not be able to display and query the layout of the design. These programs will only process the necessary data from the OA files. The usage of this command is recommended in dynamic power analysis, and static and dynamic rail analysis for large (>5M components) designs to obtain better performance during analysis. You will be able to save the memory and runtime that takes to load the design during analysis.

The OA database files must be specified in the top-down order of the design hierarchy, that is, top-level OA, followed by block-level OA database files.

Parameters

- help	Outputs a brief description that includes type and default information for each <code>specify_lib</code> parameter.
<pre>-input {{top_librarytop_celltop_view} {block1_libraryblock1_cellblock1_view} ... {blockn_libraryblockn_cellblockn_view}}</pre>	
	List of OA databases.

Examples

- The following Tcl shows the use model of the `specify_oa` command to perform dynamic power analysis:

```
specify_oa -input {{TopOaLib UL20 layout} {BlockOaLib super_filter1 layout}  
{BlockOaLib super_filter2 layout} {BlockOaLib super_filter3 layout} {BlockOaLib  
super_filter4 layout} {BlockOaLib super_filter5 layout} }  
specify_spef ...  
read_twf ...  
set_default_switching_activity -comb_clockgate_ratio 1 -period 2.01ns -input_activity  
0.2 -icg_ratio 1  
set_power_analysis_mode -default_supply 1.1 -method dynamic_vectorless ...
```

specify_spef

```
specify_spef spef_file_list
```

Specify the name of the SPEF file(s) for dynamic power analysis. When you use the `specify_spef` command, you can run the dynamic power and rail analysis flow without loading design data, thereby, improving the runtime and memory footprint for the whole flow. You must have an external TWF file to run this flow as the timing engine will not be invoked from within Voltus.

<code>-help</code>	Outputs a brief description that includes type and default information for each <code>specify_spef</code> parameter.
<code>spef_file_list</code>	List of SPEF files.

For more information, refer to *Running Analysis without Design Loading* in the "[Importing Design](#)" chapter of the *Voltus User Guide*.

Examples

- The following command specifies the SPEF file `adder_routed.spef` for dynamic power analysis:

```
specify_spef ./adder_routed.spef
```

write_lef_abstract

```
write_lef_abstract
[-help]
fileName
[-add_obs_layers layer_list]
[-cutObsMinSpacing]
[-cutObsToExposeRouting]
[-design_boundary point_list]
[-excludeObsLayers layer_list]
[-excludePinLayers layer_list]
[-extractBlockObs]
[-extractBlockPGPinLayers layer_number_list]
[-extractWellLayerObs well_layer_list[-extractWellObsByRow margin]]
[-ignoreBumpOnPin {all pg signal}]
[-noCutObs]
[-obsSpacing {val layers [val2 layers ...]} [-obsSpacingPerLayer]]
[-portForEachStripePin]
[-property]
[-specifyTopLayer layer_number [-obs_above_top_layer layer+]]
[-stripePin [-selected]]
[-stripePin [-PGPinLayers layer_number_list]]
[-stripePin [-PGNets net+]]
[ -5.6 | -5.7 | -5.8 | -6.0]
```

Generates design abstract (LEF) information for the current routed block-level design. By default, `write_lef_abstract` copies all the current design PG and signal pins to the LEF and creates cut-outs in the obstructions (OBS shapes) around the pins.

In a bottom-up flow, you normally create a simple LEF abstract that contains:

- one obstruction (OBS) shape per routing layer
- signal pins located along the block boundary
- add power pins for the top metal layer PG stripes (see `-stripePin` and `-PgpPinLayers` options)
- embedded bump pins shapes are also written out as LEF pin shapes

If the block used four routing layers out of a possible six layers, you would run the following command:

```
write_lef_abstract -noCutObs -stripePin -PgpPinLayers 4 -specifyTopLayer 4
```

Parameters

<code>-help</code>	Prints a brief description that includes the type and default information for each <code>write_lef_abstract</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <pre>man write_lef_abstract</pre>
-5.6 -5.7 -5.8 -6.0	
	Specifies the LEF version number. This value overrides any value you set with the <code>lefDefOutVersion</code> variable. <i>Default:</i> The software uses the current <code>lefDefOutVersion</code> value. The default value for <code>lefDefOutVersion</code> is 5.8.
<code>-add_obs_layers layer_list</code>	Writes the shapes on on the specified layers as OBS shapes in the LEF. These layers cannot be routing or cut layers.
<code>-cutObsMinSpacing</code>	Creates cut outs in the blockages around pins using minimum width to calculate the spacing. This option cannot be used with <code>-noCutObs</code>. 
<code>-cutObsToExposeRouting</code>	Exposes routing within the specified halo distance to a LEF model.

<code>-design_boundary point_list</code>	Specifies a list of points defining the design boundary polygon for the output LEF (a rectangle would use 4 points). It is used instead of the real design boundary to compute the LEF SIZE, OBS shapes to block routing layers, and OBS shapes on the OVERLAP layer if it is non-rectangular. The LEF will also include any special routes that are outside the design boundary as OBS shapes unless they are converted to PIN shapes based on other options. It does not matter if PIN shapes are inside or outside the design boundary, they are always written out. This is intended for a block that only has bumps and/or top-level RDL routing to the bumps that are outside the "real" block boundary. The real block boundary can be specified with this option to shrink the SIZE and OBS shapes to just include the active area of the block, and let the bumps and RDL routing stick outside. An important limitation is that any instances or regular routing shapes outside the design boundary will be ignored. It is assumed that placement and routing blockages are used to prevent this outside the "real" block boundary.
<code>-excludeObsLayers layer_list</code>	Ignores the specified layer when saving OBS in the macro LEF file. You can use this parameter to set the layer as the following format: ◦ One layer: For example, if you specify – <code>excludeObsLayers {AP}</code>, the AP layer is not saved to OBS. ◦ Multiple layers: For example, if you specify – <code>excludeObsLayers {M7 M8}</code>, M7, M8 layers are not saved to OBS.
<code>-excludePinLayers layer_list</code>	Specifies not to create PIN shapes on the specified layers.

<pre>-extractBlockObs</pre>	Extracts all obstruction (OBS) information from the instantiated <code>CLASS BLOCK</code>, <code>CLASS PAD AREAIO</code>, and <code>CLASS RING</code> macros and writes out the obstructions to the output LEF file. OBS information is extracted for layers that are not already being completely covered under the normal routing layer range and for cut layers between routing layers that are completely covered.
<pre>-extractBlockPGPinLayers layer_number_list</pre>	
	Extracts the power and ground pin information from any <code>CLASS BLOCK</code>, <code>RING</code>, <code>PAD</code>, or <code>COVER</code> macros with pins on layers in the specified layer list and writes out the pin information to the LEF file. The layer names can be either names in the technology file like <code>metal1</code>, or technology independent names like <code>M1</code> or <code>1</code> for the first routing layer, <code>M2</code> or <code>2</code> for the second routing layer. For example, to create PG pins on layers <code>metal3</code> and <code>metal4</code>, use: <pre>-extractBlockPGPinLayers {metal3 metal4}</pre> Or <pre>-extractBlockPGPinLayers {3 4}</pre>
<pre>- extractWellLayerObs well_layer_list</pre>	Extracts OBS from instances on the specified well layers. If you have DRC spacing value checks for the well layers in your LEF, you can copy the well layer OBS shapes out to the LEF abstract so the well shapes can be checked at the next level up.

<code>-extractWellObsByRow margin</code>	Extracts OBS from the bounding box of rows plus the margin on the specified well layers. You can create well layer OBS shapes in the LEF abstract for all the standard cell rows, with additional margin in microns. This is useful if the standard-cell LEF abstracts do not have well shapes in them but you want to see well OBS in the abstract. This option requires the <code>-extractWellLayerObs</code> option to indicate which layers to create.
<code>fileName</code>	Specifies the LEF output file.
<code>-ignoreBumpOnPin {all pg signal}</code>	By default, pin shapes of bump cells (e.g. LEF MACRO CLASS BUMP) are written out as LEF pin shapes. This option can be used to suppress those LEF pin shapes depending on the type of net they are connected to.
<code>-noCutObs</code>	Creates a single obstruction shape for each routing layer. <i>Default:</i> Creates cut-outs (obstructions) in the blockages for signal pins, power and ground pins, vias, and power and ground stripes. This option cannot be used with <code>-cutObsMinSpacing</code>.
<code>-obs_above_top_layer layer+</code>	Writes out OBS shapes rather than one large abstract OBS shape for special-routes on these layers that are not PIN shapes. The layers will have OBSSPACING FULLDRC added to the LEF so the OBS shapes on those layers are treated as 'real' shapes for DRC checking, and not abstract obstructions. This is intended for blocks that have some sparse special-routing above the blocks "normal" top-layer set with the <code>-specifyTopLayer</code> option (for example, it might be a few special-routes for clocks that match up with the chip clock distribution but must be visible inside the block). Only special-routes are currently supported.

<pre>-obsSpacing {val layers [val2 layers ...]}</pre>	Writes out the LEF58_OBSSPACING property for the MACRO. This overrides the global USEMINSPACING OBS setting and sets the spacing required between routing and the OBS shapes of just this MACRO. <i>val</i> = 'min' for min-spacing, and <i>layers</i> = {} for all layers. The order of the arguments is the order in the LEF file. In LEF, the last value for a given layer "wins" over previous values for that macro. For example, <code>-obsSpacing {min {} 1.0 {m2 m3} 2.0 {m5}}</code> would add this to the LEF: <pre>PROPERTY LEF58_OBSSPACING " OBSSPACING MIN ; OBSSPACING 1.0 LAYER m2 LAYER m3 ; OBSSPACING 2.0 LAYER m5 ; "</pre> So, OBS shapes in this MACRO on every layer would use SPACING <min-spacing> for that specific layer, then m2, m3 are changed to use SPACING 1.0, and finally m5 is changed to use SPACING 2.0.
<pre>-obsSpacingPerLayer</pre>	Writes out the spacing for each layer under the layer definition. This parameter does not work without – <code>obsSpacing</code>. Data_type: bool, optional To assign the value to all layers, you should set the layers as "{}". For example, if you specify <code>write_lef_abstract -obsSpacing {1.0 {}} -obsSpacingPerLayer</code>, it means the spacing of all layers is 1.0 <pre>Macro A ... OBS Layer M1 SPACING 1.0 ; RECT ...; Layer M2 SPACING 1.0 ; RECT ...; ...;</pre>

	<pre> END ... END A </pre> The following example sets the spacing of all layers as 1.0, except M2. <pre> write_lef_abstract -obsSpacing {1.0 {} 2.0 {M2}} - obsSpacingPerLayer Macro A ... OBS Layer M1 SPACING 1.0 ; RECT ... ; Layer M2 SPACING 2.0 ; RECT ... ; Layer M3 SPACING 1.0 ; RECT ... ; ... END ... END A </pre>
<code>-PGNets net+</code>	Writes out PG stripes only on these PG nets. If you specify a net name that does not exist in the design, or the net is not a PG net, tool will give an error and stop.
<code>-PGpinLayers layer_number_list</code>	

	Writes out power and ground stripes on the specified layer numbers as power and ground pins. The layer names can be either names in the technology file like metal1, or technology independent names like M1 or 1 for the first routing layer, M2 or 2 for the second routing layer. For example, to create PG pins on layers metal3 and metal4, use: <pre>-PGPinLayers {metal3 metal4}</pre> Or <pre>-PGPinLayers {3 4}</pre> Note: You must specify the <code>-stripePin</code> parameter in order to use this parameter. <i>Default:</i> The top metal layer
<code>-portForEachStripePin</code>	Creates a separate LEF port for each pin shape created by <code>-stripe_pins</code> .
<code>-property</code>	Writes out all the user-defined design properties.
<code>-selected</code>	Writes out only the selected PG stripes as LEF pin shapes.
<code>-specifyTopLayer layer_number</code>	

	Creates obstructions (OBS shapes) covering the block only for layers up to the specified layer. The layer name can be either a name in the technology file like metal1, or a technology independent names like M1 or 1 for the first routing layer, M2 or 2 for the second routing layer, etc. For example, to specify metal4 as the top layer, use: <pre>-specifyToplayer metal4</pre> Or <pre>-specifyTopLayer 4</pre> By default, all routing layers in the technology file are fully obstructed with OBS shapes across the entire block (possibly cut around pin shapes). If this block does not use some of the highest routing layers, you should use this option to limit the OBS shapes to below layer.
<code>-stripePin</code>	Writes out power and ground stripes on the top metal layer as power and ground pins. You can use this parameter in a bottom-up methodology, when you create abstracts for blocks that you will use at the next higher level of a hierarchical design.

Example

Suppose you have a design that was routed on only the first 5 routing layers, even though the technology has higher metal layers.

You would use the following command to include any PG stripes on metal5 as pins in the block LEF to allow connecting to them from above, and to only create OBS for routing layers up to layer 5.

```
write_lef_abstract my.lef -specifyTopLayer 5 -stripePin -PgpinLayers 5
```

If your design also included a block inside that has PG pins on metal5 with no PG stripes in the current design covering them, you can include layer 5 PG pins from the internal blocks using the following command:

```
write_lef_abstract my.lef -specifyTopLayer 5 -stripePin -extractBlockPGPinLayers 5
```

If you also want to include metal4 PG stripes as pins, and metal4 PG pins from the block, you would use the following command:

```
write_lef_abstract my.lef -specifyTopLayer 5 -stripePin -Pgpinlayers {4 5} -  
extractBlockPGPinLayers {4 5}
```

write_rcdb

```
write_rcdb
```

```
[-help]
```

```
<dirname>
```

Saves the parasitic data that you specify in a Voltus session. The software by default does not save the parasitic data even if the design has extracted RC data. Use the [read_rcdb](#) command to restore the parasitic data.

You can restore the parasitic data on systems that are same as those used for saving the data. For example, if the data was saved on Linux in 32bit mode, you can restore it on Linux 32 bit mode only.

 Restoring RCDB that you generated using any version of software prior to 7.1.2 is not supported and might result in an error. Use `write_rcdb` to generate a new RCDB.

Parameters

<code>-help</code>	Prints out the command usage.
<code>dirname</code>	Specifies the name of the parasitic file. Note: The parasitic file must have the <code>.rcdb.d</code> extension.

Examples

- The following example writes the parasitic data to a directory called `myparasitics.rcdb`:

```
write_rcdb myparasitics.rcdb.d
```

write_twf

write_twf

-help

outfile

[-exclude_instances *string*]

[-format <*string*>]

[-include_instances *string*]

[-include_lib_cells *string*]

[-pin]

[-power_compatible]

[-ssta_sigma_multiplier <*float*>]

[-third_party_file_format <*string*>]

[-view <*string*>]

[-voltage_threshold <*string*>]

Generates a Timing Window File (TWF). The TWF file mainly contains the earliest and the latest possible arrival times that a signal may arrive on a net or a pin.

Parameters

-help	Prints out the command usage.
-exclude_instances <i>string</i>	Specifies a list of hierarchical instances to be excluded from the TWF file.
-format < <i>string</i> >	Specifies the name of format.
-include_instances <i>string</i>	Specifies a list of hierarchical instances to be included in the TWF file.
-include_lib_cells <i>string</i>	Specifies the library cells that require the input timing window.

<code>outfile</code>	Writes the TWF file to the specified file name. The <code>outfile</code> parameter must be the last argument in the list.  To write a compressed file, add the <code>.gz</code> extension to the file name.
<code>-pin</code>	Generates a TWF for each pin instead of each net. <i>Default:</i> Generates a TWF for the crosstalk analysis tool, which contains timing windows on each net.
<code>-power_compatible</code>	Generates a timing windows file that is suitable for power analysis.
<code>-ssta_sigma_multiplier value</code>	Specifies a sigma multiplier between 0 and 3. The value that you specify increases the arrival window by that amount and worst slack value is generated. The transition and impedance values are not impacted.
<code>-third_party_file_format</code> <code><string></code>	Specifies the file format for third party format.
<code>-view <string></code>	Specifies name of the analysis view.
<code>-voltage_threshold</code> <code>{lower_thresholdupper_threshold}</code>	Specifies voltage threshold values for generating a Timing Window File (TWF). All the slews in the TWF file will be scaled to these values. The lower threshold value must be less than the higher threshold value. <i>Value range:</i> 0 to 100

Examples

- The `write_twf -voltage_threshold {20 80}` command writes the following in the TWF file:
(VOLTAGE_THRESHOLD 20 80)

Multiple-CPU Processing Commands

- [get_distribute_host](#)
- [get_multi_cpu_usage](#)
- [set_distribute_host](#)
- [set_multi_cpu_usage](#)

get_distribute_host

```
get_distribute_host  
[-help]  
[-args]  
[-custom_script]  
[-env_script]  
[-hosts]  
[-mode]  
[-queue]  
[-reportsLsfInfo <file>]  
[-resource]  
[-shellTimeout]  
[-single_cpu_lsf_args]  
[-timeout]  
[-waitForLsfInfo <integer>]
```

Displays settings for the [set_distribute_host](#) command.

Parameters

<code>-help</code>	Prints out the command usage.
<code>-args</code>	Displays the additional arguments when the software is running in the Sun Grid Engine (SGE) or Load Sharing Facility (LSF) mode.
<code>-custom_script</code>	Displays the name of the custom script when the software is running in <code>custom</code> mode.
<code>-env_script</code>	Displays the user-defined shell environment script that is run for each of the slave hosts.
<code>-hosts</code>	Displays the list of host names when the software is running in <code>rsh/ssh</code> mode.
<code>-help</code>	Outputs the command usage and a brief description about the command parameters.
<code>-mode</code>	Displays the distributed processing mode. The possible values are <code>local</code> , <code>sge</code> , <code>rsh</code> , <code>ssh</code> , <code>lsf</code> , <code>custom</code> , or <code>0</code> . The command displays <code>0</code> when the software is not in multiple-CPU processing mode.
<code>-queue</code>	Displays the queue name when the software is running in <code>sge</code> or <code>lsf</code> mode.
<code>-reportLsfInfo file</code>	
	Specifies to restore the LSF information to the specified file. This parameter can only be used when <code>set_distribute_host -reportLsfInfo</code> is specified in the current session. You must specify the <code>get_distribute_host -reportLsfInfo</code> only after the clients exit to obtain the client CPU time. If you specify this command before the client exits, the command gives a warning message.
<code>-resource</code>	Displays the resource string when the software is running in <code>sge</code> or <code>lsf</code> mode.
<code>-shellTimeout</code>	Displays the number of seconds the host machine waits for <code>rsh/ssh</code> machines to become available for multiple-CPU processing.
<code>-single_cpu_lsf_args</code>	

	Displays the lsf arguments when the software is running in <code>lsf</code> mode for a single CPU application.
<code>-timeout</code>	Displays the default timeout for a submitted job that will never start.
<code>-waitForLsfInfo value</code>	
	Specifies the timeout period required to wait for task exit and generate the LSF information.

Examples

- The following command displays the current setting for the `-mode` parameter:

```
get_distribute_host -mode  
local
```

- The following command saves the LSF information to the `lsf.log` file:

```
get_distribute_host -reportLsfInfo lsf.log
```

get_multi_cpu_usage

```
get_multi_cpu_usage  
[-help]  
[-autoPageFaultMonitor]  
[-cpuPerRemoteHost]  
[-keepLicense]  
[-localCpu]  
[-remoteHost]  
[-threadInfo]  
[-verbose]
```

Displays the requested number of threads or hosts.

To request threads or hosts, use [set_multi_cpu_usage](#).

Parameters

-autoPageFaultMonitor	
	Displays the warning rank set for performance issues caused by major page faults
-cpuPerRemoteHost	
	Displays the number of CPUs available on every remote host for Superthreading.
-help	Outputs the command usage and a brief description about the command parameters.
-keepLicense	Displays the current setting for the <code>set_multi_cpu_usage -keepLicense</code> parameter.
-localCpu	Displays the number of available local CPUs for multi-threading.
-remoteHost	Displays the number of available remote hosts for distributed processing or Superthreading.
-threadInfo	Displays the level of informational messages for each thread.
-verbose	Displays a message with the current multiple-CPU setting.

Examples

- The following command displays the requested number of threads for multi-threading:

```
get_multi_cpu_usage -localCpu
```

- The following command displays the requested number of remote hosts for Superthreading or distributed processing:

```
get_multi_cpu_usage -remoteHost
```

- The following command displays the requested number of CPUs per remote hosts for Superthreading:

```
get_multi_cpu_usage -cpuPerRemoteHost
```

set_distribute_host

```
set_distribute_host  
[-help]  
[-add string]  
[-args string]  
[-custom]  
[-custom_script string]  
[-custom_script_list string]  
[-local]  
[-lsf]  
[-nc]  
[-noWaitTaskComeUp]  
[-queue string]  
[-remove string]  
[-reportLsfInfo]  
[-resource string]  
[-shellTimeout integer]  
[-single_cpu_lsf_args string]  
[-ssh]  
[-timeOut integer]  
[-env_script string]
```

Specifies the multiple-CPU processing configuration for distributed processing. The software finds hosts from the last `set_distribute_host` command specified before a distributed processing command starts.

Use `get_distribute_host` to display the current settings for the `set_distribute_host` command.

 This command is required for distributed processing.

Parameters

```
-add {hostName1 hostName2 ...}
```

	Adds the specified hosts to the <code>rsh/ssh</code> configuration. If you specify more than one host, separate each host name with a space. Each time you specify a host, the software uses it to run one client process. To run more than one client process on a host, specify it more than once. For example, if you have two machines that each has two CPUs and you want to use both CPUs in both machines, type a command like the following: <pre>set_distribute_host -rsh -add {<i>machine1 machine2</i> \ <i>machine1 machine2</i>}</pre>
<code>-args arguments</code>	
	Specifies any additional arguments you need to provide to the Load Sharing Facility (LSF) job submission command.
<code>-custom -custom_script script</code>	
	Specifies a custom script to run distributed processing.
<code>-custom_script_list {{{string1} count1} {{string2} count2} ...{{stringN} countN}}</code>	
	Specifies to customize the launch of multiple jobs, based on resources required for these jobs. This parameter allows you to specify multiple launch strings in order of priority with a maximum allowed job count for each string. If you request for N number of jobs, the command starts submitting jobs starting from <i>string1</i>, until the corresponding count is reached, at which point it starts submitting the next string in the list. Therefore, the command submits the first "<i>count1</i>" jobs with <i>string1</i>, the next "<i>count2</i>" jobs with <i>string2</i> and so on. It stops when the total submitted jobs has reached the requested number (N).
<code>-env_script "script"</code>	

	Specifies a user-defined shell environment script that is run for each of the slave hosts. The parameter allows you to pass these environment settings to the slave hosts by including them in the shell environment script. The license variables used by the software are also automatically passed to the slave hosts to avoid license errors.
<code>-help</code>	Outputs the command usage and a brief description about the command parameters.
<code>-local</code>	Runs all distributed processing jobs on the master machine.
<code>-lsf</code>	Specifies a Load Sharing Facility configuration. If you do not specify any parameters for <code>-lsf</code> , the software uses default settings. Note: To use this parameter, LSF must already be set up (typically by specifying the <code>LSF_ENVDIR</code> and <code>LSF_SERVERDIR</code> environment variables) and <code>bsub</code> must be in your search path. Contact your LSF administrator for details.
<code>-nc</code>	Specifies to use network computer for distributed processing.
<code>-noWaitTaskComeUp</code>	Specifies to not wait for tasks that are delayed or are not likely to occur.
<code>-queue queue_name</code>	
	Specifies the queue for the LSF configuration.
<code>-remove {hostName1hostName2 ... all}</code>	
	Removes the specified hosts or all hosts from the <code>rsh/ssh</code> configuration. If you specify more than one host, separate each host name with a space.

<code>-reportLsfInfo</code>	Specifies to report LSF information in a single report. This parameter allows you to collect the system statistics on the status of the distributed jobs. It has the following information: • Host name for master and clients • Number of CPUs on each machine that are being used by a process • Job ID/PID for each client process • Total memory used on each machine • Total virtual memory used on each machine • Total CPU time used on each machine You must specify this parameter with the <code>-lsf</code> parameter. To get the LSF information, specify the <code>get_distribute_host -reportLsfInfo</code> command parameter. You must specify the <code>get_distribute_host -reportLsfInfo</code> only after the clients exit to obtain the client CPU time. If you specify this command before the client exits, the command gives a warning message.
<code>-resource resource_string</code>	
	Specifies a resource string for the LSF queue. Note: The correct resource string is specific to your installation. Contact your LSF administrator for the appropriate parameters and values.
<code>-shellTimeout seconds</code>	
	Specifies the number of seconds the host machine waits for <code>rsh/ssh</code> machines to become available for multiple-CPU processing. Default: 5
<code>-single_cpu_lsf_args lsf_arguments</code>	

	Specifies the <code>bsub</code> options. For information, see the LSF documentation. Note: You must use this parameter to adjust the LSF argument for a single CPU application. Alternatively, use the <code>-args</code> parameter to adjust the LSF argument for an application that runs in the Superthreading mode.
<code>-timeout</code>	Specifies the default timeout for a submitted job that will never start. The timeout value should include the waiting time for both the host and the license. This functionality prevents the master machine from hanging when the clients do not come up for some reason. After timeout, a warning message appears informing you that the distributed CPUs did not come online within the expected time. The master will continue waiting for the clients and it is up to you to continue waiting or kill the run if there are problems with the Distributed Resource Management (DRM) system. A sample warning message is given below: Task id 2 did not come up within the specified timeout period (300 seconds). Use <code>set_distribute_host -timeout</code> to change the value. This may be a real error due to incorrect or incomplete platform/machine specification provided via <code>set_distribute_host</code> to your distributed resource management system (LSF/Sun Grid Engine etc.). If running locally (<code>set_distribute_host -local</code>), the current machine may be overloaded with too many jobs and swapping (check free memory and CPUs with <code>top</code>). For local runs, consider reducing the number of CPUs you are using via the <code>set_multi_cpu_usage</code> command or free up some memory by killing other memory intensive processes. <i>Default:</i> 30 (local/rsh/custom) and 3600 (lsf)
<code>-ssh</code>	Specifies a secure shell configuration.

Examples

rsh

The following command creates an `rsh` configuration and adds hosts `host10`, `host11`, and `host12`:

```
set_distribute_host -rsh -add {host10 host11 host12}
```

The following commands create an `rsh` configuration with `host1` and `host2` as remote hosts and then adds `host3` and `host4` to the configuration:

```
set_distribute_host -rsh -add {host1 host2}
```

```
set_distribute_host -rsh -add {host3 host4}
```

LSF

The following command uses an existing LSF environment, and relies on the general LSF set up by the system administrator. In this example, the name of the queue implies that all the machines have 4 Gb of memory.

```
set_distribute_host -lsf -queue mem4G
```

The following command schedules a LSF job named `bigQueue` to run on a Linux machine for a maximum of 30 minutes. No other jobs can run on the machine while `bigQueue` is running.

```
set_distribute_host -lsf -queue bigQueue -args {-x -W 30} \  
-resource {OSNAME==Linux}
```

The following command schedules a LSF job named `smallQueue` to run on Linux machines with minimum speed of 2000 MHz.

```
set_distribute_host -lsf -queue smallQueue -args {-W 20} \  
-resource {OSNAME==Linux && SPEED>2000}
```

Note: Check with your LSF administrator for the correct values for the resource string, as the parameters and their values differ in each installation. For example, the parameter name for the speed of the machine might be `MHZ` instead of `SPEED` (as in the preceding example), and the units might be specified in gigahertz instead of megahertz.

The following command specifies to report LSF information:

```
set_distribute_host -lsf -args "-q lnx64" -reportLsfInfo
```

You can use the following command to save the LSF information in the specified file:

```
get_distribute_host -reportLsfInfo lsf.log
```

Local

The following command runs all distributed processing jobs on the machine on which you are running the master version of the Voltus software:

```
set_distribute_host -local
```

Custom

The following command runs distributed processing with a custom script:

```
set_distribute_host -custom -custom_script {bsub -q bigQueue -x -R "OS==Linux &&  
memory>20000"}
```

The following command runs distributed processing with a custom script list:

```
set_distribute_host -custom -custom_script_list {{{bsub -P ICD -W 2 -q admin -R  
"OSNAME==Linux && OSREL==EE50"}1} {{rsh hpc1 -e} 1}}
```

set_multi_cpu_usage

```
set_multi_cpu_usage
[-acquireLicense integer]
[-autoPageFaultMonitor {0 | 1 | 2 | 3}]
[-cpuPerRemoteHost integer]
[-keepLicense {true | false}]
[-licenseList {string}]
[-localCpu {integer | max}]
[-releaseLicense]
[-remoteHost integer]
[-reset]
[-threadInfo {0 | 1 | 2}]
[-verbose]
[-help]
```

Specifies the number of threads to use for multi-threading, or the maximum number of computers to use for distributed processing, or the maximum number of computers and the number of threads to use for Superthreading. Optionally, reports usage information.

 This command is required for multi-threading, distributed processing, and Superthreading.

Parameters

-acquireLicense integer	
	Acquires licenses to enable the specified number of CPUs. For example, if you specify -acquireLicense 5, the software checks out enough licenses to enable 5 CPUs. After the licenses are checked out, they can be used for multi-threading, distributed processing, or superthreading.
-autoPageFaultMonitor {0 1 2 3}	

	Specifies to set a warning rank for performance issues caused by major page faults. The parameter supports the following values: • 0 - specifies to disable the reporting of major page faults. • 1 - specifies to set the warning rank as 1. This means $\frac{PF}{total\ PF} > 0.8$, $b > 10$, and $wa > 85\%$ • 2 - specifies to set the warning rank as 2. This means $\frac{PF}{total\ PF} > 0.5$, $b > 5$, and $wa > 65\%$ • 3 - specifies to set the warning rank as 3. This means $\frac{PF}{total\ PF} > 0.2$, $b > 1$, and $wa > 40\%$ where PF is the number of major page faults from the software, $total\ PF$ is the total number of major page faults in the machine, b is the number of blocked processors, and wa is CPU waiting time.
<code>-cpuPerRemoteHost integer</code>	
	Specifies the number of CPUs on each of the remote machines. This parameter is required for Superthreading. Note: For Superthreading, you must use this parameter in conjunction with the <code>-remoteHost</code> parameter. <i>Default:</i> 1
<code>-help</code>	Outputs the command usage and a brief description about the command parameters.
<code>-keepLicense {true false}</code>	
	Specifies whether to keep the acquired multiple CPU-licenses until the current session ends. Specify this parameter before running any commands that require multiple-CPU applications. To release all multiple-CPU licenses immediately, use the <code>releaseLicense</code> option. <i>Default:</i> <code>true</code>

<code>-licenseList {tslxl tslal edsl edsxl}</code>		
	Specifies the list or order of licenses the software should use for checking out licenses during multiple-cpu processing. The parameter does not support an empty license list {}. You must specify at least once license from the following list: tslxl, tslal, edsl, and edsxl .	
<code>-localCpu</code> <code>{integer max}</code>		
	Specifies the number of CPUs on the local machine . This parameter is required for multi-threading. <i>Default:</i> 1 When <code>-localcpu 1</code> command is specified, multi-threaded timing propagation is enabled.	
<code>-</code> <code>releaseLicense</code>		
	Releases all multiple-CPU license(s) immediately. By default, the software holds multiple-CPU licenses until the end of the current session. To specify that the software release multiple-CPU licenses after every multiple-CPU command runs, use <code>-keepLicense</code>.	
<code>-remoteHost</code> <code>integer</code>	Specifies the number of remote machines. This parameter is required for distributed processing and Superthreading. <i>Default:</i> 0	
<code>-reset</code>	Resets the specified parameters back to default values.	
<code>-threadInfo {0 1 2}</code>		
	Reports usage information. <i>Default:</i> 0 Specify one of the following values:	
	0	Does not write messages to the log file.

	1	Writes the final message to the log file. For example, All threaded jobs finished (10 elapsed sec: 0 processor sec (parent), 0 system sec (parent), 0 processor sec (threads), 0 system sec (threads)).
	2	Writes additional starting/ending information for each thread. For example, Starting threaded job 1... Starting threaded job 2... Starting threaded job 3... Ending threaded job 2 (1 elapsed sec, 0 processor sec, 0 system sec, 1.160M). Ending threaded job 3 (2 elapsed sec, 0 processor sec, 0 system sec, 0.895M). Ending threaded job 1 (10 elapsed sec, 0 processor sec, 0 system sec, 1.172M). All threaded jobs finished (10 elapsed sec: 0 processor sec (parent), 0 system sec (parent), 0 processor sec (threads), 0 system sec (threads)).
-verbose		Displays messages when changing the multiple-CPU settings.

Verify Commands

- [verify_connectivity](#)
- [verify_PG_short](#)
- [verify_power_via](#)

verify_connectivity

```
verify_connectivity
[-help]
[-allPGPinPort]
[-append]
[-conn_loop | -geom_loop | -geom_connect]
[-divide_power_net]
[-error integer]
[-geom_connect]
[-geom_loop]
[-marker_on_highest_layer]
[-net net_names | -selected]
[-no_antenna]
[-no_floating_metal]
[-no_open]
[-no_soft_PG_connect]
[-no_unconn_pin]
[-no_unrouted_net]
[-no_weak_connect]
[-preferred_top_layer integer]
[-raw_viol_mark]
[-remove_old_open_vio]
[-report filename]
[-selected]
[-type {all | special | regular}]
[-use_new_open_vio]
[-use_virtual_connection]
[-warning value]
```

Detects conditions such as opens, unconnected wires (geometric antennas), unconnected pins, loops, partial routing, and unrouted nets; generates violation markers in the design window; reports violations. Running this command does not have database impact unless you save the design, which also saves the violation markers.

The Verify Connectivity feature uses [set_multi_cpu_usage](#) and other multi-CPU commands for multi-threading. For more information, see the [Multiple-CPU Processing Commands](#) chapter in the *Voltus Text Command Reference* document.

Parameters

-append	Displays incremental results in the Violation Browser. By default, violation markers are over-written with new results during each <code>verify _connectivity</code> run. When you specify this parameter, violation markers are appended to the violations of the previous <code>verify_connectivity</code> run.
-allPGPinPort	
	Verifies all PG ports connection.
-conn_loop	
	Checks for connectivity loops in regular wires. Detects connectivity loops based on the end points of the center line of a regular wire segment or the center of a via.
-divide_power_net	
	Divides power nets into four subareas for connectivity verification. Use this parameter to decrease memory usage in 32-bit machines with limited memory. This parameter might increase or decrease the run time, depending on the design.
-error integer	
	Specifies the maximum number of errors to report. The command stops when the maximum value is reached. <i>Default: 1000</i>
-geom_connect	
	Checks for connectivity violations of regular wires. Uses a geometrical model instead of the center-line model. In other words, if the wires overlap at any point, they are considered to be connected--they do not have to connect at the center line. Use this parameter if you manually change the routing or use a third-party router that does not route using the centerline connection routing technique.
-geom_loop	
	Checks for loop violations of regular nets using a geometrical model. The nets do not have to overlap on the center line. When you specify this parameter, the Innovus software does not perform any other connectivity checks. Use this parameter if you use a third-party router that does not route using the centerline connection routing technique. In this case, the <code>-conn_loop</code> parameter might not detect connectivity loop violations.
-marker_on_highest_layer	

	Reports the opening on the highest layer of the pin, which has more than one shape.
<code>-net net_names</code>	
	Specifies that connectivity should be verified for specified nets only. If you specify more than one net, separate the net names with space and surround the list of net names with braces or quotes. You can use wildcards (* and ?) when you specify net names. Note: The <code>-net</code> and <code>-selected</code> parameters are mutually exclusive. Default: If you do not specify either the <code>-nets</code> or the <code>-selected</code> parameter, <code>verify_connectivity</code> verifies all nets.
<code>-no_antenna</code>	
	Ignores violations due to unconnected wires (also called geometrical antennas or dangling wires).
<code>-no_floating_metal</code>	
	Ignores floating metals that are not connected to any pin shapes.
<code>-no_open</code>	
	Ignores open violations. Note: This parameter automatically turns on the <code>-no_unconn_pin</code> parameter, because open and unconnected pin verification are performed together.
<code>-no_unconn_pin</code>	
	Ignores violations due to pins that are not connected to any other objects. Note: This parameter is specified automatically when you specify the <code>-no_open</code> parameter.
<code>-no_unrouted_net</code>	
	Ignores nets that are not routed.
<code>-no_weak_connect</code>	
	Disables checking for routing to more than one port of the weakly connected pin ports.
<code>-no_soft_PG_connect</code>	
	Disables checking of soft Power/Ground connects. By default, <code>verify_connectivity</code> checks the connectivity on masterslice layers. If <code>-no_soft_PG_connect</code> option is specified, connectivity on these layers is ignored.
<code>-preferred_top_layer integer</code>	

	Specifies the preferred top metal layer. When this parameter is used, <code>verify_connectivity</code> does not check the specified top layer and above layers.	
<code>-remove_old_open_vio</code>		
	Removes old open violations.	
<code>-raw_viol_mark</code>		
	Displays violation markers for opens as the bounding box of the island. By default, violation markers for opens are displayed as polygons that include all wires, pins, and vias that connect to the island.	
<code>-report filename</code>		
	Specifies the report file for connectivity violation data.	
<code>-selected</code>		
	Specifies that connectivity should be checked for selected nets. Note: The <code>-net</code> and <code>-selected</code> parameters are mutually exclusive. <i>Default:</i> If you do not specify either the <code>-nets</code> or the <code>-selected</code> parameter, <code>verify_connectivity</code> verifies all nets.	
<code>-type {all regular special}</code>		
	Specifies the type of wires to verify. <code>verify_connectivity</code> is only concerned with the type of the wire and via (special or regular) and not the type of the pin. Irrespective of the <code>type</code> specified, all unconnected terminals (special and regular) are always reported unless <code>-noUnConnPin</code> is on. <i>Default:</i> <code>all</code> Note: To check connectivity for whole nets, you do not need to specify a type. Note: A pin-open violation is listed under VC DRC type "Problem(s) (ENCVFC-92): Pieces of the net are not connected together". Such a violation means the isolated open island only includes some pins' shapes that come from at least two different ports or pins. Choose one of the following:	
	<code>all</code>	Checks all wires, including those that have been previously verified.
	<code>regular</code>	Reports all unconnected pins and open islands that do not include any special wire and via. Does not check a net that has no regular objects (wire, via, pin).
	<code>special</code>	Reports all unconnected pins and open islands that do not include any regular wire and via. Does not report isolated open islands that have any regular wire and via.
<code>-use_new_open_vio</code>		

	Uses an open violation marker to show unconnected objects.
<code>-use_virtual_connection</code>	
	Implies a virtual connection for all bumps and external I/O pins of the same net. Set this parameter to override default behavior of <code>verify_connectivity</code> , in which bumps and external I/O pins of the same net are not considered to be interconnected. It is useful in flip chip designs, where the power bumps are connected outside the chip.
<code>-warning</code> <i>value</i>	
	Specifies the maximum number of warnings to report. <i>Default:</i> 50 <i>Range:</i> 0 to 1000000 <i>Type:</i> Integer

Examples

- The following command verifies connectivity for all nets and generates a maximum of 50 error and 50 warning messages:
`verify_connectivity -type all -error 50 -warning 50`
- The following command reports special unconnected pins and open islands that do not include any regular wire and via
`verify_connectivity -type special -no_antenna -error 1000 -warning 50`

verify_PG_short

```
verify_PG_short
[-help]
[-area {x1 y1 x2 y2}]
[-net net_name]
[-no_cell_blkcg]
[-no_routing_blkcg]
[-report filename]
```

Checks for power and ground shorts between two geometries belonging to different nets. The command performs power and ground short check between the following:

- PG and PG nets
- PG and signal nets
- PG and other special net

This command enables you to check for power and ground shorts in the design before performing any kind of power-grid analysis. You can use this command after restoring the physical design (`read_design -physical_data *.enc` or `read_def filename`).

Parameters

<code>-area {x1 y1 x2 y2}</code>	
	Specifies the coordinates of the area to verify. For example, <code>verify_PG_short -area {0 0 100 100}</code> . The unit of coordinates is micron.
<code>-help</code>	Outputs a brief description that includes type and default information for each <code>verify_PG_short</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <code>man verify_PG_short</code> .
<code>-net net_name</code>	Specifies the name of the power or ground net to be verified.
<code>-no_cell_blkcg</code>	Specifies to ignore cell blockages during during power and ground short checking.
<code>-no_routing_blkcg</code>	Specifies to ignore routing blockages during during power and ground short checking.
<code>-report filename</code>	Specifies the name of the report file that contains the violation information.

Example

- The following command checks for power and ground shorts in the area specified and generates a detailed report named shorts.rpt.

```
verify_PG_short -area {0 0 1600 1600} -report shorts.rpt
```


verify_power_via

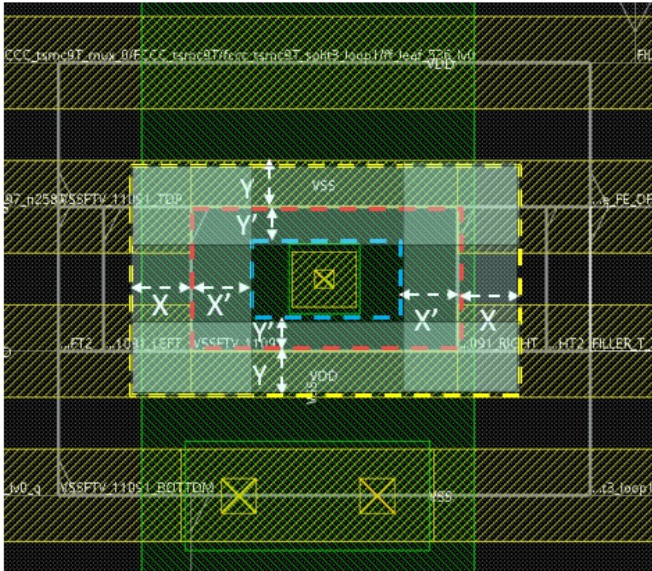
```
verify_power_via
[-help ]
[-append]
[-area {x1 y1 x2 y2}]
[-check_wire_pin_overlap]
[-edge_to_edge]
[-error integer]
[-exclude_region_file check_file]
[-fill]
[-ignore_objects {drc_fill | io_wire}]
[-layer_range {bottom_layer top_layer}]
[-net {net_names}]
[-non_orthogonal_check [-distance {x y}]]
[-report file]
[-shielding]
[-stacked_via]
[-via_util value]
[-what_if_report filename]
[-width_range string]
```

This command has a variety of power-rail overlap checks to look for missing power-grid vias. By default, it checks that orthogonal power-routes on adjacent routing layers have a via between them at every intersection. For example, there should be a via at the point where a metal3 power stripe overlaps a metal2 power stripe. By default, horizontal routes get checked only for overlaps with vertical routes. If the power shapes are polygon, the polygon is split into rectangles and the rectangles are used to decide the direction. Support for checking stacked vias between non-adjacent routing layers is optional. Violations are highlighted in the layout window, and a text report is generated with the location of the missing vias.

Parameters

-help	Outputs a brief description that includes type and default information for each <code>verify_power_via</code> parameter. For a detailed description of the command and all of its parameters, use the man command: <code>man verify_power_via</code> .
-append	Specifies that all previous <code>verify_power_via</code> settings will be kept.
-area x1 y1 x2 y2	Specifies the coordinates of the area to be checked.
-check_wire_pin_overlap	Specifies that wire overlaps of pin shapes should also be checked. By default, only wire overlaps with wires are checked.

<code>-distance {x y}</code>	Specifies distances for finding missing via within a specified window for a non-orthogonal check, with <i>x</i> and <i>y</i> representing the distance in the horizontal and vertical directions, respectively. When the <i>x</i> or <i>y</i> distance is provided, <code>verify_power_via</code> checks if the wire is parallel with another wire. If yes, it checks: • The via center to via center spacing between parallel wires against the value <i>x</i> (<i>y</i>) • The via center to wire end distance against the value <i>x</i>/2 (<i>y</i>/2) If the actual distance is greater than the required distance in either case, it marks a violation. When the <i>x</i> or <i>y</i> distance is not provided or set to 0, the via to via distances are not checked. Note: This parameter can only be specified with the <code>-non_orthogonal_check</code> parameter.
<code>-edge_to_edge</code>	Checks the non-orthogonal distance from edge to edge within a layer.
<code>-error integer</code>	Specifies the maximum number of errors to report.
<code>-exclude_region_file check_file</code>	Ignores missing VIA checks within the specified <code>check_file</code> regions for a cell extension area. The <code>check_file</code> format can be a set of the following values: • <code>cell</code>: Specifies the cell type. • <code>ext_layer_bottom</code>: Specifies the extension area bottom layer. • <code>ext_layer_top</code>: Specifies the extension area top layer. • <code>ext_x</code>: Specifies the extension length in the X direction. • <code>ext_y</code>: Specifies the extension length in the X direction. • <code>exc_x</code>: Specifies the exception length in the X direction. • <code>exc_y</code>: Specifies the exception length in the Y direction. Example 1: The following set of <code>check_file</code> formats ignores missing VIA check within the yellow rectangle (in the output), but checks missing VIA within the blue rectangle: <pre>FTV_CELL, B_M0, B_M1, X, Y, X', Y' INV_D2, M0, M1, X2, Y2, X2', Y2'</pre> Following is the output:



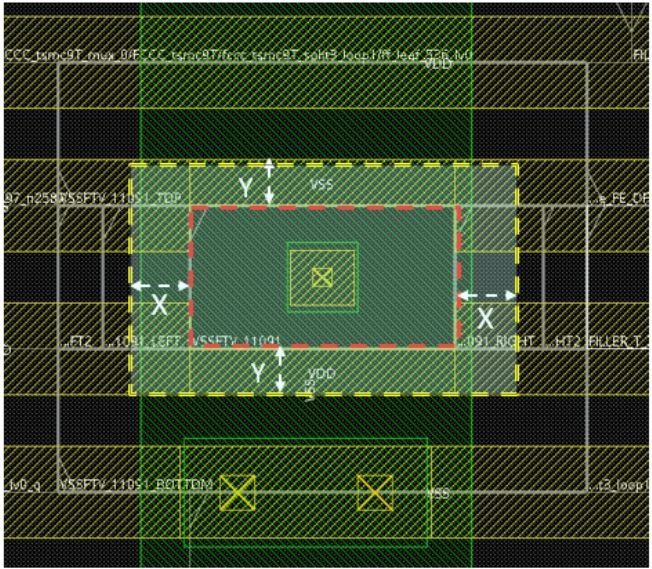
- Extension area
- FTV cell
- Exception area

Example 2:

The following set of `check_file` formats ignores missing VIA check within the yellow rectangle (in the output):

```
FTV_CELL, B_M0, B_M1, X, Y
INV_D2, M0, M1, X2, Y2
```

Following is the output:



- Extension area
- FTV cell
- Exception area

`-fill`

Specifies that tied-off metal fill should also be checked. Floating fill is still ignored.
Default: Metal fill will be ignored.

Voltus IC Power Integrity Solution Text Command Reference
Verify Commands

<code>-ignore_objects {drc_fill io_wire}</code>	Skips checking the power vias of the specified wire type.
<code>-layer_range {bottom_layer top_layer}</code>	Checks for missing stacked vias between the bottom and top layers. It also checks for missing vias between all adjacent intermediate layers, but does not check for stacked vias between the intermediate layers. You can specify full layer names like <code>METAL1</code>, or abbreviations like <code>M1</code> or <code>1</code>. For example, <code>verify_power_via -layer_range {M2 M5}</code> checks for missing stacked vias between overlapping <code>M2</code> and <code>M5</code>, and also missing vias for overlapping <code>M2/M3</code>, <code>M3/M4</code>, <code>M4/M5</code>. It checks the missing vias between all adjacent layer intersections but does not check for missing stacked vias between bottom and top layers. If <code>bottomLayer</code> is specified but <code>topLayer</code> is not, it assumes <code>topLayer</code> is one layer up.
<code>-net {net_names}</code>	Specifies the names of the power nets that will be checked. A single net or a list of nets enclosed in curly braces({}) or quotes (" ") can be specified. Wildcards (* and ?) are supported. For example, when you specify <code>verify_power_via -net {v*}</code>, all PG net names having initial letter as v are checked. Note: Currently, wildcards are supported for checking PG nets only. <i>Default:</i> All power grid nets.
<code>-non_orthogonal_check</code>	Checks the overlap area for non-orthogonal crossing wires (e.g. wires that are both in the same direction) in adjacent as well as non-adjacent layers. Checks for missing vias between two or more parallel wires. The via center-to-center distance between parallel wires should be less than or equal to the specified spacing If this parameter is not specified, horizontal wires get checked only against vertical wires. Note: You can use <code>-distance {x y}</code> parameter with <code>-non_orthogonal_check</code> to find missing vias within a specified window. When the <code>x</code> or <code>y</code> distance is provided, <code>verify_power_via</code> checks if the wire is parallel with another wire. If yes, it checks the via center-center spacing and via center-wire end distance between parallel wires. If the actual distance is greater than the required distance, it marks a violation.
<code>-report file</code>	Specifies the name of the output file for the report.
<code>-shielding</code>	Specifies that shielding wires should also be checked. By default, shielding wires are not checked.
<code>-stacked_via</code>	Checks for missing vias between all non-adjacent as well as adjacent layers.

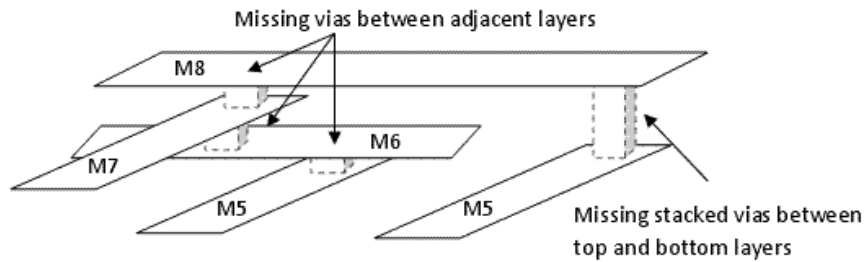
<code>-via_util value</code>	Checks if the cut area in the metal intersection area is sufficient. $\text{viaUtil} = \text{Total cut area} / \text{Metal intersect area}$ If the actual ratio is less than specified ratio, <code>verify_power_via</code> marks it as violation. Note: When <code>-via_util</code> is specified, the <code>verify_power_via</code> command will not check for missing vias.
<code>-what_if_report filename</code>	Writes out <code>verify_power_via</code> violations data into a ECO file that can be used directly as input for whatIf rail analysis. When you specify this parameter, the missing via information generated can be used as input for what-if virtual wires.
<code>-width_range string</code>	Specifies the metal width to be considered. <code>verify_power_via</code> uses the specified width range to filter wires. Only the bottom and top wires that fall in the range are checked. The <code>string</code> value takes the following format: <pre>-width_range {min_width max_width [layer_name ...] [min_width max_width [layer_name ...]] ... }</pre> • If <code>min_width</code> is 0, all wires with width less than specified <code>max_width</code> will be checked. • If <code>max_width</code> is 0, all wires with width greater than specified <code>min_width</code> will be checked. • If <code>layer_name</code> is specified in the string, <code>width_range</code> is enabled only for the specified layer. • If no <code>layer_name</code> is specified in the string, <code>width_range</code> is enabled for all non-specified layers. Note: Use braces or quotation marks to enclose the string value for <code>width_range</code>.

Examples

- Specifies that all vias for all powergrid nets will be checked, except for metal fill and reports it to a file called `power_via.report`:

```
verify_power_via -report power_via.report
```
- The following command checks for missing stacked vias between METAL5 and METAL8 layer intersections on VSS net. In addition, it checks for missing vias between METAL5-METAL6, METAL6-METAL7 and METAL7-METAL8 layer intersections. The violations will be reported in `powerVia.rpt` file and highlighted in GUI.

```
verify_power_via -layer_range {METAL5 METAL8} -net VSS -report powerVia.rpt
```



- The following command checks that the overlapping power routes on adjacent layers have vias that cover at least 70% of the overlap area.

```
verify_power_via -via_util 0.7
```

- The following command checks that the overlapping power routes have vias that cover at least 70% of the overlap area for layers between METAL3 and METAL4.

```
verify_power_via -via_util 0.7 -layer_range {METAL1 METAL4}
```

- The following command checks for wire overlaps for all pin shapes included, enclosed, or touched by the specified area:

```
verify_power_via -area {100 100 1000 1000} -check_wire_pin_overlap
```

GUI Commands

- `add_gui_marker`
- `add_gui_shape`
- `createMarker`
- `createPGPin`
- `deselectAll`
- `deselectInst`
- `deselectInstByCellName`
- `dumpToGIF`
- `list_gui_marker`
- `read_temperature_map_file`
- `remove_gui_marker`
- `reportSelect`
- `selectInst`
- `selectInstByCellName`
- `selectInstOnNet`
- `selectPGPin`
- `set_power_rail_display`
- `set_power_rail_layers_nets`
- `zoomBox`
- `zoomIn`
- `zoomOut`
- `zoomSelected`

- [zoomTo](#)

add_gui_marker

```
add_gui_marker  
[-help]  
-color <string>  
-name <string>  
-pt {x y}  
-type {X TICK STAR}
```

Places a marker to indicate location on the design layout.

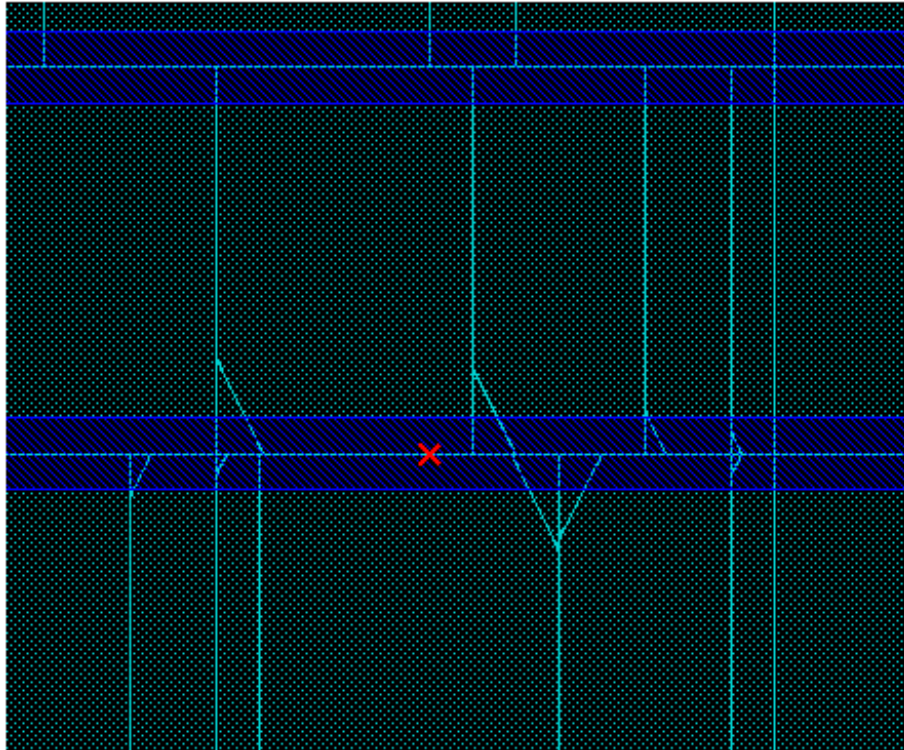
Parameters

-help	Outputs the command usage and a brief description about the command parameters.
-color <string>	Specifies the color of the marker. You can specify 16 colors for the marker. These are: red blue green yellow magenta cyan pink orange brown purple violet teal olive gold maroon wheat
-name <string>	Specifies the name of the marker.
-pt {x y}	Specifies the location {x y} at which the marker will be placed on the design layout.
-type {X TICK STAR}	Specifies the shape of the marker. There are three shapes for the markers: X, TICK, and STAR.

Example

- The following command shows that a X type marker of color RED has been placed at the location {100 100}.

```
add_gui_marker -name aa -color red -pt {100 100} -type X
```



add_gui_shape

```
add_gui_shape
[-help]
-arrow
-layer layerName
[-line {x1 y1 x2 y2 ...} [-arrow ] ]
-name guiShapeName
-polygon {x1 y1 x2 y2 ...}
[-width width]
{-rect {x1 y1 x2 y2} | -line {x1 y1 x2 y2 ...} | -polygon {x1 y1 x2 y2 ...}}
```

The command `add_gui_shape` adds a shape to the specified custom layer. You can use the command at any point in the design flow. The shapes added by the command are to annotate shapes and do not show up in a DEF output.

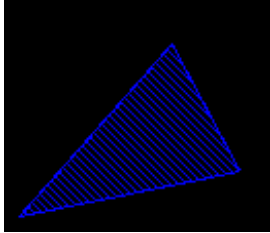
Parameters

<code>-help</code>	Prints a brief description that includes type and default information for each <code>add_gui_shape</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <pre>man add_gui_shape</pre>
<code>-arrow</code>	Draws an arrow in the middle of the line segment to indicate the start to end direction. This parameter can only be used with <code>-line</code>.
<code>-layer</code> <i>layerName</i>	Specifies the GUI layer name to use or create for the rectangle/line. If the specified layer does not exist, it will be created automatically.
<code>-line {x1 y1 x2 y2 ...}</code>	Creates a line, with <i>x1 y1 x2 y2 ...</i> specifying the coordinates for the line segments. The values are specified in microns.
<code>-name</code> <i>guiShapename</i>	The name of the GUI shape.
<code>-polygon {x1 y1 x2 y2 ...}</code>	Creates a polygon shape, with <i>x1 y1 x2 y2 ...</i> specifying the coordinates for the polygon segments. The values are specified in microns.
<code>-rect {x1 y1 x2 y2}</code>	Coordinates of the lower-left and upper-right points of the rectangle. The values are specified in microns.
<code>-width width</code>	Specifies the width of the shape border in pixels. The minimum (default) value is 1 and the maximum is 7.

Examples

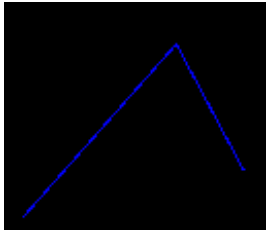
- The following command adds a triangular shape in layer 1:

```
add_gui_shape -layer 1 -polygon {95.841 -645 96.592 -644.147 96.926 -644.777}
```



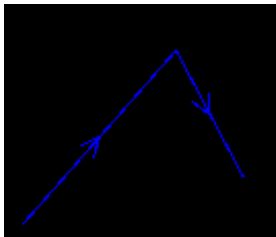
- The following command adds a line shape with two segments:

```
add_gui_shape -layer 1 -line {95.841 -645 96.592 -644.147 96.926 -644.777}
```



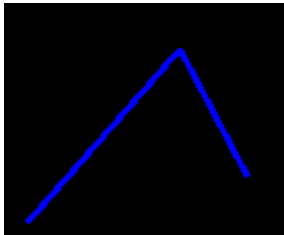
- The following command adds a line shape with two segments and direction arrows:

```
add_gui_shape -layer 1 -line {95.841 -645 96.592 -644.147 96.926 -644.777} -arrow
```



- The following command adds a line shape of 3-pixel width:

```
add_gui_shape -layer 1 -line {95.841 -645 96.592 -644.147 96.926 -644.777} -width  
3
```



createMarker

```
createMarker  
[-help]  
[-bbox { x1 y1 x2 y2 } | -poly { x1 y1 x2 y2 ...}]  
[-desc description]  
[-layer layerName]  
[-rulemap]  
[-subtype subtypeName]  
[-tool toolName]  
[-type typeName]
```

Creates markers for violations in the database and imports DRC markers generated by other software applications. Use this command if you want to import only a specific marker or check a specific marker that has been created by a third party tool.

Parameters

<code>-help</code>	Outputs a brief description that includes type and default information for each <code>createMarker</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <pre>man createMarker</pre>
<code>-bbox { x1 y1 x2 y2 }</code>	Specifies the area of the bounding box of the marker. For example, <code>createMarker -bbox 100 100 100.5 100.5</code>. Type: Real
<code>-desc description</code>	Describes the marker. For example, <code>createMarker -desc "the user mark is non default rule spacing check"</code>. Default: "" Type: String
<code>-layer layerName</code>	Specifies the layer. For example, <code>createMarker -layer METAL2</code>. Type: String
<code>-poly { x1 y1 x2 y2 ... }</code>	Specifies the area of the bounding polygon of the marker.
<code>-rulemap</code>	Specifies whether the marker needs to be created through <code>ruleMap</code>.
<code>-subtype subtypeName</code>	Specifies the marker subtype. For example, <code>createMarker -subtype user_spacing</code>. Default: Other Type: String
<code>-tool toolName</code>	Specifies the source software application. For example, <code>createMarker -tool Calibre</code>. Default: Other Type: String
<code>-type typeName</code>	Specifies the marker type. For example, <code>createMarker -type user_verify</code>. Default: Other Type: String

Example

- The following command generates markers for tool named `Calibre` with type `METAL1` and subtype `S.1`:

```
createMarker -bbox { 1 2 1 2 } -tool Calibre -type METAL1 -subtype S.1
```

The violation browser displays the marker in the above example as follows:

```
+ Calibre
  + METAL1
    + S.1
      ...
```

- Suppose the cadence LPA tool has created the following marker on layer `METAL3`:

```
CDNLitho;SPACING; Severity: 3; MARKER_INFO: "CD = 45.39"
```

```
bbox = (468.248, 63.511) (468.310, 63.599)
```

The following command creates a corresponding marker in the design:

```
createMarker -bbox {468.248 63.511 468.310 63.599} -layer M3 /
               -type CDNLitho -subtype Spacing
```

Related Topics

- "Identifying and Viewing Violations" chapter in the *Innovus User Guide*.

createPGPin

```
createPGPin
[-help]
{-onDie {-selected | -net netName}
[-dir {undefined | input | output}]
[-length float]} |
{pgPinName [-net netName]
[-geom layerName llx lly urx ury]}
[-width float]
```

Creates a power/ground pin as per the specified coordinates of the physical shape.

If the `-geom` parameter is not specified, only the logical pin is created.

If the `-geom` parameter is specified, the physical pin is also created.

If a logical power/ground pin exists before this command is run, the net name will be ignored if it is specified.

If a logical power/ground pin does not exist before this command is run:

- If a net has been specified with the `-net` parameter, the power/ground pin will be created and attached to the net specified.
- If a net name has not been specified, the net name is assumed to be the same as the pin name. In this case, the power/ground pin will be created and attached to that nets.

Parameters

<code>-dir {undefined input output}</code>	
	Specifies the direction of the power/ground pin. You can specify one of the following: • <code>undefined</code>: Specifies that the direction of the pin is not defined. • <code>input</code>: Specifies that the direction of the pin is input. • <code>output</code>: Specifies that the direction of the pin is output.
<code>-geom layerName llx lly urx ury</code>	

	Specifies the geometry of the physical pin. • <i>layerName</i>: Specifies the layer on which the power/ground pin will be created. The <i>layerId</i> can range from 1 to n, where, the maximum value for n is the top most routing layer. Note: Layers can also be specified using layer names (for example, Metal1, Metal2, Metal3) or the LEF layer names. • <i>llx</i>: Specifies the lower-left x coordinate, in microns, of the power/ground pin. • <i>lly</i>: Specifies the lower-left y coordinate, in microns, of the power/ground pin. • <i>urx</i>: Specifies the upper-right x coordinate, in microns, of the power/ground pin. • <i>ury</i>: Specifies the upper-right y coordinate, in microns, of the power/ground pin.
-help	Outputs a brief description that includes the type and default information for each <i>createPGPin</i> parameter. For a detailed description of the command and all of its parameters, use the man command: man createPGPin
-length <i>float</i>	Specifies the length of the power/ground pin. Default: Same as the width
-net <i>netName</i>	Specifies the name of the net to which the power/ground pin will be attached.
-onDie	Creates power/ground pin on the die.
<i>pgPinName</i>	Specifies the name of the power/ground pin.
-selected	Creates power/ground pin on the die, for all selected stripes.
-width <i>float</i>	Specifies the width of the power/ground pin. Default: width of the stripe.

Examples

- This example is for the case where a logical pin name has not been defined prior to running the command and a net name has been specified.

The following command creates a power/ground pin named pgpin_A on layer Metal2 and attaches the pin to the net net_A. The pin coordinates are 7 (llx), 3 (lly), 17 (urx), and 23 (ury).

```
createPGPin -geom Metal2 7 3 17 23 -net net_A pgpin_A
```

- This example is for the case where a logical pin name has not been defined prior to running the command and a net name has not been specified.

The following command creates a power/ground pin named pgpin_B on layer Metal3 and attaches the pin to the net pgpin_B. The pin coordinates are 7 (llx), 3 (lly), 17 (urx), and 23 (ury). As the net name was not specified, the net name is assumed to be the same as the pin name.

```
createPGPin -geom Metal3 7 3 17 23 pgpin_B
```

- This example is for the case where a logical pin name has already been defined prior to running the command.

The following command creates a power/ground pin named pgpin_C on layer Metal5. The pin coordinates are 7 (llx), 3 (lly), 17 (urx), and 23 (ury). In this case, the logical pin name was already defined prior to running the command, and so a net name is not required. Even if a net name were specified, it would have been ignored.

```
createPGPin -geom Metal5 7 3 17 23 pgpin_C
```

- This example is for the case where the net name is not specified and the -geom parameter has not been specified.

The following command creates a logical pin names pgpin_D. The net name is assumed to be the same as the pin name and no physical pin is created.

```
createPGPin pgpin_D
```

- This example is for the case where the pin is created on the die attached to net vdd with a width of 2 and a length of 3.

```
createPGPin -onDie -net vdd -width 2 -length 3
```

Related Topics

- "Power Menu" chapter of the *Innovus Menu Reference*

deselectAll

```
deselectAll  
[-help]
```

Deselects all selected nets. Once the nets are deselected, they are no longer highlighted in the design display window.

Use this command after selecting one or more nets.

Parameters

-help	Outputs a brief description of the <code>deselectAll</code> command. For a detailed description of the command, use the man command: <code>man deselectAll</code>
-------	---

deselectInst

```
deselectInst
[-help]
instName
[-query_power_rail_attr]
```

Deselects the specified instance. Use this command after selecting an instance.

Parameters

-help	Outputs a brief description that includes the type and default information for each <code>deselectInst</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <code>man deselectInst</code>
instName	Specifies the name of the instance.
-query_power_rail_attr	
	Specifies to query the power and rail attributes for an instance. You can click the selected instance or press the Q key to display the attributes in the Attribute Viewer window. When the parameter is not specified, the tool would not query the power and rail attributes, resulting in this information not being available in the Attribute Viewer window (displayed as NA).

deselectInstByCellName

```
deselectInstByCellName  
[-help]  
cellName
```

Deselects an instance by cell name. Use this command after selecting an instance by cell name.

Parameters

<i>cellName</i>	Specifies the name of the cell.
-help	Outputs a brief description that includes the type and default information for each <code>deselectInstByCellName</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <pre>man deselectInstByCellName</pre>

dumpToGIF

`dumpToGIF [-help] filename`

Saves a snapshot of the current screen to a GIF file with the specified name in the current directory. You can use this command at any stage of the design flow.

Parameters

<code>-help</code>	Prints a brief description that includes type and default information for each <code>dumpToGIF</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <code>man dumpToGIF</code>.
<code><i>filename</i></code>	Specifies the name of the output file as a string.

Example

The following example captures the current screen and saves it to `test1.gif` in the current directory.

```
dumpToGif test1.gif
```

Related Topics

- "Write To GIF File" in the [Tools Menu](#) chapter of the *Voltus Menu Reference*.

list_gui_marker

```
list_gui_marker  
[-help]
```

Gives a list of GUI markers in the design layout.

Parameters

-help	Prints out the command usage.
-------	-------------------------------

Example

The following command shows a list of GUI markers:

```
list_gui_marker
```

The following is displayed:

```
{name:aa  coordinate:(100.000 100.000) }
```


read_temperature_map_file

```
read_temperature_map_file  
[-help]  
-file <string>
```

Loads the tile-based temperature map file generated by Sigrity Celsius to be used for displaying temperature.

Parameters

-help	Outputs the command usage and a brief description about the command parameters.
-file <string>	Specifies the name of the tile-based temperature map file generated by Sigrity Celsius.

remove_gui_marker

```
remove_gui_marker  
[-help]  
{-name <string> | -all }
```

Removes a marker from the design layout.

Parameters

-help	Outputs the command usage and a brief description about the command parameters.
-name <string>	Specifies the name of the marker to be removed from the design layout.
-all	Removes all markers from the design layout.

reportSelect

```
reportSelect  
[-help]  
-file filename
```

Reports the properties of one or more selected objects in the design display area in the main console window and the log file. You can use this command after selecting one or more objects in the design display area.

Parameters

<code>-help</code>	Outputs a brief description that includes the type and default information for each <code>reportSelect</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <code>man reportSelect</code>
<code>-file</code> <i>filename</i>	Specifies the name of the output file that reports the properties of the selected object.

Example

The following command generates a report file `object.rpt`, which displays the properties of the selected object:

```
reportSelect -file object.rpt
```

The following is the output format of the report file:

```
Object Type : IO Cell  
Name : IOPADS_INST/Ptdspip00  
No. of Terminals : 2  
Cell Type : PDIDG2  
Cell Width : 40.0  
Cell Height : 235.0  
Location : 253.78 0.0  
Location Origin : Lower Left  
Orientation : R0  
Status : PLACED  
Routing Halo : None  
InstGroup : None
```


selectInst

```
selectInst
[-help]
instName
[-query_power_rail_attr]
```

Selects an instance and highlights it in the design display window. You cannot specify more than one instance; however, you can use a wildcard (*). You can use this command after importing or restoring the design.

Parameters

-help	Outputs a brief description that includes the type and default information for each <code>selectInst</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <code>man selectInst</code>
instName	Specifies the name of the instance.
-query_power_rail_attr	
	Specifies to query the power and rail attributes for an instance. You can click the selected instance or press the Q key to display the attributes in the Attribute Viewer window. When the parameter is not specified, the tool would not query the power and rail attributes, resulting in this information not being available in the Attribute Viewer window (displayed as NA).

Examples

- The following command queries the power and rail attributes for instance u0:

```
selectInst u0 -query_power_rail_attr
```

selectInstByCellName

```
selectInstByCellName  
[-help]  
cellName
```

Selects an instance by cell name. You can use this command after importing the design.

Parameters

<code>-help</code>	Outputs a brief description that includes the type and default information for each <code>selectInstByCellName</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <code>man selectInstByCellName</code>
<code><i>cellName</i></code>	Specifies the name of the cell.

selectInstOnNet

```
selectInstOnNet  
[-help]  
netName
```

Selects an instance on a net. You can use this command after importing the design.

Parameters

<code>-help</code>	Outputs a brief description that includes the type and default information for each <code>selectInstOnNet</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <code>man selectInstOnNet</code>
<code><i>netName</i></code>	Specifies the name of the net.

selectPGPin

```
selectPGPin
[-help]
[-all]
[-area llx lly urx ury]
[-layer layerId | {layerIdList}]
[-net netName]
```

Selects power/ground pins as per the specified parameters and reports the number of selected power/ground pins.
You can use this command after importing the design.

Parameters

-all	Selects all the power/ground pins.
<i>-area llx lly urx ury</i>	
	Specifies a bounding box for the area in which power/ground pins are to be selected.
-help	Outputs a brief description that includes the type and default information for each <code>selectPGPin</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <code>man selectPGPin.</code>
<i>-layer layerId {layerIdList}</i>	
	Specifies the layer(s) on which the power/ground pins are to be selected. Note: Layers can be specified using the LEF layer names or layer ID numbers.
<i>-net netName</i>	Specifies the name of the net whose attached power/ground pins are to be selected.

set_power_rail_display

```
set_power_rail_display
[-help]
[-customized_range {value1 value2 ... value9}]
[-enable_auto_apply {true | false}]
[-enable_result_browser {true | false}]
[-enable_percentage_range {true | false}]
[-enable_rlrp {true | false}]
[-enable_voltage_sources {true | false}]
[-filter_color {on | off | color_name}]
[-filter_max max_value]
[-filter_min min_value]
[-grid {on | off}]
[-legend {off ne nw se sw}]
[-plot plot_type]
[-range_max max_value]
[-range_min min_value]
[-reset_color_scale {true | false}]
[-thermal_layer_name thermal_layer]
[-tile_count_x value]
[-tile_count_y value]
[-tile_size_x value]
[-tile_size_y value]
```

Controls the display of the main window widgets, such as legend, result browser, voltage source, filter range, and RLRP path.

Parameters

- help	Outputs a brief description that includes type and default information for each report_power_rail_results parameter. For a detailed description of the command and all of its parameters, use the man command: man report_power_rail_results.
-customized_range {value1 value2 ... value9}	

	Specifies the color scale range values for the 8 filters between the minimum and maximum values of the color gradient based (<i>Basic</i>) filter. The value list must contain 9 values. The filter ranges for the 8 buckets are evenly divided by default. This parameter can be used to customize the filter ranges. You can either modify the color range using the GUI (Basic button) or the <code>-customized_range</code> parameter. Example: <pre>set_power_rail_display -plot ir -customized_range { 1 10 12 13 14 15 16 17 49 }</pre>
	<code>-enable_auto_apply {true false}</code>
	Specifies to automatically redraw the layout after modifying the color scale range values. This parameter allows you to enable/disable the <i>Auto Apply for Color Scale</i> checkbox of the Power Rail Plots form using the Tcl script instead of the GUI option. The default value of this parameter is <code>false</code>.
	<code>-enable_percentage_range {true false}</code>
	Specifies to display the color gradients for the following plots by percentage in the GUI: • IR Drop • IVDD • IVDN The percentage value is based on the nominal voltage value of each rail/instance. It displays high percentage voltages in red and low percentage voltages in blue. <i>Default:</i> <code>false</code>
	<code>-enable_result_browser {true false}</code>
	When set to <code>true</code> it enables the result browser. <i>Default:</i> <code>false</code>
	<code>-enable_rlrp {true false}</code>

	When set to <code>true</code> it enables the display of the RLRP path. The rail analysis must include RLRP analysis (<code>set_rail_analysis_mode -enable_rlrp_analysis true</code>) in order to have the required data for this functionality. <i>Default:</i> <code>false</code>
<code>-enable_voltage_sources {true false}</code>	
	Displays voltage sources (power pads) using white pixels in the layout canvas. <i>Default:</i> <code>false</code>
<code>-filter_color {on off color_name}</code>	
	Allows to set the color of the filtered out resistors specified using the scissor filter range. By default, the filtered resistors are not displayed. If you want to view these resistors, you can use this parameter to enable the visibility of resistors and set desired colors for them. The valid values of this parameter are: • <code>on</code> – enables the visibility of the filtered out resistors. • <code>off</code> - disables the visibility of the filtered out resistors. • <code>color_name</code> - enables the visibility of filtered out resistors and set the color of resistors per the specified color name.
<code>-filter_max max_value</code>	
	Specifies the maximum value of the scissor filter range that will be plotted. You can use the scissor filter range to filter out the resistors and nodes that you do not want to view, and obtain statistics only for the instances in the specified filter range. Optional, but if specified, the <code>-plot</code> option is required, and requires both <code>-filter_min</code> and <code>-filter_max</code> to be specified.
<code>-filter_min max_value</code>	
	Specifies the minimum value of the scissor filter range that will be plotted. Optional, but if specified, the <code>-plot</code> option is required, and requires both <code>-filter_min</code> and <code>-filter_max</code> to be specified.
<code>-grid {on off}</code>	

	Allows you to control the grid display when viewing the Voltage Source Current (VC) and Voltage across Package Model (VU) plots. The default value of this parameter is <code>on</code>. You can set this parameter to <code>off</code> to hide the grid view for better display and easier analysis of the plot data.
	<code>-legend {off ne nw se sw}</code>
	Displays the filter color and range details. When you set <code>set_power_rail_display -legend</code> to one of the location (<code>ne nw se sw</code>), this opens the legend at the specified location on the design layout. Use <code>set_power_rail_display -legend off</code> to close the legend. <i>Default:</i> <code>off</code>.
	<code>-plot plot_type</code>
	Plots the specified plot type. See Plot Types for a description of each of the plot types.
	<code>-range_max max_value</code>
	Specifies the maximum value of the color scale. Optional, but if specified, the <code>-plot</code> option is required, and requires both <code>-range_min</code> and <code>-range_max</code> to be specified.
	<code>-range_min min_value</code>
	Specifies the minimum value of the color scale. Optional, but if specified, the <code>-plot</code> option is required, and requires both <code>-range_min</code> and <code>-range_max</code> to be specified.
	<code>-reset_color_scale {true false}</code>
	Specifies to reset the maximum (<code>-range_max</code>) and minimum (<code>-range_min</code>) values of the color scale to the default plot data values. For example, when you specify <code>-plot ir</code>, the default value for <code>-range_min</code> is 0 and <code>-range_max</code> is 100. You can modify the color scale, such as set <code>-range_min</code> to 0 and <code>-range_max</code> to 20. To reset the color scale of the IR drop plot to default, you can specify the <code>-reset_color_scale</code> parameter.
	<code>-thermal_layer_name thermal_layer</code>

	Displays the specified layer of the temperature map. Example: <pre>set_power_rail_display -thermal_layer_name VIA_7</pre>
	<code>-tile_count_x value</code>
	Specifies the number of tiles in the x direction. This parameter is used to resize the tile power density plot. You can either specify the tiles by count (<code>tile_count_x</code> and <code>tile_count_y</code>) or specify the tiles by size (<code>tile_size_x</code> and <code>tile_size_y</code>). When you resize the tile power density (<code>tpd</code>) plot in the batch or GUI mode, the software automatically invokes the <code>report_power</code> command and resizes the plot. This feature allows you to resize the <code>tpd</code> plot without having to restore the power database (<code>power.db</code>) again.
	<code>-tile_count_y value</code>
	Specifies the number of tiles in the y direction. This parameter must be specified with the <code>tile_count_x</code> parameter.
	<code>-tile_size_x value</code>
	Specifies the horizontal dimension of each tile polygon in μm. This parameter is used to resize the tile power density plot. You can either specify the tiles by count (<code>tile_count_x</code> and <code>tile_count_y</code>) or specify the tiles by size (<code>tile_size_x</code> and <code>tile_size_y</code>). When you resize the tile power density (<code>tpd</code>) plot in the batch or GUI mode, the software automatically invokes the <code>report_power</code> command and resizes the plot. This feature allows you to resize the <code>tpd</code> plot without having to restore the power database (<code>power.db</code>) again. The following is an example of specifying tiles by size: <pre>set_power_rail_display -plot tpd -tile_size_x 100 -tile_size_y 100</pre>
	<code>-tile_size_y value</code>
	Specifies the vertical dimension of each tile polygon in μm. This parameter must be specified with the <code>tile_size_x</code> parameter.

Plot Types

Plot Type	Description
<code>ac</code>	Available/feasible capacitance (Farad) Displays the feasible decoupling capacitance that can be added in the area. The feasible capacitance in the area is calculated based on the filler cells placement. The program matches the placed filler cells with the decap cells available in the power-grid library and computes the feasible capacitance. The capacitance of the decap cell is stored in the power-grid library.
<code>cap</code>	Ggrid capacitance (Farad) Plots the capacitance of the extracted power-grid network.
<code>cvil</code>	Customized File 1 Plots the user-specified instance voltage file.
<code>dd</code>	Decoupling capacitance (decap) density (Farad/μm^2) Plots decap density of the design. The decap density is the cell internal decoupling capacitance in the unit area (μm^2).
<code>effr</code>	Effective resistance (ohm) - instance based Display the effective resistance results for all instances.
<code>node_reff</code>	Effective resistance (ohm) - node based Display the effective resistance results for all nodes.
<code>ipc</code>	Instance Peak Current Display the peak current of each instance.
<code>ivdn</code>	Instance Voltage Drop (Volts) – net-based Plots the net-based instance voltage drop.
<code>ivdd</code>	Instance Voltage Drop (Volts) – domain-based Plots the domain-based instance voltage drop. For domain based, at least one power and one ground net must be selected.

fd	Filler cell density ($1/\mu\text{m}^2$) Displays the filler cell density per unit area (μm^2). The plot can be used to analyze the feasibility of adding decap cells by replacing the filler cells and not changing the placement of the logic cells.
freq	Frequency domain (Hz) Displays the associated clock frequency of all instances in the design. The instances associated with multiple clock domains are displayed using the fastest clock frequency. This plot can be used to analyze and debug the power distribution in the design.
ip	Instance total power (mW) The total power consumed by each instance is displayed. The total power is the sum of the internal, switching and leakage power of the instance. The power data is read from the power database and can be specified using the <code>-power_db</code> option or it is queried directly from memory, if power calculation is run during the session.
ip_i	Instance internal power (mW) The power consumed by charging and discharging of the internal interconnect and device capacitances of the instance.
ip_l	Instance leakage power (mW) The power consumed by devices when not switching.
ip_s	Instance switching power (mW) The power consumed by charging and discharging of the interconnect or output load.
ipd_i	Specifies internal power density in $\text{mW}/\mu\text{m}^2$.
ipd_l	Specifies leakage power density in $\text{mW}/\mu\text{m}^2$.
ipd_s	Specifies switching power density in $\text{mW}/\mu\text{m}^2$.
ipd_t	Specifies total power density in $\text{mW}/\mu\text{m}^2$.
ir	IRdrop or voltage drop (Volts) Displays the IRdrop across each extracted power-grid segment in the design.

jrms	Specifies current density as j/j_{max} based on the <i>Irms</i> current. This plot is only supported for dynamic analysis. A <i>jrms</i> gif file will be saved in the state directory that shows where the <i>Irms</i> based EM violation occurred. This plot type allows you to verify the analysis results of "jrms" using the GUI.
load	Loading capacitance (Farad) Displays the output loading capacitance driven by the instance. This plot can be used to analyze and debug the regions with high switching power.
pi	Power-switch (gate) current violation: I/I_{dsat} Displays the ratio of current through the power-switch and the saturation current (<i>I_{dsat}</i>) of the power-switch. This plot can be used to analyze and debug whether the current through power-switches exceeds the saturation current characterized for the power-switch devices. The saturation current and on-resistance of the power-switches are characterized and stored inside the power-grid library of the power-switch cell. A ratio of greater than 1 means that the current requirement of the power-gated block can not be met with placed power-switch instance. Either the power-switch needs to be made larger or power-switch instances need to be added.
ptpd	Profiling Tile Power Density Displays the tile-based power density plot for the vector profiling flow. You can enable this plot using the <code>read_power_rail_results - profiling_power_file file</code> command. You can also view the power density for the different tile co-ordinates in the <i>Details</i> tab of the Result Browser.
pV	Voltage drop across power-switch (Volt) Displays the IRdrop across power-switch instances. This plot can be used to analyze and debug the IRdrop inside the power-gated block, e.g. if the IRdrop across power-switches is already high, the IRdrop inside the power-gated block will be much higher. In this case, the power-switch placement will have to be refined in order to resolve the IRdrop problem.

rc	Resistor current (Amp) Displays the current across power-grid resistor segments. This plot can be used to analyze and debug how the current flows from the voltage sources into the rest of the design, e.g. high current should flow from pads to the top-level routing level layers and then to the lower level metal layers. This plot can also be used to debug current density violations (r_j).
rlrp	The total resistance of the instance along the least resistive path. Displays the total resistance of the instance along the least resistive path, meaning the resistance of the instance along the path from where it gets the most amount of current. This plot can be used to analyze and debug the power-grid integrity early in the design stage. An instance with high resistance is likely to have high static or dynamic IRdrop depending on the vector that is simulated later in the design cycle. This plot can be also used during decap optimization, since regions with high dynamic IRdrop and high resistance may not be improved upon by adding decaps, because the effectiveness of decoupling capacitance depends on the resistance of the power-grid network. The plot displays the RLRP paths for power (purple color) and ground (white color) nets in different colors for the identification of paths and ease of analysis. You can also highlight the least resistive path of the instance using the GUI and selecting the instance with left mouse button. The <i>RLRP Path</i> checkbox should be enabled.
res	Grid resistance (ohm) Displays the total resistance of the least resistive path from each node to voltage source. This plot can be used to view high resistance regions of the power-grid. It is useful during IRdrop and decap analysis to identify weak power-grid connectivity. The GUI also allows you to view the resistor-based RLRP path. To view the resistor-based RLRP path: 1. Select the <i>res</i> Rail plot, and the <i>RLRP Path</i> checkbox. 2. Select a resistor. The GUI highlights the least effective resistance path to the nearest voltage source, and opens the <i>Resistance Path</i> form.

rij	Resistor current density, J/Jmax. Displays the current density (current per unit area) violation on each power-grid segment using the ratio of calculated current density and maximum allowed current density. A ratio of greater than 1 means that the current density limit of the segment is violated. The current density limits are supplied by the foundry and can be read during analysis using the <code>set_rail_analysis_mode -em_models</code> command.
rs	Specifies the plot type for resistor sensitivity.
sem_avg	Average Current Density Displays average current limit violations in the design. This plot is enabled when you specify <code>verify_AC_limit -method avg</code>.
sem_peak	Peak Current Density Displays peak current limit violations in the design. This plot is enabled when you specify <code>verify_AC_limit -method peak</code>.
sem_rms	RMS Current Density Displays RMS current limit violations in the design. This plot is enabled when you specify <code>verify_AC_limit -method rms</code>.
slack	Instance slack (Seconds) Displays worst instance slack. The slack data is queried from either the timing database or if the power database is specified using <code>-power_db</code> option, then the worst slack data is read from the power database. When using an external TWF, the slack data is read from the TWF.
tc	Tap current (Amp) Displays the current consumed by the instance devices or current taps. During rail analysis, it is based on the power consumption of the instance and is distributed inside the cell. The current distribution inside the cell is based on the extracted device netlist stored inside the power-grid library of the cell. The current distribution is generally based on the device width, length, and spice models. This plot can be used to analyze and debug IRdrop inside the macro.

td	Transition density (Hz) Displays the worst transition density (activity * frequency) of the instance. This plot can be used to analyze and debug activity distribution and high power density regions.
thermal	Instance thermal Displays the instance temperature distribution for the chip. The plot shows tile-based temperature for instances. When you click on an instance, the <i>Attribute Viewer</i> window opens and displays the instance temperature in the <i>Thermal</i> field.
unc	Unconnected segments Displays power-grid segments that are disconnected from the voltage source (power pads). This plot can be used to debug unusually high IRdrop in the region.
vc	Voltage source current (Amp) Displays current through the voltage sources (power pads). This plot can be used to analyze the current carrying capacity of the power-pad.
vu	Voltage drop across package (Volt) Displays voltage drop across package. If analysis is run with package model attached to the power pads, this plot displays the IRdrop due to package RLCK elements.

Examples

- The following command specifies that `div` data will be plotted without loading design DEF:

```
set_power_rail_display\  
-plot div
```
- The following command enables the visibility of the filtered out resistors without changing the color:

```
set_power_rail_display -filter_color on
```
- The following command enables the visibility of the filtered out resistors and also changes the color to red:

```
set_power_rail_display -filter_color red
```

Related Topics

- "[Static Power, IRdrop and EM Analysis](#)" in the *Voltus User Guide*

set_power_rail_layers_nets

```
set_power_rail_layers_nets
[-help]
[-design_opacity value]
[-enable_switch_nets {all|net_names} |
-disable_switch_nets {all|net_names} ]
[-enable_report_layers {all|layer_names} |
-disable_report_layers {all|layer_names} |
-layers {{name report0/1 visible0/1}}{...}} ]
[-enable_visible_layers {all|layer_names} |
-disable_visible_layers {all|layer_names} |
-layers {{name report0/1 visible0/1}}{...}} ]
[-enable_visible_sem_layers {all|layer_names} |
-disable_visible_sem_layers {all|layer_names} ]
[-instance_voltage_method {Worst | Best | Avg | WorstAvg}]
[-instance_voltage_window {1| 0}]
[-nets {{all_power 0/1}}{all_ground 0/1}} |
-enable_nets {names} |
-disable_nets {names} ]
[-rail_analysis_opacity value]
[-show_layer_net_setting_list {true | false}]
[-via_size {1 | 2 | 3 | 4 | 5 | 6}]
```

Controls the visibility of the layers, the opacity of the rail analysis or design database, and the visibility of power/ground nets.

Parameters

- help	Outputs a brief description that includes type and default information for each set_power_rail_layers_nets parameter. For a detailed description of the command and all of its parameters, use the man command: man set_power_rail_layers_nets.
-design_opacity value	

	Specifies the design opacity value for the design database. The opacity range is from 0-100%, the default value is 60.
<code>-disable_nets {names}</code>	
	Hides the specified nets. This parameter is mutually exclusive to the <code>-enable_nets</code> and <code>-nets</code> parameters.
<code>-disable_report_layers {all layer_names}</code>	
	Hides the reporting (user-defined layers to be used for processing) of layers. You can use <code>all</code> to hide all layers, or list the specific layers that are not to be displayed. This parameter is mutually exclusive to the <code>-enable_report_layers</code> and <code>-layers</code> parameters.
<code>-disable_switch_nets {all net_names}</code>	
	Hides the specified switch nets. You can use <code>all</code> to hide all the switch nets, or list the specific switch nets that are not to be displayed. This parameter is mutually exclusive to the <code>-enable_switch_nets</code> parameter.
<code>-disable_visible_layers {all layer_names}</code>	
	Specifies which layers should not be made visible when plotting. You can use <code>all</code> to hide all layers, or list the specific layers that are not to be displayed. This parameter is mutually exclusive to the <code>-enable_visible_layers</code> and <code>-layers</code> parameters.
<code>-disable_visible_sem_layers {all layer_names}</code>	
	Specifies which signal EM layers should not be made visible when plotting. You can use <code>all</code> to hide all layers, or list the specific layers that are not to be displayed. This parameter is mutually exclusive to the <code>-enable_visible_sem_layers</code> parameter.
<code>-enable_nets {names}</code>	

	Displays only the specified nets. This parameter is mutually exclusive to the <code>-disable_nets</code> and <code>-nets</code> parameters.
<code>-enable_report_layers {all layer_names}</code>	
	Specifies to select the user-defined layers to be used for processing. You can use <code>all</code> to select all layers, or list the specific layers that are to be selected. This parameter is mutually exclusive to the <code>-disable_report_layers</code> and <code>-layers</code> parameters.
<code>-enable_switch_nets {all net_names}</code>	
	Displays the specified switch nets. You can use <code>all</code> to display all the switch nets, or list the specific switch nets that are to be displayed. This parameter is mutually exclusive to the <code>-disable_switch_nets</code> parameter.
<code>-enable_visible_layers {all layer_names}</code>	
	Specifies which layers should be made visible when plotting. You can use <code>all</code> to display all layers, or list the specific layers that are to be made visible. This parameter is mutually exclusive to the <code>-disable_visible_layers</code> and <code>-layers</code> parameters.
<code>-enable_visible_sem_layers {all layer_names}</code>	
	Specifies which signal EM layers should be made visible when plotting. You can use <code>all</code> to select all layers, or list the specific layers that are to be selected. This parameter is mutually exclusive to the <code>-disable_visible_sem_layers</code> parameter.
<code>-instance_voltage_method {Worst Best Avg WorstAvg}</code>	

	Allows different methods of computing effective instance voltage during rail analysis. Using this option, you can write out the Best, Worst, or Average effective instance voltage between power and ground nets. The WorstAvg method computes the average instance voltage during each evaluation window defined by the IV Window field, and reports the worst average instance voltage out of all the windows. If IV Method is WorstAvg and IV Window is whole, then this method will report average voltage across all time steps in simulation. Default : Worst
<pre>-instance_voltage_window {1 0}</pre>	
	Controls the EIV plot. EIV is obtained by processing the voltage waveforms and finding the worse-case effective voltage between the power and ground pins during a specific window. There are two options to view EIV plots: • 1 – both switching and timing windows (window EIV) • 0 – worst of the entire rail simulation (worst EIV)
<pre>-layers {{name report0/1 visible0/1}}{...}}</pre>	
	Controls the visibility of layers. You can enable or disable the reporting and visibility of specific layer. For example, <code>-layer {{M1 1 1} {M2 0 0}}</code> means enable reporting and visibility of layer M1, and disable reporting and visibility of layer M2. This parameter is mutually exclusive to the <code>-disable_report_layers</code>, <code>-disable_visible_layers</code>, <code>-enable_visible_layers</code>, and <code>-enable_report_layers</code> parameters.
<pre>-nets {{all_power 0/1}{all_ground 0/1}}</pre>	
	Controls the display of all the power and ground nets. When <code>all_power</code> is set to 0, hides all power nets, and when set to 1, displays all power nets. When <code>all_ground</code> is set to 0, hides all ground nets, and when set to 1, displays all ground nets. This parameter is mutually exclusive to the <code>-enable_nets</code> and <code>-disable_nets</code> parameters.

<code>-rail_analysis_opacity value</code>	
	Specifies the opacity value for the rail analysis output. The opacity range is from 0-100%, the default value is 100.
<code>-show_layer_net_setting_list {true false}</code>	
	Displays the current status (OFF/ON) of all layers and nets in the console. The following is an example of the output: <pre>#-----# # Layers Report Visible # # RD ON ON # # M0 ON ON # # #-----# # PowerNets Status # # VDD ON # # VDDm ON # #-----# # GroundNets Status # # VSS ON # #-----#</pre>
<code>-via_size {1 2 3 4 5 6}</code>	
	Specifies to modify and control the size of vias in the GUI. The default via size is 1. The possible via size values are 1, 2, 3, 4, 5, and 6.

Examples

- The following command specifies the layers that will be visible:

```
set_power_rail_layers_nets \  
-enable_visible_layers{RD M1 V12 M2 V23 M3 V34 M4 V45 M5 V56 M6} \  
-enable_nets {vdd vdd_1}
```

- The following commands describe the use model for enabling and disabling switch nets. Here, consider a net `VDD` with three switch nets `VDD_sw0`, `VDD_sw1`, and `VDD_sw2`:

- The following command hides the switch net `VDD_sw0`:

```
set_power_rail_layers_net -disable_switch_nets {VDD {VDD_sw0}}
```

Displays `VDD_sw1` and `VDD_sw2`.

- The following command hides all the switch nets:

```
set_power_rail_layers_net -disable_switch_nets {VDD {all}}
```

- The following command displays the switch net `VDD_sw0`:

```
set_power_rail_layers_net -enable_switch_nets {VDD {VDD_sw0}}
```

- The following command displays all the switch nets:

```
set_power_rail_layers_net -enable_switch_nets {VDD {all}}
```

Related Topics

- "[Static Power, IRdrop and EM Analysis](#)" in the *Voltus User Guide*

zoomBox

```
zoomBox  
[-help]  
llx lly urx ury
```

Zooms in the viewable window to the area that encloses the specified coordinates.

Parameters

<code>-help</code>	Prints a brief description that includes type and default information for each <code>zoomBox</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <code>man zoomBox</code> .
<code>llx</code>	Specifies the lower-left x coordinate for the area.
<code>lly</code>	Specifies the lower-left y coordinate for the area.
<code>urx</code>	Specifies the upper-right x coordinate for the area.
<code>ury</code>	Specifies the upper-right y coordinate for the area.

Example

The following command zooms in the viewable window to the area enclosed by the lower-left coordinates (450 531) and the upper-right coordinates (550 631)

```
zoomBox 450 531 550 631
```

zoomIn

zoomIn [-help]

Zooms in the viewable window.

Parameters

-help	Prints a brief description of the <code>zoomIn</code> command. For a detailed description of the command, use the <code>man</code> command: <code>man zoomIn</code> .
-------	--

zoomOut

zoomOut [-help]

Zooms out from the viewable window by two times.

Parameters

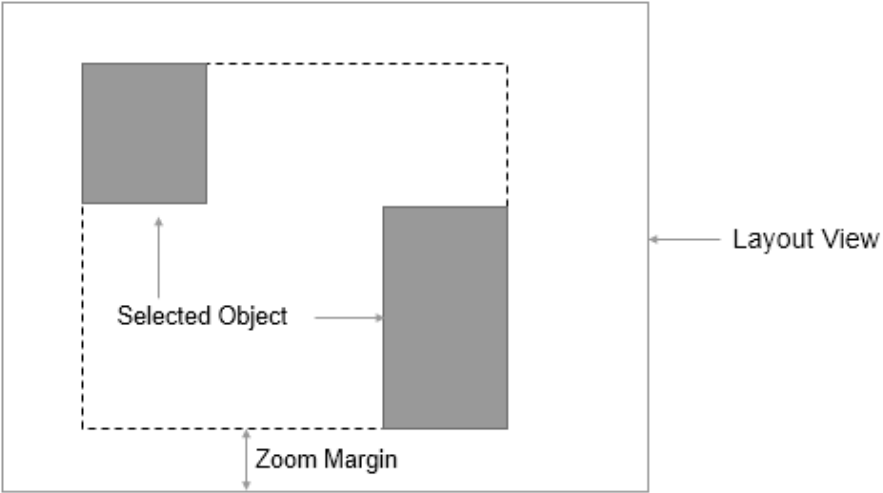
-help	Prints a brief description of the <code>zoomOut</code> command. For a detailed description of the command, use the <code>man</code> command: <code>man zoomOut</code> .
-------	--

zoomSelected

```
zoomSelected
[-help]
[-margin value]
```

Zooms in the viewable window to the area that encloses the selected objects.

Parameters

-help	Prints a brief description of the <code>zoomSelected</code> command. For a detailed description of the command, use the <code>man</code> command: <code>man zoomSelected</code>.
-margin value	Specifies the zoom margin ratio as an integral value. The zoom margin is the minimum spacing between the bounding box of the selected object(s) and the layout view.  The diagram shows a large rectangle labeled 'Layout View' on the right. Inside this rectangle, there is a smaller solid rectangle labeled 'Selected Object' on the left. A dashed rectangle encloses the 'Selected Object' and extends towards the right edge of the 'Layout View'. The space between the 'Selected Object' and the dashed rectangle is labeled 'Zoom Margin' with a double-headed arrow. The minimum value is 0 and the maximum value is 99. <i>Default: 50</i>

zoomTo

```
zoomTo
[-help]
x y
[-radius value]
```

Zooms in the viewable window to the point defined by the coordinates.

Parameters

<code>-help</code>	Prints a brief description that includes type and default information for each <code>zoomTo</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <code>man zoomTo</code>.
<code>-radius value</code>	Specifies the radius of the zoom area in microns, with center at the point specified by <code>x</code> and <code>y</code>. <code>value</code> is of type float. <i>Default:</i> 100 microns
<code>x y</code>	Specifies x and y coordinates of the zoom point.

Example

- When the following command is run, the display zooms in to point with x=100 and y=200. The display area is centered at this point and has the default radius of 100 microns:

```
zoomTo 100 200
```

- When the following command is run, the display zooms in to point with x=100 and y=200. The display area is centered at this point and has a radius of 50 microns:

```
zoomTo 100 200 -radius 50
```

Power-Grid Library Commands

- [check_pg_library](#)
- [generate_pg_library](#)
- [generate_xpgv](#)
- [generate_tech_pg_lib](#)
- [merge_pg_library](#)
- [rename_pg_library](#)
- [set_advanced_pg_library_mode](#)
- [set_pg_library_mode](#)
- [trim_pg_library](#)
- [validate_pg_library](#)

check_pg_library

```
check_pg_library
[-help]
power_library_paths
[-cell_list_file file]
[-check_compatibility {PGV1 PGV2 ...} | -lef_consistency]
[-check_parameters]
[-command_dump]
[-integrity_check]
[-libgen_command_file filename]
[-list]
[-metal_density_map]
[-output file]
[-report]
[-report_detail_multiple_voltage_cap]
[-rule_file file]
[-skip_layer_list_file filename]
[-skip_signal_net_check {true | false}]
[-summary]
[-tap_node_text_dump]
[-tech_layers]
[-total_currents]
```

Specifies to perform a comprehensive check of power-grid views (PGV) to be used in rail analysis. These checks include:

- Verify that all the PGV views exist: Early/IR/EM
- Report the dynamic PWL and static current
- Perform a compatibility check on version, layermap and resistivity parameters
- Check any missing information, such as temperature, qrcTechFile location, and so on.

Parameters

<code>-help</code>	Outputs the command usage.
<code>power_library_path</code>	
	Default: None (Mandatory) Specifies the absolute path to the power library (.cl) file. You can run multiple power library files (.cl) at the same time.
<code>-cell_list_file file</code>	
	Default: None Specifies a file containing the list of cells for which the LEF consistency checks will be done. This parameter allows you to perform the consistency checks only for the user-defined cells. The format of the input file is: <pre>CELL1 CELL2 CELL3</pre>
<code>-check_compatibility {PGV1 PGV2 ...}</code>	

Default: None

Checks for compatibility between the specified cell libraries. It checks whether the following parameters are the same for the libraries that are to be merged:

- Time Units
- Capacitance Units
- Layout Scale Factor
- Manufacturing Grid

If one or more of these parameters are not the same, a warning message appears stating that the libraries are not compatible and cannot be merged. It also specifies the parameters that are different.

Based on the above electrical/physical parameter check, a compatibility report (compatibility.report) with pass/fail is generated. Fail implies that the library is not compatible and cannot be used in Voltus rail analysis.

`-check_parameters`

Default: None

Checks for the following electrical parameters of the specified cell libraries:

- CINT_MIN and CINT_MAX: Intrinsic device capacitance
- CG_MIN and CG_MAX: Grid capacitance
- RON_MIN and RON_MAX: On-resistance for powergates
- IDSAT_MIN and IDSAT_MAX: Saturation current for powergates
- ILEAK_MIN and ILEAK_MAX: Leakage current for powergates
- IAVG_MIN and IAVG_MAX: Static current
- IPEAK_MIN and IPEAK_MAX: Dynamic peak current
- ESR_MIN and ESR_MAX: Equivalent series resistance

For powergates, `-check_parameters` will check the V_{ds} - I_{ds} table for the correct trend. It checks whether:

- for a given V_{gs} , I_{ds} increases with V_{ds}
- for a given V_{ds} , I_{ds} increases with V_{gs}

where;

V_{gs} is voltage across gate and source

V_{ds} is voltage across drain and source

I_{ds} is current from drain to source (of a transistor)

This parameter allows you to check the powergate parameters for all the voltages of a multi-voltage characterized powergate PGV.

`-command_dump`

	Default: None Specifies to print the settings used during PGV characterization to a text file called <code>command_list.txt</code>. This parameter allows you to determine the user-specified or default settings used during PGV generation for the specified power library (.cl).
<code>-integrity_check</code>	
	Specifies to load all the views from a power-grid library to ensure that the library contains the required data. This parameter checks if there are any missing files or views, and that the PGV is usable in rail analysis.
<code>-lef_consistency</code>	
	Default: None Checks for LEF consistency of the cells in the specified cell libraries. It reads the LEF and PGV files, and compares the given LEF file with the LEF file used in PGV generation.
<code>-libgen_command_file filename</code>	
	Specifies to enable the additional LibGen variables <code>copy_intersecting_obs_vias_to_ports</code> and <code>input_type check_lef_pgv_compatibility</code>, respectively, to verify the consistency between the generated PGV and the original LEF file.

-list	Default: None Specifies to list the PGVs and the cells in each PGV. The parameter allows you to determine the name of the PGV library that contains a specific cell and determine whether a cell is there in multiple libraries. The format of the output file is: <pre>PGV Library: <Number of PGV libraries> <Lib_1> <TOOL VERSION> <PGV VERSION> <GENERATE_DATE> <Path> <Lib_2> <TOOL VERSION> <PGV VERSION> <GENERATE_DATE> <Path> ... Duplicated Cells: <Number of cells in multiple libraries> <CELLNAME> <Lib_1 Lib_2 Lib_3> Cell List: <Number of cells in a library> <CELLNAME> <LIB#></pre>
-metal_density_map	
	Default: None Generates the metal density map report file (<i><pgvname>.metal_density</i>) in the PGV output directory. This file contains information about the macro size, number of tiles in the x and y direction, metal layer ID, layer name, and metal density values for each tile. Usage Guidelines: This parameter is used with the <code>-calculate_metal_layer_density</code> parameter of the <code>set_pg_library_mode</code> command.
-output <i>dir_name</i>	
	Default: The default is the current working directory. Specifies the name of the output directory in which the power-grid library report is created.

<code>-report</code>	Default: None Specifies to generate the power-grid library report. The report name is <code><pgvname>.report</code>.
<code>-report_detail_multiple_voltage_cap</code>	
	Default: None Specifies to generate a voltage-dependent capacitance table that contains the detailed capacitance values of the given power-grid library. The capacitance table file is called <code><pgvname>.cap_table</code>. Usage Guidelines: This parameter is used with the <code>-enable_multi_voltage_cap_generation</code> parameter of the <code>set_pg_library_mode</code> command.
<code>-rule_file file</code>	

	Default: None Specifies the rule file name used for parameter checking. This file contains the list of electrical parameters to be checked. File Format: <pre> CINT_MIN <min cap value> CINT_MAX <max cap value> RON_MIN <value> RON_MAX <value> IDSAT_MIN <value> IDSAT_MAX <value> ILEAK_MIN <value> ILEAK_MAX <value> IPEAK_MIN <value> IPEAK_MAX <value> IAVG_MIN <value> IAVG_MAX <value> ESR_MIN <value> ESR_MAX <value> </pre> Usage Guidelines: This parameter must be specified with the – <code>check_parameters</code> parameter.
	<pre>-skip_layer_list_file filename</pre>
	Default: None Specifies the name of a file containing a list of layers to ignore while checking PGV and LEF compatibility. Its format is one layer per line. Usage Guidelines: This parameter must be specified with the – <code>check_compatibility</code> parameter.
	<pre>-skip_signal_net_check {true false}</pre>

	Default: <code>false</code> Specifies to skip LEF consistency checks for signal nets. This forces skipping of the signal pin checks when performing a comprehensive check of power-grid views to be used in rail analysis. The <code>-skip_signal_net_check</code> parameter can be used if you want to check LEF consistency of only the power and ground pins.
<code>-summary</code>	
	Default: None Specifies to generate a detailed report of the library contents for the specified cells that can be used for debugging purpose.
<code>-tap_node_text_dump</code>	
	Default: None Specifies to generate a detailed report of the tap current distribution for each interface node. The report name is <code>tapNodeDetailReport.txt</code>. It lists both the peak and average current values, capacitance value, and the number of taps for each interface node.
<code>-tech_layers</code>	
	Default: None Specifies to report technology layer information and layer mapping.
<code>-total_currents</code>	
	Default: None Specifies to report the static current for each power/ground net, and dynamic PWL for each mode. For each mode, it reports the peak and average current, and the number of points for power supply nets. When specified, the total current is reported in <code><lib_name>.total_current</code> for each library.

Examples

- The following command creates a report for the library std_cells.cl:

```
check_pg_library std_cells_fast/std_cells_fast.cl -report
```

- The following command checks for LEF consistency:

```
read_lib -lef ddk.0.lef
read_lib -lef ddk.1.lef
check_pg_library -lef_consistency \
"/../library1.cl\
/ ../library2.cl"
```

- The following command displays the list of layers (defined in the technology file) on stdout:

```
check_pg_library -tech_layers ./check_param/powergate/stdcells.cl
```

- The following command checks for compatibility between the specified cell libraries and generates a compatibility report (compatibility.report) in the output directory:

```
check_pg_library -output out -check_compatibility {/ ../lib1.cl / ../lib2.cl}
```

- The following command specifies to list the cells corresponding to the specified libraries, pgv1.cl and pgv2.cl:

```
check_pg_library -list {/ ../pgv1.cl / ../pgv2.cl}
```

- The following command checks for electrical parameters of the stdcells.cl library:

```
check_pg_library -output OUT -check_parameters ./stdcells.cl
```

The following is a sample output of this command:

	PARAM	VALUE	RANGE	PASS/FAIL
Cell: Cell1				
Net VDDG				
	Device Cap	4.50e-14	(1.00e-18,1.00e-11)	PASS
Net VSS				
	Device Cap	2.61e-14	(1.00e-18,1.00e-11)	PASS
Net VDD				
	Device Cap	2.13e-14	(1.00e-18,1.00e-11)	PASS
Power Gate SLEEP				
	Ron	1.84e+01	(2.00e+01,1.00e+03)	FAIL

```
Ileak      7.01e-08      (1.00e-15,1.00e-06)      PASS
Idsat      3.36e-02      (1.00e-09,1.00e-02)      FAIL
```

Trend Checks:

```
Ron Trend    : PASS
Idsat Trend   : PASS
Ileak Trend  : PASS
```

- The following command reports the static current for each power/ground net:

```
check_pg_library -total_currents ../pg.cl
```

The following is a sample output of this command:

```
Net VSS
Static current = -0.00021181

Net VDD
Static current = 0.000211809

MODE1
Trigger Condition: ( !CEN & !WEN )
Net VDD
    Peak = 0.0138308
    Avg = 0.000276053
    Number of points 190

Net VSS
    Peak = -0.0166879
    Avg = -0.000300184
    Number of points 190

MODE2
...
```

- The following command generates a capacitance table (`powergate_stdcells.cap_table`) that contains the detailed capacitance values of the `powergate_stdcells.cl` library:

```
check_pg_library -report_detail_multiple_voltage_cap ../powergate_stdcells.cl
```

The following is a snippet of the capacitance table file (`powergate_stdcells.cap_table`):

```
=====
CellName      NetName      Voltage      Capacitance
-----
powercell     vccm          0.1000       4.586e-14
powercell     vccm          0.2825       4.102e-14
...
```

- The following command specifies to output the settings used during generation of macros_pll.cl to a text file called command_list.txt:

```
check_pg_library -command_dump macro_pgv/macros_pll.cl
```

The following is a sample snippet of the command_list.txt file:

```
temperature 25
spice_subckts ../spice.sp
spice_models ../spectre_load.sp
extraction_tech_file ../tech.tch
gds_files ../file1.gds
stop@via VIA_1
spice_subckts_xyscaling 1000
use_powergate_data_from_lib 0
default_frequency 2e+08
decap_frequency 1e+08
```

- The following command skips layer checks for the list of layers specified in the sub_layers.txt file:

```
check_pg_library -check_compatibility -skip_layer_list_file sub_layers.txt
```

```
----sub_layers.txt----
```

```
VNW
```

```
VPW
```

- The following command generates the power-grid library report:

```
check_pg_library -output report -report ../pg.cl
```

The following is a sample output of this command:

```
=====
Library path name:      /it/./library
Library name:          pg
Hierarchical library:   NO
User login:            user
Date:                  2015-Sep-03 03:58:45 (2015-Sep-03 10:58:45 GMT)
ToolIdentifier:         VOLTUS
Application version:    15.20-d127_1
Number of cells in library: 1
=====
```

Cell	PowerNets	Capacitance	PowerGrid Views

cell11			
	VSS(0.0000)	5.3e-12	EARLY(255 taps) EM(8827 taps)
IR(8827 taps)			
	VDD(0.9000)	6.8e-12	EARLY(193 taps) EM(7534 taps)
IR(7534 taps)			
Cell Type = MACRO		Cell_Status: PASS	
INFO: Dynamic Tap Current views were created for this cell. MODE1 has Conditional input = "(!CEN & !WEN) " and Conditional pin = "CLK RISE".			
INFO: Dynamic Tap Current views were created for this cell. MODE2 has Conditional input = "(CEN & WEN) " and Conditional pin = "CLK RISE".			

Related Topics

- "[Power-Grid Library Generation](#)" in the *Voltus User Guide*

generate_pg_library

```
generate_pg_library
[-help]
[-library_prefix prefix]
[-output dir_name]
```

Generates the output for power-grid library generation.

Parameters

-help	
	Outputs the command usage.
-library_prefix <i>prefix</i>	Default: If this parameter is not specified, the default PGV for tech-only is <code>techonly.cl</code>, for standard cells is <code>stdcells.cl</code>, and for macros is <code>macros_cellname.cl</code>. Specifies a prefix for the power-grid library. When this parameter is specified, the PGV for each cell type is as follows: • <code>techonly</code> - <code>prefix_techonly.cl</code> • <code>stdcells</code> - <code>prefix_stdcells.cl</code> • <code>macros</code> - <code>prefix_cellname.cl</code> You can generate macro PGVs without the "<code>macro_</code>" string in the name and without any prefix by specifying an empty string for <code>-library_prefix</code>, as shown below: <pre>generate_pg_library -output <i>output_folder</i> -library_prefix ""</pre>
-output <i>dir_name</i>	

Default: The default is the current working directory.

Specifies the name of the output directory in which power-grid libraries are created.

Note: If you specify both `-output dir_name` and `-library_prefix prefix` parameters, the command generates the PGV in the *dir_name* directory with PGV name `prefix_<PGVname>.cl`.

Examples

- The following command generates a power-grid library in the *temp* folder:

```
generate_pg_library -output temp/
```

- The following command specifies to generate a technology library with the default PGV name *techonly* and prefix *ab*:

```
set_pg_library_mode -celltype techonly ...
```

```
generate_pg_library -output ./ -library_prefix ab
```

The generated library name will be `ab_techonly.cl`

- The following command specifies to generate a macro library with the default PGV name *<cell_name>* and prefix *ab*:

```
set_pg_library_mode -celltype macros ...
```

```
generate_pg_library -output ./ -library_prefix ab
```

The generated library name will be `ab_<cell_name>.cl`

generate_xpgv

```
generate_xpgv
-cell cell_name
[-inst_list inst1 inst2 .. instn ]
[-libgen_include file_name]
[-mvmf_config config_file]
[-output_dir directory_name]
[-rail_include file_name]
[-reuse_block_state_directory directory_name]
-state_directory directory_name
```

Creates power-grid view (xPGV) models for large digital blocks. The block xPGV can be generated either after the signoff dynamic IR drop analysis or after the dynamic Early Rail Analysis (ERA) of the block. In both cases, the xPGV is generated using the R-C grid of the block and currents captured at the via layer during the analysis. These block-level xPGVs can then be instantiated during the next level of the design or the top-level full-chip analysis. The next level rail analysis will run faster with the block-level xPGVs because these runs use the power-grid that is modeled inside the xPGV.

Parameters

- help	Outputs the command usage.
-cell <i>cell_name</i>	
	Default: None (Mandatory) Specifies the block name for which xPGV model is generated.
-inst_list <i>inst1 inst2 .. instn</i>	
	Default: None Specifies the block instances for which xPGV model is generated.

<code>-libgen_include file_name</code>	
	Default: None Specifies a file that includes additional library generation options that will be run for xPGV model generation.
<code>-mvmf_config config_file</code>	
	Default: None Specifies to generate a single xPGV model with different functional modes. Using this parameter, you can specify a file that includes the different modes and the mapping state directory for each mode. When <code>-mvmf_config</code> is specified, the block-level currents are saved for multiple voltages and frequency (MvMf) per each functional mode in a single PGV. During xPGV instantiation, the different modes are automatically selected based on the power domain to which each mode is connected. The format of the <code>config_file</code> file is: <pre><modeName1> <stateDir_1> <modeName2> <stateDir_2> ... <modeNameN> <stateDir_N></pre> Before generating MvMf-xPGV, you need to perform multiple rail analysis runs for different corners to generate different state directories (<code>stateDir_*</code>) to be included in the configuration file.
<code>-output_dir directory_name</code>	
	Default: current working directory Specifies the name of the output directory in which the power-grid library report is created.
<code>-rail_include file_name</code>	
	Default: None Specifies a file that includes additional rail analysis options that will be run for xPGV model generation.
<code>-reuse_block_state_directory directory_name</code>	

Default: None

Specifies an existing rail analysis state directory to skip block extraction. The software assumes that the design data (DEFs and GDS) and power-grid libraries are not changed, and will generate xPGV using the extraction data inside the specified state directory. If the state directory does not contain the necessary design and power-grid database files, it will issue an error message and provide information on the missing files.

The `-reuse_block_state_directory` parameter is used if the entire design that is analyzed in the rail run should be used to create the xPGV. However, this parameter must not be used in case you require to generate xPGVs for each instance (sub-blocks) of a block. That is, for generating xPGVs for the sub-blocks B and C of the block A (rail analysis was performed on block A), you need to re-extract the block-level grids of B and C as part of xPGV generation for accurate current calculation. In this case, `-reuse_block_state_directory` must not be specified.

For net based analysis, you should specify net-based state directory, as shown below:

- `generate_xpgv -reuse_block_state_directory VDD_25C_avg_1`

For domain based analysis, you should specify the domain's state directory, as shown below:

- `generate_xpgv -reuse_block_state_directory domain_25C_dynamic_1`

A new state directory will be generated for the new analysis with soft links to the old state directory files that were reused.

`-state_directory` *directory_name*

Default: None

(Mandatory) Specifies the path to the rail analysis state directory.

Examples

- The following command creates an xPGV library `hier_block.cl`:
`generate_xpgv -state_directory dynamic_rail/core_125C_dynamic_1/ -cell hier_block -output_dir ../xpgv/`
- The following command creates an xPGV library `super_filter_A` with multiple functional modes specified in the `mode.config` file:

```
generate_xpgv -cell super_filter_A -mvmf_config mode.config -output_dir  
./mode_mvmf -rail_include rail.inc
```

The following is a snippet from the `mode.config` file:

```
label1 dynamic_rail/core_25C_dynamic_1  
label2 dynamic_rail/core_25C_dynamic_1
```

Related Topics

- "[Power-Grid Library Generation](#)" in the *Voltus User Guide*

generate_tech_pg_lib

```
generate_tech_pg_lib  
[-help]  
[-decap_cells cell_list]  
[-default_power_voltage value]  
[-esd_cells cell_list]  
[-filler_cells cell_list]  
[-keep {true | false}]  
[-lef_layermap filename]  
[-output_dir dir_name]  
[-power_gate_file filename]  
-tech_file filename  
[-tech_only {true | false}]
```

Generates technology power-grid library on the fly. The generated tech-PGV will be deleted after the session is over. If this command is specified, the Power Analysis and Rail Analysis engines will automatically use the on the fly generated tech-PGV. The purpose of generating the tech-only PGV on the fly is that you do not need to keep a track of the tech-only PGVs. Thus, it ensures that the tech-only PGV is generated from the correct Quantus technology file.

The tech-only PGV library will be generated under the working directory and will be called by Power/Rail Analysis on the fly, as shown in the following log file snippet:

```
Begin Loading PGV Libraries for Power Calculation  
./pgv/techonly.cl  
Ended Loading PGV Libraries for Power Calculation:  
(cpu:0:00:00,real=0:00:00,mem(process/total/peak)=275.25MB/2585.14MB/1194.52MB)  
  
Begin Processing Cell Libraries for Rail Analysis  
  
Merging libraries:  
./pgv/techonly.cl  
../../../../LIBS/voltus_pgv/macro_pgv/macros_sram.cl  
../../../../LIBS/voltus_pgv/stdcells_pgv/stdcells.cl  
  
Output library: ./work/lib_merged.cl
```

Parameters

<code>-help</code>	
	Outputs the command usage.
<code>-decap_cells cell_list</code>	
	Default: None Specifies the names of cells that have explicit decoupling capacitance for use in decap optimization. These cells are nearly equivalent to feedthrough cells, except that there is typically an active device between the power and ground nets to provide extra decoupling capacitance. Power-grid view library generation uses this list of cells to ensure that a capacitance value is associated with each cell. If no capacitance is associated with a cell, a warning message will be issued.
<code>-default_power_voltage value</code>	
	Default: 1.2 Specifies voltage for the power pins which are present in LEF.
<code>-esd_cells cell_list</code>	
	Default: None Specifies a list of Electrostatic Discharge (ESD) cells. You can use this parameter to characterize ESD cells. When you specify this parameter, the list of ESD cells are later used in the <code>analyze_esd_network</code> command while loading the power-grid view library.
<code>-filler_cells cell_list</code>	
	Default: None List the names of the filler or feedthrough cells that have no GDS data. These cells can be swapped out for decaps during decap optimization.
<code>-keep {true false}</code>	
	Default: <code>false</code> Specifies that the generated library should be retained after the session completes.

`-lef_layermap filename`

Default: None

Specifies a file that provides the mapping of layers between the LEF and technology file. This parameter is optional. Library Generation automatically generates the layermap file using the technology LEF (containing all layer and via definitions) and Quantus Tech (extraction library) files, therefore, a LEF layermap file is not a mandatory input. The automatically generated layermap file is located in the PGV temp directory.

The following is the format and sample content of the auto-generated file (lefdefLayerMap_AutoGenerated.txt):

<u>Layer Type</u>	<u>tech layername</u>	<u>lefdef</u>	<u>lefdef layername</u>
via	VIA_9	lefdef	VIA9
metal	METAL_1	lefdef	METAL1

You can specify this parameter if you want to provide your own layermap file.

`-output_dir dir_name`

Default: current working directory

Specifies the name of the output directory in which tech-only power-grid library is created.

`-power_gate_file filename`

Default: None

Specifies a file with the information that technology library generation needs to characterize a powergate cell. Specifies the cell name, unswitched power pin, switched power pin, on resistance in ohms, saturation current in milliamps, and leakage current in milliamps.

File Format:

cell supply switched Ron Idsat Ileak

Examples:

The following is an example of multiple footer powergate:

```
FOOTBUFVHSV16 VSSG VSS 63 3.167 0.000128252
FOOTBUFVHSV32 VSSG VSS 63 3.167 0.000128252
FOOTBUFVHSV64 VSSG VSS 63 3.167 0.000128252
FOOTBUFVHSV8 VSSG VSS 63 3.167 0.000128252
```

`-tech_file filename`

	Default: None (Mandatory) Specifies the name of the technology file that will be used for extraction.
<code>-tech_only {true false}</code>	
	Default: <code>false</code> Specifies to generate the technology PGV without any cells.

Examples

- The following command generates a tech-only power-grid library:

```
generate_tech_pg_lib -tech_file $tech_path/qrcTechFile -filler_cells *FILL* -  
decap_cells *DCAP* -default_power_voltage 0.9
```

merge_pg_library

```
merge_pg_library
[-help]
[-force {true | false}]
-library_list_file file_name
[-library_prefix prefix]
[-merge_ddv_currents {true | false}]
[-merge_single_voltage_pgvs {true | false}]
[-output directory_name]
[-remove_tech {true | false}]
[-validate_rail_merge {true | false}]
```

Specifies to merge power-grid libraries. Using this command, you can merge multiple same-type power-grid libraries into one consolidated power-grid library.

Parameters

- help	Outputs the command usage.
-force {true false}	
	Default: false Specifies to force merge PGVs with different resistivity and layers. It uses the technology information of the first library.
-library_list_file <i>file_name</i>	
	Default: None (Mandatory) Specifies the file containing the list of libraries (one per line) to be merged. File Format: PGV1 PGV2 PGV3

<code>-library_prefix <i>prefix</i></code>	
	Default: None Specifies a prefix for the generated power-grid library.
<code>-merge_ddv_currents {true false}</code>	
	Default: <code>false</code> Specifies to merge the tap currents from multiple DDV PGVs having different voltages. The parameter can be used during rail analysis to consider multiple single-voltage DDVs for a particular cell like a multi-voltage PGV for that cell. Usage Guidelines: This parameter is applicable only for macro PGV generation (<code>-celltype macros</code>).
<code>-merge_single_voltage_pgvs {true false}</code>	
	Default: <code>false</code> Specifies to merge all the single voltage PGVs into a multi-voltage PGV for better handling of the library characterization flow. This parameter is used in a scenario when you have generated multiple PGVs for different voltages, instead of generating a single multi-voltage PGV that has voltage-dependent capacitance.
<code>-output <i>directory_name</i></code>	
	Default: current working directory Specifies the name of the output directory in which the merged power-grid library will be created. The merged library is called <code>merged_lib.cl</code>. If you specify both <code>-output <i>output_dir</i></code> and <code>-library_prefix <i>prefix</i></code> parameters, the command generates the PGV in the <code><i>output_dir</i></code> directory with PGV name <code><prefix>.cl</code>.
<code>-remove_tech {true false}</code>	
	Default: <code>false</code> Specifies to remove technology dependency from the specified or merged library. The newly created library will not contain the technology information.
<code>-validate_rail_merge {true false}</code>	

Specifies to validate the merged PGV in the same way as done when merging PGV during rail analysis. When this parameter is specified, the log file reports error, warning, and info messages if the tool finds any issues during PGV merging. The parameter allows you to check the PGV merging issues as reported during rail analysis, without having to run the full design.

`-validate_rail_merge` is used only for validation purpose, and the merged PGV is not to be used for analysis.

Examples

- The following command merges the power-grid libraries specified in the `merge_lib.list` file into one consolidated power-grid library:

```
merge_pg_library -library_list_file "merge_lib.list" -output ./ -library_prefix  
"new_merged"
```

The content of the `merge_lib.list` file is:

```
library1.cl  
library2.cl  
library3.cl
```

- The following command merges the power-grid libraries specified in the `lib.txt` file into one consolidated power-grid library and validates the merged library:

```
merge_pg_library -library_list_file lib.txt -force true -validate_rail_merge true
```

The content of the `lib.txt` file is:

```
fast_allcells.cl  
pv_library.cl
```

Related Topics

- "[Power-Grid Library Generation](#)" in the *Voltus User Guide*

rename_pg_library

```
rename_pg_library
[-help]
-cell_list_map_file filename
-merge_library_list_file filename
-output directory_name
-output_library new_library_name
```

Specifies to copy and rename the power-grid view library. The command also allows you to rename the cells of the specified power-grid library.

Parameters

- help	Outputs the command usage.
-cell_list_map_file filename	
	Default: None (Mandatory) Specifies a file containing the list of cells to be renamed. File Format: <pre><old cell name1> <new cell name1> <old cell name2> <new cell name2> ...</pre>
-merge_library_list_file filename	

	Default: None (Mandatory) Specifies a file containing the power-grid library to be renamed. If you specify multiple libraries, they are merged into one consolidated output library. File Format: <pre><path to library1> <path to library2> <path to library3></pre>
<code>-output directory_name</code>	
	Default: None (Mandatory) Specifies the name of the output directory in which the new power-grid library is created.
<code>-output_library new_library_name</code>	
	Default: None (Mandatory) Specifies the name of the new power-grid library.

Examples

- The following command specifies to rename the input library to `new_lib`:

```
rename_pg_library -merge_library_list_file new.list -cell_list_map_file cell.list
-output_library new_lib -output .
```

Related Topics

- "[Power-Grid Library Generation](#)" in the *Voltus User Guide*

set_advanced_pg_library_mode

```
set_advanced_pg_library_mode
[-abort_on_extraction_errors {true|false}]
[-add_port_labels file_name]
[-assume_foreigns {true | false}]
[-assume_foreigns_mode [1 | 0]]
[-cell_accura_data_file file]
[-cell_per_distributed_host value]
[-cell_pinnet_map_file file]
[-circuit_include_file thunder.inc]
[-cluster_via_rule {{via_layer1 number_of_equidistant_vias}...}]
[-cluster_via_size value]
[-common_supply_pins {net_name+}]
[-create_static_view_from_dynamic_view {true | false}]
[-damping_decap_cell_list {cell1cell2 ..}]
[-damping_decap_frequency value]
[-decap_frequency value]
[-default_frequency value]
[-default_power_voltage value]
[-delete_ddv_fsdb_files {true|false} ]
[-disable_powergate_rampup_simulation{true | false}]
[-distribute_current_to_switch_net {true | false}]
[-enable_ac_simulation_for_IO_characterization {true | false}]
[-enable_spectre_netlist_flow {true | false} ]
[-enable_subconductor_layers {true | false}]
[-enable_via_based_current_distribution {true | false}]
[-esd_cells {cell_list}]
[-esd_cells_list_file filename]
[-esd_device_list {device1device2 .... deviceN}]
[-esd_parameters_file filename]
[-esd_pin_list {pin1pin2 .... pinN}]
[-exclude_tap_region_file filename]
[-extraction_command_file file]
[-followpins_tap_layer {lowest_lef_pin_layer | highest_lef_pin_layer |
all_lef_pin_layers}]
[-followpins_interface_node_layer {lowest_lef_pin_layer | highest_lef_pin_layer |
all_lef_pin_layers}]
[-force_tap_node_distance {true | false}]
[-generate_bulk_pin_cap_separately {true | false}]
```

```

[-generate_graybox_data {true | false} ]
[-generate_itfnode_x_direction {true | false}]
[-generate_itfnode_y_direction {true | false}]
[-generate_itfnodes_at_via_layer {true | false}]
[-generate_pin_intrinsic_cap {true | false}]
[-generate_pin_intrinsic_cap_list {netname1 netname2...}]
[-ignore_pg_nets {{cellnamenetname}+}]
[-input_port_value_list {pin1value1pin2value2 ....pinNvalueN}]
[-interface_node_location_file filename]
[-lef_layer_ignore_list {lef_layername +}]
[-lef_layer_ignore_list_file filename]
[-lef_pin_short {true | false}]
[-lef_pin_short_cell_file filename]
[-lef_pin_short_cell_list {cell1 cell2 ...}]
[-libgen_command_file file]
[-macro_parasitic_file filename]
[-marker_layermap filename]
[-mcpu_rerun_countcount]
[-pg_arc_file filename]
[-pgv_path path]
[-powergate_characterization_voltages { val1val2val3 .... }]
[-powergate_multi_enable_characterization_method {enable_all | enable_single} ]
[-process_bulk_diffusion_ports {true|false}]
[-remove_emview_dangling_resistor {true|false}]
[-retain_generated_pgv_on_error {true | false}]
[-reuse_electrical_data {true | false}]
[-ron_measure_threshold value]
[-schematic {true|false} ]
[-skip_switch_net_extraction {true | false}]
[-source_location_file {filename}]
[-spectre_path value]
[-spectre_standalone_simulation {true | false}]
[-stdcell_characterization_voltage_value {val1val2val3 ...}]
[-strict_input_check {true|false}]
[-tap_node_distance value]
[-techgen_dir directory]
[-thunder_command_file file]
[-trim_metal_layer_map file]
[-use_embedded_spectre {true | false}]
[-use_peak_current_distribution {true | false}]
[-use_physical_view_from_techpgv {true | false}]
[-verbosity {true | false}]
[-via_based_current_config_file filename]

```

```
[-well_cap_file filename]
[-xtc_command_file filename]
[-xtc_include_file_for_qdv filename]
```

Specifies the advanced power-grid library generation features. If the `set_advanced_pg_library_mode` command is specified twice, the last specified command takes precedence.

Parameters

<code>-abort_on_extraction_errors {true false}</code>	
Default: <code>true</code> Specifies whether the software should exit with an error message or continue when it encounters extraction errors. Usage Guidelines: This parameter is applicable only for macro PGV generation (<code>-celltype macros</code>).	
<code>-add_port_labels file_name</code>	
Default: None Specifies the name of a file containing details about the label of a switched rail. Using this parameter, you can add a label for the switched rail manually in the specified port file, if the switched rail is not labeled. File Format: <pre>MACRO cell_name PIN <pinname> <direction> <x-coord in microns> <y-coord in microns> <layer></pre> For more information on the format, refer to the "Port File" section in the "File Formats" chapter of <i>Voltus User Guide</i>. Usage Guidelines: This parameter is applicable only for macro PGV generation (<code>-celltype macros</code>).	
<code>-assume_foreigns {true false}</code>	

Default: `false`

Enables the Library Characterization engine to process LEF files in which FOREIGN statements are not defined as part of the macro definition. When generating detailed power-grid views, the engine assumes that FOREIGN statements are defined as part of the macro definition in the LEF file.

When this option is set to `true`, the engine processes the LEF file by using the name defined by the MACRO statement rather than requiring a FOREIGN statement. The `-assume_foreigns` parameter ensures that the FOREIGN statement is placed at 0,0. Therefore, if the MACRO statement has an offset, you must provide a FOREIGN statement that contains an offset. This option works for offsets that are 0,0 or the inverse of the origin.

Usage Guidelines:

This parameter is applicable only for macro PGV generation (`-celltype macros`).

`-assume_foreigns_mode [1 | 0]`

Default: `1`

Determines how the Library Characterization engine processes cells for which FOREIGN statements are assumed. If set to 0, the engine assumes that the cell for which FOREIGN statements are assumed is placed at (0,0). When you set it to 1, the engine assumes that the FOREIGN cell is placed at the inverse of the macro origin.

Usage Guidelines:

This parameter is applicable only for macro PGV generation (`-celltype macros`).

`-cell_accura_data_file file`

Default: None

Specifies the name of a file containing data about the electrical characteristics of cells. Voltus library simulator requires this data to generate currents during power-grid view generation.

File Format:

```
CELL cellname
{
<FREQ=value>
<FA_VALUE=value>
<AVG_PWR=value>
<CLK_FILE=filename>
<FA_FILE=filename>
<AVG_PWR_FILE=filename>
}
```

For more information on the format, refer to the "[Cell Accura Data File](#)" section in the "**File Formats**" chapter of *Voltus User Guide*.

Usage Guidelines: This parameter is applicable only for macro PGV generation (-celltype macros).

-cell_per_distributed_host value

Default: 50

Specifies the number of cells to be submitted per host for distributed PGV generation.

Usage Guidelines: This parameter is applicable only for standard cell PGV generation (-celltype stdcells).

-cell_pinnet_map_file file

Default: None

Specifies the mapping between the cell LEF pin names and the internal names of the net to which those pins are attached. It is used when multiple pins are attached to a single net.

File Format:

```
cellname net_name pin_name
```

For more information on the format, refer to the "[Cell Pin Net Map File](#)" section in the "**File Formats**" chapter of *Voltus User Guide*.

```
-circuit_include_file simulator.inc
```

Default: None

Specifies the name of the input file for Voltus library simulator in the Library Characterization command file.

Usage Guidelines:

This parameter is applicable only for standard cell and macro PGV generation (`-celltype stdcells | macros`).

```
-cluster_via_rule {{via_layer1 number_of_equidistant_vias}...}
```

Controls the number of vias to cluster on a layer basis. The VIA clustering rule specified using this parameter will override the default clustering rule for PGV generation. You can use this parameter when you need clustering at VIA1, or different clustering for different layers. The default cluster size for PGV generation is 64 for all via layers except VIA1.

Example:

The following command will cluster 100 equidistant VIA1 cuts, 200 equidistant VIA2 cuts, and 300 equidistant VIA7 cuts:

```
set_advanced_pg_library_mode -cluster_via_rule {{VIA1 100} {VIA2 200} {VIA7 300}}
```

```
-cluster_via_size value
```

Default: 64

Specifies that power-grid extraction for library cell needs to perform 64 (8x8) via clustering inside PGV for all via layers above VIA1 between Metal1 and Metal2.

The following command specifies to perform 16 (4x4) via clustering:

```
set_advanced_pg_library_mode -cluster_via_size 16
```

Usage Guidelines:

This parameter is applicable only for macro PGV generation (`-celltype macros`).

```
-common_supply_pins{net_name+}
```

Default: None	Identifies the generic supply net or nets which could act as either a power or ground supply net. No wildcards are allowed for the specification of net names. Generic supply nets are used for pads or bump cells that could be connected to either a power or ground net. They will be connected to the correct power-grid based on the DEF or GDS file.
<code>-create_static_view_from_dynamic_view {true false}</code>	
Default: false	Specifies to generate static power-grid view by averaging of dynamic currents in the multi-mode characterization flow. When this parameter is specified, the software performs averaging of the dynamic currents of all the modes and saves it as static current in the power-grid view.
<code>-damping_decap_cell_list {cell1 cell2 ..}</code>	
Default: None	Specifies a list of damping decap cells.
Usage Guidelines:	
This parameter is applicable only for technology and standard cell PGV generation (<code>-celltype techonly stdcells</code>).	
<code>-damping_decap_frequency value</code>	
Default: 200kHz	Specifies the frequency of operation of the damping decap cells.
Usage Guidelines:	
This parameter is applicable only for standard cell PGV generation (<code>-celltype stdcells</code>).	
<code>-decap_frequency value</code>	

Default: 1GHz

Specifies the decoupling capacitance frequency used during PGV generation. Intrinsic capacitance of cells has a significant dependency on frequency used during PGV generation. PGVs characterized with higher frequency may lead to pessimism during dynamic analysis due to less impact of intrinsic capacitance of the cells characterized with higher frequency. Using this parameter, you can specify the dominant frequency to be used for PGV generation.

Usage Guidelines: This parameter is applicable only for standard cell PGV generation (`-celltype stdcells`).

`-default_frequency value`

Default: 200MHz

Specifies the default frequency of signal nets in hertz (Hz); this is used to calculate the current and capacitance values during the creation of detailed power-grid views.

For example, the following command specifies that the default switching frequency is 100 MHz:

- `set_advanced_pg_library_mode -default_frequency 100e+06`

Usage Guidelines:

This parameter is applicable only for macro PGV generation (`-celltype macros`).

`-default_power_voltage value`

Default: None

Specifies the default voltage for power nets. This parameter assigns a default value to power nets for which voltage has not been defined using the `-power_pins` parameter.

`-delete_ddv_fsdb_files {true|false}`

Default: false

Specifies to delete the intermediate FSDB files generated in the Spice deck (user-specified) method of current characterization. This parameter allows you to retain or delete the huge FSDB files.

Usage Guidelines:

This parameter is applicable only for macro PGV generation (`-celltype macros`).

```
-disable_powergate_rampup_simulation {true | false}
```

Specifies whether to run power gate ramp-up simulation during power-grid view generation. When this parameter is set to `true`, the `Ids-Vds` table will not be present in PGV, therefore, this PGV cannot be used for the power-up or rush current analysis flow. This parameter can be specified to reduce the run time when you require only the `RON/Idsat/Ileak` and `Cap` values in PGV.

`disable_powergate_rampup_simulation` is used when power gate parameters are specified using the `set_pg_library_mode -powergate_parameters_file` parameter. This parameter can only be used for the standard cell PGV generation.

Default: `false`

```
-distribute_current_to_switch_net {true | false}
```

Default: `false`

Distributes the PWL waveform, which is specified to the always-on power net, to the taps of both the always-on power net and the related switched power nets in the fine-grain memory characterization flow. By default, the current waveform is distributed only to the taps of the always-on power net when you specify the PWL to the always-on power net.

Usage Guidelines:

This parameter is applicable only for macro PGV generation (`-celltype macros`).

```
-enable_ac_simulation_for_IO_characterization {true | false}
```

Default: `false`

Specifies to perform an AC analysis based coupled capacitance calculation for IO cells. When set to `false`, the software performs DC analysis based coupled capacitance calculation for IO cells.

The AC analysis based coupled capacitance calculation is slower than the DC analysis based coupled capacitance calculation.

Usage Guidelines:

This parameter is applicable only for macro PGV generation (`-celltype macros`).

```
-enable_spectre_netlist_flow {true | false}
```

Default: <code>false</code> Specifies to use the Spectre simulator to read the Spice/Spectre netlist for the standard cells characterization flow. This option should be used when the netlist is in Spectre format, or when the netlist is encrypted by Spectre.	
<pre>-enable_subconductor_layers {true false}</pre>	
Default: <code>true</code> for tech-only PGV and <code>false</code> for standard cells and macros Allows to control the generation of the sub-conductor layers in the PGV. When this parameter is set to <code>false</code>, the sub-conductor layers are not generated in the PGV.	
<pre>-enable_via_based_current_distribution {true false}</pre>	
Default: <code>false</code> Specifies to enable location-based via current modelling during GDS-based macro power-grid view generation. When set to <code>true</code>, defines the current values only on the user-defined via location. This parameter allows you to define average and peak current values for specific-location vias. The <code>-enable_via_based_current_distribution</code> parameter must be specified with the <code>-via_based_current_config_file</code> parameter. If the parameter is set to <code>false</code>, current is uniformly distributed across all vias in the macro PGV.	
<pre>-esd_cells {cell_list}</pre>	
Default: None Specifies a list of Electrostatic Discharge (ESD) cells. You can use this parameter to characterize ESD cells. When you specify this parameter, the list of ESD cells are later used in the <code>analyze_esd</code> command while loading the power-grid view library. As a result, you do not need to specify the ESD cell names with the <code>-esd_cell_list</code> parameter of the <code>analyze_esd</code> command. Usage Guidelines: This parameter is applicable only for technology and standard cell PGV generation (<code>-celltype techonly stdcells</code>).	
<pre>-esd_cells_list_file filename</pre>	

Default: None

Specifies a file containing a list of ESD cells that are later used in the `analyze_esd` command while loading the power-grid view library. As a result, you do not need to specify the ESD cell names with the `-esd_cell_list` parameter of the `analyze_esd` command.

The `-esd_cells_list_file` parameter is the same as the existing `-esd_cells` parameter of the `set_advanced_pg_library_mode` command, except that the `-esd_cells_list_file` parameter allows you to specify a list of ESD cells in a file while the `-esd_cells` parameter allows you to specify the ESD cells in the command line itself.

The following is an example of this parameter:

```
set_advanced_pg_library_mode -esd_cells_list_file {./Esd_Cell_list.txt}
```

File Format:

```
CELL_1 <type>  
CELL_2  
...  
CELL_n
```

Here,

- the first column specifies the ESD cell name
- the second column specifies the ESD cell type, such as DIODE and CLAMP. This is optional.

The following is a snippet of the input file:

```
HEADER_SWITCH CLAMP
```

Usage Guidelines:

This parameter is applicable only for technology and standard cell PGV generation (`-celltype techonly | stdcells`).

```
-esd_device_list {device1 device2 .... deviceN}
```

Default: None

Specifies the list of ESD devices. The ESD device information is used for accurate ESD analysis as it takes into account the internal grid from the ports to the ESD devices. The followings inputs are required for ESD tagging during macro PGV generation:

- an xDSPF file
- ESD device list

Usage Guidelines:

This parameter is applicable only for macro PGV generation (`-celltype macros`).

```
-esd_parameters_file filename
```


Default: None

Specifies resistance information for the following types of ESD cells during ESD cell characterization:

- Diodes
- RC Clamp

When this parameter is specified, the software uses these electrical parameters from the generated PGV for ESD analysis.

File Format:

```
Cell <cell name> <Ron> <Roff> <power_pin> <ground_pin> <voltage>  
Type <esd cell type> <Ron> <Roff> <power_pin> <ground_pin> <voltage>
```

Here,

- `Cell` and `Type` are keywords
- `<cell name>` specifies the ESD cell name
- `<esd cell type>` specifies the ESD cell type, such as DIODE and CLAMP
- `RON` is the resistance of the complete cell when it is switched on.
- `ROFF` is the resistance of the complete cell when it is switched off.
- `power_pin` is the name of the power pin connected to the power net
- `ground_pin` is the name of the ground pin connected to the ground net
- `voltage` is the threshold value between the power and ground pin.
- When you specify the cell name, the `RON` and `ROFF` would be applied only for that cell, and when you specify the cell type, the `RON` and `ROFF` would be applied for all the ESD cells of the particular type.

The following is a snippet of the input file:

```
Cell HEADER_SWITCH 10 5 pwr_pin gnd_pin 0.4
```

Usage Guidelines:

This parameter is applicable only for technology and standard cell PGV generation (`-celltype techonly | stdcells`).

```
-esd_pin_list {pin1 pin2 ....pinN}
```

Default: None

Specifies the list of ESD pins. This parameter tags the user-specified pin as the ESD pin before running the `analyze_esd` command, and ensures that the software traces only this ESD power pin to bump and extracts resistance. The ESD cell pins that are tagged using this parameter are listed in the `.report` file.

Usage Guidelines:

This parameter is applicable only for technology and standard cell PGV generation (`-celltype techonly | stdcells`).

```
-exclude_tap_region_file filename
```

Default: None

Specifies to exclude certain tap points during macro PGV generation. This parameter helps to avoid false IR-drop violations inside macros caused by inadvertently assigning evenly-distributed tap current to certain undesired tie high/low paths or shielding paths at certain via layers. The Spice netlist and models are not mandatory parameters for excluding the tap points in the macro PGV generation flow.

File Format:

```
REGION X1 Y1 X2 Y2 <netname1>
REGION X3 Y3 X4 Y4 <netname2>
```

where, `REGION` keyword specifies that the coordinates following it is a tap current region. The unit for coordinates is micron. `<netname>` is optional, and allows you to specify the exclude region for each net. If you do not specify `<netname>`, the tap points in the specified rectangular region are excluded for all power and ground nets.

Usage Guidelines:

This parameter is applicable only for macro PGV generation (`-celltype macros`).

```
-extraction_command_file file
```

Default: None

Specifies the file name of an extraction command file.

Usage Guidelines:

This parameter is applicable only for macro PGV generation (`-celltype macros`).

```
-followpins_tap_layer {lowest_lef_pin_layer | highest_lef_pin_layer |  
all_lef_pin_layers}
```

Default: all_lef_pin_layers

Specifies the LEF metal layer used to generate current taps. The possible arguments are:

- **lowest_lef_pin_layer:** The tool generates current taps only on the lowest LEF metal layer. For example, if both METAL1 and METAL2 metal layers exist for the power/ground pin in the cell LEF, the tool uses METAL1 layer as the tap layer.
- **highest_lef_pin_layer:** The tool generates current taps only on the highest LEF metal layer. For example, if both METAL1 and METAL2 metal layers exist for the power/ground pin in the cell LEF, the tool uses METAL2 layer as the tap layer.
- **all_lef_pin_layers:** The tool generates current taps on all LEF metal layers. For example, if both METAL1 and METAL2 metal layers exist for the power/ground pin in the cell LEF, the tool uses both METAL1 and METAL2 layers as the tap layer.

```
-followpins_interface_node_layer {lowest_lef_pin_layer | highest_lef_pin_layer |  
all_lef_pin_layers}
```

Default: all_lef_pin_layers

Specifies the LEF metal layer used to generate interface nodes. The possible arguments are:

- **lowest_lef_pin_layer:** The tool generates interface nodes only on the lowest LEF metal layer. For example, if both METAL1 and METAL2 metal layers exist for the power/ground pin in the cell LEF, the tool will only generate the interface nodes on the METAL1 layer.
- **highest_lef_pin_layer:** The tool generates interface nodes only on the highest LEF metal layer. For example, if both METAL1 and METAL2 metal layers exist for the power/ground pin in the cell LEF, the tool will only generate the interface nodes on the METAL2 layer.
- **all_lef_pin_layers:** The tool generates interface nodes on all LEF metal layers. For example, if both METAL1 and METAL2 metal layers exist for the power/ground pin in the cell LEF, the tool generates the interface nodes on both METAL1 and METAL2 layers.

```
-force_tap_node_distance {true | false}
```

Default: false

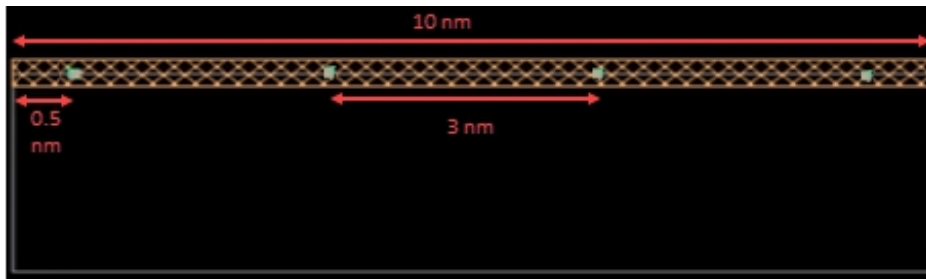
Specifies to create the interface nodes exactly at the user-specified distance. The `-tap_node_distance` option of the `set_advanced_pg_library_mode` command uniformly places the interface nodes so the actual distance between the nodes might be less than the user-specified distance. Therefore, you can set the `-force_tap_node_distance` option to `true` to have maximum specified distance between the nodes.

Example:

If for the LEF segment of length 10 nm, the maximum allowed distance is 3nm. So, the maximum tap integer is given by $10\text{nm}/3\text{nm} = 3$

Since the distance between interface nodes is fixed as 3nm, so the total distance used by interface nodes is $3 \times 3\text{nm} = 9\text{nm}$

The side-edge distance is $(\text{Total length} - \text{Total distance used by interface nodes})/2 = 1/2 = 0.5\text{nm}$



```
-generate_bulk_pin_cap_separately {true | false}
```

Default: `false`

Specifies to generate separate capacitance values for body bias pins in the library report. This parameter allows you to check the bulk pin capacitance based on the Spice simulation. When set to `false`, the software would sum up the capacitance values for the body bias pins and power/ground pins.

The following is a snippet of the library report:

```
cell_A
      VPP(0.9350)      2.3e-16      EARLY(1 taps) EM(1 taps) IR(1
taps)
      VBB(0.0000)      2.3e-16      EARLY(1 taps) EM(1 taps) IR(1
taps)
      VDD(0.9350)      8.1e-15      EARLY(1 taps) EM(1 taps) IR(1
taps)
      VSS(0.0000)      8.1e-15      EARLY(1 taps) EM(1 taps) IR(1
taps)
Cell Type = STDCELL                      Cell_Status: PASS
```

Here, the blue-highlighted text shows the capacitance values for the `VPP` and `VBB` body bias pins.

Usage Guidelines:

This parameter is applicable only for the standard cell PGV generation (`-celltype stdcells`).

`-generate_graybox_data {true | false}`

Default: `true`

Controls the generation of gray box data and storage of this data in the cell library. It is recommended to set this parameter to `false` to reduce the size of the PGV significantly (from 10MB to 1MB) without loss of accuracy.

Usage Guidelines:

This parameter is applicable only for macro PGV generation (`-celltype macros`).

`-generate_itfnodex_direction {true | false}`

Default: <code>true</code> When the LEF port geometry is bigger than the tap node distance (<code>set_advanced_pg_library_mode -tap_node_distance</code>), multiple interface nodes are generated. You can specify this parameter to generate the interface nodes along the X direction. When set to <code>false</code>, interface nodes are not generated along the specified direction.	
<code>-generate_itfnodes_x_direction {true false}</code>	
Default: <code>true</code> When the LEF port geometry is bigger than the tap node distance (<code>set_advanced_pg_library_mode -tap_node_distance</code>), multiple interface nodes are generated. You can specify this parameter to generate the interface nodes along the Y direction. When set to <code>false</code>, interface nodes are not generated along the specified direction.	
<code>-generate_itfnodes_y_direction {true false}</code>	
Default: <code>false</code> Specifies to generate interface nodes at via locations in scenarios where the tap current source is a VIA layer.	
<code>-generate_itfnodes_at_via_layer {true false}</code>	
Default: <code>false</code> Specifies to apply the intrinsic capacitance generated during library generation to all the nets in the datasheet-based current characterization flow. This parameter will override the value of capacitance specified using <code>DATASHEET_PARAMETER</code> in the trigger file.	
<code>-generate_pin_intrinsic_cap {true false}</code>	
Default: <code>false</code> Specifies to apply the intrinsic capacitance generated during library generation to all the nets in the datasheet-based current characterization flow. This parameter will override the value of capacitance specified using <code>DATASHEET_PARAMETER</code> in the trigger file.	
<code>-generate_pin_intrinsic_cap_list {netname1 netname2...}</code>	

Specifies the list of nets for which you want to apply the generated intrinsic capacitance in the datasheet-based current characterization flow. This parameter will override the value of capacitance specified using `DATASHEET_PARAMETER` in the trigger file.

Example:

```
set_advanced_pg_library_mode -generate_pin_intrinsic_cap_list {VDDA VDD1
...}
```

```
-ignore_pg_nets {{cellname netname}+}
```

Default: None

Skips the specified power or ground nets in PGV generation.

```
-input_port_value_list {pin1 value1 pin2 value2 ....pinN valueN}
```

Default: None

Specifies the voltage conditions of the signal inputs during power-grid library characterization. A typical usage model of this parameter include setting the SET or RESET signal port to be the same voltage as supply or ground (tie high/low) to obtain the correct model of operation.

The following example illustrates the use model:

```
set_advanced_pg_library_mode -input_port_value_list { RESETVDD 3.3 }
```

Usage Guidelines:

This parameter is applicable only for standard cell PGV generation (`-celltype stdcells`).

```
-interface_node_location_file filename
```

Default: None

Specifies a file containing the location of interface nodes where tap currents are to be generated. The specified file includes the cell name, net name, layer name, and location of the nodes. This parameter allows to generate current taps at user-specified locations during PGV generation.

File Format:

```
CELL <cell_name>
NET <net_name>
LAYER <layer_name> (x1,y1), (x2,y2)
```

Example:

```
CELL cellA
NET VDD
LAYER M0 (0,0.15), (0.585,0.177), (0.2,0.15), (0.3,0.177)

NET VSS
LAYER M0 (0.485,0.019)

CELL cellB
NET VDD
LAYER M0 (0.49,0.008), (0.24,0.008)
```

Usage Guidelines:

This parameter is applicable only for technology and standard cell PGV generation (-celltype techonly | stdcells).

```
-lef_layer_ignore_list {lef_layername+}
```

Default: None

Specifies the names of the LEF layers that power-grid library generation is to ignore when it creates the cell library.

```
-lef_layer_ignore_list_file filename
```


Default: None

Specifies the name of a file containing a list of the LEF layers that power-grid library generation is to ignore when it creates the cell library. Each line in the file must consist of a single LEF layer name.

File Format:

```
layer1
layer2
...
layern
```

```
-lef_pin_short {true | false}
```

Default: false

Enables shorting of the unconnected LEF pins and ports to establish electrical connectivity. When set to `true`, shorts the interface nodes for all the cells that are being processed.

```
-lef_pin_short_cell_file filename
```

Default: None

Specifies the name of a file containing a list of cells and the net name, metal layer name, and resistance value for each layer. It is used to short the LEF pins and ports by the user-specified resistance value. When this parameter is specified, only the given list of nets are shorted with the user-defined resistance values. If a resistance value is not specified for a layer, the default resistance value of 0.001 is used.

File Format:

```
CELL <cell_name>
<net_name> <layer_name> <resistance_value>
```

Examples:

```
CELL CellA
VDD M1 15
VSS M1 10

CELL CellB
VSS M1 12
VDD M1 12
```

```
-lef_pin_short_cell_list {cell1 cell2 ...}
```

	Default: None Specifies a list of cells for which the the LEF pins and ports are shorted by the default resistance value of 0.001ohm. When this parameter is specified, all the nets of the given cells are shorted using the resistance value of 0.001ohm.												
<code>-libgen_command_file file</code>													
	Default: None Specifies the file name of a power-grid library generation command file.												
<code>-macro_parasitic_file filename</code>													
	Default: None Specifies the name of the SPEF file that provides signal net parasitics while creating a detailed dynamic view in Library Characterization. The SPEF should be extracted in the decouple_rc mode. Note: A SPEF file is required only when you specify your own netlist in the Netlist-based flow. The SPEF file is a requirement when the netlist does not have signal caps. Usage Guidelines: This parameter is applicable only for macro PGV generation (<code>-celltype macros</code>).												
<code>-marker_layermap filename</code>													
	Default: None Specifies the GDS layer mapping file containing the list of marker GDS layers that are used for layer operations during XTC processing. Marker layers are layers which do not have corresponding tech layers. This parameter is used to support extraction of Metal-Insulator-Metal Capacitor (MIMCAP) geometries during power-grid library characterization. File Format: <table><tr><th><u>Layer Type</u></th><th><u>marker layername</u></th><th><u>GDS keyword</u></th><th><u>gds layer number</u></th></tr><tr><td>metal</td><td>CTM_O</td><td>gds</td><td>77 1</td></tr><tr><td>metal</td><td>CBM_O</td><td>gds</td><td>88 1</td></tr></table> Usage Guidelines: This parameter is applicable only for macro PGV generation (<code>-celltype macros</code>).	<u>Layer Type</u>	<u>marker layername</u>	<u>GDS keyword</u>	<u>gds layer number</u>	metal	CTM_O	gds	77 1	metal	CBM_O	gds	88 1
<u>Layer Type</u>	<u>marker layername</u>	<u>GDS keyword</u>	<u>gds layer number</u>										
metal	CTM_O	gds	77 1										
metal	CBM_O	gds	88 1										
<code>-mcpu_rerun_count count</code>													

Specifies to rerun the cells that failed due to machine issues during PGV library generation. This parameter allows you to specify the number of times the failed cells can be resubmitted in the distributed processing mode.

Default: 0

`-pg_arc_file filename`

Specifies the name of the user-defined PG arc file that contains the power-ground data to generate the power-ground correspondence view in PGV. This correspondence view is reported in the `PGV.summary` report.

File Format:

```
CELL cellname
{Power_pin_name1} {gnd_pin_name1}
{Power_pin_name2} {gnd_pin_name2}
...
CELL cellname2
```

Example:

The following is an example of the power-ground correspondence view that gets generated from the user-specified PG arc file:

User-specified PG arc file:

```
CELL      cell1
{VDDA}      {VSSA}

{VDDA1 VDD}  {VSS}
```

Power-ground correspondence view:

Pin Name	Correspondence Name
VDDA	VSSA
VDDA1	VSS
VDD	VSS
VSSA	VDDA
VSS	VDD
VSS	VDDA1

	<code>-pgv_path path</code>
	Specifies the path of the cell library from which the electrical data will be used to create a new PGV.
	<code>-powergate_characterization_voltages { val1 val2 val3 }</code>
	Default: None Supports characterization of power gates at multiple voltages. You can specify multiple values of voltages so that the library characterization engine will characterize the power gates at each of the voltages. The software will then be able to get the characterized cell from the cell library for a particular voltage. Usage Guidelines: This parameter is applicable only for standard cell PGV generation (<code>-celltype stdcells</code>).
	<code>-powergate_multi_enable_characterization_method {enable_all enable_single}</code>

Default: `enable_all`

Specifies the Ron/I_{dsat} characterization method for power gate cells with multi-enable pins. This parameter allows you to control the generation of RON and Id-V_d curve for each enable pin. The possible characterization methods are:

- `enable_all`: All the enable pins are switched ON for Ron/I_{dsat} characterization. The generated PGV model will contain a single Ron resistor with the total equivalent resistance value. The cell library report displays the equivalent Ron value of all the enable pins. This is the default method.
- `enable_single`: Only one enable pin is switched ON at a time for Ron/I_{dsat} characterization, and the other pins are switched OFF. The generated PGV model will contain total number of Ron resistors that is equal to the number of enable pins. The cell library report displays Ron values separately for each enable pin.

This parameter is supported only for standard cells when the power gate parameters are specified using the `-powergate_parameters_file` option. For multi-enable power gate cells, I_{leak}/R_{off} are always calculated by keeping all the enable pins OFF.

Examples:

```
set_pg_library_mode ... -powergate_parameters_file ./pgl.txt
set_advanced_pg_library_mode -powergate_multi_enable_characterization_method
enable_all
```

Usage Guidelines:

This parameter is applicable only for standard cell PGV generation (`-celltype stdcells`) when the power gate parameters are specified using the `-powergate_parameters_file` parameter.

`-process_bulk_diffusion_ports {true|false}`

Default: `false`

Specifies to create the `DIFF` layer ports of body bias pins in PGV.

By default, library generation ignores ports on the diffusion layer. In order to process body bias pin's ports on the diffusion layer for electrical analysis on body bias network, set this parameter to `true` during library generation.

`-remove_emview_dangling_resistor {true|false}`

Default: <code>false</code>	Specifies to restore or remove dangling resistors from power-grid views. It is recommended to minimize dangling resistors for optimal analysis of IR drops and accurate Blech length calculation at the top level in EM analysis.
<code>-retain_generated_pgv_on_error {true false}</code>	
Default: <code>false</code>	Specifies to retain an incomplete cell library created during PGV generation even if the tool exits with a fatal error. When set to <code>false</code> , the tool deletes the incomplete cell library. This parameter is recommended for use during initial library creation and debugging.
<code>-reuse_electrical_data {true false}</code>	
Default: <code>false</code>	Specifies to reuse the electrical data from the old PGV and only process the physical part in the new PGV.
<code>-ron_measure_threshold value</code>	
Default: <code>0.99</code>	Specifies the switched pin voltage during Ron simulation. The specified value will be multiplied by the VDD voltage (<code><value>*vdd_vol</code>) and applied to the switched pin during Ron simulation. You can specify a value between 0 and 1.
Usage Guidelines:	
This parameter is applicable only for standard cell PGV generation (<code>-celltype stdcells</code>).	
<code>-schematic {true false}</code>	
Default: <code>false</code>	Specifies whether the subcircuit netlist is a schematic netlist.
<code>-skip_switch_net_extraction {true false}</code>	

Default: `false`

Specifies to skip extraction of switched nets inside the GDS file for a fine-grain memory cell. The software will add the capacitance from the switched net to the always-on net. The software will thus consider the cell as a regular memory without switched/power-gated nets. When this parameter is specified, you do not need to specify the port file (`set_advanced_pg_library_mode -add_port_labels`) as the switched nets are not traced. The final PGV will be created with the always-on and VSS nets.

The power-gate library report file will contain information about the missing switched net, and the capacitance and current that was contributed by the switched net.

Usage Guidelines:

This parameter is applicable only for macro PGV generation (`-celltype macros`).

`-source_location_file {filename}`

Default: None

Specifies the name of the file containing the locations of the voltage sources to be connected to the pad cells.

File Format:

For more information on the source location file, see [Source Location File Format](#).

Usage Guidelines:

This parameter is applicable only for macro PGV generation (`-celltype macros`).

`-spectre_path value`

Default: None

Specifies an external Spectre path for PGV generation. This parameter allows you to use a different Spectre version than the one embedded with the software.

Note: To specify an external Spectre path for PGV generation, the `-use_embedded_spectre` option should be set to `false`.

Usage Guidelines:

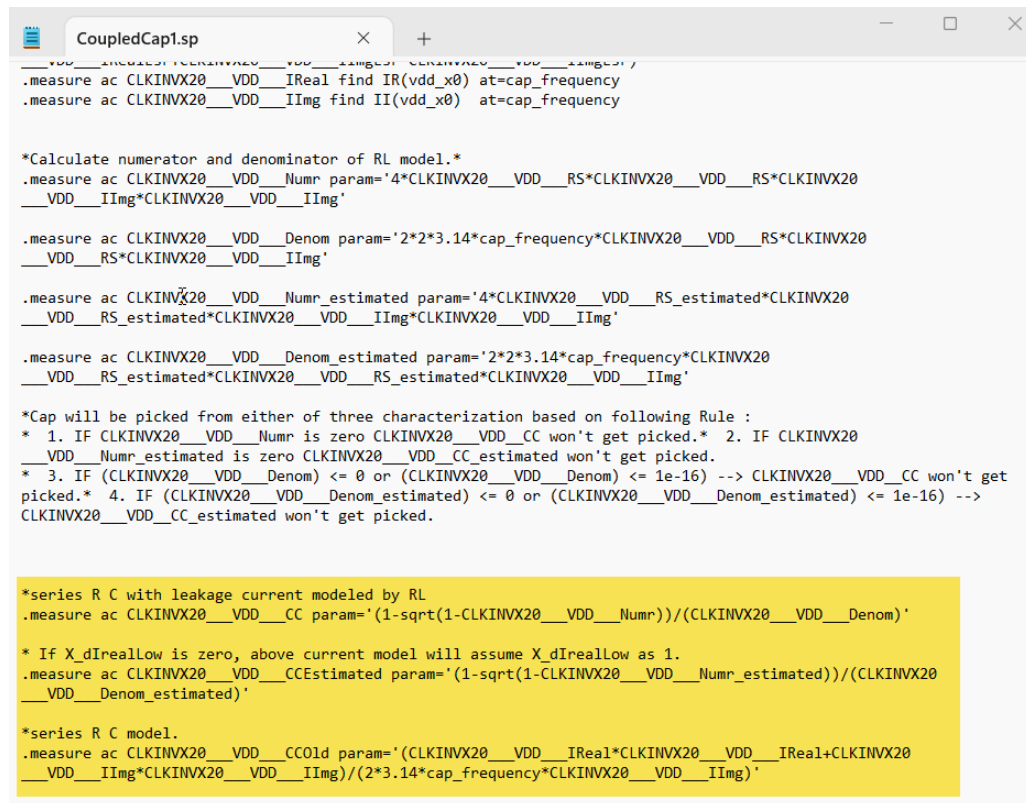
This parameter is applicable only for standard cell and macro PGV generation (`-celltype stdcells|macros`).

`-spectre_standalone_simulation {true | false}`

Default: false

Specifies to include computation equations (that is, `.measure` statements) in Spectre setup file for the verification of power-gate parameters (Idsat, I leakage, and Ron) and coupled capacitance. You can use this setup file directly in Spectre to understand and verify the underlying computational processes.

The table below displays a snippet of the computation equations extracted from the `.sp` file:



```

CoupledCap1.sp
...
.measure ac CLKINX20__VDD__IReal find IR(vdd_x0) at=cap_frequency
.measure ac CLKINX20__VDD__IImg find II(vdd_x0) at=cap_frequency

*Calculate numerator and denominator of RL model.*
.measure ac CLKINX20__VDD__Nnumr param='4*CLKINX20__VDD__RS*CLKINX20__VDD__RS*CLKINX20__VDD__IImg*CLKINX20__VDD__IImg'

.measure ac CLKINX20__VDD__Denom param='2*2*3.14*cap_frequency*CLKINX20__VDD__RS*CLKINX20__VDD__RS*CLKINX20__VDD__IImg'

.measure ac CLKINX20__VDD__Nnumr_estimated param='4*CLKINX20__VDD__RS_estimated*CLKINX20__VDD__RS_estimated*CLKINX20__VDD__IImg*CLKINX20__VDD__IImg'

.measure ac CLKINX20__VDD__Denom_estimated param='2*2*3.14*cap_frequency*CLKINX20__VDD__RS_estimated*CLKINX20__VDD__RS_estimated*CLKINX20__VDD__IImg'

*Cap will be picked from either of three characterization based on following Rule :
* 1. IF CLKINX20__VDD__Nnumr is zero CLKINX20__VDD__CC won't get picked.* 2. IF CLKINX20__VDD__Nnumr_estimated is zero CLKINX20__VDD__CC_estimated won't get picked.
* 3. IF (CLKINX20__VDD__Denom) <= 0 or (CLKINX20__VDD__Denom) <= 1e-16 --> CLKINX20__VDD__CC won't get picked.* 4. IF (CLKINX20__VDD__Denom_estimated) <= 0 or (CLKINX20__VDD__Denom_estimated) <= 1e-16 --> CLKINX20__VDD__CC_estimated won't get picked.

*series R C with leakage current modeled by RL
.measure ac CLKINX20__VDD__CC param='(1-sqrt(1-CLKINX20__VDD__Nnumr))/(CLKINX20__VDD__Denom)'

* If X_dIrealLow is zero, above current model will assume X_dIrealLow as 1.
.measure ac CLKINX20__VDD__CCEstimated param='(1-sqrt(1-CLKINX20__VDD__Nnumr_estimated))/(CLKINX20__VDD__Denom_estimated)'

*series R C model.
.measure ac CLKINX20__VDD__CC01d param='(CLKINX20__VDD__IReal*CLKINX20__VDD__IReal+CLKINX20__VDD__IImg*CLKINX20__VDD__IImg)/(2*3.14*cap_frequency*CLKINX20__VDD__IImg)'

```

This parameter is applicable only for standard cell PGV generation (`-celltype stdcells`).

`-stdcell_characterization_voltage_value {val1 val2 val3 ...}`

Default: None

Specifies capacitance characterization voltages when generating standard cell PGVs containing voltage-dependent capacitance table for the specified cell. This parameter allows you to compute capacitance based on the user-defined voltage values. The `-stdcell_characterization_voltage_value` parameter must be specified with the `set_pg_library_mode -enable_multi_voltage_cap_generation true` parameter.

If this parameter is not specified with `-enable_multi_voltage_cap_generation`, capacitance is calculated based on the software-derived voltage range.

When the `-stdcell_characterization_voltage_value` parameter is specified, the library generation engine prints the capacitance values and corresponding voltages in the report file, and the rail analysis engine prints the capacitance used for each instance.

Examples:

```
set_pg_library_mode -ground_pins VSS -power_pins {VDD 0.9 TVDD 0.9 } -
celltype stdcells \
...
-enable_multi_voltage_cap_generation true
set_advanced_pg_library_mode -stdcell_characterization_voltage_value {1.0
1.2 1.5 2.0 3.3}
```

Usage Guidelines:

This parameter is applicable only for standard cell PGV generation (`-celltype stdcells`).

`-strict_input_check {true|false}`

Default: false

Specifies that the software should not create any view for the cell if the EM view is not generated. When set to `true`, the software displays an error message if the Spice netlist or GDS is not specified, and gives details about the cell name and the reason for not generating the view.

Usage Guidelines:

This parameter is applicable only for macro PGV generation (`-celltype macros`).

`-tap_node_distance value`

Default: 50 microns	Specifies to set the maximum distance between interface nodes when they get generated in power-grid library generation. Library generation creates interface nodes on port geometries in LEF and Extraction, uses them for top level connectivity tracing and also to connect the cell/block onto top level routing. The interface node generation is purely based on port geometries (tap points are placed one per port). For ports greater than <code>-tap_node_distance</code>, they are placed at <code>-tap_node_distance</code> apart. If the port pin geometry in LEF is bigger than the setting of the <code>-tap_node_distance</code> parameter, at least two nodes will get generated per port geometry.
<code>-techgen_dir directory</code>	
Default: None	Specifies the techgen directory.
<code>-thunder_command_file file</code>	
Default: None	Specifies the file name of a thunder command file.
Usage Guidelines:	This parameter is applicable only for standard cell and macro PGV generation (<code>-celltype stdcells macros</code>).
<code>-trim_metal_layer_map file</code>	
Specifies the name of the file containing the mapping between a metal layer and its corresponding trim layer. When this parameter is specified, the trim layer shapes are automatically removed from the routing layer shapes to correctly extract all the nets in the generated PGV.	
File Format:	<code>metal_layer_name trim_layer_name</code> <pre>M0 CM0 M1 CM1 M2 CM2</pre>
<code>-use_embedded_spectre {true false}</code>	

Default: <code>true</code>	When set to <code>false</code>, allows you to specify a different Spectre version than the one embedded with the software. Library Generation will automatically pick Spectre from users' path. Usage Guidelines: This parameter is applicable only for standard cell and macro PGV generation (<code>-celltype stdcells macros</code>).
<code>-use_peak_current_distribution {true false}</code>	
Default: <code>false</code>	Specifies to use the maximum currents of devices in the Spice netlist for current distribution in macros. When set to <code>true</code>, enables peak current distribution for faster PGV generation. The default (<code>false</code>) behavior is to enable activity propagation based current distribution.
<code>-use_physical_view_from_techpgv {true false}</code>	
Default: <code>false</code>	Specifies to change the tap node distance of LEF based PGVs by only generating a new tech PGV, without regenerating the entire PGV. You can pass the new generated library with the previously generated PGVs during rail analysis, so that the updated tech PGV is used for tap node distance. The electrical parameters (currents, capacitances) are considered from original PGVs. Usage Guidelines: This parameter is applicable only for standard cell and macro PGV generation (<code>-celltype stdcells macros</code>).
<code>-verbosity {true false}</code>	
Default: <code>false</code>	Controls the amount of information displayed in the console and written to the log files during power-grid library generation. You can use this parameter for detailed troubleshooting and controlling the verbosity level in the log file.
<code>-via_based_current_config_file filename</code>	

Specifies the name of the configuration file containing tap current details for location-based via current generation. If `-enable_via_based_current_distribution` is set to `true`, this parameter must be specified.

Configuration File Format:

```
#-----
CURRENT_UNIT <default_value>
INPUT_SLEW <default_value>
OUTPUT_SLEW <default_value>
MAX_DISTANCE <Threshold distance for snapping via tap if the exact xy
location is not found as specified>
DELAY <default_value>
PERIOD <default_value>
INTRINSIC_CAP { power_net cap ground_net cap}
#-----
ID      PGPIN_NAME      X      Y      AVG_CURRENT      PEAK_CURRENT
#-----
```

Configuration File Example

```
#-----
CURRENT_UNIT mA
INPUT_SLEW 20ps
OUTPUT_SLEW 10ps
MAX_DISTANCE 2.5
DELAY 5ps
INTRINSIC_CAP { VDD 40nF VSS 5nF }
#-----
ID      PGPIN_NAME      X      Y      AVG_CURRENT      PEAK_CURRENT
#-----
PVDD_1  VDD      16.3090  7.6470  50.0373  100.037
PVDD_2  VDD      16.6900  7.6470  50.0373  100.037
PVDD_3  VDD      17.0700  7.6470  50.0373  100.037
...
```

When this feature is enabled, the tool by default generates a report “dropped_via_location_taps.txt” in the output directory to give a list of via IDs that were dropped because they were not within the snapping threshold limit. By default, the snapping threshold is zero. It is controlled by “MAX_DISTANCE” in the configuration file.

`-well_cap_file filename`

	Default: None Specifies the name of a file containing a list of cells and the well capacitance value for each. File Format: <pre>UNIT unit_in_farads CELL cell_name cap_value</pre> For more information on the format, refer to the "Cell Well Cap File" section in the "File Formats" chapter of <i>Voltus User Guide</i>.
	<pre>-xtc_command_file filename</pre>
	Default: None Specifies the name of the XTC command file. Usage Guidelines: This parameter is applicable only for macro PGV generation (<code>-celltype macros</code>).
	<pre>-xtc_include_file_for_qdv filename</pre>
	Default: None Specifies the file name of an XTC include file used during characterization. For example, you can include XTC commands to ignore GDS cells, to blackbox lower-level hierarchy cells, or to perform some basic GDS layer operations. Usage Guidelines: This parameter is applicable only for macro PGV generation (<code>-celltype macros</code>).

Examples

- The following command creates a power-grid library:

```
set_advanced_pg_library_mode \  
-libgen_command_file library_inc.cmd  
-thunder_command_file simulator.cmd
```

- In the following example, power-grid library generation ignores the LEF *metal1* and *metal3* layers when it creates the cell library:

```
set_advanced_pg_library_mode -lef_layer_ignore_list {metal1 metal3}
```

- The following line indicates that a file named *nolef.txt* contains a list of the LEF layer names that power-grid library generation should ignore:

```
set_advanced_pg_library_mode -lef_layer_ignore_list_file {nolef.txt}
```

- The following commands tags the VDD and TVDD pins as the ESD pins:

```
set_advanced_pg_library_mode -esd_cells {IOESD} -esd_pin_list {VDD TVDD}
```

- The following example specifies that FOREIGN statements are to be assumed for cells with no FOREIGN statements and that the FOREIGN cells should be placed at the inverse of the origin:

```
-assume_foreigns true  
-assume_foreigns_mode 1
```

- The following example specifies that FOREIGN statements are to be assumed for cells with no FOREIGN statements, and the FOREIGN cells should be placed at (0,0):

```
-assume_foreigns true  
-assume_foreigns_mode 0
```

- The following example enables the reuse of electrical data from an old PGV to create a new PGV:

```
set_advanced_pg_library_mode -pgv_path <path> -reuse_electrical_data true
```

Source Location File Format

The voltage source location file format is as follows:

```
<MIN_SPACING spacing_value>  
<DENSITY density_value>  
CELL cell_name
```

```
<[PORT|DETAILED] {>
NET net_name
<[PORT|DETAILED] {>
lef_layer_name <PORT|DETAILED> x y [x y]
lef_layer_name <PORT|DETAILED> x y [x y]
<>
<>
```

This source location file syntax includes the following parameters:

- **NET net_name**
These specifications apply to the voltage source locations that follow until Library Generation encounters a new CELL or NET line. You can specify voltage sources as single points or as rectangles. If you specify them as rectangles, Library Generation generates a grid of voltage sources inside these rectangles.
- **PORT | DETAILED**
The source location file can specify voltage sources to be used with port or detailed views.

```
lef_layer_name <PORT|DETAILED> x y [x y]
```

Lines beginning with a LEF layer name specify the actual location of the voltage sources.

You can specify this location as a single point by providing one set of values for x and y. These x and y coordinates are specified in microns and use the same coordinate system as that used for LEF ports in the LEF file.

Alternatively, you can specify the voltage source locations as a rectangle by giving two sets of x and y locations. In this case, Library Generation attempts to generate a 10-by-10 grid of voltage sources inside this area. From a rectangle specification, it does not create voltage sources closer than 1 micron apart, so if the rectangle is less than about 10 microns on a side, it generates fewer voltage sources.

In the source location file, the location of the voltage sources is specified by using a LEF layer name and location in LEF coordinate space. However, because of the way LibGen uses this data internally, no LEF geometries are actually required to exist at the location where the voltage sources are to be placed. Library Generation uses this data internally with the layout of the block (that is, the GDS data) during detailed power-grid view generation, so it only requires that there be layout (GDS) geometries at the location of the voltage sources. Similarly, the LEF layer name used in the source location file does not actually have to be included in any of the input LEF files; it must be defined only in the layer map used by Library Generation.

This is an example of the contents of the voltage source location file:

```
CELL BUMP
NET vcc
METAL1 11.0 16.0
METAL1 0.0 15.0 22.1 17.5
```

Here is an example file using MIN_SPACING and DENSITY to create a 10 x 10 grid of voltage sources on the VSS net in the detailed view of the NAND1 cell:

```
DENSITY 10  
MIN_SPACING 1  
CELL BUMP  
DETAILED  
NET VSS
```

Related Topics

- ["Power-Grid Library Generation"](#) in the *Voltus User Guide*
- [File Formats](#)

set_pg_library_mode

```
set_pg_library_mode
[-help]
[-add_ploc_cells cell1 cell2 ...]
[-bulk_ground_pins { ground_pin_list }]
[-bulk_power_pins { pin1 voltage1 ... pinN voltageN }]
[-calculate_metal_layer_density {true | false}]
[-cell_decap_file filename ]
[-cell_list_file filename ]
[-cell_ploc_details filename]
-celltype { techonly | stdcells | macros }
[-current_distribution {propagation | dynamic_simulation trigger_file | current_region
region_file}]
[-custom_label label]
[-decap_cells cell_name_list]
[-default_area_cap value]
[-default_power_voltage value]
[-design_qrc_layermap file]
[-enable_device_size_current_distribution {true | false}]
[-enable_distributed_processing {true|false} ]
[-enable_multi_voltage_cap_generation {true | false}]
[-exclude_cell_list_file filename]
[-extraction_tech_file file]
[-filler_cells cell_name_list ]
[-fine_grain_memory_cell {true | false}]
[-gds_files file_list]
[-gds_layermap file ]
[-generate_port_using_gds mbb_layer_name ]
[-ground_pins { ground_pin_list }]
[-ignore_gds_cells {gds_cell_name_list}]
[-internal_ground_pins ground_pin_list]
[-internal_power_pins power_pin_list]
[-layer_tap_node_distance { {LayerName1tap_node_distance} {LayerName2tap_node_distance}
... }]
[-layers_to_be_ignored_during_tap_creation {layername+}]
[-layers_to_have_ports_created {layername+}]
[-lef_layermap file ]
[-liberty_files file_list]
[-macros_config_file filename]
[-max_metal_density_tile_dimension {maxlengthmaxwidth}]
```

```
[ -non_zero_ground_pins { pin1voltage1 ... pinN voltageN } ]
[ -oasis_files file_list ]
[ -oasis_layermap file ]
[ -pin_shape_current_distribution filename ]
[ -power_pins { pin1voltage1 ... pinN voltageN } ]
[ -powergate_finegrain_simulation {true | false} ]
[ -powergate_parameters {{ cell supply switchedRon Idsat Ileak enablePinName
Control_Pin_Logic} + } ]
[ -powergate_parameters_file filename ]
[ -spice_cornerscorner_list ]
[ -spice_modelsfile_list ]
[ -spice_netlist_list_file filename ]
[ -spice_subckts file_list ]
[ -spice_subckts_xyscaling value ]
[ -stop@via layername ]
[ -stop@via_file filename ]
[ -tap_node_distance_file filename ]
[ -tech_gds_layermap filename ]
[ -tech_oasis_layermap filename ]
[ -temperature temp_value_in_degree_celcius ]
[ -tsv_subckt_modelfile_list filename ]
[ -use_lefonly_for_detailedview_generation {true | false} ]
[ -use_powergate_data_from_lib {true|false}]
```

Specifies what kind of power-grid library will be created. This command is used to define the type of input and configuration for generating power-grid libraries. You must specify this command before running the `generate_pg_library` command.

Parameters

-help	
	Outputs the command usage.
-add_ploc_cells cell1 cell2 ...	

Default: None	Specifies the name of the low-dropout (LDO) cell for which a voltage source is to be added. You can specify a list of cells. This parameter is applicable to the LDO cells only, and must be used when creating voltage sources for these cells. When this parameter is specified, all the output pins will be made voltage sources using the highest voltage value of the cell. The <code>-add_ploc_cells</code> parameter is used with the <code>-cell_ploc_details</code> parameter.
<code>-bulk_ground_pins { ground_pin_list }</code>	
Default: None	Specifies the names of nets that connect transistor bulk terminals to the ground net. This information is used to prevent any current from being assigned to these nets. If a net is connected to any transistor terminals other than a bulk terminal, you must specify that net with the <code>-ground_pins</code> parameter.
<code>-bulk_power_pins { pin1 voltage1 ... pinN voltageN }</code>	
Default: None	Specifies the names of nets that connect transistor bulk terminals to the power net. This information is used to prevent any current from being assigned to these nets. If a net is connected to any transistor terminals other than a bulk terminal, you must specify that net with the <code>-power_pins</code> parameter.
<code>-calculate_metal_layer_density {true false}</code>	
Default: <code>false</code>	Enables metal layer density calculation for macros. When this parameter is set to <code>true</code>, Library Generation calculates and stores the area density information for each metal layer in macros for accurate thermal analysis. Metal density for macros is calculated by dividing the macro cell into tiles, and computing the area of polygons in each tile. The calculated metal density values are used in the Power Analysis flow to generate the metal power density map. This power map file is later used as input by Celsius Thermal Solver, the thermal analysis tool for packages and boards. Usage Guidelines: This parameter is applicable only for macro PGM generation (<code>-celltype macros</code>).
<code>-cell_decap_file filename</code>	

Default: None

Specifies the name of a file containing a list of cells and the decoupling capacitance value for each. You can also specify the series resistance value along with the decoupling capacitance value to accurately model the cell decoupling capacitance. The cell decap file format allows you to specify capacitance for multiple voltage values to support multi-voltage capacitance simulation for standard cell and macro PGV generation.

File Format:

```
UNIT <X>
CELL cell_name
PIN pin_name capacitance resistance
```

or

```
UNIT <X>
CELL cell_name
VOLTAGE value1
PIN pin_name capacitance resistance
...
....
```

VOLTAGE *value2*

```
PIN pin_name capacitance resistance
...
...
```

Unit is optional, default is $1e-15$. The *value* for each cell will be multiplied by the unit, except if engineering notation is provided.

Examples:

```
UNIT 1
CELL cell1

VOLTAGE 1.0
PIN VDD 1e-18
PIN VSS 1e-18

VOLTAGE 0.9
PIN VDD 1e-18
PIN VSS 1e-18
```

```
-cell_list_file    filename
```

Default: None

Specifies a file containing a list of cells that will be processed.

The `-cell_list_file` parameter is required when `set_pg_library_mode -celltype macros` is selected. The program spawns the library generation job in serial mode for each macro in the cell list file. A separate library for each macro is generated. This facilitates debugging when library generation of any macro fails. It also helps in improving the overall performance.

File Format:

```
CELL1
CELL2
CELL3
```

```
-celltype { techonly | stdcells | macros }
```

Default: None

(Mandatory) Specifies the type of library creation.

- `techonly` - Creates a technology Library that contains the tech file, area capacitance specification, decap/filler/powergate cells, and Tech view for all the cells. A technology library is a requirement for running rail analysis.
- `stdcells` - Generates a power-grid library for all the standard cells using the Spice netlist. The stdcells power-grid library does not contain the tech file. Power-grid is connected at the LEF pin, and all the capacitance are derived from simulation. The power-grid library contains three kind of power-grid views (PGVs), Early/IR/EM, and the views are selected based on the accuracy mode of rail analysis.
- `macros` - Generates a power-grid library for memory, I/Os and macros using the Spice and GDS netlist. GDS is mandatory for the cell type macros. The `macros` power-grid library does not contain the tech file. Power-grid is connected down to contact/specified via, and all the capacitance are derived from simulation. The power-grid library contains three kind of PGVs, Early/IR/EM, and the views are selected based on the accuracy mode of rail analysis.

```
-cell_ploc_details filename
```

Default: None

Specifies a file that contains all the information required to add a voltage source at the desired location for the power pad and LDO cells. This file includes the pad cell name, pin name, layer name, and domain name.

File Format:

<i>cell_name</i>	<i>pin_name</i>	<i>layer_name</i>	<i>domain_name</i>
PADCELL	VDD	Metal3	VDD

- *pin_name* is the power/ground name for pad cells, and is the output pin name for LDO cells. The pin name could be represented by using the wildcard character "*" to indicate that voltage sources are to be added to all pins of a power pad/LDO cell.
- *domain_name* is the pin name for pad cells. For LDO cells, it is the power or ground name whose voltage needs to be used.

Examples:

```
set_pg_library_mode \
    -ground_pins           {VSS 0 ExtVSS 0} \
    -power_pins            {VDD 1.2 ExtVDD 1.2} \
    -celltype              techonly \
    ...
    -add_ploc_cells        {CELL1 CELL2 CELL3} \
    -cell_ploc_details     /VoltageSource/config.txt \
```

The following is a snippet of the config.txt file:

```
CELL1 VDD Metal3 VDD
CELL1 VSS Metal2 VSS
CELL1 Z Metal2 VDD
CELL2 Z Metal3 VSS
CELL3 * VDD
```

Usage Guidelines:

This parameter is applicable only for all cell types (`-celltype techonly | stdcells | macros`).

```
-current_distribution {propagation | dynamic_simulation trigger_file | current_region region_file}
```

	Default: <code>propagation</code> Specifies the method of current distribution inside the cell. • Propagation uses Voltus library simulator to determine the propagation activity to distribute the current. • Dynamic simulation uses a trigger file to run detailed analysis to determine current distribution. For more information on the format, see Trigger File Format. • Current region specifies the amount of current to distribute within a cell region. For more information on the format, see Current Region File. Usage Guidelines: This parameter is applicable only for macro PGV generation (<code>-celltype macros</code>).
<code>-custom_label label</code>	
	Default: None Saves the specified label in the PGV for easy identification. This label can be used for revision control, label for Spice netlist/model, and internal keywords. The label will be stored as "Custom Label" in VERSION.TXT in the .cl.
<code>-decap_cells cell_name_list</code>	
	Default: None Specifies the names of cells that have explicit decoupling capacitance for use in decap optimization. These cells are nearly equivalent to feedthrough cells, except that there is typically an active device between the power and ground nets to provide extra decoupling capacitance. Power-grid view library generation uses this list of cells to ensure that a capacitance value is associated with each cell. If no capacitance is associated with a cell, a warning message will be issued. This parameter supports wildcards (*).
<code>-default_area_cap value</code>	

Default: 0.0 (unit is fF per micron-square). You must provide a value for area-based decoupling capacitance. Specifies the default amount of area-based decoupling capacitance in a cell. Usage Guidelines: This parameter is applicable only for technology PGV generation (<code>-celltype techonly</code>).												
<code>-default_power_voltage value</code>												
Default: 1.2 Specifies voltage for the power pins which are present in LEF but not specified using the <code>-power_pins</code> parameter. As a result, the <code>-power_pins</code> parameter of the command is optional. The default voltage value specified using <code>-default_power_voltage</code> is added to all power pins of the cell. If both the parameters <code>-power_pins</code> and <code>-default_power_voltage</code> are not specified, the default voltage of 1.2 will be used.												
<code>-design_qrc_layermap file</code>												
Default: None Specifies a file that provides the mapping of the layer names in the LEF design file to the layer names in the Quantus technology file. File Format: <table><thead><tr><th><u>lefdef layername</u></th><th><u>technology layername</u></th></tr></thead><tbody><tr><td>M1</td><td>METAL_1</td></tr><tr><td>VIA1</td><td>VIA_1</td></tr><tr><td>M2</td><td>METAL_2</td></tr><tr><td>VIA2</td><td>VIA_2</td></tr><tr><td>...</td><td></td></tr></tbody></table>	<u>lefdef layername</u>	<u>technology layername</u>	M1	METAL_1	VIA1	VIA_1	M2	METAL_2	VIA2	VIA_2	...	
<u>lefdef layername</u>	<u>technology layername</u>											
M1	METAL_1											
VIA1	VIA_1											
M2	METAL_2											
VIA2	VIA_2											
...												
<code>-enable_device_size_current_distribution {true false}</code>												
Default: false Specifies to compute the tap current distribution for a macro based on device sizes (Width, Length), without performing any simulation. This parameter is used for macro PGV generation in a scenario when the Spice netlist is available but the Spice models are not available.												

<code>-enable_distributed_processing {true false}</code>	
	Default: <code>false</code> Specifies to enable parallel processing for the Library Generation flow.
<code>-enable_multi_voltage_cap_generation {true false}</code>	
	Default: <code>false</code> Specifies to compute capacitance values over a range of input voltages, including the nominal voltage. It generates 10 values from 0.1 to 1.3 times the vdd/vss voltage. When set to <code>true</code>, it generates a PGV that contains the voltage-dependent capacitance table for the specified cell. This parameter allows you to use the accurate cell capacitance at different supply voltages in power-up analysis and in different voltage domains in dynamic analysis. Usage Guidelines: This parameter is applicable only for standard cell PGV generation (<code>-celltype stdcells</code>).
<code>-exclude_cell_list_file filename</code>	
	Default: None Specifies a file containing a list of cells (one per line) which should be excluded from power-grid library generation. Usage Guidelines: This parameter is applicable only for technology and standard cell PGV generation (<code>-celltype techonly stdcells</code>).
<code>-extraction_tech_file file</code>	
	Default: None Specifies the name of the technology file that will be used for extraction.
<code>-filler_cells cell_name_list</code>	

	Default: None List the names of the filler or feedthrough cells that have no GDS data. These cells can be swapped out for decaps during decap optimization. This parameter supports wildcards (*). Usage Guidelines: This parameter is applicable only for technology and standard cell PGV generation (<code>-celltype techonly stdcells</code>).
<pre>-fine_grain_memory_cell {true false}</pre>	
	Default: <code>false</code> Specifies to tag the cell as the fine-grain cell in the PGV library when characterizing the cell, and ensures that the software automatically uses this information to identify the fine-grain memory cells in the rail analysis flow. This parameter is used with the <code>-internal_power_pins</code> and <code>-internal_ground_pins</code> parameters to specify the list of switch power/ground pins. It is recommended to use these parameters when characterizing fine-grain cells to avoid any potential cell type identification conflict in the rail analysis flow. Usage Guidelines: This parameter is applicable only for macro PGV generation (<code>-celltype macros</code>).
<pre>-gds_files file_list</pre>	
	Default: None Specifies the names of the GDSII files used during power-grid library generation. Usage Guidelines: This parameter is a mandatory input for macro PGV generation (<code>-celltype macros</code>).
<pre>-gds_layermap file</pre>	

Default: None

Specifies a file that provides the mapping between the library layers (qrcTechFile layers) and the GDS layers.

Sample File Format:

layer_type	layer_name	gds	gds_layer_no
via	VIA9	gds	59

Usage Guidelines:

This parameter is a mandatory input for macro PGV generation (`-celltype macros`).

```
-generate_port_using_gds mbb_layer_name
```

Default: None

Identifies the layer name that library generation uses to compute the placement bounding box for GDSII cells that are placed by the DEF file.

Usage Guidelines:

This parameter is applicable only for macro PGV generation (`-celltype macros`).

```
-ground_pins { ground_pin_list }
```

Default: None

Specifies the names of the nets considered ground nets for all cells for which the ground nets are not specified.

```
-ignore_gds_cells {gds_cell_name_list}
```

Default: None

Ignores the specified cells during macro PGV generation. When this parameter is specified, the Library Generation engine does not extract data for these cells. This parameter supports wildcards.

Examples:

The following command does not extract data for the `cell13` cell:

```
set_pg_library_mode -ignore_gds_cells {cell13}
```

```
-internal_power_pins power_pin_list
```

	Default: None Specifies the list of switch power pins. Usage Guidelines: This parameter is applicable only for macro PGM generation (<code>-celltype macros</code>).
<code>-internal_ground_pins ground_pin_list</code>	
	Default: None Specifies the list of switch ground pins. Usage Guidelines: This parameter is applicable only for macro PGM generation (<code>-celltype macros</code>).
<code>-layer_tap_node_distance { {LayerName1 tap_node_distance} {LayerName2 tap_node_distance} ... }</code>	
	Default: None Specifies tap node distances for certain layers. Using this parameter, you can customize the tap node distance for each layer during power-grid library generation. The tap node distance is in micron. If the tap node distance is specified for a single layer, double brackets <code>{{}}</code> are still required.
<code>-layers_to_be_ignored_during_tap_creation {layername+}</code>	
	Specifies the names of the layers that library generation is to ignore when it creates the current taps. This parameter can be used to suppress the generation of any nodes or taps on layers that should not be extracted. Some technologies include layers that are only used to influence the extraction of other layers.
<code>-layers_to_have_ports_created {layername+}</code>	
	Specifies the names of the layers for which ports are to be created. This parameter can be used to create ports for the poly and diffusion layers in the technology file. By default ports are only created on metal layers.
<code>-lef_layermap file</code>	

Default: None

Specifies a file that provides the mapping of layers between the LEF and technology file. If this parameter is not specified, Library Generation automatically generates the layermap file using the technology LEF (containing all layer and via definitions) and Quantus Tech (extraction library) files. The automatically generated layermap file (`lefdefLayerMap_AutoGenerated.txt`) is located in the PGV temp directory. You can specify this parameter if you want to provide your own layermap file.

File Format:

<u>Layer Type</u>	<u>tech layername</u>	<u>lefdef</u>	<u>lefdef layername</u>
via	VIA_9	lefdef	VIA9
metal	METAL_1	lefdef	METAL1

`-liberty_files file_list`

Default: None

Specifies to directly read decoupling capacitance and resistance from the specified Liberty file for standard cell and macro PGV generation.

The following is an example of the Liberty construct required for capacitance/resistance:

```
intrinsic_parasitic () {  
    when : "I";  
  
    intrinsic_capacitance (VDD) {  
        value : 0.000248992;  
    }  
  
    intrinsic_resistance (VSS) {  
        related_output : ZN;  
        value : 1.89195;  
    }  
  
    intrinsic_capacitance (VSS) {  
        value : 0.000639008;  
    }  
}
```

If there are multiple `when` conditions, the average intrinsic resistance/capacitance value is taken.

Usage Guidelines:

This parameter is applicable only for standard cell and macro PGV generation (`-celltype stdcells | macros`).

```
-macros_config_file filename
```

Default: None

Specifies to provide configuration details for each macro cell per Library Generation run. This is an additional input for `macros` cell type wherein you can provide macro specific Spice and GDS.

File Format:

```
CELL <cell1>
GDS_FILE <gds_file1>
SUBCKT_FILE <subckt_file1>
DECAP_FILE <decap_file1>
TRIGGER_FILE <trigger_file1>
POWER_GATE_PARAMS supply <supplyName> switched <switchedName>
CELL <cell2>
GDS_FILE < gds_file1 >
SUBCKT_FILE < subckt_file1 >
...
```

`CELL` is the keyword to specify a new cell that is to be characterized. This is a mandatory keyword. This keyword is followed by optional keywords, like `GDS_FILE` and `SUBCKT_FILE`, that are specific to the cell.

Usage Guidelines:

This parameter is applicable only for macro PGV generation (`-celltype macros`).

```
-max_metal_density_tile_dimension {maxlength maxwidth}
```

Default: 50µm

Specifies the tile dimensions, wherein `maxlength` denotes maximum length and `maxwidth` denotes maximum width in µm. The number of tiles in the X and Y direction is determined using the length and width dimension.

Usage Guidelines:

- This parameter is applicable only for macro PGV generation (`-celltype macros`).
- This parameter should only be used when the `-calculate_metal_layer_density` parameter is set to `true`.

```
-non_zero_ground_pins { pin1 voltage1 ...pinN voltageN }
```

Default: None Specifies the name and voltage of the nets considered ground nets for all cells for which the elevated ground nets need to be specified. This parameter allows you to provide a list of non-zero ground be it positive or negative for the ground nets. If the same name has been specified for both the parameters <code>-non_zero_ground_pins</code> and <code>-ground_pins</code>, then <code>-non_zero_ground_pins</code> will be given priority. Usage Guidelines: This parameter is applicable only for standard cell and macro PGV generation (<code>-celltype stdcells macros</code>). Example: <pre>set_pg_library_mode -non_zero_ground_pins {VSS -1.05 VSSA 0.08 VSSA1 1.09}</pre>	<pre>-oasis_files file_list</pre>
Specifies the names of the OASIS files used during power-grid library generation. Usage Guidelines: This parameter is a mandatory input for macro PGV generation (<code>-celltype macros</code>).	<pre>-oasis_layermap file</pre>
Specifies a file that provides the mapping between the library layers (qrcTechFile layers) and the OASIS layers. File Format: <pre>layer_type layer_name oasis oasis_layer_no via VIA9 oasis 59</pre> Usage Guidelines: This parameter is a mandatory input for macro PGV generation (<code>-celltype macros</code>).	<pre>-pin_shape_current_distribution filename</pre>

Default: None

Specifies a current distribution file that allows you to state how current should be distributed across multiple pin shapes of power and ground pins. Multiple pin shapes exist for VDD and VSS pins, and current drawn through each pin shape is non-uniform depending upon the output bit that is switching. This parameter allows you to define current distribution for multi-height multi-bit flip-flop (MBFF) characterization. The parameter allows you to accurately model the current through each pin shape.

File Format

```
CELL <cell name>
PGPIN VDD: (<LEF_layer_name>, <LEF_coordinates>) : (<LEF_layer_name>,
<LEF_coordinates>)
PGPIN VSS: (<LEF_layer_name>, <LEF_coordinates>) : (<LEF_layer_name>,
<LEF_coordinates>)
PIN <signal_pin_name> <current_ratio_for_VDD> <current_ratio_for_VSS>
```

Here,

- *<LEF_layer_name>, <LEF_coordinates>* specifies to assign the current ratio to the pin shape occupying the coordinates.
- *LEF_layer_name* the layer name to which the current taps are to be created

Example

```
CELL NAND3X8
PGPIN VDD: (Metal1,1,1) : (Metal1,4,1)
PGPIN VSS: (Metal1,2,2) : (Metal2,5,2) : (Metal2,8,2)
PIN Q1 0.00:1.00 0.00:0.00:1.00
PIN Q2 0.90:0.10 0.95:0.05:0.00
```

```
-power_pins { pin1 voltage1 ...pinN voltageN }
```

Default: None

Specifies the name and voltage of the nets considered power nets for all cells for which the power nets are not specified. If both the parameters `-power_pins` and `-default_power_voltage` are not specified, the default voltage of 1.2 will be used.

```
-powergate_finegrain_simulation {true|false}
```

Default: false

Specifies to enable finegrain powergate simulation.

```
-powergate_parameters {{cell    supply    switched    Ron    Idsat    Ileak
enablePinName    Control_Pin_Logic} + }
```

Default: None

Specifies the information that technology library generation needs to characterize a powergate cell. It includes the cell name, unswitched power pin, switched power pin, on resistance in ohms, saturation current in milliamps, leakage current in milliamps, and control pin logic.

If ron, idsat, or ileak values are specified, they will be used to override the data extracted by Voltus library simulator.

Control_Pin_Logic can have the following values:

- enableHigh - control pin(s) of the control logic will be connected to the power net to turn on the power gate instances.
- enableLow - control pin(s) of the control logic will be connected to the ground net to turn on the power gate instances.

The control pin logic allows you to specify whether the enable pin of the specified power gate is active high or active low, thus ensuring that the appropriate voltages are applied to the enable pins.

You can specify multiple values for on-resistance, saturation current, and leakage current for the same cell based on different enable pin names.

This parameter also allows you to specify multiple functional modes to enable different combinations of control pins in each mode. For multiple functional modes, the tool calculates RON for each mode separately and generates a single PGV for the specified modes.

Format for Multiple Functional Modes:

```
set_pg_library_mode -powergate_parameters { {cell_name mode_name1 {supply switch
Ron Idsat Ileak Control_Pin_Logic} mode_name2 {supply switch Ron Idsat Ileak
Control_Pin_Logic}... mode_nameN {supply switch Ron Idsat Ileak
Control_Pin_Logic} }
```

Examples:

- For control pin logic:

```
-powergate_parameters { {HEADER_SWITCH TVDD VDD enableHigh} }
```
- For Single Powergate:

```
-powergate_parameters {{HDRSID0 TVDD VDD}}
```

- For 2 Header Powergate:

```
-powergate_parameters {{HEAD16_A12TH_C35 VDDG VDD} {HEAD8_A12TH_C35 VDDG VDD}}
```

- For Multiple Footer Powergate:

```
-powergate_parameters {{FOOTBUFVHSV16 VSSG VSS} {FOOTBUFVHSV32 VSSG VSS} {FOOTBUFVHSV64 VSSG VSS} {FOOTBUFVHSV8 VSSG VSS}}
```

- For Multiple Functional Modes:

```
-powergate_parameters { {HEADER_SWITCH S1 {TVDD VDD 750 .5 4.0e-8V} S3 {TVDD1 VDD 350 .3 5.0e-8V} S2 {TVDD VDD 450 .5 4.0e-8V} } {RING_SWITCH TVDD VDD 750 .5 4.0e-8} }
```

- For Different Enable Pins:

```
-powergate_parameters {{Cell11 VDDG VDD 300 5e-8 1500 Enable1 } {Cell11 VDDG VDD 280 4e-8 200 Enable2 }}
```

Usage Guidelines:

This parameter is applicable only for the technology PGV generation (`-celltype techonly`).

```
-powergate_parameters_file filename
```

Default: None

Specifies a file that contains details of power switches that have multiple control pins, input pins, and power nets. The control pins, input pins, and power nets can have either sweep voltages or user-defined voltages.

This parameter also allows you to specify multiple functional modes to enable different combinations of control pins in each mode. For multiple functional modes, the tool calculates RON for each mode separately and generates a single PGV for the specified modes.

File Format:

```
CELL <cell_name>
PGATE_PAIR <AON_PIN> <SW_PIN> <CTRL_PIN> <Polarity|VoltageValues>
PGATE_PAIR <AON_PIN> <SW_PIN> <CTRL_PIN> <Polarity|VoltageValues>
...
PWR_PIN <net_name> {voltage_values}
PWR_PIN <net_name> {voltage_values}
...
INPUT_PIN <pin_name> <Polarity|VoltageValues>
INPUT_PIN <pin_name> <Polarity|VoltageValues>
```

```
...
PGATE_OFF_PIN <CTRL_PIN> <Polarity|VoltageValues>
PGATE_OFF_PIN <CTRL_PIN> <Polarity|VoltageValues>
PGATE_OFF_PIN <CTRL_PIN> <Polarity|VoltageValues>
...
CHARACTERIZATION_VOLTAGES { val1 val2 val3 .... }
END
```

File Format for Multiple Functional Modes:

```
CELL <cell_name>
MODE_NAME mode_name1
PGATE_PAIR <AON_PIN> <SW_PIN> <CTRL_PIN> <Polarity|VoltageValues>
PWR_PIN <net_name> {voltage_values}
INPUT_PIN <pin_name> <Polarity|VoltageValues>
PGATE_OFF_PIN <CTRL_PIN> <Polarity|VoltageValues>
CHARACTERIZATION_VOLTAGES <VoltageValues>
END_MODE

MODE_NAME mode_name2
PGATE_PAIR <AON_PIN> <SW_PIN> <CTRL_PIN> <Polarity|VoltageValues>
PWR_PIN <net_name> {voltage_values}
INPUT_PIN <pin_name> <Polarity|VoltageValues>
PGATE_OFF_PIN <CTRL_PIN> <Polarity|VoltageValues>
CHARACTERIZATION_VOLTAGES <VoltageValues>
END_MODE

...
END
```

- PGATE_PAIR and CHARACTERIZATION_VOLTAGES are mandatory keywords.
- PGATE_PAIR is the keyword to specify the always-on power pin, switched power pin, and control pin names, and the voltage values at which these pins are simulated. If you specify voltage values, then the software uses these user-defined values during power gate characterization. Alternatively, if you specify polarity (high/low), then the software uses the sweep voltage values (CHARACTERIZATION_VOLTAGES) during power gate characterization. If you do not specify both polarity or voltage values, then the software by default uses the sweep voltage values (CHARACTERIZATION_VOLTAGES) during power gate characterization.
- CHARACTERIZATION_VOLTAGES is the keyword to specify sweep voltage values. The number of voltage values specified with the PGATE_PAIR keyword must match with the

number of voltage values specified with the `CHARACTERIZATION_VOLTAGES` keyword.

- `PWR_PIN` is the keyword to specify additional power pins of the cell other than the always-on and switched power pins.
- `INPUT_PIN` is the keyword to specify additional signal pins of the cell other than the control pins. If you specify voltage values, then the software uses these user-defined values during power gate characterization. Alternatively, if you specify polarity (high/low), then the software uses the sweep voltage values (`CHARACTERIZATION_VOLTAGES`) during power gate characterization. If you do not specify both polarity or voltage values, then the software by default uses the sweep voltage values (`CHARACTERIZATION_VOLTAGES`) during power gate characterization.
- `PGATE_OFF_PIN` is the keyword to specify the control pin names and the voltage values at which the off-resistance is computed. If you specify voltage values, then the software uses these user-defined values during power gate characterization. Alternatively, if you specify polarity (high/low), then the software uses the sweep voltage values (`CHARACTERIZATION_VOLTAGES`) during power gate characterization. If you do not specify both polarity or voltage values, then the software by default uses the sweep voltage values (`CHARACTERIZATION_VOLTAGES`) during power gate characterization.

Examples:

Example 1: User-specified voltage values for control pins, input pins, and power nets

```
CELL cellA
PGATE_PAIR AONVDD SWVDD SLEEPN {0.600 0.660 0.700 0.750 0.800 }
PWR_PIN VDD_PS {0.7 0.8 0.9 1.0 1.1 }
INPUT_PIN PS_DISABLE {0.600 0.660 0.700 0.750 0.800 }
CHARACTERIZATION_VOLTAGES {0.600 0.660 0.700 0.750 0.800 }
END
```

Example 2: User-specified polarity

```
CELL cellb
PGATE_PAIR TVDD VDD SLEEPN
PWR_PIN VDD_SIDE
INPUT_PIN S1 HIGH
CHARACTERIZATION_VOLTAGES {0.600 0.660 0.700 0.750 0.800 }
END
```

Example 3: powergate_param_file.txt

```
CELL HEADER_SWITCH
```

```
MODE_NAME S1
PGATE_PAIR TVDD VDD NSLEEPIN LOW
CHARACTERIZATION_VOLTAGES {0.9 1.0 1.1}
END_MODE

MODE_NAME S2
PGATE_PAIR TVDD VDD NSLEEPIN HIGH
CHARACTERIZATION_VOLTAGES {0.9}
END_MODE

MODE_NAME S3
PGATE_PAIR TVDD1 VDD NSLEEPIN LOW
CHARACTERIZATION_VOLTAGES {0.9 1.1}
END_MODE

END
```

Usage Guidelines:

This parameter is **applicable only** for standard cell and macro PGV generation (`-celltype stdcells | macros`).

```
-spice_corners  corner_list
```

Default: None

Specifies a list of SPICE corners. This is an optional parameter. If you specify SPICE corner(s), the number of corner items should be equal to the number of SPICE model files specified in `-spice_models file_list`. One Spice model can map to many Spice corners but not vice versa.

Examples:

- if there are three model files but only two corners, a blank corner definition is required to match the model-corner pair, as shown below:

```
-spice_models { model1.scs model2.scs model3.scs }
-spice_corners {{corner1} {} {corner2}}
```

- if a SPICE model1 has three corners and model2 has two corners, you should specify:

```
-spice_models { spice_1.model spice_2.model }
-spice_corners { {s11 s12 s13} {s21 s22} }
```

Based on the above parameter description, the following information gets updated in the library generation file:

```
spice_model_file spice_1.model {s11 s12 s13}
spice_model_file spice_2.model {s21 s22}
```

Usage Guidelines:

This parameter is a mandatory input for standard cell and macro PGV generation (`-celltype stdcells | macros`).

`-spice_models file_list`

Default: None

Specifies the names of the SPICE model files used by the SPICE netlist file (`-spice_subckts`).

Some Spice process models use `wnom/Wn` and `lnom/Ln` instead of `w` and `l` (device model parameters names for length and width) for MOS models. Voltus recognizes and supports these alternate parameter names.

Usage Guidelines:

This parameter is a mandatory input for standard cell and macro PGV generation (`-celltype stdcells | macros`).

`-spice_netlist_list_file filename`

Default: None Specifies a file containing all the spice netlists to be used for library generation, the format is one netlist per line. These spice netlist files must contain the subcircuit definitions for the cells being processed by power-grid library generation. Usage Guidelines: This parameter is applicable only for standard cell and macro PGV generation (<code>-celltype stdcells macros</code>).	<code>-spice_subckts file_list</code>
Default: None Specifies the names of the SPICE netlist files containing the subcircuits corresponding to the cells being processed during power-grid library generation. These subcircuit netlists will be used to calculate pin capacitances using Voltus library simulator. Usage Guidelines: This parameter is a mandatory input for standard cell and macro PGV generation (<code>-celltype stdcells macros</code>).	<code>-spice_subckts_xyscaling value</code>
Default: The default scale factor is 1000 considering that the SPICE netlist coordinate unit is um. Specifies the current region scale factor for the Netlist-based flow. The current region scale factor is the factor by which the netlist coordinates are scaled before finding the closest node to attach the tap currents. The netlist-based flow fails when there is a mismatch in the units of the PGDB coordinates (in nm) and the coordinates in the SPICE netlist causing improper current tap creation. To match these coordinates, you need to specify the current region scale factor. Usage Guidelines: This parameter is applicable only for macro PGV generation (<code>-celltype macros</code>).	<code>-stop@via layertype</code>

Default: Via1

Stops the extraction of the power-grid network inside a cell at the specified via layer.

However, this parameter will not be honored if the port on the signal/power pin is on a layer below the stop@via layer. In this case, the grid at the port (lower) layer will be extracted to maintain connectivity and to ensure that the power-grid network is integrated for IR drop analysis.

Usage Guidelines:

This parameter is applicable only for macro PGV generation (`-celltype macros`).

`-stop@via_file filename`

Specifies the name of a file containing a list of cells and the corresponding via layers at which extraction of the power-grid network should be stopped. However, this functionality is applicable only for the cells that are mentioned in the cell list file as well.

File Format:

`<cell_name> <stop@_layer_name>`

Sample File Format:

```
Cell11 Via5
Cell12 Via9
```

Usage Guidelines:

This parameter is applicable only for macro PGV generation (`-celltype macros`).

`-tap_node_distance_file filename`

Default: None

Specifies a file containing the distance between tap nodes for the given list of cells. The specified file includes the cell name and tap node distance in micron. The default unit for tap node distance is `micron`, and the default tap node distance is `50 microns`.

This parameter allows you to customize the tap node distance for each cell during power-grid library generation.

File Format:

`cell_name tap_node_distance`

Examples:

```
OA_cellA 1
OA_cellC 1
```

Usage Guidelines:

This parameter is applicable only for standard cell and technology PGV generation (`-celltype stdcells|techoonly`).

`-tech_gds_layermap filename`

Specifies a file that provides the mapping of the layer names in the Quantus technology file to the layer names in the GDS file.

File Format:

<u>technology</u>	<u>layername</u>	<u>use</u>	<u>gds_number</u>	<u>data_type</u>
METAL_1		text	131	0
VIA_1		x	51	0
METAL_2		text	132	0
VIA_2		x	52	0
...				

If `-design_qrc_layermap` is specified along with `-tech_gds_layermap`, then the format of `-tech_gds_layermap` is treated as:

<u>lefdef</u>	<u>layername</u>	<u>use</u>	<u>gds_number</u>	<u>data_type</u>
M1		text	131	0

`-tech_oasis_layermap filename`

Specifies a file that provides the mapping of the layer names in the Quantus technology file to the layer names in the OASIS file.

File Format:

technology_layername	use	oasis_number	data_type
METAL_1	text	131	0
VIA_1	x	51	0
METAL_2	text	132	0
VIA_2	x	52	0
...			

If `-design_qrc_layermap` is specified along with `-tech_oasis_layermap`, then the format of `-tech_oasis_layermap` is treated as:

lefdef_layername	use	oasis_number	data_type
M1	text	131	0

`-temperature temp_value_in_degree_celcius`

Default: 25

Specifies the temperature in degree celcius used in library characterization for thermal aware EM/IR analysis.

`-tsv_subckt_modelfile_list filename`

Specifies the name of external Through Silicon Via (TSV) subckt model files to be used for 3D-IC extraction. You can specify a list of files separated by space. Each file contains the subckt model information for TSV. The specified file has the Spice format.

A sample file is shown below:

```
.subckt TSV top bottom
R1 top n2 0.03075
R2 n2 bottom 0.03075
C1 n2 0 28f
.ends
```

`-use_lefonly_for_detailedview_generation {true|false}`

Default: `false`

Specifies to generate a detailed PGV with the bump LEF without loading a GDS file. You can specify this parameter to perform the RC extraction of a hybrid bump or via cell while skipping the extraction of its top and bottom routing metal layers. When set to `true`, the top and bottom metal layers are preserved as geometries in the library and promoted to the top level, whereas the via layer shapes are dropped and their resistance is used directly from the PGV.

```
-use_powergate_data_from_lib {true|false}
```

Default: `false`

Specifies to read powergate parameters `ron`, saturation current, and leakage current from the Liberty files specified using the existing `-liberty_files` parameter. This parameter allows you to obtain the powergate parameters from a Liberty file when a Spice netlist is not available, or when the Liberty file contains the powergate parameters.

Usage Guidelines:

This parameter is applicable only for technology and standard cell PGV generation (`-celltype techonly | stdcells`).

Examples

- The following command sets the power library mode:

```
set_pg_library_mode
-extraction_tech_file RCgen.tch
-lef_layermap lefdef.map
-celltype techonly
-power_pins { VDD 1.08 VDDO 1.08 VDDG 1.08 }
-ground_pins { VSS GND VSSG }
-cell_list_file built_cells_list
-temperature -40
-spice_models models/vld1_usage.1
-spice_corners { ss_lib }
-spice_subckts { typical_max_m40.spice }
-powergate_parameters { { cell1 VDDG VDD } { cell2 VDDG VDD } }
-current_distribution propagation
```

- The following command generates a PGV that contains the voltage dependent capacitance table for the specified cells:

```
set_pg_library_mode -celltype stdcells -cell_list_file cellfile -
extraction_tech_file ../../qrcTechFile -lef_layermap ../../lefdef.layermap -
temperature 125 -power_pins { vcc 1.26 vccpg 1.26 vccr 1.26 } -ground_pins { vss }
-powergate_parameters {{powercell_pg1 vcc vccpg} {powercell_pg1 vcc vccr}
{powercell_pg2 vcc vccpg} {powercell_pg2 vcc vccr}} -spice_corners { tt } -
spice_models spice_model/libspice -spice_subckts { powercell_pg.net
powercell_pg2.net } -enable_multi_voltage_cap_generation true
```

The PGV report includes the following information:

```
INFO: PGV contains voltage dependent cap table for this cell
```

Trigger File Format

For the current characterization flow, you must specify the `-current_distribution { dynamic_simulation trigger_file }` parameter. The format of the trigger file is:

Trigger File Format	Description
#Method of current characterization	
CURRENT_CHARACTERIZATION_METHOD <method> <SIMULATION/DOTLIB/PWL/FSDB/DATASHEET>	• SIMULATION -> Vector-based flow • DOTLIB -> Liberty based Flow • PWL - User-defined PWL flow • FSDB - Spice total current output captured in the FSDB format • DATASHEET - Datasheet details of any memory
#Memory cell name	
CELL <cellname1>	Name of the memory cell
#Mode and conditional details for trigger	

<pre>LEAKAGE_CURRENT { <PGPin1> <value1> ... <PGPinN> <valueN> }</pre>	Specifies the leakage current for both power and ground pins, or just power pins. Dynamic Power Analysis will process <code>LEAKAGE_CURRENT</code> specified in the PGV trigger file, and use it when a macro cell is idle or dynamic switch pattern is not defined (defined as "-"). In the following example, the second cycle will use <code>LEAKAGE_CURRENT</code> defined in the PGV trigger file: <pre>set_power -cell x -dynamic_switch_pattern { READ - READ }</pre> You must specify the leakage current for all power pins. If leakage current is not defined for some pins, then those pins will get 0 leakage current when processed by Power Analysis. <code>LEAKAGE_CURRENT</code> should be specified in the trigger file as a global, and specified outside the mode specification, as shown below: <pre>CURRENT_CHARACTERIZATION_METHOD PWLCELL cell1 LEAKAGE_CURRENT { VSS 2p VDD 17n }MODE_NAME READ static... END_MODEEND</pre>
<pre>MODE_NAME <model> CONDITIONAL_INPUT <conditional input statement for cellname1></pre>	Mode name and conditional input statement for the cell. Conditional input of current for the given mode.
<pre>CONDITIONAL_PIN <pin_name> (rise fall both)</pre>	Conditional pin statement for the cell. Signal pin for which the tool should rise or fall.
<pre>CONDITIONAL_STIMULUS_FILE <usim1.txt></pre>	Simulation vector detail in the given mode.
<pre>USER_PWL_FILE <pwl1.txt></pre>	User-specified PWL file that contains information about the PWL waveforms (power/ground currents). The current for ground net should be specified as a negative value.

FSDB_START_TIME <time> FSDB_END_TIME <time>	FSDB Simulation based mode details
DATASHEET_PARAMETER < {pg_pin1 Capacitance Leakage} {pg_pin2 Capacitance Leakage}>	Datasheet-based mode details
END_MODE	End mode section
#Datasheet clock required parameters	
DATASHEET_INPUT_SLEW <time> DATASHEET_OUTPUT_SLEW <time> DATASHEET_DELAY <time> DATASHEET_PARAMETER < { power_pin cap leakage_pwr } >	• DATASHEET_INPUT_SLEW - Clock input slew of the conditional pin • DATASHEET_OUTPUT_SLEW - Rising slew of the output pin Q • DATASHEET_DELAY - Mode delay as the read_access and write_access time • DATASHEET_PARAMETER - Datasheet parameters include power pin voltage, load capacitance, and leakage power. Load capacitance is mode dependent.
#FSDB required parameters	
FSDB_PATH <path> FSDB_NET_NAME <net> [<generic_name>] FSDB_START_TIME <time> FSDB_END_TIME <time> FSDB_HIER_CHAR <hier_char, for hier DB>	• FSDB file path • Power net name and its FSDB probe name • FSDB start and end time • Hierarchical separator for hierarchical database. The default separator is "." Single FSDB file having total current for ALL power/ground pins with all mode current in it
#<Other mode independent parameters>	


```
DOTLIB_LIST {a list of memory dotlibs  
files}  
MODE_COUNT  
MODE_CONFIG_FILE <file_path>  
SIM_START_TIME <time>  
SIM_STOP_TIME <time>  
SIM_STEP_SIZE <step_size>  
STIMULUS_FILE <file_path>  
INCLUDE_FILE <file_path>
```

- **DOTLIB_LIST** - Specifies a list of memory dotlibs.
- **MODE_COUNT** - Specifies the maximum number of modes to be generated. Valid values are either a positive number or 'all'. 'all' means stimulus for all power groups are to be generated. The default value is 4. If you set this parameter to `all`, it may result in high runtime as it may generate too many modes.
- **MODE_CONFIG_FILE** - Specifies the file containing extra information for memory cell. This file contains the following information - signal slew, signal active value and memory write information.
- **SIM_START_TIME** - Specifies the simulation start time. The default unit is in nanoseconds (ns).
- **SIM_STOP_TIME** - Specifies the simulation stop time. The default unit is in nanoseconds (ns).
- **SIM_STEP_SIZE** - Specifies the step size for power-grid analysis. The default value of step size is 20 picoseconds (ps). This parameter supports other units such as, ns and fs. This parameter supports unit-prefixes (pico, femto, nano) combined with units of measure (ps, fs, ns), but does not support units itself ("second", "Ohm", "Farad") without unit-prefix.
- **STIMULUS_FILE** - Specifies the stimulus file containing the trigger information for all input ports.
- **INCLUDE_FILE** - Specifies the name of the simulation include file.

#End cell section	
END	End cell section

Current Region File Format

The following shows the `current_region_file` format:

```
UNIT CURRENT default_unit_in_amps
UNIT CAPACITANCE default_unit_in_farads
CELL cell_name
<PROP [FLAT | HIER]>
NET net_name
LAYER layer_name
[ REGION x1 y1 x2 y2 current_value <cap_value> | REGION current_value <cap_value> ]
```

UNIT	Defines the default scale to be used with the current and capacitance values read from the current region file. These scales apply to any value that is not followed by an explicit unit specification. Valid current values are: A (Amps), mA (miliAmps), uA (microAmps), and nA (nanoAmps). Valid capacitance values are: F(Farads), mF (miliFarads), uF (microfarads), nF (nanofarads), pF (picoFarads), and fF (femtoFarads).
CELL <i>cell_name</i>	This command specifies the name of the cell. All the following NET, LAYER, and REGION commands will apply to this CELL until another CELL specification is encountered. Wildcards can be used in the cell_name.
<PROP [FLAT HIER]>	If the keyword FLAT is applied, LibGen will not perform hierarchical tracing for the given net in this cell. If the keyword HIER is presented, LibGen will perform hierarchical tracing for the given net in this cell. This provides the ability for LibGen to automatically specify a current value to all instances of a cell in the design hierarchy. The assumption is that all of these instances of the cell will consume the same power. The default is HIER.

NET <i>net_name</i>	This command specifies that all the LAYER and REGION commands following it applies to the <i>net_name</i>. This stays in effect until another NET or CELL specification is encountered. Wildcards can be used in the <i>net_name</i>.
LAYER <i>layer_name</i>	This command specifies that all the REGION commands following it apply to the <i>layer_name</i>. This stays in effect until another LAYER, NET, or CELL command is encountered. The <i>layer_name</i> must be a library layer name. If the layer is not found, LibGen will issue an error message and exit. You must provide a valid layer name for LibGen to run. If you specified a valid layer, but LibGen does not find any via in the assigned current region, LibGen will issue a warning message, but the process will still continue and exit successfully. In this case, you may get a 0 current value for the specified cell.
REGION <i>x1 y1 x2 y2</i> <i>current_value</i> <i>cap_value</i>	This command specifies the current value and optionally the capacitance value to be associated with the rectangular region specified by the bounding box (<i>x1 y1 x2 y2</i>). Current taps will be created at the locations of the vias or contacts specified by the most recent LAYER command. These current taps will be attached to the layer that is below the specified via or contact layer. Coordinate values are assumed to be in GDSII coordinate units and will be interpreted according to the scale and precision settings contained in the GDSII file (or spice netlist with device XY location) that contains the cell. The value of the current drawn by each current tap will be 1/n of the value specified for the current region, where n is the number of taps within the current region. The second variation of the REGION command without the bounding box is valid only if there is only one current region. In this case LibGen can automatically uses the bounding box of the cell for the region. You can specify a single region or multiple regions per cell.

Example of Current Region File

```
UNIT CURRENT uA
UNIT CAPACITANCE fF
CELL an4_80
NET vcc
LAYER CONT
REGION 0 14000 28900 17000 3.0 5.0
NET gnd
LAYER CONT
REGION 1.5e-2mA
```

Example of a Dynamic PWL Region File (Multiple Regions)

```
CELL Cell1
  NET VSS
    PROP HIER
    LAYER via1_r
      REGION 1 10 188 333 { 0 -1 1 -9 2 -8 3 -7 4 -6 5 -5 6 -4 7 -3 8 -2 9 -
1 } 1e-12
      REGION 121 10 185 80 { 0 -1 1 -90 2 -80 3 -70 4 -60 5 -50 6 -40 7 -30 8 -20 9 -10
} 4e-12
    NET VDDPR
      PROP HIER
      LAYER via1_r
        REGION 1 10 188 333 { 0 -1 1 9 2 8 3 7 4 6 5 5 6 4 7 3 8 2 9 1 } 5e-12

    NET VDDP
      PROP HIER
      LAYER via1_r
        REGION 1 10 188 333 { 0 -1 1 9 2 8 3 7 4 6 5 5 6 4 7 3 8 2 9 1 } 5e-12

    NET VDDAR
      PROP HIER
      LAYER via1_r
        REGION 1 10 188 333 { 0 -1 1 9 2 8 3 7 4 6 5 5 6 4 7 3 8 2 9 1 } 5e-12

    ...
```

Related Topics

- "[Power-Grid Library Generation](#)" in the *Voltus User Guide*

trim_pg_library

```
trim_pg_library
[-help]
[-delete_cells cell1 cell2 ...]
[-delete_cells_file file_name]
-library_list_file file_name
[-output directory_name]
-output_library library_name
```

Specifies to remove a list of cells from the specified PGV. This command allows you to trim the size of a cell library by removing unwanted cells, which can reduce the overall time required for extraction.

Parameters

- help	Outputs the command usage.
-delete_cells cell1 cell2 ...	
	Default: None Specifies the name of a cell or a list of cells to be removed from the power-grid library.
-delete_cells_file file_name	
	Default: None Specifies the name of a file containing the list of cells (one per line) to be removed from the power-grid library. File Format: cell1 cell2 cell3
-library_list_file file_name	

	Default: None (Mandatory) Specifies the name of a file containing the library or libraries (one per line) that are to be trimmed. If multiple libraries are given, Library Generation merges the multiple same-type power-grid libraries into one consolidated power-grid library. File Format: <pre>PGV1 PGV2 PGV3</pre>
<code>-output <i>directory_name</i></code>	
	Default: current working directory Specifies the name of the output directory in which the trimmed power-grid library will be created.
<code>-output_library <i>library_name</i></code>	
	Default: None (Mandatory) Specifies the name of the trimmed power-grid library created after removing the specified cells.

Examples

- The following command removes the cell `AND2X2` from the power-grid libraries specified in the `library_list.txt` file and generates one consolidated power-grid library `trimlibrary`:

```
trim_pg_library -library_list_file library_list.txt -output_library trimlibrary -
delete_cells AND2X2
```

Related Topics

- "[Power-Grid Library Generation](#)" in the *Voltus User Guide*

validate_pg_library

```
validate_pg_library
[-help]
[-activity value]
-cell cell_name
[-clock {clk1 clk2 ...}]
[-disable_rail_analysis {true | false}]
[-dynamic_analysis_duration value]
[-enable_xp {true | false}]
[-extraction_include_file filename]
-extraction_tech_file tech_file_path
[-frequency value]
[-input_pin_set_case_analysis_high {pin1 pin2 ...}]
[-input_pin_set_case_analysis_low {pin1 pin2 ...}]
[-liberty liberty_file]
[-max_ir_drop value]
[-net_capacitance value]
[-output output_dir]
[-pgv_mode_dynamic_switch_pattern pattern]
[-pin_capacitance value]
-power_grid_library_path pgv_path
[-power_grid_library_view view_name]
[-power_include_file filename]
[-power_pad_size value]
[-rail_analysis_type {static | dynamic}]
[-rail_include_file_begin filename]
[-rail_include_file_end filename]
[-rail_trigger_switching_pattern pattern]
[-sdc_file filename]
[-set_power_switching_pattern trigger_file]
[-set_total_power value]
[-step_size value]
[-supply_voltage value]
-tech_lef_file tech_lef_file_path
[-temperature value]
[-transition_time time]
[-twf_path twf_location]
[-user_defined_ploc_file filename]
```


Specifies to validate a macro PGV. The command takes a macro PGV and Liberty file as input, and performs IR Drop based validation by running static and dynamic power and rail analysis. In this flow, Library Generation will read the input PGV and generate a dummy DEF file that has an instance of the macro (one-cell design). It will then run static/dynamic power and rail analysis on this design. You can analyze the power and rail results to check for hotspots, connectivity, and current distribution inside the PGV.

Parameters

<code>-help</code>	
	Outputs the command usage.
<code>-activity value</code>	
	Default: 0.2 Specifies the activity value for input pins. The activity specifies the number of times an input pin switches in a clock cycle.
<code>-cell cell_name</code>	
	Default: None (Mandatory) Specifies the name of the cell to be validated.
<code>-clock {clk1 clk2 ...}</code>	
	Default: None Specifies the list of clock pins.
<code>-disable_rail_analysis {true false}</code>	
	Default: false Specifies to disable rail analysis during macro PGV validation. The following is the behavior of <code>-disable_rail_analysis true</code> with the <code>-enable_xp</code> parameter: - if <code>-enable_xp</code> is set to <code>true</code>, the tool will run only power analysis in the XP mode - if <code>-enable_xp</code> is set to <code>false</code>, the tool will enable only power analysis in the non-XP or normal mode

<code>-dynamic_analysis_duration value</code>	
	Default: 10ns Specifies the simulation period that dynamic power analysis will use. It is the duration for which a window is selected for calculation of IR drop during power analysis. Usage Guidelines: This parameter is applicable only to the dynamic power analysis flow.
<code>-enable_xp {true false}</code>	
	Default: false Specifies to validate the macro PGV in the extensively parallel (XP) mode.
<code>-extraction_include_file filename</code>	
	Specifies the extractor (ZX) commands and variables in the form of an include file. This parameter is applicable only for the standalone report generation flow in the XP mode (<code>validate_pg_library -enable_xp true</code>).
<code>-extraction_tech_file tech_file_path</code>	
	Default: None (Mandatory) Specifies the path of the extraction technology file.
<code>-frequency value</code>	
	Default: 100Mhz Specifies the clock frequency.
<code>-input_pin_set_case_analysis_low {pin1 pin2 ...}</code>	
	Default: None Specifies the list of pins to be kept at a logical low. The <code>set_case_analysis</code> command in the SDC file will have the same effect as the set pin to constant value command described here.
<code>-input_pin_set_case_analysis_high {pin1 pin2 ...}</code>	

	Default: None Specifies the list of pins to be kept at a logical high. The <code>set_case_analysis</code> command in the SDC file will have the same effect as the set pin to constant value command described here.
<code>-liberty liberty_file</code>	
	Default: None Specifies the name of the Liberty file for power calculation. Usage Guidelines: This parameter is optional for the static power analysis flow, and is mandatory for the dynamic power analysis flow. In the static flow, you need to specify either <code>-liberty</code> or <code>-set_total_power</code>. If both the parameters are specified, the <code>-set_total_power</code> parameter will be honored.
<code>-max_ir_drop value</code>	
	Default: 5 (in percentage) Specifies the maximum IR drop allowed.
<code>-net_capacitance value</code>	
	Default: 1pF Specifies the value of the net capacitance. Usage Guidelines: This parameter is applicable only to the dynamic power analysis flow.
<code>-output output_dir</code>	
	Default: current working directory Specifies the name of the output directory. This directory saves the power and rail reports, and the design output (such as .lef, .def, and so on).
<code>-pgv_mode_dynamic_switch_pattern pattern</code>	

Default: None	Specifies the switching patterns for the functional modes. The use model of this parameter is same as the <code>set_power -dynamic_switch_pattern</code> parameter.
In the following example, the mode <code>write</code> will switch in the 1st clock cycle, the mode <code>read</code> will switch in the 2nd and 4th clock cycles, and none of the modes will switch in the 3rd clock cycle:	
<pre>validate_pg_library -pgv_mode_dynamic_switch_pattern { write read - read }</pre>	
Usage Guidelines:	This parameter is applicable only to the dynamic power analysis flow.
<code>-pin_capacitance value</code>	
Default: 1pF	Specifies the value of the pin capacitance.
Usage Guidelines:	This parameter is applicable only to the dynamic power analysis flow.
<code>-power_grid_library_path pgv_path</code>	
Default: None	(Mandatory) Specifies the path of the power-grid library file.
<code>-power_grid_library_view view_name</code>	
Default: None	Specifies the power-grid view name.
<code>-power_include_file filename</code>	
Default: None	Specifies a power analysis include file. This file specifies certain commands of the dynamic engine that do not have a Voltus equivalent.
Usage Guidelines:	This parameter is applicable only to the dynamic power analysis flow.
<code>-power_pad_size value</code>	

	Default: 50 (in Micron) Specifies the distance between two adjacent DEF voltage sources.
<code>-rail_analysis_type {static dynamic}</code>	
	Default: static Specifies whether static or dynamic analysis will be done.
<code>-rail_include_file_begin filename</code>	
	Default: None Provides a file that includes additional rail options that will be run prior to the <code>analyze_rail</code> command. This can be used for those legacy rail analysis commands that do not have a Voltus equivalent.
<code>-rail_include_file_end filename</code>	
	Default: None Provides a file that includes additional rail options that will be run after the <code>analyze_rail</code> command. This can be used for those legacy rail analysis commands that do not have a Voltus equivalent.
<code>-rail_trigger_switching_pattern pattern</code>	
	Default: None Specifies a dynamic trigger file. When a Macro EM view with current signature is used in a design, it needs to be determined when this block/cell is triggered. The dynamic trigger file is used to determine this. The use model of this parameter is same as the <code>set_rail_analysis_mode -dynamic_trigger_file</code> parameter.
<code>-sdc_file filename</code>	
	Default: None Specifies the name of the timing constraints file.
<code>-set_power_switching_pattern trigger_file</code>	

	Default: None Specifies the stimulus pattern on the pins of an instance and cell. Specify this parameter only in the dynamic mode for the type instance and cell. The use model of this parameter is same as the <code>set_power -dynamic_switch_pattern</code> parameter.
<code>-set_total_power value</code>	
	Default: .001 Specifies the total power for the design in Watts. This power is distributed based on cell area.
<code>-step_size value</code>	
	Default: 20ps Specifies the simulation step size, in picoseconds. Usage Guidelines: This parameter is applicable only to the dynamic power analysis flow.
<code>-supply_voltage value</code>	
	Default: 1V Specifies the voltage of each power supply pin.
<code>-tech_lef_file tech_lef_file_path</code>	
	Default: None (Mandatory) Specifies the path of the technology LEF file.
<code>-temperature value</code>	
	Default: 125 degrees Sets the temperature for static power calculation.
<code>-transition_time time</code>	
	Default: 100ps Specifies the transition time for specific pins or instances in the design. Usage Guidelines: This parameter is applicable only to the dynamic power analysis flow.

<code>-twf_path twf_location</code>	
Default: None	
Specifies the location of the Timing Window File (TWF) for noise and power calculation.	
<code>-user_defined_ploc_file filename</code>	
Specifies the name of a file containing net names and corresponding user-defined ploc or power pad files (.pploc). The .pploc file includes power pin or voltage source locations for power and ground nets. The parameter gives you the flexibility to include either user-specified or tool-generated voltage sources. The following is a sample file with the list of nets and power pad files: <pre> ----ploc.list---- #<NET_NAME> <Pointer to ploc file> VDD input/ploc/VDD.pploc VDDA input/ploc/VDDA.pploc VDDF input/ploc/VDDF.pploc VSS input/ploc/VSS.pploc </pre> When specified, the voltage source location files (.pploc) in the given <i>ploc.list</i> file will be used for library generation. If the pploc file is missing for a few nets in the <i>ploc.list</i> file, then library generation will automatically generate .pploc for those nets.	

Examples

- The following command specifies to validate the cell `cell1` by running static power and rail analysis:

```
validate_pg_library -cell cell1 -liberty 125c.lib -clock {CLKW CLKR} -  
power_grid_library_path macros_cell.cl -extraction_tech_file caps.tch -  
tech_lef_file tech.lef -output validate_pg_output_static -rail_analysis_type  
static
```

- The following command specifies to validate the cell `cell1` by running dynamic power and rail analysis

```
validate_pg_library -cell cell1 -liberty 125c.lib -clock {CLKW CLKR} -  
power_grid_library_path macros_cell.cl -temperature 122 -extraction_tech_file  
caps.tch -tech_lef_file tech.lef -output validate_pg_output_dynamic -  
rail_analysis_type dynamic
```

Power Calculation Commands

- [analyze_thermal](#)
- [create_thermal_model](#)
- [dump_unannotated_nets](#)
- [encrypt_thermal_file](#)
- [extract_metal_density](#)
- [get_default_switching_activity](#)
- [get_dynamic_power_simulation](#)
- [get_power_analysis_mode](#)
- [map_activity_file](#)
- [merge_vtm](#)
- [propagate_activity](#)
- [query_power_data](#)
- [read_activity_file](#)
- [report_annotated_parasitics](#)
- [report_instance_power](#)
- [report_power](#)
- [reset_power_activity](#)
- [restore_power_database](#)
- [set_default_switching_activity](#)
- [set_dynamic_power_simulation](#)
- [set_inst_temperature_file](#)
- [set_metal_density_options](#)
- [set_power](#)
- [set_power_analysis_mode](#)
- [set_power_calc_temperature](#)
- [set_power_include_file](#)
- [set_power_output_dir](#)
- [set_switching_activity](#)
- [set_thermal_analysis_mode](#)
- [set_twf_attribute](#)

- [set_virtual_clock_network_parameters](#)
- [write_power_constraints](#)
- [write_tcf](#)

analyze_thermal

```
analyze_thermal
-help
```

Specifies to invoke Celsius Thermal Solver to run die-only thermal analysis. The command generates the die temperature map file that is used by Voltus to run temperature-aware power/IR/EM analysis. Before specifying this command, use the `set_thermal_analysis_mode` command to set the parameters for performing chip-centric thermal analysis.

Parameters

-help	Outputs the command usage and a brief description about the command parameters.
-------	---

Examples

- The following command specifies to run die-only thermal analysis:
`analyze_thermal`

Related Topics

- [set_thermal_analysis_mode](#)

create_thermal_model

```
create_thermal_model
[-help]
[-leakage_temperature_scale_table {$temp1 $scalefactor1 $temp2 $scalefactor2 $temp3 $scalefactor3 .....}]
[-leakage_temperature_scale_table_file filename]
[-metal_density_file filename]
[-power_db path_to_power_database]
[-vtm_SI_unit {true | false}]
[-vtm_conductivity_inputs filename]
[-vtm_densitytable_tile {Xint Yint}]
[-vtm_file filename]
[-vtm_format {simple | stack} ]
[-vtm_format_version {new | default} ]
[-vtm_header_include_file filename]
[-vtm_leakage_temp {temp1 temp2 temp3} ]
[-vtm_output_dir directory]
[-vtm_powertable_tile {Xint Yint} ]
[-vtm_tile {Xint Yint}]
[-vtm_tile_size {$X_um $Y_um}]
```

Sets the parameters for extracting the metal density information. This command must be specified after the `extract_metal_density` command.

Parameters

- help	Outputs the command usage and a brief description about the command parameters.
-leakage_temperature_scale_table {\$temp1 \$scalefactor1 \$temp2 \$scalefactor2 \$temp3 \$scalefactor3}	
	Specifies the global scale factor for a temperature value to enable scaling of the leakage power values coming from the input dotlib file. This parameter allows you to scale the power values only at certain specific temperatures. This parameter allows you to generate a multi-temperature leakage power map table from a single input dotlib file in the power map generation flow. <code>-leakage_temperature_scale_table</code> and <code>-leakage_temperature_scale_table_file</code> are mutually exclusive.
-leakage_temperature_scale_table_file filename	

	Specifies a file containing the user-defined temperature values and the corresponding scale factors for each library, cell, or instance. The format of the file is: <pre>#Type Name temp1 temp2 temp3 temp4 Lib library_name scalefactor@temp1 scalefactor@temp2 scalefactor@temp3 scalefactor@temp4 Cell cell_name scalefactor@temp1 scalefactor@temp2 scalefactor@temp3 scalefactor@temp4 inst instance_name scalefactor@temp1 scalefactor@temp2 scalefactor@temp3 scalefactor@temp4</pre> This parameter allows you to generate a multi-temperature leakage power map table from a single input dotlib file in the power map generation flow. <code>-leakage_temperature_scale_table</code> and <code>-leakage_temperature_scale_table_file</code> are mutually exclusive. File Format Example for the Type Lib: <pre>#Type Name 25 50 125 150 Lib fast 1 2 10 100 Lib bufao 1 2 10 100 Lib header 1 2 10 100 Lib pso_ring_wc 1 2 10 100</pre>
	<code>-metal_density_file filename</code>
	Specifies the VTM metal density file generated by the <code>extract_metal_density</code> command.
	<code>-power_db path_to_power_database</code>
	Specifies the path to the power database with cell boundary box, location, and rotation information. This is generated by specifying the <code>set_power_analysis_mode -save_bbx true</code> parameter.
	<code>-vtm_SI_unit {true false}</code>
	Specifies the unit definition for power and length. The possible arguments are: <code>true</code>: unit will be in m and W. <code>false</code>: unit will be um and mW. Default is <code>true</code> with unit m(meter) for length and W(Watt) for power.
	<code>-vtm_conductivity_inputs filename</code>
	This parameter is same as the <code>set_metal_density_options -thermal_conductivity_file</code> parameter. When you want to overwrite the thermal conductivity defined in the metal density file, you can provide a new thermal conductivity file using the <code>create_thermal_model -vtm_conductivity_inputs</code> command.
	<code>-vtm_densitytable_tile {Xint Yint}</code>
	Specifies the number of tiles in the X and Y directions for the metal density tables only.
	<code>-vtm_file filename</code>
	Specifies the name of the output Voltus thermal model file.
	<code>-vtm_format {simple stack}</code>

	Generates a tile-based power map file in the simple or stack format to perform accurate thermal analysis. The possible arguments are: • <code>simple</code>: generates only power tables without metal density tables. • <code>stack</code>: generates both power and metal density tables.
	<code>-vtm_format_version {new default}</code>
	Determines whether to generate the default or the new power map file format. This parameter allows you to have different tile number between the power table and the metal density table. When set to <code>new</code>, the power map file format with non-uniform tile numbers for the power (<code>-vtm_powertable_tile</code>) and metal density tables (<code>-vtm_densitytable_tile</code>) can be generated.
	<code>-vtm_header_include_file filename</code>
	Allows you to define extra header information like file location.
	<code>-vtm_leakage_temp {temp1 temp2 temp3}</code>
	Specifies the leakage temperature values used for trilib (single voltage but different temperatures dot libs) in the power calculation flow to output user-defined temperature points' leakage power tables.
	<code>-vtm_output_dir directory</code>
	Specifies the directory path to the Voltus thermal model output folder.
	<code>-vtm_powertable_tile {Xint Yint}</code>
	Specifies the number of tiles in the X and Y directions for the power tables only.
	<code>-vtm_tile {Xint Yint}</code>
	Specifies the number of tiles in the X and Y directions for the generated Voltus thermal model file. The default is 10x10.
	<code>-vtm_tile_size {\$X_um \$Y_um}</code>
	Specifies the unit tile size in the X and Y directions for the generated thermal model file.

Examples

- The following command is an example of the trilib Tcl command, that is, uses the temperature in `vtm_leakage_temp` to calculate the leakage:

```
create_thermal_model \  
-power_db powerdb \  
-metal_density_file vtm_MD_db/my_chip.rpt \  
-vtm_output_dir ./vtm_output \  
-vtm_leakage_temp {20 50 80}
```

- The following command is an example of the mix-usage Tcl command, that is, some instances have trilib data while other instances only have a single library:

```
create_thermal_model \  
-power_db powerdb \  
-metal_density_file vtm_MD_db/my_chip.rpt \  
-vtm_output_dir ./vtm_output \  
-vtm_leakage_temp {20 50 80} \  
-leakage_temperature_scale_table {0 1 20 2 50 3 100 10}
```

Related Topics

- [extract_metal_density](#)

dump_unannotated_nets

```
dump_unannotated_nets  
[-help]  
[-file filename]  
-type annotationType
```

Shows the percentage of nets annotated by a specific type and also prints the names of all nets that are not annotated by that type.

Annotation information is stored on a per-file basis for activity, VCD, TCF, and user commands to provide detailed annotation reports when requested.

`dump_unannotated_nets` can be invoked multiple times with different report types (`-type`). You must specify this command before `report_power`.

Parameters

<code>-file <i>filename</i></code>	Specifies the text file name that includes the annotation report information. If no file is specified, the report will be written to the default output file <code>annotation_type.log</code> , where <i>type</i> is taken from the <code>-type</code> argument specified.
<code>-help</code>	Outputs the command usage and a brief description about the command parameters.

`-type`
annotationType

The type can be any of the following:

- `all`
Shows annotation reports for all types.
- `activity`
Shows annotation data for all types of activity specifications including TCF, VCD, SDC, TWF, and user commands that set activity.
- `cap`
Shows annotations from the SPEF files.
- `fsdb`
Shows annotations from the FSDB files.
- `saif`
Shows annotations from the SAIF files.
- `tcf`
Shows annotations from the TCF files.
- `timing`
Shows annotations from the TWF files. This type is applicable only to the dynamic power analysis flow.
- `vcd`
Shows annotation from the VCD files.

Example

- The following command dumps the nets in the netlist that are missing in the vcd file:

```
dump_unannotated_nets -type vcd -file vcd.log
```

The net names are written in the file `vcd.log`.

- The following command dumps the nets in the netlist which do not have an activity annotated from any source (VCD/TCF/SDC/TWF/user defined) and writes the net names in the file `unannotatednets.log`:

```
dump_unannotated_nets -type all -file unannotatednets.log
```

encrypt_thermal_file

```
encrypt_thermal_file
-help
-input ASCII.txt
-output $encrypted_conductivity.txt
```

Specifies to encrypt the file containing the thermal conductivity value for layers, and to protect foundry's thermal conductivity parameters used for power map generation. This encrypted file is then specified with the `report_power - thermal_conductivity_inputs` parameter for generating a tile-based power map file.

Parameters

-help	Prints out the command usage.
-input ASCII.txt	
	Specifies the name of the ASCII thermal file that needs to be encrypted. This file contains the thermal conductivity parameters used for power map generation. The <code>-input</code> parameter supports both layer type and layer name based thermal conductivity file. The following is a snippet of the input thermal conductivity file content (layer type based file): <pre>Metal 430 Silicon 250 Dielectric 0.6</pre>
-output \$encrypted_conductivity.txt	
	Specifies the name of the encrypted output thermal file. The following is a snippet of the encrypted thermal conductivity file content: <pre>Metal \$encryption_value Silicon \$encryption_value Dielectric \$encryption_value</pre>

Examples

- The following command encrypts the specified input file (`format_1.txt`) to generate the encrypted output file (`format_1_encr.txt`):

```
encrypt_thermal_file -input encrypt/format_1.txt -output encrypt/format_1_encr.txt
```

The encrypted file (`format_1_encr.txt`) is then specified for generating a tile-based power map file, as shown below:

```
report_power \  
-outfile                static.rpt \  
-thermal_power_map_tile  {5 5}\  
-thermal_power_map_format stack \  
-thermal_power_map_file  powermap_format1_encr_in.txt \  
-thermal_conductivity_inputs ./encrypt/format_1_encr.txt
```

extract_metal_density

extract_metal_density
-help

Specifies to extract the metal density information. The command generates the Voltus Thermal Model (VTM) for Celsius only if you do not want to include power information. Before specifying this command, use the `set_metal_density_options` command to set the parameters for extracting the metal density information.

Parameters

-help	Outputs the command usage and a brief description about the command parameters.
-------	---

Examples

- The following command specifies to run extract metal density:
`extract_metal_density`

Related Topics

- `set_metal_density_options`

get_default_switching_activity

```
get_default_switching_activity
[-help]
[-clip_activity_to_domain_freq]
[-clock_gates_enable]
[-clock_gates_output]
[-clock_gates_output_ratio]
[-comb_clockgate_ratio]
[-duty]
[-global_activity]
[-icg_ratio]
[-input_activity]
[-macro_activity]
[-period]
[-seq_activity]
```

Displays the following information about a `set_default_switching_activity` parameter in the system log file and in the system console:

- Parameter name
- Current value

If you do not specify a parameter, the software displays information for all of the `set_default_switching_activity` parameters. The `get_default_switching_activity` command prints only the user-specified value for the parameters. If a parameter has not been set using the `set_default_switching_activity` command, the `get_default_switching_activity` command does not output any information in the console.

Parameters

<code>-help</code>	Outputs a brief description that includes type and default information for each <code>get_default_switching_activity</code> parameter. For a detailed description of the command and all of its parameters, use the man command: <code>man set_default_switching_activity</code> .
<code>parameter_names</code>	Displays information for the specified parameters. You can specify one or more parameters. See <code>set_default_switching_activity</code> for description of the parameters you can specify.

Examples

The following command displays the current setting of the `-period` parameter:

```
get_default_switching_activity -period
```

The software displays the following information:

```
-period {20}
```

get_dynamic_power_simulation

```
get_dynamic_power_simulation  
[-help]  
[-activity_pattern]  
[-period]  
[-resolution]
```

Displays the following information about a `set_dynamic_power_simulation` parameter in the system log file and in the system console:

- Parameter name
- Current value

If you do not specify a parameter, the software displays information for all of the `set_dynamic_power_simulation` parameters. The `get_dynamic_power_simulation` command prints only the user-specified value for the parameters. If a parameter has not been set using the `set_dynamic_power_simulation` command, the `get_dynamic_power_simulation` command does not output any information in the console.

Parameters

<code>-help</code>	Outputs a brief description that includes type and default information for each <code>get_dynamic_power_simulation</code> parameter. For a detailed description of the command and all of its parameters, use the man command: <code>man set_dynamic_power_simulation</code> .
<code>parameter_names</code>	Displays information for the specified parameters. You can specify one or more parameters. See <code>set_dynamic_power_simulation</code> for description of the parameters you can specify.

Examples

The following command displays the current setting of the `-period` parameter:

```
get_dynamic_power_simulation -period
```

The software displays the following information:

```
-period {20}
```

get_power_analysis_mode

```
get_power_analysis_mode
[-quiet]
```

Returns the following information about a `set_power_analysis_mode` parameter in the log file and in the software console:

- Parameter name
- Current value
- Type (Boolean, string, and so on)
- Whether the current value was set by user

If you do not specify a parameter, the software returns information for all of the `set_power_analysis_mode` parameters.

Parameters

- quiet	Returns the current settings of all <code>set_power_analysis_mode</code> parameters in Tcl list format.
------------	---

Example

- The following command returns the current settings for all `set_power_analysis_mode` parameters:

```
get_power_analysis_mode \
-honor_negative_energy true # bool, default=true \
-ignore_control_signals true # bool, default=true \
-leakage_scale_factor_for_temp 1 # float, default=1 \
-off_pg_nets {} # string, default="" \
-x_transition_factor 0.5 # float, default=0.5 \
-z_transition_factor 0.25 # float, default=0.25
```

- The following command returns the current settings for all `set_power_analysis_mode` parameters in Tcl list format only:

```
get_power_analysis_mode -quiet
```

The software returns the following information:

```
{honor_negative_energy true} {ignore_control_signals true} {leakage_scale_factor_for_temp 1} {off_pg_nets  
{}} {x_transition_factor 0.5} {z_transition_factor 0.25}
```

map_activity_file

```
map_activity_file
[-help]
[-gate_block block_name]
[-golden {rtl | gate}]
[-reset]
[-rtl2gate mapping_file]
[-rtl_block block_name]
[-two_column_mapping_file mapping_file]
```

Specifies instance name mapping between RTL netlist and GATE level netlist. Power analysis uses this instance name mapping to annotate the output nets of this instance from the pin-based RTL VCD or TCF. The mapping file used for instance name mapping is generated by Conformal.

You must specify this command before `report_power`.

Parameters

-help	Outputs a brief description that includes the type and default information for each <code>map_activity_file</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command <code>man map_activity_file</code>.
-gate_block block_name	
	Specifies the GATE activity block name to map the block-level RTL mapping files to the top-level RTL activity files. The GATE block name is appended before the GATE level entries in the mapping file to match the entries in the top-level RTL activity file.
-golden {rtl gate}	
	Specifies whether RTL or Gate is Golden. The one not specified as golden will be considered as Revised. Currently the Conformal MatchPoint file does not specify this. Default: rtl
-reset	Specifies to reset and ignore all the previous mapping files specified using the <code>map_activity_file</code> command. Default: All the previous mapping files specified using the <code>map_activity_file</code> command are loaded and used.
-rtl2gate mapping_file	

	Specifies instance name mapping between RTL netlist and GATE level netlist. RTL level VCD or TCF are obtained from simulation on RTL Verilog netlist. Power analysis uses this instance name mapping to annotate the output nets of this instance from the pin based RTL VCD or TCF. The mapping file used for instance name mapping is generated by Conformal.
<code>-rtl_block <i>block_name</i></code>	
	Specifies the RTL activity block name to map the block-level RTL mapping files to the top-level RTL activity files. The RTL block name is appended before the RTL level entries in the mapping file to match the entries in the top-level RTL activity file.
<code>-two_column_mapping_file <i>mapping_file</i></code>	
	Specifies a file that contains the instance name mapping between the RTL netlist and GATE level netlist. It also supports mapping between instance pin names in the GATE level netlist and instance names in the RTL netlist. This parameter can be used to specify a mapping file instead of the LEC generated mapping file (<code>-rtl2gate <i>mapping_file</i></code>) to map points from RTL to Gate netlist. The format of the two column mapping file is: <pre>#Gate name => # RTL name gate_inst_name1 gate_inst_pin_name1 => rtl_inst_name1 ...</pre> Here, the first column corresponds to the gate-level instance name or the instance pin name and the second column corresponds to the RTL-level instance name.

Example

- The following command specifies a mapping file `rtl_map`:

```
map_activity_file -rtl2gate rtl_map
```
- The following command performs instance name mapping between RTL netlist and GATE level netlist, wherein the GATE level netlist is Golden and the RTL netlist is Revised; it specifies the mapping file `MapFile.alt`

```
map_activity_file -rtl2gate MapFile.alt -golden gate
```
- The following commands appends the RTL block name and GATE block name before the RTL and GATE level entries in the mapping file to match the entries in the top-level RTL activity file:

```
map_activity_file -rtl2gate LEC.top \
    -rtl_block "core/block_top" \
    -gate_block "core/block_top"
```

```
map_activity_file -rtl2gate LEC.tile.top \
    -rtl_block "core/tile_instance\[0\]/tile_inst" \
    -gate_block "core/tile_instance_0_tile_inst"
```
- The following commands specify to ignore the previous mapping file (`old.map`), and use the modified mapping file (`updated.map`):

```
map_activity_file -rtl2gate old.map
```

```
map_activity_file -reset
map_activity_file -rtl2gate updated.map
```

An example of the mapping file output from Conformal is described as follows:

```
Mapped points: SYSTEM class
1-th mapped points:
  (G) + 1 PI /x_ick
  (R) + 595 PI /x_ick
2-th mapped points:
  (G) + 2 PI /x_jreset_cp_p
  (R) + 594 PI /x_jreset_cp_p
3-th mapped points:
  (G) + 3 PI /x_mreset_cp_p
  (R) + 593 PI /x_mreset_cp_p
.....
60-th mapped points:
  (G) + 4582 DFF /cpexec0/cpddecls0/gi_opls/q_reg_reg[35]
  (R) + 3856 DFF /cpexec0/cpddecls0/gi_opls/q_reg_reg_35_
61-th mapped points:
  (G) + 4583 DFF /cpexec0/cpddecls0/gi_opls/q_reg_reg[34]
  (R) + 3855 DFF /cpexec0/cpddecls0/gi_opls/q_reg_reg_34_
62-th mapped points:
  (G) + 4633 DFF /cpexec0/cpddecex0/gi_opex/q_reg_reg[21]
  (R) + 3805 DFF /cpexec0/cpddecex0/gi_opex/q_reg_reg_21_
63-th mapped points:
  (G) + 4634 DFF /cpexec0/cpddecex0/gi_opex/q_reg_reg[20]
  (R) + 3804 DFF /cpexec0/cpddecex0/gi_opex/q_reg_reg_20_
64-th mapped points:
  (G) + 4635 DFF /cpexec0/cpddecex0/gi_opex/q_reg_reg[19]
  (R) + 3803 DFF /cpexec0/cpddecex0/gi_opex/q_reg_reg_19_
65-th mapped points:
  (G) + 4636 DFF /cpexec0/cpddecex0/gi_opex/q_reg_reg[18]
  (R) + 3802 DFF /cpexec0/cpddecex0/gi_opex/q_reg_reg_18_
```

where, **G** refers to the Golden netlist name and **R** refers to the Revised netlist name.

By default, the tool assumes that the Golden netlist name refers to the RTL net name and Revised netlist refers to the GATE level net name.

Related Topics

- ["RTL Activity File Flow in Static Power Calculation"](#) in the *Voltus User Guide*

merge_vtm

```
merge_vtm
[-help]
-config filename
[-rundir directory]
[-merged_file filename]
[-format {simple | metal | stack}]
```

Specifies to import the top-level and other block-level Voltus thermal models (VTMs) having the specified design configuration to generate a merged uniform single VTM for Celsius usage.

Parameters

-help	Outputs the command usage and a brief description about the command parameters.
-config filename	Specifies the name of the design configuration file. The format of the file is: <pre>\$instance_name \$Design_name \$X(um),\$Y(um) \$Cell_orientation \$VTM_file format</pre> The file includes: • instance name • design name • X and Y locations of the instance • LEF/DEF style orientation for the instance • path to the thermal map file • format of the thermal map file Example: <pre>octa octa 0,0 N vtm_output_top/top.vtm.txt both inst_8 super_filter 62.97,47.035 N vtm_output_block/block.vtm.txt both inst_1 super_filter 1157.09,765.605 FE vtm_output_block/block.vtm.txt both inst_2 super_filter 777.085,765.605 E vtm_output_block/block.vtm.txt both inst_3 super_filter 397.085,765.605 W vtm_output_block/block.vtm.txt both inst_4 super_filter 17.085,765.605 FW vtm_output_block/block.vtm.txt both inst_5 super_filter 1127.97,47.035 S vtm_output_block/block.vtm.txt both inst_6 super_filter 772.97,47.035 FS vtm_output_block/block.vtm.txt both inst_7 super_filter 417.97,47.035 FN vtm_output_block/block.vtm.txt both</pre>
-format {simple metal stack}	

	Specifies the format of the thermal map file. The possible arguments are: • <code>simple</code>: contains only power tables. • <code>metal</code>: contains only metal density tables • <code>stack</code>: contains both power and metal density tables.
<code>-merged_file filename</code>	
	Specifies the name of the merged VTM.
<code>-rundir directory</code>	
	Specifies the output folder for the merged thermal model.

Examples

- The following command specifies to generate a merged VTM:

```
merge_vtm -config /Run_Merge_VTM/design_both.cfg -rundir merged_vtm_out_both -merged_file merge_vtm.txt
```
- The following command specifies to generate a merged VTM containing only power tables:

```
merge_vtm -config /Run_Merge_VTM/design_both.cfg -rundir merged_vtm_out_simple -merged_file merge_vtm.txt -
format simple
```
- The following command specifies to generate a merged VTM containing only metal density tables:

```
merge_vtm -config /Run_Merge_VTM/design_both.cfg -rundir merged_vtm_out_metal -merged_file merge_vtm.txt -
format metal
```
- The following command specifies to generate a merged VTM containing both power and metal density tables:

```
merge_vtm -config /Run_Merge_VTM/design_both.cfg -rundir merged_vtm_both_both -merged_file merge_vtm.txt -
format stack
```

Related Topics

- [set_metal_density_options](#)

propagate_activity

```
propagate_activity
-help
[-set_net_freq {true | false}]
```

Propagates the activity file in the database after it is read in using the [read_activity_file](#) command. This command propagates the activity for all primary inputs, nets, and other devices in the design whose activity has not been previously defined through user attributes.

`propagate_activity` can be used to propagate activities in the design and check for activity annotation before running power calculation. This is not a mandatory step as execution of `report_power` will also do the same and then do power calculation based on the propagated activities.

Parameters

- help	Prints out the command usage.
-set_net_freq {true false}	
	Specifies that activity from the generated TCF file needs to be used to capture net frequency for signal EM analysis. When this parameter is set, you do not have to read the TCF file to set net frequency.

Example

- The following commands read and propagate the activity file in the database:

```
read_activity_file
propagate_activity
```

Related Topics

- "[Signal ElectroMigration](#)" in the *Voltus User Guide*

query_power_data

```
query_power_data
[-help]
[<in_file <file1[<file2>] ...>>]
[-average_current]
[-clock_average]
[-dump_pwl]
[-frame <string>]
[-instance_list <inst_list>]
[-instance_list_file <filename>]
[-list]
[-output <string>]
[-output_tap_name <tap_name>]
[-peak_current]
[-peak_time]
[-pin]
[-stdout]
[-time_steps]
[-total_current <input_current_file> ]
```

The `query_power_data` command allows you to query the binary current files (.ptiavg/.ptipeak) that Power Calculation/Rail Analysis creates. It reads the tap current data in the current files and reports various information for debugging.

Parameters

- help	Outputs the command usage and a brief description about the command parameters.
<in_file <file1[<file2>] ...>>	
	Specifies the names of the VDD and VSS instance voltage pti files.
-average_current	
	Specifies to write the average current of all instances.
-clock_average	
	Specifies to write the clock based current report.
-dump_pwl	
	Specifies to write the tap current in the PWL format. You can use this parameter to view the contents of the binary data file.
-frame <string>	
	Specifies intervals for the clock_average report.

<code>-instance_list <inst_list></code>	
	Specify the list of instances to be dumped in the ASCII format, one instance per line. If not specified, it prints all instances. The <code>-instance_list</code> parameter is specified with the <code>-dump_pwl</code> , <code>-peak_current</code> , <code>-peak_time</code> , and <code>-average_current</code> parameters.
<code>-instance_list_file filename</code>	
	Specifies a file with the list of instances to be dumped in the ASCII format.
<code>-list</code>	
	Specifies to write the tap names from the input file.
<code>-output <string></code>	
	Specifies the name of the output file.
<code>-output_tap_name <tap_name></code>	
	Specifies the name of the output tap in the new current file, which is created using the <code>-total_current</code> parameter.
<code>-peak_current</code>	
	Specifies to write the peak current of all instances.
<code>-peak_time</code>	
	Specifies to write the peak current time of all instances.
<code>-pin</code>	
	Specifies to print the pin names with the instance names. The <code>-pin</code> parameter is specified with the <code>-dump_pwl</code> and <code>-list</code> parameters.
<code>-stdout</code>	
	Specifies to print the report to standard output.
<code>-time_steps</code>	
	Specifies to summarize the minimum current, maximum current, average current, and total current for each interval in the data file. You can use this data to estimate which interval in a transient response might contain the worst-case data.
<code>-total_current <input_current_file></code>	

Specifies to create a new current (.ptiavg) file containing the total current for all taps of a net. This parameter can be used if you have a very large current file generated by the Power Calculation engine and want to see only a total current waveform in the SimVision waveform viewer.

input_current_file is the current file that will be read in (output from the Power Calculation engine).

Use the following syntax to create a total current file:

```
query_power_data -total_current <input_current_file> -  
output <output_total_current_file>
```

 **-output** is used to specify a user-defined output current filename, and is not a mandatory parameter. If not specified, the output file will be generated automatically with the name of the input file suffixed with .ptiavg.

Example

- The following command creates a total current waveform file **VDD** from the input file **dynamic_VDD.ptiavg**:

```
query_power_data -output_tap_name vdd_tap -total_current dynamic_VDD.ptiavg -output VDD
```


read_activity_file

```
read_activity_file
name
[-help]
[-block block_name ]
[-cell cell_name ]
[-end { time1 time2 ...timen } ]
[-format { VCD | TCF | SAIF | FSDB | PHY | SHM}]
[-hier_separator separator ]
[-name_mapping_rule file]
[-reset]
[-rtl {true|false}]
[-scale_duration scalefactor ]
[-scope scope_name ]
[-set_net_freq {true | false}]
[-start { time1 time2 ...timen } ]
[-start_time_shift value ]
[-weight value]
[-zero_delay {true|false}]
```

Specifies the name and type of activity file to be used as input. This command must be used when specifying activity files for static or dynamic power calculation.

Parameters

-help	Prints out the command usage.
-block <i>block_name</i>	
	Specifies the name of the FSDB/SAIF/TCF/VCD block to map a sub-block activity file with a top level Verilog file. Provides the ability to support multiple activity files based upon block instance names. The block argument is added to the FSDB/SAIF/TCF/VCD name before comparing it to the Verilog file.
-cell <i>cell_name</i>	

	Specifies the master cell name for the activity file. This parameter allows you to support master-cell-based activity specification. That is, you need to specify only the master cell name for the activity file, instead of specifying all the instances that were instantiated from a master cell. Example Let us consider a design that has a cell named "super_filter" with four block instances "super_filter1", "super_filter2", "super_filter3", and "super_filter4": <u>Instance-Based Use Model:</u> <pre>read_activity_file -format VCD -block super_filter1 read_activity_file -format VCD -block super_filter2 read_activity_file -format VCD -block super_filter3 read_activity_file -format VCD -block super_filter4</pre> <u>Master-Cell-Based Use Model:</u> <pre>read_activity_file -format VCD -cell super_filter</pre>
	<pre>-end {time1 time2 ...timen }</pre>
	When you use the vector-driven method of performing dynamic instance-based power-consumption calculation, you can specify the part of the activity file from which you want power analysis to use for the power calculation. This option specifies the end time of this range and is used in conjunction with the -start option. Specifies the end time as time-value pairs to report the average power across non-overlapping multiple windows specified in the activity file for a block, level of hierarchy, or other part of the design that you want to analyze. Units are in seconds (s), milliseconds (ms), microseconds (us), nanoseconds (ns), or picoseconds (ps). Default: The time unit is defined in the first .lib file read during design import. You can specify the \$worst_power_window_end variable with the -end parameter to take the worst case end time from the last vector profiler run. Not required for TCF and SAIF. Note: You can read multiple windows in a single VCD file by specifying multiple pairs of the start time and end time for the non-overlapping multiple windows specified in the activity file. The use model for reading multiple windows is: <pre>read_activity_file -format VCD design.vcd -start {start1 start2 start3} -end {end1 end2 end3}</pre> The <i>start1/2/3</i> and <i>end1/2/3</i> values should be in the ascending order. Multiple current waveforms representing multiple VCD windows are merged into a single continuous current waveform. To capture the waveform for signals at the edge of user windows, a buffer time is added between the windows being stitched using the <code>set_power_analysis_mode -settling_buffer</code> parameter. This buffer time ensures that the stitched waveform is contiguous. The generated waveform hence has the duration of user windows and the intervening settling buffers. The default settling buffer period is 400ps.
	<pre>-format { VCD TCF SAIF FSDB PHY SHM }</pre>
	Specifies the net activity of the design using an activity file in the VCD, TCF, FSDB, SAIF, PHY, or SHM format. The arguments specified with the -format parameter are case-insensitive. • The VCD file format uses the toggle count information as the basis for the power consumption calculation. These files can be in compressed (zipped) format using the .gz extension. • The TCF file format contains data on the switching activity of the nets in the design. The switching activity includes the toggle count information and the probability of the net or pin being in the logic 1 state. These files can be in compressed (zipped) format using the .gz extension. Supports extended TCF which contains transition density for X and Z as shown in the following extended TCF

example:

```
"out" : "0.0 0" "0.005000 2" "0.0 0"
```

The fields are defined as follows:

Field 1: net or pin name

Field 2: duty and transition density for transition of 0-1

Field 3: duty and transition density for transition of X(0-X-1)

Field 4: duty and transition density for transition of Z(0-Z-1)

- The SAIF file format contains aggregate switching activity information on the nets and/or ports in the design. It does not contain time-based information. This file format is used for quick average power analysis of the design.
- The FSDB file format specifies the net activity of the design using an activity file in FSDB format. Voltus supports the FSDB version up to 6.0.

Use Model

```
read_activity_file -format FSDB fsdbfile
```

```
report_power
```

(or)

```
report_vector_profile
```

Example

```
read_activity_file -format FSDB test.fsdb -scope top
```

- The PHY (Palladium database format generated in the Palladium environment) format contains the simulation activity data generated during emulator-based simulation. The PHY format can be used in all vector-based power analysis flows (vector profiler, static power analysis and dynamic vector-based power analysis).

Use Model

```
read_activity_file-format PHY <path to PHY database directory>
```

Example

```
read_activity_file -format PHY ./phy2/trace.phy -scope test
```

Notes:

- The xeDebug software must be present in the `PATH` environment. Add xeDebug in the path using the following command:

```
set path = (<install_dir>/bin $path)
```

The xeDebug software version should be the same as the one using which the PHY database is generated.
- The PHY database generated from the following UXE/VXE version is supported:
UXE18.6.0 onwards
VXE18.6.0 onwards
- When reading a PHY format activity file, you require a DPA (DPA_ToggleCountFormat) license.
- For information on error messages displayed while reading the PHY database, refer to the `xe.msg` file in the PHY database directory.
- Make sure that the PHY database is not locked before invoking the Voltus software.
- Make sure that the xeDebug software is not already running on the machine.
- Ensure that the following directories are present in the top-level PHY database directory: `*.phy`, `PDB`,

	cellList, QTDB, dbFiles, and .design. - It is recommended that the write permission is assigned on the entire PHY database. In case the write permission cannot be granted, you must set the <code>SWFV_FREEZE_PHY</code> environment variable to 1. - If you want to run parallel Voltus executables on the same top-level PHY directory with multiple *.phy databases, it is suggested to create local copies of the multiple top-level directories, with links to the original PHY database. This will allow local copies of the PHY database to acquire separate locks. - The Simulation History Manager (SHM) database format is a record of the data signal changes that occur during design simulation. The SHM database consists of a directory, typically with the ".shm" suffix. The directory contains two disk files, namely *.trn and *.dsn. For more information about the SHM format, refer to the <i>SimVision Analysis Environment</i> manual available on the Cadence Online Support (COS) website. Example
<code>-hier_separator separator</code>	
	Specifies the separator character in the hierarchical net names, bus names, and pin names for the block or other part of the design that you want to analyze. It must be the same as the separator character in the design netlist. Default: slash (/)
<code>-name_mapping_rule file</code>	
	Specifies to enable rule-based mapping to map RTL vectors to gate-level netlist. This parameter helps in resolving name mismatch between RTL and gate level netlists. The name mapping rule file format is: <pre>Name_Mapping_Rule_file # RTL # Gate <rtlstring> <gatestring></pre> where, each line is a rule. The string in the first column(RTL) is what we want to replace with the string in the second column(Gate). The rules in every line are added onto the previous rules. For example, to replace all the square [] brackets with under score _ for RTL-to-Gate matching, the rule file will be: <pre>#RTL #GATE [_] _</pre> Note: <code>-name_mapping_rule</code> is not supported when the <code>set_power_analysis_mode - comprehensive_automapping</code> parameter is set to true.
<code>-reset</code>	If <code>-reset</code> is specified, all previous activity files specified using the <code>read_activity_file</code> command will be reset and ignored. Default: All the previous activity files specified using the <code>read_activity_file</code> command are loaded and used (They are additive).
<code>-rtl {true false}</code>	

	Specifies to process the vector as an RTL vector and perform state propagation. The propagation behavior is global and the full design is propagated. RTL vectors are zero-delay vectors so delay annotation is always done for signals from this vector file. Default: <code>false</code>
<code>-scale_duration scalefactor</code>	
	Scales the FSDB/SAIF/TCF/VCD duration value by the specified scale factor. For example, an FSDB scale of 2 will double the FSDB duration value. Note: The <code>-start</code> and <code>-end</code> parameters of the <code>read_activity_file</code> command are post scaled.
<code>-scope scope_name</code>	
	Specifies the FSDB/SAIF/TCF/VCD scope, that is, the name of the module within the activity file associated with the block or other part of the design that you want to analyze. The scope argument is removed from the FSDB/SAIF/TCF/VCD name before comparing it to the Verilog file.  If the scope value has backslash '\ ' or square brackets '[]', you must specify the scope within curly brackets <code>{{.....}}</code> to ensure that '\ ' or '[]' are correctly processed by the Tcl Interpreter. In the following example, the scope value is specified within curly brackets as it contains '\ ' and '[]': <pre>read_activity_file -scope {{reg_based_buffer.reg_inst\sv_xpol_trans_q_reg[0][14][7][sv_re] [4]}}</pre> If '\ ' and '[]' are not there, curly brackets are not required. Example The following command specifies the <code>u1</code> module within the VCD file as the module associated with the part of the design being analyzed, and specifies the <code>dut_5buf_full_chip</code> block instance as the block to map: <pre>read_activity_file \ -format VCD \ -scope adder/u1 \ -start {0ps 2000ps} \ -end {1100ps 3100ps} \ -block ../vcd/dut_5buf_full_chip</pre>
<code>-set_net_freq {true false}</code>	
	This option should be set when an activity from a TCF or VCD file needs to be used to capture net frequency for signal EM analysis (<code>verify_AC_limit</code>). In addition, you must use the <code>verify_AC_limit -use_db_freq</code> option to read the toggle rates from the activity file. Default: <code>false</code> Example The following example reads a TCF file which is used to capture net frequency for signal EM analysis: <pre>read_activity_file -format TCF -set_net_freq true design.tcf verify_AC_limit -use_db_freq -report signal_em.rpt</pre>
<code>-start { time1 time2 ... timen }</code>	

	When you use the vector-driven method of performing dynamic instance-based power-consumption calculation, you can specify the part of the activity file from which you want power analysis to use for the power calculation. This option specifies the start time of this range and is used in conjunction with the <code>-end</code> option. Specifies the start time as time-value pairs to report the average power across non-overlapping multiple windows specified in the VCD file for a block, level of hierarchy, or other part of the design that you want to analyze. Units are in seconds (s), milliseconds (ms), microseconds (us), nanoseconds (ns), or picoseconds (ps). Default : The time unit is defined in the first .lib file read during design import. You can specify the <code>\$worst_power_window_start</code> variable with the <code>-start</code> parameter to take the worst case start time from the last vector profiler run. Not required for TCF and SAIF. Note: You can read multiple windows in a single VCD file by specifying multiple pairs of the start time and end time for the non-overlapping multiple windows specified in the activity file. The use model for reading multiple windows is: <pre>read_activity_file -format VCD design.vcd -start {start1 start2 start3} -end {end1 end2 end3}</pre> The <code>start1/2/3</code> and <code>end1/2/3</code> values should be in the ascending order. Multiple current waveforms representing multiple VCD windows are merged into a single continuous current waveform. To capture the waveform for signals at the edge of user windows, a buffer time is added between the windows being stitched using the <code>set_power_analysis_mode -settling_buffer</code> parameter. This buffer time ensures that the stitched waveform is contiguous. The generated waveform hence has the duration of user windows and the intervening settling buffers. The default settling buffer period is 400ps.
	<code>-start_time_shift value</code>
	Specifies to shift the start time of the specified activity files. When you specify the <code>-scale_duration</code> parameter with the <code>-start_time_shift</code> parameter, it scales the start time shift along with the start/end time.
name	The name of the activity file.
	<code>-weight value</code>
	Specifies the weight number of the TCF file. 1.0 is the default value. The value should be a floating number that is smaller than 1.0. This parameter allows you to merge multiple TCF files at different function modes for static power estimation of a design. In the following example, TCF <i>file1</i>, <i>file2</i>, and <i>file3</i> has the weight numbers 0.3, 0.2, and 0.4, respectively. <pre>read_activity_file -format TCF file1 -weight 0.3 read_activity_file -format TCF file2 -weight 0.2 read_activity_file -format TCF file3 -weight 0.4</pre> Here, <i>file1</i>, <i>file2</i>, and <i>file3</i> will be merged together to represent the toggle and duty for the nets/pins in the design. The relative weight of 0.3/0.9, 0.2/0.9, and 0.4/0.9 would be used for static power calculation. In this example, 0.9 is the sum of weights.
	<code>-zero_delay {true false}</code>
	Specifies to process the vector as a zero-delay vector and perform delay annotation. Default : false

Activity Precedence

The following commands can define activity:

```
read_activity_file file
    [-format { VCD | TCF | SAIF | FSDB }]
    [-hier_separator separator ]
    [-start time ]
    [-end time ]
    [-reset]
    [-scope scope_name ]
    [-block block_name ]

set_default_switching_activity
    [-input_activity factor ]
    [-seq_activity factor ]
    [-period value ]
    [-duty value ]
    [-global_activity factor ]
    [-hier hierarchy _name]
    [-clock_gates_output activity_factor ]

set_switching_activity
    [-reset]
    [-clock clock_name]
    [-period value ]
    [-unclocked]
    [-activity factor | -density transition_density ]
    [-net net_name | -port port_name | -pin pin_name ]
    [-duty value ]
    [-inst]
```

The precedence of activity is as follows:

1. User-Defined: Activities applied to specific nets/pins/ports
2. Activity File: Activity specification through a VCD/FSDB/SAIF/TCF format
3. Clock Gates Output Activity: Activity specification to the output of all clock gates
4. SDC/TWF constants: Set constants to a list of pins or ports for use by the timing engine (timing analysis)
5. Hierarchical Global Activity: Activity specification for a specific hierarchy
6. Global Activity: Activity specification for all instances that are part of the data network.
7. Sequential Element Activity: Activity specification applied to the output of all sequential elements.
8. Primary Input Activity: Activity specification for all primary inputs

If you specify multiple activity data for the same items, the power engine will use the one with the higher precedence.

Examples

- The following command reads the compressed ap_wait_test_pll1_mod.vcd.gz VCD file that contains the toggle count information for the power consumption calculations, specifies the crm_ap module within the VCD file as the module associated with the part of the design being analyzed, specifies the slash character (/) as the hierarchical separator for hierarchical net names, bus names, and pin names for the part of the design being analyzed, and specifies the start (180002ns) and end (189802ns) times from the VCD file for the part of the design being analyzed:

```
read_activity_file -format VCD \  
-scope testbench/top/ap/mcu_platform/crm_ap \  
-hier_separator \  
-start 180002ns \  
-end 189802ns \  
ap_wait_test_pll1_mod.vcd.gz
```

- The following command reads the compressed A.vcd.gz VCD file that contains the toggle count information for the power consumption calculations, specifies A1 as the block to map a sub-block VCD file with a block instance at the top-level, and specifies the start (180002ns) and end (189802ns) times from the VCD file for the part of the design being analyzed:

```
read_activity_file -format VCD \  
-block ap/A1 \  
-start 180002ns \  
-end 189802ns \  
A.vcd.gz
```

- The next example specifies a VCD activity file called dmac_mac.vcd for the scope top/dma_dut.

```
read_activity_file -format VCD -start 10ns -end 20ns \  
-scope top/dma_dut dmac_mac.vcd
```

- The following command specifies multiple pairs of the start time and end time for the multiple windows in the VCD file:

```
read_activity_file \  
-format VCD \  
-scope adder/u1 \  
-start {0ps 2000ps} \  
-end {1100ps 3100ps} \  
-block ../vcd/dut_5buf_full_chip.vcd
```

- The following command specifies to read the RTL-level VCD activity file:

```
read_activity_file -format VCD -scope scope_top -start 300ns -end 700ns /DesignData/VCD/rtl.vcd -block  
inst_C1 -rtl true  
set_power_analysis_mode -method vector_profile -vector_profile_mode transient -worst_window_type full -  
power_grid_library /DesignData/PGV/library_pv.cl
```

Related Topics

- "[Dynamic Power and IRDrop Analysis](#)" in the *Voltus User Guide*

report_annotated_parasitics

```
report_annotated_parasitics
[-help]
[{-list_annotated}
[-list_broken_net]
[-list_extra_pin_net]
[-list_float_net]
[-list_nodriver_net]
[-list_noload_net ]
[-list_not_annotated ]
[-list_real_net ]
[-list_supply_net ]
[-list_zero_cap_net]
[-max_missing {max_missing_int}]]]
[-net_file string]
[-nets string]
[-view {viewName}]
[> {write_fileName}]
[>> {append_fileName}]
```

Reports the back-annotated parasitics of the design. By default only summary report is generated. The software groups all the nets by net-types and annotation-status (annotated or not-annotated).

Parameters

-help	Outputs a brief description that includes type and default information for the <code>report_annotated_parasitics</code> parameters. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <code>manreport_annotated_parasitics</code> .
>> {append_fileName}	Appends the report to the specified file name.
-list_annotated	Generates a list of annotated nets.
-list_broken_net	Reports broken nets in the design.
-list_extra_pin_net	Generates a list of nets that have extra pins in the SPEF but not in the Verilog.
-list_float_net	Generates a list of floating nets.
-list_nodriver_net	Lists one terminal nets that have no driver.
-list_noload_net	Lists one terminal nets that have no load.
-list_not_annotated	Generates a list of not annotated nets.
-list_real_net	Lists all real nets.
-list_supply_net	Lists all supply nets.
-list_zero_cap_net	Generates a list of zero capacitance nets.
-max_missing {max_missing_int}	Limits the number of missing annotations to the specified number of annotations. <i>Default: 1000</i>
-nets string	Specifies the list of net names for which annotation summary is required.
-net_file filename	Accepts a file name, which contains a list of net names, as input. When this parameter is specified, the parasitic annotation summary is provided only for the listed nets. The net names in the file can be specified as one net per line.
-view viewname	Reports the back-annotated parasitics for the specified analysis view. You can specify this parameter only when the software is in multi-mode multi-corner timing analysis mode. <i>Default: Reports against the default analysis view</i>
> {write_fileName}	Writes the report to the specified file name.

Example

- The following command generates a list of annotated and not-annotated nets:

```
report_annotated_parasitics -list_annotated -list_not_annotated -list_real_net -list_float_net -
list_nodriver_net \ + -list_noload_net -list_supply_net -max_missing 1000

# list syntax: type: net_name [nrCap Cap nrXCap XCap nrRes Res]
# unit: pF, Ohm
```

Voltus IC Power Integrity Solution Text Command Reference

Power Calculation Commands

```
# List the first 82 of total 82 real nets:
Annotated Real Net: [seg3/w7] [ 10 0.0690723 25 0.0512442 9 219.251 ]
Annotated Real Net: [seg3/w5] [ 14 0.0373353 24 0.0265601 13 161.843 ]
:
:
Annotated Real Net: [CLK3] [ 2 0.00046867 0 0 1 18.872 ]
Annotated Real Net (broken): [CLK4] [ 1 0.0002379 0 0 2 0.0002 ]
# List the first 20 of total 20 not annotated real nets:
Not Annotated Real Net: [seg3/wout]
Not Annotated Real Net: [CLK44]
:
:
Not Annotated Real Net: [out1]
Not Annotated Real Net: [in1]
# No annotated supply net.
# List the first 4 of total 4 not annotated supply nets:
Not Annotated Supply Net: [gnd]
Not Annotated Supply Net: [GND]
Not Annotated Supply Net: [VDD]
Not Annotated Supply Net: [vdd]
# No annotated floating net.
# No not annotated floating net.
# No annotated no driver net.
# List the first 9 of total 9 not annotated no driver dangling nets:
Not Annotated Dangling Net (no driver): [seg3/n8]
Not Annotated Dangling Net (no driver): [seg3/n111111]
:
:
Not Annotated Dangling Net (no driver): [in]
Not Annotated Dangling Net (no driver): [out5]
# List the first 8 of total 8 annotated no load dangling nets:
Annotated Dangling Net (no load): [seg3/n14] [ 1 0.00520893 0 0 0 0 ]
Annotated Dangling Net (no load): [seg2/n14] [ 1 0.00636874 0 0 0 0 ]
:
:
Annotated Dangling Net (no load): [n6] [ 2 0.1086 0 0 1 0.0001 ]
Annotated Dangling Net (no load): [n5] [ 2 0.1065 0 0 1 0.0001 ]
# List the first 16 of total 16 not annotated no load dangling nets:
Not Annotated Dangling Net (no load): [seg3/u14:QN]
Not Annotated Dangling Net (no load): [seg3/u9:QN]
:
:
Not Annotated Dangling Net (no load): [u2:QN]
Not Annotated Dangling Net (no load): [in4]
# Summary of Annotated Parasitics:
+-----+
| Net Type          | Count | Annotated (%) | Not Annotated (%) |
+-----+-----+-----+-----+
| total              | 135   | 90 66.7%      | 45 33.3%          |
+-----+-----+-----+-----+
| no drive (*)       | 9     | 0 0.0%        | 9 100.0%          |
| 1-term:no load     | 24    | 8 33.3%       | 16 66.7%          |
| real net (complete) | 81    | 61 75.3%     | 20 24.7%          |
| real net (broken)  | 21    | 21 100.0%    | 0 0.0%            |
+-----+-----+-----+-----+
```

Voltus IC Power Integrity Solution Text Command Reference

Power Calculation Commands

Note for Net Types marked "(*)": Such nets are never timed, but reported here for informational purpose.

+-----+				
Annotated	Res (KOhm)	Cap (pF)	XCap (pF)	
+-----+-----+-----+-----+				
Count	982	1011	1152	
Value	20.0869	7.7289	5.4598	
+-----+				

report_instance_power

```
report_instance_power  
[-help]  
instance  
[-file filename]  
[-inst_file filename]
```

Generates a text based report for the specified instance. This command enables you to determine how the different components of static power are calculated. You can use the report to debug static power numbers after performing power analysis (`report_power`).

This command is available through both the static and dynamic power engines. This command is supported in the dynamic power engine by including this command in the power include file through the command `set_power_include_file`.

Note: For dynamic power engine, the `report_instance_power` command does not generate text reports incrementally. If the dynamic run has been performed and you need the debug information for a static power number, you should re-run power analysis.

 The `report_instance_power` command is not supported in the event-based power analysis flow.

Parameters

<code>-file filename</code>	
	Writes out the static power analysis report into the file that you specify.
<code>-help</code>	Outputs the command usage and a brief description about the command parameters.
<code>instance</code>	
	Specifies the instances to include in the instance power report.
<code>-inst_file filename</code>	
	Specifies a file that includes a list of instances. The parameter allows you to report and debug power numbers for multiple instances. A sample file is given below: <pre>INST1 INST2 INST3</pre>

Example

- The following command specifies an instance power report file `debug.rep` for the instance `top/i_1`:

```
report_instance_power -file debug.rep top/i_1
```

Related Topics

- "[Static Power, IRdrop and EM Analysis](#)" in the *Voltus User Guide*
 - "Debugging Instance Power"

report_power

```
report_power
[-help]
[-block block_name]
[-cap]
[-cap_unit unit]
[-cell { cell_list } ]
[-cell_fit_file filename]
[-cell_list_file filename]
[-cell_type {all | {macro io combinational sequential clock_combinational clock_sequential} }}]
[-clock_domain names ]
[-clock_gating_efficiency]
[-clock_network {all | { clock_list } }}]
[-cluster_gating_efficiency]
[-comb_seq_power]
[-compress compression_ratio]
[-count_seq_elements_in_clock_network]
[-distribute]
[-die_instance_name dieinstname]
[-exclude_cells {cell_list}]
[-exclude_cells_file filename ]
[-exclude_instances {instance_list}]
[-exclude_instances_file filename ]
[-float_precision value]
[-format { simple | detailed }}]
[-group_type user_defined_group_name]
[-hierarchical_instances {inst1 inst2 inst3 ...}]
[-hierarchy {all | hierarchy_level} ]
[-include_group_instance_count]
[-inst_list_file filename]
[-instances { instance_list } ]
[-leakage]
[-net [-nworst number_of_nets]]
[-no_wrap]
[-o directory]
[-outfile name]
[-output directory]
[-output_cell_fit_file filename]
[-pg_net {all | pg_net_name_list }}]
[-pg_pin]
[-power_db_directory directory]
[-power_density_tiles value]
[-power_density_tiles_row_col {value1 value2}]
[-power_density_tiles_size {value1 value2}]
[-power_domain {all | { power_domain_list } } ]
[-power_unit unit ]
[-print_memory_power]
[-register_gating_efficiency]
[-report_prefix prefix]
[-sort {internal | switching | leakage | total}}]
[-stat]
[-thermal_conductivity_inputs file_name]
[-thermal_leakage_temp {temp1 temp2 temp3 ...}]
```

```
[-thermal_leakage_temperature_scale_table { $temp1 $scalefactor1 $temp2 $scalefactor2 $temp3 $scalefactor3
.....}]
[-thermal_leakage_temperature_scale_table_file filename]
[-thermal_material_file filename]
[-thermal_power_map_bottom_layer layer_name]
[-thermal_power_map_densitytable_tile {Xint Yint}]
[-thermal_power_map_file file_name]
[-thermal_power_map_format {simple | stack }]
[-thermal_power_map_format_version {new|default}]
[-thermal_power_map_header_include_file filename]
[-thermal_power_map_powertable_tile {Xint Yint}]
[-thermal_power_map_SI_unit {true | false}]
[-thermal_power_map_tile {Xint Yint}]
[-thermal_power_map_top_layer layer_name]
[-threshold value ]
[-threshold_voltage_group { all | group_name }]
[-time_based_report]
[-time_unit unit]
[-toggle_rate]
[-use_geometry_cell_size {true | false}]
[-view view_name ]
```

Reports global as well as specific instance power, clock network power, clock domain power, and net switching power. It also reports the power for power domains and specific power nets. Units are reported in milliwatts.

This command starts the execution of power analysis (both static and dynamic). The `report_power` command has a character limit of 1015.

 The parameters of the `report_power` command, except `-cap`, `-clock_gating_efficiency`, `-register_gating_efficiency`, and `-cluster_gating_efficiency`, are not supported in the dynamic only mode (`set_power_analysis_mode -disable_static true`). These parameters are only supported in the static mode. However, the `-power_domain` parameter is not supported even if the static power mode is enabled. You can use the `set_power_include_file` command to pass the `report_power` command options in the dynamic analysis mode.

The `report_power` command allows generation of multiple power reports with a single `report_power` command. This allows you to generate all the static reports during dynamic analysis, and removes the need to run incremental `report_power` in static power analysis using a Verilog netlist. By default, a detailed power summary report, and a short summary will be printed at the beginning of all reports. To enable this flow, you need to specify the following parameters:

- `-output/o`
- `-report_prefix`

The following `report_power` command parameters are supported by the multi-report flow, both in static and dynamic power analysis:

- `-cell`
- `-clock_gating_efficiency`
- `-register_gating_efficiency`
- `-cluster_gating_efficiency`

- `-net`
- `-instances`
- `-pg_net`
- `-clock_network`
- `-hierarchy`

The following parameters are not supported in the multi-report flow:

- `-cell_type`
- `-power_domain`
- `-clock_domain`
- `-clock_gating_efficiency` and `-clock_network` in the same run

 In the static power analysis flow, you can run the `report_power` command after the `restore_power_database` command without loading the design. The following command is an example of the use model:
`restore_power_database -file power.db`
`report_power -output same -report_prefix report -clock_network {all} -instances {*}`

The following parameters of the `report_power` command are supported without design loading:

- `-cell`
- `-cell_type`
- `-instances`
- `-clock_network`
- `-clock_domain`
- `-pg_net`
- `-pg_net -format detailed`
- `-hierarchy`
- `-hierarchical_instances`

Parameters

<code>-help</code>	Prints out the command usage.
<code>-block <i>block_name</i></code>	

	Specifies to generate the different power analysis reports for a block. This parameter is useful in viewing and debugging power consumption issues for a specific block after the power run is complete. Example: The following command will create the summary and pg_net reports for the design block {A/B}: <pre>report_power -block {A/B} -outfile blockB.rpt -pg_net -pg_pin {all} -instances {*}</pre> Following are the sample block-specific reports: <pre>blockB.rpt.avppower blockB.rpt.pgnetpwr blockB.rpt.pgpinpwr blockB.rpt.instpwr</pre>
-cap	Dumps out the list of net, pin, and total capacitances in three different columns.
<code>-cap_unit unit</code>	
	Sets the capacitance unit to be used for power report generation. The default unit is pF (picoFarads). By default, the capacitance unit in <code>report_power</code> is Farad (F). To get the capacitance in pF, use the command <code>report_power -cap_unit pf</code>.
<code>-cell { cell_list }</code>	
	Specifies the cells to include in the power report. Accepts wildcards (*)
<code>-cell_fit_file filename</code>	
	Specifies a file that accepts a cell-based FIT input table with two columns: cell name and FIT value. This parameter is used to generate an instance-level FIT report based on the cell-level data. Instance-based FIT report is supported only in the XP mode. The format of the file is: File Format Example for the FIT Input File: <pre>##Cell Name FIT ----- CellA .25 CellB .5</pre>
<code>-cell_list_file filename</code>	
	Specifies a file containing a list of cells that are to be included in the power reports.
<code>-cell_type {all {macro io combinational sequential clock_combinational clock_sequential}}</code>	
	Specifies the cell type to include in the power report. Default: all When you specify the cell types, <code>-cell_type combinational</code> or <code>sequential</code>, the power report now does not show the clock combinational or clock sequential cells.
<code>-clock_domain names</code>	

	Generates detailed instance power reports for the specified clock domains. Wildcards are not accepted. When this parameter is not specified, the <code>report_power</code> command provides a summary of clock power for instances with the clock pins driven by a clock domain. In contrast, <code>report_power -clock_domain</code> reports a clock power summary for instances with any pin driven by a clock domain, provided the instance is part of the clock network. Usage <pre>report_power -clock_domain name -outfile filename</pre>
<code>-clock_gating_efficiency</code>	
	Generates a report that includes the Clock Gating Efficiency (CGE) for all clock gating instances as well as different hierarchies in the design. Average CGE for different hierarchies is also reported. Clock Gating Efficiency is calculated as: $CGE = (\text{toggles at clock gate output} / \text{toggles at clock gate input})$ $\text{Average CGE} = (\text{avg of all CGEs above})$ When you generate a CGE report using the command <code>report_power -clock_gating_efficiency -outfile cge.rpt</code>, the following information will be displayed: • Clock-gating Efficiency Report - for each clock domain, it includes the toggle rate, number of registers, number of clock gates, average clock toggle at registers, average toggle savings at registers, and average toggle savings histogram. • Hierarchical View of Average Toggle Savings - number of clock gates and average toggle savings for each hierarchical module in the design
<code>-clock_network {all { clock_list } }</code>	
	Reports the power consumption of the clock network, including the power for generated clocks. Use <code>clock_list</code> to report the power consumed by specific clocks. Default: <code>all</code>
<code>-cluster_gating_efficiency</code>	
	Specifies to generate the cluster efficiency report in the RGE report. This report gives the CGE/RGE metrics for the registers at the fanout of each clock gate instance in the design.
<code>-comb_seq_power</code>	
	Specifies to generate a report that gives the ratio of combinational versus sequential power for all modules of a given hierarchy. You must specify this parameter with the <code>-instances</code> parameter.
<code>-compress compression_ratio</code>	
	Specifies to compress the power analysis report files in the GNU Zip format. The compressed files will have the suffix <code>.gz</code>. <code>compression_ratio</code> is from 0-9. 0 indicates that the report files are not to be compressed. The compression levels vary from 1 to 9. 1 provides the fastest compression speed but at a lower ratio, so the file size will be larger. 9 provides the highest compression ratio but at a lower speed. Default: 0

Voltus IC Power Integrity Solution Text Command Reference
Power Calculation Commands

<code>-count_seq_elements_in_clock_network</code>	
	Includes the leaf flip-flop power in the clock network power report. <i>Default:</i> The leaf flip-flop power will not be included in the clock network power report.
<code>-die_instance_name <i>dieinstname</i></code>	
	Specifies the die instance name for multi-die analysis.
<code>- distribute</code>	Enables standalone distributed power analysis. This parameter allows you to perform dynamic power analysis in the multi-CPU mode without running rail analysis.
<code>-exclude_cells {<i>cell_list</i>}</code>	
	Specifies a list of cells that should be excluded from power analysis. This parameter is supported in both Verilog and DEF-based flows with the <code>set_power_analysis_mode</code> set to either <code>-method static</code> or <code>-method event_based</code> .
<code>-exclude_cells_file <i>filename</i></code>	
	Specifies a file containing a list of cells (one per line) which should be excluded from power analysis. This parameter is supported only in the DEF-based flow with the <code>set_power_analysis_mode</code> set to either <code>-method static</code> or <code>-method event_based</code> .
<code>-exclude_instances {<i>instance_list</i>}</code>	
	Specifies a list of instances that should be excluded from power analysis. This parameter is supported only with the following method: <code>set_power_analysis_mode -method event_based</code> .
<code>-exclude_instances_file <i>filename</i></code>	
	Specifies a file containing a list of instances (one per line) which should be excluded from power analysis. This parameter is supported only with the following method: <code>set_power_analysis_mode -method event_based</code> .
<code>-float_precision <i>value</i></code>	
	Sets the precision value to be used for power report generation. The default value is 4.
<code>-format { <i>simple</i> <i>detailed</i> }</code>	

	Specifies the type of report summary required for static power reporting. The possible arguments of this parameter are: • <code>simple</code> - reports the amount of power consumed by all power nets in a condensed format, breaking down power based on the switching/internal/leakage components. • <code>detailed</code> - reports the amount of power consumed by all power nets in a detailed format. The detailed format includes additional information, such as output pin capacitance (<code>Cpin Sum</code>), output net capacitance (<code>Cnet Sum</code>), Total Load Capacitance (<code>Cpin Sum + Cnet Sum = Total Load_Cap</code>), maximum frequency at the input and output pins (<code>Max_Clk_Freq</code>), maximum activity factor at the input and output pins (<code>Max_Activity</code>), cell name of the instance (<code>Cell Name</code>), clock or data cell (<code>C/D</code>), and library used in power analysis for the specific cell (<code>Lib</code>). The libraries are numbered in the header section. The name of the generated report file is <code>report.*.detailed.rpt</code>. Currently, the <code>-format detailed</code> report is only available for the <code>-pg_net</code> report.
<code>-hierarchical_instances {inst1 inst2 inst3 ...}</code>	
	Specifies to report all the power content for the specified hierarchical instances/ blocks. It reports the <code>Total Power</code> (a breakup of power components) and <code>Group</code> (a breakup of power by cell type) of the specified block at each hierarchy. To generate a complete hierarchical instance power report, it is recommended to specify <code>-hierarchical_instances</code> with <code>-report_prefix</code>. The report is written in the specified output directory. If you specify <code>-outfile</code> instead of <code>-report_prefix</code>, an instance file is output without hierarchy-level details. <code>-report_prefix</code> generates the following 3 reports: • <code><prefix>.rpt</code> - reports power details for all instances in the design. • <code><prefix>.hierpwr.rpt</code> - reports hierarchical power, that is, power summary, internal, switching, leakage, and total power for all hierarchical instances. • <code><prefix>.hierinstpwr.rpt</code> - reports power details for the requested block instances. Note: <code>-hierarchical_instances</code> must be specified with <code>-hierarchy</code> to include hierarchy-level power report. When <code>-hierarchical_instances</code> is used without <code>-hierarchy</code>, only top-level summary of the power consumption is written.
<code>-hierarchy {all hierarchy_level }</code>	
	Specifies the hierarchy level that is included in the power report. You can specify the <code>hierarchy_level</code> as a number that corresponds to a specific hierarchy level in the design. A <code>hierarchy_level</code> of 0 specifies the top level. <i>Default:</i> <code>all</code>
<code>-include_group_instance_count</code>	
	Specifies to report the instance count for each cell type. When specified, you can view the number of instances for the sequential, macro, IO, combinational, clock sequential, and clock combinational cell types in the power report (<code>power.rpt</code>). The parameter is supported in both the static and dynamic power analysis flows. This parameter allows you to verify if there are any missing instances.
<code>-instances {instance_list}</code>	

	Specifies the instances to include in the power report. The power report provides the amount of power consumed by a specified list of hierarchical or leaf-level instances. <i>instance_list</i> accepts wildcards (*) for leaf-level instances only. An instance with 0 power will not be listed in the power report.																								
<code>-inst_list_file filename</code>																									
	Specifies a file containing a list of instances that are to be included in the power reports.																								
<code>-leakage</code>	Reports only leakage power of the design.																								
<code>-net</code>	Reports the net switching power, as well as the load capacitance, toggle rate, and switching values for each net in the design.																								
<code>-no_wrap</code>	Displays the <code>report_power -cell_type all</code> output on a single line in the report output file. Example <table><tr><td>Instance</td><td>Toggle</td><td>Internal Power</td><td>Switching Power</td><td>Leakage Power</td><td>Total Power</td><td>Percentage (%)</td><td>Cell Name</td></tr><tr><td colspan="8">-----</td></tr><tr><td>dma/ram0</td><td>7.102e+08</td><td>5.58</td><td>0.002307</td><td>0.02424</td><td>5.607</td><td>18.19</td><td>/data/cellA</td></tr></table>	Instance	Toggle	Internal Power	Switching Power	Leakage Power	Total Power	Percentage (%)	Cell Name	-----								dma/ram0	7.102e+08	5.58	0.002307	0.02424	5.607	18.19	/data/cellA
Instance	Toggle	Internal Power	Switching Power	Leakage Power	Total Power	Percentage (%)	Cell Name																		

dma/ram0	7.102e+08	5.58	0.002307	0.02424	5.607	18.19	/data/cellA																		
<code>-nworst number_of_nets</code>																									
	Provides n top windows with maximum activity or power in the vector profile report. The default value is 10. Note: You must use the <code>-nworst</code> parameter in conjunction with the <code>-net</code> parameter.																								
<code>-outfile filename</code>																									
	Specifies the name of the report file in which the top-level summary of the power consumption information is written. You can include the directory name. You can convert a power report file to the gzip file format by specifying <code>- outfile filename.gz</code> . Note: If you do not specify this parameter, the software directs the power report to the command line, as well as the log file, and no power report is written to a separate file.																								
<code>-output/-o directory</code>																									
	Specifies the name of the power analysis output directory that saves the power reports. The default is the current working directory. This parameter replaces the need to specify <code>set_power_output_dir</code> ; if both are specified, <code>-output</code> has the higher priority.																								
<code>-output_cell_fit_file filename</code>																									
	Specifies the name of the output file for the generated FIT report, which includes three columns: instance name, corresponding cell name, and computed FIT value. This parameter is specified with the <code>-cell_fit_file</code> parameter. The output file is generated in the power output directory. File Format Example for the FIT Output File: <table><tr><td>#Instance Name</td><td>Cell Name</td><td>FIT</td></tr><tr><td colspan="3">#####</td></tr><tr><td>inst_A/al_inst/g5</td><td>INV_X1A_A9TL</td><td>0.25</td></tr><tr><td>inst_A/al_inst/g8</td><td>INV_X1B_A9TL</td><td>0.25</td></tr><tr><td colspan="3">....</td></tr><tr><td colspan="2">Total FIT:</td><td>115</td></tr></table>	#Instance Name	Cell Name	FIT	#####			inst_A/al_inst/g5	INV_X1A_A9TL	0.25	inst_A/al_inst/g8	INV_X1B_A9TL	0.25				Total FIT:		115						
#Instance Name	Cell Name	FIT																							
#####																									
inst_A/al_inst/g5	INV_X1A_A9TL	0.25																							
inst_A/al_inst/g8	INV_X1B_A9TL	0.25																							
....																									
Total FIT:		115																							

<code>-pg_net {all pg_net_name_list }</code>	
	When set to <code>all</code>, reports the amount of power consumed by all power nets. If you specify a list, reports the power consumed by each instance that is connected to the specified list of power nets. Note: When using this parameter, the static power calculation engine always performs propagation and power calculation. <i>Default:</i> <code>all</code>
<code>-pg_pin</code>	Reports power consumed by all PG pins in a design.
<code>-power_db_directory directory</code>	
	Specifies the location of the power database that the Power Calculation engine uses for generating standalone power reports. Note: Before specifying the <code>-power_db_directory</code> parameter, ensure that a power database is created in the binary format by using the <code>set_power_analysis_mode -create_binary_db true</code> parameter.
<code>-power_domain {all { power_domain_list } }</code>	
	Specifies all power domains or lists power domains to be included in report. It prints a summary of power break down for each domain before listing power for each instance in the domain.
<code>-power_density_tiles value</code>	
	Specifies the number of tiles used for power density calculation. <i>Default:</i> <code>100</code> Note: When dividing a design into equal-sized tiles, it might not always fit perfectly into a square grid. For example, if you want to create NxN tiles for the top design, the design's size might not align exactly with these dimensions. This means the tiles in the last row and the last column could be different in size to cover the remaining part of the design. All other tiles, however, in the (N-1)x(N-1) area, will be the same size.
<code>-power_density_tiles_row_col {value1 value2}</code>	

Defines the number of tiles by rows * columns. *value1* is the number of tile rows and *value2* is the number of tile columns.

The following example specifies to perform power density aware vector profiling using 2 tile rows and 2 column rows:

```
set_power_analysis_mode -method vector_profile -vector_profile_mode power_density -
worst_step_size 10ns
report_power -power_density_tiles_row_col {2 2} -time_based_report -outfile tiling1.rpt
```

The following is the vector profile report for this command:

```
Tile 0_0_0 5.06 5.33 14.95 14.555
Tile 1_0_1 14.95 5.33 24.84 14.555
Tile 2_1_0 5.06 14.555 14.95 23.78
Tile 3_1_1 14.95 14.555 24.84 23.78
...
...
```

Note: When dividing a design into equal-sized tiles, it might not always fit perfectly into a square grid. For example, if you want to create NxN tiles for the top design, the design's size might not align exactly with these dimensions. This means the tiles in the last row and the last column could be different in size to cover the remaining part of the design. All other tiles, however, in the (N-1)x(N-1) area, will be the same size.

`-power_density_tiles_size {value1 value2}`

Defines the absolute tile size. *value1* is the horizontal width of a tile (in micron) and *value2* is the vertical height of a tile (in micron).

`-power_unit unit`

Sets the power unit to be used for power report generation. The default unit is mW (miliAmps).

`-print_memory_power`

Specifies to report power values for the memory and macro cells under separate groups (Memory and Macro).

The following is a snapshot of the power report showing separate groups for memory and macro cells:

Group	Internal Power	Switching Power	Leakage Power	Total Power	Percentage (%)
Sequential	0.4031	0.1283	0.006583	0.538	18.38
Memory	0.1624	0.004345	0.011	0.1778	6.074
IO	0	0	0	0	0
Physical-Only	0	0	2.91e-06	2.91e-06	9.945e-05
Combinational	0.8711	1.05	0.01579	1.937	66.2
Clock (Combinational)	0.002302	0.01018	7.654e-05	0.01255	0.429
Clock (Sequential)	0.01177	0.008387	0.000347	0.0205	0.7005
Macro (non-mem)	0.2317	0.007259	0.001234	0.2402	8.208
Total	1.682	1.209	0.03503	2.926	100

`-group_type user_defined_group_name`

Specifies to report power (internal/switching/leakage/total) for a user-defined collection of cells that are stated under a power group in the power summary report. You can use the following Tempus commands to create power groups:

- `define_property` - define a property.
- `set_property` - assign the defined property (group type) and its value (power group) to a list of cells.

The use model of the `-group_type` parameter is:

```
report_power -group_type user_defined_group_type
```

Here, `user_defined_group_type` is the user-defined power group type applied to a list of cells in a design. When the `-group_type` parameter is specified, the command reports power for the power groups associated with the specified group type in the power summary report.

Example:

The following command defines the `poly_bias` property (group type) on library cells:

```
define_property poly_bias -object_type lib_cell -type string
```

The following commands apply the property `poly_bias` with the group name `P0` and `P10` to the collection of cells:

```
set_property [get_lib_cells -of_objects [get_cells -hier * -filter {ref_name=~*PB0*}]] poly_bias P0
set_property [get_lib_cells -of_objects [get_cells -hier * -filter {ref_name=~*PB10*}]] poly_bias P10
```

The following command reports power for the user-defined power groups that are associated with the property `poly_bias`:

```
report_power -outfile basic_group_type.rpt -group_type poly_bias
```

The following is a snippet of the power report that shows the user-defined power groups `P0` and `P10` that are associated with the property `poly_bias`, in addition to the existing groups (sequential, macro, IO, and so on):

User Group	Internal Power	Switching Power	Leakage Power	Total Power	Percentage (%)
P0	0.01498	0.0002017	0.01765	0.03283	15.63%
P10	0.1268	0.04026	0.01011	0.1772	84.37%
Total	0.1418	0.04047	0.02776	0.21	100%

Group	Internal Power	Switching Power	Leakage Power	Total Power	Percentage (%)
Sequential	0.02958	0.0003768	0.01458	0.04453	16.62
Macro	0	0	0	0	0
IO	0	0	0	0	0
Combinational	0.008217	0.002633	0.01025	0.0211	7.877
Clock (Combinational)	0.1239	0.04421	0.007419	0.1755	65.51
Clock (Sequential)	0.01773	0.008783	0.0002459	0.02676	9.988
Total	0.1794	0.056	0.03249	0.2679	100

`-register_gating_efficiency`

	Generates a report that includes the Register Gating Efficiency (RGE) and Data Aware Gating Efficiency (DAGE) for all sequential cell instances as well as different hierarchies in the design. Average RGE and DAGE for different hierarchies is reported. It also displays a section on ICG cluster information, that is, information about registers at the fanout of each ICG instance along with their RGE metrics. Register Gating Efficiency is calculated as: $\text{RGE} = 1 - (\text{toggles at register clock pin} / \text{root clock toggles})$ Note: Root clock means the clock defined in SDC containing register timing constraints, which can be the generated clock. It can be back traced from the register clock pin to the clock definition point. When you generate an RGE report using the command <code>report_power -register_gating_efficiency -outfile rge.rpt</code>, the following information will be displayed: • Register-gating Efficiency Report - for each clock domain, it includes the toggle rate, number of registers, number of clock gates, average clock toggle at registers, average toggle savings at registers, and average toggle savings histogram. • Register Gating Opportunities Report - this report is sorted based on the Q/CLK toggle ratio. When a register's Q/CLK ratio is greater than .25 (25%), Q toggles every other clock cycle. However, when it is less than 25%, there is a gating opportunity to reduce power. Data Aware Gating Efficiency is calculated as: $\text{DAGE} = 1 - ((\text{toggles at clock pin} - 2 * \text{toggles at Q pin}) / (\text{root clock toggles}))$																								
-report_prefix	prefix																								
	Specifies a prefix for the report in which the top-level summary of the power consumption information is written. All the new report names will be <code>prefix.*.rpt</code>, and the detailed summary will be <code>prefix.rpt</code>.																								
-sort	{internal switching leakage total}																								
	Sorts the instance-based power consumption data by <code>internal</code>, <code>switching</code>, <code>leakage</code>, or <code>total</code> power. Default: <code>total</code>																								
-stat	Reports a set of statistics on the net status after propagation, for example, duty, transition density, or activity. This parameter is useful in viewing and debugging user-defined activity factors after the power run is complete. Example: <pre>report_power -time_based_report -distribute -outfile power.rpt -stat</pre> <pre>-----</pre> <pre>-- Net status after propagation --</pre> <pre>-----</pre> <table><tr><th>Net</th><th colspan="2">Transition Density</th><th>Duty</th><th>Activity</th><th>Ref. Freq</th></tr><tr><td>VDDA1</td><td>0</td><td>1</td><td>0</td><td>1.25e+08</td><td></td></tr><tr><td>VSSA1</td><td>0</td><td>0</td><td>0</td><td>1.25e+08</td><td></td></tr><tr><td>...</td><td></td><td></td><td></td><td></td><td></td></tr></table>	Net	Transition Density		Duty	Activity	Ref. Freq	VDDA1	0	1	0	1.25e+08		VSSA1	0	0	0	1.25e+08		...					
Net	Transition Density		Duty	Activity	Ref. Freq																				
VDDA1	0	1	0	1.25e+08																					
VSSA1	0	0	0	1.25e+08																					
...																									
-thermal_conductivity_inputs	file_name																								

Specifies a file containing the thermal conductivity value for layers. You can use this parameter to include the thermal conductivity value of the following layers in the power map file:

- Silicon
- Metal
- Dielectric
- Micro Bump (optional for 3DIC design)
- Fill material in micro bump layer (optional for 3DIC design)

The thermal conductivity file format is:

Section 1 (layer type based thermal conductivity value)

```
#default
$layer_type1 $thermal_conductivity_parameter1
$layer_type2 $thermal_conductivity_parameter2
...
```

Section 2 (layer name based thermal conductivity value)

```
#perlayer
$layer1_name $thermal_conductivity_parameter1 $dielectric_thermal_conductivity_1
$layer2_name $thermal_conductivity_parameter2 $dielectric_thermal_conductivity_2
...
```

Here,

- You can either specify both Section 1 (layer type based) and Section 2 (layer name based), or specify one of the two sections. If both sections are specified, then layer name based section takes precedence over layer type based section.
- The layer name based section has thermal conductivity value and its dielectric layer's thermal conductivity value for each metal/via layer.
- If section 2 has no parameters defined, the software will use the default parameters defined in the section 1.

Example:

```
###default
Metal      430
Silicon    250
Dielectric 0.6
```

```
###Perlayer
Metal1     430    0.4
Via1       410    0.5
Metal2     390    0.5
Via2       410    0.5
...
```

For information on the thermal conductivity input file and power map file, refer to the "Static Power, IRdrop and EM Analysis" chapter in *Voltus User Guide*.

```
-thermal_leakage_temp {temp1 temp2 temp3 ...}
```

Defines the temperature to calculate leakage for the power map file.

```
-thermal_leakage_temperature_scale_table {$temp1 $scalefactor1 $temp2 $scalefactor2 $temp3 $scalefactor3
.....}
```

	Specifies the global scale factor for a temperature value to enable scaling of the leakage power values coming from the input dotlib file. This parameter allows you to scale the power values only at certain specific temperatures. This parameter allows you to generate a multi-temperature leakage power map table from a single input dotlib file in the power map generation flow. <code>-thermal_leakage_temperature_scale_table</code> and <code>-thermal_leakage_temperature_scale_table_file</code> are mutually exclusive.
<code>-thermal_leakage_temperature_scale_table_file filename</code>	
	Specifies a file containing the user-defined temperature values and the corresponding scale factors for each library, cell, or instance. The format of the file is: <pre>#Type Name temp1 temp2 temp3 temp4 Lib library_name scalefactor@temp1 scalefactor@temp2 scalefactor@temp3 scalefactor@temp4 Cell cell_name scalefactor@temp1 scalefactor@temp2 scalefactor@temp3 scalefactor@temp4 inst instance_name scalefactor@temp1 scalefactor@temp2 scalefactor@temp3 scalefactor@temp4</pre> This parameter allows you to generate a multi-temperature leakage power map table from a single input dotlib file in the power map generation flow. <code>-thermal_leakage_temperature_scale_table</code> and <code>-thermal_leakage_temperature_scale_table_file</code> are mutually exclusive. <u>File Format Example for the Type Lib:</u> <pre>#Type Name 25 50 125 150 Lib fast 1 2 10 100 Lib bufao 1 2 10 100 Lib header 1 2 10 100 Lib pso_ring_wc 1 2 10 100</pre>
<code>-thermal_material_file filename</code>	
	Specifies the name of a file containing the user-specified substrate layer thickness. The layer thickness value for the power map generation flow can either be a user-specified value or derived from the technology file. If both the values are available, the user-specified value takes precedence.
<code>-thermal_power_map_bottom_layer layer_name</code>	
	Writes zero metal density tables for layers lower than the user-specified layer in the power map file. This parameter prevents duplication of metal density tables when merging the top and block thermal power map files.
<code>-thermal_power_map_densitytable_tile {Xint Yint}</code>	
	Specifies the number of tiles in the X and Y directions for the metal density table.
<code>-thermal_power_map_file file_name</code>	
	Generates the power map file with the specified name in the output directory.
<code>-thermal_power_map_format {simple stack }</code>	
	Generates a tile-based power map file in the simple or stack format to perform accurate thermal analysis. You must use the stack format to support multiple dies in a package for thermal analysis.

<code>-thermal_power_map_format_version {new default}</code>	
	Determines whether to generate the default or the new power map file format. When set to <code>new</code>, the power map file format with non-uniform tile numbers for the power (<code>-thermal_power_map_powertable_tile</code>) and metal density tables (<code>-thermal_power_map_densitytable_tile</code>) can be generated. The following are the features of the new report format: • version number added to the header "Version : 2020.0" • nx/ny under the "DIE_AREA" section has been removed • XCOR/YCOR for each layer has been removed • nx/ny has been added for the POWER and METAL_DENSITY sections
<code>-thermal_power_map_header_include_file filename</code>	
	Specifies a file that contains comments or additional information to be included in the header section of the power map file.
<code>-thermal_power_map_powertable_tile {Xint Yint}</code>	
	Specifies the number of tiles in the X and Y directions for the power table.
<code>-thermal_power_map_SI_unit {true false}</code>	
	Specifies to change chip thermal power map units to SI in the thermal power map file. This parameter was introduced to support the thermal power map file format used by foundry. When this parameter is set to <code>true</code>, the values for all the power map parameters (length, power, and thermal conductivity) will be reported in the SI units. • Length Unit: default is <code>um</code> and SI is <code>m</code> • Power Unit: default is <code>mW</code> and SI is <code>W</code> • Metal/Silicon Thermal Conductivity Unit: default is <code>mW/(um * C)</code> and SI is <code>W/(m * C)</code> <i>Default:</i> <code>true</code>
<code>-thermal_power_map_tile {Xint Yint}</code>	
	Defines the tile number in the X and Y direction. By default 10*10 will be used.
<code>-thermal_power_map_top_layer layer_name</code>	
	Writes zero metal density tables for layers higher than the user-specified layer in the power map file. This parameter prevents duplication of metal density tables when merging the top and block thermal power map files.
<code>-threshold value</code>	
	Filters out the instances whose total power dissipation is less than the specified value. Units are in milliwatts. <i>Default:</i> <code>0</code>
<code>-threshold_voltage_group { all group_name }</code>	

	Reports leakage by voltage threshold group in the power report. This parameter supports the argument <code>all</code> (default value) or any other string, such as <code>hvt</code> or <code>lvt</code>. <i>Default:</i> <code>all</code>
<code>-time_based_report</code>	
	Writes out a detailed VCD profiling report into the file that you specify.
<code>-time_unit unit</code>	
	Sets the time unit to be used for power report generation. The default unit is <code>ns</code> (nanoseconds).
<code>-toggle_rate</code>	
	Specifies to dump out instance-based activity to the outfile. The toggle rate for each instance is based on the clock frequency to which it belongs. If multiple clocks are reaching the instance, then the fastest clock is used as a reference.
<code>-use_geometry_cell_size {true false}</code>	
	Specifies whether to use the actual geometry or placement bounding box for macro PGM cell size during generation of tile-based power map file. This parameter is used when a macro's power is distributed across multiple tiles. When this parameter is set to <code>true</code>, the geometry-based bounding box will be used for power map generation and power will be distributed evenly across tiles. <i>Default:</i> <code>false</code>
<code>-view view_name</code>	
	Specifies the active timing view that was created using the multi-mode multi-corner (MMC) <code>set_analysis_view -setup -hold</code> command. Power analysis is performed for the specified view. The command prints the MMC view name in the output file even if you do not specify the <code>-view</code> parameter. If <code>report_power -view view_name</code> is set, it will override the view set in the <code>set_power_analysis_mode -view</code> command. However, if you specify the <code>set_analysis_view -dynamic</code> and <code>-leakage</code> parameters, then the view set using these <code>set_analysis_view</code> command parameters has higher priority over the <code>report_power -view</code> command. <i>Default:</i> Uses current view in Voltus.

Examples

- The following command reports the amount of power consumed by the clock nets `clk` and `test_clk` (including leaf flip-flop power) and writes the output information to a power report file named `clk_pwr.rep`:

```
report_power -clock_network{clk test_clk} \
-count_seq_elements_in_clock_network -outfile clk_pwr.rep
```
- The following command reports the amount of power consumed down to hierarchy level 3 from the top and writes the output information to a power report file named `hier.rep`:

```
report_power -hierarchy 3 -outfile hier.rep
```
- The following command compresses the power report file `pg_net.rep` with `gzip`:

```
report_power -outfile pg_net.rep.gz
```

- The following command reports the amount of power consumed by leaf-level instances that match `ECO*` and writes the output information to a power report file named `inst.rep`:

```
report_power -instances ECO* -outfile inst.rep
```
- The following command reports the amount of power consumed by all cells that match `buf*6` and writes the output information to a power report file named `cell.rep`:

```
report_power -cell buf*6 -outfile cell.rep
```
- The following command reports the amount of power consumed by all I/O and sequential cells and writes the output information to a power report file named `cell_type.rep`:

```
report_power -cell_type {io sequential} -outfile cell_type.rep
```
- The following command sorts the instance-based power consumption data by `leakage` power and writes the output information to a power report file named `sort.rep`:

```
report_power -sort leakage -outfile sort.rep
```
- The following command reports the total power of all instances whose total power dissipation is more than 1 milliwatt and writes the output information to a power report file named `threshold.rep`:

```
report_power -threshold 1 -outfile threshold.rep
```
- The following command reports the net switching power, load capacitance, toggle rate, and switching values for each net in the design, reports the top 100 nets with the highest net switching power, and writes the output information to a power report file named `net.rep`:

```
report_power -net -nworst 100 -outfile net.rep
```
- The following command reports the amount of power consumed by each power domain and writes the output information to a power report file named `pd_all.rep`:

```
report_power -power_domain all -outfile pd_all.rep
```
- The following command reports the amount of power consumed by each power domain `PD2` and writes the output information to a power report file named `pd.rep`:

```
report_power -power_domain PD2 -outfile pd.rep
```
- The following command reports the amount of power consumed by view `view_max-hp-max-lp1` and writes the output information to a power report file named `view.rep`:

```
report_power -view view_max-hp-max-lp1 -outfile view.rep
```
- The following command reports the amount of power consumed by each instance connected to power nets `vdd` and `vdd1` and writes the output information to a power report file named `pg_net.rep`:

```
report_power -pg_net {vdd vdd1} -outfile pg_net.rep
```
- The following command reports the leakage for the threshold voltage group `HVT`:

```
report_power -leakage -threshold_voltage_group { HVT }
```
- The following command writes out a detailed vector profiling report for the combinational and macro cell types in the `vectorprofile.rpt` file:

```
report_power -time_based_report -outfile vectorprofile.rpt -cell_type {combinational macro}
```
- The following command is an example of generating multiple reports at the same time:

```
report_power -output static_power -report_prefix design -net {name1} -instance {inst_name}
```

The following reports will be generated:

```
static_power/design.rpt
```

```
static_power/design.netpwr.rpt
static_power/design.instpwr.rpt
```

- The following command specifies to report power for combinational cells on the clock network.

```
report_power -cell_type clock_combinational -output static_power -report_prefix power_clk
```

- The following command specifies to report the power content for the `inst1` hierarchical instance in the `report.hierinstpwr.rpt` report:

```
report_power -output static -report_prefix report -clock_network {all} -instances * -no_wrap -hierarchy
{all} -hierarchical_instances inst1 -cell_type {all}
```

- The following command generates a power map file in the simple format for the specified tile:

```
report_power -outfile powermeter.txt -thermal_power_map_file powermap.txt -thermal_power_map_tile { 4 6 } -
thermal_power_map_format simple
```

Example of the output of the power report.

```
*-----
* Version 64-bit
* Date & Time: 2008-Jan-27 15:49:51 (2008-Jan-27 23:49:51 GMT)
*-----
* Design: dtmf_chip
*
* Liberty Libraries used:
* /sev/DTMF_CHIP/TIMING/max/scmetropmk_cmos10lp_rvt_ss_1p08v_125c.lib
* /sev/DTMF_CHIP/TIMING/max/scmetropmk_cmos10lp_rvt_ss_0p8v_1p08v_125c.lib
* /sev/DTMF_CHIP/TIMING/max/scmetropmk_cmos10lp_rvt_ss_0p8v_125c.lib
* .....
*
* Power Domain used:
* Rail: VDD_AO Voltage: 1.32
* Rail: VDD_TDSPCore Voltage: 1.32
* Rail: VDD_TDSPCore_R Voltage: 1.32
*
* Parasitic Files used:
* /sev/DTMF_CHIP/SPEF/dtmf_chip.decoup.spef
*
* DEF Files used:
* /sev/DTMF_CHIP/DEF/dtmf_chip.def
*
* Power Units = lmW
* Time Units = 1e-09 secs
* report_power
*-----
Total Power
*-----
Total Internal Power: 0.01943(62.6774%)
Total Switching Power: 0.01057(34.0968%)
Total Leakage Power: 0.001(3.2258%)
Total Power: 0.031
*-----
Group Internal Switching Leakage Total Percentage
```


Voltus IC Power Integrity Solution Text Command Reference

Power Calculation Commands

	Power	Power	Power	Power	(%)
Sequential	0.0006738	0.0002215	4.045e-05	0.0009358	3.019
Macro	0.0004916	1.515e-05	0.0002609	0.0007677	2.477
IO	0.01743	0.009085	0.0003172	0.02683	86.56
Combinational	0.0008324	0.001247	0.0003815	0.002461	7.941
Total	0.01943	0.01057	0.001	0.031	100

Rail	Voltage	Internal	Switching	Leakage	Total	Percentage
		Power	Power	Power	Power	(%)
VDD_AO	1.32	0.0008238	0.0003404	0.0002694	0.001434	4.625
VDD_TDSPCore	1.32	0.001126	0.001022	0.0003931	0.002541	8.197
VDD_TDSPCore_R	1.32	5.833e-06	3.253e-07	8.371e-07	6.996e-06	0.02257

Clock	Internal	Switching	Leakage	Total	Percentage
	Power	Power	Power	Power	(%)
m_digit_clk	0	0	0	0	0
m_ram_clk	0	0	0	0	0
m_dsram_clk	0	0	0	0	0
m_spi_clk	1.018e-06	1.09e-06	2.572e-08	2.134e-06	0.006884
m_rcc_clk	1.282e-05	1.513e-05	2.672e-07	2.821e-05	0.09103
m_clk	5.039e-05	7.278e-05	7.658e-07	0.0001239	0.3998
refclk	1.11e-05	3.017e-05	2.488e-07	4.152e-05	0.134
Total	7.533e-05	0.0001192	1.307e-06	0.0001958	0.6317

```

* Power Distribution Summary:
* Highest Average Power: IOPADS_INST/Prefclk (PBCSCT2B): 0.003727
* Highest Leakage Power: DTMF_INST/ROM_512x16_0_INST (rom_512x16A): 8.899e-05
* Total Cap: 2.46221e-10 F
* Total instances in design: 8525
* Total instances in design with no power: 24
* Total instances in design with no activity: 24
* Total Fillers and Decap: 8

```

- The following command creates a leakage report (see example report below):

```
report_power -leakage
```

```

*-----
*
* Date & Time: 2008-Aug-24 21:34:17 (2008-Aug-25 04:34:17 GMT)
*

```

Voltus IC Power Integrity Solution Text Command Reference

Power Calculation Commands

```

*-----
*
*      Design: dma_mac
*
*      Liberty Libraries used:
*          DATA/timing_libs//tcbn65lp_LVL_c060217wc0d90d9.lib
*          DATA/timing_libs//tcbn65lpcgwc.lib
*          ...
*
*      Power Domain used:
*          Rail: VDDlu  Voltage: 0.84
*          Rail: VDDm  Voltage: 1.08
*          Rail: VDD   Voltage: 1.08
*
*      Power Units = 1mW
*
*      Time Units = 1e-09 secs
*
*      report_power -outfile Leakage.rep -leakage
*

```

Total Power

Total Leakage Power: 0.1879

Group	Leakage Power	Percentage (%)
-------	------------------	-------------------

Sequential	0.0127	6.756
Macro	0.1556	82.8
IO	0	0
Combinational	0.01962	10.44

Total	0.1879	100
-------	--------	-----

Rail	Voltage	Leakage Power	Percentage (%)
------	---------	------------------	-------------------

VDDau	0	0	0
VDDlu	0.84	0.06477	34.47
VDDm	1.08	0.003587	1.909
VDD	1.08	0.1195	63.62

Voltus IC Power Integrity Solution Text Command Reference

Power Calculation Commands

Clock	Leakage Power	Percentage (%)
phy_rxclk_2	0	0
phy_txclk_2	0.0004786	0.2547
phy_rxclk_1	0	0
phy_txclk_1	3.39e-05	0.01804
clk	0.001723	0.9171
Total	0.002236	1.19

```

* Power Distribution Summary:
* Highest Average Power: ethernet_mac_2/tx_fifo_module_from_dma/fifo1/dual_port_
  ram_4_tx_fifo/ram2P1024x32/ram (TSDN65LPA1024X32M8F):      0.02424
* Highest Leakage Power: ethernet_mac_2/tx_fifo_module_from_dma/fifo1/dual_port_
  ram_4_tx_fifo/ram2P1024x32/ram (TSDN65LPA1024X32M8F):      0.02424
* Total Cap: 1.01625e-10 F
* Total instances in design: 29235
* Total instances in design with no power: 3596
* Total instances in design with no activity: 115
* Total Fillers and Decap: 0

```

Total leakage power = 0.187903 mW

Cell usage statistics:

```

Library tsdn65lpa128x16m8f_1v08, 1 cells ( 0.003900% ) , 0.01016 mW (5.407063% )
Library tsdn65lpa1024x32m8f_1v08, 6 cells ( 0.023402% ), 0.145427 mW (77.394568% )
Library tcbn65lp_LVL_c060217wc0d90d9 , 120 cells ( 0.468037% ), 0.000175605 mW ( 0.093455% )

Library unknown, 3881 cells ( 15.137096% ), 0.00630072 mW ( 3.353178% )
Library tcbn65lpcgwc, 118 cells ( 0.460236% ), 0.000625092 mW ( 0.332667% )
Library tcbn65lplvtwc, 3762 cells ( 14.672959% ), 0.0110529 mW ( 5.882233% )
Library tcbn65lpwc, 17751 cells ( 69.234370% ), 0.0141619 mW ( 7.536835% )

```

reset_power_activity

```
reset_power_activity  
[-help]
```

Resets all activity in the database. You can specify this command to reset all activity and power information in a given session.

 You can reset activity from an activity file using `read_activity_file -reset`, reset activity for nets, pins, and ports using `set_switching_activity -reset`, and reset the default switching activities using `set_default_switching_activity -reset`.

Parameters

<code>-help</code>	Prints out the command usage.
--------------------	-------------------------------

restore_power_database

```
restore_power_database
[-help]
-file {file1 file2 file3 ...}
[-hierarchy {prefix1 prefix2 prefix3 ...}]
[-hierarchy_separator "/" ]
```

Restores any `power.db` file in the Voltus/Innovus session. This command can be used after restoration of a design to view the static power results or incremental report generation without having to do power analysis again.

Note: You will be able to restore the power database that you created previously, only if you created the power database using the `set_power_analysis_mode -create_binary_db true` command. Also, the command loads power databases created in the 9.1 release only, and not in the older releases.

- The command can map a hierarchical instance to a flat netlist.
- It can restore multiple power dbs for blocks and top level, that are generated separately.
- When the information for a net or an instance is present in multiple power dbs, the last one will overwrite.

Parameters

- help	Outputs a brief description that includes the type and default information for each <code>restore_power_database</code> parameter. For a detailed description of the command and all of its parameters, use the man command <code>man restore_power_database</code>.
-file {file1 file2 file3 ...}	
	Specifies the name of the power database file.
-hierarchy {prefix1 prefix2 prefix3 ...}	
	Specifies to map each instance in the file with a prefixN/instance (for example, prefixA/inst) in the flat netlist, where '/' is the hierarchy separator. This is an optional parameter. <i>Default:</i> all
-hierarchy_separator "/"	
	Specifies the separator character in the hierarchical net names, bus names, and pin names for the block or other part of the design that you want to analyze. It must be the same as the separator character in the design netlist. This is an optional parameter. <i>Default:</i> slash (/)

Use Model

```
read_design
restore_power_database -file power.db
report_power # for text based power reports
```

Example

For each instance in file 'file2' a prefix "A" is added to instance name such as instC, A/instC. No prefix is added to 'file1' since prefix1 is none for file1.

```
restore_power_database -file {file1 file2} -hierarchy {none A}
```

set_default_switching_activity

```
set_default_switching_activity
[-black_box_density value]
[-black_box_duty value]
[-block master_cell_name]
[-clip_activity_to_domain_freq {true | false}]
[-clock_gates_enable {activity_factor}]
[-clock_gates_output {activity_factor}]
[-clock_gates_output_ratio num]
[-comb_clockgate_ratio num]
[-duty value]
[-global_activity factor]
[-hier hierarchy_name]
[-icg_ratio num]
[-input_activity factor]
[-macro_activity factor]
[-name activity_name]
[-period value]
[-pg_net netname]
[-reset]
[-reset_type { global_activity | seq_activity ... }]
[-seq_activity factor]
```

Specifies the switching activity for all primary inputs, nets, and other devices in the design whose activity has not been previously defined through user attributes, the toggle count format (TCF) file, the value change dump (VCD) file, or the tracing of the clock network.

You can use this command to specify user-defined activities at the design level.

Parameters

-black_box_density value	
	Specifies to set the default transition density on all the output pins of all blackboxes in the design. The -black_box_density and -black_box_duty parameters must be used together. If one or both of these parameters are not specified, then the software will use the average transition density from all the input pins of the blackboxes.
-black_box_duty value	
	Specifies to set the default duty on all the output pins of all blackboxes in the design. The -black_box_density and -black_box_duty parameters must be used together. If one or both of these parameters are not specified, then the software will use the average transition density from all the input pins of the blackboxes.
-block master_cell_name	

	Specifies the name of the master cell for activity setting. The parameter allows you to define the user-defined switching activity for a block partition. This parameter is useful for assigning block-based activity. The <code>-block</code> parameter is applicable only to the block-based distributed power flow.
	<code>-clip_activity_to_domain_freq {true false}</code>
	Specifies to perform domain-based activity clipping. This parameter clips data paths to 1x frequency whereas the clock networks are still at 2x frequency. This parameter allows you to have more control to determine the rate of clipping with respect to frequency.
	<code>-clock_gates_enable {activity_factor}</code>
	Specifies the activity at the enable pin(s) of the clock gating cells.
	<code>-clock_gates_output {activity_factor}</code>
	Specifies the activity at the output pin(s) of the clock gating cells. This works for both ICGs as well as combinational clock gating cells. Example: <pre>set_default_switching_activity -global_activity 0.02 -clock_gates_output 2.0 (clock is transparent)</pre> Transition density at the output pin(s) of clock gating cells is calculated by: $\text{transition_density} = \text{clock_domain_frequency} * \text{clock_gates_output_value}$ If the fastest clock frequency is 1 GHz for a clock gating cell, then: $\text{transition_density} = (1\text{e}9) * 2.0 = 2,000,000,000$
	<code>-clock_gates_output_ratio num</code>
	Specifies to control transition density at the output pin(s) of the integrated and combinational clock gating cells. This parameter is equivalent to specifying the parameters <code>-icg_ratio</code> and <code>-comb_clockgate_ratio</code> together with the same value. For example, the <code>set_default_switching_activity -clock_gates_output_ratio 0.6</code> is equivalent to <code>set_default_switching_activity -icg_ratio 0.6 -comb_clockgate_ratio 0.6</code>. Note: This parameter has a cascading effect when there are multiple levels of ICG cells on the clock path. For example, when <code>-clock_gates_output_ratio</code> is set to 0.5, the first level ICG cell will have 0.5 ratio of the root clock's toggle density, the second level ICG cell will have $0.5 * 0.5 = 0.25$ ratio of the root clock's toggle density, and the third level ICG cell will have $0.5 * 0.5 * 0.5 = 0.125$ ratio of the root clock's toggle density, and so on. Note: For dynamic power analysis, fastest clock on the path is used for calculating output toggle density. However, for static power analysis, weighted averaging of input toggle densities is used. Example for static power analysis: <pre>set_default_switching_activity -global_activity 0.02 -clock_gates_output_ratio 1.0</pre> Transition density for either a combinational or an integrated clock gating cell with 'N' inputs is calculated by: $\text{output_pin_transition_density} = (\text{summation}) \text{weighted_input_transition_density} = \text{clock_gates_output_ratio} * [(\text{duty_input_pin1} * \text{Td_input_pin1}) + (\text{duty_input_pin2} * \text{Td_input_pin2}) + \dots + (\text{duty_input_pinN} * \text{Td_input_pinN})]$ Given a 2:1 clock Mux and assuming duty of 0.5 for all input pins: Td is the toggle density for each input pin of the instance (Td_pinSel , Td_pinI0 , Td_pinI1). $\text{output_pin_transition_density} = 1.0 * [0.5 * \text{Td_pinSel} + 0.5 * \text{Td_pinI0} + 0.5 * \text{Td_pinI1}]$

<code>-comb_clockgate_ratio num</code>	
	Specifies to set the propagation ratio for combinational clock gate cell outputs. When specified, this parameter sets the output activity of any instance that is identified as a combinational clock gating cell to <i>num</i> times the fastest transition density arriving on the instance's clock inputs. This parameter cannot be specified with the <code>-hier</code> parameter. Notes: • This parameter does not apply to combinational clockgates with a constant data input, that is, a select pin tied low. • For dynamic power analysis, fastest clock on the path is used for calculating output toggle density. However, for static power analysis, weighted averaging of input toggle densities is used. For a detailed static power analysis example, refer to '<code>-clock_gates_output_ratio</code>', limiting calculations only to combinational clock gates.
<code>-duty value</code>	Specifies the duty cycle, which is the probability that the signal is a logical 1. Note: You must specify the <code>-period</code> parameter when assigning a duty. Default: 0.5 (equivalent to a 50 percent duty cycle)
<code>-global_activity factor</code>	
	Specifies the average number of times that all unset nodes switch in a clock cycle. If not specified the fastest domain frequency will be used unless one turns off clock domains. Note: Due to activity precedence, the <code>-global_activity</code> and <code>-seq_activity</code> parameters of the <code>set_default_switching_activity</code> command are mutually exclusive. If both the parameters are specified, <code>-global_activity</code> takes precedence.
<code>-hier hierarchy_name</code>	
	A hierarchical name that the global activity is being assigned to. If not provided, the whole design is assumed.
<code>-icg_ratio num</code>	
	Specifies to set the propagation ratio for integrated clock gate (ICG) cell outputs. When specified, this parameter sets the output pin of all identified ICG instances to <i>num</i> times the fastest transition density arriving on clock inputs of the instance. This parameter cannot be specified with the <code>-hier</code> parameter.
<code>-input_activity factor</code>	
	Specifies the average number of times that a primary input switches in a clock cycle. It can be any positive number. Default: 0.2 (Switches once every five clock cycles.)
<code>-macro_activity factor</code>	

	Specifies the activity factor for macros in a design. The allowed values for this parameter are between 0 to 1. If you set the macro activity to .30, 30% of macros in the design would be selected for switching in a given cycle during dynamic simulation. This parameter is applicable only to the state-propagation-based dynamic analysis flow (<code>set_power_analysis_mode -method dynamic_vectorless -enable_state_propagation true</code>). Default: 0.2
<code>-name activity_name</code>	
	Allows you to set the activity level for the specified jitter activity name. You can define a low/high activity level for jitter analysis. Examples: <pre>set_default_switching_activity -name low -clock_gates_output 0 -global_activity 0.01 set_default_switching_activity -name high -clock_gates_output 2 -global_activity 0.2</pre>
<code>-period value</code>	Specifies the default operating period (1/frequency). It is the period that the activity is referenced to. If not specified, the software uses the <i>dominant</i> clock in the design. The specified period will only be applied to nets that do not have reference clock (ref_clock) associated with the net. If no clock is specified, the software generates an error and prompts you to enter one. Units in seconds (s), milliseconds (ms), microseconds (us), nanoseconds (ns), or picoseconds (ps). Note: If you enter the period as a negative number, the software reverts back to using the default frequency. Default: The time unit is defined in the first .lib file read during design import.
<code>-pg_net netname</code>	
	Specifies the name of the power net for domain-based switching activity assignment. The parameter allows you to define different switching activities for different domains within the same DEF block during stable flop scheduling. When this parameter is specified, the <code>voltus_power.stateprop.swichsrc</code> and <code>voltus_power.stateprop.swichinst</code> reports are affected because the flop toggle values will change. The parameter is applicable only to the dynamic vectorless flow. The <code>-pg_net</code> parameter must be specified with <code>set_power_analysis_mode -honor_default_switching_activity_in_scheduling true</code> for domain-based switching activity assignment.
<code>-seq_activity factor</code>	
	Specifies the activity of outputs of the sequential logic. This value will be used at the output ports of flops for propagating activity. If the sequential activity is not specified, propagation will be done for the full design starting from the input ports.
<code>-reset</code>	Resets all specified options back to default values.
<code>-reset_type { global_activity seq_activity ... }</code>	
	Specifies to selectively reset the global specifications. Note: You must use the <code>-reset_type</code> parameter in conjunction with the <code>-reset</code> parameter.

Activity Precedence

See "[Activity Precedence](#)".

Examples

- The following command specifies that the primary inputs in the design will switch once every ten clock cycles, with a clock period of 7520 ns and a duty cycle of 50 percent:

```
set_default_switching_activity -input_activity 0.10 -period 7520ns -duty .5
```

- The following command specifies that all unswitched nodes in the design will switch once every ten clock cycles, with a clock period of 7520 ns and a duty cycle of 50 percent:

```
set_default_switching_activity -global_activity 0.10 -period 7520ns -duty .5
```

- The next example defines the switching activity, sequential activity, and the period:

```
set_default_switching_activity -input_activity 0.2 -period 10 \  
-seq_activity 0.1
```

- The next example sets the default global activity to 0.5 for the hierarchy `dma_dut`:

```
set_default_switching_activity -global_activity 0.5 -hier dma_dut
```

- The following command resets the sequential and global activity specification:

```
set_default_switching_activity -reset -reset_type  
{seq_activity,global_activity}
```

- The following command will define the input activity, sequential activity and macro activity for the design block A:

```
set_default_switching_activity -input_activity 0.1 -seq_activity 0.1 -macro_activity 0.1 -block A
```

Related Commands

- `read_activity_file`
- `get_activity`
- `reset_power_activity`
- `set_switching_activity`

set_dynamic_power_simulation

```
set_dynamic_power_simulation
[-help]
[-activity_pattern {time1 activity_name1 time2 activity_name2 ....}]
[-period value]
[-reset]
[-resolution value]
```

Specifies the period and resolution for a dynamic power simulation. This command is used to specify the simulation period in the dynamic vectorbased/vectorless and power-up flows.

Generally, the number of simulation steps (simulation period divided by resolution) should be selected such that the number is around 1000. Around 20 clock cycles of the dominant clock specified as period and 20/50ps specified as resolution will capture all switching events in the design. This can be fine-tuned to balance the run time and accuracy. For example, if the dominant clock frequency is 1GHz, and the clock period is 1ns, the recommended simulation period is 20ns and resolution is 20ps, as shown in the following example:

```
set_dynamic_power_simulation -period 20ns -resolution 20ps
```

In this case, the number of simulation steps is 1000.

Parameters

-help	Outputs a brief description that includes type and default information for each set_dynamic_power_simulation parameter. For a detailed description of the command and all of its parameters, use the man command: man set_dynamic_power_simulation.
-activity_pattern {time1 activity_name1 time2 activity_name2}	

	Divides the total simulation time based on the windows given in the <code>activity_pattern</code> parameter. For a particular window, it uses the defined activity specified with the <code>set_default_switching_activity</code> command. When the user-defined low cycle and high cycle time is specified for the input/sequential activity, all the clocks will be in the low or high cycle until completely past the cycle transition time. Following is the use model of this parameter: • <code>set_default_switching_activity -name low -input_activity 0.05 -seq_activity 0.1 -clock_gates_output 0.2</code> • <code>set_default_switching_activity -name high -input_activity 0.1 -seq_activity 0.2 -clock_gates_output 2.0</code> • <code>set_dynamic_power_simulation -period 15ns -resolution 50ps -activity_pattern {0 high 5ns low 10ns high}</code> Here, high activity is used for 0ns to 5ns, low activity is used for 5ns to 10ns, and high activity is used for 10ns to 15ns.
<code>-period</code> <i>value</i>	Specifies the simulation period that dynamic power analysis will use. The supported time units are <code>s</code> , <code>ms</code> , <code>us</code> , <code>ns</code> , and <code>ps</code> . If no explicit time unit is specified, the default <code>ns</code> (nanosecond) will be applied.
<code>-reset</code>	Resets all specified options back to default values.
<code>-resolution</code> <i>value</i>	Specifies the transient time step size or resolution. The supported time units are <code>s</code> , <code>ms</code> , <code>us</code> , <code>ns</code> , and <code>ps</code> . If no explicit time unit is specified, the default <code>ps</code> (picosecond) will be applied.

Examples

- The following command sets up the period and resolution for a dynamic simulation:
`set_dynamic_power_simulation -period 5ns -resolution 100ps`

set_inst_temperature_file

```
set_inst_temperature_file  
[-help]  
file  
[-reset]
```

Specifies the instance temperature file to support thermal-aware leakage. You can use this command to do thermal-aware power calculation. Based on the temperature specified in the file, the software will calculate temperature and leakage power.

In the presence of a trilib set, the internal and leakage power will be interpolated as the per the given temperature specification.

Parameters

-help	Prints out the command usage.
<i>file</i>	Specifies the name of the instance temperature file.
-reset	Resets the temperature file.

set_metal_density_options

```
set_metal_density_options
[-metal_density_include_file filename]
-power_grid_library pgv_path
[-reuse_md_db_path MD_db_path]
-tech_file tech_file_path
-thermal_conductivity_file thermal_file
[-user_defined_metal_density_file user_defined_md_file]]
[-vtm_densitytable_tile {Xint Yint} ]
[-vtm_densitytable_tile_size {Xsize Ysize}]
[-vtm_header_include_file user_defined_header_content_file]
-vtm_path output_path
```

Sets the parameters for extracting the metal density information. This command must be specified before the `extract_metal_density` command.

Parameters

<code>-metal_density_include_file filename</code>	
	Specifies the file for listing the additional variables that can be included when extracting metal density information. For example, you can specify " <code>setvar report_with_lef_layer_names true</code> " to output the LEF layer names.
<code>-power_grid_library pgv_path</code>	
	Specifies the path to the power cell libraries (.cl).
<code>-reuse_md_db_path MD_db_path</code>	
	Specifies to reuse the generated metal density database for <code>extract_metal_density</code> . When specified, the PGV/tech file specified with <code>set_metal_density_options</code> will be ignored.
<code>-tech_file tech_file_path</code>	
	Specifies the path to the extraction technology file to be used for top-level power-grid extraction.
<code>-thermal_conductivity_file thermal_file</code>	

Specifies a file containing the thermal conductivity value for layers. You can use this parameter to include the thermal conductivity value of the following layers in the power map file:

- Silicon
- Metal
- Dielectric
- Micro Bump (optional for 3DIC design)
- Fill material in micro bump layer (optional for 3DIC design)

The thermal conductivity file format is:

Section 1 (layer type based thermal conductivity value)

```
#default
$layer_type1 $thermal_conductivity_parameter1
$layer_type2 $thermal_conductivity_parameter2
...
```

Section 2 (layer name based thermal conductivity value)

```
#perlayer
$layer1_name $thermal_conductivity_parameter1 $dielectric_thermal_conductivity_1
$layer2_name $thermal_conductivity_parameter2 $dielectric_thermal_conductivity_2
...
```

Here,

- You can either specify both Section 1 (layer type based) and Section 2 (layer name based), or specify one of the two sections. If both sections are specified, then layer name based section takes precedence over layer type based section.
- The layer name based section has thermal conductivity value and its dielectric layer's thermal conductivity value for each metal/via layer.
- If section 2 has no parameters defined, the software will use the default parameters defined in the section 1.

Example:

```
###default
Metal      430
Silicon    250
Dielectric 0.6

###Perlayer
Metal1     430    0.4
Via1       410    0.5
Metal2     390    0.5
Via2       410    0.5
...
```

```
-user_defined_metal_density_file user_defined_md_file)
```


Allows you to define the metal density in the user-defined metal density flow.

The format of the user-defined metal density file is:

```
CELL <cell_name>
CELL_AREA [ xmin ymin xmax ymax | x1 y1 x2 y2 x3 y3 x4 y4 ... x1 y1]      # Must specify if cell PGV is not
imported
LAYER <layer_name(s)>                                     # Layer to be defined metal density; multiple layers are
available (LEF layer name based)
ALL <metal_density_value>                                     # To define metal density for whole cell area as 'ALL'
only 'ALL' can be used now
END_LAYER
END_CELL
```

Example:

```
CELL Macro1
CELL_AREA 0 0 32 16
LAYER Metal1 Metal2 Metal3
ALL 0.2
END_LAYER
LAYER Via1 Via2
ALL 0.05
END_LAYER
END_CELL

CELL Macro2
CELL_AREA 0 0 32 16
LAYER Metal1 Metal2 Metal3
ALL 0.2
END_LAYER
LAYER Via1 Via2
ALL 0.05
END_LAYER
END_CELL
```

`-vtm_densitytable_tile {Xint Yint}`

Specifies the number of tiles in the X and Y directions for the metal density table.

`-vtm_densitytable_tile_size {Xint Yint}`

Specifies the size of tiles instead of tile number in the X and Y directions for the metal density table.

`-vtm_header_include_file user_defined_header_content_file`

Allows you to define extra header information like file location.

`-vtm_path output_path`

Specifies the directory path to the metal density file output folder.

Examples

- The following command specifies to set the metal density parameters:

```
set_metal_density_options \  
-power_grid_library "$DesignDir/PGV/tech_pgv/techonly.cl" \  
-tech_file $DesignDir/QRC/qrcTechFile \  
-vtm_path ./vtm_MD_db \  
-thermal_conductivity_file $DesignDir/MISC/thermal_conduct.txt \  
-vtm_densitytable_tile {10 10} \  
-vtm_header_include_file $DesignDir/MISC/user_header.txt
```

Related Topics

- [extract_metal_density](#)

set_power

```
set_power
[-help]
value
[-ascii_power_file filename]
[-cap value]
[-cell cell_name value]
[-clock {all| clock_name }]
[-custom_macro_pwl filename]
[-dynamic_async_mode { mode_name }]
[-default_dynamic_pgv_mode { mode_name }]
[-dynamic_switch_event { switching_events }]
[-dynamic_switch_pattern pattern]
[-exclude_clock <string>]
[-exclude_ddv_modes {pattern_list}]
[-fanout_limit value]
[-force]
[-include_ddv_modes {pattern_list}]
[-instance inst_list value]
[-internal]
[-leakage]
[-no_propagation]
[-pg_net railname]
[-pg_pin {pg_pin}]
[-pin pin_name pin_power ]
[-pwl]
[-repeat]
[-reset]
[-scale_factor value]
[-signal_net netname]
[-slew value]
[-static_mode { custom_label_name | mode_name }]
[-sticky]
[-switching value ]
[-trigger_time_adjustment time]
```

Specifies the amount of power for all or part of the design. This command is used to define user-specified power on cells and instances. You must specify this command before `report_power`.

The `set_power` command can accept a cell/instance list using the `get_cells/get_lib_cells` commands. You can use the `get*` commands to create a collection of cells/instances whose name matches the supplied pattern list. Therefore, providing you with the flexibility in providing lists through use of regular filter patterns. This feature is supported in both static and dynamic analysis engines.

The following is the priority for the `set_power` parameters:

Priority	Command Parameters
1	-instance -internal/-switching/-leakage
2	-cell -internal/-switching/-leakage

3	-instance
4	-cell
5	-clock
6	whole power in design / -exclude_clock / -pg_net

Higher priority parameter will overwrite the setting of the lower priority parameter.

Parameters

-help	Prints out the command usage.
-ascii_power_file filename	Specifies the name of an instance list file. This file contains the list of instances in the following format: <pre>instance_name power_value <pin_name></pre> Here, • <i>instance_name</i> is the instance name to which power is to be distributed. • <i>power_value</i> is the power value for the instance in watts. • <i>pin_name</i> is an optional argument to specify the pin name. When specified, allows you to assign power to the specific pin of the instance. • The three fields should be in one line, and each instance should be in a new line. The following is an example of the -ascii_power_file parameter: <pre>set_power -ascii_power_file pg_pin_1.txt</pre> The following is a snippet of the specified file (pg_pin_1.txt): <pre>#Instance Name Power (W) Power Pin ring/PSO_ring_1_155 3.403e-09 VDD ring/PSO_ring_1_155 3.403e-09 TVDD ring/PSO_ring_1_154 3.403e-09 VDD ring/PSO_ring_1_154 3.403e-09 TVDD ...</pre>

<pre>-cell <i>cell_name</i> <i>value</i></pre>	Specifies the cell name. If the cell or instance name is not specified, the power will be distributed to all rails. If you want to specify power for a group of cells/instances, use the <code>get_cells/get_lib_cells</code> commands. The <code>-cell</code> parameter can accept these commands to create a collection of cells/instances whose name matches the supplied pattern list. For example, <code>-cell[get_lib_cells NAND*] 2mw</code>
<pre>-cap <i>value</i></pre>	Specifies the capacitance value to all instances with nets more than or equal to the fanout limit (<code>-fanout_limit</code>).
<pre>-clock {all <i>clock_name</i> }</pre>	Specifies the power value for clock networks. You can specify the power value for a specific clock network or the clock network power value for the whole design.  This parameter cannot be used with the <code>-cell</code>, <code>-instance</code> and <code>-pg_net</code> parameters.
<pre>-custom_macro_pwl <i>filename</i></pre>	Specifies to use a custom trigger file to apply instance or cell PWL waveforms. Given one master cell PWL waveform, the software can automatically generate scaled PWL waveforms of the other companion cells defined in the trigger file proportional to the cell internal power.  This parameter must be used with <code>set_power -dynamic_switch_pattern</code> in the vectorless flow. For more information on the trigger file format, refer to Trigger File Format.
<pre>- default_dynamic_pgv_mode { <i>mode_name</i> }</pre>	Specifies the current characterization mode to be used during dynamic power and jitter analyses. This parameter can be specified when using a PGV with multiple functional modes. When this parameter is specified, the Power Calculation engine uses this preferred mode in scheduling the switching events for the macro blocks, and saves these events in the trigger file. The use model of this parameter is: <pre>set_power -cell {} -instance {} - default_dynamic_pgv_mode { <i>mode_name</i> }</pre> This parameter is mutually exclusive to the <code>-dynamic_switch_pattern</code> parameter. If a cell or instance has both the <code>-dynamic_switch_pattern</code> and <code>-default_dynamic_pgv_mode</code> parameters defined in the same Tcl script, <code>-dynamic_switch_pattern</code> takes higher priority.

<pre>-dynamic_async_mode { mode_name }</pre>	Specifies to trigger asynchronous macros based on the length of the PWL waveform to create a single continuous waveform. This parameter allows you to specify the asynchronous mode for an instance or a cell. The start and end of the PWL saved in the PGV must have similar current value in order to avoid sudden jump in the current. Use model: <pre>set_power -dynamic_async_mode {mode} [-instance {name} -cell {name}]</pre> Example: <pre>set_power -cell pll -dynamic_asynchronous_mode {READ_C}</pre> Here, power analysis will trigger the <code>READ_C</code> mode and automatically splice the waveform based on its length.
<pre>-dynamic_switch_event { switching_events }</pre>	Specifies the switching events for cells, instances, and pins along with their rise and fall time. <code>switching_events</code> is a string of toggle time followed by “r” or “f”, where “r” is for rise toggle and “f” is for fall toggle. The <code>-dynamic_switch_event</code> parameter should not be specified with the <code>-repeat</code> parameter.
<pre>-dynamic_switch_pattern pattern</pre>	Specifies stimulus pattern on the pins of an instance and cell. Specify this parameter only in the dynamic mode for the type <code>instance</code> and <code>cell</code>. You can also use the <code>set_power -pin</code> parameter with the <code>-dynamic_switch_pattern</code> parameter to apply pin-specific dynamic switch pattern. That is, you can apply different switching patterns on different pins, as shown in the following example: <pre>set_power -instance FF14 -dynamic_switch_pattern { r - - f - - } -pin Q</pre> The <code>-pin</code> parameter can be specified for both the <code>-instance</code> and <code>-cell</code> parameters. Pin-based dynamic switch pattern has the following limitations: • “-instance *” and “-cell *” are not supported. All other wildcards except “*” is supported. For example “AND*” or “top*” is supported. • The value of the <code>-no_propagation</code> parameter should be the same for all pins. The <code>pattern</code> is applied to all pins (input and output) on the instance/cell. It is a string. <code>pattern</code> specifies the switching pattern. If there is more than one clock domain with timing windows on a pin, only the windows on the fastest clock will be used, and the pattern will be applied to that clock period. It is a string of 0, 1, r, and f values. 0 for cycles not to switch and 1 for

cycles to switch. `r` is for rise only and `f` for fall only.

The pattern string should include enough digits to cover all of the clock cycles in the simulation period. If there are not enough digits, the pattern will be padded with 0 values at the end. If there are too many, the pattern will be truncated to the number of applicable cycles. Transitions will be scheduled in the center of each applicable timing window.

⚠ Notes

- 0 and 1 is supported only in the probability-based vectorless flow. It is not supported in the state-propagation-based vectorless flow and the vector-based flow.
- In case of macros, you can specify the mode names, that is, which mode switches in which clock cycle `-dynamic_switch_pattern { mode1 - mode2 - }`.
- The dynamic switch pattern (DSP) events can be state propagated. Instance power is reported for instances directly annotated using DSP or those receiving activity through state propagation.
- The dynamic switch pattern does not apply to clock network instances.

You can specify “-” to indicate that instances will not switch in a particular clock cycle. If “-” is the first value in the string of values specified within {}, you must insert a space before “-”, as shown below:

- no switching: `-dynamic_switch_pattern { - - }`
- no switching in the 1st clock period: `-dynamic_switch_pattern { - r }`

Examples:

In the following example, instance will not switch in the 4th clock cycle:

```
set_power -dynamic_switch_pattern { f r f - r }
```

In the following example, the mode `write` will switch in the 1st clock cycle, the mode `read` will switch in the 2nd and 4th clock cycle, and none of the modes will switch in the 3rd clock cycle:

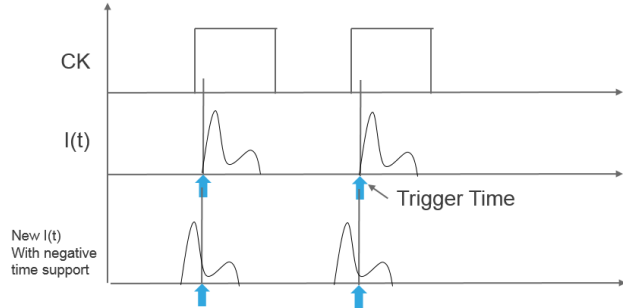
```
set_power -dynamic_switch_pattern { write read - read }
```

	⚠ Wildcards are supported only for <code>-dynamic_switch_pattern</code> with <code>-pin</code>. When <code>-dynamic_switch_pattern</code> is applied for <code>-instance</code> or <code>-cell</code> without the <code>-pin</code> parameter, wildcards are not supported.
<code>-exclude_clock</code>	Excludes clock networks when specifying a scale factor or power specification for the whole design. This parameter cannot be used with the <code>-cell</code>, <code>-instance</code> and <code>-pg_net</code> parameters.
<code>-exclude_ddv_modes {pattern_list}</code>	Excludes the current characterization (or Dynamic Detailed View) modes that match with the specified patterns. This parameter allows you to control the modes to be switched, instead of switching all the current characterization modes defined in the power-grid view. Use Model <pre>set_power -instance -cell name -exclude_ddv_modes {pattern_list}</pre> Here: • <code>pattern_list</code> is a list of mode names, separated by space or comma • the instance, cell, and pattern names can contain wildcard characters
<code>-fanout_limit value</code>	Specifies the fanout limit for nets. If an instance has nets more than or equal to the fanout limit, its switching/internal/leakage power is taken to be zero. When this parameter is specified, the user-defined power value is applied to all the driver instances that have nets more than or equal to the fanout limit.
<code>-force</code>	Specifies to apply the <code>set_power</code> command even if there are Verilog mismatches. For dynamic or DEF-based static analysis, this parameter allows you to apply <code>set_power</code> to the DEF instances that do not exist in Verilog.
<code>-help</code>	Outputs the command usage.

<pre>-include_ddv_modes {pattern_list}</pre>	Includes only the current characterization (or DDV) modes that match with the specified patterns. This parameter allows you to control the modes to be switched, instead of switching all the current characterization modes defined in the power-grid view. Use Model <pre>set_power -instance -cell name -include_ddv_modes {pattern_list}</pre> Here: • <i>pattern_list</i> is a list of mode names, separated by space or comma • the instance, cell, and pattern names can contain wildcard characters
<pre>-internal</pre>	Specifies the target internal power of the design, cell, or instance. To specify internal power for cells and instances, you can use the <code>-internal</code> parameter in conjunction with the <code>-cell</code> or <code>-instance</code> parameter.
<pre>-instance inst_list value</pre>	Specifies the instance name. If the cell or instance name is not specified, the power will be distributed to all rails.
<pre>-leakage</pre>	Specifies the target leakage power of the design, cell, instance, or power net. To specify leakage power for cells and instances you can use <code>-leakage</code> option in conjunction with <code>-cell</code> or <code>-instance</code> option. To specify leakage power of a certain power net in the design you can use <code>-pg_net</code> option along with <code>-leakage</code> option.
<pre>-no_propagation</pre>	Modifies the behavior of the <code>-dynamic_switch_pattern</code> parameter so that the user-defined switch pattern is applied only after state-propagation is complete. The use model of the command parameter is: <pre>set_power -dynamic_switch_pattern {pattern} - no_propagation</pre> The <code>-no_propagation</code> parameter is only applicable in conjunction with the <code>-dynamic_switch_pattern</code> parameter. When the <code>-no_propagation</code> parameter is added to the <code>-dynamic_switch_pattern</code> parameter, the dynamic power engine performs state propagation first, overrides the single instance switching event by the user-defined dynamic switch pattern, and does no further propagation.

<code>-pg_net railname</code>	Optional. The specified power or pwl for the cell, instance or design is applied to the specified rail. When not specified, the power or pwl is applied to all the rails associated with the instance. This option is generally used for MSMV cells. When this option is not specified the behavior is different for pwl and power options. When not specified and power option is used, the power is applied for the whole design, cell, or instance. When not specified and pwl option is used, the pwl is applied to every rail of the cell or instance. Note: When <code>set_power -pg_net</code> is defined for power rails, the scaled power is also applied for ground rails to ensure current symmetry. Therefore, by default, sum of power rails is same as sum of ground rails. If you do not want to scale ground power, you must define a scaling of 1 for ground rails, as shown in the following example: <pre>set_power -pg_net VSS -scale_factor 1</pre>
<code>-pg_pin {pg_pin}</code>	Specifies the switch pattern for a specific PG pin of an instance. This parameter must be specified with the <code>-dynamic_switch_pattern</code> parameter.
<code>-pin pin_name pin_power</code>	Specifies pin-wise power numbers for a specific instance. This parameter allows you to control the power assigned to power ground pins of an instance. Following is the use model of this parameter: <pre>set_power -instance <inst_name> -pin <VDDA> <pin_power> set_power -instance <inst_name> -pin <VDDDB> <pin_power></pre>
<code>-pwl</code>	Specifies that power argument (<code>power</code> <code>pwl</code>) is a waveform rather than a power number.
<code>-repeat</code>	Repeats dynamic switch pattern of all the clock cycles in the simulation period.
<code>-reset</code>	This parameter causes all previous <code>set_power</code> commands to be ignored. Note: You can selectively reset the instance/cell specifications. In addition, you can also selectively reset the cell and instance specifications of a list of objects through the <code>get_cells</code> and <code>get_lib_cells</code> commands.

<code>-scale_factor value</code>	Scales the power value by the specified scale factor. This parameter allows you to set a user-defined scale factor in the trigger file to scale the dynamic current of the PGV. To modify the scale factor, use the <code>-reset</code> parameter with the <code>-scale_factor</code> parameter. Example: <pre>set_power -leakage -reset -scale_factor 5 - instance A</pre>
<code>-signal_net netname</code>	Specifies the signal net name for power assignment. When this parameter is specified, the driver instances connected to the signal net will be assigned the user-defined power value.
<code>-slew value</code>	Specifies the slew value to all instances with nets more than or equal to the fanout limit (<code>-fanout_limit</code>).
<code>-static_mode {</code> <code> custom_label_name </code> <code> mode_name }</code>	Specifies the mode name or the custom label name associated with a cell or instance for computing instance current from PGV with static multi-voltage multi-frequency (MvMf) modes. This current is used to compute the instance's internal power, and its switching and leakage power components would be set to zero. The use model of this parameter is given below: <pre>set_power -instance -static_mode READ</pre> If you specify custom label of a static MvMf mode, currents would be taken from it directly. Alternatively, if only mode name is specified for a cell/instance, the software snaps to the nearest custom label defined within this mode, based on the instance's voltage and frequency to get the currents.  If you specify a dynamic mode with <code>set_power -static_mode</code>, average current from the mode will be used in the static MvMf flow.
<code>-sticky</code>	Applies the simulation based PWL waveform as is, to the specified cell/instance for dynamic power analysis. Use this option only if you have specified <code>-pwl</code>.
<code>-switching value</code>	Specifies the target switching power of the design, cell, or instance. To specify switching power for cells and instances, you can use the <code>-switching</code> parameter in conjunction with the <code>-cell</code> or <code>-instance</code> parameter.

<code>-trigger_time_adjustment time</code>	Specifies to adjust the trigger time of the current waveforms for the various functional modes (read/write/idle) of the custom macros. This parameter allows you to start the waveform before the trigger time (negative time support), as shown below:  The specified time will be applied to the computed trigger time for each mode in the trigger files generated by power analysis. For example, if the trigger time (from timing window/vector) is computed as 1ns, 2ns, and 3ns, and if you specify <code>set_power -trigger_time_adjustment -100ps</code>, the software will adjust the trigger time to .9ns, 1.9ns, and 2.9ns.
<code>value</code>	Specifies power in milliwatts during static power calculation or piecewise linear (<code>-pwl</code>) current waveform, during dynamic power calculation for cell or instance. The command accepts a unit specification (e.g. 10w). The format of the PWL waveform is: <code>{time1 current1 time2 current2 ...time_n current_n}</code> Time is in ns and current in mA. Example: <code>set_power -pwl -pg_net {} -cell BUF2MTL {0 0 10 0 10.1 5 10.5 0}</code> Here, at 0ns its value is 0mA, at 10ns its value is 0mA, at 10.1ns its value is 5mA, and so on.

Examples

- The following command sets power for vdd rail to 10 milliwatts:
`set_power -pg_net vdd 10mw`
- The following command sets a power value of .005 milliwatts on power net vdd for instance clk_in_neg_L14_I1:
`set_power -pg_net vdd -instance Top/A/clk_in_neg_L14_I1 .005`
- The following command sets a power value of .005 milliwatts for any cell named INVX2 in the design:
`set_power -cell INVX2 .005mw`
- The following command defines a piecewise linear waveform of {0 0 10 0 10.1 5 10.5 0} for the cell of type BUF2MTL for all

power rails.

```
set_power -pwl -pg_net {} -cell BUF2X2MTL {0 0 10 0 10.1 5 10.5 0}
```

- The following command specifies the leakage power for the cell DFFRX1.

```
set_power -leakage -cell DFFRX1 10mw
```

- The following commands specify the target leakage power for power nets VDD1 and VDD2.

```
set_power -leakage -pg_net VDD1 100mw
```

```
set_power -leakage -pg_net VDD2 200mw
```

- The following command specifies the PWL description of current for the instance SEHD130_1024X32X1CM8. The `-sticky` option tells the software to use the specified PWL simulation based waveforms as is.

```
set_power -pwl -instance SEHD130_1024X32X1CM8 -sticky {0 0 1 1 2 2 7 0}
```

- The following command scales the total power of the instance i_1 by a scale factor of 2:

```
set_power -scale_factor 2 -instance i_1
```

- The following command sets the total power of the design to 20mw while keeping the clock network power the same:

```
set_power 20mw -exclude_clock {all}
```

- The following command assigns a switching power of 3mw to all instances of the cell AND1:

```
set_power -cell AND1 -switching 3mw
```

- The following command assigns 2mw power to all cells that match the specified pattern:

```
set_power -cell[get_lib_cells NAND*] 2mw
```

- The following command specifies that mode1 switches in the first cycle, mode2 switches in the third cycle and no switching in others:

```
set_power -dynamic_switch_pattern { mode1 0 mode2 0 }
```

- The following command resets power of all cells that match the specified pattern:

```
set_power -reset -cell[get_cells A/*]
```

- The following command is an example of cell-based switching:

```
set_power -cell HS65_LS_* -dynamic_switch_pattern {1101}
```

- The following command specifies that the switching pattern value 1010 will be applied to all the instance names starting with m_inst:

```
set_power -instance m_inst* -dynamic_switch_pattern {1010}
```

- The following command does not schedule DDV mode scan*:

```
set_power -cell { * } -exclude_ddv_modes { scan* }
```

- The following command does not schedule DDV mode scan* for instance "A1":

```
set_power -instance { A1 } -exclude_ddv_modes { scan* }
```

- The following command only schedules DDV mode names with the `*read*` and `*write*` pattern:

```
set_power -cell { * } -include_ddv_modes { *read*, *write* }
```

- The following commands specify the toggle time for the pin names Q7 and Q6:

```
set_power -dynamic_switch_event { 1.1ns r 5.5ns f } -instance clk_b_i2 -pin Q7
```

```
set_power -dynamic_switch_event { 2.2ns r 6.6ns f } -instance clk_b_i2 -pin Q6 -no_propagation
```

- The following command specifies the toggle time for the cell name BUF8:

```
set_power -dynamic_switch_event { 2.22ns r 6.66ns f } -cell BUF8
```

Trigger File Format

For specifying a custom trigger file, you must use the `-custom_macro_pwl` parameter. The format of the trigger file is:

Trigger File Format	Description
#Cell names	
MASTER_CELL <cellname1>	Name of the reference cell.
COMPANION_CELLS <cellname1 cellname2 ... >	Name of the cells that will use scaled PWL from the master cell.
#Mode and conditional details for trigger	
MODE_NAME <model> CONDITIONAL_INPUT <conditional input statement for cellname1>	Mode name and conditional input statement for the cell. Conditional input of current for the given mode.
CONDITIONAL_PIN <pin_name> (rise fall both)	Conditional pin statement for the cell. Signal pin for which the tool should rise or fall.
CONDITIONAL_STIMULUS_FILE <usim1.txt>	Simulation vector detail in the given mode.
USER_PWL_FILE <pwl1.txt>	User-specified PWL file that contains information about the PWL waveforms (power/ground currents). For an example of a PWL file, see PWL Format Example .
END_MODE	End mode section
#End cell section	
END	End cell section

Example of a Custom Macro Power Trigger File

```
MASTER_CELL          SRAM_A1
COMPANION_CELLS      { SRAM_A2, SRAM_A3, SRAM_A4, SRAM_A5 }
    MODE_NAME WRITE
        CONDITIONAL_INPUT = ( !SD & !SLP & !CEB & !WEB )
        CONDITIONAL_PIN = CLK { RISE }
        USER_PWL_FILE write_rise.txt
    END_MODE
    MODE_NAME READ
        CONDITIONAL_INPUT = ( !SD & !SLP & !CEB & WEB )
        CONDITIONAL_PIN = CLK { RISE }
        USER_PWL_FILE read_rise.txt
    END_MODE
...
END
```

Example of “USER_PWL_FILE” (Multiple regions)

UNIT CURRENT mA

UNIT TIME ns

NET VSS

REGION { 0 0 0.1 -1 0.2 -2 0.3 -3 0.4 -10 0.5 -50 0.6 -40 0.7 -30 0.8 -20 1 0 2 0 3 0 3.9 0 4 -
0.1 4.5 -0.2 5 -4 5.5 -3 6 -2 7 0 }

NET VDD

REGION { 0 0 0.1 1 0.2 2 0.3 3 0.4 10 0.5 50 0.6 40 0.7 30 0.8 20 1 0 2 0 3 0 3.9 0 4 0.1 4.5 0.2
5 4 5.5 3 6 2 7 0 }

NET VDDM

REGION { 0 0 0.1 1 0.2 2 0.3 3 0.4 10 0.5 50 0.6 40 0.7 30 0.8 20 1 0 2 0 3 0 3.9 0 4 0.1 4.5 0.2
5 4 5.5 3 6 2 7 0 }

set_power_analysis_mode

```
set_power_analysis_mode
[-help]
[-reset]
[-add_simulation {true | false}]
[-adjust_input_activity_in_iterations {true | false}]
[-adjust_macro_activity_in_iterations {true | false}]
[-annotation_detail_report {true | false}]
[-auto_twf_delay_annotation {true | false}]
[-average_rise_fall_cap {true | false}]
[-binary_db_name filename ]
[-block_independent_peakpower {true | false} ]
[-boundary_gate_leakage_file filename]
[-boundary_gate_leakage_report {true | false}]
[-boundary_leakage_cell_property_file filename]
[-boundary_leakage_edge_annotation_file filename]
[-boundary_leakage_multi_pgpin_support {true | false}]
[-boundary_leakage_pessimism_removal {true | false}]
[-boundary_leakage_PXE_support {true | false}]
[-bulk_pins {bulk_pin_list}]
[-capacity {low|medium|high}]
[-case_insensitive_mapping {true | false}]
[-clock_source_as_clock {true | false}]
[-comprehensive_automapping {true | false}]
[-compress_reports {true|false|1-9}]
[-constant_override {true | false}]
[-corner {min|max}]
[-create_binary_db {true | false}]
[-create_driver_db {true | false}]
[-create_gui_db {true | false}]
[-current_generation_method { avg | peak }]
[-decap_cell_list cell_list ]
[-default_frequency value]
[-default_slew value]
[-default_supply_voltage value]
[-detailed_view_current_only {true | false} ]
[-disable_clock_gate_clipping {true | false}]
[-disable_leakage_scaling {true | false}]
[-disable_static {true | false}]
[-distribute_switching_power {true | false}]
[-distributed_combine_report_format {none | detailed | reduced}]
[-distributed_setup file]
[-domain_based_clipping {true | false} ]
[-dynamic_glitch_filter value]
[-dynamic_scale_clock_by_frequency { freq_A factor_A freq_B factor_B freq_C factor_C ... }]
[-dynamic_scale_clock_by_name { clk_A factor_A clk_B factor_B clk_C factor_C ... }]
[-dynamic_vectorless_ranking_methods {load | clock | vector_activity}]
[-effective_load_cap_method {lumped | max | avg | min}]
[-effective_load_cap_slew_threshold_limit value]
[-enable_auto_queue {true | false}]
[-enable_auto_mapping {true | false}]
[-enable_disk_mapping {true | false}]
```



```
[-enable_duty_prop_with_global {true | false}]
[-enable_dynamic_current_slew_load_interpolation {true | false}]
[-enable_dynamic_scaling {true | false}]
[-enable_flop_state_propagation {true | false}]
[-enable_force_propagation {true | false}]
[-enable_generated_clock {true | false}]
[-enable_input_net_power {true | false}]
[-enable_interactive_reports {true | false}]
[-enable_mt_in_vectorbasedflow {true | false} ]
[-enable_mt_reports {true | false}]
[-enable_mt_state_propagation {true | false} ]
[-enable_pba_for_tempus_pi {true | false}]
[-enable_power_target_flow {true | false}]
[-enable_rtl_vectorbased_dynamic_analysis {true | false}]
[-enable_scan_report {true | false}]
[-enable_single_cycle_scheduling {true | false}]
[-enable_slew_based_ccs_pin_cap {true | false}]
[-enable_stable_clock_gating {true | false}]
[-enable_stable_flop_scheduling {true | false}]
[-enable_state_propagation {true | false}]
[-enable_tempus_pi {true | false}]
[-enable_xp {true | false}]
[-enhanced_blackbox_avg {true | false}]
[-enhanced_blackbox_max {true | false}]
[-equivalent_annotation {true | false}]
[-event_based_leakage_power {true | false}]
[-external_load_config_file filename]
[-extraction_tech_file filename]
[-extractor_include filename]
[-fanout_limit value ]
[-flatten_xpgv_block_instances filename]
[-force_library_merging {true | false}]
[-from_x_transition_factor value]
[-from_z_transition_factor value]
[-generate_activity_mapping_report {true | false}]
[-generate_current_for_rail railnames ]
[-generate_flop_ranking_data directoryname]
[-generate_leakage_power_map_based_on_calculated_leakage {true | false}]
[-generate_static_report_from_state_propagation {true | false}]
[-handle_glitch {true|false}]
[-handle_transport_glitch {true | false}]
[-handle_tri_state {false|true} ]
[-hier_delimiter character]
[-honor_combinational_logic_on_clock_net {true | false}]
[-honor_default_switching_activity_in_scheduling {true|false}]
[-honor_negative_energy {true | false}]
[-honor_net_activity {true | false}]
[-honor_non_mission_leakage {true | false} ]
[-hybrid_analysis {true | false}]
[-ignore_control_signals {true | false}]
[-ignore_data_phase_for_clk {true | false}]
[-ignore_end_toggles_in_profile {true | false}]
[-ignore_glitches_at_same_time_stamp {true | false}]
[-ignore_inout_pin_cap {true | false}]
[-ignore_macro_leakage_scale_for_temp {true | false}]
```

```
[--ignore_unconstraint_net_with_timing_data {true | false}]
[--include_seq_clockpin_power {true | false}]
[--include_timing_in_current_file {true | false}]
[--ir_derated_timing_view view_name]
[--keep_clock_gate_ratio_in_iterations {true | false}]
[--leakage_scale_factor_for_temp scale ]
[--library_preference {voltage | eesm_ccsp}]
[--macro_pgv_leakage {true | false}]
[--macro_toggle_pin_percentage percentage ]
[--mbff_toggle_behavior { simultaneous | independent | sbff }]
[--mbff_toggle_pin_percentage percentage ]
[--memory_current_scaling_method {1 | 2 | 3}]
[--merge_switched_net_currents {true | false}]
[--method { static | dynamic_vectorless | dynamic_vectorbased | dynamic_mixed_mode | event_based |
vector_profile}]
[--min_leaf_count value]
[--multi_scenario_simulation {true | false}]
[--off_pg_bulk_leakage {true | false}]
[--off_pg_nets net _list]
[--output_current_data_prefix prefix ]
[--partition_count value]
[--partition_spef {true | false}]
[--partition_twf {true | false}]
[--pin_based_twf {true | false}]
[--power_grid_library { library _list } ]
[--power_include_initial_x_transitions {true | false}]
[--power_match_state_for_logic_x value ]
[--precision value]
[--pre_simulation_empty_period period]
[--pre_simulation_period period]
[--pre_simulation_power_exclude_period period]
[--quit_on_activity_coverage_threshold threshold_value ]
[--read_rcdb {true | false}]
[--relax_arc_match {true | false}]
[--report_black_boxes {true | false}]
[--report_clock_instance_switching_info {true | false}]
[--report_idle_instances {true | false}]
[--report_instance_switching_info {all | output_logic | none} ]
[--report_instance_switching_list filename]
[--report_library_usage {true | false}]
[--report_missing_bulk_connectivity {true | false}]
[--report_missing_input {true | false}]
[--report_missing_nets { true|false }]
[--report_scan_chain_stats {true | false}]
[--report_stat {true | false}]
[--report_time_display_fraction_digits value]
[--report_twf_attributes {detailed | summary}]
[--reuse_flop_ranking_data directoryname]
[--reuse_flop_ranking_data_hier blockname]
[--save_bbox {true | false}]
[--scale_to_sdc_clock_frequency {true | false}]
[--scan_chain_activity {design_name chain_name activity_number} | {vectorless activity_number}]
[--scan_chain_name {design_name chain_name pattern ...}]
[--scan_control_file filename]
[--scan_mbff_chain_type type]
```

```
[-settling_buffer value]
[-show_cell_usage_stats {true | false}]
[-smart_window {true | false}]
[-split_bus_power {true | false}]
[-start_time_alignment {true | false}]
[-state_dependent_leakage {true | false}]
[-stateprop_ignore_unannot_pins {true | false}]
[-static_multi_mode_scenario_file filename]
[-static_netlist {verilog | def}]
[-switching_power_on_rise_only {true | false}]
[-thermal_input_file file]
[-thermal_leakage_temperature_scale_table_file filename]
[-to_x_transition_factor value]
[-to_z_transition_factor value]
[-transition_factor_based_duty {true | false } ]
[-transition_time_method {min | avg | max}]
[-twf_delay_annotation {min | avg | max}]
[-twf_load_cap {min | avg | max | ignore}]
[-unified_power_switch_flow {true | false}]
[-use_cell_leakage_power_density {true | false}]
[-use_fastest_clock_for_dynamic_scheduling {true | false}]
[-use_lef_for_missing_cells {true | false}]
[-use_only_ddv_current {true | false}]
[-use_physical_partition {true | false}]
[-use_twf_stable_randomized_arrival_delay {none | data_only | all}]
[-use_zero_delay_vector_file {true | false}]
[-vector_profile_mode {activity | event_based | power_density | transient}]
[-worst_case_vector_activity {true | false}]
[-worst_step_size value]
[-worst_window_count value]
[-worst_window_coverage { top | tile } ]
[-worst_window_reports {full | worst | both}]
[-worst_window_size value]
[-worst_window_type { power | activity | delta_power | delta_activity | ir | critical_inst_power } ]
[-write_boundary_leakage_edge_annotation {true | false}]
[-write_default_uti {true | false}]
[-write_dynamic_currents {true | false}]
[-write_profiling_db {true | false}]
[-write_simulation_db {true | false}]
[-write_static_currents {true | false}]
[-x_count_transition_using_3_states {true | false}]
[-x_transition_factor value ]
[-z_transition_factor value ]
[-zero_delay_vector_toggle_shift value]
```

Sets the parameters for doing the power analysis.

Parameters

<code>-help</code>	Prints out the command usage.
--------------------	-------------------------------

`-add_simulation {true | false}`

Default: false

Specifies to add simulation if there are no simulation vectors or if all primary sources are not annotated by vectors. This parameter can be used to ensure complete design coverage. Event simulation is done for primary sources like primary input, flops, and macros.

Usage Guidelines:

This parameter is only applicable to the event-based power analysis flow (`set_power_analysis_mode -method event_based`).

`-adjust_input_activity_in_iterations {true | false}`

Default: true

Specifies whether to keep the input activity fixed or automatically adjust the activity value for each iteration of state propagation.

Usage Guidelines:

This parameter is only applicable to the power target flow (`set_power_analysis_mode -enable_power_target_flow true`).

`-adjust_macro_activity_in_iterations {true | false}`

Default: true

Specifies whether to keep the macro activity fixed or automatically adjust the activity value for each iteration of state propagation.

Usage Guidelines:

This parameter is only applicable to the power target flow (`set_power_analysis_mode -enable_power_target_flow true`).

`-annotation_detail_report {true | false}`

Default: false

Specifies to generate a detailed annotation report per instance in the Vector Profiling flow (`set_power_analysis_mode -method vector_profile`). When specified, it creates the `AnnotationDetail.rpt` file in the user-specified output directory.

`-auto_twf_delay_annotation {true | false}`

Default: false

Specifies to automatically detect the zero-delay vectors and perform delay annotation. As a result, you do not need to specify additional parameters (`set_power_analysis_mode -use_zero_delay_vector_file` and `read_activity_file -zero_delay`) to indicate the vectors that have delay annotation.

Usage Guidelines:

This parameter is only applicable to the event-based power analysis flow (`set_power_analysis_mode -method event_based`).

`-average_rise_fall_cap {true | false}`

	Default: <code>false</code> When set to <code>true</code>, the software uses the average of rise and fall capacitance from the liberty file. This parameter can be specified for all library formats. Usage Guidelines: This parameter is not applicable to the <code>-static_netlist def flow</code>.
<code>-binary_db_name filename</code>	
	Default: The default file name is <code>power.db</code> Specifies the name of the binary power database file (<code>filename.db</code>) used for analyzing power plots. This database contains instances, locations, and power. The generated power database allows you to perform GUI interactive debugging using the <i>Power & Rail Plots</i> and <i>Power Debug</i> forms. You can specify this parameter in scenarios where more than one database is generated and you need to track which one was generated in which mode when restoring the database, or if your design has several blocks and analysis is done on one block all the time and all the databases are named <code>power.db</code>. Usage Guidelines: You must specify this parameter with the <code>-create_binary_db true</code> parameter. This parameter is applicable to both the static and dynamic power calculation flows.
<code>-block_independent_peakpower {true false}</code>	
	Default: <code>false</code> Specifies to identify the worst power interval of each block, and run power analysis of the full design using the worst interval of each block. This parameter allows you to perform power and current analysis for design blocks that have independent peak power intervals. When set to <code>false</code>, power analysis is performed using the common worst interval found using simulation of all vectors together.
<code>-boundary_gate_leakage_file filename</code>	
	Specifies a list of Liberty side files required for boundary gate leakage (context-aware leakage) calculation. This file defines the boundary current for different threshold voltage (Vt) layers at different operating conditions. The file includes: - multiple leakage tables corresponding to different operating conditions. - leakage probability factor for different abutments (when abutting instances are on the same Vt layer or when they are on different Vt layers). When <code>-boundary_gate_leakage_file</code> is specified, the <code>power.boundarygateleakage.rpt</code> file is dumped by default. This file contains the design-level summary for the three combinations of Vt select fail ratio (0, 1 and the one specified with <code>HighVT_select_fail_ratio</code>).
<code>-boundary_gate_leakage_report {true false}</code>	
	Default: <code>false</code> Generates a verbose report for every abutted instance-edge for boundary leakage calculation. This detailed report is useful in debugging and understanding the leakage power calculation for an instance. The generated report name is <code>boundarygateleakage.detailed.rpt</code>.

`-boundary_leakage_cell_property_file filename`

Specifies the list of edge information files containing cell properties.

`-boundary_leakage_edge_annotation_file filename`

Specifies the edge annotation file with updated Vt layers at a given edge location to be used during boundary leakage calculation. This parameter is used to provide custom Vt layers for an abutted edge at a given location to be used during power computation.

Note: When running the edge annotation feature, the user-defined VT will have precedence over the one from the cell properties file.

`-boundary_leakage_multi_pgpin_support {true | false}`

Default: false

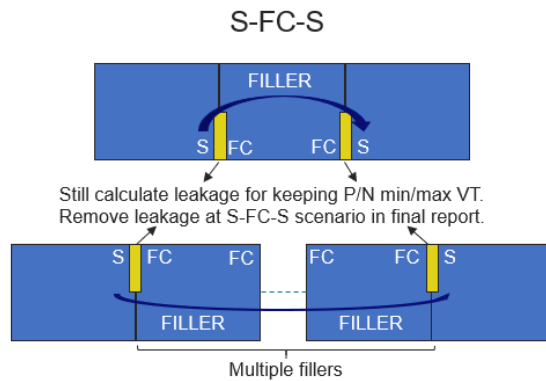
Enables support for cells with multiple power/ground pins.

`-boundary_leakage_pessimism_removal {true | false}`

Default: true

Enables pessimism removal for the Source to FCs to Source abutment type. This abutment type normally has 0 CNOD leakage. Therefore, the non-zero leakage computed at the starting S-FC and terminating FC-S boundaries needs to be removed for accurate estimation. This parameter is specified to remove the non-zero leakage or pessimism.

Revised CNOD leakage = CNOD Leakage – Leakage (S-FC-S).

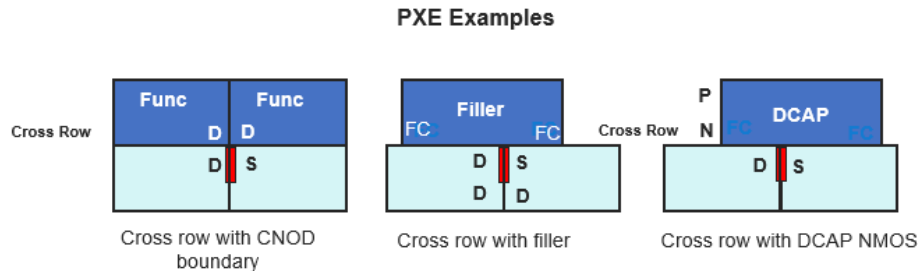


`-boundary_leakage_pxe_support {true | false}`

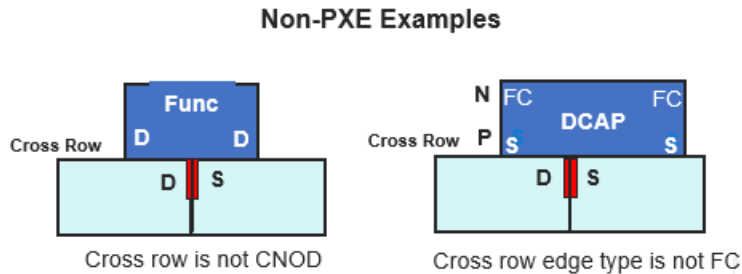
Default: `false`

Enables the PXE feature. PXE is used for calculation of CNOD leakage by cross row environment. The T structure or inverted T structure with both edges of the type FC is PXE.

The following diagram illustrates examples of PXE:



Every scenario other than the ones mentioned in PXE examples is non-PXE. The following diagram illustrates examples of non-PXE:



- If this parameter is set to `false`, Voltus does not consider the relationship by cross row environment.
- If this parameter is set to `true`, Voltus will find out the PXE or non-PXE structure and look up different leakage tables for obtaining the leakage value.

`-bulk_pins {bulk_pin_list}`

Default: None

Defines power and ground bulk LEF pins for the design. Use this parameter when library cells have bulk pins defined in LEF but the liberty file does not contain power associated with these pins. If a liberty file does not have body bias definition, power analysis does not distribute power to the body bias domain.

Usage Guidelines:

This parameter is only applicable to the static power calculation flow. It does not support the `-static_netlist def` and dynamic power calculation flows.

`-capacity {low|medium|high}`

	Enables segmented power processing. When this parameter is specified, the software performs concurrent processing of vector simulation annotation, power calculation, and report construction. This parameter enables performance optimization with some trade-off on memory consumption and runtime. The arguments of the <code>-capacity</code> parameter are: • <code>low</code> - reduces memory for a flat run. • <code>medium</code> - further reduces memory with moderate increase in runtime (1.2X). • <code>high</code> - maximum memory reduction at about 2X runtime. The <code>-capacity</code> parameter is used for reducing memory when processing long simulation vectors. In case of the FSDB and SHM format activity file, the runtime can increase up to 2X.
<code>-case_insensitive_mapping {true false}</code>	
	Default: <code>false</code> Specifies whether instance name mapping between the RTL netlist and GATE level netlist in the mapping file is case-sensitive or case-insensitive. When set to <code>true</code>, name mappings are case-insensitive.
<code>-clock_source_as_clock {true false}</code>	
	Default: <code>false</code> Specifies to use the correct clock frequency for activity calculation. If you stop the arrival of the clock signal by setting <code>set_case_analysis</code> to 0, the transition density at the output net of the flop is calculated using the topological source clock frequency instead of the default frequency. When this option is set to <code>false</code>, then the Verilog flow will stop clock propagation for any disable clock path that is set with <code>set_case_analysis</code>. Usage Guidelines: This parameter is not applicable to the <code>-static_netlist def flow</code>.
<code>-comprehensive_automapping {true false}</code>	
	Default: <code>false</code> Specifies to perform all-inclusive instance name mapping between the RTL netlist and GATE level netlist. When set to <code>true</code>, the software honors all possible naming rules (includes handling of characters like <code>['', ']', ':', '/', '\', '_', ':'</code> and handling of strings like <code>"MBFF"</code>, <code>"_reg"</code>, <code>"_MB_"</code> etc.) for better annotation coverage. This feature is applicable to both file-based mapping (<code>map_activity_file -rtl2gate -two_column_mapping_file</code>) and automatic mapping (<code>set_power_analysis_mode -enable_auto_mapping</code>). When <code>-comprehensive_automapping</code> is specified with either <code>-rtl2gate</code> or <code>-two_column_mapping_file</code>, the RTL mapping entries are given priority over the automatic rule-based mapping entries and are used to annotate the design objects.
<code>-compress_reports {true false 1-9}</code>	

	Specifies to compress the power output files in the GNU Zip format. The compressed files will have the suffix .gz. The use model of this parameter is: <pre>set_power_analysis_mode -compress_reports {true false 1-9}</pre> • true: Sets the compression option with compression level 6. • false: default, no compression • 1-9: the compression levels vary from 1 to 9. 1 provides the fastest compression speed but at a lower ratio, so the file size will be larger. 9 provides the highest compression ratio but at a lower speed.
<code>-constant_override {true false}</code>	
	Default: false Specifies to give higher precedence to propagated constants defined using the <code>set_case_analysis</code> command. It allows propagated constants to override global activity.
<code>-corner {min max}</code>	
	Default: max Defines the library corner for the non-MMMC setup (<code>read_lib -min -max -typ</code>). • min corner means the power calculator will use min timing libraries for power calculation. • max corner means the power calculator will use max timing libraries for power calculation. Usage Guidelines: This parameter is applicable to both the static and dynamic power calculation flows.
<code>-create_binary_db {true false}</code>	
	Default: false A value of true creates a binary database of power results. This database contains instances, locations, and power. The generated power database allows you to perform GUI interactive debugging using the <i>Power & Rail Plots</i> and <i>Power Debug</i> forms. Usage Guidelines: This parameter is applicable to both the static and dynamic power calculation flows.
<code>-create_driver_db {true false}</code>	

	Default: <code>false</code> Specifies to generate all the flop-based drivers/receivers list. This parameter is required to run the noise margin calculation flow, and differential voltage calculation in the rush-current analysis flow. The driver/receiver pairs are generated only for the state-elements (flops) as noise margins will be calculated only for the driver-receiver pairs where flop is a receiver. Noise margins are calculated with worst voltages seen on the driver-receiver pins. The driver/receiver list is saved in the <code>power2rail_config.json</code> file located in the power output directory. Usage Guidelines: This parameter is applicable only to the dynamic power calculation flow.
<pre>-create_gui_db {true false}</pre>	
	Default: <code>false</code> Specifies to create a binary database of power results in the XP mode. When set to <code>true</code>, the tool creates sliced power database file for each rail partition and saves it to the <code>gui.db</code> sub-directory under the power output directory (example: <code>power_dir/part_*/gui.db/rail_part_*/power.x.db</code> ($x = 1, 2, \dots, 14$). The generated power database allows you to perform GUI interactive debugging using the <i>Power & Rail Plots</i> and <i>Power Debug</i> forms. Usage Guidelines: This parameter is applicable to both the static and dynamic power calculation flows.
<pre>-current_generation_method { avg peak }</pre>	
	Default: <code>avg</code> Specifies the method of instance current discretization (or continuous current waveform generation) based on the user-specified time step size (<code>set_dynamic_power_simulation -resolution <value in ps></code>). The current waveform generated for an instance depends on the time step size/resolution for dynamic power simulation. The recommended value for <code>-resolution</code> is 20ps. The two possible arguments of this parameter are: • <code>peak</code> - Specifies to use the peak current value of each time step. The output current filename is <code>filename.ptipeak</code>. It is recommended to use the <code>peak</code> method when using the Composite Current Source Power (CCSP) Liberty files. This is because the CCSP library provides real current waveforms from simulation that is used directly by the software. • <code>avg</code> - Specifies to use the average current value of each time step. The output current filename is <code>filename.ptiavg</code>. It is recommended to use the <code>avg</code> method when using the non-CCSP (NLPM) library. In this case, the Voltus power engine heuristically constructs a triangular waveform from the look-up table. Usage Guidelines: • This parameter is applicable only to the dynamic power calculation flow. • It is used only for instance current waveform generation, and does not apply to total current waveform generation. When generating total current (composite) waveform, the peak current values of all instances are summed up for each step. • For very small step size, the average and peak methods will converge.
<pre>-default_frequency value</pre>	

	Default: 100MHz Specifies to set the default frequency in the MHz unit for net not annotated by TWF. You can specify a numerical value without unit.
<code>-default_slew value</code>	
	Default: For static analysis CTE flow, the default slew = 0. For TWF flow, the default slew = 0.1 * default frequency in MHz. Specifies to set the default slew in the ns unit for net not annotated by TWF. You can specify a numerical value without unit. Usage Guidelines: This option depends on <code>-default_frequency</code> for the TWF flow. If <code>-default_frequency</code> is not specified, then it will take frequency of the shortest clock period.
<code>-default_supply_voltage value</code>	
	Default: 1.0 Specifies to set a default voltage for power nets in a scenario where the power engine cannot determine the voltage of these nets.
<code>-detailed_view_current_only {true false}</code>	
	Default: true Controls instance power reporting for cells that have power information available in PGV. When set to <code>true</code>, specifies to skip writing of the switching and leakage current and use only the internal current from DDV PGV. When set to <code>false</code>, specifies to report the switching and leakage current values in addition to the internal current from DDV PGV. If a cell is defined in both PGV and dotlib, leakage is computed from dotlib. It is recommended to set this parameter to <code>true</code> because that would prevent over-estimation of the instance current.
<code>-disable_clock_gate_clipping {true false}</code>	
	Default: true Specifies to enable clock gate output clipping so that the output transition density of an ICG cell does not exceed the input clock pin transition density.
<code>-disable_leakage_scaling {true false}</code>	
	Default: false Specifies to exclude leakage power during scaling factor computation when target power is specified using the <code>set_power</code> command. When <code>set_power_analysis_mode -disable_leakage_scaling</code> is set to <code>true</code>, the leakage power does not get scaled, and only the internal and switching components are scaled to meet the user-specified total power. This parameter is relevant only for the <code>set_power</code> commands for which total power is specified, and has no effect for the <code>set_power</code> command specified with the <code>-internal</code>, <code>-switching</code>, or <code>-leakage</code> parameters.
<code>-disable_static {true false}</code>	

	Default: false When set to true in the dynamic vector-based or dynamic vectorless flows, the command performs only dynamic analysis and turns-off static power calculation. Note: When you perform static power calculation in the dynamic power analysis flow, you can use either the <code>-disable_static</code> or <code>-write_static_currents</code> parameter to enable static power calculation and write static current file. If only <code>-disable_static</code> is set to false and <code>-write_static_currents</code> is not set, the tool automatically sets <code>-write_static_currents</code> to true. Similarly, if <code>-write_static_currents</code> is set to true, and <code>-disable_static</code> is not set, the tool automatically sets <code>-disable_static</code> to false.
	<pre>-distribute_switching_power {true false}</pre>
	Default: false Specifies to equally distribute the switching power among all drivers if a net has more than one driver. When set to false, stores all switching power on one of the net drivers.
	<pre>-distributed_combine_report_format {none detailed reduced}</pre>
	Specifies to generate consolidated XP power reports. This parameter allows you to combine power reports that are stored across multiple partition directories to give a single unified view of power data. The possible arguments of this parameter are: • none - generates distributed reports. • reduced - generates consolidated reports for the following: ◦ power.missingdata.* (static and dynamic flows) ◦ power.stateprop.* (dynamic flow) ◦ power.stats.setpower (dynamic flow) • detailed - generates consolidated reports for the following: ◦ power.rpt (static and dynamic flows) ◦ power.instpwr.rpt (static flow) ◦ power.lib.rpt (static flow) ◦ power.pgpinpwr.rpt (static flow) ◦ power.pgnet.detailed.rpt ◦ power.missingdata.* (static and dynamic flows) ◦ power.pgnet.detailed.rpt (static and dynamic flows) ◦ power.stateprop.* (dynamic flow) ◦ power.stats.setpower (dynamic flow) ◦ pg_trigger.txt (dynamic flow) ◦ trigger.txt (dynamic flow) ◦ *.totalcurrent (dynamic flow) ◦ voltus_power.cellstat (static and dynamic flows) ◦ voltus_power.ptifiles (static and dynamic flows)

`-distributed_setup file`

Default: ""

Specifies a file containing the customized setup details for running power analysis in the distributed mode.

The specified file has 4 columns:

`INST/CELL <name> block <power.inc>`

where,

- `INST/CELL`: Keyword indicating cell or instance to be run in the distributed mode. Cell can be useful for repeated DEF blocks.
- `name`: Name of the hierarchical block or instance.
- `block`: Keyword `block` implies that only the lower level DEF will be included
- `power.inc`: name of the power include file. This column is optional.

The unspecified instance/cell will be run with the top level as a single group. Each distributed block/instance should have corresponding TWF and SPEF to improve runtime.

Note: Each block will be run independently from other blocks, thus the external load or cell seen at the pin of the block can be different from flat analysis. If the partition boundary analysis is important, distributed processing is not recommended.

Usage Guidelines:

- This parameter is not applicable to the `-static_netlist verilog` flow.

`-decap_cell_list cell_list`

Default: ""

Specifies the physical only cells, such as decap cells, to include in the power report.

Usage Guidelines:

- This parameter is applicable only to the static power calculation flow.
- This parameter is not applicable to the `-static_netlist def` flow.

`-domain_based_clipping {true | false}`

Default: false

Determines the frequency to be used to clip propagated activity that is too high. When set to `false`, the frequency of the fastest clock in the design is used. If you set this parameter to `true`, the fastest clock domain on the net is used to clip propagated activity.

`-dynamic_glitch_filter value`

	Default: -1 Sets the absolute threshold time (in ns) for filtering glitches in net toggles. When you specify a time value for this parameter, glitches in net toggles that are equal to or smaller in width than the specified time value are removed. Usage Guidelines: This parameter is applicable only to the state-propagation-based vectorless flow (<code>set_power_analysis_mode -method dynamic_vectorless -enable_state_propagation true</code>).
<code>-dynamic_scale_clock_by_frequency { freq_A factor_A freq_B factor_B freq_C factor_C ... }</code>	
	Specifies the scale factors for the clock frequency values. This parameter allows you to perform frequency scaling for clocks running at relatively low frequencies, thereby ensuring higher toggle coverage for limited simulation period. The parameter is applicable only to the dynamic vectorless flow. The following units are supported for frequency: Hz, KHz, MHz, and GHz.
<code>-dynamic_scale_clock_by_name { clk_A factor_A clk_B factor_B clk_C factor_C ... }</code>	
	Specifies the scale factors for the low frequency clocks. This parameter allows you to perform frequency scaling for clocks running at relatively low frequencies, thereby ensuring higher toggle coverage for limited simulation period. The parameter is applicable only to the dynamic vectorless flow. The following units are supported for frequency: Hz, KHz, MHz, and GHz.
<code>-dynamic_vectorless_ranking_methods {load clock vector_activity}</code>	
	Default: "" Specifies to change the ranking method from the default to the following: <code>load</code> - Prioritizes scheduling cells with higher fanout <code>clock</code> - Prioritizes scheduling cells associated with faster clocks <code>vector_activity</code> - Prioritizes scheduling cells with higher activity Usage Guidelines: - This parameter is applicable only to the state-propagation-based vectorless flow (<code>set_power_analysis_mode -method dynamic_vectorless -enable_state_propagation true</code>). - This parameter is not applicable to the <code>-static_netlist verilog</code> flow.
<code>-effective_load_cap_method {lumped max avg min}</code>	
	Default: <code>lumped</code> Specifies the method to determine the slew ratio. This parameter has the following arguments: <code>lumped</code> - the slew ratio is 1, which indicates that a lumped capacitance is used. <code>max</code> - the slew ratio is calculated as $\text{max}(\text{slew of all drivers}) / \text{max}(\text{slew of all receivers})$. <code>min</code> - the slew ratio is calculated as $\text{min}(\text{slew of all drivers}) / \text{min}(\text{slew of all receivers})$. <code>avg</code> - the slew ratio is calculated as $\text{avg}(\text{slew of all drivers}) / \text{avg}(\text{slew of all receivers})$.
<code>-effective_load_cap_slew_threshold_limit value</code>	

	Default: 0 Specifies the minimum threshold value for slew ratio to prevent effective capacitance from being derated beyond realistic limits. You can specify a float value between 0 and 1.
<code>-enable_auto_queue {true false}</code>	
	Default: false Specifies to automatically determine the queue for power jobs. This parameter is used in the Voltus-XP distributed power flow with hierarchical DEFs. In the XP flow, partitions are divided into multiple queues depending on the number of CPUs (For example, Default queue: 1 CPU, Small queue: 2 CPUs, Medium queue: 4 CPUs, Large queue: 8 CPUs, and so on). By default, all power partitions run on the same queue. When the parameter is set to <code>true</code>, the blocks can run on different queues depending on the size (number of instances) of the blocks. The <code>-enable_auto_queue</code> parameter will be beneficial when there are uneven power block partitions. For example, if there are some blocks with 20-30M instances and there are some with 3-5M instances. Using this parameter, bigger blocks will be automatically assigned to a bigger queue, resulting in better runtime.
<code>-enable_auto_mapping {true false}</code>	
	Default: false Specifies to automatically perform instance name mapping between the RTL netlist and GATE level netlist. This parameter allows you to map the RTL vector to the Gate level netlist without a mapping file. You can use this parameter with the <code>read_activity_file -name_mapping_rule <filename></code> command parameter to specify a file containing customized or design-specific rules. Usage Guidelines: This parameter is applicable only to the vector-based static/dynamic power calculation flow (<code>set_power_analysis_mode -method static dynamic_vectorbased dynamic_mixed_mode</code>).
<code>-enable_disk_mapping {true false}</code>	
	Default: false Specifies to enable disk caching of the toggles. Therefore, the toggles are stored in the disk instead of keeping them in memory. This has made large vectors or long simulation duration runs possible even with lesser available memory. Usage Guidelines: This parameter is only applicable to the event-based power analysis flow (<code>set_power_analysis_mode -method event_based</code>) in the XP mode.
<code>-enable_duty_prop_with_global {true false}</code>	
	Default: false Specifies to propagate duty cycle with the global switching activity setting for all data nets in the dynamic vectorless analysis flow. Usage Guidelines: This parameter is applicable to the static power calculation and probability-based dynamic vectorless flows. It is not supported in the state-propagation-based vectorless flow (<code>set_power_analysis_mode -enable_state_propagation true</code>).

`-enable_dynamic_current_slew_load_interpolation {true | false}`

Default: false

Specifies to use the charge-based slew load interpolation. When this parameter is set to `true`, Power Analysis takes dynamic currents from the 4 nearest slew/load values and performs either a 4-way (lower slew, lower load, upper slew, upper load) or a 2-way (lower slew, upper slew OR lower load, upper load) interpolation to calculate the dynamic current waveform. The parameter allows you to derive a more realistic dynamic current waveform based on the exact slew load values from the .lib dynamic current table.

`-enable_dynamic_scaling {true | false}`

Default: false

Enables `set_power` scaling for dynamic power analysis. The power and scaling specifications are used to scale dynamic current on both the power and ground currents of the specified instance/cell.

This parameter honors the user-defined average (static) power for leaf-level instances and hierarchies (instances/cells/modules) along with power net specification in the dynamic current scaling flows such that the average of the dynamic current is equal to the assigned average power. When you specify this parameter, the average of the dynamic current of these leaf-level instances will match the scaled static power numbers.

Usage Guidelines:

- This parameter is applicable to the probability-based vectorless flow. It is not supported in the state-propagation-based vectorless flow (`set_power_analysis_mode -enable_state_propagation true`).
- This parameter is not applicable to the `-static_netlist verilog` flow.

`-enable_flop_state_propagation {true | false}`

Default: false

Enables accurate clock state propagation for sequential cells. When this parameter is set to `true`, flops propagate in a manner that toggles on the output pins are in sync with the clock. This parameter helps to align the output pins' toggles with the clock toggles.

`-enable_force_propagation {true | false}`

Default: false

Enables force propagation to ensure that the output toggles are according to the clock signal rather than the input pin states. This parameter assigns clock activity to outputs when input propagation is not feasible, thereby enhancing overall activity.

`-enable_generated_clock {true | false}`

Default: true

Specifies to get generated clock frequency during activity propagation for static power analysis.

Usage Guidelines:

- This parameter is applicable only to the static power calculation flow.
- This parameter is not applicable to the `-static_netlist def` flow.

`-enable_input_net_power {true | false}`

	Default: <code>false</code> Specifies to calculate the switching power of input nets. Usage Guidelines: This parameter is applicable only to the static power calculation flow.
<code>-enable_interactive_reports {true false}</code>	
	Default: <code>false</code> Specifies to generate multiple dynamic reports based on the power analysis run done for a netlist. When this parameter is set to <code>true</code>, you can generate various types of power reports interactively using the <code>report_power</code> command without the need to perform power calculation on the same netlist again. The <code>-enable_interactive_reports</code> parameter is useful when you need to generate reports for the unaltered netlist within the same power analysis session. Usage Guidelines: This parameter is applicable only to the event-based power analysis flow with area-based partitioning (<code>-use_physical_partition</code>) in the XP mode.
<code>-enable_mt_in_vectorbasedflow {true false}</code>	
	Default: <code>false</code> Specifies to enable multi-threading for the VCD/FSDB based dynamic vector-based flows. To enable multi-threading in the vector-based flows, set this parameter to <code>true</code>. Usage Guidelines: • This parameter is applicable only to the dynamic vector-based flow. • This parameter is not applicable to the <code>-static_netlist verilog</code> flow.
<code>-enable_mt_reports {true false}</code>	
	Default: <code>false</code> Specifies to enable multi-threading for generation of power calculation reports. When set to <code>true</code>, it improves the memory and runtime for power reporting in large designs with more than 100 million instances.
<code>-enable_mt_state_propagation {true false}</code>	
	Default: <code>true</code> Specifies to enable multi-threading in state propagation to help improve the runtime for large simulations. For example, when the parameter was specified for a design with 300 million instances and simulation duration of 120 microseconds, the runtime has improved from 21 hours to 1.5 hours. Usage Guidelines: This parameter is only applicable to the event-based power analysis flow (<code>set_power_analysis_mode -method event_based</code>).
<code>-enable_pba_for_tempus_pi {true false}</code>	

	Default: <code>false</code> Specifies to enable the Path-Based Analysis (PBA) for slack calculation in the Tempus PI flow. By default, the Graph-Based Analysis (GBA) is used. Usage Guidelines: This parameter is only applicable to the state-propagation-based vectorless flow (<code>set_power_analysis_mode -method dynamic_vectorless -enable_state_propagation true</code>).
<code>-enable_power_target_flow {true false}</code>	
	Default: <code>false</code> Specifies to enable the power target flow. When set to <code>true</code>, the tool allows you to specify power targets for full-chip power analysis. You can use the <code>set_power</code> command to specify the power targets for full-chip, physical block, logical block, and power/ground net. When these targets are specified, Voltus will automatically scale the toggle rates of clock gate ratio, input, macro, and sequential activities to meet the specified power targets. Usage Guidelines: This parameter is only applicable to the state-propagation-based vectorless flow (<code>set_power_analysis_mode -method dynamic_vectorless -enable_state_propagation true</code>) in the XP mode.
<code>-enable_rtl_vectorbased_dynamic_analysis {true false}</code>	
	Specifies to take an RTL or partial VCD/FSDB file as input and use that for dynamic vector-based flow. This parameter allows you to estimate current even for instances which are missing from the activity file. You must use this parameter in conjunction with the <code>-method dynamic_vectorbased</code> parameter. Default is <code>false</code>.
<code>-enable_scan_report {true false}</code>	
	Default: <code>false</code> Specifies to generate all the scan mode analysis reports. The following reports will be generated: • <code>voltus_power.scanff.duplicated.rpt</code> - flip-flops that are specified in multiple chains • <code>voltus_power.scanff.missing.rpt</code> - flip-flops that are missing in the scan control file but exist in the design • <code>voltus_power.scanff.nonassigned.rpt</code> - flip-flops that do not exist in the design and are dropped from the scan control file • <code>voltus_power.scanff.nonscan.rpt</code> - assigned flip-flops that have no scan-out pin • <code>voltus_power.scanff.nonswitched.rpt</code> - non-switching flip-flops in the scan control file • <code>voltus_power.scanff.rpt</code> - detailed scan chain report containing global statistics for all scan chains and statistics for each scan chain in the design • <code>voltus_power.scanff.stateconflict.rpt</code> - flip-flops that have conflicting state assignment
<code>-enable_single_cycle_scheduling {true false}</code>	

	Default: <code>false</code> Schedules macro instances in every dominant clock cycle in the dynamic vectorless flow. This parameter can be used achieve faster toggle coverage. After the scheduling of every instance and attaining the maximum toggle coverage, the scheduler cycles back to the same switch source element and then schedules it again with the opposite polarity. This parameter helps in achieving the same toggle coverage within half the simulation period compared to dual cycle scheduling. When the parameter is set to <code>false</code>, the macros are scheduled every two cycles wherein the adjacent cycle is used for reversing the transition polarity.
<code>-enable_slew_based_ccs_pin_cap {true false}</code>	
	Default: <code>false</code> Specifies to use the Composite Current Source (CCS) capacitance value based on the slew given in the input pin section of the library cell in the Liberty file. If it is set to <code>true</code>, the tool honors the CCS pin capacitance; if it is set to <code>false</code>, it takes <code>max_capacitance</code> from <code>.lib</code>. This parameter can be used when timing windows and/or a Tempus timing session is not available.
<code>-enable_stable_clock_gating {true false}</code>	
	Default: <code>false</code> Perform balanced scheduling of ICGs independent of the predecessor ICG state. Usage Guidelines: This parameter is only applicable to the state-propagation-based vectorless flow (<code>set_power_analysis_mode -method dynamic_vectorless -enable_state_propagation true</code>).
<code>-enable_stable_flop_scheduling {true false}</code>	
	Default: <code>false</code> Perform balanced scheduling of flops independent of the ECOs done on the design. Usage Guidelines: This parameter is only applicable to the state-propagation-based vectorless flow (<code>set_power_analysis_mode -method dynamic_vectorless -enable_state_propagation true</code>).
<code>-enable_state_propagation {true false}</code>	
	Specifies to enable the vectorless state-propagation-based dynamic analysis flow. Default is <code>false</code>. For more information, refer to the State-Propagation-Based Vectorless Methodology section in the "<i>Dynamic Power and IRDrop Analysis</i>" chapter of the <i>Voltus User Guide</i>.
<code>-enable_tempus_pi {true false}</code>	

	Default: <code>false</code> Specifies to enable the Tempus Power Integrity (Tempus PI) analysis flow. When this parameter is set to <code>true</code>, data path scheduling is timing-aware, that is, scheduling is based on timing slacks. For more information, refer to the "<i>Tempus Power Integrity Analysis</i>" chapter in <i>Voltus User Guide</i>. Usage Guidelines: This parameter is only applicable to the state-propagation-based vectorless flow (<code>set_power_analysis_mode -method dynamic_vectorless -enable_state_propagation true</code>).
<code>-enable_xp {true false}</code>	
	Default: <code>false</code> Specifies to run standalone report generation in the XP mode.
<code>-enhanced_blackbox_avg {true false}</code>	
	Default: <code>false</code> Specifies to use the average toggle rate of related inputs for enhanced blackbox propagation.
<code>-enhanced_blackbox_max {true false}</code>	
	Default: <code>false</code> Specifies to use the maximum toggle rate of related inputs for enhanced blackbox propagation.
<code>-equivalent_annotation {true false}</code>	
	Default: <code>false</code> Specifies to enable equivalent annotation for Q or Qbar pins when only one of them is annotated.
<code>-event_based_leakage_power {true false}</code>	
	Default: <code>false</code> Specifies to calculate state-dependent leakage power. When this parameter is set to <code>true</code>, it calculates leakage power and current based on the state of an instance at each time point. For example, at time 0ns, instance instA is in state !ZN (0 on input, 1 on output), and at time 71ns, the instance is in state I!ZN (1 on input, 0 on output). The leakage power is calculated based on the different states of instA at different time points.
<code>-external_load_config_file filename</code>	

	Default: None Specifies to honor the user-specified load capacitance values for power calculation. This parameter allows you to specify an external load capacitance file when you have a timing window file without the <code>EXTERNAL_LOAD</code> constructs. File Format: The format of the user-specified input file is: <pre>INST CELL blockName external_load_file</pre> Here, the format of <code>external_load_file</code> is: <pre>NET PORT name load_value</pre> Examples: <u>Configuration file:</u> <pre>INST TOP top_wcard.load</pre> <u>External load file (<code>top_wcard.load</code>):</u> <pre>UNIT pF NET result 0.02 NET overflow 0.02</pre>
<code>-extraction_tech_file filename</code>	
	Specifies the extraction technology file to be used for top-level power-grid extraction.
<code>-extractor_include filename</code>	
	Specifies the extractor (ZX) commands and variables in the form of an include file. This parameter is applicable only for the standalone report generation flow in the XP mode (<code>set_power_analysis_mode - enable_xp true</code>).
<code>-fanout_limit value</code>	
	Specifies the fanout limit for nets. If an instance has nets more than or equal to the fanout limit, its switching/internal/leakage power is taken to be zero.
<code>-flatten_xpgv_block_instances filename</code>	

	Specifies a file that includes a list of block instances that use DEF instead of xPGV for power analysis, that is, specify the instances that are not to be used as xPGV. This parameter allows you to specify a few instances of the same block that will use DEF, and the others that will use xPGV at the top-level. If you do not specify this parameter, and if both DEF and xPGV are given for a block, xPGV takes precedence over DEF. File Format: <pre><block_name> <block_instance_name All None></pre> In this file, you can specify if a block is using a DEF by specifying the block instance name, or with the <code>All</code> or <code>None</code> keywords. <code>All</code> indicates that all instances are using DEF, and <code>None</code> indicates that none of the instances are using DEF. Example: For a design with three blocks, if you want to use the DEF and xPGV as following: <pre>Block_1 - > A1 (xPGV) & A2 (DEF) Block_2 - > B1 & B2 (xPGV for both instances) Block_3 - > C1 & C2 (DEF for both instances)</pre> The specified file will contain the following information: <pre>Block_1 A2 Block_2 None Block_3 All</pre>
<code>-force_library_merging {true false}</code>	
	Default: <code>false</code> Specifies to force merge PGVs even if there are some minor conflicts in the merging libraries. The default value of this parameter is <code>false</code>. This parameter is applicable only for the standalone report generation flow in the XP mode (<code>set_power_analysis_mode -enable_xp true</code>).
<code>-from_x_transition_factor value</code>	
	Specifies how transitions from X to 0/1 are counted. The default value is <code>0.5</code> .
<code>-from_z_transition_factor value</code>	
	Specifies how transitions from Z to 0/1 are counted. The default value is <code>0.25</code> .
<code>-generate_activity_mapping_report {true false}</code>	

	Default: <code>false</code> Specifies to generate the <code>activity_mapping.rpt</code> report in the vector-based dynamic power flow. This report gives the list of signal name mappings between the Golden (RTL netlist - input name in the vector file) and Revised (Gate netlist - annotated design object name) netlists that were used in the flow. The report displays the list of annotated and unannotated objects and helps to debug the mapping file for unannotated design objects. <code>activity_mapping.rpt</code> supports activity mapping done through the following methods: • User-specified mapping file (<code>map_activity_file -two_column_mapping_file</code>) • Conformal-generated RTL mapping file (<code>map_activity_file -rtl2gate</code>) • Voltus auto-generated mapping file (<code>set_power_analysis_mode -enable_auto_mapping</code>). Usage Guidelines: This parameter is only applicable to the vector-based dynamic flow (<code>set_power_analysis_mode -method dynamic_vectorbased</code>).
<code>-generate_current_for_rail railnames</code>	
	Generates current files for the specified rail. By default, the command generates current files for all power rails.
<code>-generate_flop_ranking_data directoryname</code>	
	Specifies the name of the directory where the flop ranking data is saved. Using this parameter, you can save the flop ranking data that can be reused across different versions of the design database for the same block or transferred from the top-level to the block-level run. This parameter is used with the <code>-reuse_flop_ranking_data</code> parameter.
<code>-generate_leakage_power_map_based_on_calculated_leakage {true false}</code>	
	Default: <code>false</code> Specifies to scale the instance leakage power by directly multiplying the user-specified scaling factor with the final calculated leakage power value, instead of scaling the leakage power value from the input dotlib. If this parameter is not specified, the user-specified scaling factor will be applied on the leakage power values from the input dotlib.
<code>-generate_static_report_from_state_propagation {true false}</code>	
	Default: <code>false</code> Specifies to directly generate the static power analysis result based on the RTL-VCD input and state-propagation. When this parameter is specified, the RTL vector power analysis flow does not require the generation of a TCF/FSDB file. Usage Guidelines: • This parameter is applicable only to the dynamic propagation-based flow (vector-based/vectorless/RTL/Mixed-mode). • This parameter is not applicable to the <code>-static_netlist verilog</code> flow.
<code>-handle_glitch {true false}</code>	

	Default: <code>false</code> A transition is defined as an inertial glitch when the time difference between two sequential toggles are less than half of the rise transition time + fall transition time in the vector-based static power calculation. Both static and dynamic current are derated for glitch transitions. The same derate factor is used for both power and current. If set to <code>true</code>, glitch power will be identified and calculated according to this definition and it will be reported in a separate column. Switching or internal power calculation does not include power due to glitches. If set to <code>false</code>, glitch power will not be identified, and the respective power will be calculated as normal signal switches. Hence, the total power will be a little higher.
<code>-handle_transport_glitch {true false}</code>	
	Default: <code>false</code> Transport glitch is referred to as functional toggles for a net in a single clock period before it reaches a stable value. You can use this parameter to report the internal and switching power values for transport glitch. The transport glitch power is reported in the following reports: • Average Power Summary Report (<code>power.rpt.avgpwr</code>) - reported in the Total, Group, Clock, and Rail power sections. • Instance Power Report (<code>power.rpt.instpwr</code>) - reported for all instances. • Hierarchy Power Report (<code>power.rpt.hierpwr</code>) - reported for all hierarchies. This parameter is automatically set to <code>true</code> with <code>-handle_glitch true</code>. That is, reports both inertial and transport glitch power. When set to <code>false</code>, only the inertial glitch power is reported.
<code>-handle_tri_state {false true}</code>	
	When set to <code>true</code>, all tristate device enable pin values will be taken into account when determining the propagation of the activity through tristate gates. Default: <code>false</code>
<code>-hier_delimiter character</code>	
	Specifies the hierarchical delimiter in the DEF file. The default hierarchical delimiter is a forward slash (/) but can be changed by setting the <code>-hier_delimiter</code> parameter. You can use this parameter to override the existing hierarchical delimiter.
<code>-honor_combinational_logic_on_clock_net {true false}</code>	
	Specifies the behavior of the combination logic on the clock net. • <code>true</code> (default) - specifies to honor activity propagation through combinational logic. • <code>false</code> - specifies that the combination logic will be treated like a combinational clockgate.
<code>-honor_default_switching_activity_in_scheduling {true false}</code>	

	Default: <code>false</code> Controls whether default switching activity settings are applied during hierarchy/domain-based scheduling. This parameter must be used for switching activity assignment using the <code>set_default_switching_activity</code> <code>-pg_net</code> or <code>-hier</code> parameters in the dynamic vectorless flow.
<code>-honor_negative_energy {true false}</code>	
	A value of <code>true</code> specifies that Voltus will keep negative internal energy numbers from the <code>.lib</code> internal power table. When set to <code>false</code>, negative power is treated as 0. Default: <code>true</code>
<code>-honor_net_activity {true false}</code>	
	Specifies to use activity from the <code>Net</code> section of the Switching Activity Interchange Format (SAIF) file for power calculation and optimization, and to ignore the pin-based state-dependent path delay (SDPD) activity. The default value of this parameter is <code>true</code>. When this parameter is set to <code>false</code>, the power calculation engine can use both the net-based and pin-based activity. In this case, precedence will be given to activity from net or pin that appears later in the SAIF file.
<code>-honor_non_mission_leakage {true false}</code>	
	Supports modeling of well-to sub-leakage information when the circuit power supply is off for the level shifter cells. This is referred to as the non-mission mode leakage power calculation. The leakage power for a level shifter cell depends on the state of each PG pin. The Liberty file contains sections for PG pin conditions of each mode/state and non-mission mode leakage power for each PG pin of that mode. When this parameter is set to <code>true</code>, Voltus identifies the <code>pg_pin</code> mode and assigns non-mission mode leakage power associated with the mode. The use model of the non-mission mode leakage power calculation feature is: <pre>set_power_analysis_mode -honor_non_mission_leakage true set_power_analysis_mode -off_pg_nets {VDD VDDO}</pre> Example: Let us consider a level shifter cell with the PG mode “<code>retained_VDDO_off</code>” and the following <pre>pg_pin_condition: pg_setting_value (retained_VDDO_off) { pg_pin_condition : "!VBB*VDD*VDDI "; pg_pin_active_state (VBB, V1); pg_pin_active_state (VDD, V2); pg_pin_active_state (VDDI, V2);</pre> When you specify a list of PG nets to be turned off along with the <code>-honor_non_mission_leakage</code> parameter, all cells/instances that have the PG state “<code>retained_VDDO_off</code>” associated with the specified PG pin condition, will use the corresponding non-mission leakage power.
<code>-hybrid_analysis {true false}</code>	
	Default: <code>false</code> Specifies to perform both vector-based and vectorless power analysis in a single session.
<code>-ignore_control_signals {true false}</code>	

	Specifies that control signals will be ignored when propagating activity. It is recommended to set this parameter to <code>true</code>. When set to <code>true</code>, the control pins (reset, preset) will not impact the sequential instance's output pin activity during activity propagation. <i>Default:</i> <code>true</code>
<code>-ignore_data_phase_for_clk {true false}</code>	
	Specifies to ignore data frequency in the clock path for the clock instances in the design. When this parameter is set to <code>true</code>, the software propagates only the fastest clock pin frequency instead of the data pin frequency for activity propagation through clock network. In the default mode (<code>false</code>), the software picks the fastest frequency between the data phase and clock phase on that pin. <i>Default:</i> <code>false</code>
<code>-ignore_glitches_at_same_time_stamp {true false}</code>	
	Specifies to ignore glitches at the same transient time in the vector-based static and dynamic power analysis flow. When <code>-ignore_glitch_at_same_time_stamp</code> is set to <code>true</code>, only the last signal value change is honored, and all the preceding value changes at the same time stamp are ignored. The last signal value will overwrite the previous value for the same signal at the same time stamp, so the last value will represent the state of the signal at that time stamp. <i>Default:</i> <code>true</code>
<code>-ignore_inout_pin_cap {true false}</code>	
	When set to <code>true</code>, ignores the bidirectional pin capacitances (<code>direction : inout</code>) defined for I/O cells in the .lib file, when calculating switching power and internal power; this also includes the default capacitance value for <code>inout</code> pins. <i>Default:</i> <code>false</code>
<code>-ignore_macro_leakage_scale_for_temp {true false}</code>	
	Default: <code>false</code> The <code>-leakage_scale_factor_for_temp</code> parameter is used to perform linear scaling of the leakage power for temperature in both the standard cell and macro libraries. The <code>-ignore_macro_leakage_scale_for_temp</code> parameter allows you to selectively apply the scale factor only for standard cells, and not use the scale factor for macro cells.
<code>-ignore_end_toggles_in_profile {true false}</code>	
	By default, the software profiles the toggles in the user-selected vector time window by steps including the toggles at the start/end boundary of the window. In each step, the toggles on the start boundary edge of the step is counted in the current step; the toggles on the end boundary edge is counted in the next step. When this parameter is set to <code>false</code>, the toggles on the end boundary edge is also counted in the last step, as there is no next step to it. This may lead to double count of the power profile for the last step, especially for zero-delay vector profiling. The default of this parameter is <code>false</code>. When this parameter is set to <code>true</code>, the software ignores the toggle on the end boundary of the last step.
<code>-ignore_unconstraint_net_with_timing_data {true false}</code>	

	Default: <code>false</code> Specifies to ignore unconstrained nets/pins from toggle generation. These nets/pins do not have an associated clock but contain valid timing data with arrival windows in the TWF file. If set to <code>false</code>, the tool applies default frequency toggles to these unconstrained nets, which could cause false IR violation warnings due to high voltage drop.
<code>-include_seq_clockpin_power {true false}</code>	
	Default: <code>false</code> Reports the clock pin power of flip-flops as part of the clock network power.
<code>-include_timing_in_current_file {true false}</code>	
	Default: <code>false</code> Specifies to write timing bits for non-switching instances in the current files during the state-propagation-based flow. When specified, rail analysis will report effective instance voltage (EIV) for both switching and non-switching instances. If the parameter is set to <code>false</code>, rail analysis will report EIV for the switching instances only. Usage Guidelines: This parameter is only applicable to the state-propagation-based vectorless flow (<code>set_power_analysis_mode -method dynamic_vectorless -enable_state_propagation true</code>).
<code>-ir_derated_timing_view view_name</code>	
	Specifies the name of the timing view used to calculate timing slack and to rank the flops in the Tempus PI flow. This parameter must be specified if there are multiple views. The <code>-ir_derated_timing_view</code> parameter defines the timing view used to detect the IR-sensitive paths using a lower voltage than the actual analysis. This lower voltage simulates the IR-drop for each instance in the design. The software uses this view to prioritize the end-points that need to be prioritized for switching.
<code>-keep_clock_gate_ratio_in_iterations {true false}</code>	
	Default: <code>false</code> Specifies whether to keep the clock gate output (CGO) fixed or automatically adjust the value for each iteration of state propagation. Usage Guidelines: This parameter is only applicable to the power target flow (<code>set_power_analysis_mode -enable_power_target_flow true</code>).
<code>-leakage_scale_factor_for_temp scale</code>	
	Performs a linear scaling of the leakage power for temperature in all libraries. This replaces the k-factor for temperature in the .lib file. For process and voltage, the scaling will continue to be based on the corresponding k-factors in the .lib file. A value of 0.8 scales the leakage power in the library to 80 percent. A value of 1.2 scales the leakage power in the library to 120 percent. Default: <code>1</code>
<code>-library_preference {voltage ecdsm_ccsp}</code>	

	Instances of the same cell can be attached to different power domains in a design. If you have specified different PVT libraries, then the software will by default bind each instance to the closest library definition. You can use this parameter to select either the closest Effective Current Source Model (ECSM)/Composite Current Source-power (CCSP), or the closest voltage library for the instance: • voltage: when specified, enables library binding to the closest voltage library. • ecsm_ccsp: when specified, enables library binding to the closest ECSM/CCSP library. The default is voltage.
<code>-macro_pgv_leakage {true false}</code>	
	Default: true Controls leakage power reporting for cells that have power information available in both dotlib and PGV. When the parameter is set to false, the leakage current from Liberty is used and when the parameter is set to true, the leakage current from PGV is used.
<code>-macro_toggle_pin_percentage percentage</code>	
	Default: 50% Specifies the percentage of output data pins (ignoring the control pins) to be scheduled for a macro cell. This parameter is applicable to the state-propagation based vectorless flow. The default macro pin toggle percentage is 50%.
<code>-mbff_toggle_behavior { simultaneous independent sbff }</code>	
	Specifies the toggle behavior of multi-bit flip-flops (MBFF). The possible arguments are: • simultaneous: The multiple bits of the MBFF will toggle or not toggle at the same time. The MBFF will be considered as one flip-flop. • independent: The multiple bits of the MBFF will toggle independently. The MBFF will be considered as one flip-flop. • sbff: The MBFF will be considered as multiple Single-Bit Flip-Flops (SBFFs). Each bit will be considered as one flip-flop, and will toggle independently. The default value is independent.
<code>-mbff_toggle_pin_percentage percentage</code>	
	Specifies the percentage of Multi Bit Flip Flop (MBFF) pins to be scheduled. This parameter is applicable to the state-propagation based vectorless flow. This parameter will only work when -mbff_toggle_behavior is set to independent.
<code>-memory_current_scaling_method {1 2 3}</code>	
	Specifies to enable scaling of the NLPM-based currents by a user-specified scale factor for macro cells that are switching. The applicable arguments are 1, 2, and 3, which means enable a scaling factor of 0.5, 0.25, and 0.1 respectively.
<code>-merge_switched_net_currents { true false}</code>	

Specifies to merge switched net currents into always-on currents. For designs with a large number of switched nets, this parameter helps improve performance by merging switched net instance currents with always-on instance currents. However, when this parameter is set to `true`, you will not be able to use the generated current files to run power-up analysis.

Default: `false`

```
-method { static | dynamic_vectorless | dynamic_vectorbased | dynamic_mixed_mode | event_based |  
vector_profile }
```

Specifies the type of analysis to be performed. You can perform two types of power analysis, namely static and dynamic.

- `static` - specifies to perform static power analysis to calculate average power.

Dynamic power analysis can be a vector-based or vectorless approach.

- `dynamic_vectorbased` - uses the full VCD output of a logic simulator to identify the instances that are switching and when they switch.
- `dynamic_vectorless` - uses the timing arrival window information from a static timing analysis tool to determine when instances switch. A timing arrival window describes when a signal can change within a clock cycle. An additional algorithm then determines the instances that switch.
- `dynamic_mixed_mode` - specifies to combine the vector-based and vectorless (mixed) dynamic analysis. This flow can be used in the following cases:

- **case 1:** vectors are not available for every block of a full-chip design.

In this case, the software uses the vectorless methodology for blocks with incomplete vectors

- **case 2:** blocks with vectors having unannotated combinational/ sequential logic.

In this case, the software performs state propagation for the missing combinational/ sequential logic. It uses the vectorless methodology for a logic cone that does not have the primary inputs or flops annotated from the vector, and then performs state propagation

To enable the dynamic mixed mode analysis flow, you must specify the following commands:

```
set_default_switching_activity -clock_gates_output 2.0 -global_activity <activity>
set_power_analysis_mode -method dynamic_mixed_mode
read_activity_file -scope <> -block <> -> (for available blocks)
```

- `event_based` - specifies to enable event-based power analysis. This method supports multiple power analysis flows simultaneously. When specified, the tool performs accurate state-based power estimation for all events of a scenario in a design. Event-Based Power Analysis is applicable to all event-based vectors, such as VCD, FSDB, SHM, and PHY.

You can specify this method to generate and analyze multiple types of power analysis flow reports in a single run. The following table shows the additional options required for the different flows:

Power Analysis Flows	Additional Options for <code>-method event_based</code>
Static	no additional option required
Average Power & Current	<code>-write_static_currents</code>
Dynamic Vectorbased	<code>-write_dynamic_currents</code>
Mixed Mode	<code>-add_simulation</code>
Dynamic Vectorless	
Profiling	<code>-worst_step_size</code>

- `vector_profile` - specifies to enable vector profiling. This is used to identify windows with maximum activity and power which can then be used to drive dynamic vector-based power analysis.

Default: `static`

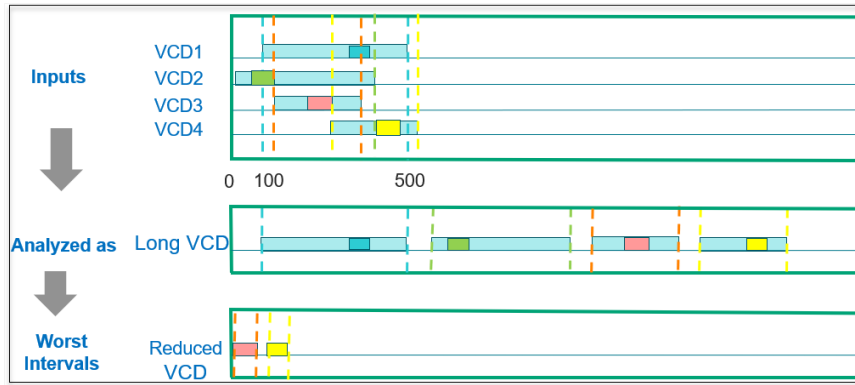
`-min_leaf_count value`

Specifies the minimum number of leaf instances required for writing the hierarchy data in the profiling database. It allows you to restrict the number of hierarchies for which waveforms are reported in the profiling database when using both the `-write_profiling_db` and `report_power -hierarchy` parameters.

The default value of this parameter is 0. Any hierarchy whose leaf instance count in its complete sub-tree is less than the `-min_leaf_count` value will be skipped from getting reported in the profiling database. This parameter can be used to skip very small hierarchies and as a result save on the database writing time and disk space.

`-multi_scenario_simulation {true | false}`

Specifies to create a long vector by combining multiple vectors and to identify the worst power intervals across the combined vector. The long vector includes all scenarios that are used sequentially and are non-overlapping.



This parameter allows you to perform power and current analysis using worst peak power interval across multiple simulation vectors.

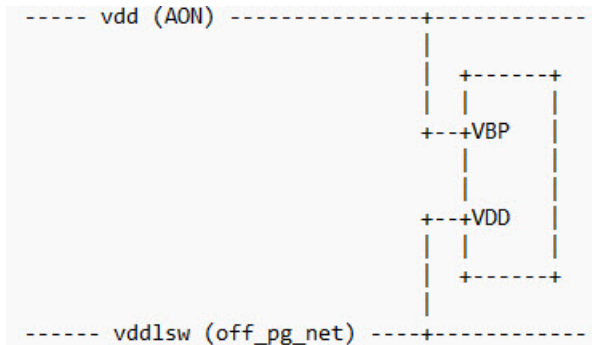
Default: false

`-off_pg_bulk_leakage {true | false}`

Default: false

Specifies to account for the leakage power through an instance's bulk pin when it is connected to a power-ground (PG) rail in the ON state.

The following diagram illustrated that the instance contributes bulk pin leakage to the ON state rail (**vdd**) through the **VBP** pin and zero leakage to the OFF state rail (**vddlsw**) through the **VDD** pin.



<code>-off_pg_nets net_list</code>	The name of the power rails that will be turned off. The net name can contain instance hierarchy if appropriate. This data will be taken into account when figuring out the list of power gates which are switched off. Note: A net is turned off if all connected power gates are turned off by control signals. When a net is off, all instances connected to that net will contribute zero power.
<code>-output_current_data_prefix prefix</code>	Specifies a prefix to the static or dynamic current files generated by the software. When power analysis is done in the multi-CPU mode, the output current data directory name and the underlying current files will also use the same prefix by default.
<code>-partition_count value</code>	Default: 0 Specifies the number of physical partitions or jobs for a distributed run in the area-based partitioning flow. Usage Guidelines: • This parameter is only applicable to the event-based power analysis flow (<code>set_power_analysis_mode -method event_based</code>) in the XP mode. • This parameter must be specified with the <code>-use_physical_partition</code> parameter.
<code>-partition_spef {true false}</code>	Default: false Specifies to generate block-level SPEF files from a flat SPEF file. This parameter allows you to process input SPEF data and automatically generate partition-level SPEF chunks. The parameter is supported only in the XP mode for full-chip designs using flat SPEF files. The use model of this parameter is: <pre>specifiy_spef { flat.spef.gz } set_power_analysis_mode -partition_spef true</pre>
<code>-partition_twf {true false}</code>	Default: false Specifies to read the input TWF data (flat or hierarchical TWF files), and generate block-level TWFs. The default is false. This parameter must be specified when the block/partition level TWF files are not available.
<code>-pin_based_twf {true false}</code>	

	Specifies whether to generate pin-based TWF or net-based TWF. The default value is <code>false</code>. - When set to <code>false</code>, a net-based TWF file is generated. Net-based TWF file is smaller in size and therefore results in faster processing. - When set to <code>true</code>, a pin-based TWF file is generated. Pin-based TWF file is bigger in size and requires longer processing time, however, is more accurate because the slews numbers are pin-based. Note: This parameter must be used with the <code>read_sdc</code> command.
<code>-power_grid_library { library_list }</code>	
	Specifies the name of the Cadence power cell libraries (<code>.c1</code>). You can also specify a list of defined power-grid libraries having decap cells tagged in them. Note: When specifying a list of PGVs to power analysis using the <code>set_power_analysis_mode</code> command, you must specify the technology library as the first PGV library in the list, followed by the standard and macro PGV libraries. The selection between the standard and macro PGV library is order based. Therefore, if multiple power-grid libraries are specified for a cell, the power engine maps the cell with the first library after the technology library.
<code>-power_include_initial_x_transitions {true false}</code>	
	Controls how the initial X state toggle should be counted. The initial X state means the state at time 0. The default value is <code>true</code>. This means that the software counts the power caused by the <code>x->0</code> or <code>x->1</code> toggle. When set to <code>false</code>, the software does not count the power caused by the initial X state toggle.
<code>-power_match_state_for_logic_x value</code>	
	Controls how the logic X will be evaluated in the boolean function of the 'when' state of a power table. The default value is 0. The parameter uses the following arguments: 0 - regards X as 0 1 - regards X as 1 - x/X - regards X as neither 0 nor 1, that is, whenever there is any X logic in it, the boolean function will be evaluated false.
<code>-precision value</code>	
	Specifies precision of decimal range [1-8] to ensure consistent decimal places are displayed for each power component. The default value is 8. This parameter is used for precision handling in <code>report_instance_power</code> and <code>report_power</code>.
<code>-pre_simulation_empty_period period</code>	
	Specifies to add an empty pre-simulation period before simulation during IR drop analysis. Power analysis is not done for the pre-simulation period and flat dynamic currents are created.
<code>-pre_simulation_period period</code>	
	Specifies to set a pre-simulation period for IR drop analysis. Pre-simulation is used for generating power analysis reports and dynamic currents.

<code>-pre_simulation_power_exclude_period <i>period</i></code>	
	Specifies to exclude power analysis during the pre-simulation period used for IR drop analysis. Pre-simulation is used only to create dynamic currents and not used for power reports.
<code>-quit_on_activity_coverage_threshold <i>threshold_value</i></code>	
	Specifies the activity (VCD/FSDB/TCF) coverage threshold value. This parameter allows you to control the acceptable activity coverage that you are expecting and enforce the software to quit when it does not meet the criteria. When you specify the threshold value, the software exits if the real activity coverage is below the threshold.
<code>-read_rcdb {true false}</code>	
	Specifies to write a SPEF file containing only the total C for signal nets and pass it to the Dynamic Power engine. Dynamic power analysis requires only the total C for signal nets. Therefore, instead of writing a large SPEF file (with distributed RC) from the RCDB, this parameter specifies to create and pass a SPEF file with only total C for all the signal nets. This significantly reduces the parasitic annotation time in the Dynamic Power engine. <i>Default</i> : false
<code>-relax_arc_match {true false}</code>	
	Specifies to relax arc match when the exact arc is not found in a .lib. When set to <code>true</code> , the internal energy is calculated as an average of multiple arcs with relaxed arc matching. If this parameter is set to <code>false</code> , relaxed arc search is not done and 0 internal energy is returned. <i>Default</i> : true
<code>-report_black_boxes {true false}</code>	
	Specify this parameter to report cells that are used as black boxes. A black box is a cell which is either missing from the liberty file or has no functionality defined in the liberty file. <i>Default</i> : false This is an optional parameter.
<code>-report_clock_instance_switching_info {true false}</code>	
	Specifies to generate a report for clock instances that switched during the state-propagation based vectorless flow. The generated report contains the toggle time, instance name, input pin event, and output pin event for each switching event of the clock instance.
<code>-report_idle_instances {true false}</code>	

Specifies to generate a report (`idleinstance.rpt`) that lists all the idle or non-switching instances.

The default value is `false`.

The following is a snippet of the `idleinstance.rpt` report:

```
DFF22/DFF      F
DFF42/DFF      F
...
```

Here, the first column is the instance name and the second column is the type of instance (`F` indicates flop and `C` indicates combinational).

```
-report_instance_switching_info {all | output_logic | none}
```

Specifies to generate a report for combinational instance switched during state-propagation. In the output file, it contains the toggle time, instance name, input pin event, and output pin event for each switching event of the combination gate. The possible arguments of this parameter are:

- `all` - Specifies to report instance toggle information for both input and output pins. When this parameter is set to `all`, the switching of both the input and output pins of all the instances are reported in the `voltus_power.stateprop.switchinst` file and the switching of flops are reported in the `voltus_power.stateprop.switchsrc` file.
- `output_logic` - Specifies to report instance toggle information for the output pins only. When this parameter is set to `output_logic`, the switching of the output pins of all the instances are reported in the `voltus_power.stateprop.switchinst` file and the switching of flops are reported in the `voltus_power.stateprop.switchsrc` file.
- `none` - Disables generation of instance toggle report. This is the default option. When this parameter is set to `none`, only switching of flops are reported in the `voltus_power.stateprop.switchsrc` file. The file does not report switching of instances.

The default value is `none`. This parameter is used for generating switching details of state-propagation and should be specified only for debugging purpose.

The following is a snippet of the `voltus_power.stateprop.switchinst` report:

```
*-----
*Switch Time   Instance Name   Pin Name   Logic
*-----
*
  0.097         B11/BUFF      I          rise
  0.097         DFF11/DFF     Q          rise
...
```

Here,

- `Switch Time`: switching time of instances that can be either combinational or sequential
- `Instance Name`: instance name
- `Pin Name`: input/output pin of the instance
- `Logic`: present logic of the pin that can be either "rise" or "fall"

```
-report_instance_switching_list filename
```

Specifies to generate a report (`voltus_power.stateprop.switchlist`) containing all the switching times for the specified instances when the `-enable_state_propagation` parameter is set to `true`.

The format of the specified file is:

```
instname1
instname2
...
```

The following is the format of the `voltus_power.stateprop.switchlist` report:

```
*-----
*Instance Name   Pin Name   Pin Direction   Frequency   Switch Time
*-----
*
```

Here,

- **Instance Name:** instance name that can be either combinational or sequential
- **Pin Name:** name of the input/output pin of the instance
- **Pin Direction:** indicates whether the pin is an INPUT (I) or OUTPUT (O) pin
- **Frequency:** clock frequency
- **Switch Time:** gives the switching pattern for the instance, that is, `<time1> <logic> <time2> <logic>`. The logic of the pin can be "r" or "f". r indicates rise and f indicates fall.

The following is a snippet of the `voltus_power.stateprop.switchlist` report:

```
*-----
*Instance Name   Pin Name   Pin Direction   Frequency   Switch Time
*-----
*
   DFF11/DFE      QN         O             100         0.106 f 10.107 r
   DFF11/DFE      Q          O             100         0.097 r 10.099 f
   DFF11/DFE      CP         I             100         0.059 r 5.056 f 10.059 r 15.056 ...
...
```

```
-report_library_usage {true | false}
```

	Default: false Specifies to report library usage statistics in a text report. The report contains information such as, cell name, library format used (CCSP/ECSM/NLPM) for each cell, library name, and so on. This parameter allows you to perform library integrity checking for debugging purpose. The following is the use model of the <code>-report_library_usage</code> parameter: <pre>set_power_analysis_mode -method static dynamic_vectorless dynamic_vectorbased \ ... -report_library_usage true report_power -output <power_dir> -report_prefix <prefix></pre> The report is saved at <code><power_dir>/<prefix>.lib.rpt</code>. File Format: <table><tr><td>Instance Name</td><td>Cell Name</td><td>Net Name</td><td>Pin Name</td><td>Operational Voltage</td><td>Library Voltage</td><td>Library Format</td><td>Missing Tables</td><td>Library Name</td></tr></table> Examples: <table><tr><td>Instance Name</td><td>Cell Name</td><td>Net Name</td><td>Pin Name</td><td>Operational Voltage</td><td>Library Voltage</td><td>Library Format</td><td>Missing Tables</td><td>Library Name</td></tr><tr><td>INV_178</td><td>INV</td><td>VDD, VDDA</td><td>VDDG, VDD</td><td>0.9</td><td>1.0</td><td>NLDM</td><td>leakage, timing</td><td>A</td></tr><tr><td>INV_179</td><td>INV</td><td>VDDC</td><td>VDDG</td><td>0.75</td><td>0.75</td><td>CCSP, NLDM</td><td>leakage</td><td>B</td></tr></table> Usage Guidelines: This parameter must be specified with the <code>report_power -report_prefix</code> parameter.	Instance Name	Cell Name	Net Name	Pin Name	Operational Voltage	Library Voltage	Library Format	Missing Tables	Library Name	Instance Name	Cell Name	Net Name	Pin Name	Operational Voltage	Library Voltage	Library Format	Missing Tables	Library Name	INV_178	INV	VDD, VDDA	VDDG, VDD	0.9	1.0	NLDM	leakage, timing	A	INV_179	INV	VDDC	VDDG	0.75	0.75	CCSP, NLDM	leakage	B
Instance Name	Cell Name	Net Name	Pin Name	Operational Voltage	Library Voltage	Library Format	Missing Tables	Library Name																													
Instance Name	Cell Name	Net Name	Pin Name	Operational Voltage	Library Voltage	Library Format	Missing Tables	Library Name																													
INV_178	INV	VDD, VDDA	VDDG, VDD	0.9	1.0	NLDM	leakage, timing	A																													
INV_179	INV	VDDC	VDDG	0.75	0.75	CCSP, NLDM	leakage	B																													
<code>-report_missing_bulk_connectivity {true false}</code>																																					
	Specifies to report missing bulk pins in the missing input report (<code>-report_missing_input</code>). Usage Guidelines: This parameter is not applicable to the <code>-method static -static_netlist verilog flow</code>.																																				
<code>-report_missing_input {true false}</code>																																					
	Specifies to report missing netlist information in input files, such as library, LEF/DEF, PGV, SPEF, TWF, and missing logical connectivity for cell instance (PGNET). The name of the report file is: <code><prefix>_missingdata.<spef lefdef lib pgv twf pgnet></code> Following is the use model: <pre>set_power_analysis_mode -report_missing_input true report_power -output <dirname> -report_prefix <prefix></pre> Usage Guidelines: This parameter is not applicable to the <code>-method static -static_netlist verilog flow</code>.																																				
<code>-report_missing_nets { true false }</code>																																					
	Controls reporting of missing nets in both static and dynamic power analysis. This parameter controls reporting for TWF and activity files in static analysis, and controls reporting for TWF/SDC/SPEF and activity files in dynamic analysis. Default: false																																				

<pre>-report_scan_chain_stats {true false}</pre>	Specifies to generate a report file called <code>voltus_power.scanchain.stat</code> during scan mode analysis that contains scan chain statistics. The default value is <code>false</code>. The report has 2 parts: • Global Statistics: this section has the cumulative statistics of all scan chains in the design • Chain Statistics: this section has the statistics for each scan chain in the design The Global and Chain Statistics sections have the following information: • Total: total instances specified in the control file or in a scan chain • AssignedFF: flip-flops defined in the scan control file which exist in the design • SwitchedFF: flip-flops that have switching event at the output • Non-ScanFF: flip-flops that have no scan-out pin • DuplicatedFF: flip-flops that are specified in multiple chains • StateConflictFF: flip-flops that have conflicting state assignment
<pre>-report_stat {true false}</pre>	Reports a set of statistics on the instance power, instance power density, clock power, or transition density. This parameter is useful in viewing and debugging user defined activity factors. <i>Default:</i> <code>false</code>
<pre>-report_time_display_fraction_digits value</pre>	Sets the number of fractional digits or decimal places displayed for the instance switching time values in the <code>voltus_power.stateprop.switchinst</code> file. This parameter displays fractional digits that show the exact time difference if there are two events (rise/fall) of an instance switching at the same time. <i>Default:</i> <code>-1</code>
<pre>-report_twf_attributes {detailed summary}</pre>	

Controls the generation of the TWF attribute report based on the TWF attributes that are specified using the `set_twf_attribute` command.

The possible arguments are:

- `summary` - generates a summarized static timing analysis report “`voltus_power.twf_attribute.summary`” that displays the number of user-defined TWF attributes (delay, slew, and slack values for both the rise and fall transitions) and the number of annotated TWF attributes for dynamic power analysis. The following is a snippet of the report:

```
STATISTICS:
shift_arrival_time    User Defined    Annotated    Annotated/Total(%)
rise_slew             5              4             80
fall_slew             2              2             100
rise_delay            2              2             100
fall_delay            5              4             80
rise_slack             5              4             80
fall_slack            2              2             100
fall_slack            2              2             100
```

- `detailed` - generates a detailed static timing analysis report “`voltus_power.twf_attribute.detailed`” that includes the summary report, and a detailed report of the rise and fall transitions for the TWF attributes of each annotated instance/pin. It also includes the shift arrival time report.

This argument also allows you to generate a report `voltus_power.twf.rpt` containing the TWF statistics to highlight clock-frequency and delay used for the static and dynamic power calculation.

The TWF statistics report is generated only if:

- the `read_twf` command is specified before `set_power_analysis_mode`
- the `set_power_analysis_mode -method static -static_netlist def` OR `set_power_analysis_mode -method dynamic_vectorless | dynamic_vectorbased` command is specified.

The format of the `voltus_power.twf.rpt` report is:

```
# Voltus TWF Statistics
# Units: Freq (Hz), Delay (second), Slew (second)
# TWF File List:
#     TWF_file1
#     TWF_fileN
```

```
USER DEFINED TWF ATTRIBUTE STATISTICS
<Clock Freq> <Min Rise_Delay> <Rise Delay> <Max Rise Delay> <Min Fall Delay> <Fall Delay>
<Max Fall Delay> <Net>
```

The `USER DEFINED TWF ATTRIBUTE STATISTICS` section is included in the report only if the TWF attributes are defined using the `set_twf_attribute` command.

Example:

#check all instances of inst_C1 get shifted by 3ns

```
set_twf_attribute -pin inst_C1 -shift_arrival_time 3
```

#check all instances of inst_D1 get shifted by 3ns

```
set_twf_attribute -net inst_D1 -shift_arrival_time 3
```

```
set_power_analysis_mode -method dynamic_vectorless -create_binary_db false -disable_static
false -enable_state_propagation true -current_generation_method peak -power_grid_library
/./library_pv.cl -report_twf_attributes summary
```

<code>-reset</code>	Resets all or specified parameters to their default values. When you specify only the <code>-reset</code> parameter, the software resets all parameters for the <code>set_power_analysis_mode</code> command. When resetting specific parameters, the <code>-reset</code> parameter must be the first followed by the specific parameter. For example, you can specify the following command to reset only the <code>-method</code> parameter while keeping all other parameters' values unchanged: <pre>set_power_analysis_mode -reset -method</pre> Note: When resetting a specific parameter, you must not specify any arguments for that parameter.
<code>-reuse_flop_ranking_data</code> <i>directoryname</i>	Specifies the name of the directory where the flop ranking data to be reused is placed.
<code>-reuse_flop_ranking_data_hier</code> <i>blockname</i>	Allows you to choose the block/hierarchy whose ranking data is to be used for the block-level analysis. This parameter is used when the flop ranking data is transferred from a top-level analysis where blocks are flattened to another block-level analysis. The <code>-reuse_flop_ranking_data_hier</code> parameter enables you to specify the required block if the top-level contains multiple instantiations of the same DEF block.
<code>-save_bbox</code> {true false}	Default: false Specifies to save the boundary box of each instance, its placement, and rotation at the top level. When set to <code>true</code>, generates the power database with each instance boundary box, placement, and rotation information at the top level. Usage Guidelines: This parameter must be specified when generating the Voltus thermal model.
<code>-scale_to_sdc_clock_frequency</code> {true false}	

Specifies to enable the VCD clock frequency to be scaled up to the SDC clock frequency. This allows you to match the VCD clock to the SDC clock to run analysis. By default, the clock frequency is scaled based on the activity file (Default: `false`). When you set the `-scale_to_sdc_clock_frequency` parameter to `true`, the power report and current waveforms apply the after-scaled clock frequency.

The default behavior for clock frequency scaling in the RTL and Gate level VCD flows is different, as described below:

- **RTL VCD** - By default, the clock frequency is scaled to the SDC clock frequency (`-scale_to_sdc_clock_frequency true`) when the RTL VCD flow is enabled (`set_power_analysis_mode -enable_rtl_vectorbased_dynamic_analysis true`).
You can disable clock frequency scaling by setting the `set_power_analysis_mode -scale_to_sdc_clock_frequency` parameter to `false`.
- **Gate VCD** - By default, scaling of the clock frequency is disabled (`-scale_to_sdc_clock_frequency false`).
You can enable the VCD clock frequency to be scaled up to the SDC clock frequency by setting the `set_power_analysis_mode -scale_to_sdc_clock_frequency` parameter to `true`. This works only when `set_power_analysis_mode -use_zero_delay_vector_file` is set to `true`, which means that the gate VCD is a zero-delay gate VCD.

```
-scan_chain_activity {design_name chain_name activity_number} | {vectorless activity_number}
```

Specifies to either provide vectorless activity for all scan chains or provide a specific activity for a specific chain. This parameter allows you to specify vectorless or user-defined activity to scan chains.

The following are the two use models:

- Generate the scan control file for a specific chain:
`set_power_analysis_mode -scan_chain_activity {design_name chain_name activity_number}`
- Generate the scan control file for all scan chains:
`set_power_analysis_mode -scan_chain_activity {vectorless activity_number}`

The `-scan_chain_activity` parameter is supported only in the XP mode.

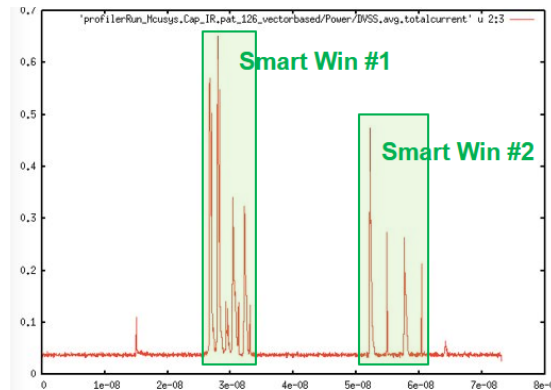
```
-scan_chain_name {design_name chain_name pattern ...}
```

	Specifies to automatically generate a scan-chain control file from a scan-chain based DEF file for the shift mode scan-chain analysis. This parameter allows you to create a control file for a large-sized DEF containing long chains or for multiple-DEF files. The generated scan-chain control file is saved in the power output directory. The use model of the parameter is: • <code>-scan_chain_name {all pattern}</code> - generate scan control file for all scan chains. Example: <code>-scan_chain_name {all 11010}</code> • <code>-scan_chain_name {design_name chain_name pattern ...}</code> - generate scan control file for a specific chain. Here, “<i>design_name</i>” is the design name of the block DEF, or the “top” keyword that means the top-level DEF. Example: <code>-scan_chain_name {top scan_chain_1 11010 BlockA scan_chain_1 10010 ... }</code> Using this parameter, you can generate the control file for both flat and hierarchical DEFs.
<code>-scan_control_file filename</code>	
	Specifies the scan control file name required to run the Scan Mode Analysis flow. The scan control file contains details about the scan analysis mode used for testing. For more information, refer to Scan Control File Format.
<code>-scan_mbff_chain_type type</code>	
	Specifies the type of multi-bit scan flip-flop. • <code>liberty</code> - specifies to use the syntax in Liberty to decide the multi-bit flip-flop type. • <code>serial</code> - specifies to pick the first flop as the starting flop, and treat the scan chain as serial, regardless of .lib. • <code>parallel</code> - specifies to treat the scan chain as parallel, regardless of .lib.
<code>-settling_buffer value</code>	

	Specifies the settling buffer time (in ps unit) between multiple windows of a VCD file. This parameter is used when you specify multiple pairs of the start time and end time for the non-overlapping multiple windows specified in an activity file (using the <code>read_activity_file</code> command). Multiple current waveforms representing multiple VCD windows are merged into a single continuous current waveform. To capture the waveform for signals at the edge of user windows, a buffer time is added between the windows being stitched. This buffer time ensures that the stitched waveform is contiguous. The generated waveform hence has the duration of user windows and the intervening settling buffers. The default settling buffer period is 400ps. You must specify the buffer value without unit. The following is an example of this feature: <pre>set_power_analysis_mode -verbosity true \ -settling_buffer 200 -report_missing_nets true read_activity_file \ -format VCD -scope adder/u1 \ -start {0ps 2000ps} -end {1100ps 3100ps} \ -block ../vcd/dut_5buf_full_chip.vcd</pre>
	<pre>-show_cell_usage_stats {true false}</pre>
	Default: false Provides cell usage statistics alongside power values, indicating the number of cells used from a specific library and the corresponding power percentage. This helps identify libraries with the greatest and least impact on power consumption. You must ensure that this parameter is specified before using the <code>report_power</code> command, as demonstrated below: <pre>set_power_analysis_mode -show_cell_usage_stats true report_power -outfile p1.rpt</pre> Following is a snippet from the report: <pre>Cell usage statistics: Library jlos_h , 88 cells (7.351713%) , 6.56434e-06 mW (0.554089%) Library jlos_l , 123 cells (10.275689%) , 0.000799809 mW (67.511081%) Library jlos , 986 cells (82.372597%) , 0.000378334 mW (31.934830%)</pre> Usage Guidelines: This parameter is applicable only to the static power calculation, Verilog-based (<code>-static_netlist verilog</code>) flow.
	<pre>-smart_window {true false}</pre>

Default: false

Specifies to enable the smart window feature to extract peak power scenarios from a vector-based activity file. A smart window is a variable size window to capture maximum regions of high power dissipation. An activity file can have multiple variable-sized regions that are ranked in the descending order of power metric.



The use model of the parameter is:

```
set_power_analysis_mode -smart_window true
set_power_analysis_mode -worst_step_size value
set_power_analysis_mode -worst_window_type [ power | activity | delta_power | delta_activity |
ir | critical_inst_power ]
```

`-split_bus_power {true | false}`

A value of `false` applies the internal power number to each individual bit.

A value of `true` divides the internal power number by bus width, before applying it to the bus bit.

Default: false

`-state_dependent_leakage {true | false}`

When set to `false`, power analysis performs state independent leakage power calculation.

Default: true

`-stateprop_ignore_unannot_pins {true | false}`

Default: false

Specifies to ignore unannotated pins in the functional logic of output pins for the RTL propagation flow. This parameter is set to `true` if there are unannotated input pins for instances and such pins are used in the functional logic of output pins that are propagated. For example output pin Q is propagated with functional logic A & B where A is unannotated and might cause wrong propagation for the Q pin.

`-static_multi_mode_scenario_file filename`

	Specifies a file with information on instance scaling scenario for the static multi-mode IR analysis flow. When this parameter is specified, Power Analysis creates the multi-mode static current file that is used during rail analysis. The format of the input file is: <pre>INST_NAME PIN_NAME SCALE_FACTOR_1 SCALE_FACTOR_N</pre> Here, the first column indicates the macro instance name, the second column indicates the pin name, followed by multiple current scaling factors. Each column of the scaling factor is referred to as a scenario. You must specify positive values only for scale factors. The following is a snippet from the specified input file: <pre>top/macroA VDD 1 0.5 0.5 1 top/macroA VDDM 0.5 0.5 0.5 1 top/macroB VDD 0.5 1 0.5 0.5</pre> If scale factors are not specified for certain scenarios, the tool will automatically use the instance's last stored scale factor value to complete current file generation. In the following example, scale factor value 0.45 will be used in the 3rd and 4th scenario for instanceC, and scale factor value 0.35 will be used in the 4th scenario for instanceD: <pre>instanceA VDD 0.45 0.55 0.65 0.31 instanceB VDD 0.31 0.5 0.67 0.21 instanceC VDD 0.25 0.45 instanceD VDD 0.31 0.25 0.35</pre>
	<pre>-static_netlist {verilog def}</pre>
	Specifies to perform static power analysis using a Verilog or DEF only netlist. By default, the static power analysis is performed using the Verilog netlist as the primary netlist. Using this parameter, you can specify DEF when a DEF netlist with logical connectivity is available, and use it as primary and not consider the Verilog netlist. This parameter is also enabled when <code>set_rail_analysis_mode -report_power_in_parallel</code> is true for static power computation. Default: verilog
	<pre>-start_time_alignment {true false}</pre>
	Specifies to align all the signals of the activity file at time zero. When set to true, the simulation period is <code>max(endtime - starttime)</code>. When it is false, the simulation period is <code>max(endtime) - min(starttime)</code>. Default: true Note: This parameter is not supported in the vector profiling flow. In the vector profiling flow, the start time will always be the <code>min(starttime)</code> and the end time will always be <code>max(endtime)</code>.
	<pre>-switching_power_on_rise_only {true false}</pre>
	Default: false Specifies to perform switching power analysis only for rise toggles. Usage Guidelines: The parameter is only applicable to the event-based power calculation flow (<code>set_power_analysis_mode -method event_based</code>). This parameter impacts only the static power report, and does not affect the dynamic power results.
	<pre>-thermal_input_file file</pre>

Specifies to read a power map file to perform thermal analysis. Thermal input file contains die area divided into tiles, and temperature value for each tile. This temperature of each tile is used to calculate power of all the instances that fall in that tile's bounding box.

The format of the thermal input file is:

```
DIE_TEMPERATURE_MAP
DIE_AREA    xmin(um) ymin(um) xmax(um) ymax(um) nx(int) ny(int)

NUMBER_OF_LAYERS    nLayers(int)
LAYER    layer_name(string)
TEMPERATURE
XCOOR    x0(0)    x0(1)    x0(2)... ..x0(nx-1)
YCOOR    y0(0)    y0(1)    y0(2)... ..y0(ny-1)

T0(0,0)    T0(1,0)    ...    T0(nx-1,0)
T0(0,1)    T0(1,1)    ...    T0(nx-1,1)

:
T0(0,ny-1)    T0(1,ny-1)    T0(nx-1,ny-1)
END_TEMPERATURE
END_LAYER
END_DIE_TEMPERATURE_MAP
```

where,

- **DIE_AREA** specifies the coordinates of the four points of die outline as well as the tile number in X and Y direction.
- **NUMBER_OF_LAYERS** is the number of layers in each die instance
- **LAYER** specifies the LEF layer name
- **XCOOR** and **YCOOR** are the coordinates of each tile
- **T0(0,0)** is the temperature value assigned to a tile at a specific coordinate

```
-thermal_leakage_temperature_scale_table_file filename
```

Specifies a file containing the user-defined temperature values and temperature scale factors for each instance. This parameter allows you to use the current file (.ptiavg) with re-adjusted leakage power values to perform thermal-aware IR drop analysis.

When this parameter is specified, the generated power will include the scaled leakage power values. The `-thermal_leakage_temperature_scale_table_file` parameter must be specified with the `-thermal_input_file` parameter to import the temperature map file generated by Celsius.

The following is the format of the temperature-aware leakage scaling table:

```
#Type      Instance_Name      temp1      temp2      temp3      temp4
(Type: Lib/Cell/inst)

inst      inst_name1      scalefactor@temp1      scalefactor@temp2      scalefactor@temp3
scalefactor@temp4
inst      inst_name2      scalefactor@temp1      scalefactor@temp2      scalefactor@temp3
scalefactor@temp4
```

File Format Example:

```
#Type      Name      25      50      125      150
inst      ring/b213      1      2      5      10
inst      ring/b212      1      2      5      10
inst      ring/b211      1      2      5      10
inst      ring/b210      1      2      5      10
```

`-to_x_transition_factor value`

Specifies how 0/1 to X transitions are counted. The default value is 0.5.

`-to_z_transition_factor value`

Specifies how 0/1 to Z transitions are counted. The default value is 0.25.

`-transition_factor_based_duty {true | false}`

Default: false

Specifies that duty for X and Z toggles is calculated using the X and Z transition factor of toggle specified using the following parameters:

- `-from_x_transition_factor`
- `-from_z_transition_factor`
- `-to_x_transition_factor`
- `-to_z_transition_factor`

The default value of false specifies to use the duty value given with the `set_default_switching_activity/set_switching_activity -duty` parameter.

`-transition_time_method {min | avg | max}`

	Default: The default value is: • <code>min</code> for static power analysis using the Verilog netlist (<code>-static_netlist verilog</code>). • <code>avg</code> for static power analysis using the DEF netlist (<code>-static_netlist def</code>) and dynamic power analysis. Specifies the minimum, maximum, or average transition time method that will be used with the integrated timer or the external TWF.
<code>-twf_delay_annotation {min avg max}</code>	
	Allows you to choose between min, max, or avg arrival times from the timing window file (TWF). The arrival time is annotated to the vectors processed from the RTL VCD/FSDB Zero-Delay flows. The default value is <code>avg</code>.
<code>-twf_load_cap {min avg max ignore}</code>	
	Allows you to ignore or select minimum, maximum, or average value of TWF external load or capacitance for power calculation. The <code>ignore</code> argument ignores the <code>EXTERNAL_LOAD</code> statements during TWF parsing in the power calculation flow. This argument allows you to control the inclusion of external load on the block peripheral or output ports. It may be useful to specify this argument when user-specified load capacitance values (<code>-external_load_config_file</code>) are being used. <i>Default:</i> <code>max</code>
<code>-unified_power_switch_flow {true false}</code>	
	Default: <code>false</code> Specifies to enable the unified analysis of the always-on and switch nets. When set to <code>true</code>, the power-up and always-on analyses are performed in a single flow, wherein the switch nets and always-on nets are simulated together resulting in improved performance and accuracy.
<code>-use_cell_leakage_power_density {true false}</code>	
	Specifies to use library leakage density times area (<code>default_leakage_power_density</code>), if defined, instead of the library's default cell leakage power (<code>default_cell_leakage_power</code>). This is used when leakage power for cells are missing or not defined. The default value of this parameter is <code>true</code>.
<code>-use_fastest_clock_for_dynamic_scheduling {true false}</code>	
	Specifies to use only the respective fastest clock associated with each net or pin to schedule events for a given simulation period. The default value of this parameter is <code>false</code>.
<code>-use_lef_for_missing_cells {true false}</code>	
	Allows a combination of LEF and PGV in the dynamic analysis flow. The default value of this parameter is <code>false</code>. If there are cells that are common to LEF and PGV, the PGV cell will get the precedence.
<code>-use_only_ddv_current {true false}</code>	

	Default: <code>true</code> Controls the use of static current of a detailed dynamic power-grid view (DDV PGV) cell for static power calculation. The parameter is set to <code>true</code> by default, that is, the DDV PGV static current is not used in the static power calculation flow. If you want to use the DDV PGV static current for static power calculation, set this parameter to <code>false</code>.
<code>-use_physical_partition {true false}</code>	
	Default: <code>false</code> Enables tile or area-based partitioning of a design for distributed processing. When this parameter is specified, the top-level design is divided into physical tiles, which are then grouped to ensure that each group or partition has the same instance count. Each partition is considered and processed as a separate job. Usage Guidelines: • This parameter is only applicable to the event-based power analysis flow (<code>set_power_analysis_mode -method event_based</code>) in the XP mode. • This parameter must be specified with the <code>-partition_count</code> parameter.
<code>-use_twf_stable_randomized_arrival_delay {none data_only all}</code>	
	Specifies to annotate randomized arrival time based on net names. The parameter ensures that when there is a change in design, the net delay is not impacted. This parameter is supported in both the static and dynamic flows. The possible arguments are: • <code>none</code> - disables randomized timing window annotation. • <code>all</code> - performs randomized annotation for both the clock and data nets. • <code>data_only</code> - performs randomized annotation only for the data nets.
<code>-use_zero_delay_vector_file {true false}</code>	
	Enables zero delay mode in vector-based dynamic analysis to avoid pessimism in current estimation. When set to <code>true</code>, the software reads the cycle accurate (zero delay) VCD/FSDB file and adds the timing/delay information from the Timing Analyzer or external TWF file to the VCD/FSDB file. Default: <code>false</code>
<code>-vector_profile_mode {activity event_based power_density transient}</code>	

	Default: <code>event_based</code> Specifies the method used to perform vector profiling. The possible arguments are: • <code>activity</code> - performs activity profiling. This parameter generates only the worst activity report, and does not generate the average power report. The name of the file is as specified using the <code>report_power - outfile</code> parameter. • <code>event_based</code> - generates the average power report. This is the average power of all events in the user-specified window. This report includes a summary based on the power components, cell type, power rail, and clock network. The worst window report is captured in a file specified with the <code>report_power - outfile</code> parameter, and the report is suffixed by <code>'.avgpower'</code>. When <code>-outfile</code> is not specified, the <code>vectorprofile.report.avgpower</code> report file is created. • <code>power_density</code> - performs area or power density profiling. For this method, power is reported for each tile of the design, where design can be considered to be physically divided into equal area tiles. By default, a design is divided in 10x10 tiles. • <code>transient</code> - generates the event-based average power report and current files for the worst power window. Usage Guidelines: This parameter must be specified with the <code>-method vector_profile</code> parameter.
<code>-worst_case_vector_activity {true false}</code>	
	Default: <code>false</code> Specifies to use the worst activity value when multiple vectors are specified in the static power calculation flow.
<code>-worst_step_size value</code>	
	Default: Total simulation duration/100 Specifies the step size or resolution for vector profiling. Activity/ Power profiling is carried out for every step size that you specify. When this parameter is used with <code>-worst_window_size</code>, the specified step size is the resolution at which window sliding is performed. The supported time units are s, ms, us, ns, and ps. If no explicit time unit is specified, the default ns will be applied.
<code>-worst_window_count value</code>	
	Default: 1 (implements single worst window) Specifies the number of worst windows from the last vector profile run to be used for dynamic power analysis. It provides n top windows with maximum activity/power/IR. This parameter is used when you need to combine dynamic power analysis and vector profiling for multiple worst-case windows in a single run.
<code>-worst_window_coverage { top tile }</code>	

	Default: <code>top</code> Specifies how the worst window type is applied for multi-window analysis in the vector profiling flow. The following are the possible options of this parameter: • <code>top</code>: worst window selection for the full design • <code>tile</code>: interval selection as per the worst tile power Use Model: <pre>set_power_analysis_mode -method event_based -worst_window_coverage tile report_power -outfile power.rpt -power_density_tiles 100</pre> This flow should be run with <code>-method event_based</code>. When the <code>-worst_window_coverage tile</code> parameter is specified, you should include any one of the following parameters to specify the total number of tiles: • <code>report_power -power_density_tile_size " 15 15 "</code> • <code>report_power -power_density_tiles_row_col " 2 2 "</code> • <code>report_power -power_density_tiles 100</code> The software generates the <code>power.rpt.density</code> report that has the power density tiles profile report with an additional section "Intervals covering unique tiles having worst power density".
	<pre>-worst_window_reports {full worst both}</pre>
	Specifies to generate static power analysis reports for the specified worst window type. The possible arguments are: • <code>full</code>: creates the static power report for the full simulation window specified with the <code>-start/-end</code> parameters of the <code>read_activity_file</code> command. • <code>worst</code>: creates the static power report for the worst window only. • <code>both</code>: creates the static power report for both the full simulation and worst windows. The default value is <code>full</code>. The worst window report is created with <code>‘.1’</code> added in the file name, as shown below: • Static reports for full simulation: <code>power.rpt.avgpwr</code>, <code>power.rpt.instpwr</code> • Static reports for worst window: <code>power.rpt.1.avgpwr</code>, <code>power.rpt.1.instpwr</code>
	<pre>-worst_window_size value</pre>
	Default: <code>None</code> Specifies the duration of dynamic analysis. This parameter is used for sliding window vector profiling that involves sliding the profiling windows by using a very small resolution value to get the window with the worst current profile. <code>-worst_window_size</code> must be specified with <code>-worst_step_size</code>. The supported time units are s, ms, us, ns, and ps. If no explicit time unit is specified, the default ns will be applied.  This parameter is not supported with net-based activity profiling (<code>set_power_analysis_mode -method vector_profile -vector_profile_mode activity</code>).

```
-worst_window_type { power | activity | delta_power | delta_activity | ir | critical_inst_power }
```

Default: power

Specifies the window type to be used for multi-window analysis. The possible arguments are:

- power - worst instance power
- activity - worst instance activity
- delta_power - worst dp/dt (rate of change of power)
- delta_activity - worst da/dt (rate of change of activity)
- ir - worst IR
- critical_inst_power - worst power for critical/IR-sensitive instance (worst timing and IR)

```
-write_boundary_leakage_edge_annotation {true | false}
```

Default: false

Writes an edge annotation summary file for boundary leakage calculation. This parameter is used to report location-based Vt layer information on all the abutted edges in the design. The generated report name is boundarygateleakage.edgeannotation.rpt.

```
-write_default_uti {true | false}
```

Default: true

For instances that are not hooked to any power/ground rail, the current for these instances are reflected in the default static current files:

- static_default_ground_rail.ptiavg
- static_default_power_rail.ptiavg

```
-write_dynamic_currents {true | false}
```

	Default: <code>false</code> Specifies to create dynamic current files either for full simulation duration or for user-specified duration. Usage Guidelines: This parameter is only applicable to the event-based power analysis flow (<code>set_power_analysis_mode -method event_based</code>). Example: <pre>set_power_analysis_mode -method event_based -write_dynamic_currents true - enable_state_propagation true read_activity_file sim.vcd -format VCD report_power -outfile power.rpt</pre>
<code>-write_profiling_db {true false}</code>	
	Default: <code>false</code> Writes out profiling databases which can later be viewed as histograms using the SimVision interface. You can specify this parameter during both the activity and power vector profiling. The profiling databases support instance level and power/ground net histograms. It creates an output file name with the <code>*.trn</code> extension.
<code>-write_simulation_db {true false}</code>	
	Default: <code>false</code> Specifies to save functional waveforms for each instance when parsing a vector-based activity file. When set to <code>true</code>, creates SimVision waveform database (SHM database) for toggle simulation used during power analysis, which can later be viewed as histograms using the SimVision interface. The simulation database (<code>shmdb.trn</code>) is generated in the power analysis output directory.
<code>-write_static_currents {true false}</code>	
	Default: <code>false</code> A value of <code>true</code> tells Voltus to generate the current data files per net. Note: When you perform static power calculation in the dynamic power analysis flow, you can use either the <code>-disable_static</code> or <code>-write_static_currents</code> parameter to enable static power calculation and write static current file. If only <code>-disable_static</code> is set to <code>false</code> and <code>-write_static_currents</code> is not set, the tool automatically sets <code>-write_static_currents</code> to <code>true</code>. Similarly, if <code>-write_static_currents</code> is set to <code>true</code>, and <code>-disable_static</code> is not set, the tool automatically sets <code>-disable_static</code> to <code>false</code>.
<code>-x_count_transition_using_3_states {true false}</code>	
	Default: <code>false</code> Specifies to calculate the toggle rate using 3 states, pre-previous, previous and current states.
<code>-x_transition_factor value</code>	

	When using VCD/FSDB, one can specify how transitions to and from X are counted. These transitions are defined as shown in Table 5-1. Transitions to and from the X state will be treated as full transitions multiplied by the specified factor. Note: 0 -> X -> 1 or 1 -> X -> 0 count as full transitions and they are not controlled by the parameter. <i>Default:</i> 0.5
<code>-z_transition_factor value</code>	
	When using VCD/FSDB, one can specify how transitions to and from Z are counted. These transitions are defined as shown in Table 5-1. Transitions to and from the Z state will be treated as full transitions multiplied by the specified factor. Note: 0 -> XZ-> 1 or 1 -> Z-> 0 count as full transitions and they are not controlled by the parameter. <i>Default:</i> 0.25
<code>-zero_delay_vector_toggle_shift value</code>	
	Specifies to shift the current waveform in the zero-delay VCD flow. This parameter is useful in removing the idle period for the current waveform result with a long idle period (small flat current) before the first high current peak because of delay annotation. The parameter allows to offset the idle period from the delay values. The use model is given below: <code>set_power_analysis_mode -zero_delay_vector_toggle_shift -3.0ns</code> The default value is 0ns. The supported time units are s, ms, us, ns, and ps. The default unit is ns. You can specify a negative or a positive value. A positive value means right shift in the timeline or add more delay to the original arrival time, and a negative value means left shift in the timeline or trim the unwanted idle period. The <code>-zero_delay_vector_toggle_shift</code> parameter enables you to make a more realistic current waveform that can be easily aligned with the other current waveforms. Note: This parameter is similar to the <code>set_twf_attribute -shift_arrival_time</code> parameter, with the only difference that <code>-zero_delay_vector_toggle_shift</code> supports both positive and negative values while <code>-shift_arrival_time</code> supports only a positive value.

Table 5-1 Transitions from/to X and Z values and

Transition	Definition	Default
to/from X	[0 or 1 or Z] <-> X	0.5
to/from Z	[0 or 1] <-> Z	0.25

Scan Control File Format

The use model for the Scan Mode Analysis flow is:

```
set_power_analysis_mode -method dynamic_vectorbased -scan_control_file filename
```

The format of the scan control file is:

```
MODE = [ SHIFT | VECTORLESS]
VECTORLESS_GLOBAL_ACTIVITY = <factor> [Between 0 - 1]
<chain_name> <flop_inst> <initial_state> <master_cell_name> <scale> <mode> <scan_in_pin> <scan_out_pin>
<scan_en_pin>
```

Note that <chain_name> is optional, <initial_state> is optional in vector-less mode, and <master_cell_name>, <scale>, <mode>, <scan_in_pin>, <scan_out_pin>, <scan_en_pin> are optional and only needed for the first internal scan flop inside one macro.

Where,

- Mode specifies the mode used for scan design analysis. The two modes are: SHIFT and VECTORLESS.
- VECTORLESS_GLOBAL_ACTIVITY specifies the activity specification used in the VECTORLESS mode.
- chain_name is the name of the scan chain.
- initial_state is the initial state of all scan flops.
- master_cell_name is the name of the macro.
- scale is the scaling factor applied to the current.
- mode is mode name of the scan mode in the PGV view.
- scan_in_pin is the macro's scan_in pin name.
- scan_out_pin is the macro's scan_out pin name.
- scan_en_pin is the macro's scan_enable pin name.

Examples

- The following command uses the negative internal power from the .lib files during analysis, specifies that the transitions that are to and from the Z state will be multiplied by .2, and specifies that the transitions that are to and from the X state will be multiplied by .45:

```
set_power_analysis_mode -honor_negative_energy true \
-z_transition_factor .2 \
-x_transition_factor .45
```

- The following command resets the -honor_negative_energy parameter to its default value:

```
set_power_analysis_mode -reset -honor_negative_energy
```
- The following command resets all set_power_analysis_mode parameters to their default values:

```
set_power_analysis_mode -reset
```

- The following command ignores leakage power, and scales only the switching and internal power by the specified scale factor:

```
set_power -instance top.block2 2mW
set_power_analysis_mode -disable_leakage_scaling true
```

- The following command sets up dynamic power analysis.

```
set_power_analysis_mode \
    -method dynamic_vectorless \
    -corner max \
```

```
-create_binary_db true \  
-write_static_currents true \  
-honor_negative_energy true \  
-ignore_control_signals false \  
-power_grid_library { fast_allcells.cl }
```

- The following command adds a prefix SM_ to the current data directory and the underlying current files:

```
set_power_analysis_mode -ouput_current_data_prefix SM_
```

The following current files are generated:

```
SM_dynamic_VDD.ptiavg  
./SM_PTIData/SM_dynamic_1_VDD.ptiavg  
./SM_PTIData/SM_dynamic_2_VDD.ptiavg
```

- The following command is another example of dynamic power analysis.

```
set_power_analysis_mode \  
-method dynamic_vectorless \  
-off_pg_nets VDDlu \  
-create_binary_db true \  
-write_static_currents true \  
-honor_negative_energy true \  
-ignore_control_signals false \  
-power_grid_library {accurate_stdcells.cl TSDN65LPA1024X32M8F.cl}
```

- The following command is an example of the state-propagation-based dynamic vectorless flow, and also specifies to generate a report (voltus_power.stateprop.switchlist) containing all the switching times for the instances specified in the inst_list file:

```
set_power_analysis_mode -method dynamic_vectorless -enable_state_propagation true -  
report_instance_switching_list inst_list
```

- The following is an example of running power analysis in the distributed mode:

```
set_distribute_host -rsh -add "m1 m2"  
  
##The following blue-highlighted parameters specify the number of remote hosts and number of CPUs per remote  
host  
set_multi_cpu_usage -remoteHost 8 -cpuPerRemoteHost 2 -localCpu 16  
  
##The following blue-highlighted parameter specifies that all the blocks included in the dp_setup.txt file will be  
run in parallel  
set_power_analysis_mode -method dynamic_vectorless -distributed_setup dp_setup.txt
```

The following is the content of the dp_setup.txt file:

```
INST top.block1 block  
INST top.block4 block  
CELL block2 block
```

In the above example, 4 power distributed jobs are created for: top.block1, top.block4, hierarchical cell block2, and any other hierarchical instances not specified in dp_setup.txt that are run with the top-level.

- The following is an example of performing standalone power report generation in the XP mode:

```
set_power_analysis_mode -method dynamic_vectorless \  

```



```
-enable_state_propagation true \  
-power_grid_library { /pgv_dir/tech_pgv/techonly.cl /pgv_dir/stdcell_pgv/stdcells.cl  
/pgv_dir/macro_pgv/macros_pll.cl } \ -extraction_tech_file /qrcTechFile -enable_xp true  
...  
report_power
```

- The following command specifies to enable area-based partitioning, and distribute the event-based analysis into four jobs:

```
set_power_analysis_mode -method event_based -enable_xp true -use_physical_partition true -partition_count  
4
```

- The following command specifies to enable interactive generation of power reports:

```
set_power_analysis_mode -method event_based -use_physical_partition true -partition_count 2 -  
enable_interactive_reports true -enable_xp true
```

- The following command specifies to enable disk caching of toggles:

```
set_power_analysis_mode -method event_based -enable_disk_mapping true -enable_xp true
```

set_power_calc_temperature

```
set_power_calc_temperature  
-help  
temperature
```

Sets the temperature for static power calculation.

In the presence of a trilib set, the internal and leakage power will be interpolated as the per the given temperature specification.

Parameters

<code>-help</code>	Prints out the command usage.
<code>temperature</code>	Specifies the temperature for static power calculation. Note: The static power calculation software gets the temperature value of each instance from the following sources (in order of precedence): • Instance temperature file • <code>set_power_calc_temperature</code> command • The current operating condition temperature

set_power_include_file

```
set_power_include_file
[-help]
file
[-block master_cell_name]
[-reset]
```

Specifies a power analysis include file. You can use the `set_power_include_file` command to specify certain commands of the dynamic engine which do not have a Voltus equivalent. For more information on these commands, refer to the [Power Analysis Options](#) chapter.

The `set_power_include_file` command is required only if the `set_power_analysis_mode -static_netlist` is set to `def`.

Note: The `set_power_include_file` command is used to pass these legacy power analysis commands and `report_power` command options only in the dynamic mode (vectorbased and vectorless). If `set_power_analysis_mode` is set to `static`, the `set_power_include_file` command is ignored.

Parameters

<code>-help</code>	Prints out the command usage.
<code>file</code>	Name of the include file. This file will include all commands that cannot be translated to Voltus commands. In addition, you can also include the <code>report_power</code> command options to generate different power reports.
<code>-block master_cell_name</code>	
	Specifies the master cell name to which the powermeter options are to be applied. The parameter allows you to apply certain commands of the dynamic engine only to a specific block partition. This parameter allows you to apply the include options to a specific block instead of applying them globally to all blocks. The <code>-block</code> parameter is applicable only to the block-based distributed power flow.
<code>-reset</code>	Resets all options back to default values.

Examples

- The following command specifies a power include file of `power.inc`:

```
set_power_include_file power.inc
```

- The following example shows the contents of the `power.inc` file.

power.inc file must use the pre-defined design and power objects as "design" # and "ChipPwr" respectively. The power output object is not supported.

```
design calCellsToIgnore BUFX1
ChipPwr calRailVoltage VDD 1.2
```

```
report_power -net -outfile Net.rep  
report_power -cell {INV*} -outfile Invpower.rep
```

set_power_output_dir

```
set_power_output_dir dir  
[-help]  
[-reset]
```

Specifies the name of the directory that power information will be output to.

Parameters

<i>dir</i>	Specifies the name of the directory in which power analysis output information is written.
-help	Prints out the command usage.
- reset	Resets all options back to default values.

Examples

- The following command specifies the output directory called `static_power_AvallOn` in which the power information is written.

```
set_power_output_dir static_power_AVallOn
```

set_switching_activity

```
set_switching_activity
[-help]
[-activity factor | -density transition_density ]
[-cell cell_name]
[-clock clock_name]
[-comb_clockgate_ratio num]
[-duty value]
[-force]
[-hier hierarchy_name]
[-icg_ratio num]
[-inst instance_name]
[-net net_name | -port port_name | -pin pin_name | -input_port port_name | -output_port port_name | -bidir_port port_name]
[-period value]
[-quiet]
[-reset]
[-scale_factor value]
[-unclocked]
```

Specifies activity for nets, pins, and ports. This command is used to specify user-defined activities at specific pins/nets/ports. You must specify this command before `report_power`.

The `set_switching_activity` command can accept a net/pin/port list using the `get_pins/get_ports/get_nets` commands. You can use the `get*` commands to create a collection of nets/pins/ports whose name matches the supplied pattern list. Therefore, providing you with the flexibility in providing lists through use of regular filter patterns. This feature is supported in both static and dynamic analysis engines.

 If the pin/instance/net names have backslash '\ or square brackets '[]', you must specify the name within curly brackets {{...}} to ensure that '\ or '[]' are correctly processed by the Tcl Interpreter. In the following example, the pin name is specified within curly brackets as it contains '\ and '[]':

```
set_switching_activity -cell LDN_INV_4 -pin {{I1_1\A [0]}} -density 0.6 -duty 0.31
```

If '\ and '[]' are not there, curly brackets are not required.

Parameters

-help	Prints out the command usage.
-activity factor	
	Specifies the activity for a net, pin, or top-level port. The <i>activity_factor</i> specifies the number of times the net, pin, or port switches in a clock cycle. Enter any positive number. If the activity is set to 0.5, the activity happens once for every two clock cycles. If the activity is set to 2.0, the activity happens two times per clock cycle. There is no default value.

<code>-bidir_port port_name</code>	
	Specifies the name of the bidirectional port. The port names are in the following format: <code>bidirectional-port</code> . This parameter supports wildcards (*).
<code>-cell cell_name</code>	
	Specifies the activity on all output pins for all instances of the specified cell. The <code>set_switching_activity -cell</code> command parameter works without the <code>-pin</code> parameter, and sets activity on all pins. If you need to specify the pins on which activity should be set, then use the <code>-pin</code> parameter.
<code>-clock clock_name</code>	
	Assigns global activity factor to all nets in domain of <code>clock_name</code> if and only if <code>clock_name</code> is the fastest clock in the domain set. To use this option, <code>-clock</code> and <code>-activity</code> must be used without any other options.
<code>-comb_clockgate_ratio num</code>	
	Specifies to set the propagation ratio for combinational clock gate cell outputs. When specified, this parameter sets the output activity of any instance that is identified as a combinational clock gating cell to <code>num</code> times the fastest transition density arriving on the instance's clock inputs. You can specify this ratio for a net, pin, port, or clock. This parameter cannot be specified with the <code>-hier</code> parameter.
<code>-density transition_density</code>	
	Specifies the transition density for a specific net, pin, or top-level port. Transition density specifies the number of transitions per second. It can be any positive number. There is no default value. Note: If you specify the <code>-density</code> parameter in conjunction with the <code>-period</code> parameter, the software issues a warning and the <code>-period</code> parameter is ignored.
<code>-duty value</code>	
	Specifies the duty cycle, which is the probability that the signal is a logical 1. The <code>-period</code> parameter must be specified to add a duty. If not specified, the <code>set_switching_activity</code> command picks this value from the external TWF or SDC through the Common Timing Engine (CTE). Default: Uses the duty specified by the <code>set_default_switching_activity</code> command if the <code>-duty</code> parameter is not specified.

<code>-force</code>	Specifies to apply user-defined activities for specific nets, pins, and ports, without having to load a verilog netlist using the <code>set_top_module</code> command. When the <code>set_top_module</code> command is not specified, the <code>set_switching_activity</code> command does not support the <code>-cell</code> parameter, and the net/pin/port list using the <code>get_pins/get_ports/get_nets</code> commands.
<code>-hier hierarchy_name</code>	
	A hierarchical name that an activity is being assigned to.
<code>-icg_ratio num</code>	
	Specifies to set the propagation ratio for integrated clock gate (ICG) cell outputs. When specified, this parameter sets the output pin of all identified ICG instances to <i>num</i> times the fastest transition density arriving on clock inputs of the instance. You can specify this ratio for a net, pin, port, or clock. This parameter cannot be specified with the <code>-hier</code> parameter.
<code>-input_port port_name</code>	
	Specifies the name of the input port. The port names are in the following format: <i>input-port</i> . This parameter supports wildcards (*)
<code>-inst instance_name</code>	
	Specifies the activity on a specific instance.
<code>-net net_name</code>	
	Specifies the name of the net. This parameter supports wildcards (*).
<code>-output_port port_name</code>	
	Specifies the name of the output port. The port names are in the following format: <i>output-port</i> . This parameter supports wildcards (*)
<code>-period value</code>	
	Specifies the period that the activity is referenced to. If not specified, a default frequency is used. If a negative number is specified, the static power calculation engine reverts back to the default frequency. Units in seconds (s), milliseconds (ms), microseconds (us), nanoseconds (ns), or picoseconds (ps). Note: If you specify the <code>-period</code> parameter in conjunction with the <code>-density</code> parameter, the software issues a warning and the <code>-period</code> parameter is ignored. Default: The time unit is defined in the first <code>.lib</code> file read during design import. If the time unit is not defined, the period specified in the <code>set_default_switching_activity</code> command is used.

<code>-pin pin_name</code>	
	Specifies the name of a pin. The pin names are in the following format: <i>instance-name + default-hierarchy-separator + pin</i> . This parameter supports wildcards (*)
<code>-port port_name</code>	
	Specifies the name of a top-level port. The port names are in the following format: <i>top-level-port</i> . This parameter supports wildcards (*)
<code>-quiet</code>	Suppresses all error and warning messages generated when the <code>set_switching_activity</code> command is run.
<code>-reset</code>	Resets all specified options back to default values. The reset option should be the first option specified, otherwise all options set prior to this will be reset. Note: You can selectively reset the pin/port/net specifications. In addition, you can also selectively reset the pin/port/net specifications of a list of objects through the <code>get_pins/get_ports/get_nets</code> commands.
<code>-scale_factor value</code>	
	Specifies to scale frequencies of the specified clock domains. This parameter allows you to scale transition densities of all instances associated with the clock domain by the specified scale factor to compute internal and switching power. This parameter must be specified with the <code>-clock</code> parameter. The <code>-scale_factor</code> parameter is only applicable to static power and dynamic vectorless power analysis.
<code>-unclocked</code>	Applies <i>activity</i> (with reference <i>period</i>) to all nets that have no clock domain (<code>-unclocked</code>). To use this option, <code>-unclocked</code> must be used with <code>-activity</code> and <code>-period</code> with no other options.

Activity Precedence

See "[Activity Precedence](#)".

Examples

- The following command specifies that net n189 will switch once every ten clock cycles, with a clock period of 7520ns and a duty cycle of 50 percent:

```
set_switching_activity -net n189 -activity 0.10 -period 7520ns -duty .5
```

- The following command sets the transition density of port out_cpu_top to 20000 transitions per second:

```
set_switching_activity -port out_cpu_top -density 20000
```

- The following command applies a transition density of 1e+06 to all input ports that match the specified pattern:

```
set_switching_activity -port[get_ports "mode*" -filter {port_direction==in}] -density 1e+06
```

- The following command resets the previously defined activity on all nets that match the specified pattern:

```
set_switching_activity -reset -net[get_nets n23*]
```

- The following command specifies the clock gate ratio for the clock domain sys_clk:

```
set_switching_activity -clock sys_clk -icg_ratio 0.8 -comb_clockgate_ratio 0.5
```

- The following command specifies to set the default switching activity for all other clock domains of the design:

```
set_default_switching_activity -icg_ratio 1.0 -comb_clockgate_ratio 1.0
```

set_thermal_analysis_mode

```
set_thermal_analysis_mode
[-help]
[-boundary_condition_file filename]
[-power_map_file filename]
[-temperature_map_file filename]
[-thermal_output_dir directory_name]
[-tool_path path]
```

Sets the parameters for performing chip-centric thermal analysis after generation of the Voltus thermal model (VTM). This command must be specified before the `analyze_thermal` command.

Parameters

- help	Outputs the command usage and a brief description about the command parameters.
-boundary_condition_file <i>filename</i>	
	Specifies the name of the boundary condition file. The format of the boundary condition file is: <pre>#Length Unit : um #Temperature Unit: C die_thickness=100 front Tfix=25 back Tfix=25 top Tfix=50 bottom Tfix=50 left Tfix=25 right Tfix=25</pre>
-power_map_file <i>filename</i>	
	Specifies the name of the generated power map file.
-temperature_map_file <i>filename</i>	
	Specifies the name of the output temperature map file. Each analysis run would create a directory named “ThermalDir_<number>” automatically. Soft links to the power mapping file and boundary condition file would be created inside the directory. The analysis’s final result is a temperature map named as “<DieName>_TemperatureMap.dat”, and is saved in this directory.
-thermal_output_dir <i>directory_name</i>	
	Specifies a custom output directory for thermal analysis results instead of using the default <code>ThermalDir</code> . This parameter gives you better control over where your output files are stored, making it easier to manage results across multiple runs.
-tool_path <i>path</i>	

	Specifies the full path to the bin folder of the thermal analysis tool (Celsius).
--	---

Examples

- The following command specifies to set thermal analysis parameters:

```
set_thermal_analysis_mode -tool_path $CELSIUSTOOL \  
-boundary_condition_file $bcDir/bc_100.txt \  
-temperature_map_file temperatureMap_100 \  
-thermal_output_dir thermal_dir_new \  
-power_map_file vtm_output/top.vtm.txt
```

Related Topics

- [extract_metal_density](#)

set_twf_attribute

```
set_twf_attribute
[-help]
[-clear_twf_attr]
[-clock clock_name]
[-clock_edge {rise | fall}]
[-clock_freq value]
[-fall_delay {min:max or single_value}]
[-fall_slack {min:max or single_value}]
[-fall_slew {min:max or single_value}]
[-net net_name]
[-pin pin_name]
[-reset]
[-rise_delay {min:max or single_value}]
[-rise_slack {min:max or single_value}]
[-rise_slew {min:max or single_value}]
[-shift_arrival_time time]
[-type {data | clock}]
```

Specifies static timing analysis information. The command allows you to override timing parameters, such as slew and clock, on desired pins/nets from an existing timing windows file (TWF).

Parameters

-clear_twf_attr	
	Specifies to reset the TWF file values for a specific pin or net. The use model of this parameter is: set_twf_attribute -clear_twf_attr -pin/-net <name>
-clock clock_name	
	Specifies the clock name. This indicates that the nets and pins that are timing constrained have arrival times (timing windows) caused by this clock.
-clock_edge {rise fall}	
	Specifies the capturing clock edge direction: rise or fall.
-clock_freq value	
	Specifies the clock frequency. You must specify the -clock parameter along with the -clock_freq parameter to assign the clock for which the frequency needs to be updated.
-fall_delay {min:max OR single_value}	
	Specifies the fall delay for the pin. This parameter is not supported in the static power analysis flow.

<code>-fall_slack {min:max or single_value}</code>	
	Specifies the fall slack for the pin.
<code>-fall_slew {min:max or single_value}</code>	
	Specifies the fall slew for the pin.
<code>-net net_name</code>	
	Specifies the net name. The <code>-net</code> and <code>-pin</code> parameters are mutually exclusive.
<code>-pin pin_name</code>	
	Specifies the pin name. The <code>-net</code> and <code>-pin</code> parameters are mutually exclusive.
<code>-reset</code>	Resets all specified options back to default values.
<code>-rise_delay {min:max or single_value}</code>	
	Specifies the rise delay for the pin. This parameter is not supported in the static power analysis flow.
<code>-rise_slack {min:max or single_value}</code>	
	Specifies the rise slack for the pin.
<code>-rise_slew {min:max or single_value}</code>	
	Specifies the rise slew for the pin.
<code>-shift_arrival_time time</code>	

Specifies to filter the idle period of the current waveform in the dynamic power analysis flow. This parameter is useful in removing the idle period of the current waveform result with long idle period (small current) before the 1st high current peak because of TWF delay annotation. The parameter allows to offset idle period from the TWF delay values.

The use model is given below:

```
set_twf_attribute -shift_arrival_time 3.0ns
```

The default value is 0ns. The supported time units are s, ms, us, ns, and ps. The default unit is ns. You must specify a positive value only. A positive value means shift right in the time line or add more delay to the original arrival time.

You can specify the `-clock` parameter to limit the changes to a clock domain. In addition, you can further add the `-pin/-net` parameter to limit the changes to a block. The following are examples of shifting different delays to different clock domains because different clocks may have different clock insertion delays at the top level:

```
set_twf_attribute -shift_arrival_time 1.2ns -clock clk_a
set_twf_attribute -shift_arrival_time 2.3ns -clock clk_b
```

If a clock domain is not specified, the delay will be applied to all the specified nets/pins.

Note: The final rise/fall delays for the annotated nets/pins after the shift has been applied are reported under the “Shift Arrival Time Report” section of the detailed TWF Attributes report. This report is generated using the

```
set_power_analysis_mode -report_twf_attributes detailed
```

parameter.

The following example illustrates a sample command and report snippet:

Command:

```
set_twf_attribute -shift_arrival_time 3.0ns
set_power_analysis_mode -method dynamic_vectorless ... -
report_twf_attributes detailed
```

Report:

Clock domain annotated

- CK1_1
- CK1_2
- CK2_1
- CK2_2

Rise Delay	Fall Delay	Net Name
1e-09	1e-09	CK1_1
1e-09	1e-09	CK1_2
1e-09	1e-09	CK2_1
...		

```
-type {data | clock}
```

Specifies whether the net or pin is part of the clock or data network.

Examples

- The following command specifies to use the following TWF attributes for the net `netB1`:

```
set_twf_attribute -net netB1 -rise_delay 2.1978:2.1978 -rise_slew 0.05:0.05 -rise_slack 0.2741:3.6961  
-fall_delay 2.1819:2.1819 -fall_slew 0.05:0.05 -fall_slack 0.237:3.6948 -clock CK1_1 -clock_freq 100 -type  
data
```

- The following commands reset the TWF values for the "DFF11/DFF/Q" pin to default, and then sets the rise/fall delay:

```
set_twf_attribute -clear_twf_attr -pin "DFF11/DFF/Q"  
set_twf_attribute -pin "DFF11/DFF/Q" -rise_delay 0.35 -fall_delay 0.45 -clock CLK1
```


set_virtual_clock_network_parameters

```
set_virtual_clock_network_parameters
[-help]
[-cell cell_name]
[-clock list_of_clocks]
[-library library_name]
[-max_fanout value]
[-reset]
[-wire_load_model wireload_model]
```

Specifies to estimate power for a virtual clock tree. When the command is specified, the software reports the power of the clock network based on the buffers inserted in the clock tree. It also reports the number of buffers used for implementing the clock tree and the depth of the clock tree.

You can use this command for fast power estimation of clock network power of a pre-CTS netlist.

This command should be invoked before running the `report_power` command.

Parameters

-help	Outputs a brief description that includes the type and default information for each <code>set_virtual_clock_network_parameters</code> parameter. For a detailed description of the command and all of its parameters, use the man command <code>man set_virtual_clock_network_parameters</code> .
-cell <i>cell_name</i>	
	Specifies the name of the buffer or inverter cell that has to be used to build the clock tree.
-clock <i>list_of_clocks</i>	
	Specifies the list of clocks for which the clock tree needs to be implemented. The default is for all the clocks.
-library <i>library_name</i>	
	Specifies the name of the library which contains the cell.
-max_fanout <i>value</i>	
	Specifies the maximum fanout that the buffer cell can drive.
-wire_load_model <i>wireload_model</i>	
	Specifies the wire load model used to compute the wire load model for the nets in the clock network.
-reset	Resets all specified parameters back to default values.

Example

The following command specifies to use the cell `BUFX12`, with a maximum fanout value of `2`, to build a virtual clock tree:

```
set_virtual_clock_network_parameters -cell { BUX12 } -library typical -max_fanout 2 -wire_load_model B0X0
```

write_power_constraints

```
write_power_constraints  
[-help]  
[-outfile filename]
```

Writes out all the power constraints such as the power analysis mode, activity file specifications, simulation period specifications, and so on, into a file. You can use this command at any time during the session to dump out the power commands/ options specified till that point.

You can source this file into other Voltus/Tempus/Innovus session in order to use the same settings.

Parameters

<code>-help</code>	Outputs a brief description that includes the type and default information for each <code>write_power_constraints</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command <code>man write_power_constraints</code> .
<code>-outfile</code> <i>filename</i>	Specifies the output file into which the power constraints are to be written.

Example

```
write_power_constraints -outfile test1.tcl
```

write_tcf

```
write_tcf file
[-help]
<file>
[-block block_name]
[-exclude_hier_insts list_of_instances]
[-macro_output]
[-pin]
[-primary_input]
[-seq_output]
```

Writes out the propagated switching activity information to a toggle count format (TCF) file. The TCF file contains switching activity information, toggle count information, and the probability of the net or pin being in the logic1 state.

You must specify this command after the `set_power_analysis_mode` command and before the `report_power` command.

Following is the behavior of the `write_tcf` command in different modes:

- **Command File Mode:** If the `write_tcf` command is specified multiple times within a command file, only the last instance of the command will be honored. This means that any earlier instances will be overridden by the final specification.
- **Interactive Mode:** In contrast, if the `write_tcf` command is specified multiple times during an interactive session, each instance of the command will be honored. This allows for multiple configurations or actions to be performed in sequence.

Parameters

-help	Prints out the command usage.
<file>	File name
-block <i>block_name</i>	Specifies to generate the block-level TCF from the top-level design. When specified, it writes out the block-level duty and activity in the generated TCF file. The use model of this parameter is: <code>write_tcf -pin -block blockname TCF_filename</code>
-exclude_hier_insts <i>list_of_instances</i>	Excludes the specified hierarchical modules or instances when writing out a pin-based TCF. This parameter allows you to ignore the activity generated for the excluded instances. The use model of this parameter is: <code>write_tcf -pin -exclude_hier_insts {list_of_instances}</code>
-macro_output	Specifies to generate a TCF file that includes all the activities for the macro outputs in the design.

<code>-pin</code>	Specifies to write out pin-based TCF.
<code>-primary_input</code>	Generates a TCF file that includes only the primary inputs in the design.
<code>-seq_output</code>	
	Generates a TCF file that includes all the activities for the sequential outputs in the design.
<code>file</code>	Specifies the name of the output TCF file.

Example

- The following command writes the propagated switching activity information to TCF file `mmm.tcf`:
`write_tcf mmm.tcf`
- The following command writes a pin-based TCF for the whole design:
`write_tcf -pin pin_design.tcf`
- The following command writes out a TCF which only includes activity information for the primary inputs and sequential outputs in the design:
`write_tcf -primary_input -seq_output Power.tcf`
- The following command writes a net-based TCF for module block1:
`write_tcf -block top.A1.B1 block.tcf`
- The following command writes a pin-based TCF for module block1:
`write_tcf -pin -block top.A1.B1 block.tcf`
- The following command writes out a pin-based TCF that excludes the specified hierarchical instances:
`write_tcf -pin -exclude_hier_insts {DTMF_INST/CORE_INST DTMF_INST/RAM_128x16 DTMF_INST/RESULTS_INST }`

Rail Analysis Commands

- [analyze_esd_network](#)
- [analyze_esd_voltage](#)
- [analyze_jitter](#)
- [analyze_package](#)
- [analyze_rail](#)
- [analyze_resistance](#)
- [analyze_signal_resistance](#)
- [analyze_voltage_regulator](#)
- [calculate_didt](#)
- [calculate_differential_voltage](#)
- [calculate_noise_margin](#)
- [create_current_region](#)
- [create_die_model](#)
- [create_hier_view](#)
- [create_power_pads](#)
- [create_power_pads_regions](#)
- [create_what_if_shape](#)
- [debug_irdrop](#)
- [extract_package](#)
- [map_die_package](#)
- [generate_esd_rlrp_report](#)
- [generate_power_up_report](#)
- [get_ir_insight](#)
- [get_voltus_insight_mode](#)
- [read_power_rail_results](#)
- [report_power_rail_results](#)
- [scale_what_if_capacitance](#)
- [scale_what_if_current](#)

- [scale_what_if_resistance](#)
- [set_advanced_package_options](#)
- [set_advanced_rail_options](#)
- [set_die_model](#)
- [set_dynamic_rail_simulation](#)
- [set_multi_die_analysis_mode](#)
- [set_net_group](#)
- [set_offchip_package_trace](#)
- [set_package](#)
- [set_pg_nets](#)
- [set_power_data](#)
- [set_power_pads](#)
- [set_rail_analysis_domain](#)
- [set_rail_analysis_mode](#)
- [set_voltus_insight_mode](#)
- [set_voltage_regulator_module](#)
- [view_dynamic_movie](#)
- [view_dynamic_waveform](#)
- [view_package_results](#)

analyze_esd_network

```
analyze_esd_network
[-help]
<run_name>
[-clamp_lowest_layer_taps {true | false}]
[-clamp_pin_short_file pin_shorting_file]
-config_file rule_filename
[-output directory]
[-remove_layers_below layer_name]
[-ta | -task_assistant]
-type {net | domain}
[-use_power_pad {true | false}]
```

Performs different types of effective resistance checks, such as bump to clamp, bump to bump, and clamp to clamp, and current density analysis during an ESD zap event. This command checks whether every power or ground bump has an ESD device, and the device is placed in such a way that it does not violate the effective resistance or current density limit.

The objective of ESD Checker is to easily identify the violations caused by high effective resistance or the current density above the specified threshold value.

Parameters

<code>-help</code>	Outputs the command usage and a brief description about the command parameters.
<code><run_name></code>	Mandatory parameter. Specifies the domain or net name. The result directory is prefixed with <code><run_name></code> .
<code>-clamp_lowest_layer_taps {true false}</code>	
	Specifies to keep only the lowest layer of clamp taps for effective resistance analysis. The default value is <code>false</code>. - When set to <code>true</code>, the ESD pin nodes on the lowest layer are shorted, and effective resistance checks are performed to the shorted node on the lowest layer. - When set to <code>false</code>, the ESD pin nodes on all layers are shorted, and effective resistance checks are performed to the shorted pin nodes of clamps on different layers.
<code>-clamp_pin_short_file pin_shorting_file</code>	
	Specifies the list of pin layers to be shorted for bump-to-clamp and clamp-to-clamp resistance checks. If multiple layers for the same pin are specified, then these layers are shorted with each other. The format of the file is: <pre>#Cell specification CELL <cell name> <pin> <layer> #Instance specification INST <instance name> <pin> <layer></pre> In this file, multiple layers for the same cell or instance can be specified by adding multiple rows for the same cell or instance name corresponding to each layer. If the <code>-clamp_pin_short_file</code> parameter is not specified, the software shorts all pin nodes of the lowest ESD cell pin layer, and performs effective resistance checks to the shorted node on the lowest layer. Note: If the <code>-clamp_pin_short_file</code> parameter is used in conjunction with <code>-clamp_lowest_layer_taps</code>, the <code>-clamp_pin_short_file</code> parameter will override the settings of <code>-clamp_lowest_layer_taps</code>.
<code>-config_file rule_filename</code>	
	Mandatory parameter. Specifies the name of the rule file that includes all the ESD checks to be performed as a set of rules. It also specifies the effective resistance threshold limit. For more information, refer to the ESD Rule Specification section in the "ESD Analysis" chapter of <i>Voltus User Guide</i> .
<code>-output directory</code>	
	Specifies the name of the output directory in which the ESD analysis results are saved.
<code>-remove_layers_below layer_name</code>	

	Specifies the layer name below which all layers are dropped from analysis. This parameter can be used in scenarios where the ESD cells connect to the PG grid at high layers, and therefore the impact of the lower PG layers is very small on effective resistance calculation and current density checks.
<code>-ta -task_assistant</code>	
	Opens ESD Analysis Task Assistant, a task-based quick help functionality for the ESD analysis flow. The information in the Task Assistant is organized by topics relevant to the task you are performing. It answers frequently asked questions, such as format of the rule file, use model of the command, or supported ESD checks.
<code>-type {net domain}</code>	
	Mandatory parameter. Specifies whether to run rail analysis on a domain or a net.
<code>-use_power_pad {true false}</code>	
	Specifies that the x,y locations of the bump cells are to be obtained from the power pad file. This parameter can be used to support designs for which bump cells are not visible. The default value is false.

Examples

- The following commands specify to run ESD analysis on the domain `top`:

```
set_rail_analysis_mode -method static -accuracy hd -power_grid_library ./techonly.cl -ict_em_models
./Data/ESD_em.ict
analyze_esd_network top -config_file ./esd_new_rule.txt -type domain -output ESD_analysis -
use_power_pad true
```

analyze_esd_voltage

```
analyze_esd_voltage
[-help]
-current_distribution_layer layer
-driver_receiver_file filename
-esd_pin_location_file filename
[-detailed_report {true | false}]
[-maximum_violations value]
-net NetName
[-report filename]
[-threshold value]
-total_current value
```

Performs IR Drop analysis for every driver/receiver pair and reports voltage drop above the given limit. Use this command to provide the current injection points (sinks), the current distribution layer (source), and the total ESD current to be distributed. The command generates a report containing all the violations where potential difference across driver/receiver is greater than the given threshold.

Parameters

- help	Outputs the command usage and a brief description about the command parameters.
-current_distribution_layer <i>layer</i>	
	Specifies to distribute current to all nodes of the given layer. This is the LEF/DEF layer name. This is a required parameter.
-detailed_report {true false}	
	Specifies to report the differential voltage for each zap point above the specified threshold. It gives the x,y location of the zap point and the zap point name. Using this parameter, you can fix all the differential voltage violations in a single run. This parameter will generate the report file with the same name as specified using the <code>-report</code> parameter and with the <code>_detail.rpt</code> suffix. Default: false The following is an example of the parameter: <pre>analyze_esd_voltage -net VSS -driver_receiver_file ./driverdb_railVSS -esd_pin_location_file VSS.pp -current_distribution_layer METAL5 -total_current 8 -report FULL_new_gnd_esd_cdm.rpt - detailed_report true -threshold 2</pre>
-driver_receiver_file <i>filename</i>	

	Specifies the name of the driver/receiver file of the net. This file is created by the <code>-create_driver_db</code> parameter of the <code>set_power_analysis_mode</code>. This is a required parameter. The file has multiple columns, wherein the first column is the receiver, and the remaining columns are drivers to the receiver instance. The following is an example of the file content: <pre>padding/I_RCV_7 io_toplevel/padding/I_DECAP io_toplevel/padding/</pre> In this example: <code>padding/I_RCV_7</code> is the receiver instance. <code>io_toplevel/padding/I_DECAP</code> and <code>io_toplevel/padding/</code> are driver instances.
<code>-esd_pin_location_file filename</code>	
	Specifies the name of the ESD pin location file. This is a required parameter. The format of the file is: <pre>#Pin_name x coord y coord layer gnd_pin_1 2284.6 -2898.05 m4</pre>
<code>-maximum_violations value</code>	
	Specifies the number of max worst violations to be reported. This is an optional parameter. When this value is specified, only the specified number of top violations will be reported. If a value is not specified, voltage drop for all driver/receiver pairs will be reported.
<code>-net NetName</code>	
	Specifies the name of the net to be analyzed. This is a required parameter.
<code>-report filename</code>	
	Specifies the name of the output report file in which details of the driver-receiver violation pair is written. This is an optional parameter. The report is filtered in descending order of differential voltage. The output file has four columns: • First column - Differential voltage (V) • Second column - Driver name (node location(um)) • Third column - Receiver name (node location(um)) • Fourth column - ESD pin name (location(nm)) The following is the sample ESD file format: <pre>14.1324 pmc/a_ip_pmc_chassis_c55fg (2050.875 -2760.085 ap) io_toplevel/padding/I_ESDCLAMP_39 (1428.200 2884.250 m4) gnd_pin_22 (1458.200 2898.050 m4)</pre>
<code>-threshold value</code>	

	Specifies the minimum allowed driver/receiver voltage for filtering the report. This is an optional parameter. When a threshold value is specified, driver/receiver pairs with differential voltage greater than the specified threshold value will be reported. If a threshold value is not specified, voltage drop for all driver/receiver pairs will be reported.
<code>-total_current value</code>	
	Specifies the total ESD current to be distributed. The default current unit is amperes. This is a required parameter.

Example

- The following example performs static IR Drop analysis to check the voltage drop between all driver and receiver pairs:

```
analyze_esd_voltage -net gnd -driver_receiver_file ./POWER/driverdb_railgnd -esd_pin_location_file  
./epinfile -current_distribution_layer m1 -total_current 8 -report FULL_new_gnd_esd_cdm.rpt
```

analyze_jitter

```
analyze_jitter
[-help]
[-cycle_interval <integer>]
[-eiv_end_time time]
[-eiv_file file_path]
[-eiv_start_time time]
[-eiv_vdd_vss net_names]
[-enable_spectre_x {true | false}]
[-force_delete_output_dir]
[-from pin_name]
[-from_fall pin_name]
[-from_rise pin_name]
[-levels_combine value]
[-lsf_job_string job]
[-max_spectre_lic value]
[-mode {period cycle_to_cycle time_interval_error backward_compatible}]
[-model_file filename]
[-no_hard_stop_at_to_pins {true | false}]
[-outdir directory_name]
[-run_simulation]
[-skip_stage_fanout value1 value2]
[-spectre spectre_path]
[-spectre_cmd_options options]
[-subckt_file file_path]
[-through through_points]
[-to pin_name]
[ [ > | >> ] ]
```

Analyzes jitter for the user-defined clocks and calculates clock jitter for all the clock end points or user-defined end points. The `analyze_jitter` command profiles the cycle-to-cycle EIV values from rail analysis, applies the two worst-case cycle-to-cycle EIV values for two simulations, measures the delta delays for each of the clock tree cells, and computes the effective jitter at the end point.

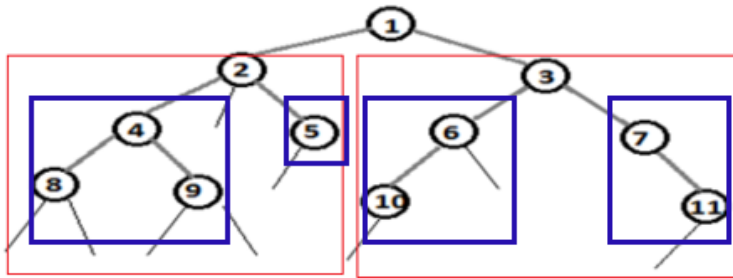
Parameters

-help	Outputs a brief description that includes type and default information for each <code>analyze_jitter</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man analyze_jitter</code> command.
-cycle_interval <integer>	

	Specifies the cycle interval to be considered while selecting the EIV values of each instance for jitter computation. The default value is 1, where all the EIV values are considered for jitter computation. When 'N' is specified, the <code>analyze_jitter</code> command considers only the EIV entries in step size of 'N', starting from the value specified by the <code>-start_cycle</code> parameter.
<code>-eiv_end_time time</code>	
	Specifies the end time of the EIV timing window.
<code>-eiv_file file_path</code>	
	Specifies the path to the effective instance voltage files for accounting IR Drop in jitter computation.
<code>-eiv_start_time time</code>	
	Specifies the start time of the EIV timing window.
<code>-eiv_vdd_vss net_names</code>	
	Specifies the power and ground net name for each EIV file provided in the <code>-eiv_file</code> parameter.
<code>-enable_spectre_x {true false}</code>	
	Enable spectreX license checkout when launching spectre.
<code>-force_delete_output_dir</code>	
	Completely remove the directory given by the user in the <i>Outdir</i> option.
<code>-from pin_name</code>	
	Creates the spice deck with mesh net starting from the specified from pin.
<code>-from_fall pin_name</code>	
	Creates the spice deck with mesh net starting from the falling transition of the specified from pin.
<code>-from_rise pin_name</code>	
	Creates the spice deck with mesh net starting from the rising transition of the specified from pin.
<code>-levels_combine value</code>	

Specifies the number of levels above the leaf node from which sub trees are to be created. This parameter specifies the level at which the sub trees are to be combined. The default value is 0, which indicates that a separate tree will be created for each leaf node. After determining the nodes to be combined together, the tree is broken into smaller trees, and jitter analysis on these smaller trees can be done in parallel.

Let us consider the following diagram:



Here,

For levels_combine = 0:

T1: 1, 2, 4, 8

T2: 1, 2, 4, 9

T3: 1, 2, 5

T4: 1, 3, 6, 10

T5: 1, 3, 7, 11

For levels_combine (blue box) = 1:

T1: 1, 2, 4, 8, 9

T2: 1, 2, 5

T3: 1, 3, 6, 10

T4: 1, 3, 7, 11

For levels_combine (red box) = 2:

T1: 1, 2, 4, 5, 8, 9

T2: 1, 3, 6, 7, 10, 11

`-lsf_job_string job`

Specifies a handle to the farm job for launching the simulation.

`-max_spectre_lic value`

Specifies the number of Spectre license available to run multiple Spectre instances simultaneously.

`-mode {period cycle_to_cycle time_interval_error backward_compatible}`

Runs the `analyze_jitter` command natively - with the specified mode.

Default: period

`-model_file filename`

	Specifies the model file to be included in generated Spice.
<code>-no_hard_stop_at_to_pins {true false}</code>	
	When set to <code>false</code> (default mode), the spice deck is generated only for traced paths starting from the <code>-from</code> pin and ending at the <code>-to</code> pin. It does not traverse through the fanout branches that do not lead to the <code>-to</code> specified pin. When set to <code>true</code>, traversal occurs through the whole fanout cone of the <code>-from</code> pin till it hits a clock pin of a flop, or any stop point specified with the <code>-to</code> parameter.
<code>-outdir <i>directory_name</i></code>	
	Specifies the name of the output directory that will contain the generated jitter analysis reports: • <code>summary.jitter</code> • <code>clock_branch1.jitter</code> • <code>clock_branch2.jitter</code>
<code>-run_simulation</code>	
	Enables path simulation in addition to generating the spice deck.
<code>-skip_stage_fanout <i>value1 value2</i></code>	
	Takes two positive integers in the format "N1 N2". When specified, scans up to the N1 stage in the fanin cone of each end point, and skips the segments where a stage is found to have fanout greater than N2.
<code>-spectre <i>spectre_path</i></code>	
	Specifies the location of the Spectre version that should be used for simulation. If the path is not specified with this option, the software will check if you have the Spectre location defined in your path.
<code>-spectre_cmd_options <i>options</i></code>	
	Specifies the command line options for Spectre.
<code>-subckt_file <i>file_path</i></code>	
	Specifies the path of the Spice sub-circuit file that contains the Spice information. This information is necessary to get accurate Spice information.
<code>-through <i>through_points</i></code>	
	Creates the spice deck with mesh net going by the <code>-through</code> points.
<code>-to <i>pin_name</i></code>	

	Creates the Spice deck with mesh net ending at the specified to pins.
> <i>filename</i>	
	Redirects the <code>-run_simulation</code> output to the specified file.
>> <i>filename</i>	
	Appends the <code>-run_simulation</code> output to the specified file.

Examples

- The following command creates jitter reports for the clock path starting from the `i_REC_CLK` and places the results directories in the `jitter_output` directory:

```
analyze_jitter -eiv_vdd_vss {VDD VSS} -eiv_file /IRDROP/PD_25C_dynamic_1/Reports/VDD_VSS.win_avg.rpt -
model_file ../../include.ss -subckt_file ../../worst.spi -spectre
/icd/flow/MMSIM/MMSIM161/ISR9/lrx86/tools.lrx86/bin/spectre -no_hard_stop_at_to_pins true -outdir
jitter_output -from i_REC_CLK -spectre_cmd_options {+aps +spice +mt=8 +postlayout +dcopt } -
lsf_job_string {/grid/sfi/farm/bin/bsub -q rnd -P VOLTUS:16.1:PV:regTest -W 140:00 -R "OSNAME==Linux
&& OSBIT==64 && (OSREL==EE60||OSREL==EE50)" -Is } -run_simulation -levels_combine 2 -max_spectre_lic
100
```

Related Topics

- "[Static Power, IRdrop and EM Analysis](#)" and "[Dynamic Power and IRDrop Analysis](#)" in the *Voltus User Guide*

analyze_package

```
analyze_package  
[-help] <domain>  
[-computer_resources string]  
[-output <directoryname>]  
-pkg_bga_name <bga_circuitname>  
-pkg_die_name <die_circuitname>  
-pkg_pin_current_file <filename>  
-result_name <filename>  
[-spd_file filename]  
-type <package_analysis_type>  
-workspace <workspace_name>
```

Specifies how the package analysis will be performed, and to run package analysis.

Parameters

<code>-help</code>	Outputs the command usage and a brief description about the command parameters.
<code>domain</code>	Specifies the rail analysis domain name.
<code>-computer_resources string</code>	
	Specifies the computer resources for the package analysis run. This parameter allows you to customize the host distribution/CPU usage for package analysis. The default setting is <code>"-smt 0 -local -cn localhost -cpus 8"</code> .
<code>-output <directoryname></code>	
	Specifies the output directory name of the package analysis results. By default, the package model will be created in the specified workspace.
<code>-pkg_bga_name <bga_circuitname></code>	
	Defines the ball grid array (BGA) circuit name in the package model.
<code>-pkg_die_name <die_circuitname></code>	
	Defines the die circuit name in the package model.
<code>-pkg_pin_current_file <filename></code>	
	Specifies the name of the generated pin current file from rail analysis that contains the package pin names and the current values. It is automatically generated after the package model is created.
<code>-result_name <filename></code>	
	Specifies the name of the .xml result file.
<code>-spd_file filename</code>	
	Specifies an existing Sigrity .spd file for package analysis. This parameter allows you to specify a .spd file instead of the workspace name (package model database) of the package analysis (PowerDC) tool.
<code>-type <package_analysis_type></code>	
	Specifies the type of package analysis. Currently, the software supports only DC package analysis.
<code>-workspace <workspace_name></code>	
	Specifies the workspace name for package analysis (powerDC). The file name extension is .pdcx.

Example

- The following example specifies the package analysis details and starts package analysis:

```
analyze_package ALL \  
-type dc \  
-workspace ../../sigrity/rak_pdc_irdrop/test.pdcx \  
-pkg_bga_name FPBGA144_Signal02 \  
-pkg_die_name CD1_Signal01 \  
-output rak_pdc_irdrop_voltus_cpf \  
-result_name rak_pdc_irdrop_voltus \  
-pkg_pin_current_file ../../spa_cpf/ALL_125C_avg_1/Reports/super_filter_pkg_iv.rpt
```

analyze_rail

```
analyze_rail
-help
[-type {domain | net}]
[-output directory_name]
name
```

Runs rail analysis on a net or domain. This command must be specified after defining the rail analysis mode properties using the `set_rail_analysis_mode` command.

Parameters

help	Prints out the command usage
<i>name</i>	Specifies the name of the domain or net that you want to analyze.
-output <i>directory_name</i>	
	Specifies the name of the directory in which to store the output directories created during the analysis. <i>Default:</i> Current working directory.
-type {domain net}	
	Specifies whether to run rail analysis on a domain or a net. A domain is a list of power or ground nets to analyze together. See set_power_pads for more details. You can specify the variable "ALL" to analyze all domains.

Examples

- The following command performs an analysis for the vdd net and places the results directories in the analysis_out directory:
`analyze_rail -type net -output analysis_out vdd`
- The next command performs rail analysis on TDSP power domain and writes the results in current working directory.
`analyze_rail -type domain -output ./ TDSP`
- The following command performs rail analysis on all power domains and writes the results in the current working directory:
`analyze_rail -type domain ALL -output ./`

Related Topics

- "[Static Power, IRdrop and EM Analysis](#)" and "[Dynamic Power and IRDrop Analysis](#)" in the *Voltus User Guide*

analyze_resistance

```
analyze_resistance
[-help]
[-cell master_cell_name]
[-die_mode {3dic}]
[-domain_threshold value]
[-exclude_list_file filename]
[-force_reextraction {true | false}]
[-instance_list {instance1 pin1 instance2 pin2...}]
[-instance_list_file filename]
[-instance_pair_list {instance1 instance2 pin1 pin2 ...}]
[-instance_pair_list_file filename]
-net netName | -domain domainName
[-net_threshold value]
[-node_list {{nodeX nodeY METAL_LAYER [LABEL]}}...}]
[-node_list_file filename]
[-node_pair_list {node1X node1Y METAL_LAYER1 node2X node2Y METAL_LAYER2...LABEL}]
[-node_pair_list_file filename]
[-output filename]
[-output_dir directory_name]
[-print_cell_name]
[-region {x1 y1 x2 y2 layer number_of_nodes region_name}]
[-report_limit value]
[-threshold value]
```

Performs effective resistance analysis. Effective resistance computes effective resistance between two nodes on the power-grid, or from a node to all the voltage sources in the design, or from an instance to all the voltage sources in the design.

You can run this command without performing power calculation and IRdrop analysis. The flow requires you to load the design, set power nets, set power pads, and run resistance analysis. This command enables the tool to consider the entire power structure, including embedded ring structure within IO cells if any.

Note: The `analyze_resistance` full chip analysis supports running the `set_rail_analysis_mode -enable_rlrp_analysis` command to perform least resistance path analysis (RLRP) in conjunction with effective resistance analysis (REFF) in the DP mode. However, RLRP and REFF analysis should be done separately in the XP mode. When performing the point-to-point effective resistance analysis (`analyze_resistance`) for the power/signal nets in the XP mode, disable the RLRP analysis (`set_rail_analysis_mode -enable_rlrp_analysis false`).

Parameters

- help	Outputs a brief description that includes the type and default information for each <code>analyze_resistance</code> parameter. For a detailed description of the command and all of its parameters, use the man command <code>man analyze_resistance</code>.
-----------	--

<code>-cell master_cell_name</code>	
	Reports effective resistance for all instances of the specified master cell name. The <code>-cell</code> parameter improves the usability of effective resistance calculation by allowing you to specify only the cell name of the instances you want to analyze. You can specify multiple cells for performing effective resistance analysis of multiple cells simultaneously.
<code>-die_mode {3dic}</code>	
	Specifies to perform multi-die effective resistance analysis. This parameter must be specified with the <code>set_rail_anlysis_mode -die_mode 3dic</code> command.
<code>-domain domainName</code>	
	Specifies the domain name that is to be analyzed.
<code>-domain_threshold value</code>	
	Specifies the minimum allowed effective resistance (effective resistance sum of each net) to instance. For specific instance list analysis, the instances below the threshold value will be reported as "PASS", while the instances above the threshold value will be reported as "FAIL". For full chip analysis, the instances below the threshold value will not be reported in the domain report. The default value is 0.
<code>-exclude_list_file filename</code>	
	Specifies the name of a file containing a list of cells or instances that will not be processed during the resistance analysis flow. The specified cells/instances will be ignored from report generation. The use model of this parameter is: <pre>analyze_resistance -exclude_list_file <file_name></pre> File Format: <pre><CELL INST> <name></pre> Example: <pre>CELL BUMP_MASTER CELL ESD_MASTER INST INSTA</pre>
<code>-force_reextraction {true false}</code>	
	Specifies to force a re-extract of resistances.
<code>-instance_list {instance1 pin1 instance2 pin2...}</code>	
	Specifies the list of instances for which effective resistance is to be calculated. <i>pin</i> is an optional argument. By default, the tool will use the instance pin associated with the net being analyzed. The <i>pin</i> argument can be used in special cases when two different pins of an instance are connected to the same net. Note: The <code>-instance_list</code> and <code>-instance_list_file</code> parameters are mutually exclusive to the <code>-instance_pair_list</code> and <code>-instance_pair_list_file</code> parameters. This means that you can specify <code>-instance_list</code> and <code>-instance_list_file</code> together but you cannot specify <code>-instance_list</code> and <code>-instance_list_pair</code> together.
<code>-instance_list_file filename</code>	

	Specifies the name of the instance list file. This file contains the list of instances in the same format as mentioned for the <code>-instance_list</code> parameter. The file allows you to specify a large number of instances. Note: The <code>-instance_list</code> and <code>-instance_list_file</code> parameters are mutually exclusive to the <code>-instance_pair_list</code> and <code>-instance_pair_list_file</code> parameters. This means that you can specify <code>-instance_list</code> and <code>-instance_list_file</code> together but you cannot specify <code>-instance_list</code> and <code>-instance_list_pair</code> together.
<code>-instance_pair_list {instance1 instance2 pin1 pin2 ... }</code>	
	Specifies the instance pair list to enable effective resistance calculation between two instances in the design. <code>pin1</code> and <code>pin2</code> are pins of <code>instance1</code> and <code>instance2</code>, respectively. Note: The <code>-instance_list</code> and <code>-instance_list_file</code> parameters are mutually exclusive to the <code>-instance_pair_list</code> and <code>-instance_pair_list_file</code> parameters. This means that you can specify <code>-instance_list</code> and <code>-instance_list_file</code> together but you cannot specify <code>-instance_list</code> and <code>-instance_list_pair</code> together.
<code>-instance_pair_list_file filename</code>	
	Specifies the name of the instance pair list file. This file contains the instance pair list in the same format as mentioned for the <code>-instance_pair_list</code> parameter. The file allows you to specify a large number of instance pairs. <code>-instance_pair_list_file</code> and <code>-instance_pair_list</code> are mutually exclusive. Note: The <code>-instance_list</code> and <code>-instance_list_file</code> parameters are mutually exclusive to the <code>-instance_pair_list</code> and <code>-instance_pair_list_file</code> parameters. This means that you can specify <code>-instance_list</code> and <code>-instance_list_file</code> together but you cannot specify <code>-instance_list</code> and <code>-instance_list_pair</code> together.
<code>-net netName</code>	
	Specifies the net name that is to be analyzed.
<code>-net_threshold value</code>	

The effective resistance requirements for different power nets vary depending on the design. For example, an array or periphery supply to an SRAM needs to be signed off at a lower effective resistance threshold compared to the core supplies. This parameter allows you to specify different threshold values for different nets.

The use model of this parameter is given below:

```
set_rail_analysis_domain -name ALL -pwrnets {vdd vddar vdda vdd1 vdd2} -gndnets vss
analyze_resistance -threshold 50 -net_threshold {{vdd 50} {vddar 20} {vdda 35} {vss 40}} -domain
ALL -output_dir Reffective
```

In this example, the threshold value for different nets are:

```
vdd -> 50
vddar -> 20
vdda -> 35
vss -> 40
vdd1 -> 50
vdd2 -> 50
```

Here, for all the remaining nets for which a value is not specified using the `-net_threshold` parameter, the value specified with the `-threshold` parameter is applied.

The different threshold values will have an impact on the list of instances reported in the `<state_dir>/Reports/<net>/<net>.effr` report.

```
-node_list {{nodeX nodeY METAL_LAYER [LABEL]}}...
```

Specifies the list of user-defined nodes for effective resistance calculation between the specified user-defined nodes and all voltage sources. You can add a list of nodes based on the region on a given layer. `LABEL` is any name chosen by you to identify node entries in the list. This is optional but helps during debugging.

Note: The `-node_list` and `-node_list_file` parameters are mutually exclusive to the `-node_pair_list` and `-node_pair_list_file` parameters. This means that you can specify `-node_list` and `-node_list_file` together but you cannot specify `-node_list` and `-node_pair_list` together.

```
-node_list_file filename
```

Specifies the name of the node list file. This file contains the list of user-defined nodes in the same format as mentioned for the `-node_list` parameter. The file allows you to specify a large number of nodes.

You can specify multiple nets for node list based resistance analysis. The use model for multiple nets is:
`analyze_resistance -domain <domain name> -node_list_file <filename>`

Here,

- the `-domain` parameter will specify the domain name, that is, the domain with nets that is required to be analyzed.
- the `-node_list_file` parameter will include the multiple net names

The format for specifying multiple nets is:

```
NET <netname1>
node1X node1Y METAL_LAYER [LABEL]
node2X node2Y METAL_LAYER [LABEL]

NET <netname2>
node1X node1Y METAL_LAYER [LABEL]
node2X node2Y METAL_LAYER [LABEL]
```

Note: The `-node_list` and `-node_list_file` parameters are mutually exclusive to the `-node_pair_list` and `-node_pair_list_file` parameters. This means that you can specify `-node_list` and `-node_list_file` together but you cannot specify `-node_list` and `-node_pair_list` together.

```
-node_pair_list {node1X node1Y METAL_LAYER1 node2X node2Y METAL_LAYER2...LABEL}
```

Specifies the node pair list for effective resistance calculation between two nodes on the power-grid. You can add a list of nodes based on the region on a given layer. LABEL is any name chosen by you to identify node entries in the list. This is optional but helps during debugging.

Note: The `-node_list` and `-node_list_file` parameters are mutually exclusive to the `-node_pair_list` and `-node_pair_list_file` parameters. This means that you can specify `-node_list` and `-node_list_file` together but you cannot specify `-node_list` and `-node_pair_list` together.

```
-node_pair_list_file filename
```

	Specifies the name of the node pair list file. This file contains the node pair list in the same format as mentioned for the <code>-node_pair_list</code> parameter. The file allows you to specify a large number of node pairs. You can specify multiple nets for node pair list based resistance analysis. The use model for multiple nets is: <code>analyze_resistance -domain <domain name> -node_pair_list_file <filename></code> Here, - the <code>-domain</code> parameter will specify the domain name, that is, the domain with nets that is required to be analyzed. - the <code>-node_pair_list_file</code> parameter will include the multiple net names The format for specifying multiple nets is: <pre>NET <netname1> node1X node1Y METAL_LAYER1 node2X node2Y METAL_LAYER2 [LABEL] node3X node3Y METAL_LAYER3 node4X node4Y METAL_LAYER4 [LABEL] NET <netname2> node1X node1Y METAL_LAYER1 node2X node2Y METAL_LAYER2 [LABEL] node3X node3Y METAL_LAYER1 node4X node4Y METAL_LAYER4 [LABEL]</pre> Note: The <code>-node_list</code> and <code>-node_list_file</code> parameters are mutually exclusive to the <code>-node_pair_list</code> and <code>-node_pair_list_file</code> parameters. This means that you can specify <code>-node_list</code> and <code>-node_list_file</code> together but you cannot specify <code>-node_list</code> and <code>-node_pair_list</code> together.
	<code>-output filename</code>
	Specifies the name of the output report file that contains the report header and the sorted list for resistance values. Note: This parameter should not be specified in the XP flow as the XP reports are consolidated under the state directory by default.
	<code>-output_dir directory_name</code>
	Specifies the name of the directory in which the output report file is saved.
	<code>-print_cell_name</code>
	Specifies to print cell names in the effective resistance report. This parameter allows you to sort resistance values by cell names.
	<code>-region {x1 y1 x2 y2 layer number_of_nodes region_name}</code>
	Analyzes resistance in the specified region. You can draw a box on a specific layer, and specify the number of nodes to be selected (maximum limit of 100 nodes). This parameter allows you to compute effective resistance on a given layer across the chip and determine how the resistance is changing for the design from top to bottom layer. You can assign a name for the region.
	<code>-report_limit value</code>
	Specifies the number of instances printed in the report to save disk space.
	<code>-threshold value</code>

	Specifies the minimum allowed effective resistance to display node/node-pair/instance/instance-pair that have a resistance value above this threshold value. If you specify threshold as a non zero value, then instances with resistance value above that threshold will be tagged as FAIL and the rest will be tagged as PASS.
--	--

Examples

- The following command performs effective resistance calculation for the list of nodes and instances:

```
analyze_resistance -net VSS -node_list { 231 1061.6 M1 } -instance_list_file instance.list
```
- The following command specifying a list of user-defined nodes and a node list file for effective resistance calculation:

```
analyze_resistance -net VDD -node_list {{ 213.556 133.469 M1 I123 }} -node_list_file  

../node_list.txt.1 -output reff_rpt
```
- The following is an example of specifying a list of instances for effective resistance calculation:

```
analyze_resistance -instance_list {{INV11 VDD} {INV21 VDD}} -net VDD
```
- The following is an example of specifying a list of user-defined nodes for effective resistance calculation:

```
analyze_resistance -net VDD -node_list {{800 11.88 M4} {999.768 11.88 M4} {1000.296 11.880 M4}} -  

output out.txt
```
- The following is an example of specifying node pair list for effective resistance calculation:

```
analyze_resistance -net VDD -node_pair_list {195.728 133.369 M1 213.556 133.469 M1} -output  

out_nodelist.txt
```
- The following is an example of specifying instance pair list for effective resistance calculation:

```
analyze_resistance -net GND -instance_pair_list {inst1 inst2 GND GND} -output outfile -threshold 100
```
- The following command performs effective resistance calculation for full chip analysis:

```
analyze_resistance -threshold 13.0 -domain_threshold 25.0 -domain ALL
```

Sample Report:

```
NET: VSS VDD
# Threshold: 25.000
# Total Effective resistance for instances sorted from high to low
# PASS/FAIL      REFF  INSTANCE      PIN...
FAIL            25.664  clk__L1_I1      VDD (51.52%)    VSS (48.48%)
FAIL            25.35   clk__L2_I1      VDD (52.16%)    VSS (47.84%)
FAIL            25.037  clk__L3_I1      VDD (51.56%)    VSS (48.44%)
```

- The following command performs effective resistance calculation for instance list analysis:

```
analyze_resistance -domain ALL -threshold 13.0 -domain_threshold 25.0 -instance_list_file inst.list
```

Sample Report:

```
NET: VSS VDD
# Threshold: 25.000
# Total Effective resistance for instances sorted from high to low
# PASS/FAIL      REFF  INSTANCE      PIN...
FAIL            25.664  clk__L1_I1      VDD (51.52%)    VSS (48.48%)
FAIL            25.35   clk__L2_I1      VDD (52.16%)    VSS (47.84%)
FAIL            25.037  clk__L3_I1      VDD (51.56%)    VSS (48.44%)
PASS            24.724  clk__L4_I1      VDD (52.21%)    VSS (47.79%)
PASS            23.047  inst_pll        VDD (50.55%)    VSS (49.45%)
```

- The following command reports reff for all instances of the cell type CKBD8:

```
analyze_resistance -net VDD -cell CKBD8
```

The following is a sample reff report generated using the `-cell` parameter:

```
*****

# Layer Representation: LEF
# Coordinate Unit: um
# Resistance Unit: Ohm
NET: VDD

# Threshold:      0.000
# Effective resistance for instances sorted from low to high

# PASS/FAIL      REFF      INSTANCE      PIN
-                3.5752    add_94_37_I2/g695    VDD
-                3.7521    add_94_37_I2/g694    VDD
```

Related Topics

- ["Static Power, IRdrop and EM Analysis"](#) in *Voltus User Guide*

analyze_signal_resistance

```
analyze_signal_resistance
[-help]
[-instance_pair_list_file filename]
-net net_name
[-output_dir directory]
[-report filename]
[-threshold value]
```

Computes effective resistance for signal nets from a given instance1/pin1 to instance2/pin2 accounting all applicable manufacturing effects. You can use the effective resistance output file to determine whether the critical routes are within the specified resistance limits.

Parameters

- help	Outputs the command usage and a brief description about the command parameters.
-instance_pair_list_file filename	
	Specifies the file name containing the signal net instance-pin pairs for which effective resistance needs to be computed. The format of the file is: INST1 PIN1 INST2 PIN2
-net net_name	
	Specifies the name of the signal net.
-output_dir directory	
	Specifies the name of the output directory.
-report filename	
	Specifies the name of the report file in which details of effective resistance between instance-pin pairs are written.
-threshold value	
	Specifies the minimum allowed effective resistance between instance-pin pairs.

Examples

- The following command computes effective resistance for the signal net `n_13190`:

```
analyze_signal_resistance -net n_13190
```
- The following command computes effective resistance for the signal net `n_13190`, and writes the effective resistance details in the report `n_13190.rpt`:

```
analyze_signal_resistance -net n_13190 -report n_13190.rpt
```
- The following command computes effective resistance for signal net instance-pin pairs specified in the file `instance.list`:

```
analyze_signal_resistance -net n_0 -instance_pair_list_file instance.list
```

analyze_voltage_regulator

```
analyze_voltage_regulator
name
[-results_directory directory_name]
-type {domain | net}
```

Runs low dropout (LDO) analysis on a net or domain. This command must be specified after defining the rail analysis mode properties using the `set_rail_analysis_mode` command.

Parameters

name	Specifies the name of the domain or net that you want to analyze.
-results_directory directory_name	
	Specifies the name of the directory in which to store the output directories created during the analysis. Default: Current working directory.
-type {domain net}	
	Specifies whether to run LDO analysis on a domain or a net. A domain is a list of power or ground nets to analyze together. You can specify the variable "ALL" to analyze all domains.

Examples

- The following command performs an analysis for the vdd net and places the results directories in the analysis_out directory:
`analyze_voltage_regulator -type net -results_directory analysis_out vdd`
- The next command performs rail analysis on TDSP power domain and writes the results in current working directory.
`analyze_voltage_regulator -type domain -results_directory ./ TDSP`

Related Topics

- ["Static Power, IRdrop and EM Analysis"](#) and ["Dynamic Power and IRDrop Analysis"](#) in the *Voltus User Guide*

calculate_didt

```
calculate_didt
[-help]
-net_name net_name
-nworst value
-state_directory dir
[-window_steps value | -window_size value ]
```

Generates the on-die di/dt information from an existing dynamic rail analysis run and saves it in a text file. This command allows you to determine the worst N di/dt events for each bump and for the sum of all bumps (vc_sum).

Parameters

- help	Prints a brief description that includes the type and default information for each <code>calculate_didt</code> parameter. For a detailed description of the command and all of its parameters, use the man command: <code>man calculate_didt</code>
<code>-net_name <i>net_name</i></code>	
	Specifies a net for which the di/dt report is to be generated.
<code>-nworst <i>value</i></code>	
	Specifies the number of worst windows with the largest di/dt values to be reported per bump.
<code>-state_directory <i>dir</i></code>	
	Specifies the name of the state directory where the information on the bump-based di/dt information from the previous run resides.
<code>-window_steps <i>value</i></code>	
	Specifies the transient time step size or resolution of the sliding window. The default value is 5.
<code>-window_size <i>value</i></code>	
	Specifies the duration of the sliding window in seconds. Note: The <code>-window_size</code> and <code>-window_steps</code> parameters are mutually exclusive. When both the parameters are specified, the <code>-window_size</code> parameter takes precedence. If you do not specify either of the two parameters, the results will be based on a default of 5 resolution steps.

Examples

- The following command generates the on-die di/dt information:

```
calculate_didt -nworst 3 -window_steps 10 -net_name VDD10 -state_directory  
./dynamic_rail/core_125C_dynamic_1
```

Related Topics

- "[Static Power, IRdrop and EM Analysis](#)" in the *Voltus User Guide*

calculate_differential_voltage

```
calculate_differential_voltage
[-help]
-diffv_limit value
-power_directory directory_name
-state_directory directory_name
```

Determines the differential voltage between a receiver/driver pair during ramp-up analysis. This command can be used to mitigate the risk of high short-circuit current for a driver-receiver instance pair due to slow/fast (differential) ramp-up voltage at the output of the power-switch cell connected to the driver/receiver cell.

The `calculate_differential_voltage` command reports the differential potential for those receiver/driver pairs that are greater than the specified differential voltage threshold, and the time interval at which the differential voltage remains greater than the threshold. You must specify the `set_power_analysis_mode -create_driver_db true` parameter before specifying this command.

The following is the format of the output report:

```
# <driverName>    <receiverName>    <peak_diffv>    <minT>    <maxT>
```

Parameters

- help	Outputs the command usage and a brief description about the command parameters.
-diffv_limit value	
	Specifies to report differential voltage for the receiver/driver pairs that exceed the specified threshold limit.
-power_directory directory_name	
	Specifies the path to the power directory to access the driver-receiver pairs.
-state_directory directory_name	
	Specifies the path to the state directory generated by rail analysis to access the EIV or timing database.

Examples

- The following command lists all the driver-receiver pairs that exceed the differential voltage limit of 0.001:

```
calculate_differential_voltage -diffv_limit 0.001 -state_directory dynamic_rail/core_105C_dynamic_1 -  
power_directory dynamic_power
```

The following is a snippet of the generated report:

#	<driverName>	<receiverName>	<peak_diffv>	<minT>	<maxT>
	column/MSV_I2	column/MULT8_reg_1_4	0.7382	5e-11	2.75e-09
	column/HIER_INST	column/MULT8_reg_1_4	0.7382	5e-11	2.75e-09
	...				

Related Topics

- "[Static Power, IRdrop and EM Analysis](#)" and "[Dynamic Power and IRDrop Analysis](#)" in the *Voltus User Guide*

calculate_noise_margin

```
calculate_noise_margin
[-help]
-noise_limit_file input_file
-power_directory dir
-state_directory dir
```

Specifies to calculate noise margins using the Effective Instance Voltage (EIV) method, that is, calculate noise margins with worst voltages seen on the driver-receiver pins. You need to specify the `set_power_analysis_mode - create_driver_db true` parameter to generate the driver-receiver database.

To run this command, you need to specify the default or cell-specific noise limit, signal noise, guardband, rail directory, and power directory.

The command generates the noise margin output file in the state directory. The format of the EIV-based noise margin output file (`powernetname_groundnetname.noise_margin`) is:

Margin_High(V)	Margin_Low(V)	N M H (V)	N M L (V)	V o h (V)	V o l (V)	V i h (V)	V i l (V)	S i g n a l _ N o i s e (V)	G u a r d b a n d (V)	D r i v e r p i n	R x I n s t p i n
NMH - signal noise - guardband	NMI - signal noise - guardband	V o h - V i h	V i l - V o l	V D D _ d r i v e r - V S S _ r e c e i v e r	V S S _ d r i v e r - V S S _ r e c e i v e r	(l - n o i s e _ l i m i t * (V D D _ r e c e i v e r - V S S _ r e c e i v e	n o i s e _ l i m i t * (V D D _ r e c e i v e r - V S S _ r e c e i v e	S i g n a l _ n o i s e _ v a l u e s _ s p e c i f i e d i n t h e n o i s e l i m i t f i l e	G u a r d b a n d v a l u e s _ s p e c i f i e d i n t h e n o i s e l i m i t f i l e	D r i v e r p i n n a m e	R e c e i v e r p i n n a m e

er) noise limit is the noise limit value specified in the noise limit file

Parameters

- help	Prints a brief description that includes the type and default information for each <code>calculate_noise_margin</code> parameter. For a detailed description of the command and all of its parameters, use the man command: <code>man calculate_noise_margin</code>
<code>-noise_limit_file input_file</code>	

	Specifies the name of the input noise limit file that contains the default or cell-specific noise limit, signal noise, and guardband. The format of the noise limit file is: <pre>default_noise_limit <value> default_signal_noise <value> default_guard_band <value> <cell_name> <noise_limit> <signal_noise> <guard_band></pre> The following is a snippet of the noise limit file: <pre>default_noise_limit 0.46 default_signal_noise 0.1 default_guard_band 0.05</pre>
<code>-power_directory dir</code>	
	Specifies the path to the power directory to access driver-receiver pairs.
<code>-state_directory dir</code>	
	Specifies the path to the state directory generated by rail analysis to access the EIV or timing database.

Examples

- The following command calculates noise margin:

```
calculate_noise_margin -noise_limit_file ./noiselimit -state_directory
dynamic_rail/core_105C_dynamic_1 -power_directory dynamic_power
```

Related Topics

- "[Static Power, IRdrop and EM Analysis](#)" in the *Voltus User Guide*

create_current_region

```
create_current_region
[-help]
-reset | {-region x1 y1 x2 y2
[-current value | -current_pwl_file filename]
[-die_instance_name dieinstname]
[-file filename]
[-label name]
[-layer layername]
[-intrinsic_cap value]
[-series_resistance value]
[-loading_cap value]
[-on_interdie_bump {true | false}]
[-net net_name]
[-polygon {x1 y1 x2 y2 x3 y3 ..x1 y1}]
[-pwrnet net_name]
[-gndnet net_name]
}
```

Specifies static or dynamic current regions on global power-grids over the placed macro to accurately capture high and low current consuming regions during rail analysis.

You can use this command prior to rail analysis to specify current regions or power distribution over the top of power-grid on a user-defined layer. These current regions are generally specified for a placed macro for which power-grid view library is not available, and power/current distribution inside this macro could possibly impact IRdrop in neighboring regions. These current regions can also be defined over top-level power-grid for an unplaced macro during early stages of power-planning.

 When you specify PWL currents over a region that includes instances, the current is distributed on nodes in the specified layer and not to each instance. The number of nodes to which current is distributed are determined by the tool so as to preserve the shape of the waveform.

Parameters

-current value	
	In the static mode (set_rail_analysis_mode -method static), the parameter specifies a single current value in mAmp. In the dynamic mode (set_rail_analysis_mode -method dynamic) the parameter specifies a PWL current waveform in time (in ns) and current (in mA) format. For example, 0ns 0mA 0.5ns 1mA 1ns 2mA 3ns 1mA 4ns 0.
-current_pwl_file filename	

	Specifies a PWL current file. The file has two columns, time (in ns) and current value (in mA), as shown below: <pre>time1 value1 time2 value2</pre> The <code>-current_pwl_file</code> and <code>-current</code> parameters are mutually exclusive.
<code>-die_instance_name dieinstname</code>	
	Specifies the name of the die to which you need to apply a current region. This parameter is only applicable to the multi-die analysis flow (<code>set_rail_analysis_mode -die_mode multi-die</code>). The parameter allows you to accurately assign current to power and ground nets in specific geometric regions for individual dies during rail analysis in 3D-IC designs.
<code>-file filename</code>	
	Specifies the name of a file containing the list of current regions for each die. <code>-file</code> can be used only with the <code>-die_instance_name</code> parameter. The other parameters of the <code>create_current_region</code> command can be included within the specified file. Example: <pre>create_current_region -file create_current_region_die0.tcl -die_instance_name Die0 create_current_region -file create_current_region_die1.tcl -die_instance_name Die1</pre> ###File content in <code>create_current_region_die0.tcl</code> <pre>-net VDD -current 50mA -layer M2 -region "10 10 100 100" -net VDD -current 50mA -layer M2 -region "100 100 200 200" -net VSS -current 100mA -layer M1 -region "10 10 200 200"</pre> ###File content in <code>create_current_region_die1.tcl</code> <pre>-net VDD -current 150mA -layer M2 -region "10 10 50 50" -net VDD -current 50mA -layer M2 -region "100 100 200 200" -net VSS -current 200mA -layer M1 -region "10 10 200 200"</pre>
<code>-help</code>	Outputs a brief description that includes the type and default information for each <code>create_current_region</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command <code>man create_current_region</code>.
<code>-gndnet net_name</code>	
	Specifies a ground net name on which the specified current is to be applied.
<code>-intrinsic_cap value</code>	
	Specifies the cell intrinsic capacitance in pico farads (pf). This is required parameter when set in the dynamic mode (<code>set_rail_analysis_mode -method dynamic</code>). This parameter is disabled in static mode.
<code>-label name</code>	
	Specifies a name to a specific current region. The parameter allows to assign a user-specific name to the region, instead of using the default name generated by the tool. By assigning a unique name, you can easily identify and manage the regions. For instance, instead of having generic names like "Region 1," using specific names like "VDD_area1" or "VSS_area2" can provide immediate context.
<code>-layer layername</code>	

	Specifies a layer name on which the scaling will be performed. Multiple commands can be issued to scale resistance on multiple layers. This is a required parameter.
<code>-loading_cap value</code>	
	Specifies the cell loading capacitance in pico farads (pf). This is required parameter when set in the dynamic mode (<code>set_rail_analysis_mode -method dynamic</code>). This parameter is disabled in static mode.
<code>-net net_name</code>	
	Specifies a net name on which the specified current is to be applied.
<code>-on_interdie_bump {true false}</code>	
	Specifies to create the tap nodes only on non-vsrc PLOC nodes. When the option is set to <code>true</code>, the number of total tap changes to include the tap nodes only on non-vsrc PLOC. You can check the change in total taps in the “Grid Statistics of Net XXX” section of the log file at <code><state directory>/logs/design_report/dmg/ext_report.log</code>
<code>-polygon {x1 y1 x2 y2 x3 y3 ..x1 y1}</code>	
	Specifies a rectilinear current region where the current will be distributed within. The rectilinear region enables you to specify multiple x,y points to add current regions in the areas that are not rectangular in shape. When specified, the last xy coordinate location will be the same as the first xy location. The xy coordinates will be in the counterclockwise sequence.
<code>-pwrnet net_name</code>	
	Specifies a power net name on which the specified current is to be applied.
<code>-region x1 y1 x2 y2</code>	
	Specifies the region on the layer across which the specified current and capacitance (in dynamic mode) is distributed by the rail analysis engine. The units of x and y should be in microns. This is a required parameter.
<code>-reset</code>	Resets all the previous <code>create_current_region</code> command settings.
<code>-series_resistance value</code>	
	Specifies the series resistance (ESR) value attached to each tap point in the current region definition.

Examples

- The following command specifies static current region:

```
create_current_region -current 2mA -layer METAL_1 -region "100 100 200 200"
```
- The following command specifies dynamic current region:

```
create_current_region -current {0ns 0mA 0.5ns 1mA 1ns 2mA 3ns 1mA 4ns 0} -layer METAL_1 -region "200 200 400 400" -intrinsic_cap 40 -loading_cap 20
```

- The following command resets all `create_current_region` parameters to their default values:

```
create_current_region -reset
```

- The following command specifies rectilinear current regions for die instance `Die1`:

```
create_current_region -net VDD -current 50mA -layer Metal5 -polygon "10 10 100 10 100 100 10 100 10 10  
" -die_instance_name Die1
```

```
create_current_region -net VDD -current 50mA -layer Metal5 -polygon "100 100 200 100 200 200 100 200  
100 100" -die_instance_name Die1
```

- The following command specifies the static current region with the region name `VDD_area1`:

```
create_current_region -label VDD_area1 -region "0.0 0.0 87.3 107.2" -layer M1 -current 1 mA -net VDD
```

Related Topics

- "[What-If Rail Analysis](#)" in the *Voltus User Guide*

create_die_model

```
create_die_model
[-accuracy {xd | hd }]
[-disable_isrc_node_zero { true|false }]
[-disable_global_node_zero { true|false }]
[-effective_rc]
[-effective_rc_deck]
[-enable_package_pin_map { true|false }]
[-enable_reference_node]
[-enable_xp {true | false}]
[-generate_sipi_port_definition_file {true | false}]
[-grid_reduction_cut_layer layer_name]-help
[-ignore_port_grouping_conflict]
[-local_reference_ground {true | false}]
[-max_ports_per_net integer ]
[-multi_die {true | false}]
[-output_directory directory_name]
[-state_directory directory_name]
[-port_grouping_by_ploc_files { true|false }]
[-port_grouping_file filename]
[-port_grouping_tiles_file filename]
[-probing_node_file file_name]
[-probing_region_file filename]
[-rail_analysis_domain domain_name | -net net_name ]
[-repeat time ]
[-suppress_negative_res { true|false }]
[-temp_directory_name directory]
[-use_sigrity_repeat]
```

Specifies to generate a reduced n-port die model in the form of equivalent die parasitic and current profile after rail analysis to perform system-level analysis with extracted package and board models. Resonance and electro-magnetic interference like system-level analyses can be performed using this die-model in package designing software like Allegro Package Design and Sigrity.

 This command requires a VTS-XL license.

Parameters

```
-accuracy {xd | hd }
```

	Specifies the accuracy mode for die-model generation. <i>Default:</i> <code>hd</code> • <code>xd</code>: <code>xd</code> or accelerated definition accuracy mode is used for faster die model generation with slight reduction in RC accuracy. • <code>hd</code>: <code>hd</code> or high definition accuracy mode is used for most accurate die model generation.
<code>-disable_isrc_node_zero { true false }</code>	
	Specifies to disable node zero for current sources (isrc) in a generated die-model. The isrc node zero is used to connect decoupling capacitance and the current source in the generated die-model. This parameter is added to generate a die model without isrc node zero. By default, a virtual ground is used to connect decoupling caps and the current source in the generated die-model. When set to <code>true</code>, generates die model without the virtual ground and uses the ground network for connecting decoupling caps and current sources. It also creates the <code><BLOCK>_nportModel.sp.isrc_balanced</code> file in the die-model directory. This file has information on the current sources and their port connections. The <code>.subckt</code> file generated for Spice simulation points to the <code>.isrc_balanced</code> file. <i>Default:</i> <code>false</code>
<code>-disable_global_node_zero { true false }</code>	
	Specifies to disable the global node zero in a generated die model. • When set to <code>true</code>, generates die models without the virtual ground and uses the ground network for connecting decoupling caps and current sources. • When set to <code>false</code>, a virtual ground is used to connect decoupling caps and the current source in the generated die-model. <i>Default:</i> <code>true</code>
<code>-enable_package_pin_map { true false }</code>	

	Specifies to support the 6-column style MCP format to map package pins to circuit ports. The following are the old and new MCP file formats: The original 5-column format is: <vsrc_name> <Ckt_port> <Net_name> <X coord> <Y coord> The new 6-column format is: <package_pin> <Ckt_port> <Net_name> <X coord> <Y coord> <vsrc_name> Example: Original format: <pre>chip1/bmp_100_6_12 D1_bmp_100_6_12 VDDE 350.000 147.000 chip2/bmp_122_3_15 D2_bmp_122_3_15 VDD2 321.000 983.800</pre> New format: <pre>bmp_100_6_12 D1_bmp_100_6_12 VDDE 350.000 147.000 chip1/bmp_100_6_12 bmp_122_3_15 D2_bmp_122_3_15 VDD2 321.000 983.800 chip2/bmp_122_3_15</pre>
-effective_rc	Generates an additional report file (.txt) containing the effective resistance and capacitance information. The parameter is used for generation of a table for effective resistance and capacitance across multiple operating frequencies. The following is a sample report format: <pre># Net:VDD_AO, Net:VSS # frequency # rdie # cdie 1.0000000e+07 4.73148443 2.47034747e-11 1.2589254e+07 4.90209173 2.46916698e-11 # Net:VDD_external, Net:VSS # frequency # rdie # cdie 1.0000000e+07 -183.231085 6.65466673e-12 1.2589254e+07 -164.007129 6.55112569e-12</pre>
-effective_rc_deck	

	Specifies to include the Rdie/Cdie (effective resistance/capacitance) calculation for the die-model frequency domain simulation setup in the Spice (.sp) format. The following is a snippet of the .sp file with the frequency-domain setup for Rdie/Cdie calculation: <pre>***** * Frequency domain analysis to capture input impedance. .AC DEC 10 10Meg 5G .PRINT AC RDIE=PAR('VR(VSS_in,VDD_AO_in)') CDIE=PAR('1/(6.283185*freq*(VI(VDD_AO_in)- VI(VSS_in)))') *****</pre>
<code>-enable_reference_node</code>	
	Specifies to replace the global node zero with the reference port in the generated die model.
<code>-enable_xp {true false}</code>	
	Specifies to generate the die model in the Extensively Parallel (XP) mode in which processing is distributed over a large number of machines. <i>Default:</i> false
<code>-generate_sipi_port_definition_file {true false}</code>	
	Generates a ports definition file during die-model creation. When set to <code>true</code>, the port definitions are output to the <code>port_defintion.txt</code> file, which is later used by Sigrity SystemPI for connecting the die model to the package design. <i>Default:</i> false
<code>-grid_reduction_cut_layer layer_name</code>	
	Specifies a cut via layer below which all the layers will be reduced. This feature minimizes data and enables efficient die model generation.
<code>-ignore_port_grouping_conflict</code>	

	Specifies to not generate an error message in case there is a conflict between the user-defined port grouping and the default port groups (voltage sources that are grouped by mapping to the same package terminal). For example, vsrca and vsrcaB are both mapped to the package terminal PKG_T, but in the user-specified grouping file, vsrca and vsrcaB belong to two groups. When this parameter is specified, the tool will not generate an error message if there is a conflict, but instead will group the voltage sources according to the user-specified port grouping file. If this parameter is not specified and there is a conflict in port grouping, an error will be generated.
<code>-local_reference_ground {true false}</code>	
	Specifies to assign a ground source as reference ground locally for sources in the current file <code>.isrc</code>. You can specify the reference node in the PLOC file as "RefGnd=PortName" to compute the isrc of port by considering reference ground locally during die-model generation. <i>Default:</i> <code>false</code> Example: <pre>create_die_model \ -local_reference_ground true \ -port_grouping_by_ploc_files true \ -generate_SIPi_port_definition_file true \ -disable_isrc_node_zero true \ (make isrc with ref ground) -disable_global_node_zero false (keep global node zero in ckt)</pre>
<code>-max_ports_per_net integer</code>	
	Specifies the maximum number of ports allowed for each net in a generated die model. If you do not specify this parameter, the software by default enables port grouping with number of ports set to 200. If the number specified through this parameter is 0, port grouping will be disabled. The maximum number of ports per net should be 0 or a positive value. A negative value will be treated as 0. Note: The <code>-max_ports_per_net</code> parameter is only meaningful for N-port die model creation. The 2-port die model creation always groups all ports together.
<code>-multi_die {true false}</code>	

	Specifies to generate a complete die model for a TSV/SiP design. <i>Default:</i> false
<code>-output_directory</code> <i>directory_name</i>	
	Specifies the directory in which you save the die-model files.
<code>-port_grouping_by_ploc_files</code> {true false}	
	Specifies to read the PLOC group information from the user-defined PLOC files. You can specify the port information in the PLOC file, as shown below: Vsource_5_VDD 21985.00 23785.00 AP VDD p18 # with net and group Vsource_7_VDD 22385.00 23785.00 AP VDD p18 C=1e-09 L=1e-11 # with capacitance and inductance When <code>-port_grouping_by_ploc_files</code> is set to true, the die model flow will read the user-specified port grouping information and generate a reduced die model. This parameter is mutually exclusive with the <code>-port_grouping_file</code> parameter.
<code>-port_grouping_file</code> <i>filename</i>	

	Specifies a text file containing the user-defined pin grouping details on a per power/ground net basis. When this parameter is specified, a reduced die model is generated that honors the user-specified port grouping. This die model has a reduced RC netlist and <code>.isrc</code> when compared with the original un-grouped die model. The degree of reduction will depend on how many pins are being grouped. The format of the pin grouping file is as follows: <pre>NET <netname> <grouped pin name> <vsrc names which have to be grouped> <grouped pin name> <vsrc names which have to be grouped> NET <netname> <grouped pin name> <vsrc names to be grouped> <grouped pin name> <vsrc names to be grouped> END</pre> where, • NET is a keyword followed by the power or ground net name • grouped pin name is the user defined port name • vsrc names are the actual design voltage sources that need to be grouped
<code>-port_grouping_tiles_file filename</code>	
	Specifies to group power ground ports by a user-defined tile array. All the ports in a specific tile are grouped together. The use model of the parameter is: <pre>create_die_model -port_grouping_tiles_file tiles.txt</pre> The following is the format of the port grouping tiles format is: <pre>NET <net> BBOX <x1 y1 x2 y2> ROWS <num rows> COLUMNS <num columns></pre> Here, • the keywords are highlighted in blue. • <code><net></code> is the power supply net name • <code><x1 y1 x2 y2></code> are the coordinates of the bounding box lower and upper corners. This is optional. If the bounding box is not specified, the default is the chip bounding box. • <code><num rows></code> is the number of rows in the tile array • <code><num columns></code> is the number of columns in the tile array This parameter allows you to partition the voltage sources of

the die model in to several port groups. If the same grouping pattern is used for the voltage sources of the die model and the package pins, it will help in matching the die ports and package terminals, and connecting the die model to the package netlist.

The grouping information is available in the MCP header of the generated die model file, as shown below:

```
* [REM]
* [Power Nets]
* vsrc_821 VDD2_GROUP_BUMP_tile01 VDD2 6316.25 20
* vsrc_820 VDD2_GROUP_BUMP_tile01 VDD2 623.75 20
* vsrc_822 VDD2_GROUP_BUMP_tile00 VDD2 5000 40
* vsrc_823 VDD2_GROUP_BUMP_tile00 VDD2 5002 40
.....
*[Ground Net]
* vsrc_100 VSS_GROUP_BUMP_tile01 VSS 6300 25
* vsrc_101 VSS_GROUP_BUMP_tile01 VSS 6250 25
* vsrc_102 VSS_GROUP_BUMP_tile00 VSS 4000 40
* vsrc_103 VSS_GROUP_BUMP_tile00 VSS 4002 40
```

Here

- the first column is the voltage source name.
- the second column is the port name under which the voltage sources are grouped. It also displays the tile number to check a specific tile's power and group port during simulation.
- the third column is the net name.
- the fourth and fifth columns indicate the X and Y coordinates, respectively. The unit of x/y coordinate location is micrometer (μm).

-probing_node_file *file_name*

	Specifies the probe node file containing the probe point on a die. The file format for specifying probing nodes is as follows: <pre><user defined unique probe name> <location = x> <location = y> <layer> <Power/ground net name></pre> You can also specify the package node voltages and resistor currents that are to be probed in the following format: <pre><probe name> pkg ir <pkg circuit node name> <probe name> pkg rc <pkg circuit resistor name></pre> Example <pre>probe_1 18 16 METAL_1 VDD # pkg node voltage probe_pkg_1 pkg ir GND.in0 probe_pkg_6 pkg ir VDD.in2 # pkg resistor current probe_pkg_cur_1 pkg rc R_1_1 probe_pkg_cur_2 pkg rc R_5_2</pre> Note: The units of x and y should be in microns.
<pre>-probing_region_file filename</pre>	
	Specifies a probe node region file to define a probe region and use all points in the region as probe contact points on the die. The file format for specifying a probing region is as follows: <pre>region_label x1 y1 x2 y2 layer_name net_name</pre>
<pre>-rail_analysis_domain domain_name -net net_name</pre>	
	Specifies the domain name or net name for which the die-model is to be generated. When the domain name is specified, coupled die-model is generated and it includes all power and ground nets associated with the specified domain. When net name is specified, decoupled die-model is generated for the specified net. Before specifying this command parameter, the power net name must be specified using the <code>set_pg_nets</code> command. This is a required option and one of them must be specified.

<code>-repeat time</code>	Specifies the repeat time in ns. When this parameter is specified without the <code>-use_sigrity_repeat</code> parameter, the generated die model has the dynamic current profile of the die repeated to the specified time. By default, the die model consists of the default simulation period saved in dynamic current files during IR drop analysis. When this parameter is specified along with the <code>-use_sigrity_repeat</code> parameter, the software forces to write out repeating waveforms in the Sigrity specific format. If the <code>-use_sigrity_repeat</code> parameter is specified, the value specified through the <code>-repeat</code> parameter indicates the starting point of the repeating waveform.
<code>-state_directory directory_name</code>	
	Specifies the top-level state directory that is generated by the static or dynamic IR drop analysis. This state directory contains analysis of all nets associated with the power domain; For example, <code>core_25C_dynamic_1</code>. This is a required parameter.
<code>-suppress_negative_res {true false}</code>	
	Specifies to suppress the negative resistor and capacitor (RC) values during die-model generation. When this parameter is set to <code>true</code>, the generated netlist contains positive RC values with voltage-controlled (VC), current-controlled(CC), and function-controlled (FC) sources to ensure accuracy and stability in the die models.
<code>-temp_directory_name directory</code>	
	Specifies a directory to save temporary files for die-model generation.
<code>-use_sigrity_repeat</code>	When this parameter is specified along with the <code>-repeat</code> parameter of the <code>create_die_model</code> command, the software forces to write out repeating waveforms in the Sigrity specific format. If the <code>-use_sigrity_repeat</code> parameter is specified, the value specified through the <code>-repeat</code> parameter indicates the starting point of the repeating waveform. If <code>-use_sigrity_repeat</code> is specified without specifying <code>-repeat</code>, time 0 is used for the starting point. The default unit for the value is "ns". Note: When the <code>-use_sigrity_repeat</code> parameter is specified, the software will write the stimulus current in the specific Sigrity format, which cannot be used by other Spice like simulators.

Examples

- The following command creates an n-port die model having the dynamic current profile of the die repeated upto 50ns, using the state directory `allDomains_25C_dynamic_1` and saves the generated n-port die-model file in the `n_port_diemodel` output directory:

```
create_die_model
-state_directory irdrop_allOn/allDomains_25C_dynamic_1
-rail_analysis_domain allDomains -output_directory n_port_diemodel -repeat 50
```

- The following example shows the use model of the `-effective_rc` parameter to create a report file containing the effective resistance and capacitance information:

```
create_die_model
-effective_rc -state_directory staticRailResults/ALL_125C_avg_1/
-output_directory static_die_model -rail_analysis_domain ALL
```

Related Topics

- "[Package Analysis](#)" in the *Voltus User Guide*

create_hier_view

```
create_hier_view
-help
-library_name name
[-block_lef lef_file]
[-block_gds gds_file]
[-gds_port_file gds_port_file]
[-cell_name name]
[-cluster_vial_ports {true | false}]
[-cluster_via_rule { {via_layer1 number_of_equidistant_vias}...}]
[-cluster_via_size value]
[-compress_powergrid_database {true | false}]
[-def list_of_DEF_files]
[-extraction_mode {fast | accurate}]
[-extraction_work_directory path_to_extraction_work_directory]
[-force_library_merging {true | false}]
[-ground_nets {pin1 pin2 ...pinN}]
[-ignore_fillers {true | false}]
[-ignore_decaps {true | false}]
[-ignore_shorts {true | false}]
[-libgen_command_file file]
[-max_viaccluster_mode {true | false}]
[-oaRef OA_reference_libs]
[-output_dir dir]
[-parasitic_extractor_command_file filename]
[-power_nets {pin1 value1 pin2 value2 ...pinN valueN}]
[-powergrid_view_libraries primitive_cells.cl]
```

Creates a hierarchical power-grid view. The hierarchical PGV generation of a block is part of the top-level Hierarchical Rail Analysis.

Parameters

-help	Prints out the command usage.
-block_lef <i>lef_file</i>	
	Specifies the name of the block LEF file. The <i>lef_file</i> must have all power pin port geometries defined for the hierarchical block.
-block_gds <i>gds_file</i>	
	In absence of block LEF, you can specify block GDS and port label file to generate hierarchical power-grid view.
-cell_name <i>name</i>	Specifies the block cell name as specified in the block DEF file.

<code>-cluster_vial_ports {true false}</code>	
	Designs with follow-pin routing on M1 and M2 layers have standard cells library LEF with M1, M2, and VIA1 ports. This parameter specifies to cluster VIA1 ports for such cells in order to improve overall extraction and rail analysis performance. <i>Default:</i> <code>true</code> This parameter is set to <code>true</code> by default to improve VIA1 port handling in designs with M1 and M2 follow-pin routing. This parameter gives significantly better performance by clustering equidistant VIA1 ports defined in cell LEF without significant loss of accuracy. However, if you specify a clustering rule, the user specification takes precedence over automated VIA1 clustering.
<code>-cluster_via_rule { {via_layer1 number_of_equidistant_vias}...}</code>	
	Controls the number of vias to cluster on a layer basis. The VIA clustering rule specified using this parameter will override the default clustering rule for a given accuracy mode. The following command will cluster 100 equidistant VIA1 cuts, 200 equidistant VIA2 cuts, and 300 equidistant VIA7 cuts: <pre>create_hier_view -cluster_via_rule { {VIA1 100}} {VIA2 200} {VIA7 300}}</pre> The rest of the VIAs will be clustered using the default clustering rule depending upon the rail analysis accuracy mode.
<code>-cluster_via_size value</code>	
	Specifies the size of the via array to cluster all via layers except VIA1 (layer between Metal1 and Metal2). The default value is <code>625</code> (25x25 via array), except when running static analysis using HD mode. The default for static HD mode is <code>16</code> (4x4), primarily to preserve more accuracy for EM analysis. <i>Default:</i> <code>625</code>
<code>-compress_powergrid_database {true false}</code>	
	Specifies to compress the output files generated by extraction and rail analysis to save disk space usage. This parameter reduces disk space requirement from the extraction and rail analysis stages at approximately 5-10% performance penalty. <i>Default:</i> <code>false</code>
<code>-def list_of_DEF_files</code>	
	Specifies a list of block DEF files for hierarchical analysis.
<code>-extraction_mode {fast accurate}</code>	

	Determines what power-grid views will be created, dependent on whether fast or accurate extraction is to be done. By default, hierarchical PGV generates both detailed and reduced views.
<code>-extraction_work_directory path_to_extraction_work_directory</code>	
	Specifies to re-use the extracted power-grid database to generate HPGV. This parameter is only supported to generate hierarchical extraction view of a block in the design. The specified extraction work directory must contain the extracted data for all the nets for which the hierarchical extraction view needs to be generated. This parameter is mutually exclusive with the <code>-def</code> parameter of the <code>create_hier_view</code> command.
<code>-force_library_merging {true false}</code>	
	Specifies to force merge PGVs with different resistivity and layers. It uses the technology information of the first library. If multiple tech-only PGVs are specified, it will take the first tech-only PGV definition and the first cell definition. If you are merging a tech-only PGV with other PGVs, it will keep the tech-only technology definition and the first cell definition.
<code>-gds_port_file gds_port_file</code>	
	In absence of block LEF, you can specify block GDS and port label file to generate hierarchical power-grid view.
<code>-ground_nets {pin1pin2...pinN}</code>	
	Specifies a list of ground nets of the block.
<code>-ignore_decaps {true false}</code>	
	Ignores decap cells during rail analysis. It ignores cells that are tagged as DECAP cells during library characterization or set as decap cells using the <code>set_rail_analysis_mode -decap_cell_list</code> command parameter. If decaps are used to preserve connectivity on the follow-pin routing, you must set this parameter to <code>false</code>. Ignoring decaps during dynamic analysis is not recommended. Default: <code>true</code> for static, <code>false</code> for dynamic
<code>-ignore_fillers {true false}</code>	

	Ignores filler cells during rail analysis if the list of filler cells are defined in PGV generation, or specified as fillers using the <code>set_rail_analysis_mode -filler_cell_list</code> command parameter. During rail analysis, ignore library cells that are tagged as FILLER cells and have no capacitance associated with the interface nodes. If fillers are used to preserve connectivity on the follow-pin routing, you must set this parameter to <code>false</code>. <i>Default:</i> <code>true</code>
<code>-ignore_shorts {true false}</code>	
	Ignores shorts that are found during extraction and continues hierarchical PGV generation. However, <code>-ignore_shorts</code> cannot be applied if the design contains GDS. <i>Default:</i> <code>false</code>
<code>-libgen_command_file file</code>	
	Specifies the file name of a power-grid library generation (libgen) command file. When specified, it will be included in the auto-generated <code>libgen.cmd</code> file at the end as <code>include <file></code>. This parameter allows you to provide a file that includes specific library generation options.
<code>-library_name name</code>	
	Specifies the name of the library that the block power-grid view will be placed in. A <code>.cl</code> extension will automatically be added to the name.
<code>-max_viaccluster_mode {true false}</code>	
	Specifies the extractor to cluster vias automatically to reduce the RC elements by clustering vias into one node, therefore reducing the IR drop analysis runtime in dynamic mode. It is based on the notion that current evenly distributes through all vias of an array via. <i>Default:</i> <code>true</code>
<code>-oaRef OA_Reference_libs</code>	
	Reads OpenAccess reference libraries.
<code>-output_dir dir</code>	Specifies the output directory that the results will be saved in.
<code>-parasitic_extractor_command_file filename</code>	

	Specifies the name of the file input to the extractor invoked by the Library Generation engine. This file contains information about the cells in the design and is used for resistance and capacitance extraction. For variables and functions that can be placed in this command file, see Extractor Options . In general, this parameter is used for debugging purposes.
<code>-powergrid_view_libraries primitive_cells.cl</code>	
	Specifies a list of primitive cell libraries for hierarchical PGV generation.
<code>-power_nets {pin1value1pin2value2 ...pinNvalueN}</code>	
	Specifies a list of power nets and the nominal voltage of those nets.

Examples

- The following command generates a hierarchical power-grid view for the hierarchical block using LEF and extraction directories for VDD and VSS nets:

```
create_hier_view -block_lef blockA.lef -extraction_work_directory work -library_name blockA -
powergrid_view_libraries ./pgv/stdcells.cl -power_nets {vdd 1.2} -ground_nets {gnd} -cell_name blockA
```

The design.lef must have all power pin port geometries defined for the hierarchical block. The generated hierarchical PGV library is blockA.cl. The library is in binary format and it stores the block's connectivity to the top-level based on the provided LEF data. It stores the extracted power-grid network and current distribution from the provided directories.

- The following command specifies to force merge PGVs with different resistivity and layers:

```
create_hier_view \
    -force_library_merging true \
    -library_name lib1 \
    -view hier_extraction_view \
    -powergrid_view_libraries {../accurate_stdcells_A.cl ../accurate_stdcells_A.cl ...
../accurate_stdcells_AN.cl } \
    -def {../design.def.gz} \
    -cell_name CELLA \
    -power_nets {VDD 1.32} \
    -ground_nets {VSS} \
    -block_lef {../design.lef} \
    -extraction_mode accurate \
generate_pg_library -output pgv
```

Related Topics

- "[Hierarchical Rail Analysis](#)" in the *Voltus User Guide*

create_power_pads

```
create_power_pads
[-append_to_vsrc_file]
[-auto_fetch]
[-cell {cell_list} ]
[-cell_pin {{cellname1 pinname1}}...{{cellnamen pinnamen}}+} ]
[-clear]
[-display]
[-format {padcell xy tsv}]
[-honor_pin_connection]
[-instance {inst_list} ]
[-instance_pin {{instname1 pinname1}}...{{instnamen pinnamen}}+} ]
[-layer {layername}]
[-net netname]
[-package_resistance value ]
[-package_capacitance value ]
[-package_inductance value ]
[-region {x1 y1 x2 y2}]
[-region_pitch {xpitch ypitch}]
[-snap_distance { true|false }]
[-vsrc_file vsrc_filename]
[[-add | -delete] -loc {x y}]
[-list]
```

Returns a list of DC sources from your design. DC sources are identified in the LEF file as CLASS PAD POWER, or in the DEF file as PIN with +USE POWER/GROUND. If a single port shape is connected to N number of wires, N number of DC sources are reported for that port shape.

You can use one of the following three methods to fetch voltage sources into the database:

- fetch all voltage sources
- fetch the voltage sources within the specified region of the specified layer controlled by the specified xPitch/yPitch
If the voltage sources with the specified xPitch/yPitch are not on the stripe, you can use the `-snap_distance` parameter to snap the voltage sources to the stripe. As a result, the voltage sources are generated only on the stripes and the xPitch/yPitch is honored. Use this feature to generate mesh type voltage sources on any process layer.
- fetch voltage source for the specified cell/cell pin/instance/instance pin

You can edit the fetched voltage sources before saving them to a voltage source file. To save the fetched list to a file, use the `-vsrc_file` parameter. By default, voltage sources will not be displayed on GUI after they are saved to file.

 The `create_power_pads` command supports hierarchical IR drop analysis for a TSV design. When you specify the `create_power_pads` command to fetch the voltage sources from the LEF file, the command not only searches for the P/G pin of the pad connected to the specified P/G net, but also fetches for the embedded bump TSV property definition for the P/G pin connected to the P/G net. The port that contains the point defined in the embedded bump property is considered as the embedded bump.

Parameters

-add	Adds the location of a new voltage source to memory.
-append_to_vsrc_file	Appends the fetched voltage sources to the existing voltage source file. You can use this parameter to fetch voltage sources in the incremental mode.
-auto_fetch	Specifies to fetch all voltage sources.
-cell {cell_list}	
	Specifies the cells from which voltage sources are determined.
-cell_pin {{cellname1pinname1}}...{{cellnamenpinnamen}}+	
	Specifies the cells and their pins from which voltage sources are determined.
-clear	Clears the display of the DC sources that were specified in the <i>Edit Pad Location</i> form.
-delete	Deletes the location of a voltage source from memory.
-display	Loads and displays the pad location data from the specified file.
-format {padcell xy tsv}	
	The format of the saved pad location file. The choices are Pad File (VoltageStorm), XY File (xy coordinates), and TSV. You can create and save the voltage sources location file with a unique name for the voltage sources (both VDD and VSS). <i>Default:</i> xy
-help	Outputs the command usage and a brief description about the command parameters.
-honor_pin_connection	
	Honors pin wire connection when fetching voltage source location from a power pad. When you specify this parameter, only pins with wire connection are output as voltage sources. You have the choice to specify whether real wire connection is to be honored or not. Typically, during the sign-off stage, power pads are already fully routed so the unconnected power pads should not be counted in as voltage sources. You can specify this using the -honor_pin_connection parameter. Alternatively, in the early stage, power pads may not have been fully routed so you do not need to set this parameter. Note: For a port with shapes at multiple layers, all layer connected shapes are fetched as voltage sources.
-instance {inst_list}	
	Specifies the instances from which voltage sources are determined.

<code>-instance_pin {{instname1pinname1}...{instnamenpinnamen}+}</code>	
	Specifies the instances and their pins from which voltage sources are determined.
<code>-layer <i>layerName</i></code>	
	Specifies the process layer in LEF on which the voltage sources are to be generated. When the <code>-layer</code> parameter is specified in conjunction with the <code>-region {x1 y1 x2 y2}</code> and <code>-region_pitch {xpitch ypitch}</code> parameters, the layer set here is the layer for which voltage sources are to be fetched. If the <code>-layer</code> parameter is not used together with the <code>-region {x1 y1 x2 y2}</code> and <code>-region_pitch {xpitch ypitch}</code> parameters, the layer set here is for the lowest layer to snap (the default value is M5).
<code>-list</code>	
	Loads and displays all voltage sources from memory.
<code>-loc {x y}</code>	
	Specifies the location at which the specified voltage source is to be added or deleted.
<code>-net <i>netname</i></code>	
	Specifies the name of a power net for which to list DC sources. <i>Default:</i> If you do not specify a net name, the power analyzer automatically lists DC sources for all nets.
<code>-package_resistance <i>value</i></code>	
	Specifies the resistance value of the package resistance-inductance-capacitance (RLC) model in Ohm.
<code>-package_capacitance <i>value</i></code>	
	Specifies the capacitance value of the package resistance-inductance-capacitance (RLC) model in Farad (F).
<code>-package_inductance <i>value</i></code>	
	Specifies the inductance value of the package resistance-inductance-capacitance (RLC) model in Henry (H).
<code>-region {x1 y1 x2 y2}</code>	
	Generates the voltage sources in the specified region.
<code>-region_pitch {xpitch ypitch}</code>	
	Specifies the region pitch in the x and y direction for the voltage sources.
<code>-snap_distance <i>value</i></code>	

	Specifies the snapping distance constraint from I/O/pad pins. If this parameter is not specified with a value, no snapping will be done. <i>Default: 0</i> Specifies whether to snap DC sources from I/O pad pins to some location on the power structure. When set to true, the software performs autofetch with snapping. For I/O pad pins that are connected to core ring, the software selects the highest layer and center of the pin as the DC source location. When I/O pad pins are not connected to core ring, the software snaps the location of the I/O pad to somewhere on the core ring. The DC source location should be within the distance of snapping distance constraint from I/O pad pins, and the layer should not be lower than the lowest layer snap constraint.
<code>-vsrc_file vsrc_filename</code>	
	Specifies the name of the voltage source file that is required to save the auto fetched result.

Examples

- The following command fetches all voltage sources of the net `VDD`:

```
create_power_pads -net VDD -auto_fetch
```
- The following command loads and displays all voltage sources from memory:

```
create_power_pads -list
```
- The following command specifies the location at which the specified voltage source is to be added:

```
create_power_pads -add -net VDD -layer METTOP -loc {500 500}
```
- The following command specifies the location from which the specified voltage source is to be deleted:

```
create_power_pads -delete -loc {500 500} -layer METTOP -net VDD
```
- The following command fetches the voltage sources within the specified region of the layer `METAL6` controlled by the specified `xpitch/ypitch`, and saves the sources to the file `report2` in the `xy` format:

```
create_power_pads -net VDD -region 100 100 3000 3000 -region_pitch 555 444 -layer METAL6 -format xy -snap_distance 100 -vsrc_file report2.pp
```

 In the first example, the output was not saved but it remains in the memory till you specify the second example. Before loading and saving the output of the second example, the first example output in the memory is saved to an auto generated dc source file incrementally, like `pp.src`, `pp1.src`, `pp2.src`, and so on.

- The following command fetches voltage source for the cell `BUMPCELL` in the `xy` format:

```
create_power_pads -net VDD -cell BUMPCELL -format xy
```
- The following command clears the voltage sources displayed in the GUI and the fetched voltage sources in the database:

```
create_power_pads -clear
```
- The following command displays the fetched and saved voltage sources for the net `VDD` in the file `report2` :

```
create_power_pads -net VDD -auto_fetch -vsrc_file report2.pp -display
```
- The following command displays voltage sources in the GUI by loading the saved voltage source file `report1`:

```
create_power_pads -vsrc_file report1.pp -display
```
- The following command saves the fetched voltage sources to the voltage source file `report1.pp`:

```
create_power_pads -vsrc_file report1.pp
```

create_power_pads_regions

```
create_power_pads_regions
[-help]
-design_stack_config filename
-pad_config {{ pad_cfg_file1 die_inst1} { pad_cfg_file2 die_inst2} { pad_cfg_file3 die_inst3}...}
-power_net_config filename
-tsv_subckt_config {{file1 top_layer bottom_layer die1} {file2 top_layer bottom_layer die2}+}
-voltage_source_layers {{metal_layer die1}+}
[-work_directory directory]
```

Generates virtual power pads and pseudo TSVs for multi-die ERA.

Parameters

<code>-design_stack_config filename</code>	
	Specifies the name of the XML file containing the design stack information. This file is from the Integrity™ 3D-IC product.
<code>-help</code>	
	Outputs a brief description that includes the type and default information for each <code>create_power_pads_regions</code> parameter. For a detailed description of the command and all of its parameters, use the man command: <code>man create_power_pads_regions</code>
<code>-pad_config {{ pad_cfg_file1 die_inst1} { pad_cfg_file2 die_inst2} { pad_cfg_file3 die_inst3}...}</code>	
	Specifies the mapping of the bump region configuration file and die instances. The format of the bump region configuration file is: <code>#Net, Region, metal layer, number</code> One net can have multiple P/G pad regions, but pad regions should not overlap. The pad regions of a net should be inside the net region. Format Example: <code>VDD, {0 0 380 0 380 180 340 310 200 310 200 380 70 150 0 150 0 0}, AP, 250</code> <code>VDD, {0 150 70 150 70 380 0 380 0 150}, AP, 200</code> <code>VSS, {0 0 380 0 380 180 340 310 200 310 200 380 70 150 0 150 0 0}, AP, 280</code> <code>VSS, {0 150 70 150 70 380 0 380 0 150}, AP, 300</code>
<code>-power_net_config filename.pwrcfg</code>	
	Specifies the name of a file containing power net inter-die mapping configuration. This file is from the Integrity™ 3D-IC product.
<code>-tsv_subckt_config {{file1 top_layer bottom_layer die1} {file2 top_layer bottom_layer die2}+}</code>	

	Specifies the TSV subcircuit configuration. For each die instance, you need to specify the TSV subcircuit file, TSV top layer, and the TSV bottom layer.
	<code>-voltage_source_layers {{metal_layer die1}+}</code>
	Specifies the voltage source layer for each die instance.
	<code>-work_directory directory</code>
	Specifies the location of the run directory.

Examples

- The following command specifies static current region:

```
create_power_pads_regions -design_stack_config ./data/Integrity3DIC/top_2Die.xml \
    -power_net_config ./data/Integrity3DIC/FCCC_top_2Die.pwrcfg \
    -pad_config { {./data/bump_cfg/U1_bump_region.cfg U1} {./data/bump_cfg/U2_bump_region.cfg U2} } \
    -tsv_subckt_config { {./data/tsv_model/tsv_rc.subckt M13 BPMS U1} } \
    -vsrc_layers { {AP U1} }\
    -work_directory ./RUN_era/work_bump/
```

Related Topics

- "[What-If Rail Analysis](#)" in the *Voltus User Guide*

create_what_if_shape

```
create_what_if_shape
[-help]
[-add -method {auto | exact}]
[-add | -select | -deselect |
 -delete | -list | -save filename | -load filename]
[-add -width width]
[-area {llx lly urx ury}]
[-display]
[-direction {hor | ver}]
[-format {txt def}]
[-layer layerName]
[-nets {net1 net2 net3...}]
[-objects {selected | all} ]
[-offset {X Y}]
[-pitch pitch]
[-remove_existing_wires]
[-save filename -append ]
[-spacing value]
[-type {wire | via}]
```

Specifies to add, select, deselect, delete, list, and display the what if wires/vias to a design.

Parameters

-help	
	Outputs the command usage.
-add	
	Specifies to add what-if wires/vias.
-append	
	Specifies to append all what-if wires/vias to the existing file specified with the <code>-save filename</code> parameter.
-area {llx lly urx ury}	
	Specifies the rectangular region in which the what if wires/vias are to be generated.
-delete	
	Specifies to delete all or selected what-if shapes by constraint <code>-net/-layer/-type</code> . You can use this parameter in conjunction with the <code>-objects</code> parameter.
-deselect	

	Specifies to deselect all what-if shapes by constraints <code>-net/-layer/-type</code> . Use this parameter in conjunction with <code>-area</code> to deselect what-if shapes overlapping with the specified area.
<code>-direction {hor ver}</code>	
	Controls the direction of the wires added.
<code>-display</code>	
	Invokes the GUI to show what-if shapes and zooms to the selected shapes. It works only in the <code>-win</code> mode.
<code>-format {txt def}</code>	
	Specifies the output file format. The <code>def</code> format specifies to create incremental DEF to load into GUI using <code>read_def -top</code> . The default output format is <code>txt</code> .
<code>-layer layerName</code>	
	Specifies the LEF layer name for wires, or layer pair for via to be added.
<code>-list</code>	
	Specifies to list all or selected what-if shapes by constraint <code>-net/-layer/-type</code> . You can use this parameter in conjunction with the <code>-objects</code> parameter.
<code>-load filename</code>	
	Specifies to load the saved what-if shape file.
<code>-method {auto exact}</code>	
	Specifies the method used to add vias: <code>auto</code>: When area and layer pair are specified, the tool automatically adds vias on all intersecting points on the specified layer pairs in the given area. <code>exact</code>: When an area and layer pair are specified, the tool adds a via of exact size of the specified area between the specified layer pair.
<code>-nets {net1 net2 net3...}</code>	
	Specifies the name of the nets for which what-if wires/vias is to be created. The what-if engine will create a shape based on the net pattern.
<code>-objects {selected all}</code>	
	Specify <code>-objects</code> in conjunction with the <code>-delete</code> or <code>-list</code> parameters to delete or list the selected what-if shapes. You can also delete or list all what-if shapes. The default value is <code>all</code>. For example, you can use the following command to delete the selected objects: <pre>create_what_if_shape -delete -objects selected</pre>
<code>-offset {X Y}</code>	

	Specifies to create metal/via from the area + offset. This parameter is useful when you run multiple what-if shape creation commands and want to keep the same area but have different offset.
<code>-pitch <i>pitch</i></code>	
	Specifies the distance between the centre line of two adjacent what-if wires (pitch) to be drawn in the specified -area in the preferred direction of the specified -layer. The direction of created wires can also be controlled using the -direction parameter. If this parameter is not provided, a single rectangular shape of size {llx,lly,urx,ury} is created.
<code>-remove_existing_wires</code>	
	Specifies to remove a wire and connected vias from the specified layer and area before adding what-if shapes.
<code>-save <i>filename</i></code>	
	Specifies to save all the in-memory what-if wires/vias to the specified file. The format of the file is: <pre>type wire net netName area llx lly urx ury layer layer1 type via net netName area llx lly urx ury layer layer1 layer2</pre> Following is an example of the file containing one wire and one via shape: <pre>type wire net VDD area 191.000000 763.000000 207.000000 772.000000 layer METTOP type via net VDD area 513.000000 497.700000 517.000000 499.100000 layer METG1 ME</pre>
<code>-select</code>	
	Specifies to select all what-if shapes by constrains -net/-layer/-type. Use this parameter in conjunction with -area to select what-if shapes overlapping with the specified area.
<code>-spacing <i>value</i></code>	
	Specify the spacing between each net. Use this parameter to control the spacing between each net. The -pitch value specified should be greater than number of nets * (width+spacing).
<code>-type {wire via}</code>	
	Specifies what kind of shape is to be created.
<code>-width <i>value1 value2 ..valuen</i></code>	
	Specifies the width of what-if wires. You can use the -width parameter to specify a separate width value for each net.

Examples

- The following command adds a what-if wire for the net `VDD` and on the LEF layer `METTOP`:

```
create_what_if_shape -type wire -net VDD -area {391.000000 663.000000 407.000000 672.000000} -layer METTOP -add
create_what_if_shape -type wire -net VDD -area {191.000000 663.000000 207.000000 772.000000} -layer METTOP -add -pitch 9 -width 0.4
```
- The following command creates a what-if via for the net `VDD` and on the LEF layers `METG1` `MET5`:

```
create_what_if_shape -type via -net VDD -area {513.000000 497.700000 517.000000 499.100000} -layer {METG1 MET5} -add -method exact
create_what_if_shape -type via -area {0 0 1000 1000} -method auto -layer {METTOP METG1} -add -net VDD
```
- The following command specifies to list selected what-if shapes:

```
create_what_if_shape -list -objects selected
```
- The following command specifies to select and highlight all wires in the win mode:

```
create_what_if_shape -select
```
- The following command specifies to select and highlight all what-if vias using area:

```
create_what_if_shape -type via -area {500 490 530 500} -select -net VDD
```
- The following command specifies to save what-if shapes into the file `whatif.shapes`:

```
create_what_if_shape -save whatif.shapes
```
- The following command specifies to delete all what-if shapes in memory:

```
create_what_if_shape -delete -objects all
```
- In the following example, for nets “`VDD VDD1 VDD2 VSS`”, three spacing is possible (between `VDD` and `VDD1`, between `VDD1` and `VDD2`, and between `VDD2` and `VSS`), that is, spacing is “`0.5 0.6 0.7`” and width of what-if wires for all nets is “`0.5`”:

```
create_what_if_shape -net {VDD VDD1 VDD2 VSS} -pitch 4 -width 0.5 -spacing {0.5 0.6 0.7} -area {0 0 12 12} -type wire -add -layer M10 -direction hor
```
- The following command will remove all wires on `M7` in the `0,0` to `100,100` region as well as any vias connected to the deleted wires:

```
create_what_if_shape -remove_existing_wires -layer {M7} -area { 0 0 100 100 }
```

debug_irdrop

```
debug_irdrop
[-help]
[-eco_report]
[-enable_xp {true | false}]
[-net netname | -domain domainname]
[-net_threshold value | -domain_threshold value]
[-non_violating_instslimit value]
[-nworst_instances value]
[-output_directory dir]
-state_directory dir
[-summary_only]
[-region {x1 y1 x2 y2 }| -nregion value]
[-tile_row_col {X Y}]
[-tile_size {X Y}]
```

Specifies to automatically identify the region with worst IRdrops and EM violations, and provide necessary information to determine and fix the root cause. The reports are consolidated for each region to find root-cause. Root cause analysis is done by identifying top 10 instances in each hotspot and report the following:

- Hotspot Region Coordinate
- Timing Library Analysis
- Power-Grid View Analysis
- Power-Grid Integrity Analysis
- IR-Drop Composition
- Root Cause Analysis

You must specify this command after running the `analyze_rail` command.

 The `debug_irdrop` command requires a VTS-XL license.

Parameters

- help	Outputs a brief description that includes the type and default information for each <code>debug_irdrop</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command <code>man debug_irdrop</code> .
-domain domainname	
	Specifies the name of the power domain.
-domain_threshold value	

	Specifies the minimum allowed voltage for the domain being analyzed. The value will be in percentage. This parameter behavior is same as the <code>set_rail_analysis_domain -threshold</code> parameter.
<code>-eco_report</code>	
	Specifies to generate a file with IR drop information that can be used by Tempus ECO to fix IR voltage drop for all instances in a design. The report (<code>eco.ird</code>) is generated in the specified output directory (<code>debug_irdrop -output_directory</code>).
<code>-enable_xp {true false}</code>	
	Specifies to identify the worst IR drop and EM violations regions in the Extensively Parallel (XP) mode wherein processing is distributed over a large number of machines. <i>Default: false</i>
<code>-net netname</code>	
	Specifies the name of the power or ground net that you want to analyze.
<code>-net_threshold value</code>	
	Specifies the minimum allowed voltage for the power net being analyzed. The value must be an absolute number. This parameter behavior is same as the <code>set_pg_nets -threshold</code> parameter.
<code>-non_violating_instslimit value</code>	
	Reports non-violating instances within hotspots (violating tiles) that are close to the threshold value. This parameter allows you to define a limit (sub-threshold) that can be used to report the non-violating instances that are close to the threshold. For example, if the <code>-net_threshold</code> or <code>-domain_threshold</code> is 60mV, and <code>-non_violating_instslimit</code> is 3mV close to the threshold, the <code>debug_irdrop</code> command will report all the violating instances above the domain/net threshold and also the non-violating instances which are in the range of 60mV-57mV. This information about the non-violating instances can be used for IR improvement in the Timing-aware IR Drop Fixing flow.
<code>-nregion value</code>	
	Specifies the number of regions to be analyzed. The default value is 3.
<code>-nworst_instances value</code>	
	Specifies the number of worst instances to be reported. The default value is 5.
<code>-output_directory dir</code>	
	Specifies the output directory in which the Irdrop and EM violation results will be saved in.
<code>-region {x1 y1 x2 y2 }</code>	
	Specifies the region containing the worst instances. The default unit of <code>x1 y1 x2 y2</code> is in database units.
<code>-state_directory dir</code>	

	Specifies the top-level state directory that is generated by the dynamic IR drop analysis. This state directory contains analysis of all nets associated with the power domain. This is the output directory in which the results of the <code>analyze_rail</code> command is stored (for example, <code>core_25C_dynamic_1</code>).
<code>-summary_only</code>	
	Specifies to generate a violation summary report that displays a summary of the IR drop violation distribution for all the cells. In addition, the report also displays the summary of the IR drop violation distribution for the standards cells. If this parameter is specified, the detailed IR report is not generated.
<code>-tile_row_col {X Y}</code>	
	Specify the number of tiles in the x and y direction. The <code>-tile_row_col</code> and <code>-tile_size</code> parameters enable tile-based analysis of the hotspot regions. Tile-based analysis divides the design into user-defined tile size, and reports IR drop violations on each tile. The default value is <code>100um^2 (10x10 tiles)</code> .
<code>-tile_size {X Y}</code>	
	Specify the size of tiles in the x and y direction. The <code>-tile_row_col</code> and <code>-tile_size</code> parameters enable tile-based analysis of the hotspot regions. Tile-based analysis divides the design into user-defined tile size, and reports IR drop violations on each tile.

Voltus IC Power Integrity Solution Text Command Reference

Rail Analysis Commands

Hot-Spot Debugger Report

* Top Worst Instances

ID	Instance Name
1	inst_I2
2	inst_I3

...

#Power-Grid Integrity Analysis

Instance	LEF Pin:PG Net	Floating PG Pins
1	VDD:VDD ; VSS:VSS	None
2	VDD:VDD ; VSS:VSS	None

...

* Number of Weakly Connected Wire Segments

VDD	VSS
0	0

#Power-Grid View Library Status

Instance	Status	Power Pins (V)	Intrinsic Cap (fF)	View Used	#Current Taps	PGV Library Path
1	PASS	VDD:1.1	VDD:13.480 VSS:13.480	Early	VDD:1 VSS:1	/data/cl/stdcells.cl

...

#IR-Drop Composition

Instance	Total IR-drop (V)	IR-drop on Power Net (V)	Ground-bounce on ground net (V)
1	0.072 (TW) 0.072 (Worst)	0.031 (TW) 0.031 (Worst)	0.043 (TW) 0.043 (Worst)

...

* Layer Based IR-Drop for Net VDD

Instance	IR Across Package (mV)	IR Across Power-Gate (mV)	Layer Based IR (mV)
			M5L 0.683
			C4L5L 0.477
1	0.035	0.000	M4L 0.394
2	...		

* Layer Based IR-Drop for Net VSS

...

#Root Cause Analysis Report

* Net: VDD

Instance	IR-Drop (V)	RLRF (Ohm)	Loading Cap (fF)	Current Density (mA/um2)	Transition Density	Frequency (MHz)	Slew
1	0.031*	66.5*	64.371*	0.022*	615384576*	307*	0.100*

...

Design Max	0.058	159	140332.844	0.061	615384640	307	0.984
Design Avg	0.003	43.7	2.585	0.002	93809752	300	0.076
Design Min	0.000	0.0988	0.000	0.000	0	25	0.002

* Net: VSS

Instance	IR-Drop (V)	RLRF (Ohm)	Loading Cap (fF)	Current Density (mA/um2)	Transition Density	Frequency (MHz)	Slew
...							

Examples

- The following command generates IR debug report for the vss net in the specified output directory:

```
debug_irdrop -state_directory dynamic_rail/core_25C_dynamic_1/VSS -net VSS -output_directory IR_DEBUG_VSS -nworst_instances 10
```
- The following command generates a violation summary report that displays a summary of the IR drop violation

distribution:

```
debug_irdrop -summary_only
```

The following is a sample summary report:

Summary For All Cells:

Worst IR drop: 0.123326V

Specified IR drop threshold: 0.09V

Worst IR drop violation: 0.033326V

Total IR drop violation: 29.9416V

IR drop violation distribution

Violation Range	Violation Count	Accumulated Violation count	Total Violation(V)	Accumulated Total Violation(V)
120mV ~ 125mV: 0.094411	3	3	0.094411	
115mV ~ 120mV: 0.953065	32	35	0.858654	
110mV ~ 115mV: 2.52854	72	107	1.57548	
105mV ~ 110mV: 5.10688	150	257	2.57834	
100mV ~ 105mV: 12.0615	586	843	6.95461	
95mV ~ 100mV: 23.1765	1518	2361	11.115	
90mV ~ 95mV: 29.9416	3013	5374	6.76508	

Summary For StdCells:

Worst IR drop: 0.123326V

Specified IR drop threshold: 0.09V

Worst IR drop violation: 0.033326V

Total IR drop violation: 29.9416V

IR drop violation distribution

Violation Range	Violation Count	Accumulated Violation count	Total Violation(V)	Accumulated Total Violation(V)
120mV ~ 125mV: 0.094411	3	3	0.094411	
115mV ~ 120mV: 0.953065	32	35	0.858654	

Voltus IC Power Integrity Solution Text Command Reference

Rail Analysis Commands

110mV ~ 115mV: 2.52854	72	107	1.57548	
105mV ~ 110mV: 5.10688	150	257	2.57834	
100mV ~ 105mV: 12.0615	586	843	6.95461	
95mV ~ 100mV: 23.1765	1518	2361	11.115	
90mV ~ 95mV:	3013	5374	6.76508	29.941

Related Topics

- *"Static Power, IRdrop and EM Analysis"* and *"Dynamic Power and IRDrop Analysis"* in *Voltus User Guide*

extract_package

```
extract_package
[-help] <domain>
[-output <directoryname>]
-pkg_bga_name <bga_circuitname>
-pkg_die_name <die_circuitname>
[-reference_net <netname>]
[-spd_file filename]
-workspace <workspace_name>
```

Specifies to start package model extraction.

Parameters

-help	Outputs the command usage and a brief description about the command parameters.
domain	Specifies the rail analysis domain name.
-output <directoryname>	
	By default, the package model will be created in the workspace. However, if you specify an output directory, then a copy of the result will be made in the new directory.
-pkg_bga_name <bga_circuitname>	
	Defines the ball grid array (BGA) circuit name in the package model. It is the PCB component in the package model, which eventually connects to the PCB model.
-pkg_die_name <die_circuitname>	
	Defines the die circuit name in the package model. When specified, the software automatically creates a mapping between the die name in the package extraction tool and the die name in the software.
-reference_net <netname>	
	Specifies the ground net that is selected as reference net for pin-based model generation. This is the ground net that is being referred to by the power nets.
-spd_file filename	
	Specifies an existing Sigrity .spd file for package extraction. This parameter allows you to specify a .spd file instead of the workspace name (package model database) of the package extraction tool (XtractIM).
-workspace <workspace_name>	
	Specifies the workspace name (package model database) of the package extraction tool (XtractIM). The file name extension is .ximx.

Example

- The following example specifies the package models details and starts extraction:

```
extract_package ALL \  
-workspace ../rak_xim_pkg_model/C1_Voltus_4L.ximx \  
-pkg_bga_name FPBGA144 \  
-pkg_die_name CD1 \  
-output rak_xim_pkg_model_voltus_cpf \  
-reference_net VSS
```

map_die_package

```
map_die_package
[-help]
[-output directory]
-package_model_file filename
{-die_mcp_header filename | -net_pad_file_pair {{<net1> <pad file1>} {<net2> <pad file2>}...}}
```

The command performs the following functions:

- launch the *MCP Editor* GUI to make manual connections between package and die pins.
- map the voltage source files with the package model to check and correct mapping information before rail analysis.
- generate a mapping file that can be used in previous versions of the software in case there is no mapping between die and package pins.

The command has two types of inputs:

- Voltage source files in the xy format and a package model file with the MCP header.
- A circuit file with the MCP header and a package model file with the MCP header.

The output will be the same for both types of inputs:

- A kct mapping file containing only the MCP header, with the name `pkg_model_base_name + "_map"+suffix`.
- The file named '`un_mapped_bumps.txt`' will contain all the bumps which cannot be mapped.

Parameters

- help	Outputs the command usage and a brief description about the command parameters.
-die_mcp_header <i>filename</i>	
	Specifies a circuit file with the MCP header containing the die abstract.
-net_pad_file_pair {{<net1> <pad file1>} {<net2> <pad file2>}...}	
	Specifies the voltage source files mapping to each net.
-output <i>directory</i>	
	Specifies the output directory containing the mapping results.
-package_model_file <i>filename</i>	
	Specifies the name of the package model file.

Examples

- The following command specifies a circuit file with MCP header and a package model file with the MCP header:

```
map_die_package -die_mcp_header super_filter_mcp_ploc.ckt -package_model_file  
C1_Voltus_4L_C1_Voltus_4L_Final_PinBaseSPICE.ckt -output out_ploc
```
- The following command specifies voltage source files in the XY format and a package model file with the MCP header:

```
map_die_package -output ./out_pad -net_pad_file_pair {{VDD VDD_AO_srcs.pp} {VDD VDD_external_srcs.pp}  
{VSS VSS_srcs.pp}} -package_model_file C1_Voltus_4L_C1_Voltus_4L_Final_PinBaseSPICE.ckt
```

Related Topics

- "[Package Analysis](#)" in the *Voltus User Guide*

generate_esd_rlrp_report

```
generate_esd_rlrp_report  
[-help]  
-net netname  
[-instance_pair_list {{inst1 inst2 [pin1 pin2]}}+} | -instance_pair_list_file filename]  
[-number_threads value]  
-state_directory directory
```

Reports the least-resistance path (RLRP) for the bump-to-clamp rule type in the ESD flow. This command helps to make debugging faster when a fail status is given for a bump-to-clamp pair. Using this command, you can:

- view the RLRP path for the selected bump-to-clamp pair in the GUI
- generate a point-to-point RLRP path report that displays the resistance between each node (resistor segment) pair along the RLRP path

Parameters

- help	Outputs the command usage and a brief description about the command parameters.
<code>-net netname</code>	
	Specifies the name of the net to be analyzed.
<code>-number_threads value</code>	
	Specifies the maximum number of threads to use when reporting the least-resistance path (RLRP) for the bump-to-clamp rule type in the ESD flow. Generally, using more threads decreases the run time. There is no absolute limit to the number of threads you can use, however, it is recommended that you do not specify more threads than the number of CPUs available. The default value is 1, and the maximum value that can be specified is the total number of available CPUs.
<code>-instance_pair_list {{inst1 inst2 [pin1 pin2]}+}</code>	
	Specifies the instance pair list to enable effective resistance calculation between two instances in the design. <i>pin</i> is an optional argument. By default, the tool will use the instance pin associated with the net being analyzed. The <i>pin</i> argument can be used in special cases when two different pins of an instance are connected to the same net. <i>pin1</i> and <i>pin2</i> are pins of <i>inst2</i> respectively. If a pin name is not given, all pins associated to the specified instance are used.
<code>-instance_pair_list_file filename</code>	
	Specifies the name of the instance pair list file. This file contains the instance pair list in the same format as mentioned for the <code>-instance_pair_list</code> parameter. The file allows you to specify a large number of instance pairs. <code>-instance_pair_list_file</code> and <code>-instance_pair_list</code> are mutually exclusive.
<code>-state_directory directory</code>	
	Specifies the name of the state directory in which the ESD analysis results are saved.

Examples

- The following commands specify to generate point-to-point RLRP path report for net `VDD_AO`:

```
generate_esd_rlrp_report \
-state_directory /rak_for_esd/ESD/ALL_125C_esd_1 \
-net VDD_AO \
-instance_pair_list { {VDD_AO_vsrc1 u0} {VDD_AO_vsrc2 u0 NA VDD} }
```

generate_power_up_report

```
generate_power_up_report
[-help]
-switch_net net_name
-output_directory directory_name
-state_directory directory_name
```

Specifies to generate power-up statistics report to view the ramp up/inrush current, worst power switch voltage, and additional instance voltage data (average, median, max, and standard deviation voltage data versus time) for the powering-up rail. The following files are generated in the GIF and text formats:

- average voltage of instances (.avgV)
- max voltage of instances (.maxV)
- min voltage of instances (.minV)
- median of instance voltages (.medianV)
- standard deviation of instance voltages (.deltaV)
- total inrush current (.inrushI)
- average voltage of power gates (.avgV)
- max voltage of power gates (.maxV)
- min voltage of power gates (.minV)
- median of power gate voltages (.medianV)
- standard deviation of power gate voltages (.deltaV)

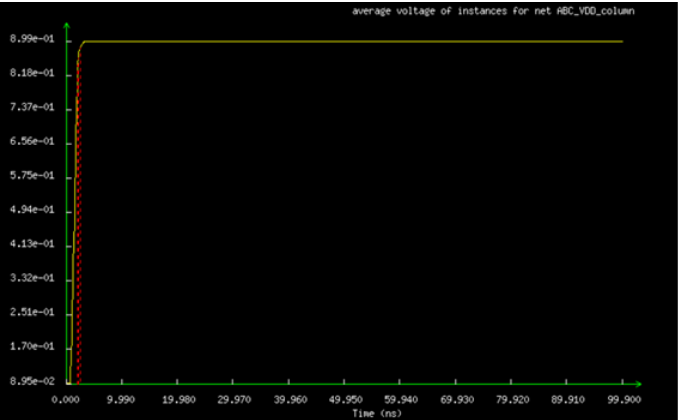
Parameters

-help	Prints out the command usage.
-switch_net <i>net_name</i>	
	Specifies the name of the powering up rail for which you want to generate detailed power-up statistics.
-output_directory <i>directory_name</i>	
	Specifies the output directory in which the power-up statistics results will be saved in.
-state_directory <i>directory_name</i>	
	Specifies the top-level state directory that is generated by IR drop analysis. This state directory contains analysis of all nets associated with the power domain; for example, core_25C_dynamic_1.

Examples

- The following command generates power-up statistics for the ABC_VDD_column net:
generate_power_up_report -net ABC_VDD_column -state_directory core_25C_dynamic_1 -output_directory rpt

The following is a snippet of the text and GIF formats with information on average voltage of instances:

Text Format (ABC_VDD_column.avgv)	GIF Format (ABC_VDD_column.avgv)
<pre>#List average voltage of instances for net ABC_VDD_column # 90% of nominal voltage at 2.1 ns # 95% of nominal voltage at 2.2 ns # 98% of nominal voltage at 2.6 ns #XLABEL# Time (ns) #YLABEL# Voltage (V) 0 0.0894674 0.1 0.0894675 0.2 0.0894676 ...</pre>	 90% Vmax at 2.100 ns 95% Vmax at 2.200 ns 98% Vmax at 2.600 ns

get_ir_insight

```
get_ir_insight
[-help]
[-cap_lower_limit value]
[-cap_upper_limit value]
-domain domainname
-domain_threshold value
[-eco_list list_of_instances ]
[-eco_mode {violation | aggressor | both}]
[-ir_drop_histogram_bin_size absolute_value_in_volts]
[-ir_drop_histogram_lower_limit absolute_value_in_volts]
[-ir_drop_histogram_upper_limit absolute_value_in_volts]
[-mode {report_by_region | predict_ir | report_aggressor | eco_report | report_only | repair_directive_only
|report_and_repair_directive}]
[-nregion value]
[-nworst_aggressors value]
[-nworst_instances value]
-output_directory directoryname
[-pdn_model directoryname]
[-report_mode {summary_only | detailed} ]
[-reset_predict]
-state_directory directoryname
[-watch_box filename]
[-watch_inst filename]
```

The `get_ir_insight` command allows you do the following:

- IR drop distribution and summary reporting by the specified IR threshold - Worst Instance Voltage Drop (WIV), Total Instance Voltage Drop (TIV), and Number of Violation Instance Voltage Drop (NVIV)
- Aggressor IR impact analysis and reporting by the specified violation and aggressor limits
- Support various Tempus IR-ECO flows (violation or aggressor-based flow with aggressor IR impact analysis and ECO IR estimation)
- IR drop prediction/estimation on input ECO changes
- IR violation reporting by regions

You must specify this command after running the `analyze_rail` command.

 The `get_ir_insight` command requires the `invs_vi` (Innovus Voltus InsightAI Option) license.

Parameters

<code>-help</code>	
	Outputs a brief description that includes the type and default information for each <code>get_ir_insight</code> parameter. For a detailed description of the command and all of its parameters, use the man command <code>man get_ir_insight</code>.
<code>-cap_lower_limit value</code>	
	Specifies the minimum load capacitance value driven by aggressor candidates in the <code>eco_report</code> mode.
<code>-cap_upper_limit value</code>	
	Specifies the maximum load capacitance value driven by aggressor candidates in the <code>eco_report</code> mode.
<code>-domain domainname</code>	
	Specifies the name of the power domain.
<code>-domain_threshold value</code>	
	Specifies the minimum allowed voltage for the domain being analyzed. The value will be in percentage. This parameter behavior is same as the <code>set_rail_analysis_domain -threshold</code> parameter.
<code>-eco_list list_of_instances</code>	
	Specifies the list of instances which were changed for <code>-mode predict_ir</code>. The format of the file is: <i><original x>,<original y>,<original orientation>,<original cell>,<x>,<y>,<orientation>,<cell>,<instance></i>
<code>-eco_mode {violation aggressor both}</code>	
	Default: <code>violation</code> Specifies the mode to be used for generating the ECO report with the <code>-mode eco_report</code> parameter. The possible options are: <code>violation</code>: Generates a list of violations in the report. <code>aggressor</code>: Generates a list of aggressors in the report. <code>both</code>: Generates a report that includes both the violations and aggressors.
<code>-ir_drop_histogram_bin_size absolute_value_in_volts</code>	
	Allows you to control the EIV histogram bin size. The default value is 0.005. In the following example, each histogram bin size is of 1mV: <pre>get_ir_insight -eiv_histogram_bin_size 0.001</pre>
<code>-ir_drop_histogram_lower_limit absolute_value_in_volts</code>	
	Specifies the lower limit of the EIV histogram range. This parameter must be used with <code>-report_mode summary_only</code> .
<code>-ir_drop_histogram_upper_limit absolute_value_in_volts</code>	

	Specifies the upper limit of the EIV histogram range. This parameter must be used with <code>-report_mode summary_only</code> .
<code>-mode {eco_report predict_ir report_aggressor report_by_region report_only repair_directive_only report_and_repair_directive}</code>	
	Default: <code>report_only</code> Specifies the mode of operation for the command. The possible options are: eco_report: Generates a file (<code>eco.ird</code>) that is used by the Tempus-ECO flow to fix the IR drop violations in a design. The <code>eco.ird</code> file is located at: <code>/output_directory/eco.ird</code>. When this option is specified, it is recommended to specify the <code>-eco_mode</code> and <code>-pdn_model</code> parameters to support the intended ECO flow. predict_ir: Specifies to generate the IR prediction for the ECO changes specified with the <code>-eco_list</code> parameter. Based on the list of instances specified with <code>-eco_list</code>, the tool performs IR prediction. You can also choose to use the <code>-watch_box</code> and <code>-watch_inst</code> parameters to specify either the area or instance you want to observe and predict. You can specify multiple ECO change lists, and all the previous accumulated ECO changes will be saved until you reset them using the <code>-reset_predict</code> parameter. This option is associated with the <code>-eco_list</code>, <code>-watch_box</code>, <code>-watch_inst</code>, <code>-reset_predict</code>, and <code>-pdn_model</code> parameters. The format of prediction output is: <pre><instance_name1> <predicted_eiv_value1> <instance_name2> <predicted_eiv_value2></pre> report_aggressor : Reports the impact of the user-specified aggressor instances. The report file is located at: <code>/output_directory/IR_impact_<nworst_instances>_<nworst_aggressors>.rpt</code>. This option is combined with the <code>-nworst_instances</code>, <code>-nworst_aggressors</code>, and <code>-pdn_model</code> parameters. The following is the report format: <pre>VIOL:<viol name> <violating ir(domain drop-threshold)> <EIV time> -AGGR1:<aggr1 name> <impact1> -AGGR2:<aggr2 name> <impact2></pre> report_by_region: Reports IR drop values by region in the report file (<code>output_directory/report.rpt region_*.rpt</code>). This option is combined with the <code>-nregion</code> and <code>-nworst_instances</code> parameters. report_only (default): Specifies to generate only the IR drop reports. When this argument is used, you can specify the type of report required using the <code>-report_mode</code> parameter. repair_directive_only: Specifies to generate the InsightAI database from the rail analysis results. report_and_repair_directive: Specifies to generate both the reports and InsightAI database from the rail analysis results.
	<code>-nregion value</code>
	Default: 3 Specifies the number of regions to be analyzed.

<code>-nworst_aggressors value</code>	
	<i>Default: 50</i> Specifies the number of aggressors that should be reported for each violation.
<code>-nworst_instances value</code>	
	<i>Default: 5</i> Specifies the number of worst instances to be reported.
<code>-output_directory directoryname</code>	
	Specifies the output directory in which the insight (IR drop and EM violation) results will be saved in.
<code>-pdn_model directoryname</code>	
	Specifies the rail state directory containing the sensitivity database. You can enable sensitivity database generation by specifying <code>set_rail_analysis_mode -create_sensitivity_db true</code>. The <code>-pdn_model</code> parameter must be specified with the <code>-mode {report_aggressor predict_ir}</code> and the <code>-eco_mode {aggressor both}</code> parameters.
<code>-report_mode {summary_only detailed}</code>	
	Controls the type of reports generated. The possible arguments are: <code>summary_only</code> (default): Includes the summary report only (saved in REPORTS/Design/Summary_Report). <code>detailed</code>: Includes both summary and detailed reports. The detailed reports instances based on domain, ranking, and range. This parameter must be specified with the <code>-mode report_only report_and_repair_directive</code> parameter. The parameter should not be specified with the <code>-mode repair_directive_only</code> parameter.
<code>-reset_predict</code>	
	Specifies to reset all the previous ECO changes. If this parameter is not specified, the previous changes in <code>-eco_list</code> will be retained for the <code>-mode predict_ir</code> runs.
<code>-state_directory directoryname</code>	
	Specifies the top-level state directory that is generated by the dynamic IR drop analysis. This state directory contains analysis of all nets associated with the power domain. This is the output directory in which the results of the <code>analyze_rail</code> command is stored (for example, <code>core_25C_dynamic_1</code>).
<code>-watch_box filename</code>	
	Specifies a file containing the list of regions, wherein only the instances inside these regions will be reported for prediction. You can write multiple regions in the file. This parameter is used with the <code>-mode predict_ir</code> parameter. The format of the file is: <pre>{{x1,y1}{x2,y2}} {{x3,y3}{x4,y4}}</pre>
<code>-watch_inst filename</code>	

Specifies a file containing the list of instances to be reported for prediction. You can write multiple instances in the file. This parameter is used with the `-mode predict_ir` parameter.

The format of the file is:

instance_1

Instance_2

Examples

- The following command specifies to perform IR prediction/estimation:

```
get_ir_insight -mode predict_ir \  
-state_directory Rail/latest \  
-pdn_model Rail/latest \  
-eco_list eco_five \  
-domain PD \  
-domain_threshold 0.1 \  
-watch_box region_watch\  
-watch_inst instance_watch\  
-reset_predict \  
-output_directory predictIR
```

The generated report will be saved at: *predict.out*

The following is the sample report:

```
CTS_buf_01236 0.828523  
CTS_buf_01248 0.829177  
CTS_buf_02745 0.817085
```

- Specifies to generate the eco.ird file for Tempus IR-ECO consumption:

```
get_ir_insight -mode eco_report \  
-state_directory Rail/latest \  
-eco_mode both \  
-domain PD \  
-domain_threshold 0.1 \  
-pdn_model Rail/latest \  
-output_directory debugIR
```

The generated report will be saved at: *debugIR/eco.ird*

The following is the sample report:

```
PGNET START  
  NET   VDD  0.900  
  NET   VSS  0.000  
PGNET END  
  
REGION START  
  INST ua/FE_n10786  1  
    IRDROP VDD:VSS  8.000e-11  0.1307  0.000e+00  7.029e-04  3.315e-03  
  INST ua/FE_n6442  1  
    IRDROP VDD:VSS  8.000e-11  0.1390  0.000e+00  7.452e-04  3.666e-03
```

- Specifies that the EIV histogram range is from 0.095 mV to 0.12 mV:

```
get_ir_insight -domain_threshold 0.1 -output_directory debug_ir -state_directory  
{rail_after/ALL_125C_dynamic_1} -domain ALL -report_mode summary_only -ir_drop_histogram_bin_size  
0.006 -ir_drop_histogram_lower_limit 0.095 -ir_drop_histogram_upper_limit 0.12
```

Sample Summary Report

Found 8826 instances with IR-drop violations in design based on eiv_method: Worst and eiv_eval_window: elapse.

Summary For All Cells :

Worst IR drop : 0.12337V

Specified IR drop threshold : 0.09V

Worst IR drop violation : 0.033327V

Total IR drop violation : 48.94V

IR drop violation distribution

Violation Range	Violation Count	Accumulated Violation count	Total Violation(V)	Accumulated Total Violation(V)
120mV ~ 125mV:	6	6	0.186857	0.186857
115mV ~ 120mV:	34	40	0.918714	1.10557
110mV ~ 115mV:	83	123	1.81664	2.92221
105mV ~ 110mV:	194	317	3.32192	6.24413
100mV ~ 105mV:	904	1221	10.5601	16.8043
95mV ~ 100mV:	2940	4161	21.3391	38.1433
90mV ~ 95mV:	4665	8826	10.7967	48.94

Related Topics

- [*"Voltus InsightAI"*](#) in *Innovus User Guide*

get_voltus_insight_mode

```
get_voltus_insight_mode
[-help]
[-config filename]
[-timing_protection_config filename]
```

Generates a configuration template after loading the design database and specifying the power/rail settings. This template is created based on the current rail settings.

Parameters

-help	Outputs a brief description that includes the type and default information for each <code>get_voltus_insight_mode</code> parameter. For a detailed description of the command and all of its parameters, use the man command <code>man get_voltus_insight_mode</code> .
-config filename	Specifies the name for the configuration file to be generated. This optional file includes details about the PG network definition, EIV method, EIV evaluation window, and a list of scenarios with their settings. By default, the configuration file is named <code>insight.config</code> . If a file with this name already exists, the suffix is automatically incremented (example, <code>insight.config1</code> , <code>insight.config2</code> , and so on).
-timing_protection_config filename	Specifies to automatically generate the timing protection configuration file. This file includes the timing degradation thresholds per timing view. The default file name is <code>insight_timing_protection.config</code> . This parameter allows you to specify the name of the file, where the timing protection configuration will be generated.

Examples

- The following command specifies to generate a configuration template for the Voltus InsightAI flow:

```
get_voltus_insight_mode
```

The following message appears:

```
Config file insight.config will be generated
```

Following is a snippet from the generated `insight.config` file:

```
#### PG Network Definition ####Nets VDD VSS

Domain_1 ALL VDD VSS

#### Global Options ####eiv_eval_window {}eiv_method {}#### Scenario Definition ###SCENARIO_DEFAULT:
SCENARIO_1 SCENARIO_2#### Scenarion Setting ###SCENARIO_1 eiv_eval_window elapse eiv_method
Worst Domain Domain_1 threshold_data 0.4 threshold_clock 0.4 Net VSS
threshold_data 0.09 threshold_clock 0.09 Net VDD threshold_data 0.81
threshold_clock 0.81SCENARIO_2 eiv_eval_window switching eiv_method WorstAvg Domain Domain_1
threshold_data 0.4 threshold_clock 0.4 Net VSS threshold_data 0.09
threshold_clock 0.09 Net VDD threshold_data 0.81 threshold_clock 0.81
```

Related Topics

- "[Voltus InsightAI](#)" in [Innovus User Guide](#)*

read_power_rail_results

```
read_power_rail_results
[-help]
[-cell_library { library1.cl}]
[-custom_value_inst_file_1 file]
[-data_type {rail | all}]
[-detail_delta_temperature_file file]
[-die_instance_name dieinstname]
[-effective_resistance_file file]
[-enable_plots plot_names]
[-esd_directory dir]
[-esd_rj dir]
[-force]
[-instance_delta_temperature_file file]
[-instances instancenames]
[-instance_voltage_method { best | worst | avg | worstavg }]
[-instance_voltage_window { timing | whole }]
[-nets net_names]
[-physical_db dir_name]
[-power_db file ]
[-power_directory power_output_directory]
[-profiling_power_file file]
[-rail_directory dir]
[-reset]
[-sem_db file]
[-short_file file]
[-thermal_file filename.txt]
[-tile_delta_temperature_file file]
[-tile_thermal_config_file filename]
[-top_cell_name topcell_name]
```

Loading power/rail analysis results into GUI.

Parameters

- help	Outputs a brief description that includes type and default information for each read_power_rail_results parameter. For a detailed description of the command and all of its parameters, use the man command: man read_power_rail_results.
-cell_library { library1.cl}	

	Specifies the single cell library that is required to load the state directory. The specified cell library must contain the technology layer information that is required to load the state directory. Therefore, you do not need to load all cell libraries used during analysis.
<code>-custom_value_inst_file_1 file</code>	
	Specifies the name of the custom value instance file (CVIF). This file can be used to display a user-specified instance voltage file in GUI. The format of the CVIF file is: <pre>value1 instancename1 value2 instancename2</pre> where, <i>value1</i> and <i>value2</i> are voltage values mapping to the specified instance. You can use the <code>cvil</code> argument of the <code>set_power_rail_display -plot</code> command parameter to view the EIV map in GUI.
<code>-data_type {rail all}</code>	
	Controls the type of data to be loaded in the GUI. The possible arguments of this parameter are: • <code>rail</code> - Loads only the rail analysis results. • <code>all</code> - Loads both the power and rail analysis results. This is the default value.
<code>-detail_delta_temperature_file file</code>	
	Specifies the name of the detailed wire-based delta temperature file from self-heating analysis.
<code>-die_instance_name dieinstname</code>	
	Specifies the name of the die whose results you need to view. This parameter is only applicable to the multi-die analysis flow (<code>set_rail_analysis_mode -die_mode multi-die</code>). The <code>-die_instance_name</code> parameter must be specified with the <code>-rail_directory</code> parameter (multi-die rail directory).
<code>-effective_resistance_file file</code>	
	Specifies the result file from effective resistance analysis (only instance-based result is supported). When the rail analysis plot type <code>effr</code> is selected and a result file is specified, the layout shows effective resistance analysis result for each instance. The <code>-effective_resistance_file</code> parameter is not supported in the Voltus-XP flow. Note: Currently, this feature only supports loading one net at a time.

<code>-enable_plots <i>plot_names</i></code>	
	Specifies the list of power or rail plots names that are to be loaded.
<code>-esd_directory <i>dir</i></code>	
	Specifies the Electrostatic Discharge (ESD) analysis directory.
<code>-esd_rj <i>dir</i></code>	
	Specifies the name of the directory where the current density/EM results generated by a previous ESD analysis run reside. This parameter allows you to selectively load the required EM result directory if there are multiple ESD EM result directories under the <code>ESD</code> directory. You must either load the rail directory (<code>-rail_directory</code>) before running this command, or specify the rail directory with the <code>-esd_rj</code> parameter.
<code>-force</code>	
	Specifies to block the pop-up window that appears when the <code>read_power_rail_results -reset</code> parameter is specified. The pop-up window displays a confirmation message for resetting the power and rail directories. Example: <pre>read_power_rail_results -reset -force</pre>
<code>-instances <i>instancenames</i></code>	
	Specifies a list of instances to load in the GUI. This parameter loads only those partitions that include the specified instances. The <code>-instances</code> parameter allows you to analyze parts of the designs, thereby reducing the loading time and memory footprint. The <code>-instances</code> parameter is applicable to the Voltus-XP flow only.
<code>-instance_delta_temperature_file <i>file</i></code>	
	Specifies the name of the instance-based delta temperature file from self-heating analysis. This is the device instance temperature.
<code>-instance_voltage_method { best worst avg worstavg }</code>	
	Loads the instance voltage data for the specified EIV computing method. If this parameter is not specified (default), the instance voltage results for all the four methods are loaded.
<code>-instance_voltage_window { timing whole }</code>	
	Loads the instance voltage data for the selected EIV window. If this parameter is not specified (default), the instance voltage results for both the timing and switching windows, and the entire rail simulation are loaded.
<code>-nets <i>net_names</i></code>	

	Loads only the specified nets in the domain-based state directory to the GUI. If this parameter is not specified with the <code>-rail_directory</code> parameter, all the nets will be loaded.
<code>-physical_db dir_name</code>	
	Specifies the name of the hierarchical design directory to be loaded to the GUI along with the rail analysis results.
<code>-power_db file</code>	
	Specifies the power database file. This file is created by using the <code>-create_binary_db</code> option of the <code>set_power_analysis_mode</code> command.
<code>-power_directory power_output_directory</code>	
	Specifies to load the power analysis results and display the power plots without loading the rail analysis results.
<code>-profiling_power_file file</code>	
	Specifies to display the tile-based power density results generated from the vector profiling flow.
<code>-reset</code>	
	Frees the loaded analysis state directory or power database from the memory.
<code>-rail_directory dir</code>	
	Specifies the rail analysis directory. You can load all the domain results by selecting the automatically generated analysis directory under the output directory (<code><domain>_<temp>_<dynamic/avg>_#</code>), or load individual nets by going one level further and selecting the net directory.
<code>-sem_db file</code>	
	Specifies to read the signal electromigration data from the specified database file generated by the <code>verify_AC_limit -report_db</code> command parameter.
<code>-short_file file</code>	
	Specifies the path to the short violations report (<code>voltus_short.report</code>). This file is generated using the <code>set_rail_analysis_mode -report_shorts true</code> command parameter, and is located in the work directory.
<code>-thermal_file filename.txt</code>	

	Specifies to load the tile-based temperature map file generated by Sigrity Celsius to be used for both power calculation and IR drop analysis. The following is an example of reading the thermal map file: <pre>read_power_rail_results -rail_directory output/static_rail/core_25C_avg_1/ -thermal_file output/power_map_by_layer.txt set_power_rail_display -plot thermal</pre>
	<code>-tile_delta_temperature_file file</code>
	Specifies the name of the tile-based delta temperature file of self-heating analysis. This is the worst instance temperature among all wire segments in a tile. This is also layer-based. The FEOL or cell layer is used to show the device instance temperature.
	<code>-tile_thermal_config_file filename</code>
	Specifies to load the configuration file that contains the mapping between the hierarchical levels with corresponding layer-based temperature map files generated by Celsius (system-level thermal analysis tool). When specified, you can view tile-based temperature distribution for each layer of the chip. The following is the format and example of the hierarchical configuration file: <pre><hierarchy> <block name> <X_tile_no,Y_tile_no> <LEF/DEF Flip/Rotation> <temperature map> TOP DIE 0,0 N /thermal_data/DIE_TemperatureMap TOP/IP1 IP1 25,50 N /thermal_data/IP1_TemperatureMap TOP/IP1/B1 B1 50,125 N /thermal_data/B1_TemperatureMap TOP/IP1/B2 B2 200,125 S /thermal_data/B2_TemperatureMap</pre> This is a mandatory parameter for the <code>tile_thermal</code> plot when loading hierarchical temperature maps.
	<code>-top_cell_name topcell_name</code>
	Specifies the name of the top module or top cell name to be loaded. This parameter must be specified with the <code>-physical_db</code> parameter.

Examples

- The following command specifies the appropriate files needed for viewing the instance voltage and power data:

```
read_power_rail_results\  
-rail_directory analysis/domain_25C_avg_1 \  
-power_db powermeter.db \  

```

- The following command specifies to load the VDD and VSS nets for the worst and best EIV methods during the entire simulation:

```
read_power_rail_results -rail_directory dynamic_rail/core_25C_dynamic_1 \  
-nets { VDD VSS } -instance_voltage_window { whole } -instance_voltage_method { worst best }
```

- The following command specifies to load both the rail directory and the ESD EM result directory:

```
read_power_rail_results -rail_directory dynamic_rail/core_25C_dynamic_1 -esd_rj  
em_rule_all_nets_VDD_VDDvsrcl_VSS_VSSvsrcl
```

- The following command specifies to load only the rail directory:

```
read_power_rail_results -rail_directory dynamic_rail/core_25C_dynamic_1 -data_type rail
```

- The following command specifies to load the hierarchical database and the rail analysis results to the GUI:

```
read_power_rail_results -rail_directory ./dynamic_rail/latest/ -physical_db /dma/Dynamic/saveDB -  
top_cell_name dma_mac
```

- The following command specifies to load the thermal configuration file and the rail directory in the XP mode:

```
read_power_rail_results -rail_directory xp_rail_directory -tile_thermal_config_file  
./thermal_data/DIE_Temperature_config  
set_power_rail_display -plot tile_thermal
```

- The following command specifies to display the tile-based power density results in the GUI:

```
read_power_rail_results -profiling_power_file ./vectorProfileResults/profile.rpt.density
```

- The following command specifies to load the power analysis results and display the power plots directly by running the power analysis without loading the rail analysis results:

```
read_power_rail_results -power_directory ./dynamic_power
```

Related Topics

- ["Static Power, IRdrop and EM Analysis"](#) in the *Voltus User Guide*

report_power_rail_results

```
report_power_rail_results
[-help]
[-append]
[-enable_filter_summary]
[-filename file ]
[-filter_max max_value]
[-filter_min min_value]
[-ignore_limit_bound]
[-layers {all | layer_names } ]
[-limit N ]
[-nets {all | all_pwr | all_gnd | net_names}]
[-plot plot_type]
[-range_max max_value]
[-range_min min_value]
[-region x1 y1 x2 y2]
[-rlrp_inst {instance_name}]
```

Generates RLRP, region-based, and rail/power/capacitance reports.

Parameters

-help	Outputs a brief description that includes type and default information for each <code>report_power_rail_results</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <code>man report_power_rail_results</code>.
-append	Appends instance reports to the current report file.
-enable_filter_summary	
	Displays a summary of the filtered data in the histogram given in the <i>Summary</i> tab of the Result Browser form.
-filename <i>file</i>	
	Specifies the name of the text based violation report file to be created. The default file name is: • for region-based report: <code>region_x1_y1_x2_y2.rpt</code> • for RLRP report: <code>rlrp_instname.rpt</code> • for plot-based report: <code>power_rail_plot.rpt</code>

<code>-filter_max max_value</code>	
	Specifies the maximum value to include in the report. Use this option to further filter out the data within the range specified. This is used to obtain statistic of filter range. Optional, but if specified, the <code>-plot</code> option is required, and requires both <code>-filter_min</code> and <code>-filter_max</code> to be specified.
<code>-filter_min max_value</code>	
	Specifies the minimum value to include in the report. Use this option to further filter out the data within the range specified. This is used to obtain statistic of filter range. Optional, but if specified, the <code>-plot</code> option is required, and requires both <code>-filter_min</code> and <code>-filter_max</code> to be specified.
<code>-ignore_limit_bound</code>	
	Specifies to ignore the maximum upper limit of 10000, and instead use the higher value given with the <code>-limit</code> parameter for specifying the number of data points to be printed in the power/rail report.
<code>-layers {all, layer_names }</code>	
	Specifies the layers to be reported. You can use <code>all</code> for reporting all layers, or list the specific layers that are to be reported. If this parameter is specified, you must specify the <code>-plot</code> parameter.
<code>-limit N</code>	
	Specifies the number of data points printed in the report. If this parameter is specified, you must specify the <code>-plot</code> parameter. Default : first 1000 data points
<code>-nets {all all_pwr all_gnd net_names}</code>	

	Specifies the nets to be used for analysis reporting. You can use <code>all</code> for reporting all nets, <code>all_pwr</code> for reporting all power nets, <code>all_gnd</code> for reporting all ground nets, or list the specific nets that are to be reported. If this parameter is specified, you must specify the <code>-plot</code> parameter. This parameter also allows you to specify the switched and always-on nets to be used for analysis reporting. The following is the use model to generate a report for the specified switched net: <pre>report_power_rail_results -plot ir -nets {switchedNetName}</pre> The following is the use model to generate a report for the specified always-on net, including its switched nets: <pre>report_power_rail_results -plot ir -nets {alwaysOnNetName} -all</pre> You must specify <code>-all</code> to include all switched nets associated with the specified always-on net.
<code>-plot plot_type</code>	
	Plots the specified plot type. See Plot Types for a description of each of the plot types.
<code>-range_max max_value</code>	
	Specifies the maximum value to report. Optional, but if specified, the <code>-plot</code> option is required, and requires both <code>-range_min</code> and <code>-range_max</code> to be specified.
<code>-range_min min_value</code>	
	Specifies the minimum value to report. Optional, but if specified, the <code>-plot</code> option is required, and requires both <code>-range_min</code> and <code>-range_max</code> to be specified.
<code>-region x1 y1 x2 y2</code>	
	Measures tap current and capacitance information in the specified region defined by x1, y1, and x2 y2 coordinates, and prints results in the log file.
<code>-rlrp_inst instance_name</code>	
	Reports the RLRP path for the specified instance. The default file name of the RLRP path report is <code><rlrp_instname.rpt></code>. You can modify the file name using the <code>-filename</code> parameter.

Plot Types

Plot Type	Description
ac	Available/feasible capacitance (Farad) Displays the feasible decoupling capacitance that can be added in the area. The feasible capacitance in the area is calculated based on the filler cells placement. The program matches the placed filler cells with the decap cells available in the power-grid library and computes the feasible capacitance. The capacitance of the decap cell is stored in the power-grid library.
activity	Displays the worst activity for the output and inout pins of every instance.
cap	Grid capacitance (Farad) Plots the capacitance of the extracted power-grid network.
dd	Decoupling capacitance (decap) density (Farad/μm^2) Plots decap density of the design. The decap density is the cell internal decoupling capacitance in the unit area (μm^2).
ivdn	Instance Voltage Drop (Volts) – net-based Plots the net-based instance voltage drop.
ivdd	Instance Voltage Drop (Volts) – domain-based Plots the domain-based instance voltage drop. For domain based, at least one power and one ground net must be selected.
fd	Filler cell density ($1/\mu\text{m}^2$) Displays the filler cell density per unit area (μm^2). The plot can be used to analyze the feasibility of adding decap cells by replacing the filler cells and not changing the placement of the logic cells.
freq	Frequency domain (Hz) Displays the associated clock frequency of all instances in the design. The instances associated with multiple clock domains are displayed using the fastest clock frequency. This plot can be used to analyze and debug the power distribution in the design.
ip	Instance total power (mW) The total power consumed by each instance is displayed. The total power is the sum of the internal, switching and leakage power of the instance. The power data is read from the power database and can be specified using the <code>-power_db</code> option or it is queried directly from memory, if power calculation is run during the session.
ip_i	Instance internal power (mW) The power consumed by charging and discharging of the internal interconnect and device capacitances of the instance.

ip_l	Instance leakage power (mW) The power consumed by devices when not switching.
ip_s	Instance switching power (mW) The power consumed by charging and discharging of the interconnect or output load.
ipd_i	Specifies internal power density in $\text{mW}/\mu\text{m}^2$.
ipd_l	Specifies leakage power density in $\text{mW}/\mu\text{m}^2$.
ipd_s	Specifies switching power density in $\text{mW}/\mu\text{m}^2$.
ipd_t	Specifies total power density in $\text{mW}/\mu\text{m}^2$.
ip_tr	Displays toggle rates for all instances. The toggle rate for an instance is calculated as the average of toggle rates of all outputs of the instance. This plot allows you to determine the design area that toggles more than the others.
ir	IRdrop or voltage drop (Volts) Displays the IRdrop across each extracted power-grid segment in the design.
jrms	Specifies current density as j/j_{max} based on the Irms current. This plot is only supported for dynamic analysis. A jrms gif file will be saved in the state directory that shows where the Irms based EM violation occurred. This plot type allows you to verify the analysis results of "jrms" using the GUI.
load	Loading capacitance (Farad) Displays the output loading capacitance driven by the instance. This plot can be used to analyze and debug the regions with high switching power.
pi	Power-switch (gate) current violation: I/I_{dsat} Displays the ratio of current through the power-switch and the saturation current (I_{dsat}) of the power-switch. This plot can be used to analyze and debug whether the current through power-switches exceeds the saturation current characterized for the power-switch devices. The saturation current and on-resistance of the power-switches are characterized and stored inside the power-grid library of the power-switch cell. A ratio of greater than 1 means that the current requirement of the power-gated block can not be met with placed power-switch instance. Either the power-switch needs to be made larger or power-switch instances need to be added.

pv	Voltage drop across power-switch (Volt) Displays the IRdrop across power-switch instances. This plot can be used to analyze and debug the IRdrop inside the power-gated block, e.g. if the IRdrop across power-switches is already high, the IRdrop inside the power-gated block will be much higher. In this case, the power-switch placement will have to be refined in order to resolve the IRdrop problem.
rc	Resistor current (Amp) Displays the current across power-grid resistor segments. This plot can be used to analyze and debug how the current flows from the voltage sources into the rest of the design, e.g. high current should flow from pads to the top-level routing level layers and then to the lower level metal layers. This plot can also be used to debug current density violations (r_j).
rlrp	The total resistance of the instance along the least resistive path. Displays the total resistance of the instance along the least resistive path, meaning the resistance of the instance along the path from where it gets the most amount of current. This plot can be used to analyze and debug the power-grid integrity early in the design stage. An instance with high resistance is likely to have high static or dynamic IRdrop depending on the vector that is simulated later in the design cycle. This plot can be also used during decap optimization, since regions with high dynamic IRdrop and high resistance may not be improved upon by adding decaps, because the effectiveness of decoupling capacitance depends on the resistance of the power-grid network. You can also highlight the least resistive path of the instance using the GUI and selecting the instance with left mouse button. The <i>RLRP Path</i> checkbox should be enabled.
res	Displays the total resistance of the least resistive path from each node to voltage source. This plot can be used to view high resistance regions of the power-grid. It is useful during IRdrop and decap analysis to identify weak power-grid connectivity.
rj	Resistor current density, J/Jmax. Displays the current density (current per unit area) violation on each power-grid segment using the ratio of calculated current density and maximum allowed current density. A ratio of greater than 1 means that the current density limit of the segment is violated. The current density limits are supplied by the foundry and can be read during analysis using the <code>set_rail_analysis_mode -em_models</code> command.
rs	Specifies the plot type for resistor sensitivity.

slack	Instance slack (Seconds) Displays worst instance slack. The slack data is queried from either the timing database or if the power database is specified using <code>-power_db</code> option, then the worst slack data is read from the power database. When using an external TWF, the slack data is read from the TWF.
tc	Tap current (Amp) Displays the current consumed by the instance devices or current taps. During rail analysis, it is based on the power consumption of the instance and is distributed inside the cell. The current distribution inside the cell is based on the extracted device netlist stored inside the power-grid library of the cell. The current distribution is generally based on the device width, length, and spice models. This plot can be used to analyze and debug IRdrop inside the macro.
td	Transition density (Hz) Displays the worst transition density (activity * frequency) of the instance. This plot can be used to analyze and debug activity distribution and high power density regions.
tpd	Displays power density map with user-specified grid size.
unc	Unconnected segments Displays power-grid segments that are disconnected from the voltage source (power pads). This plot can be used to debug unusually high IRdrop in the region.
vc	Voltage source current (Amp) Displays current through the voltage sources (power pads). This plot can be used to analyze the current carrying capacity of the power-pad.
vu	Voltage drop across package (Volt) Displays voltage drop across package. If analysis is run with package model attached to the power pads, this plot displays the IRdrop due to package RLCK elements.

Examples

- The following command will report the worst 100 IR segments on Metal 1 of the VDD net:
`report_power_rail_results -plot ir -layers M1 -limit 100 -nets VDD`
- The following command will report the IR drop of the VDD_ring switched net:
`report_power_rail_results -plot ir -net {VDD_ring}`
- The following command will report the IR drop of the VDD_AO always-on net, including its switched nets:
`report_power_rail_results -plot ir -net {VDD_AO} -all`
- The following command ignores the max limit of 10000 and prints 10008 values in the report file:
`report_power_rail_results -plot cap -limit 10008 -ignore_limit_bound`
- The following command filters the data such that it does not display the data points below 200 in the Result Browser histogram:
`report_power_rail_results -plot ivdd -filter_min 200 -enable_filter_summary`

Related Topics

- ["Static Power, IRdrop and EM Analysis"](#) in the *Voltus User Guide*

scale_what_if_capacitance

```
scale_what_if_capacitance
[-help]
-reset | {{-global | -region x1 y1 x2 y2 | -cell cell_name | -instance inst_name | -file filename}
[-intrinsic_cap value]
[-loading_cap value]
[-grid_cap value]
[-add]
[-layer {layername | all}]
[-net net_name | all]
}
```

Scales the intrinsic cell capacitance value to determine how much additional capacitance can be added in a region to reduce dynamic IR drop.

Parameters

-add	
	Specifies to add capacitance (farads) on the power-grid on a specific layer and region. The parameter works in conjunction with the existing parameters, such as <code>-intrinsic_cap</code> , <code>-grid_cap</code> , and <code>-loading_cap</code> . When this parameter is specified, the value provided with the <code>-intrinsic_cap</code> , <code>-grid_cap</code> , and <code>-loading_cap</code> parameters will be treated as absolute value instead of a scale factor value, which is the default behavior.
-cell cell_name	
	Specifies to scale the capacitance value for the specified cell.
-file filename	
	Specifies a file containing the list of instances and cells that use the same scaling factor. The format of the file is: CELL cellname INST instancename
-global -region x1 y1 x2 y2	
	You must specify either <code>-global</code> or <code>-region x1 y1 x2 y2</code> . The <code>-global</code> option scales the specified capacitances for the entire design. The <code>-region x1 y1 x2 y2</code> scales the specified capacitances in the region. The default unit of <code>x1 y1 x2 y2</code> is in database units.
-instance inst_name	
	Specifies to scale the capacitance value for the specified instance.

<code>-layer layername / all</code>	
	Specifies the LEF layer on which capacitance scaling is performed. Multiple commands can be issued to scale capacitance on multiple layers. You can specify the <code>all</code> argument to scale all the layers.
<code>-net net_name / all</code>	
	Specifies to scale the capacitance value for the specified net. Alternatively, you can specify the <code>all</code> argument to apply the scaling values across all nets simultaneously. With this argument, you can apply what-if capacitance data to all nets at once, rather than being limited to applying scaling values to a single net at a time.
<code>-help</code>	Outputs a brief description that includes the type and default information for each <code>scale_what_if_capacitance</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command <code>man scale_what_if_capacitance</code>.
<code>-reset</code>	Resets all the previous <code>scale_what_if_capacitance</code> command settings.
<code>-grid_cap value</code>	Scales power-grid capacitance of the design to the specified scale factor. The default scale value is 1.  The <code>-grid_cap</code> parameter can be specified either with the <code>-global</code> or the <code>-region</code> parameter. This parameter is not supported with the <code>-cell</code>, <code>-instance</code>, and <code>-file</code> parameters.
<code>-intrinsic_cap value</code>	
	Scales instance internal gate capacitances to the specified scale factor. The default scale value is 1.
<code>-loading_cap value</code>	
	Scales loading capacitance of the instance to the specified scale factor. The default scale value is 1.

Examples

- **Cell-based capacitance scaling** - The following command scales intrinsic capacitance on all instances of the cell *cellA* by 1e-10 ($\langle intrinsic_cap \rangle * 1e-10$):
`scale_what_if_capacitance -cell cellA -intrinsic_cap 1e-10`
- **Pin-based capacitance scaling** - The following command scales intrinsic and loading capacitance on all instances of the cell *cellA* connected to the pin *vdd* by 1e-10 ($\langle intrinsic_cap \rangle * 1e-10$ and $\langle loading_cap \rangle * 1e-10$):
`scale_what_if_capacitance -cell cellA -net vdd -intrinsic_cap 1e-10 -add -loading_cap 1e-10`
- **Region-based capacitance scaling** - The following command scales capacitance in the specified region by 1e-10 ($\langle intrinsic_cap \rangle * 1e-10$, $\langle loading_cap \rangle * 1e-10$, and $\langle grid_cap \rangle * 1e-10$):
`scale_what_if_capacitance -region {896 937 1276 1615} -intrinsic_cap 1e-10 -loading_cap 1e-10 -grid_cap 1e-10`
- The following command scales grid capacitance in the specified layer by 1e-10:
`scale_what_if_capacitance -net VDD -region { 0 0 100 100 } -layer M2 -grid_cap 1e-10`
- The following command scales intrinsic capacitance in all layers by 1e-4 for all nets:
`scale_what_if_capacitance -global -net all -layer all -intrinsic_cap 2e-3 -loading_cap 1e-4`

Related Topics

- "[What-If Rail Analysis](#)" in the *Voltus User Guide*

scale_what_if_current

```
scale_what_if_current
[-help]
[-hierarchy hier_name | -region x1 y1 x2 y2 | -global | -inst_list filename]
[-net net_name]
[-reset]
[-scale_clock_network]
[-scale value | -current value]
```

Scales current to do what-if analysis for a hierarchical partition and assess its effect on IR drop.

Parameters

-current value	
	Specifies current value in Amperes. This is used as target peak current for the given hierarchy and is then scaled appropriately. You cannot specify this option with -region x1 y1 x2 y2 or -global options.
-global	Specifies current scaling to be applied to entire design.
-help	Outputs a brief description that includes the type and default information for each <code>scale_what_if_current</code> parameter. For a detailed description of the command and all of its parameters, use the man command <code>man scale_what_if_current</code> .
-hierarchy hier_name	
	Specifies the name of the hierarchical block. For example, -hierarchy top/blockA. To specify global scaling for the design use "/". For example, -hierarchy / -hierarchy is a required parameter. This parameter allows instance-based current scaling by the specified scale factor. The parameter can be used to scale down peak current values. The use model of instance current scaling is: <code>scale_what_if_current -hierarchy <instance_name> -scale <value></code>
-net net_name	
	Scales current of the specified net.
-inst_list filename	

	Specifies to scale current for the specified nets. This parameter allows you to specify a file containing the list of instances and their scale factor. Use Model <pre>scale_what_if_current -inst_list filename -scale val</pre> The format of <i>filename</i> is: <pre>instanceName scaleFactor instanceName scaleFactor</pre> Here, <i>val</i> means the global scale factor. This value will be used if the local or instance-specific scale factor is not specified. <i>scaleFactor</i> means the local scale factor. If both global and local scale factors are specified, the local scale factor takes precedence. Default scale factor value is 1.
<code>-region x1 y1 x2 y2</code>	
	Specifies a region where current scaling is to be applied. The default unit of x and y coordinates is in microns (um).
<code>-reset</code>	Resets all the previous <code>scale_what_if_current</code> command settings.
<code>-scale value</code>	
	Specifies any value greater than 0. For example, a value of 0.5 will scale the current in half. This is a required parameter.
<code>-scale_clock_network</code>	
	When specified, scales current for all instances within the given hierarchy. When not specified, the software does not scale the instances that belong to the clock network. It scales current for the rest of the instances within the given hierarchy to the specified value. You cannot specify this option with <code>-region x1 y1 x2 y2</code> or <code>-global</code> options. This is an optional parameter.

Examples

- The following command scales current for the hierarchical block `BLOCK_C`:

```
scale_what_if_current -hierarchy top/BLOCK_C -current 0.5
```

Related Topics

- "[What-If Rail Analysis](#)" in the *Voltus User Guide*

scale_what_if_resistance

```
scale_what_if_resistance
[-help]
[-reset]
[-net netname]
[-global]
[-region x1 y1 x2 y2]
[-instance instancename]
[-cell cellname]
[-file filename]
[-scale value | -scale_ron {R_ON_scale_factor | R_OFF}]
[-auto_scale_adjacent_via_layers {true | false}]
[-layer {layername | all}]
```

Scales grid resistance globally or by region, or scales power switch resistance. In designs with power switches, the scaling of power switch resistance will be applied to cells in the always-on net as well as its switched nets in a given region.

Parameters

-auto_scale_adjacent_via_layers {true false}	
	Specifies to use auto via scaling for what-if resistance analysis. When scaling resistance on a specific metal layer, via resistance on the adjacent via layers are automatically scaled with the same scaling factor. The default value is <code>true</code>. Use <code>-auto_scale_adjacent_via_layers false</code> to keep via resistances un-scaled.
-global -region x1 y1 x2 y2 / -instance instancename / -cell cellname / -file filename	

	You must specify either <code>-global</code>, <code>-region</code>, <code>-cell</code>, <code>-instance</code>, or <code>-file</code>. - When the <code>-global</code> parameter is specified with the <code>-scale</code> parameter, it scales resistance by the specified value globally for a given layer. If the <code>-global</code> parameter is specified with the <code>-scale_ron</code> parameter, it scales the resistance value of all power switch cells globally for a given layer. This is the default. - When the <code>-region</code> parameter is specified with the <code>-scale</code> parameter, it scales the grid resistance in the specified region. If the <code>region</code> parameter is specified with the <code>-scale_ron</code> parameter, it scales the resistance of all power switch cells in the specified region. The default unit of x1 y1 x2 y2 is in database units. - The <code>-instance <i>instancename</i></code> or <code>-cell <i>cellname</i></code> parameter specifies to scale the internal resistors from the PGV for the specified instance or cell. - The <code>-file <i>filename</i></code> parameter specifies a file containing the list of instances and cells that use the same scaling factor. The format of the file is: <pre>CELL <i>cellname</i> INSTANCE <i>instancename</i></pre> This is a required parameter. Default is <code>-global</code>. The precedence for resistance scaling is as follows: 1. Instance 2. Cell 3. Region 4. Global
<code>-help</code>	Outputs a brief description that includes the type and default information for each <code>scale_what_if_resistance</code> parameter. For a detailed description of the command and all of its parameters, use the man command <code>man scale_what_if_resistance</code>.
<code>-layer <i>layername</i> / all</code>	
	Specifies the layer name (LEF layer or technology layer) on which the scaling is performed. Multiple commands can be issued to scale resistance on multiple layers. You can specify the <code>all</code> argument to scale all the layers. When the <code>all</code> argument is used, the <code>-auto_scale_adjacent_via_layer</code> parameter is disabled. This is a required parameter.
<code>-net <i>netname</i></code>	
	Scales resistance of the specified net in the domain which is being analyzed. This is a required parameter.  Specify always-on net name when scaling powergate on resistance.

<code>- reset</code>	Resets all the previous <code>scale_what_if_resistance</code> command settings.
<code>-scale value</code>	
	Specifies any value greater than 0. For example, a value of 0.5 will scale the resistance in half. This is a required parameter.
<code>-scale_ron {R_ON_scale_factor R_OFF}</code>	
	Specifies the value used to scale the RON of the power-gate instance/cell. The default value is 1. You can also specify the “R_OFF”(case insensitive) keyword to apply the off state power-gate resistance for the specified instance/cell.

Related Topics

- "[What-If Rail Analysis](#)" in the *Voltus User Guide*

set_advanced_package_options

```
set_advanced_package_options  
[-help]  
-reset |  
{[-tool_path <binary_path>] [-net_mapping_file <filename>]}
```

Specifies the path to the bin folder of package extraction and analysis tools. Also, a net mapping file needs to be specified. The mapping file is a simple two column format containing the die net name and the corresponding package net names. After running this command, use the `extract_package` command to start package model extraction.

Parameters

- help	Outputs the command usage and a brief description about the command parameters.
-tool_path <binary_path>	
	Specifies the full path to the bin folder of the package extraction and analysis tools (XtractIM and PowerDC).
-net_mapping_file <filename>	
	Specifies the name of the two column mapping file that contains the mapping between the net names of package with the net names of die.
-reset	
	Resets all specified options back to default values.

Example

- The following example specifies the path to the bin folder of package extraction and analysis tools:

```
set_advanced_package_options \  
-tool_path /home/sigrity/tools/bin/ \  
-net_mapping_file net_mapping.txt
```

set_advanced_rail_options

```
set_advanced_rail_options
[-help][-reset] | [[-voltus_rail_include_file_begin file_name1]
[-voltus_rail_include_file_end file_name2] ]
```

Provides the ability to include a file that provides a list of additional rail options that should be used for rail analysis. If you do not specify values to the `set_advanced_rail_options` command parameters, the parameter values are set to the default value, that is, empty file name.

For more information on these options, refer to appendix B: [Rail Analysis Options](#) of the *Voltus Text Command Reference*.

Parameters

-help	Outputs the command usage and a brief description about the command parameters.
-reset	Resets parameters to their default values. This means that an extra include file is not required.
<code>-voltus_rail_include_file_begin file_name</code>	
	Provides the ability to provide a file that includes additional rail options that will be run prior to the <code>analyze</code> command. This can be used for those legacy rail analysis commands that do not have a Voltus equivalent.
<code>-voltus_rail_include_file_end file_name</code>	
	Provides the ability to provide a file that includes additional rail options that will be run after the <code>analyze</code> command. This can be used for those legacy rail analysis commands that do not have a Voltus equivalent.

Examples

- The following command needs to be set to override the default precision digits:

```
set_advanced_rail_options -voltus_rail_include_file_begin vstorm.file
```

```
<vstorm.file>
```

```
setvar precision_digits 10
```

With this variable, you will get IR with 10 precision digits (10 includes all digits before and after the decimal point).

set_die_model

```
set_die_model
[-help]
-spice model_file
[-static_current {initial | average}]
[-subckt subcircuitname]
{-reset | [[-die_instance_name dieinstname [-spice model_file [-mapping mapping_file]]] ]
}
```

Specifies to import a die model on die Interposer or on package netlist.

This command gives information about the die model that will be used in the rail analysis of another die, wherein the two die are connected through die Interposer die or package netlist. For example, if you are performing IR drop analysis for a die (**die2**) on an Interposer/package design that is connected to another die (**die1**), and if the two die share the power/ground network, it is imperative to import the die model of **die1** during IR drop analysis of **die2**.

You can import multiple die models by specifying multiple `set_die_model` commands.

Parameters

<code>-die_instance_name dieinstname</code>	
	Mandatory parameter. Specifies the die instance name for die model. This is the die model of the die that needs to be imported.
<code>-help</code>	Outputs a brief description that includes the type and default information for each <code>set_die_model</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command <code>man set_die_model</code> .
<code>-mapping mapping_file</code>	
	Mandatory parameter. Specifies the name of the mapping file that contains the port connection mapping between the die model and voltage source of the chip to be analyzed. The two mapping file formats are: Mapping when die model on LEF/DEF-based Interposer Die <pre>#port_of_die_model cell die_name/VSRC_pad_name VDD_Bump_VDD_9_0 cell Interposer/Bump_DIE0_VDD_0 VDD_Bump_VDD_9_0 cell Interposer/Bump_DIE0_VDD_1 ...</pre> Mapping when die model on Package Design <pre>#port_of_die_model pkg port_of_pkg_netlist VDD_Bump_VDD_9_0 pkg VDD_3 VDD_Bump_VDD_9_0 pkg VDD_4</pre>

- reset	Resets all the previous <code>set_die_model</code> command settings to the default settings.
-spice <i>model_file</i>	
	Mandatory parameter. Specifies a file containing a SPICE subcircuit describing the die model.
-static_current {initial average}	
	Default: initial Specifies to use the average of PWL waveform as the static current for static rail analysis. By default, only the current value at $t=0$ from the PWL waveform is considered for static run. Example: <pre>set_die_model -static_current average</pre>
-subckt <i>subcircuitname</i>	
	Defines the top-level subcircuit in package files. When this parameter is not specified, the first circuit in the Spice file is used as the default top circuit.

Examples

- The following command imports two die models `Die1` and `Die2`:

```
set_die_model -die_instance_name Die1 -spice die1_model.ckt -mapping die1_port.txt
set_die_model -die_instance_name Die2 -spice die2_model.ckt -mapping die2_port.txt
```

Related Topics

- "[Package Analysis](#)" in the *Voltus User Guide*

set_dynamic_rail_simulation

```
set_dynamic_rail_simulation
[-auto_repeat]
[-reset]
[-resolution value]
[-start value]
[-stop value]
```

An optional command to set the parameters for dynamic rail analysis. By default, dynamic rail analysis uses the start time, stop time, and resolution from current data file. To analyze fewer clock cycles, you can choose different start and stop times using the `-start` and `-stop` options. It is recommended that at least one or two full cycles are simulated to fully charge and discharge the on-chip capacitance.

Parameters

-help	Outputs a brief description that includes the type and default information for each <code>set_dynamic_rail_simulation</code> parameter. For a detailed description of the command and all of its parameters, use the man command <code>man set_dynamic_rail_simulation</code>.
-auto_repeat	Specifies to repeat the dynamic current waveform till the <code>-stop</code> time specified. If <code>-stop</code> is not specified, all the waveforms will repeat till the longest waveform duration. The <code>-auto_repeat</code> parameter can be used when multiple dynamic current files have different simulation periods or when IR drop analysis needs to be run for longer duration. The following command sets the dynamic simulation time from 20ns to 100ns in which the current data will be repeated until 100ns is reached: <pre>set_dynamic_rail_simulation -start 20ns -auto_repeat -stop 100ns</pre> The <code>-repeat</code> parameter of the <code>set_power_data</code> command has higher priority than <code>set_dynamic_rail_simulation -auto_repeat</code>, as shown in the following example: <pre>set_power_data VDD1.ptiavg -repeat 50 set_power_data VDD2.ptiavg set_dynamic_rail_simulation -auto_repeat -stop 100</pre> In this example, VDD1 waveform will repeat till 50ns, while VDD2 waveform will repeat till 100ns.
-reset	Resets all the previous <code>set_dynamic_rail_simulation</code> command settings to the default settings.
-resolution value	

	By default rail analysis picks up the resolution (or step size) saved in the dynamic current files. To achieve better rail analysis performance, you can increase the step size using this option. However, using larger than two times the step size of current data file is not recommended as it can impact accuracy of dynamic IR drop analysis. You can specify the units with the value. If you do not specify a unit, the software uses the default unit. <i>Default:</i> Picoseconds (ps).
<code>-start value</code>	
	Defines the start time to be used for dynamic rail analysis. This option is used with the <code>-stop</code> option and enables you to perform a more detailed analysis in specific time ranges. You can specify the units with the value. If you do not specify a unit, the software uses the default unit. <i>Default:</i> The default unit is nanoseconds (ns). The default value is the start time from current data file. The start time in the current data file is usually 0ns.
<code>-stop value</code>	
	Defines the stop time to be used for dynamic rail analysis. This option is used with the <code>-start</code> option and enables you to perform a more detailed analysis in specific time ranges. You can specify the units with the value. If you do not specify a unit, the software uses the default unit. <i>Default:</i> The default unit is nanoseconds (ns). The default value is the end of the current data. If multiple current files are specified and they all have different simulation periods, the largest simulation period is used to decide the stop time.

Examples

- The following command sets the dynamic simulation time from 2ns to 12ns with a resolution of 100ps:

```
set_dynamic_rail_simulation -start 2ns -stop 12ns -resolution 100ps
```

set_multi_die_analysis_mode

```
set_multi_die_analysis_mode
-help
-name analysis_name
-die_list {die_instance_name1 die_instance_name2}
[-report_rlrp_path_instance_list filename]
[-stacked_die_mapping Innovus_map_file]
```

Specifies how the multi-die analysis will be performed.

Parameters

-help	Prints out the command usage.
-die_list <i>die_instance_name1 die_instance_name2</i>	
	Specifies the die instance names.
-name <i>analysis_name</i>	
	Specifies the name of the analysis.
-report_rlrp_path_instance_list <i>filename</i>	

Specifies a file containing a list of die instances (one per line) for which the RLRP path should be reported. This parameter is used to generate the RLRP path across multiple die.

The format of the specified input file is:

```
<die_name> <net_name>
```

The `-report_rlrp_path_instance_list` parameter generates a report `mdie_inst_rlrp_path.rpt` at the following location:

```
<output_dir>/<state_dir>/results/rail/.
```

The format of the report is:

```
Die Name: <die_name>
```

```
Net Name: <net_name>
```

```
Pin Name: <pin_name>
```

```
Instance Name: <inst_name>
```

```
Die   Layer/Via   Resistance(cumulative)   Ohm Length   Node1# (X,Y)
Node2# (X,Y)   Net
```

Example of an Output File:

```
Die Name: LDIE1
```

```
Net Name: VDD_AO
```

```
Pin Name: TVDD
```

```
Instance Name: ring/PSO_RING_78
```

```
Die Layer/Via   Resistance(cumulative) Ohm Length Node1# (X,Y) Node2#
(X,Y) Net
LDIE1 METAL_1    0.1238 (3.4182)        2.210   169.28500,233.27500
167.07500,233.27500 VDD_AO
LDIE1 METAL_1    0.1302 (2.7137)        2.035   169.28500,221.89000
169.28500,223.92500 VDD_AO
LDIE1 VIA_1      0.0269 (2.5835)        0.140   169.28500,221.89000
169.28500,221.89000 VDD_AO
LDIE1 METAL_2    0.0070 (2.5565)        0.145   169.28500,221.74500
169.28500,221.89000 VDD_AO
LDIE1 VIA_2      0.0875 (2.5496)        0.140   169.28500,221.74500
169.28500,221.74500 VDD_AO
LDIE1 VIA_3      0.0875 (2.4621)        0.140   169.28500,221.74500
169.28500,221.74500 VDD_AO
...
```

```
-stacked_die_mapping Innovus_map_file
```

Specifies the name of the inter die data mapping file from Innovus. The mapping file is a virtual mapping that provides the connections between any 2 stacked die with communication directly between the die (master/slave configuration).

Examples

- The following command specifies an analysis name called `tsv`, two die instances called `mother_die` and `daughter_die`, and a stacked mapping file `pwr.mapping`:

```
set_multi_die_analysis_mode -name tsv -die_list {mother_die daughter_die}  
-stacked_die_mapping ../data/pwr.mapping
```

Related Topics

- "[Through-Silicon Via and System-in-Package](#)" in the *Voltus User Guide*

set_net_group

```
set_net_group
-help
-name net_group_name
-reset
-type [power | ground]
-nets {net_list}
}
```

Specifies the list of either power or ground nets that are isolated but need to be analyzed together as a single net. The results are generated for the new grouped net. The new grouped net name must be one of the net names in the group. To analyze the grouped net, you must also specify power pads for the grouped net name.

Parameters

-help	Prints out the command usage.
<i>-name group_name</i>	
	Name of the net group. Note: The name of the net group must be one of the net names in the group. Using a net name that is not in the net group will trigger error when displaying results in the flow.
<i>-nets {net_list}</i>	
	Specifies a list of power or ground nets.
-reset	Resets all previously created net name(s) with the new grouped net name(s).
<i>-type [power ground]</i>	
	Specifies whether it is a power or ground net.

Examples

- The following example groups the power nets that will be analyzed together as a group called VDD.

```
set_net_group -name VDD -type power -nets { VDD VDD1 VDD2 VDD3 }
set_power_pads -net VDD -format xy -file VDD.pp
analyze_rail -type net VDD
```


set_offchip_package_trace

```
set_offchip_package_trace
[-help]
-mapping mapping_file
-name switched_name
-ploc_file_list list_of_bump_location_files
[-reset]
-spice spice_model_name
[-subckt subcircuitname]
```

Specifies to include RDL0 (off-chip package trace) resistance effects during the static and dynamic rail analysis. This flow allows you to analyze switched net bumps that are marked as RDL0 bumps. When this command is specified, the software automatically generates switched net bump current and bump voltage waveforms in the voltus_rail_pad.tran.ptiavg file created in the new switch net directory located under the rail state directory.

You can specify multiple set_offchip_package_trace commands to define connectivity of different bump nets.

Parameters

-help	Outputs a brief description that includes the type and default information for each set_offchip_package_trace parameter. For a detailed description of the command and all of its parameters, use the man command man set_offchip_package_trace.
-mapping mapping_file	
	Specifies the name of the mapping file that contains the mapping between the package port and switched net bump names. The format of the file is: <spice_circuit_port_name> pin <vsrcName_in_switch_net's_ploc>
-name trace_model_name	
	Specifies the trace model name or switched net bump of RDL0.
-ploc_file_list list_of_bump_location_files	
	Specifies the name of a file containing switched net names and associated ploc/power pad files. The format of the file is: SWITCHED_NET1 plocfile1 SWITCHED_NET2 plocfile2 ... The format of plocfile (power pad) is: <voltage_source_name> <x> <y> <metal_layer> The format of this file is same as specified for the set_power_pads -format xy format.

- reset	Resets all the previous <code>set_offchip_package_trace</code> command settings to the default settings.
-spice <i>model_file</i>	
	Specifies a file containing a SPICE subcircuit (RDL0 netlist or full-chip package netlist) for off-chip package trace.
-subckt <i>subcircuitname</i>	
	Defines the top-level subcircuit for off-chip package tracing.

Examples

- The following command specifies to analyze the switched net `VDDau`:

```
set_offchip_package_trace -name VDDau -ploc_file_list ploc_file_list.txt -spice pkg.ckt -mapping  
pkg_mapping.file
```

Related Topics

- "[Package Analysis](#)" in the *Voltus User Guide*

set_package

```
set_package
[-help]
-reset | {-spice model_file
[-mapping mapping_file]
[-offset {x y}]
[-die_instance_name dieinstname]
[-rotate rotation_angle_in_degrees]
[-flip {true | false}]
[-subckt subcircuitname]
[-scale scaling_factor_number]
[-mapping_type { distributed | lumped }]
}
```

Specifies the package model information to be used in power rail analysis.

Parameters

-die_instance_name dieinstname	
	Specifies the die instance name for multi-die analysis. This option is only available in multi-die analysis (set_rail_analysis_mode -die_mode multi-die).
-flip {true false}	
	Specifies to flip the die along the Y axis. <i>Default:</i> false
-help	Outputs a brief description that includes the type and default information for each set_package parameter. For a detailed description of the command and all of its parameters, use the man command man set_package.
-mapping mapping_file	
	Specifies the name of the package terminal mapping file, which contains the mapping of the terminals of the SPICE subcircuit in the package model file (Specified by the -spice model_file parameter) to the locations at the grid or top-level pins. The mapping file formats supported by the software are: • Model Connection Protocol (MCP) • Voltus-specific Format For more information on the mapping file formats, refer to the <i>Package Model and Mapping File</i> section in the "File Formats" chapter of <i>Voltus User Guide</i> .

<code>-mapping_type { distributed lumped }</code>	Specifies the type of mapping to be used in the Voltus-specific terminal mapping file that will be added in the rail analysis flow. The possible arguments are: • <code>distributed</code> – A distributed mapping file lists out the mapping for each bump on the die to its corresponding package port. The mapping file is specified with the <code>set_package -mapping</code> parameter. • <code>lumped</code> - A lumped mapping file allows for auto-grouping of the die bumps to the lumped package ports. When <code>lumped</code> is specified, the software implements a lumped package by internally creating a lumped package model and its mapping file. The format of the lumped mapping file is: <pre><terminal_name> <die vsrc> <net_name> vddcore_in die vddcore vddcore_out vsrc vddmpu</pre> In this format: ◦ the software connects “<terminal_name> die <net_name>” to the bumps belonging to <net_name> on the die ◦ the software connects “<terminal_name> vsrc <net_name>” to the voltage source of <net_name> ◦ The number of lines in the lumped mapping file are few and are equal to the number of terminals on the package model <i>Default:</i> <code>distributed</code>
<code>-offset {x y}</code>	Specifies to adjust and shift the X Y coordinates in the MCP header for each pin. When performing package analysis, the MCP is used to map the terminals of the SPICE subcircuit in the package model file to the locations at the grid or top-level pins in DEF. The die origin specified in MCP must be same as specified in DEF. If the die origin is not the same, you can use the <code>-offset</code> parameter to modify the XY location specified in the MCP file.
- reset	Resets all the previous <code>set_package</code> command settings to the default settings.
<code>-rotate rotation_angle_in_degrees</code>	

	Specifies the rotation angle of the die. The possible values are: 0/45/90/135/180/225/270/315. Rotation is always in the counter clockwise direction from the original point. Default value is 0. This information is required to do coordinate conversion for xy based pin mapping.
<code>-scale scaling_factor_number</code>	
	Specifies to scale the package model.
<code>-spice model_file</code>	
	Specifies a file containing a SPICE subcircuit describing the package model.
<code>-subckt subcircuitname</code>	
	Defines the top level subcircuit in package files.

Related Topics

- "[Package Analysis](#)" in the *Voltus User Guide*
- "[File Formats](#)" in the *Voltus User Guide*

set_pg_nets

```
set_pg_nets
-reset |
-net net_name
-voltage value
[-threshold value]
-tolerance value
-force
[-die_instance_name dieinstname]
[-lib_binding_voltage value]
[-pkg_net_name package_net_name]
[-vu_limit value]
```

Specifies the power grid nets. Using this command, you can specify power/ground net voltages in the static and dynamic power analysis flow, and the rail analysis flow.

i For the power analysis flow, you can also use the `set_dc_sources` command for specifying power/ground net voltages. In case the same net is specified through both the `set_dc_sources` and `set_pg_nets` commands, the `set_pg_nets` voltage value will take precedence. You must specify the `set_pg_nets` command before `report_power`. The `set_dc_sources` command requires the design to be loaded (`set_top_module`), whereas the `set_pg_nets` does not require design loading. Therefore, if you have an external TWF, you can run the static or dynamic power analysis without loading the design. If you specify the `set_pg_nets` command for both the power and rail analysis flows, both the flows will use the same net voltages.

Note: When specifying a ground net with a non-zero nominal operating voltage value, a threshold greater than the ground value must be specified in order for the net to be treated as the ground net.

Parameters

<code>-die_instance_name dieinstname</code>	
	Specifies the die instance name for multi-die analysis. This option is only available in multi-die analysis (<code>set_rail_analysis_mode -die_mode multi-die</code>).
<code>-force</code>	Skips the validity check on power nets.
<code>-help</code>	Outputs a brief description that includes the type and default information for each <code>set_pg_nets</code> parameter. For a detailed description of the command and all of its parameters, use the man command <code>man set_pg_nets</code> .
<code>-lib_binding_voltage value</code>	
	Specifies the voltage to be used for library binding in power analysis. This parameter binds the instances to the library that is closest to the user-specified library binding voltage.

<code>-net net_name</code>	
	Specifies the name of the power or ground net that you want to analyze.
<code>-pkg_net_name package_net_name</code>	
	Specifies the net name on the package side. By default, the package net name is same as the net name.
<code>-reset</code>	
	Specifies to clear all previously specified values for this command. This parameter can be used to honor the power and ground voltages defined in the Innovus database without impacting the regular EM-IR analysis.
<code>-threshold value</code>	
	Specifies the minimum allowed voltage on the power net or the maximum rise on the ground net, depending on the setting of the <code>-voltage</code> parameter. The <code>-threshold</code> parameter is used to verify that the power grid passes the limit check and to set the filters for an IR plot.
<code>-tolerance value</code>	
	Specifies the tolerance range of the voltage value. Specify a value from 0 to 1 (not including 1). A value of 0.2 results in a range of "voltage +20%" to "voltage -20%." <i>Default: 0.3.</i>
<code>-voltage value</code>	
	Specifies the voltage on the power rail to be analyzed.
<code>-vu_limit value</code>	
	Specifies the voltage drop limit for a net. This parameter allows you to specify the voltage threshold value for each net separately. The specified threshold value is used for the rail reports, histograms, and plots. If the threshold value is not specified for some of the nets being analyzed, the default range will be applied to those nets. <i>Default Range: 0mV to max VU</i>

Examples

- The following command specifies a VDD_TDSPCore value of 0.9 volts with a threshold of 0.89V and a tolerance of 0.3:

```
set_pg_nets -net VDD_TDSPCore -voltage 0.9 -threshold 0.890 -tolerance 0.3
```

- The following command specifies to use the library binding voltage for binding the libraries to the instances instead of `"-voltage"`:

```
set_pg_nets -net vdd -voltage 0.90 -lib_binding_voltage 0.75
```

set_power_data

```
set_power_data
-reset | {-format {current | ascii | area | ascii_current}
[-bias_voltage value]
[-cell cellname]
[-instance instance_name]
[-offset offset_value]
[-power value]
[-scale factor]
[-repeat time]
[-die_instance_name dieinstname]
[filename]
[-power_directory dir]
[-power_output_directory dir]
[-repeat_window {start end}]
[-strip_hier_instance instance]
}
```

Specifies the static and dynamic power or current files for IRdrop and EM analysis. The power data for the design can be generated using the `report_power` command. This command also has ability to read external ASCII static power files to do static rail analysis. For dynamic analysis, you must use the `report_power` command to generate dynamic current files.

This command should be specified prior to the `analyze_rail` command. You must specify the `set_power_data` command after the `report_power` command in the flow script if power data is not available.

Parameters

<code>-bias_voltage value</code>	
	Specifies the value of the bias voltage. This parameter must be set if -format is set to ascii or area.
<code>-cell cellname</code>	
	Specifies power from the block-level run to multiple instances of the specified block, so you need not to mention power for each instance separately. For example, <code>set_power_data -cell BlockA -format current {file1, file2, file3 ...}</code>
<code>-die_instance_name dieinstname</code>	
	Specifies the die instance name for multi-die analysis. This option is only available in multi-die analysis (<code>set_rail_analysis_mode -die_mode multi-die</code>).
<code>filename</code>	Specifies the name of the power data files. The file is optional if -format area is specified.

`-format {current | ascii | area | ascii_current}`

Specifies the format of the power data.

The power data format can be `current`, `ascii`, `area`, or `ascii_current`.

The parameter `-bias_voltage` must be set if `area` or `ascii` is specified.

- When using the current format option, specify the current file (*.ptiavg) that is generated by the power analysis software within Voltus. For example,

```
set_power_data -format current static_VDD.ptiavg
```

- When using the `ascii` format option, specify the ascii instance based power file, generated by the Power Calculation engine within Voltus. For example,

```
set_power_data -format ascii -bias_voltage 1.08 powermeter.pwr
```

Note: The command accepts an ASCII power file in three-column format consisting of *instance/cell name*, *power in W*, and *power pin name*. Lines starting with "#" or "" are treated as comment lines.

The format of the three-column input file is as follows:

```
<instance/cell name> <power in Watts> <power pin name>
```

Example:

```
top/inst1 0.001 vdd
top/inst2 0.001 vdd1
top/inst3 0.003 vdd3
```

where vdd, vdd1, and vdd3 are power pin names of the instance

 Although `-bias_voltage` is a required argument with `-format ascii`, when 3-column power file format is specified, the program will ignore `-bias_voltage` and will rely on associated net voltages to determine current for the pins.

- When using the `area` format option, specify the total power of the chip in Watts. This power is distributed based on cell area. Optionally, you can specify the known power for macro cells using a file, for example,

```
set_power_data -format area -bias_voltage 1.08 -power 10 cell_power.file
```

where,

`-bias_voltage` is the voltage at which power was characterized; If the analysis voltage is different, this power will be scaled accordingly.

`cell_power.file` is in two column format with *cell_name**power_in_watts*.

	When using the <code>ascii_current</code> format option, specify an ASCII current file for static analysis. The parameter accepts an ASCII file in the three-column format consisting of instance or cell name, current values in Amperes, and power pin name. The format of the three-column input file is as follows: <code><instance cell name> <current in Ampere> <power pin name></code>
<code>-help</code>	Outputs a brief description that includes the type and default information for each <code>set_power_data</code> parameter. For a detailed description of the command and all of its parameters, use the man command <code>man set_power_data</code>.
<code>-instance instance_name</code>	
	Specifies the hierarchical instance name for which the current files are included. This option is useful when power is calculated for hierarchical partitions of the design, enabling you to read power for the hierarchical partitions at the top-level. For example, <pre>set_power_data -format current -instance xy {dynamic_VDD.ptiavg dynamic_VSS.ptiavg}</pre>
<code>-offset offset_value</code>	
	Specifies the start time offset for current waveforms saved inside the current files. The default unit is in seconds; However, you can also specify the unit with the offset time value. This option is useful in the hierarchical flow to align current waveforms of various partitions. It can be also used to shift power-up rush current waveforms generated by the UltraSim software. For example, <pre>set_power_data -format current -offset 1ns -instance xy {dynamic_VDD.ptiavg dynamic_VSS.ptiavg}</pre>
<code>-power value</code>	
	Specifies the total power of the chip in Watts and is used with <code>-format area</code> option.
<code>-power_directory dir</code>	
	Specifies to read the power data from the specified directory for cell power. You can use this parameter to read <code>power_results.json</code> and automatically identify all the PTI files. If <code>power_results.json</code> is not found in the specified directory, then the power data could be retrieved from <code>dir/voltus_power_output.latest</code>. When specified, the <code>-power_directory</code> parameter takes precedence and bypasses the <code>-format</code> parameter. For example, <pre>set_power_data -cell top_design -power_directory ./power</pre>
<code>-power_output_directory dir</code>	
	Specifies the name of the directory that contains the current files required to run power analysis in parallel with rail analysis.

<code>-repeat time</code>	
	Repeats the dynamic current waveform for the specified time. You must ensure that <code>set_dynamic_rail_simulation -stop</code> is also specified and it matches the repeat duration. This option is only available for current format files during dynamic analysis. The repeat option can be used when multiple current files have different simulation periods or when IR drop analysis needs to be run for longer duration. See example below. <i>Default units: ns.</i>
<code>-repeat_window {start end}</code>	
	Specifies to extract the given window from the current waveform and repeat it. You can pick a window such that the current at the <code>start_time</code> and <code>end_time</code> are similar. This parameter allows you to identify and repeat windows where the start and end time have similar values to ensure that the repeated waveform is continuous and that there are no high current values. The unit for the start and end time is ns. The following command specifies that the pattern between {20ns 80ns} will be repeated until simulation stops: <pre>set_power_data -repeat_window {20ns 80ns}</pre>
<code>-reset</code>	Overwrites previously specified current data.
<code>-scale factor</code>	
	Specifies a scale factor to apply to the current data. You can use this parameter to scale currents to a more optimistic or pessimistic power-consumption estimate. This parameter is only for <code>-format current</code> and <code>-format ascii</code>.
<code>-strip_hier_instance instance</code>	
	Specifies to use power generated from the top-level rail analysis for block-level rail analysis. This parameter allows you to specify the top-level hierarchical instance name for which the current files are to be included. When the parameter is specified, the tool removes the prefix from the top-level and reads the top-level current (ptiavg) files during the block-level analysis. The parameter is useful when top-level power data is already available, therefore, you do not need to re-run block-level power analysis.

Examples

- The following command specifies a format of type current and a scale of 1 for the three given files.:

```
set_power_data -format current -scale 1 {static_VDD_TDSPCore.ptiavg \ static_VDD_TDSPCore_R.ptiavg
static_VSS.ptiavg}
```
- The next example specifies two current data files. The first file specifies `vdd_vdd.ptipeak`. The second one specifies `dynamic_VDD.ptiavg` and this current data will be repeated every 100ns, which is the stop time specified for the dynamic analysis.

```
set_power_data -format current vdd_vdd.ptipeak
set_power_data -format current -repeat 100ns dynamic_VDD.ptiavg
set_dynamic_rail_simulation -stop 100ns
```

`analyze_rail ...`

set_power_pads

```
set_power_pads
[-help]
-reset |{
[-auto_voltage_source_creation {true|false}]
[-consider_partial_small_tiles_for_vsrc {true | false}]
[-def_shape_only {true | false}]
[-die_instance_name dieinstname]
[-enable_switchnet_ploc_generation {true | false}]
[-format {defpin | padcell | xy | boundary | xyiv}]
[-layer vialayername]
[-net net_name / all]
[-number_of_voltage_source value]
[-file file]
[-region {x1 y1 x2 y2}]
[-short_pin_nodes {true | false}]
[-tile {x y}]
}
```

Specifies power pad information.

You can specify multiple power pad formats (_{xy} coordinates, pad cell, or power pin) for the same power net, in both, net-based and domain-based rail analysis.

 All voltage source names in the power pad location file must be unique for all nets in the domain, e.g. if voltage source name `vsrc1` is used in pad location file of net `VDD` as well as `VSS` in domain-based analysis, program will issue error and exit. In addition, the names of voltage sources in the power pad location file must be different from the names in the DEF pin section.

Parameters

-help	Prints out the command usage.
-auto_voltage_source_creation {true false}	
	Specifies to automatically create voltage sources either for all nets (-net ALL) or for a specific net (-net net_name). The voltage sources are created on the top metal of all via cuts located either on the user-specified metal layer or the design top metal layer (default). Default: false.
-consider_partial_small_tiles_for_vsrc {true false}	

	Specifies to honor voltage source generation on small tiles. Small tiles refer to tile areas less than 1/3 of the pitch*pitch area size. This parameter is used with the automatic voltage source creation feature (<code>-auto_voltage_source_creation</code>). The default value of this parameter is <code>false</code>. Example: <pre>set_power_pads -net VDD_AO -auto_voltage_source_creation true -layer VIA_5 - consider_partial_small_tiles_for_vsrc true -tile {90 90}</pre>
<code>-def_shape_only {true false}</code>	
	Specifies to generate voltage sources only on the DEF routing shapes. This parameter controls the generation of voltage sources on the cell instance shapes. When this parameter is set to <code>false</code>, voltage sources are created for both the DEF routing shapes and shapes inside cell instances. This parameter is specified with the <code>-auto_voltage_source_creation</code> parameter. Default: <code>true</code>
<code>-die_instance_name dieinstname</code>	
	Specifies the die instance name for multi-die analysis. This option is only available in multi-die analysis (<code>set_rail_analysis_mode -die_mode multi-die</code>).
<code>-enable_switchnet_ploc_generation {true false}</code>	
	Specifies to generate voltage sources on all switched nets. This parameter is specified with the <code>-auto_voltage_source_creation</code> parameter. Default: <code>false</code>
<code>-file file</code>	Specifies the name of the file containing the voltage source information. Not required for <code>-format defpin</code> option.
<code>-format {defpin padcell xy boundary xyiv}</code>	
	Specifies how the voltage source is determined. Required for <code>xy</code>, <code>padcell</code>, or <code>boundary</code>.
	<code>defpin</code> - Specifies that voltage sources will be determined by power pins.
	<code>padcell</code> - Specifies that voltage sources will be determined by pad cells.

	xy - Specifies that voltage sources will be specified by x y coordinates. You can also specify a PWL function on the X,Y location for a voltage source. When specified, Rail Analysis will read the power pad file and use the PWL waveform for the voltage source. Example: <pre>set_power_pads -format xy -file <xy_plocFile></pre> Add the PWL option in the xy file: <pre>*vsrsrc_name x y layer_name net_name [pwl=(t1,v1,t2,v2,t3,v3,...)] vsrsrc3 8.39 370.35 Metal15 VDD pwl=(0,0.6,20e-9,0.7,25e-9,0.9,30e-9 0.9,40e- 9,0.7,50e-9,0.6)</pre>
	boundary - Specifies that voltage source locations will be used for instances in the design.
	xyiv - Specifies whether the voltage sources specified in the xy power pad location file are current source or reference voltage source. The format is the same as the xy file, except the additional first and last columns to indicate the current/voltage source and current/voltage value.
	-layer vialayername
	Specifies the via layer on which the voltage sources are to be generated. If this parameter is not specified, the voltage sources will be generated on the top via layer of the design. This parameter is specified with the -auto_voltage_source_creation parameter.
	-net net_name / all
	Specifies the name of the power net. You can use all to set voltage source information for all nets in the domain or specify the net name for which the voltage source is to be specified. If you specify all to include all nets, then you must specify the net name for each voltage source in the ploc file, as shown below: <pre>vsrsrc3 8.39 370.35 Metal15 VDD_AO VDD_external_1 5.89 372.85 Metal15 VDD_external VSSvsrsrc1 3.39 375.35 Metal15 VSS</pre>
	-number_of_voltage_source value
	Specifies the number of voltage sources to be generated. You can adjust either -tile or -number_of_voltage_source to reduce the number of voltage sources created. This parameter is specified with the -auto_voltage_source_creation parameter. Default: 1 million.
	-region {x1 y1 x2 y2}
	Generates the voltage sources in the specified region. If this parameter is not specified, the software uses the entire design/layout size as the region. This parameter is specified with the -auto_voltage_source_creation parameter.
- reset	Resets all the previous set_power_pads commands

<code>-short_pin_nodes {true false}</code>	
	Specifies to short all interface nodes on a pad cell pin to a single node and create voltage source for that node. Default: <code>false</code> Example: <pre>set_power_pads -net vdd -format padcell -short_pin_nodes -file pad_cell.file</pre>
<code>-tile {x y}</code>	
	Defines the number of tiles in the X and Y direction for automatic creation of voltage sources. When <code>-tile</code> is specified, the area (<code>-region</code> or default design area) will be tiled vertically and horizontally per the given tile size (<code>x*y</code>). For each tile, one voltage source will be created on the top metal of the biggest via array. If <code>-tile</code> is not specified, the software creates voltage sources on the top metal of all via cuts located either on the user-specified via layer or the top via layer of the design. You can adjust either <code>-tile</code> or <code>-number_of_voltage_source</code> to reduce the number of voltage sources created. The <code>-tile</code> parameter is specified with the <code>-auto_voltage_source_creation</code> parameter.

Examples

- The following command specifies the power pad information for the `VDD_TDSPCore_R` net in the file `dtmf_chip.VDD_TDSPCore_R.vsrc`. The voltages sources are specified by `x y` coordinates.

```
set_power_pads -net VDD_TDSPCore_R -format xy -file \ dtmf_chip.VDD_TDSPCore_R.vsrc
```
- The following command specifies `xy`, `padcell`, and `defpin` formats for a single net (`VDD`) as follows:

```
set_power_pads -net VDD -format xy -file VDD.pp
set_power_pads -net VDD -format padcell -file VDD.padcell
set_power_pads -net VDD -format defpin
```
- The following command specifies power pad information using a combination of user-specified and automatic voltage source creation methods:
Specific setting for net VSS using automatic voltage source creation

```
set_power_pads -net VSS -auto_voltage_source_creation true -layer VIA_6
```


Global setting for all nets using automatic voltage source creation

```
set_power_pads -auto_voltage_source_creation true -net ALL -layer VIA_5
```


User-specified power pad file for net VDD_AO, therefore automatic voltage source creation feature will not be enabled for this net

```
set_power_pads -net VDD_AO -format padcell -file padcell.file -short_pin_nodes true
```


Example of power pad location file with XY coordinates for net VDD

```
*-----  
-----  
* XY power pad location file  
* vsrc name X(um) Y(um) LEF/Tech Layer <R=value ohm> <C=value F> <L=value H>  
* Package R,L and C parameters are optional  
*-----  
-----  
VDD100 5.000 793.500 M7 VDD_AO  
VDD101 1187.835 293.500 M7 VDD_External  
VDD102 695.000 5.000 M6 VDD  
VDD103 1187.835 493.500 M7 VDD  
VDD104 595.000 1293.800 M6 VDD_AO  
VDD105 595.000 5.000 M6 VDD_External
```

Example of pad cell file for VDD net,

```
*-----  
-----  
* Pad cell file for Voltus  
* cellname <r=value> <l=value> <c=value>  
* Package parameters are optional  
*-----  
-----  
PVDD1 r=2.5e-3 l=2.5e-11 c=120e-12  
PVDD2 r=2.5e-3 l=2.5e-11 c=120e-12
```

set_rail_analysis_domain

```
set_rail_analysis_domain
-help
-gndnets ground_nets
-name domain_name
-pwrnets power_net_list
-signets signal_nets
[-threshold value]
```

Specifies a rail analysis power domain.

Note: For a power-gated domain, you should only specify the "always on" net. The switched net should not be specified as part of power domain definition, because it is traced by the program automatically.

Parameters

-help	Prints out the command usage.
-gndnets <i>ground_net_list</i>	
	Specifies a list of ground nets that are in the domain.
-name <i>domain_name</i>	
	Specifies the name of the power domain.
-pwrnets <i>power_net_list</i>	
	Specifies a list of power nets that are in the domain.
-signets <i>signal_nets</i>	
	Specifies a list of signal nets.
-threshold <i>value</i>	

Specifies the minimum allowed voltage on the power net or the maximum rise on the ground net, depending on the setting of the `-voltage` parameter. The `-threshold` parameter is used to verify that the power grid passes the limit check and to set the filters for an IR plot.

Note: This parameter does not support domains with MSMV power nets.

If you have specified both net (`set_pg_nets -threshold`) and domain-based (`set_rail_analysis_domain -threshold`) thresholds, the net-based threshold will be honored.

The valid value for threshold is between 0 and 0.5 (50%). For example, `set_rail_analysis_domain -threshold 0.1` indicates that 0.1 is 10% IRdrop. If a power net in the domain has nominal voltage of 1v, 10% of threshold will translate to 100mV of maximum effective IRdrop between the power and ground net.

Examples

- The following command sets the power domain `TDSP` to include `VDD_TDSPCore_R` and `VSS` nets:

```
set_rail_analysis_domain -name TDSP -pwrnets VDD_TDSPCore_R -gndnets VSS
```

- The following command sets the power domain `TDSP` to include the nets mentioned in the specified files, `power_file` and `ground_file`:

```
set_rail_analysis_domain -name TDSP -pwrnets_list_file power_file -gndnets_list_file ground_file
```

set_rail_analysis_mode

```
set_rail_analysis_mode
-method {static | dynamic | era_static | era_dynamic}
-accuracy {xd | hd }
-power_grid_library dir_list
[-accumulate_overlapping_ddv_pwl_waveforms {true | false}]
[-analysis_view view ]
[-avg_em_analysis {true | false}]
[-block_powerup_rail netname | -powering_up_rails net_name1... netname_n ]
[-cell_eval_tap_layer_file filename]
[-cell_ignore_file filename ]
[-check_current_balanced_power_grid_em {true | false}]
[-check_thermal_aware_em {true | false}]
[-check_vsrc_placement_on_switched_net {true | false}]
[-cluster_via_rule { {via_layer1 number_of_equidistant_vias}... }]
[-cluster_via_size value]
[-cluster_vial_ports {true | false}]
[-common_res_inst_pair_file_name file]
[-compress_powergrid_database {true | false}]
[-create_pdn_model {true | false}]
[-decap_cell_list { cell1 cell2 ... celln }]
[-def_based_hierarchical_reports {true | false}]
[-default_package_capacitor value ]
[-default_package_inductor value ]
[-default_package_resistor value ]
[-die_delimiter delimiter]
[-die_mode {single|multi-die|3dic} -die_instance_name dieinstname [-design designname ]]
[-disable_analysis_types { list of analysis types }]
[-disable_em_split_ac_dc_rules {true | false}]
[-disable_parallel_extraction]
[-dynamic_trigger_file filename ]
[-eiv_average_per_window_list filename]
[-eiv_eval_gnd_window {true | false}]
[-eiv_eval_layers {layerlist}]
[-eiv_eval_layer_file filename]
[-eiv_eval_nodes {tap | port}]
[-eiv_eval_window {switching | timing | both | elapse}]
[-eiv_histogram_max value]
[-eiv_histogram_min value]
[-eiv_histogram_number_of_bucket value]
[-eiv_max_instances value]
[-eiv_method {worst best avg worstavg bestavg}]
[-eiv_pin_based_report {true | false}]
[-eiv_pin_location {true | false}]
[-eiv_print_time {true | false}]
[-eiv_report {auto | netonly | all}]
[-eiv_threshold value]
[-em_cdf_percentage value]
[-em_ignore_pgv_resistors {true | false}]
[-em_limit_scale_factor {{avg value} {rms value} {peak value}}]
```

```

[-em_models file ]
[-em_peak_analysis {true | false}]
[-em_report_line_threshold value]
[-em_rms_delta_T temp]
[-em_rms_delta_T_layer_list {{<LEF_layer_name> <delta_T_in_C>}+}]
[-em_temperature string ]
[-em_temperature_layer_list {{<LEF_layer_name> <em_temperature_in_C>}+}]
[-em_threshold value]
[-enable_2d_partition_extraction {true | false}]
[-enable_ccs_analysis {true | false}]
[-enable_distributed_processing_in_solver {true | false}]
[-enable_instance_powergate_report {true | false}]
[-enable_manufacturing_effects {true|false}]
[-enable_multi_die_connectivity_check {true | false}]
[-enable_package_battery_current_probing {true | false}]
[-enable_rc_analysis {true | false}]
[-enable_reff_analysis {true | false}]
[-enable_rlrp_analysis {true|false}]
[-enable_scheduler {true | false}]
[-enable_seb {true | false}]
[-enable_sensitivity_analysis {true | false}]
[-enable_voltage_across_vias {true|false}]
[-enable_vsrc_in_gif {true | false}]
[-enable_xp {true | false}]
[-enable_xpgv_scaling { true | false }]
[-enlarge_vsrc_in_vuvc {true | false}]
[-env_temperature temperature]
[-era_check_wires_for_generated_current_regions {true | false}]
[-era_current_distribution { unplaced | placed | all | none }]
[-era_current_distribution_factor_for_placed value]
[-era_current_distribution_layer layer_name]
[-era_current_distribution_nets {net1 net2 ...}]
[-era_current_distribution_unplaced_area {instance | diearea}]
[-era_current_region_file filename]
[-era_insert_virtual_followpins { standard | extended | none }]
[-era_insert_virtual_followpin_for_io {true | false}]
[-era_insert_virtual_via_on_layers value ]
[-era_instance_dynamic_current_file filename]
[-era_lef_layermap filename]
[-era_power_gate_file filename]
[-era_skip_virtual_via_by_type {whatif | def | all | none}]
[-era_skip_virtual_via_on_layers {{ layer1 layer2 } { layer3 layer4 } ... }]
[-era_techlib_generation {true | false}]
[-extract_subconductor_layers {true | false}]
[-extraction_tech_file filename ]
[-extractor_include filename ]
[-feol_scale_factor non_negative_integer_number]
[-filler_cell_list { cell1 cell2 ... celln }]
[-fine_pre_simulation_period value]
[-finegrain_powergate_ron {min avg max}]
[-finegrain_powergate_ron_list filename]
[-flatten_xpgv_block_instances filename]

```

```

[-force_extraction {true | false} ]
[-force_library_merging {true | false}]
[-gds_file file [-gds_offset {x y}]]
[-gds_map file ]
[-gds_purpose {metalFill | flipChip | fullChip}]
[-gds_top_cell cell_name ]
[-generate_combined_ivd_gif {true | false}]
[-generate_hier_bbv {true | false}]
[-generate_instance_effr_report file]
[-generate_instance_ir_report file]
[-generate_instance_pin_ir_report file]
[-generate_multi_voltage_library {true | false}]
[-generate_package_pin_current_voltage {true | false}]
[-gif_iv_threshold value]
[-gif_new_color_scale {true | false}]
[-gif_resolution {low | medium | high}]
[-gif_zoom_area { x1 y1 x2 y2 }]
[-gif_zoom_topcell_diearea {true | false}]
[-hier_delimiter character]
[-honor_negative_static_current {true | false}]
[-hpgv_block_lefs list_of_files]
[-hpgv_generate_view {ir | em | all}]
[-ict_em_models file]
[-ignore_decaps {true | false}]
[-ignore_duplicate_vsrc {true | false}]
[-ignore_fillers {true | false}]
[-ignore_from_reporting filename]
[-ignore_incomplete_net {true | false}]
[-ignore_nets_without_vsrc {true | false}]
[-ignore_shorts {true | false}]
[-import_what_if_shapes {true | false}]
[-inst_ignore_file filename]
[-ircx_models {RC.ircx EMIR.ircx}|{RC.ircx}]
[-lef_map lefMappingFile ]
[-lifetime value ]
[-lifetime_rating_factor_file filename]
[-limit_number_of_steps {true | false}]
[-lowest_layer_for_em_check layer_name]
[-mcp_model_mapping {{<mcp model name1> <die DEF name1>} {<mcp model name2> <die DEF name2>} ...}]
[-off_rails net_name_list ]
[-off_state_powergate_instance_file filename]
[-override_set_power_data {true | false}]
[-package_trace_connectivity {true | false}]
[-peak_em_analysis {true | false}]
[-powering_down_rails net_name1... netname_n ]
[-powerup_fast_mode {true | false}]
[-powerup_sequence_file filename ]
[-pre_simulation_period value]
[-pre_simulation_resolution value]
[-prechain_powerup_sequence_file filename]
[-print_gif_range_percentage {true | false}]
[-probe_instance_pin_waveforms {true | false}]

```

```

[-probe_instance_tap_waveforms {true | false}]
[-probe_package_interface_ports {true | false}]
[-probe_pin_voltage_list filename]
[-probe_waveform_list filename]
[-probing_node_file file_name]
[-process_bulk_pins_for_body_bias {true | false}]
[-process_techgen_em_rules {true | false}]
[-promote_pin_shapes_only {filler | decap | both} ]
[-rdl_def def_file]
[-rdl_def_list {{rdl_filename1  x1  y1  orient1} {rdl_filename2  x2  y2  orient2} ...} ]
[-rdl_orientation { N|S|W|E|FN|FS|FE|FW }]
[-rdl_placement {X Y}]
[-read_thermal_map thermal_map_file ]
[-record_inst_peak_current {true | false}]
[-record_results_start_time time ]
[-reff_detail_report {true | false}]
[-reff_domain_report_format {1 | 2 | 3 | 4}]
[-reff_eval_nodes {port | tap}]
[-reff_pin_report_layer { top | bottom | all }]
[-reff_pin_report_method { best | worst | avg} ]
[-reff_report_all {true | false}]
[-remove_duplicate_inst {true | false}]
[-report_compress_level {0 | 1-9}]
[-report_instances_missing_current_data {true | false}]
[-report_layer_based_ir_average {true | false}]
[-report_layers_above_pin_for_instance_ir {true | false}]
[-report_msmv_format {true | false}]
[-report_power_in_parallel {true | false}]
[-report_power_options option_names]
[-report_shorts {true | false}]
[-report_voltage_drop {true | false}]
[-reset]
[-reuse_state_directory dir_name]
[-rlrp_cell_report_layer filename]
[-rlrp_detail_report {true | false}]
[-rlrp_eval_nodes {port | tap}]
[-rlrp_percentage_threshold value]
[-rlrp_pin_based_report {true | false}]
[-rlrp_pin_report_method { best | worst | eiv_best | eiv_worst} ]
[-rlrp_pin_report_layer { top | bottom | all }]
[-rlrp_threshold value]
[-rms_em_analysis {true | false}]
[-save_voltage_waveforms {true | false}]
[-scale_initial_condition_current filename]
[-seb_lifetime value]
[-seb_table table_filename]
[-seb_temperature temperature]
[-set_analyze_bbox {xmin ymin xmax ymax}]
[-shorting_resistance value]
[-single_partition_extraction_layer {METAL_LAYER}]
[-single_partition_extraction_layer_list {METAL_LAYER_1 METAL_LAYER_2 .....}]
[-skip_extraction {true | false} ]

```

```
[ -snap_layer_for_current_taps file_name ]
[ -static_multi_mode_analysis {true | false} ]
[ -static_trigger_file filename ]
[ -suppress_message { message_id + } ]
[ -switchbit_file_for_eiv_calculation filename ]
[ -temp_directory_name directory ]
[ -temperature value ]
[ -topcell_orientation { N|S|W|E|FN|FS|FE|FW } ]
[ -topcell_placement {X Y} ]
[ -tsv_subckt_modelfile_list filename ]
[ -unconnected_die_pkg_pins {ignore | error | edit} ]
[ -unconnectedcell_ignore_file filename ]
[ -unified_power_switch_flow {true | false} ]
[ -use_early_view_list filename ]
[ -use_em_view_list filename ]
[ -use_ir_view_list filename ]
[ -use_rms_delta_t {true | false} ]
[ -verbosity {true | false} ]
[ -verify_logical_connectivity {true | false} ]
[ -vsrc_search_distance value ]
[ -watch_location_waveform { {layerName xCoord yCoord}+ } ]
[ -work_directory_name directory ]
[ -what_if_shapes_file filename ]
[ -write_demand_current_region_file filename ]
[ -xp_cpu_per_job_power value ]
[ -xp_cpu_per_job_simulation value ]
[ -xp_host_allocation_method {on_demand | at_startup} ]
[ -xp_purge {full} ]
[ -xp_resume {true | false} ]
[ -xp_reuse_extraction_directory directory_name ]
[ -xp_reuse_hpt_directory state directory ]
[ -xp_reuse_parse_def_directory directory_name ]
[ -xp_simulation_cpu_timeout value ]
[ -xp_simulation_min_cpu value ]
[ -xpgv_config_file filename ]
```

Specifies how the rail analysis will be performed.

Parameters

-accumulate_overlapping_ddv_pwl_waveforms {true false}	
	Default: false Specifies to accumulate the current values of Dynamic Detailed View (DDV) PWL waveforms from different trigger events when the PWL waveforms are overlapping with each other. If set to <code>true</code>, the current values of the overlapping PWL waveforms are added together for rail analysis.
-accuracy {xd hd }	

	Default: <code>hd</code> Specifies the accuracy mode of analysis.
	<code>xd</code>: <code>xd</code> or accelerated definition accuracy mode is used for early implementation stage IR/EM analysis.
	<code>hd</code>: <code>hd</code> or high definition accuracy mode is used for final verification stage IR/EM analysis.
<code>-analysis_view view</code>	
	Default: <code>None</code> Specifies analysis view created in a CPF (Common Power Format) file. Optional.
<code>-avg_em_analysis {true false}</code>	
	Default: <code>false</code> Specifies to calculate the average EM in the dynamic rail analysis run. This parameter generates the <code><net_name>.rj.avg.rpt</code> report.
<code>-block_powerup_rail netname</code>	
	Default: <code>None</code> Specifies a net for block power-up analysis. You can specify only one net for block power-up analysis at a time.
<code>-cell_eval_tap_layer_file filename</code>	
	Default: <code>None</code> Specifies to exclude the taps for a particular cell and pin of the specified layer. The parameter allows you to specify a file in the following format: <pre>CELL_NAME PIN_NAME LAYER_NAME</pre> The file supports wildcards (*) for cell name. You can specify <code>top</code>, <code>bottom</code>, or <code>all</code> for the layer name. File Format Example: <pre>=== \$file === CELL_NAME PIN_NAME LAYER_NAME BUF_* VDD METAL_1</pre>  The power-gate taps cannot be changed by this parameter.
<code>-cell_ignore_file filename</code>	

	Default: None If specified, excludes all references to the named cells in the specified file, from the design database.
<code>-check_current_balanced_power_grid_em {true false}</code>	
	Default: false Specifies to check the power-grid EM rules (power-rail terminal-via EM rule). This parameter allows EM comparison with/without this EM rule. This parameter is only applicable in TSMC N20 process nodes where the EM limit may be relaxed if certain constraints are met. Refer to TSMC DRM for more details.
<code>-check_thermal_aware_em {true false}</code>	
	Default: false Specifies to use the thermal map for EM checks. The thermal map file is specified using the <code>-read_thermal_map</code> parameter. The Rail Analysis engine uses the temperature specified in the thermal map file for EM analysis. EM limit has a strong temperature dependency. For a chip that has substantial temperature variation across the chip, the Rail Analysis engine can use the temperature specified in the thermal map file to perform accurate EM analysis.
<code>-check_vsrc_placement_on_switched_net {true false}</code>	
	Default: false Specifies to check voltage source placement on switched nets. When this parameter is set to <code>true</code>, the software checks whether the specified voltage source location file places any voltage sources on the switched net of the power-gated domain, and disables voltage source placement on the switched net. It also detects shorts and generates an error message.
<code>-cluster_vial_ports {true false}</code>	
	Default: true Designs with follow-pin routing on M1 and M2 layers have standard cells library LEF with M1, M2, and VIA1 ports. This parameter specifies to cluster VIA1 ports for such cells in order to improve overall extraction and rail analysis performance. This parameter is set to <code>true</code> by default to improve VIA1 port handling in designs with M1 and M2 follow-pin routing. This parameter gives significantly better performance by clustering equidistant VIA1 ports defined in cell LEF without significant loss of accuracy. However, if you specify a clustering rule, the user specification takes precedence over automated VIA1 clustering.
<code>-cluster_via_rule { {via_layer1 number_of_equidistant_vias}...}</code>	

	Default: None Controls the number of vias to cluster on a layer basis. The VIA clustering rule specified using this parameter will override the default clustering rule for a given accuracy mode. The following command will cluster 100 equidistant VIA1 cuts, 200 equidistant VIA2 cuts, and 300 equidistant VIA7 cuts: <pre>set_rail_analysis_mode -cluster_via_rule { {VIA1 100} {VIA2 200} {VIA7 300}}</pre> The rest of the VIAs will be clustered using the default clustering rule depending upon the rail analysis accuracy mode.
<code>-cluster_via_size value</code>	
	Default: 625 Specifies the size of the via array to cluster all via layers except VIA1 (layer between Metal1 and Metal2). The default value is 625 (25x25 via array), except when running static analysis using HD mode. The default for static HD mode is 16 (4x4), primarily to preserve more accuracy for EM analysis.
<code>-common_res_inst_pair_file_name file</code>	
	Default: None Specifies to calculate common resistance along the least resistance path (RLRP) between a pair of cells having the same voltage source. This parameter allows you to check RLRP along cell instance pairs from multiple voltage source locations. The format of the specified file is: <pre>inst_name1 inst_name2 inst_name3 inst_name4 ...</pre> The <code>-common_res_inst_pair_file_name_name</code> parameter generates a report <code><net_name>.common_res</code> in the following location: <pre><output_dir>/<state_dir>/Reports/<net_name>/</pre> The format of the report is: <pre>inst_name1 inst_name2 common_res1 pad1 inst_name3 inst_name4 common_res2 pad2 ...</pre> The following is a snippet from the report: <pre>inst_name1 inst_name2 13.4315 PAD_VDD2</pre>
<code>-compress_powergrid_database {true false}</code>	

	Default: false Specifies to compress the output files generated by extraction and rail analysis to save disk space usage. This parameter reduces disk space requirement from the extraction and rail analysis stages at approximately 5-10% performance penalty.
<code>-create_pdn_model {true false}</code>	
	Specifies to create a power delivery network model. This model will be generated in the state directory, and will be needed when running the <code>eco_opt_ir_aggressor</code> and <code>get_ir_insight -mode {report_aggressor predict_ir eco_report}</code> commands.
<code>-decap_cell_list { cell1 cell2 ... celln }</code>	
	Default: None Specifies a list of decap cells. This option is used when the decap cells are not tagged in the power-grid library. A cell characterized as a decap cell during power-grid view generation with LibGen does not need to be specified. For dynamic only.
<code>-def_based_hierarchical_reports {true false}</code>	
	Default: false Specifies to generate separate rail reports for each DEF instance. The net-based effective instance voltage, RLRP, and effective resistance reports are generated hierarchically. Reports will be generated for every net in each DEF hierarchy using the following directory structure: <pre><output_dir>/<state_dir>/Reports/HIER/hier_<HIER_ID>/<domain_name net_name>.<report_type></pre> In the Reports folder, the Rail Analysis engine will generate a hierarchical map (<code>def_hierarchy.map</code> file) during DEF parsing to associate a hierarchical ID with each DEF instance, as shown below: <pre>#hier_id hier_name #def_name 0 chip chipCore</pre>
<code>-default_package_inductor value</code>	

	Default: None Specifies the default inductance value of the package resistance-inductance-capacitance (RLC) model to use when you specify voltage source locations. If the power-grid views contain area-based voltage source locations, the following equation is used to define the inductance applied to the voltage sources: $L_i = L * \text{number_of_sources}$ where: L_i is the inductance on an individual voltage source. L is the default inductance set by - default_package_inductor. Number_of_sources is the number of voltage sources in the power-grid view. The default inductance value is specified in henrys.
-default_package_resistor value	
	Default: None Specifies the default resistance value of the package resistance-inductance-capacitance (RLC) model to use when you specify voltage source locations. If the power-grid views contain area-based voltage source locations, the following equation is used to define the resistance applied to the voltage sources: $R_i = R * \text{number_of_sources}$ where: R_i is the resistance on an individual voltage source. R is the default resistance set by - default_package_resistor. Number_of_sources is the number of voltage sources in the power-grid view. The default resistance value is specified in ohms.
-default_package_capacitor value	

	Default: None Specifies the default capacitance value of the package resistance-inductance-capacitance (RLC) model to use when you specify voltage source locations. If the power-grid views contain area-based voltage source locations, the following equation is used to define the capacitance applied to the voltage sources: $C_i = C / \text{number_of_sources}$ where: C_i is the effective capacitance on an individual voltage source. C is the default capacitance set by <code>-default_package_capacitor</code>. <code>Number_of_sources</code> is the number of voltage sources in the power-grid view. The default capacitance value is specified in <code>farads</code>.
<code>-design designname</code>	
	Default: None Specifies the DEF or GDS design name for multi-die analysis. This option is only available in multi-die analysis (<code>-die_mode multi-die</code>).
<code>-die_delimiter delimiter</code>	
	Default: / Specifies the character <code>:</code> as delimiter to separate the die instance name from the other element (net, bump, ground) names. You can specify the parameter when there is <code>/</code> character in the die instance name. Use Model: <pre>set_rail_analysis_mode -die_delimiter :</pre> Example: - When there is no <code>/</code> character in the die instance name, the <code>/</code> character is used as a delimiter in the die net name. Die instance name – <code>Die1</code> Net name – <code>Net1</code> Die net name – <code>Die1/Net1</code> - When there is <code>/</code> character in the die instance name, you can specify the <code>-die_delimiter</code> parameter to use <code>:</code> character as a delimiter in the die net name: Die instance name – <code>Die/Die1</code> Net name – <code>Net1</code> Die net name – <code>Die/Die1:Net1</code>
<code>-die_mode {single multi-die 3dic}</code>	

Voltus IC Power Integrity Solution Text Command Reference
Rail Analysis Commands

	Default: <code>single</code> Specifies whether single or multi-die analysis will be performed. The <code>3dic</code> option is used for performing multi-die rlrp analysis.
<code>-die_instance_name dieinstname</code>	
	Default: <code>None</code> Specifies the die instance name for multi-die analysis. This option is only available in multi-die analysis (<code>-die_mode multi-die</code>).
<code>-disable_analysis_types { list of analysis types }</code>	
	Default: By default, the disable list is empty. Disables specific analysis types during rail analysis. For example, <code>set_rail_analysis_mode -disable_analysis_types { rlrp vu pi pv }</code> disables <code>rlrp</code>, <code>vu</code>, <code>pi</code>, and <code>pv</code> analysis types during <code>analyze_rail</code>. The following analysis types can be disabled: <code>ir tc iv vc dd pi pv javg rj jrms rc node_effr vu</code>.
<code>-disable_em_split_ac_dc_rules {true false}</code>	
	Default: <code>false</code> Specifies to use the same AC/DC rules for the EM analysis of the signal and PG nets. When this parameter is set to <code>true</code>, the software disables separate handling of the AC/DC rules. By default (when set to <code>false</code>), the software uses separate EM rules for the signal and PG nets.
<code>-disable_parallel_extraction</code>	
	Default: Uses parallel processors for extraction Specifies to use a single processor for extracting resistance or capacitance. If this parameter is not specified, the software uses parallel processors for extraction.
<code>-dynamic_trigger_file filename</code>	

	Default: None Specifies a dynamic trigger file. When a Macro EM view with current signature is used in a design, it needs to be determine when this block/cell is triggered. The dynamic trigger file is used to determine this. At the trigger time, the dynamic current waveforms of the EM view block will be applied. Note: If you specify trigger for cells or instances using this parameter, the tool would give higher precedence to this user-defined trigger file over the auto-generated trigger files in the power analysis directory (<code>power_output_dir/trigger*.txt</code>). For more information on the trigger file format, refer to the "Dynamic Trigger File During Rail Analysis" section in the "File Formats" chapter in <i>Voltus User Guide</i>.
<code>-eiv_average_per_window_list filename</code>	
	Default: None Specifies to generate a report containing different sets of Effective Instance Voltage (EIV) for each timing/switching window for a list of cells/instances. You can use the <code>ALL</code> keyword to report EIV for every window for all cells and instances. The format of the specified file is: <pre>INST inst_name CELL cell_name ALL</pre> The report contains a set of average EIVs for each instance based on the timing/switching window in the current files. The report file name is: <pre>name.win_avg.rpt.</pre> Example: <pre>set_rail_analysis_mode -eiv_average_per_window_list ./clock_network.insts</pre> File format: <pre>INST clk__L2_I1 INST clk__L11_I2 INST clk__L13_I4 INST clk__L13_I3 INST clk__L13_I2</pre> Output EIV report format: <pre>#INST_NAME START_TIME:END_TIME EIV_AVG clk__L15_I4 1.32e-09:1.78e-09 1.06354 6.38e-09:6.8e-09 1.06217 clk__L15_I3 1.32e-09:1.68e-09 1.06273 6.38e-09:6.72e-09 1.06854 ...</pre>
<code>-eiv_eval_gnd_window {true false}</code>	
	Default: false Specifies that the ground net switching window should also be considered for the EIV computation. The switching windows from the power and ground nets will be merged into a single switching window.
<code>-eiv_eval_layers {layerlist}</code>	

	Specifies to report the EIV of nodes only on the specified layers. When this parameter is not specified, the software reports the EIV values on the worst nodes across all layers. The parameter allows you to analyze the EIV values for the user-defined layers. Example: <pre>set_rail_analysis_mode -eiv_eval_layers B_M0 -eiv_eval_nodes port</pre> This command reports only the EIV values of nodes on the B_M0 layer.
<code>-eiv_eval_layer_file filename</code>	
	Specifies a file containing a list of user-specified cells and corresponding layers for EIV reporting. The parameter allows you to analyze the EIV nodes on the user-defined cells and layers. This parameter is used for dual-sided rails (B_M0/M0) with only back-side interface/port nodes, and without the interface/tap nodes. The format of the file is: <pre><cellName> <layerList></pre> Example: The following command will report the EIV values of the instance nodes on the B_M0 layer associated with the H338 cell and report the EIV values on the instance nodes on the M0 layer: <pre>set_rail_analysis_mode -eiv_eval_layer_file file.rpt -eiv_eval_layers{M0}</pre> The following is the content of <code>file.rpt</code>: <pre>H338* B_M0</pre> When the <code>-eiv_eval_layer_file</code> parameter is not specified, the generated report is based on <code>-eiv_eval_layers</code>. If no nodes are found on the specified layer B_M0/M0, the software reports the EIV values on the worst nodes across all layers.
<code>-eiv_eval_nodes {tap port}</code>	
	Default: tap Specifies to report EIV based on the instance nodes (tap or port). By default, the software uses the tap nodes for EIV reporting.
<code>-eiv_eval_window {switching timing both elapse}</code>	

	Default: <code>both</code> Controls the window where EIV is evaluated. EIV is obtained by processing the voltage waveforms and finding the worse-case effective voltage between the power and ground pins during a specific window. There are four possible windows to evaluate EIV: • <code>switching</code> – any switching activity (current is non-leakage) reported in power analysis • <code>timing</code> – timing window information from TWF or embedded timing engine • <code>both</code> (Default) – both switching and timing window • <code>elapse</code> – entire rail simulation not filtered to a specific window Note: Although power analysis uses timing window information to generate switching activities, it is possible that switching window will be slightly outside timing window if the instance is scheduled to switch at the very beginning or the very end of the timing window. Current waveform may start prior or end later than the time of switching. For additional information on EIV window, see "effective instance voltage" in the "<i>Glossary</i>" chapter of <i>Voltus User Guide</i>.
<code>-eiv_histogram_max value</code>	
	Default: <code>0</code> Specifies the maximum voltage value of the user-defined voltage range for EIV histogram generation.
<code>-eiv_histogram_min value</code>	
	Default: <code>0</code> Specifies the minimum voltage value of the user-defined voltage range for EIV histogram generation.
<code>-eiv_histogram_number_of_bucket value</code>	
	Default: <code>8</code> Specifies the number of buckets/ranges to be used for EIV histogram generation. The data points are divided into buckets based on the range specified.
<code>-eiv_max_instances value</code>	
	Default: <code>2e-9</code> Specifies the maximum number of instances to be included in the .iv files.
<code>-eiv_method {worst best avg worstavg bestavg}</code>	

	Default: <code>worst</code> Allows different methods of computing EIV during rail analysis. Using this parameter, you can write out the best, worst, or average effective instance voltage between power and ground nets. The <code>worstavg</code> EIV method computes the average instance voltage during each evaluation window defined by the <code>-eiv_eval_window</code> parameter, and reports the worst average instance voltage out of all the windows. Similarly, the <code>bestavg</code> EIV method computes the average instance voltage during each evaluation window defined by the <code>-eiv_eval_window</code> parameter, and reports the best average instance voltage out of all the windows. If <code>-eiv_method worstavg/bestavg</code> and <code>-eiv_eval_window elapse</code> (i.e. 1 evaluation window) are specified, then this method will report average voltage across all time steps in simulation. For additional information on EIV methods, see "effective instance voltage (EIV) method" in the "Glossary" chapter of <i>Voltus User Guide</i>.
<pre>-eiv_pin_based_report {true false}</pre>	
	Default: <code>false</code> Specifies to report an individual pin's instance voltage for each instance. When enabled, the format of the instance voltage file (.iv file) will be changed to one instance pin per line with pin name added to the last column. If an instance has multiple pins connected to the same net, it will have multiple entries in the .iv file. The format of the report is: <pre><Instance Name> <Instance Voltage> <Cell Name> <Pin Names></pre> The following is a snippet of the report from the .iv file: <pre>NET INSTANCESUPPLY 18118 VDD 1.08 POWER BEGIN - inst1 1.0798 CKND12 VDD - inst2 1.0799 INVD6LVT VDD DOMAIN INSTANCESUPPLY 18118 VDD 1.08 DIFFERENTIATE - clk__L13_I3 1.0767 CKND12 VDD VSS - clk__L13_I4 1.0767 CKND12 VDD VSS</pre>
<pre>-eiv_pin_location {true false}</pre>	

	Default: <code>false</code> Specifies to report an individual pin's location for each instance. This parameter works only with the <code>-eiv_pin_based_report</code> parameter. When enabled, the format of the instance voltage file (.iv file) will be changed to one instance pin per line with the pin location added to the last column. This parameter only works for <code>ivdn</code> (net-based instance voltage drop). It is not supported for <code>ivdd</code> (domain-based instance voltage drop). That is, the pin location will be displayed only in the net-based iv report file (for example, VDD.iv file) and not in the domain-based iv report file (for example, VDD-VSS_div.iv). The format of the report is: <pre><Instance_Name> <PWR_IV> <Cell_Name> <Pin_Names> <X Y Coordinates></pre> The following is a snippet of the report from the .iv file: <pre>- INST1 1.13831 MUX2_F8_75LL VDD met1 <6152370, 5708640> - INST1 1.13831 ENOR4_F4_75LL VDD met1 <6153420, 5714520> - INST2 1.13831 BUF_F1_1SR_75LL VDD met1 <6147330, 5708640></pre>
<code>-eiv_print_time {true false}</code>	
	Default: <code>false</code> Prints the time stamp for the worst voltage in the switching window of the instance, and for the whole (elapse) window. The time stamp values are printed in the detailed EIV report. Using the time stamp information, you can determine the current signature related to the worst EIV for easy debugging. The default unit for time stamp is <code>ns</code>.
<code>-eiv_report {auto netonly all}</code>	
	Default: <code>auto</code> Specifies the type of EIV report to be generated. The possible options are: <code>auto</code> - generates domain-based .iv report for domain analysis. <code>netonly</code> - generates net-based .iv report. <code>all</code> - generates both net and domain .iv reports, if available.
<code>-eiv_threshold value</code>	
	Default: <code>1e-9</code> Specifies the EIV drop threshold in mV for reporting. Only voltage drop above the threshold will be saved into the report.
<code>-em_cdf_percentage value</code>	

	Default: 1e-9 Specifies to calculate the Cumulative Distribution Factor (CDF) derating factor based on the EM rules specified in the ICT EM file, given by the <code>-ict_em_models</code> parameter. This parameter is used to calculate the fail rate and reliability of a design. The <code>-em_cdf_percentage</code> parameter is only applicable to the ICT EM file with the <code>cdf_percentage</code> keyword. The value of the CDF percentage should be between 0 and 100. The use model of the new parameter is: <code>set_rail_analysis_mode -em_cdf_percentage 5</code> When the CDF related rule exists in the ICT EM file, the software uses the default value 1e-09 for <code>-em_cdf_percentage</code> if it is not specified.
<code>-em_ignore_pgv_resistors {true false}</code>	
	Default: false Specifies to filter out all the EM violations of resistors inside a macro (resistors coming from PGV).
<code>-em_limit_scale_factor {{avg value} {rms value} {peak value}}</code>	
	Default: None Scales the calculated EM limits by the respective factor for avg, rms or peak current limit specified by <code>{avg value} {rms value} {peak value}</code>. The software compares the current through the net against the scaled EM limit to report violations in power EM.
<code>-em_models file</code>	Default: None Specifies the name of a file that includes the electromigration (em) models. The <code>-process_techgen_em_rules</code> and <code>-em_models</code> parameters are mutually exclusive. In the static mode, if <code>-em_models</code> is not specified, or if <code>-process_techgen_em_rules</code> is not set to <code>true</code>, current density (RJ) analysis will be disabled due to lack of EM models.
<code>-em_peak_analysis {true false}</code>	
	Default: false Specifies to perform peak EM analysis. When this parameter is specified with <code>-process_techgen_em_rules true</code> or <code>-em_models</code>, both RMS and Peak EM will be reported in the same run.
<code>-em_report_line_threshold value</code>	

	Default: INT_MAX Controls the size of the EM (<code>rj.avg.rpt</code>) text report by limiting the number of lines written to the report file. This feature is useful in limiting the report size when you specify a very low EM threshold. Example: The following command reports only the top 18 lines: <code>set_rail_analysis_mode -em_report_line_threshold 18</code> The truncated EM report file has the following note: # NOTE: Aborting EM Report printing as total number of lines has exceeded the Line Threshold "em_report_line_threshold = 18"
<code>-em_rms_delta_T temp</code>	
	Default: 5C Specifies the delta temperature for the EM RMS current limit analysis.
<code>-em_rms_delta_T_layer_list {{<LEF_layer_name> <delta_T_in_C>}}+</code>	
	Default: None Specifies the delta temperature for specific LEF layers during power EM RMS current limit analysis.
<code>-em_temperature <string></code>	Default: If <code>-em_temperature</code> is not specified and <code>em_tref</code> is not included in the EM rule file, the tool uses the value set by the <code>-temperature</code> parameter. Sets a different temperature for EM analysis. As a result, you can use a different but higher temperature for conservative EM analysis.
<code>-em_temperature_layer_list {{<LEF_layer_name> <em_temperature_in_C>}}+</code>	
	Default: None Sets a different temperature for specific LEF layers during power EM AVG current limit analysis.
<code>-em_threshold value</code>	
	Default: 0.9 Specifies to set EM analysis threshold value for the resistors being reported. Using this parameter, you can control the reporting threshold. By default, rail analysis generates EM report for any resistor having current above 0.9 of its limit. Changing the value to 0 will show every resistor, and 1 will limit to the failed resistors only.
<code>-enable_2d_partition_extraction {true false}</code>	

	Default: <code>false</code> Specifies to enable the upgraded extraction algorithm that allows 2D partitioning. The 2D extraction algorithm uses an updated capacitance model from the qrcTechFile that provides better accuracy for capacitance calculation. The resistance accuracy level of the 2D extraction algorithm is the same as 1D extraction algorithm. It is recommended to use 2D partitioning as it improves the parallelism of the extraction jobs for large designs. The default value of this parameter is <code>false</code>. When the <code>-enable_2d_partition_extraction</code> parameter is set to <code>true</code>, you must provide the complete qrcTechFile using the <code>set_rail_analysis_mode -extraction_tech_file</code> parameter. This parameter does not support the 3DIC, inductance extraction, and full-chip GDS flows.
<code>-enable_ccs_analysis {true false}</code>	
	Default: <code>false</code> Specifies to enable constant current source analysis.
<code>-enable_distributed_processing_in_solver {true false}</code>	
	Default: <code>true</code> Specifies to enable distributed processing for Matrix Solver. When set to <code>true</code>, Matrix Solver will be able to use multiple hosts across the network to analyze power-grid. You can specify this parameter when peak memory of the analysis exceeds machine memory in the local multi-threaded mode. In the distributed mode, peak memory on the local machine will be reduced as child processes will be executed on network hosts. However, the master thread on the local machine will continue to consume majority of peak memory even in the distributed mode.
<code>-enable_instance_powergate_report {true false}</code>	

	Default: false Specifies to generate a report (<code>powergate.inst.rpt</code>) that summarizes information about each power gate instance. The format of the report is: <pre><inst_name> <cell_name> <Inst X> <Inst Y> <V_Ref> <Ion(total)> <PI(Ipeak/Iavg)> PV(IR)></pre> Here, • <code><inst_name></code> is the power gate instance name • <code><cell_name></code> is the name of the power gate cell • <code><Inst X></code> is the x coordinate of the power gate instance • <code><Inst Y></code> is the y coordinate of the power gate instance • <code><V_Ref></code> is the rail voltage • <code><Ion(total)></code> is the total current for enabled powergate instance • <code><PI(Ipeak/Iavg)></code> is the worst <code>I_{dsat}</code> ratio out of all the <code>R_{on}</code> values • <code><PV(IR)></code> is the voltage across the power switch using the combination for the worst <code>V(AON Pin)-V(SW Pin)</code>
<pre>-enable_manufacturing_effects {true false}</pre>	
	Default: false Specifies to honor manufacturing effects during static or dynamic analysis. The manufacturing effects are enabled by default in the <code>hd</code> accuracy mode. That is, if you specify <code>set_rail_analysis_mode -accuracy hd</code>, the <code>-enable_manufacturing_effects</code> parameter is set to <code>true</code> by default.
<pre>-enable_multi_die_connectivity_check {true false}</pre>	
	Default: false Specifies to check if voltage sources with <code>type=nonvsrc</code> have resistance path to any voltage source with <code>type=vsrc</code> inside each die. This parameter allows you to determine if the non-real voltage source (<code>type=nonvsrc</code>) is floating or does not have any resistance path to an ideal voltage source (<code>type=vsrc</code>), and to ignore floating voltage sources during rail analysis. The “<code>vsrc</code>” and “<code>nonvsrc</code>” types are given in the <code>.ploc</code> file specified with the <code>set_power_pads</code> command. When the <code>-enable_multi_die_connectivity_check</code> parameter is specified, the log file displays the list of disconnected terminals for each net. This parameter can be used in both the single-die and multiple-die analysis flows.
<pre>-enable_package_battery_current_probing {true false}</pre>	

	Default: <code>false</code> Specifies to include the battery current waveform data in the <code>tran_<net>_pad.ptiavg</code> file. This file includes the voltage of all battery side <code>power_pin</code> and <code>ground_pin</code> mapped pins.
<code>-enable_rc_analysis {true false}</code>	
	Default: <code>false</code> Enables the Resistor Current (RC) analysis. This parameter supports designs without EM rules/analysis to generate the RC plot for IR drop debugging.
<code>-enable_reff_analysis {true false}</code>	
	Default: <code>false</code> Specifies to run full-chip effective resistance analysis as part of the static or dynamic IR drop analysis. Effective resistance analysis will be performed on the domain specified with <code>analyze_rail</code>. This parameter allows you to run both resistance and IR drop analyses in a single session. The use model of this parameter is: <pre>set_rail_analysis_mode -enable_reff_analysis true analyze_rail -output <dir_name> -type domain <domain_name></pre> Usage Guidelines: • Supports both the static and dynamic rail analyses. • Supports both the net-based and domain-based analyses.
<code>-enable_rlrp_analysis {true false}</code>	
	Default: <code>false</code> Specifies to enable least resistance path analysis for all the instances in the design. The RLRP path can be displayed in the GUI using the <code>set_power_rail_display -enable_rlrp</code> command.
<code>-enable_scheduler {true false}</code>	
	Default: <code>true</code> Controls the Voltus advanced scheduler that improves data handling and multi-CPU scalability of rail analysis . When set to <code>true</code>, the scheduler can significantly improve performance and memory consumption, especially when running on large number of processors. Note: It is recommended to set <code>-enable_scheduler</code> to <code>true</code>. If set to <code>false</code>, some of the advanced extraction features will not be supported.
<code>-enable_seb {true false}</code>	
	Default: <code>false</code> Specifies to enable the Statistical ElectroMigration Budgeting (SEB) analysis.

<code>-enable_sensitivity_analysis {true false}</code>	
	Default: <code>false</code> Only available in the static mode. When set to <code>true</code>, the software performs sensitivity analysis on all power-grid segments and highlights those which can be optimized to improve the overall IRdrop profile of the design. The power-grid segments can be optimized using the <code>scale_what_if_resistance</code> command.
<code>-enable_voltage_across_vias {true false}</code>	
	Default: <code>false</code> Specifies to report voltage across vias.
<code>-enable_vsrc_in_gif {true false}</code>	
	Default: <code>true</code> Specifies to enable or disable the display of voltage sources in the generated IR drop plot GIFs. When set to <code>false</code>, disables voltage sources (power pads) from the <i>Layout</i> tab during IR drop analysis. This parameter allows you to control the display of voltage sources in the generated IR drop plot GIFs so that the plot is easily interpreted.
<code>-enable_xp {true false}</code>	
	Default: <code>false</code> Specifies to run rail analysis in the Extensively Parallel (XP) mode. When this parameter is set to <code>true</code>, the rail analysis engine distributes processing over a large number of machines in every stage of the flow (for example, extraction, reporting, GUI, and simulation) to reduce the processing time.
<code>-enable_xpgv_scaling {true false}</code>	
	Default: <code>false</code> Specifies to enable xPGV current scaling that is calculated as a ratio of voltages. If PGV voltage is V1 and analysis voltage is V2, then the scaling factor would be V2/V1. This factor is applied to each power net. In case there are multiple power nets and a single ground net, the tool will use the worst scaling factor from the power rails and apply it to the ground net. This parameter is used for scaling of currents stored in xPGV. The parameter allows you to use the same xPGV in different designs, irrespective of the voltage at which the xPGV is generated. <code>-enable_xpgv_scaling</code> and <code>-static_trigger_file</code> are mutually exclusive.

Voltus IC Power Integrity Solution Text Command Reference
Rail Analysis Commands

<code>-enlarge_vsrc_in_vuvc {true false}</code>	
	Default: <code>false</code> Specifies to increase the size of voltage sources represented for the voltage source current (vc) and voltage drop across package (vu) plots to improve visibility.
<code>-env_temperature <temperature in °C></code>	
	Default: <code>25C</code> Specifies the ambient temperature used to calculate S_{DC} for SEB analysis.
<code>-era_check_wires_for_generated_current_regions {true false}</code>	
	Default: <code>false</code> Specifies to generate unplaced current regions only in areas where actual net wires are present. The current regions are generated from the power domain information present in the design.
<code>-era_current_distribution_factor_for_placed value</code>	
	Default: <code>1</code> When you use <code>-era_current_distribution</code> and set to distribute current for placed blocks/macros, the ratio of power allocated to each placed instance is calculated based on its total area. Using this parameter, you can control the current distribution factors for the placed blocks/macros, hence power allocated for area-based power calculation. For example, if you specify 0.5, the tool will assume all placed blocks/macros to be 50% of its actual size and distribute current accordingly.
<code>-era_current_region_file filename</code>	

	Default: None Specifies a file that includes a list of regions and the amount of current to be distributed within them for the power-grid. Default units in file are mA. You can also specify rectilinear current regions in the file. #Format: LABEL <i>name</i> NET <i>netName</i> AREA <i>x1 y1 x2 y2</i> LAYER <i>layername</i> <CURRENT <i>value</i> PWL (<i>t1 i1 t2 i2 ...</i>)> INTRINSIC_CAP <<i>value</i>> LOADING_CAP <<i>value</i>> [ADD_OVERLAP SUBTRACT_OVERLAP] #Unit: current mA, cap pf, time ns, coordinate um • ADD_OVERLAP: Honors all overlapping regions, and adds current in the overlapped regions. • SUBTRACT_OVERLAP: Specifies that if two XY regions overlap, the first specified region will be honored, and the subsequent region will subtract the overlapped area to apply the region current to the remainder area. <u>Static Example :</u> <pre>label test1 net VDD area 100 200 400 500 layer M1 current 10</pre> <u>Dynamic Example :</u> <pre>label test1 net VDD area 100 200 400 500 layer M1 pwl (0ns 0mA 1ns 0mA 1.9ns 0mA 2ns 10mA) intrinsic_cap 10 loading_cap 60 label test1 net VDD_AO area 0 0 150.93 0 150.93 175.14 354.93 175.14 354.93 379.12 0 379.12 0 0 layer Metall PWL (0 1 10 1 20 025 1 100 1) INTRINSIC_CAP 1 LOADING_CAP 1</pre>
<pre>-era_current_distribution_layer <i>layer_name</i></pre>	
	Default: None Specifies the layer name for distributing unplaced current in the early rail analysis mode.
<pre>-era_current_distribution { unplaced placed all none }</pre>	

	Default: None This parameter controls the behavior of era current distribution. <code>set_power_data area</code> based power, or <code>current_region_file</code> need to be specified for ERA current distribution to work. Placed instanced without uti or ascii based power can also be considered for era current distribution. • Unplaced: Enable current distribution only for unplaced instances. If <code>set_power_data area</code> based power is specified, <code>-era_current_distribution_layer</code> will be required. • Placed: Enable current distribution for placed instances without any power specified. • All: Both unplaced and placed instances without power specified; will have ERA current. • None: Disable ERA current distribution.
<code>-era_current_distribution_nets {net1 net2 ...}</code>	
	Default: None Specifies to distribute current for unplaced instances by total die area on specified nets only. The default behavior is to distribute current for all nets. This parameter will be required when <code>-era_current_distribution_unplaced_area diearea</code> is used. For single net analysis, you can either specify a single net (for example, <code>-era_current_distribution_nets {VDD_AO}</code>) or perform the analysis without this parameter.
<code>-era_current_distribution_unplaced_area {instance diearea}</code>	
	Default: <code>instance</code> Specifies to distribute current for unplaced instances by total die area or actual instance area. When <code>diearea</code> is specified, the entire unplaced die area is considered for computation of current distribution. The die area considered excludes user-specified current regions, placed instances, and placement blockages.
<code>-era_insert_virtual_followpins { standard extended none }</code>	
	Default: <code>none</code> Specifies to generate virtual followpins. The <code>extended</code> followpins will create followpins that extend from one stripe to another. The <code>standard</code> followpins may extend to previous stripe but does not reach the next stripe. <i>Default:</i> <code>none</code>
<code>-era_insert_virtual_followpin_for_io {true false}</code>	

	Default: <code>false</code> By default, ERA virtual follow pin insertion only works in the core area. When this parameter is set to <code>true</code>, ERA follow pin routing is extended to the IO area as well.
<code>-era_insert_virtual_via_on_layers value</code>	
	Default: None Enables you to control on which layers virtual via can be inserted in early rail analysis. Using this parameter, virtual via can only be inserted between each pair of layer names provided in the design. Alternatively, you can provide wildcard, '*', as the second layer name and virtual via will be allowed to be inserted from the first layer to any layers in the design. For example, <code>-era_insert_virtual_via_on_layers { {M1 *} }</code> will allow virtual via to be inserted from M1 to any layers in the design. This parameter does not have a default value but ERA will insert virtual via for all layers if <code>-era_skip_virtual_via</code> parameters are not provided.
<code>-era_instance_dynamic_current_file filename</code>	

	Default: None Specifies an instance/cell based PWL file in the dynamic ERA flow (<code>set_rail_analysis_mode -era_dynamic</code>). This feature allows you to perform dynamic ERA without having to specify the dynamic .ptiavg/.ptiavk current files (<code>set_power_data</code>). The specified PWL should cover the entire simulation period. If the simulation period is longer than the specified PWL, the software will assume 0 current for the remaining period (<code>set_dynamic_rail_simulation -stop <time></code>). You can also use the <code>set_dynamic_rail_simulation -auto_repeat</code> parameter to repeat the dynamic current waveform till the <code>-stop</code> time specified, instead of assuming 0 current for the remaining period. The format of the file is: <pre>INSTANCE CELL <NAME> PWL { <time> <current> } pwrnet gndnet <net_name> [ADD_OVERLAP SUBTRACT_OVERLAP]</pre> • Default time unit = ns. • Default current unit = mA • <code>pwrnet gndnet</code> keyword indicates if the specified net is a power or ground net. • <code>ADD_OVERLAP</code> keyword indicates to honor all overlapping instance currents, that is, instance to instance overlap and instance to region overlap. • <code>SUBTRACT_OVERLAP</code> keyword indicates to honor instance current, and subtract its area from the overlapped X-Y region to apply the X-Y region current to the remainder area. <code>SUBTRACT_OVERLAP</code> is default. This keyword is not applicable for instance to instance overlap. The following is a snippet of the specified file: <pre>INSTANCE top/instA PWL { 0ns 0mA 0.1ns 1mA 0.2ns 0.5mA } pwrnet VDD INSTANCE top/instA PWL { 0ns 0mA 0.1ns -1mA 0.2ns -0.5mA } gndnet VSS CELL macro_A { 0 0 0.1ns 1 0.2 0.5 } ...</pre> If you have specified both the <code>-era_instance_dynamic_current_file</code> and <code>-era_current_region_file</code> parameters, and there are overlapping instances with the specified current region, the <code>SUBTRACT_OVERLAP</code> behavior is implemented by default.
<code>-era_lef_layermap filename</code>	

	Default: None If a technology library or an extraction techfile is not provided, ERA can automatically create a technology library and the layermap needed to generate the library. If the automatic layer map generation fails, you can use this parameter to manually specify a layer map file. The format of the layer map file is: File Format: <pre> Layer Type tech_layername lefdef lefdef_layername metal METAL_1 lefdef M1 via VIA_1 lefdef VIA1 metal METAL_2 lefdef M2 via VIA_2 lefdef VIA2 metal METAL_3 lefdef M3 </pre>
<code>-era_power_gate_file filename</code>	
	Default: None If a power gate PGV is not provided, this parameter specifies the name of the power gate (power switch) file that will be used to perform power gate steady-state analysis. <u>File format:</u> <pre> CELL <CELLNAME> SUPPLY <SUPPLY_PIN> SWITCHED <SWITCHNET_PIN> RON <VALUE> IDSAT <VALUE> ILEAK <VALUE> CELL HDRSID0 SUPPLY TVDD SWITCHED VDD RON 500 IDSAT 1 ILEAK 0.001 </pre>
<code>-era_skip_virtual_via_by_type {whatif def all none}</code>	
	Default: all Specifies to skip a given via type. By default, ERA generates all virtual via layer types. • whatif vias are virtual vias that have connectivity to user-defined what if shapes. • def vias are virtual vias between two metal shapes defined in DEF. • all will skip all virtual via generation. • none will insert both what-if and DEF vias. It will insert vias on all layers, unless the layers are controlled by other parameters.
<code>-era_skip_virtual_via_on_layers {{ layer1 layer2 } { layer3 layer4 } ... }</code>	
	Default: None Skips via insertion between stripes and non-stripes, on the specified LEF layer pairs.
	<code>{ layer1 layer2 } { layer3 layer4 } ...</code> : Specifies the LEF layer names.
<code>-era_techlib_generation {true false}</code>	

Voltus IC Power Integrity Solution Text Command Reference
Rail Analysis Commands

	Default: <code>true</code> Specifies to skip technology PGV generation in the early rail analysis mode. This parameter skips technology PGV generation even if the <code>-extraction_tech_file</code> parameter is specified.
<code>-extractor_include filename</code>	
	Default: <code>None</code> Specifies extractor (ZX) commands and variables in the form of an include file.
<code>-extract_subconductor_layers {true false}</code>	
	Default: <code>false</code> Specifies to perform extraction of the sub-conductor layers.
<code>-extraction_tech_file filename</code>	
	Default: <code>None</code> Specifies the extraction technology file to be used for top-level power-grid extraction. If you do not specify this parameter, the software will use the extraction technology file stored inside the power-grid view library.
<code>-feol_scale_factor non_negative_integer_number</code>	
	Default: <code>None</code> Specifies to scale the front-end-of-line (FEOL) self-heat value for all instances in the self-heating analysis flow. The delta T FEOL will be calculated based on the static power, thermal resistance, and FEOL scale factor.
<code>-filler_cell_list { cell11 cell12 ... celln }</code>	
	Default: <code>None</code> Specifies a list of filler cells. This option is used when the filler cells are not tagged in the power-grid library.
<code>-finegrain_powergate_ron {min avg max}</code>	
	Default: <code>max</code> Specifies whether the minimum, maximum, or average Ron will be used for a powergate cell in finegrain memory characterization with multiple Ron values.
<code>-finegrain_powergate_ron_list filename</code>	

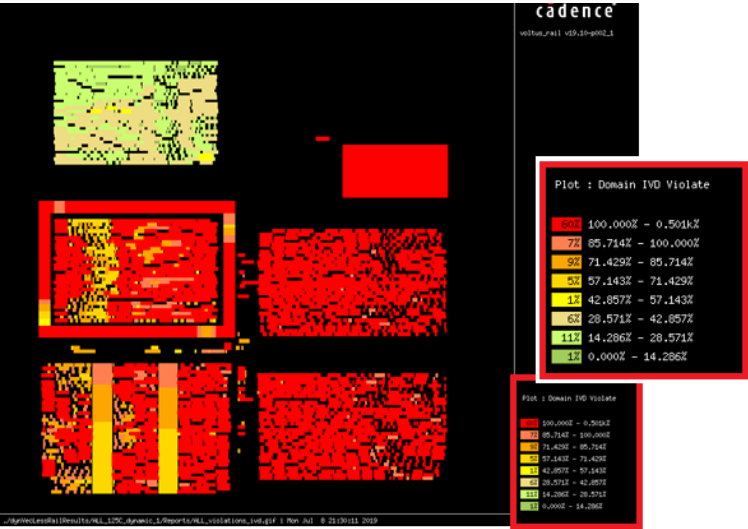
	Default: None Specifies the name of a file containing the powergate cell names and their corresponding Ron values (minimum, maximum, or average) to be used in finegrain memory characterization with multiple Ron values. The format of the file is: <pre>CELL cell1 min/max/avg CELL cell2 min/max/avg INST inst1 min/max/avg</pre> Note: The <code>-finegrain_powergate_ron</code> and <code>-finegrain_powergate_ron_list</code> parameters allow you to specify multiple Ron values in a scenario where the Ron value can change with respect to different modes of simulation.
<code>-fine_pre_simulation_period</code> <i>value</i>	
	Default: 0 Specifies the fine-grain pre-simulation period that rail analysis will use. The supported time units are s, ms, us, ns, and ps. If no explicit time unit is specified, the default ns (nanosecond) will be applied. The time step in the fine-grain pre-simulation period will be the same as the simulation period. Pre-simulation allows you to determine the current profile and analyze IRdrop of the circuit when power supply is in steady state. Note: When both the coarse-grain (<code>-pre_simulation_period</code>) and fine-grain pre-simulation periods are specified, the fine-grain pre-simulation period will start after the coarse-grain pre-simulation period. The simulation phase starts after the fine-grain pre-simulation period. Both fine-grain pre-simulation and coarse-grain pre-simulation can be specified just by themselves without requiring the other one to be specified. In the case where fine-grain pre-simulation is not specified, the simulation will start right after the coarse-grain period.
<code>-flatten_xpgv_block_instances</code> <i>filename</i>	

	Default: None Specifies a file that includes a list of block instances that use DEF instead of xPGV for rail analysis, that is, specify the instances that are not to be used as xPGV. This parameter allows you to specify a few instances of the same block that will use DEF, and the others that will use xPGV at the top-level. If you do not specify this parameter, and if both DEF and xPGV are given for a block, xPGV takes precedence over DEF. <u>Instance List File Format:</u> <pre><block_name> <block_instance_name All None></pre> In this file, you can specify if a block is using a DEF by specifying the block instance name, or with the <code>All</code> or <code>None</code> keywords. <code>All</code> indicates that all instances are using DEF, and <code>None</code> indicates that none of the instances are using DEF. <u>Example:</u> <pre>CORE /top/x1/x12/instA</pre> For a design with three blocks, if you want to use the DEF and xPGV as following: Block_1 -> A1 (xPGV) & A2 (DEF) Block_2 -> B1 & B2 (xPGV for both instances) Block_3 -> C1 & C2 (DEF for both instances) The specified file will contain the following information: <pre>Block_1 A2 Block_2 None Block_3 All</pre>
<code>-force_extraction {true false}</code>	
	Default: <code>false</code> Specifies whether the existing work directory extracted data should be used, or the design needs to be re-extracted during rail analysis.
<code>-force_library_merging {true false}</code>	
	Default: <code>false</code> Specifies to force merge PGVs with different resistivity and layers during rail analysis, wherein the first library definition will be used.
<code>-gds_file file</code>	Default: None Specifies a GDS file. Partial (RDL GDS in flip-chip) or full-chip GDS can be specified. You can view extracted parasitic network for GDS, but, in order to view GDS layout it needs to be imported as an OA database in Voltus.

<code>-gds_map file</code>	Default: None Specifies the GDS layermap for the GDS file specified using <code>-gds_file</code> parameter. The GDS layermap file format is as follows: <pre>#layer_type layer_name gds gds_layer_number <gds_data_type> <[port text]> layer via8 gds 85 layer metal9 gds 74 layer metal9 gds 126 1 text</pre>
<code>-gds_offset { x y }</code>	Default: None Specifies the offset value used to adjust and shift the GDS of a Redistribution Layer (RDL) cell in the mixed DEF+GDS flow. This parameter is available only when the <code>-gds_file</code> parameter is specified.
<code>-gds_purpose {metalFill flipChip fullChip}</code>	Default: None Specifies the data type of the GDS provided. <code>metalFill</code>: The GDS data will be used as fill metals for manufacturing effects calculation but the physical shapes will not be extracted. <code>flipChip</code>: The GDS data contains only the RDL layers, and the extracted physical shapes will be merged with the DEF netlist. <code>fullChip</code>: The GDS data contains physical shapes for the entire design.
<code>-gds_top_cell cell_name</code>	Default: None Specifies the name of the GDS top cell.
<code>-generate_combined_ivd_gif {true false}</code>	

Default: false

Specifies to generate a combined instance voltage drop GIF file for all the nets specified in the domain. This GIF allows you to see the entire design's instance voltage plot. The GIF file displays color scaling based on voltage drop as percent of different domain thresholds (print thresholds for all power domains), as shown below:



The file location of this GIF file is `<rail_dir>/Reports/ALL_violations_ivd.gif`.

`-generate_hier_bbv {true | false}`

Default: `false`

Specifies to control the generation of block boundary voltage (BBV) for all hierarchical block instances that use xPGVs. The xPGVs for the blocks can be generated using the xPGV flow (Refer to "Hierarchical Power Integrity Analysis with Extreme Modeling" in the *Voltus User Guide*). The BBV file contains realistic static or dynamic voltages for all nodes in the xPGVs starting where the top-level power-grid connects to the xPGVs. Since the BBV file is generated in context of full-chip analysis, running block-level analysis using this file gives insight to IR drop problems on the lower layers not included in the xPGV. This top-down flow can be used to identify and fix IR drop problems at the partition level without having to re-run the expensive fullchip analysis.

`generate_hier_bbv` is a global parameter so all instances of xPGV will be generated. The output is written to the state directory for each power/ground net analyzed as '`instance_name.pp`'. The file contains '`putvsrc`' commands. You can output on a per-instance basis using the '`write_block_boundary_voltage`' command in an include file, as shown below:

```
set_advanced_rail_options -voltus_rail_include_file_begin voltus_begin.inc
voltus_begin.inc:
```

```
write_block_boundary_voltage instance_name output_file_name
```

For dynamic analysis, add `-watch`:

```
write_block_boundary_voltage instance_name output_file_name -watch
```

The dynamic flow is recommended since transient information is not saved by default.

To use the output BBV file at the block level, return to the block-level execution script and change `set_power_pads -format` to `boundary` and include the BBV file:

```
set_power_pads -net net_name -format boundary -file bbv_file
```

Example of a BBV file for dynamic analysis:

* Boundary conditions for instance `sample_instance`

```
# putvsrc vsrc_# IR/Bounce x y PWLFILE pwl_file_for_node_location
putvsrc K4 Vbc_VSS_1 1.210359e-01 247.296 179.712 PWLFILE
sample_instance.pp.ptiavg 1
putvsrc K3 Vbc_VSS_2 1.210360e-01 247.296 179.712 PWLFILE
sample_instance.pp.ptiavg 2
putvsrc K4 Vbc_VSS_3 1.210097e-01 247.296 175.104 PWLFILE
sample_instance.pp.ptiavg 3
...
```

`-generate_instance_effr_report file`

	Default: None Generates a layer-based instance effective resistance report for the specified cells/instances. The specified file contains the list of cells or instances for which the Reff report is to be generated. The format of the input file is: <pre>CELL cell1 CELL cell2 INST inst1</pre> For each cell or instance specified in the input file, the report has information such as layer-based effective resistance ranges, worst node location, average statistics, and interface node resistances. The output file will be stored under each net's report folder <code><net>.instance_effr.rpt</code>. For example, <code>Reports/VDD/VDD.instance_effr.rpt</code>. Usage Guidelines: • Applicable only to the resistance analysis flow (<code>analyze_resistance</code>). • Supports both the static and dynamic rail analyses. • Supports both the net-based and domain-based analyses.
<pre>-generate_instance_ir_report file</pre>	
	Default: None Specifies to generate a layer-based IR drop report for a set of cells/instances. The <i>file</i> contains the list of cells or instances for which the IR drop report is to be generated. The format of the file: <pre>CELL cell1 CELL cell2 INST inst1</pre> The output file will be stored under each net's report folder <code><net>.instance_ir.rpt</code>. This feature is applicable for both net-based and domain-based analysis.
<pre>-generate_instance_pin_ir_report file</pre>	

	Default: None Specifies to generate a detailed report containing boundary node IR information for a set of cells/instances. The file contains the list of cells or instances for which the IR drop report is to be generated. The format of the file is: <pre>INST <instance_name> CELL <cell_name></pre> The instance pin IR report will list all the pins’ node information (layer, location) and the worst IR drop value. The output file will be stored under each net’s report folder NET/<net>.instance_pin_ir.rpt. The following is a snippet of the IR drop report: <pre>instance: <NAME> cell: <CELL> pin name: <PIN> # pin nodes: <total count> Min/Avg/Max: <min/avg/max interface node IR> For each of the interface node: IR X Y Layer</pre> The interface node output is sorted in the descending order of IR drop (that is, worst IR drop first).												
-generate_multi_voltage_library {true false}													
	Default: false When this parameter is specified, all the PGVs for each cell will be considered, and voltage-dependent capacitance information will be available for use in rail analysis, similar to the multi-voltage PGVs. The -generate_multi_voltage_library parameter is used in a scenario when you have generated multiple PGVs for different voltages, instead of generating a single multi-voltage PGV that has voltage-dependent capacitance. The following example shows the voltage-dependent capacitance table for the cell INV: <pre>=====</pre> <table><tr><td>CellName</td><td>Net Name</td><td>Voltage</td><td>Capacitance</td></tr><tr><td>INV</td><td>VDD</td><td>1.0</td><td>1.1e-15</td></tr><tr><td>INV</td><td>VDD</td><td>0.9</td><td>1.0e-15</td></tr></table> Here, there are two characterization voltage values and their corresponding capacitance values. The rail analysis engine uses both the voltage-dependent capacitance values to perform interpolation based on the two voltage values.	CellName	Net Name	Voltage	Capacitance	INV	VDD	1.0	1.1e-15	INV	VDD	0.9	1.0e-15
CellName	Net Name	Voltage	Capacitance										
INV	VDD	1.0	1.1e-15										
INV	VDD	0.9	1.0e-15										
-generate_package_pin_current_voltage {true false}													

	Default: <code>false</code> Specifies to generate a file containing the transient current or static current and voltage information of the package pins for the package analysis flow. This file is generated after rail analysis. This parameter is used in both the Sigrity Package Analysis (SPA) and non-SPA package analysis flows: • SPA flow - used in Sigrity tools to do chip-aware package analysis. This parameter is enabled by default (set to <code>true</code>) in the SPA flow. • Non-SPA flow - allows you to check the current on package pins. The format of the generated report in the static analysis mode (3 column format) is: <pre>pin_name1 current1 voltage1 pin_name2 current2 voltage2 ...</pre> The format of the generated report in the dynamic analysis mode is: <pre>.SUBCKT Subcircuit_Name <list of pins for power and ground nets> *MCP header with package pin name IA1 A1 0 PWL t1 i1 t2 i2 t3 i3 t4 i4 t5 i5 +t6 i6ENDS Subcircuit_Name</pre> Here, • <code>IA1</code> (first item) is the current source element name • <code>A1</code> (second item) and <code>0</code> (third item) are the power and ground nodes • <code>PWL</code> keyword is followed by the time and current values
<code>-gif_iv_threshold value</code>	
	Default: <code>1e-9</code> Specifies the instance voltage threshold for the combined instance voltage drop GIF for all the nets specified in the domain. The threshold is specified in voltage drop value. Any instance voltage drop value greater than the threshold will be put into the red bucket range in GIF, and the rest of the range will be linearly distributed down to the minimum instance voltage drop.
<code>-gif_new_color_scale {true false}</code>	
	Default: <code>false</code> Specifies to change the color scheme of the GIF file generated in the state directory during the static and dynamic rail analysis flow to match the GUI color scale.
<code>-gif_resolution {low medium high}</code>	

	Default: <code>medium</code> Controls the resolution of the generated GIFs after rail analysis. This option can have the following values: <code>low</code>, <code>medium</code>, and <code>high</code>.
<code>-gif_zoom_area { x1 y1 x2 y2 }</code>	
	Default: <code>None</code> Specifies to set the GIF generation bounding box to the given area (in microns).
<code>-gif_zoom_topcell_diearea {true false}</code>	
	Default: <code>false</code> Specifies to set the GIF generation bounding box to the topcell DIEAREA listed in the DEF file, instead of the total design size including RDL. If RDL is not used, this parameter has no effect. This parameter will change the bounding box of all the GIFs generated in rail analysis. Note: The <code>-gif_zoom_area</code> and <code>-gif_zoom_topcell_diearea</code> parameters are mutually exclusive. If both the parameters are specified, the <code>-gif_zoom_area</code> parameter takes precedence.
<code>-hier_delimiter character</code>	
	Default: <code>/</code> Specifies the hierarchical delimiter in the DEF file. The default hierarchical delimiter is a forward slash (/) but can be changed by setting the <code>-hier_delimiter</code> parameter. You can use this parameter to override the existing hierarchical delimiter.
<code>-honor_negative_static_current {true false}</code>	
	Default: <code>false</code> Specifies to honor negative current value in the current file (<code>.ptiavg</code>) during static rail analysis.
<code>-hpgv_block_lefs list_of_files</code>	
	Default: <code>None</code> Specifies the block LEF file for all hierarchical DEF blocks in the design. This will trigger automatic hierarchical PGV generation for the design hierarchy. The software will error out if any of the LEF files are missing for a DEF hierarchy.
<code>-hpgv_generate_view {ir em all}</code>	
	Default: <code>all</code> Specifies the view type to generate for the hierarchical PGV. The allowed values are <code>"ir"</code>, <code>"em"</code> and <code>"all"</code>. <code>"all"</code> means both <code>ir</code> and <code>em</code> views will be created. If you choose to only generate <code>ir</code> or <code>em</code> view, disk consumption can be reduced since only one view will be generated.

<code>-ict_em_models file</code>	
	Default: None Specifies the ICT-EM file which only contains the EM rules and excludes the RC extraction data in ICT. The software will use the EM rules in the specified ICT-EM file to calculate EM limits for power net EM analysis even if there are embedded EM rules in the Quantus QRC tech file.
<code>-import_what_if_shapes {true false}</code>	
	Default: false Specifies to import the what-if shapes created by the <code>create_what_if_shape</code> command for rail analysis. Note: If both the <code>-import_what_if_shapes</code> and <code>-what_if_shapes_file</code> parameters are specified, union of two sets of what-if shapes is considered for analysis.
<code>-ignore_decaps {true false}</code>	
	Default: true for static, false for dynamic Ignores decap cells during rail analysis. It ignores cells that are tagged as DECAP cells during library characterization or set as decap cells using the <code>set_rail_analysis_mode -decap_cell_list</code> command parameter. If decaps are used to preserve connectivity on the follow-pin routing, you must set this parameter to false. Ignoring decaps during dynamic analysis is not recommended.
<code>-ignore_duplicate_vsrc {true false}</code>	
	Default: true Specifies to check for multiple voltage sources with the same name in the power pad (.ploc) file. When set to true, the duplicate voltage sources are ignored. When this parameter is set to false and duplicate voltage sources are present in the .ploc file, an error will be generated. If there are duplicate voltage sources, rail analysis will report an error message like the following: <pre>** ERROR: (VOLTUS_RAIL-3647): Failed to run rail analysis due to redefined voltage source name in power pin location file(s) - /vsrc_same_name/PADs/VDD_AO.pp.</pre>
<code>-ignore_fillers {true false}</code>	
	Default: true Ignores filler cells during rail analysis if the list of filler cells are defined in PGV generation, or specified as fillers using the <code>set_rail_analysis_mode -filler_cell_list</code> command parameter. During rail analysis, ignore library cells that are tagged as FILLER cells and have no capacitance associated with the interface nodes. If fillers are used to preserve connectivity on the follow-pin routing, you must set this parameter to false.

<code>-ignore_from_reporting filename</code>	
	Default: None Specifies the name of a file containing a list of cells, instances, and cell types (filler and decap) that are not to be included in the EIV, RLRP, and REFF reports. A sample file is given below: <pre>INST AO2/MULT16_reg_2_6 CELL BUFX* CELL_TYPE FILLER</pre>
<code>-ignore_incomplete_net {true false}</code>	
	Default: false Specifies to skip power and ground nets that are not defined in the design, or have been defined but with no instance connection.
<code>-ignore_nets_without_vsrc {true false}</code>	
	Default: false Specifies if Rail Analysis should continue or exit with an error message if there are nets with no voltage sources. The possible arguments are: • <code>true</code>: prints an error message only but will allow the Rail Analysis run to continue. • <code>false</code>: exits immediately and returns to the Tcl prompt. If all the VDD/VSS nets have no voltage sources attached, the Rail Analysis run will error out and exit even if <code>-ignore_nets_without_vsrc</code> is set to <code>true</code>.
<code>-ignore_shorts {true false}</code>	
	Default: false if it is not signoff mode This parameter allows you to ignore shorts that are found during rail extraction and continue rail analysis. However, <code>-ignore_shorts</code> cannot be applied if the design contains GDS.
<code>-inst_ignore_file filename</code>	

	Specifies to exclude the LEAF instances mentioned in the specified file during rail extraction. File Format: <pre>Inst1 Inst2 . . . InstN</pre> If the instance name contains any special characters (such as ?, [, and so on) that are not treated as delimiters by default, prefix them with a backslash (\) to ensure correct processing. <table border="1"> <thead> <tr> <th>Instances to be Ignored</th><th>Pattern to be provided</th></tr> </thead> <tbody> <tr> <td><i>inst_B1/inst_1</i></td><td><i>inst_B1/inst_1</i></td></tr> <tr> <td><i>inst_B1/inst[11]</i></td><td><i>inst_B1/inst\[11\]</i></td></tr> <tr> <td><i>inst_B1/inst_3*</i></td><td><i>inst_B1/inst_3*</i></td></tr> <tr> <td><i>inst_B1/inst\[4]</i></td><td><i>inst_B1/inst\\\[4\]</i></td></tr> <tr> <td><i>inst_B1/inst\[5]</i></td><td><i>inst_B1/inst\\\[5\]</i></td></tr> </tbody> </table>	Instances to be Ignored	Pattern to be provided	<i>inst_B1/inst_1</i>	<i>inst_B1/inst_1</i>	<i>inst_B1/inst[11]</i>	<i>inst_B1/inst\[11\]</i>	<i>inst_B1/inst_3*</i>	<i>inst_B1/inst_3*</i>	<i>inst_B1/inst\[4]</i>	<i>inst_B1/inst\\\[4\]</i>	<i>inst_B1/inst\[5]</i>	<i>inst_B1/inst\\\[5\]</i>
Instances to be Ignored	Pattern to be provided												
<i>inst_B1/inst_1</i>	<i>inst_B1/inst_1</i>												
<i>inst_B1/inst[11]</i>	<i>inst_B1/inst\[11\]</i>												
<i>inst_B1/inst_3*</i>	<i>inst_B1/inst_3*</i>												
<i>inst_B1/inst\[4]</i>	<i>inst_B1/inst\\\[4\]</i>												
<i>inst_B1/inst\[5]</i>	<i>inst_B1/inst\\\[5\]</i>												
<pre>-ircx_models {RC.ircx EMIR.ircx} {RC.ircx}</pre>													
	Default: None Specifies an encrypted or unencrypted RC/EM technology (.ircx) file. The foundry-specific IRCX file, which contains the process information for extraction and EM modeling, can be used when the ICT-EM file is not available for electromigration analysis. You can either specify two IRCX files, one file containing the RC process data (RC.ircx) and the other file containing EM rules (EM.ircx), or a single IRCX file that includes both the RC and EM sections (RC.ircx). A single IRCX file is used for the mature nodes, and separate RC and EM IRCX files are used for the advanced process nodes.												
<pre>-lef_map lefMappingFile</pre>													
	Default: None Specifies the LEF layermap file, which contains the mapping of the LEF layer and the technology layer. The LEF layermap file format is as follows: <pre>layer map lef_layer_name tech_layer_name</pre>												
<pre>-lifetime value</pre>													

	Default: 87,600 hours (= 10 years) Specifies the lifetime value in hours.
<code>-lifetime_rating_factor_file filename</code>	
	Specifies a lifetime rating factor file that contains <code>jmax_life</code> values for each layer. These values are defined based on temperature and lifetime pairs. When you provide this file, EM analysis will automatically select the appropriate <code>jmax_life</code> factor for each layer according to the specific temperature-lifetime combinations defined within the file. The lifetime and EM temperature can be specified using the <code>-lifetime</code> and <code>-em_temperature</code> parameters. Following is a snippet of the lifetime rating factor file: <pre> *Rating factor of M0/V0/M1/V1/M2/V3/M3... TEMPERATURE 90 95 100 105 110 ... LifeTime ----- ----- 0.5 2.578 2.242 2.024 1.774 1.547 1 2.520 2.191 1.979 1.734 1.512 2 2.430 2.113 1.908 1.672 1.458 3 2.382 2.072 1.871 1.639 1.430 ... ----- ----- </pre>
<code>-limit_number_of_steps {true false}</code>	
	Default: <code>true</code> Specifies to limit the number of steps (simulation period, and resolution or step size) during dynamic IRdrop analysis or power-up analysis. If the number of steps during dynamic IRdrop analysis or power-up analysis exceeds 1000, the tool gives an error message and exits analysis. This parameter ensures that you are aware that there could be a problem in how the analysis is set up. If the performance risks are understood, you can set this parameter to <code>false</code> and re-run the analysis. The step-size and simulation period for the analysis is derived from current files generated by power analysis. You can explicitly set the step size and simulation period during power analysis using the <code>set_dynamic_power_simulation</code> command. During power-up analysis, the simulation period of the analysis is derived based on the turn-on time of the last power-switch in the power-up domain. If this time is too large, you may want to check if the turn-on times of the power-gates are correctly captured in the timing window file. Optionally, you can reduce the number of steps by explicitly specifying the stop time during rail analysis using the <code>set_dynamic_rail_simulation</code> command.
<code>-lowest_layer_for_em_check layer_name</code>	

	Default: None Specifies the layers that should not be displayed in the EM plots (such as, rj, jrms, and javg) and reports. <i><layer_name></i> is the layer below which the EM plots and reports will not be generated. The default value is <code>false</code>.
<code>-mcp_model_mapping {{<mcp model name1> <die DEF name1>} {<mcp model name2> <die DEF name2>} ...}</code>	
	Default: None Specifies to set the mapping between the package model and die name.
<code>-method {static dynamic era_static era_dynamic}</code>	
	Default: <code>static</code> Specifies whether static or dynamic rail analysis, or static or dynamic early rail analysis will be performed.
<code>-off_rails net_name_list</code>	
	Default: None Specifies a list of nets to exclude from analysis. Used in designs with power switches.
<code>-off_state_powergate_instance_file filename</code>	
	Specifies the list of the off-state powergates. When specified, the powergate report will show the Roff (resistance of the cell when it is switched off) value and set the status of those powergate cells to Off. The specified file lists one powergate instance name in each line. Wildcard (*) is supported in the file.
<code>-override_set_power_data {true false}</code>	
	Default: <code>false</code> Specifies to override the previous settings of the <code>set_power_data</code> command and use the last specified command setting for the same design object. This parameter can be used to honor the latest <code>set_power_data</code> command setting when the command has been specified multiple times for the same current file. Example: In the following example, the first command will be discarded and the second command will be honored: • <code>set_power_data -format current -instance XYZ dynamic_VDD.ptiavg</code> • <code>set_power_data -format current -instance XYZ dynamic_VDD.ptiavg -scale 2.0</code>
<code>-package_trace_connectivity {true false}</code>	

Voltus IC Power Integrity Solution Text Command Reference
Rail Analysis Commands

	Default: <code>false</code> When <code>-package_trace_connectivity</code> is set to <code>true</code>, connectivity tracing is enabled from the die pins to the board pins of all the elements in the package model file. When this parameter is set to <code>false</code>, connectivity tracing is disabled.
<code>-peak_em_analysis {true false}</code>	
	Default: <code>false</code> Specifies to calculate the peak EM in the dynamic rail analysis run. This parameter generates the <code><net_name>.rj.peak.rpt</code> report.
<code>-power_grid_library dir_list</code>	
	Default: None Specifies the name of the power-grid view library directories.
<code>-powerup_fast_mode {true false}</code>	
	Default: <code>true</code> Specifies to enable fast mode simulation for native power-up analysis. When the fast mode is enabled, the software uses lumped parasitic model for all instances connected to the powering up net. The fast mode native power-up analysis allows you to quickly estimate turn-on time and rush current injected in the global power-grid as the power-gated net is powering up. The fast-mode power-up analysis gives faster turn-around time and similar accuracy as accurate power-up analysis.
<code>-powerup_sequence_file</code> <code>filename</code>	
	Default: None Specifies the name of the file containing the firing time for each power gate instance. The power-up sequence file format is as follows: <pre><power gate instance name> <pin name> <always-on net> <switched net> <rise_time> <rise_slew> <fall_time> <fall_slew> <control pin voltage></pre> <code><control pin voltage></code> is optional. A sample power-up sequence file is given below: <pre>inst_name E VCC_NORMAL VCC_SWITCHED 5.50E-09 2.90E-10 5.50E-09 2.90E-10 0.84 inst_name E VCC_NORMAL VCC_SWITCHED 5.93E-09 7.23E-10 5.93E-09 7.23E-10 0.84</pre>
<code>-powering_up_rails net_name1 ... netname_n</code>	
	Default: None Specifies a list of powering up nets.

<code>-powering_down_rails net_name1... netname_n</code>	
	Default: None Specifies a list of powering down nets.
<code>-pre_simulation_period value</code>	
	Default: 0 Specifies the pre-simulation period that rail analysis will use. The supported time units are s, ms, us, ns, and ps. If no explicit time unit is specified, the default ns (nanosecond) will be applied. The pre-simulation period include the coarse-grain and fine-grain pre-simulation periods. If <code>-fine_pre_simulation_period</code> is not specified, then the entire pre-simulation period will do coarse pre-simulation.
<code>-pre_simulation_resolution value</code>	
	Default: 1e-10s Specifies the transient time step size or resolution. The supported time units are s, ms, us, ns, and ps. If no explicit time unit is specified, the default ps (picosecond) will be applied.
<code>-prechain_powerup_sequence_file filename</code>	
	Default: None Specifies a power-up sequence file that contains the pre-chain start/end and mother (data) chain start/end timing information. In the power-up flow, you can specify this pre-chain sequence file instead of a complete power-up sequence file to modify the ramp-up timing of different power switches using the timing constraint in the specified file. The timing information for the other power-gate instances is calculated using the pre-chain sequence file. The format of the file is: <pre> Prechain_start <pggate_instance_name> <pin_name> <d_time> <d_delay> <d_slew> Prechain_end <pggate_instance_name> Datachain_start <pgagate_instance_name> <pin_name><m_time> <m_delay> <m_slew> Datachain_end pggate_instance_name </pre>
<code>-print_gif_range_percentage {true false}</code>	

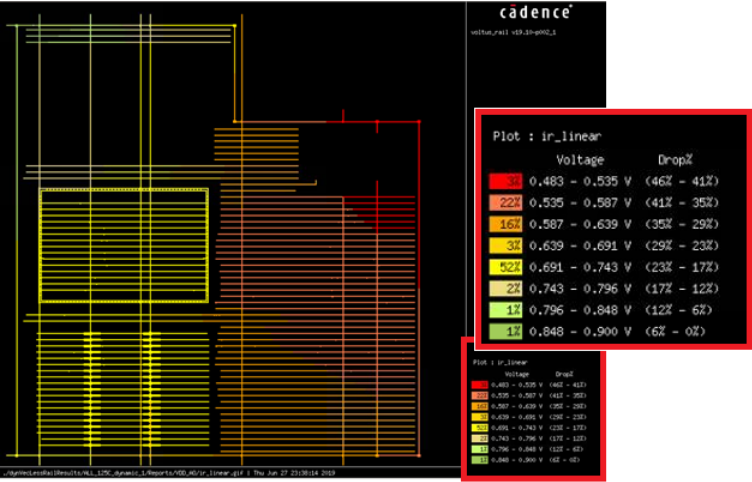
Default: false

Drop%

- **IR** (File Location: <rail_dir>/Reports/<power_net>/)
ir_linear.gif
ir_limit.gif
- **EIV** (File Location: <rail_dir>/Reports/)
<pg_pair>.<eiv_method>_eiv.gif
<pg_pair>.<eiv_method>_limit_eiv.gif

IR Drop% is calculated by $1 - (\text{voltage} / \text{ideal_voltage})$, and EIV Drop% is calculated by $1 - (\text{effective_voltage} / \text{ideal_effective_voltage})$

The following image shows the voltage drop percentage for the ir_linear plot:



-probe_instance_pin_waveforms {true | false}

Default: false

Specifies to probe current and voltage waveforms for all the pins belonging to probing instances. When -probe_instance_pin_waveforms is set to true, it writes all the voltages and currents of the pins of probing instances to the waveform files.

The use model of this parameter is:

set_rail_analysis_mode -method dynamic -probe_instance_pin_waveforms true

The watched instance has the pin names in the waveform file, as shown below:

Voltage waveform name: <instance_name>:ir

Current waveform name: TC_<instance_name>:ir

-probe_instance_tap_waveforms {true | false}

	Default: <code>false</code> Specifies to probe current and voltage waveforms for all tap nodes belonging to probing instances. When <code>-probe_instance_tap_waveforms</code> is set to <code>true</code>, it writes all the tap node voltages and currents of the instances from <code>-probe_waveform_list</code> to the waveform files. The use model of this parameter is: <pre>set_rail_analysis_mode -method dynamic -probe_instance_tap_waveforms true - probe_waveform_list instance_list_file</pre> The instance's tap nodes are named based on instance name, and its node ID and location in the waveform file, as shown below: Voltage waveform name: <code><instance_name>:N...:X...:Y_:ir</code> Current waveform name: <code>TC_<instance_name>:N...:X...:Y_...</code>
<code>-probe_package_interface_ports {true false}</code>	
	Default: <code>false</code> Specifies to probe voltages for all the ports of the top-level subcircuit and the interfaces within the package. When this parameter is specified, the rail analysis engine retrieves the package interface ports from the package model and writes them into a probing node file to watch their voltage waveforms.
<code>-probe_pin_voltage_list filename</code>	
	Default: <code>None</code> Writes the voltage of the interface nodes for the specified instance/cell to a block boundary voltage file. The software generates a block boundary voltage file with file name based on the block instance name, <code><instance_name>.pp</code> for every instance/cell in the list.
<code>-probe_waveform_list filename</code>	
	Default: <code>None</code> Specifies the name of a file containing the instance and cell name(s) for which you want to capture voltage and current waveforms. This parameter allows you to control whether to probe voltage or current waveforms or both. You can specify the optional parameter <code>PROBE <VOLTAGE CURRENT ALL></code> in the file. Default is <code>ALL</code>. When <code>VOLTAGE</code> or <code>CURRENT</code> is specified, only the voltage or current waveforms will be probed. The format of the file is: <pre>CELL/INST <name></pre> Following is an example of the file: <pre>INST I1_1 INST I2_1</pre>
<code>-probing_node_file file_name</code>	

Default: None

Specifies the probe node file containing the probe point on a die. The file format for specifying probing nodes is as follows:

```
<user defined unique probe name> <location = x> <location = y> <layer>
<Power/ground net name> type=rc/ir
```

When type is `rc`, probes resistor current, and when type is `ir`, probes node voltage. If type is not given, the tool probes node voltage by default.

Following is an example of the probing node file:

```
probe1 8.39 239.37 Metal_6 VDD_AO type=ir
probe1 8.39 234.37 Metal_6 VDD_AO type=rc
```

The generated waveform is stored in `tran_<NET>.ptiavg`.

- Voltage waveform name: `N_<x>_<y>:ir:<probe name>`
- Current waveform name: `TC_NodeRes:x_..._y_...:x_..._y_...:<probe name>`

Example

Output directory:

```
<output_dir>/ALL_125C_dynamic_1/reports/rail/waveforms/tran_VDD_AO.ptiavg
```

Waveform name in `ptiavg`:

- `N_83900_2393700_6:ir:probe1`
- `TC_NodeRes:x_83900_y_2393700:x_83900_y_1483800:probe1`

You can also specify the package node voltages and resistor currents that are to be probed in the following format:

```
<probe name> pkg ir <pkg circuit node name>
<probe name> pkg rc <pkg circuit resistor name>
```

Example

```
probe_1 18 16 METAL_1 VDD

# pkg node voltage
probe_pkg_1 pkg ir GND.in0
probe_pkg_6 pkg ir VDD.in2

# pkg resistor current
probe_pkg_cur_1 pkg rc R_1_1
probe_pkg_cur_2 pkg rc R_5_2
```

Note: The units of x and y should be in microns.

```
-process_techgen_em_rules {true | false}
```

	Default: false Processes the EM rules defined in the extraction technology file. When <code>-process_techgen_em_rules</code> is specified with <code>-extraction_tech_file</code>, the extraction technology file will be processed. If <code>-process_techgen_em_rules</code> is specified without <code>-extraction_tech_file</code>, the <code>.cl</code> model will be processed. The <code>-process_techgen_em_rules</code> and <code>-em_models</code> parameters are mutually exclusive. In the static mode, if <code>-em_models</code> is not specified, or if <code>-process_techgen_em_rules</code> is not set to <code>true</code>, current density (RJ) analysis will be disabled due to lack of EM models.
<code>-process_bulk_pins_for_body_bias {true false}</code>	
	Default: false When set to <code>true</code>, rail analysis will process current sinks on the cell's bulk pin connections. This parameter is used during body bias analysis, where a body bias net is connected to bulk pins of the cell. The bulk pins for a cell is defined in the cell library using the <code>set_pg_library_mode -bulk_power_pins -bulk_ground_pins</code> command parameter. If bulk pins are defined using <code>set_pg_library_mode -power_pins -ground_pins</code>, then the <code>-process_bulk_pins_for_body_bias</code> parameter must not be used.
<code>-promote_pin_shapes_only {filler decap both}</code>	
	Default: None Specifies to promote either only filler shapes, only decap shapes, or both types of pin shapes during rail analysis. When this parameter is specified, the interface nodes for the filler and decap cells are ignored. The <code>-promote_pin_shapes_only</code> parameter helps to reduce the resistor and node count for big designs without affecting connectivity. This parameter must be specified with <code>set_rail_analysis_mode -enable_2d_partition_extraction true</code> (enabled by default for the XP mode).
<code>-rdl_def def_file</code>	
	Default: None Specifies to instantiate the RDL instance/DEF at the top level and merge it with the DEF of the top level block being analyzed. The following is the usage model: <pre>set_rail_analysis_mode -rdl_def <def_file> -rdl_placement { X Y }</pre> For GUI display, the same parameters also work for <code>read_def</code>: <pre>read_def -rdl_def <def_file> -rdl_placement { X Y }</pre>
<code>-rdl_def_list {{rdl_filename1 x1 y1 orient1} {rdl_filename2 x2 y2 orient2} ...}</code>	

	Default: None Specifies to import the list of the RDL DEF files for the top-level rail analysis. This parameter allows to import multiple RDL DEFs with their location and rotation information in the top level. <u>Example:</u> <pre>-rdl_def_list { {RDL1.def 0 0 N} {RDL2.def 100 200 N}}</pre>
<code>-rdl_placement {X Y}</code>	
	Default: None Specifies to place the RDL at a given X,Y location (in micron).
<code>-rdl_orientation { N S W E FN FS FE FW }</code>	
	Default: None Specifies the orientation of the RDL DEF file.
<code>-read_thermal_map thermal_map_file</code>	
	Default: None Specifies to process the thermal map file to calculate resistance based on uniform temperature. Resistor scaling is supported for the tile-based power map file. Using the tile-base thermal map from Sigrity PowerDC, the software will scale the resistance value based on the temperature of the resistor's physical location, which provides more accurate IR and EM results with thermal impact. For more information on the thermal map file format, refer to the Thermal Map File section in the "File Formats" chapter of the <i>Voltus User Guide</i>.
<code>-record_inst_peak_current {true false}</code>	
	Default: false Specifies to save the instance peak current during rail analysis in the rail state directory. This parameter allows you to check and debug occurrences of abnormal instance peak current, which may result in high IR drop.
<code>-record_results_start_time time</code>	
	Default: 0 Records rail analysis results based on the specified start time. The default unit of this parameter is ns. The software will use the specified time for all types of analysis, for example, instance voltage calculation, dynamic waveforms, and power-gate optimization.
<code>-reff_detail_report {true false}</code>	

Default: false

Specifies to include the following additional columns for instance pins in the effective resistance (effr.rpt) report:

- **x** and **y**: location (x/y coordinates)
- **LAYER**: layer name
- **PIN_NODES (FLOAT/TOTAL)**: number of floating nodes per total number of nodes of the instance

By default, these columns are not included in effr.rpt.

Usage Guidelines:

- Applicable only to the resistance analysis flow (analyze_resistance).
- Supports both the static and dynamic rail analyses.
- Supports both the net-based and domain-based analyses.
- Does not affect domain_effr.rpt.

-reff_domain_report_format {1 | 2 | 3 | 4}

Default: 1

Controls the format of the effective resistance analysis domain report (domain_effr.rpt). This parameter allows you to specify the level of detail to be displayed in the domain-based analysis report.

The following table gives the report format for the four format values:

Format Value	Format
1 (1 is the default format)	# PASS/FAIL REFF INSTANCE PIN(Pin REFF Percentage) Example: <pre> FAIL 10.0 instanceName VDD1(50.00%) VDD2(30%) VSS1(10.00%) VSS2(10%) </pre>
2	# PASS/FAIL REFF INSTANCE POWER-PIN GROUND-PIN (Pin REFF Percentage) Example: <pre> FAIL 6.0 instanceName VDD1(83.33%) VSS1(16.67%) </pre>
3	# PASS/FAIL REFF INSTANCE PIN (Pin Absolute REFF Value, Pin REFF Percentage) Example: <pre> FAIL 10.0 instanceName VDD1(5.0, 50.00%) VDD2(3.0, 30%) VSS1(1.0, 10.00%) VSS2(1.0 10%) </pre>
4	# PASS/FAIL REFF INSTANCE PWRNET PIN Layer X Y % GNDNET PIN Layer X Y % Example: <pre> FAIL 22.709 instanceName VDD pinName layerName 571.340 473.060 value(37.49%) VSS pinName Layername 571.340 471.800 value(62.51%) </pre>

Usage Guidelines:

- Applicable only to the resistance analysis flow (analyze_resistance).
- Supports both the static and dynamic rail analyses.
- Applicable only to the domain-based analysis. It is not supported in the net-based analysis flow.

<code>-reff_eval_nodes {port tap}</code>	
	Default: <code>port</code> Reports effective resistance based on the port (pin) or tap nodes. By default, the software uses the port nodes for REFF reporting.
<code>-reff_pin_report_layer { top bottom all }</code>	
	Default: <code>all</code> Specifies which layer of the pin shapes are to be reported in the EDDR report. This parameter allows you specify which cell boundary pin layers will be used for the resistance report. The possible arguments of this parameter are: • <code>top</code>: Specifies to report only the pin shapes located on the highest pin layer. • <code>bottom</code>: Specifies to report only the pin shapes located on the lowest pin layer. • <code>all</code>: Specifies to report the pin shapes located on every layer. Usage Guidelines: • Applicable only to the resistance analysis flow (<code>analyze_resistance</code>). • Supports both the static and dynamic rail analyses. • Supports both the net-based and domain-based analyses.
<code>-reff_pin_report_method { best worst avg }</code>	

	Default: <code>worst</code> Specifies whether to report the best or worst effective resistance value of an instance pin with multiple nodes. The possible arguments of these parameters are: • <code>best</code>: Reports the smallest resistance value among the nodes of instance pin shapes for REFF. • <code>worst</code>: Reports the largest resistance value among the nodes of instance pin shapes for REFF. • <code>avg</code>: Reports the average resistance value among the nodes of instance pin shapes for REFF. Usage Guidelines: • Applicable only to the resistance analysis flow (<code>analyze_resistance</code>). • Supports both the static and dynamic rail analyses. • Supports both the net-based and domain-based analyses. Usage Models • <code>-reff_pin_report_method avg</code> - Reports only the average effective resistance value for instance pins with multiple nodes • <code>-reff_pin_report_method {worst avg best}</code> - Reports comprehensive effective resistance data: best, worst, and average values for instance pins with multiple nodes • This parameter supports generating multiple report types simultaneously. Each report type is automatically saved to a separate file, making it easy to access specific analysis results. Example: <pre>set_rail_analysis_mode -reff_pin_report_method { best avg worst}</pre> Output files: <pre>net.best.effr net.avg.effr net.worst.effr domain_effr.best.rpt domain_effr.avg.rpt domain_effr.worst.rpt</pre>
<pre>-reff_report_all {true false}</pre>	
	Default: <code>false</code> Reports both fail and pass instance pins in the REFF report. By default, only the "FAIL" instance pins are reported.
<pre>-remove_duplicate_inst {true false}</pre>	

	Default: <code>false</code> Specifies to remove duplicate cell instances during the DEF parsing stage of the rail analysis run. These duplicate cell instance names are from the same DEF file, and are also printed in a warning message so that you are not required to debug these disconnected instances. When this parameter is set to <code>true</code>, the duplicate cell instances within the same DEF file will not be reported as disconnected instances in the <i>Physically Disconnected Instances</i> section of the power-grid integrity report.
<code>-report_compress_level {0 1-9}</code>	
	Specifies to compress the rail output files in the GNU Zip format. The compressed files will have the suffix <code>.gz</code>. • <code>0</code>: default, no compression • <code>1-9</code>: the compression levels vary from 1 to 9. 1 provides the fastest compression speed but at a lower ratio, so the file size will be larger. 9 provides the highest compression ratio but at a lower speed.
<code>-report_instances_missing_current_data {true false}</code>	
	Default: <code>false</code> Specifies to generate a report that includes a list of all instances with missing current data. This parameter is supported in both the net-based and domain-based analyses. When this parameter is specified, the <code><net_name>.nocurrentdata.inst.rpt</code> file is created in the <code><state_directory>/<net_name>/</code> directory. In addition, the log file includes the annotation statistics for instances with current data available on a per-net basis, as shown below: <pre>#Instances missing current data: #count/#total_instances (%#percent value)</pre>
<code>-report_layer_based_ir_average {true false}</code>	
	Default: <code>false</code> Specifies to add the average IR drop value to the layer-based IR drop report and log file.
<code>-report_layers_above_pin_for_instance_ir {true false}</code>	
	Default: <code>false</code> Specifies to report nodes above the pin layer. With this parameter set to <code>true</code>, the report will also contain layers above the pin nodes to the top most routing layer. The reporting will be based on the largest rectangular bounding box of the instance.
<code>-report_power_options option_names</code>	

	Default: None This parameter is used when <code>-report_power_in_parallel</code> is set to <code>true</code>. When the <code>-report_power_in_parallel</code> parameter is specified, you will not run the <code>report_power</code> command, therefore the reporting parameters cannot be specified. Using the <code>-report_power_options</code> parameter, you can specify the required <code>report_power</code> command parameters that are to be executed in parallel with rail analysis.
<code>-report_msmv_format {true false}</code>	
	Default: <code>true</code> When set to <code>true</code>, the software generates an instance voltage file for each MSMV power domain in the design, e.g. <code>VDD_VSS.iv</code> and <code>VDDm_VSS.iv</code> where <code>VDD</code> and <code>VDDm</code> are MSMV power nets with common <code>VSS</code> net.
<code>-report_power_in_parallel {true false}</code>	
	Default: <code>false</code> Enables power analysis in parallel with rail analysis to improve the overall turnaround time. Note: When using the Voltus-XP flow with the <code>-report_power_in_parallel true</code> parameter, if there are any parameters specified in both <code>set_power_analysis_mode</code> and <code>set_rail_analysis_mode</code>, the tool will honor the settings from <code>set_rail_analysis_mode</code>. For example, the power grid library defined in <code>set_rail_analysis_mode</code> will be used for power analysis in Voltus-XP. For information on the parallel flow setup, refer to the "Run Dynamic Power Analysis and Rail Analysis in Parallel" section in the <i>Dynamic Power and IRDrop Analysis</i> chapter of the <i>Voltus User Guide</i>.
<code>-report_shorts {true false}</code>	

	Default: <code>false</code> Specifies to check for power and ground shorts between two geometries belonging to different nets. The parameter performs power and ground short check for the following: • PG and PG nets • PG and signal nets • All extracted nets When the parameter is set to <code>true</code>, if shorts are detected, the file <code>"voltus_short.report"</code> will be generated in the extraction work directory. The format of the report file is: <i>LayerName X-coord Y-coord NetName1 NetName2</i> If one of the net is not an extracted net (that is, signal net or non-extracted PG net), it will be marked as N/A in the report. The following is the use model of the parameter: <pre>set_rail_analysis_mode -report_shorts true</pre> The following is a snippet from <code>voltus_short.report</code>: <pre># Layer X Y NET NET AP 98.500 211.000 VSS N/A AP 98.500 219.550 VSS N/A AP 98.500 301.000 VDD25_LVTTL N/A AP 98.500 309.550 VDD25_LVTTL N/A M8 13007.800 2267.200 VSS VDD ...</pre>
<code>-report_voltage_drop {true false}</code>	
	Default: <code>false</code> When set to <code>true</code>, instead of reporting the actual (effective) instance voltage, all the values will be reported in terms of voltage drop/bounce. For example, if the full pwr-gnd is 1V, pwr is 0.9 and gnd is 0.1, the IV file will report 0.8 (0.9-0.1) if this parameter is set to <code>false</code>, or 0.2 (0.1drop+ 0.1bounce) if the parameter is set to <code>true</code>.
<code>-reset</code>	Default: None Specifies to reset all the <code>set_rail_analysis_mode</code> command parameters to their default values.
<code>-reuse_state_directory <i>dir_name</i></code>	

	Default: None Specifies an existing state directory. The software assumes that the design data (DEFs and GDS) and power-grid libraries are not changed, and will re-start analysis of the design using power-grid database inside the specified state directory. If the state directory does not contain the necessary power-grid database files, it will issue an error message and provide information on the missing files. For net based analysis, you should specify net-based state directory, as shown below: • <code>set_rail_analysis_mode -reuse_state_directory VDD_25C_avg_1</code> For domain based analysis, you should specify the domain's state directory, as shown below: • <code>set_rail_analysis_mode -reuse_state_directory domain_25C_dynamic_1</code> A new state directory will be generated for the new analysis with soft links to the old state directory files that were reused.
<pre>-rlrp_cell_report_layer filename</pre>	
	Default: None Specifies the cell names and layer of the pin shapes that are to be reported in the RLRP report. This parameter allows you to perform cell-based layer reporting for instance RLRP report. The format of the specified input file is: <code>#cell_name #netname #pinlayer</code> <code>netname</code> is either a netname or "ALL". If you specify <code>ALL</code> for a net name, all the nets in the analysis domain will be considered for RLRP reporting. Example of an Input File <pre>PDVDD_18_18_NT_DR_V ALL {M9,M1} PDVDD_18_18_NT_DR_V DVDD_D1 {M9,M1} PDVDD_18_18_NT_DR_V DVDD_D1 {M9,M1} VSS_D1 {M2}</pre>
<pre>-rlrp_detail_report {true false}</pre>	

	Default: <code>false</code> Specifies to trace and report the total power gate resistance along the RLRP path for each instance. The RLRP report (<code><netName>.rlrp_inst</code>) has the following additional columns for each instance: • <code>RLRP_TOTAL</code> - Total RLRP resistance including the power gate path (from the always-on and switched nets) • <code>NET_RES</code> - Total RLRP resistance excluding the power gate path • <code>PGATE_RES</code> - RLRP resistance for the power gate path • <code>PGATE_RON</code> - Number of power gate RON resistors The report allows you to trace the Ron resistors in each power gate instance and generate the RLRP report based on power gate instances. The default value of this parameter is <code>false</code>.
<code>-rlrp_eval_nodes {port tap}</code>	
	Default: <code>port</code> Reports RLRP based on the port (pin) or tap nodes. By default, the software uses the port nodes for RLRP reporting.
<code>-rlrp_percentage_threshold value</code>	
	Default: <code>100</code> Specifies to output top x% of the instances with large RLRP.
<code>-rlrp_pin_based_report {true false}</code>	
	Default: <code>false</code> Specifies to generate a pin-based RLRP report. When specified, a new column "<i>Pin</i>" is included in the RLRP report.
<code>-rlrp_pin_report_layer { top bottom all }</code>	
	Default: <code>all</code> Specifies which layer of the pin shapes are to be reported in the RLRP report. This parameter allows you specify which cell boundary pin layers will be used for the resistance report. The possible arguments of this parameter are: • <code>top</code>: Specifies to report only the pin shapes located on the highest pin layer. • <code>bottom</code>: Specifies to report only the pin shapes located on the lowest pin layer. • <code>all</code>: Specifies to report the pin shapes with best or worst resistance value among all layers.

<code>-rlrp_pin_report_method { best worst eiv_best eiv_worst }</code>	
	Default: <code>worst</code> Specifies whether to report the best or worst RLRP to an instance pin with multiple nodes. The possible arguments of these parameters are: • <code>best</code>: Reports the smallest resistance value among the nodes of instance pin shapes for RLRP. • <code>worst</code>: Reports the largest resistance value among the nodes of instance pin shapes for RLRP. • <code>eiv_worst</code>: Reports the RLRP of the node that has the worst EIV value among the tap nodes of instance pin shapes. • <code>eiv_best</code>: Reports the RLRP of the node that has the best EIV value among the tap nodes of instance pin shapes. This parameter supports generating multiple report types simultaneously. Each report type is automatically saved to a separate file, making it easy to access specific analysis results. Example: <code>set_rail_analysis_mode -rlrp_pin_report_method { best worst }</code>
<code>-rlrp_threshold <i>value</i></code>	
	Default: <code>0.0</code> Specifies to ignore instances with RLRP less than the specified threshold value.
<code>-rms_em_analysis {true false}</code>	
	Default: <code>false</code> Specifies to calculate the RMS EM in the dynamic rail analysis run. This parameter generates the <code><net_name>.rj.rms.rpt</code> report. When not specified, it will be enabled automatically if the EM model files are specified. You can set this parameter to <code>false</code> to skip the RMS EM analysis even if the EM models are provided.
<code>-save_voltage_waveforms {true false}</code>	
	Default: <code>false</code> When set to <code>true</code>, instance voltage waveforms are saved in the state directory (.ptiavg). This waveform file can be loaded in the <code>view_dynamic_waveform</code> command to probe voltage waveform for desired instances. This waveform file can be fed to Tempus to perform SPICE critical path analysis with IRdrop waveforms. It can be used in Substrate Noise Analysis as well.
<code>-scale_initial_condition_current <i>filename</i></code>	

	Default: None Specifies a file containing the current scale factor for the initial condition of the specified nets. The parameter allows you to increase/decrease the initial current at time 0 to minimize oscillation in the package and to shorten the settling time. The format of the file is: <pre>default <default scale factor for all nets> <net name> <scale factor for this net> ...</pre> The use model of this parameter is: <pre>set_rail_analysis_mode -method dynamic -scale_initial_condition_current scale_factor.file</pre> The content of <i>scale_factor.file</i> is: <pre>default 1.0 VDD 0.8 VSS 0.7</pre>
<code>-seb_lifetime value</code>	
	Default: 43800 hours (that is, 5 years) Specifies the life time of product in hours.
<code>-seb_table table_filename</code>	
	Default: None Specifies the file name of the Failures in Time (FIT) table. This file is provided by the foundry. The file has the following columns: • Layer • MTF • Sigma • Activate energy • Current density exponent
<code>-seb_temperature temperature</code>	

	Default: If not specified, the internally calculated junction temperature is used Specifies the temperature used to update the median time to failure (MTF) with respect to the ambient temperature. • If this temperature is specified, it would be used globally for all the resistors. • If this temperature is not specified, the <code>seb_temperature</code> may take the following possible values depending on the specific circumstances: ◦ If detailed delta temperature file is not used for SEB, the <code>seb_temperature</code> always takes the same value as that used for EM analysis. ◦ If detailed delta temperature file is used for SEB, the <code>seb_temperature</code> is calculated by the following equation: $\text{seb_temperature} = \text{env_temperature} + \text{delta_T}$ Where <code>delta_T</code> is the maximum temperature rise, including joule heat and coupling heat from FEOL, in the resistor's bounding box with zero extension.
<code>-set_analyze_bbox {xmin ymin xmax ymax}</code>	
	Specifies the X, Y, min and max coordinates of the bounding box in which the instances can be analyzed. The values of the coordinates are specified in microns (um). Based on physical partitioning, the bounding box coordinates analyzed by the tool could be similar or greater than the user-specified coordinates. You can view the coordinates of the actual analyzed bounding box in the info message in the main Voltus log file. The following is an example of info message that appears in the main Voltus log file: INFO: The analyzed bounding box co-ordinates of the design are { 0.0, 0.0, 355.0, 380.0 }
<code>-shorting_resistance value</code>	
	Default: 0 Specifies the minimum shorting resistance value (in ohm) assigned to resistors. When specified, those resistors with resistance equal or less than the specified value are merged to reduce the size of the R network.
<code>-single_partition_extraction_layer {METAL_LAYER}</code>	
	Specifies to extract RC using a single partition on a top metal layer. This helps to minimize extraction fracturing at partition boundaries, leading to better RC grid quality. Example: <pre>set_rail_analysis_mode -single_partition_extraction_layer {METAL_8}</pre>
<code>-single_partition_extraction_layer_list {METAL_LAYER_1 METAL_LAYER_2 }</code>	

	Specifies to extract RC using a single partition on multiple layers. This helps to minimize extraction fracturing at partition boundaries, leading to better RC grid quality. Example: <pre>set_rail_analysis_mode -single_partition_extraction_layer_list {METAL_5 METAL_8}</pre>
<code>-skip_extraction {true false}</code>	
	Default: <code>false</code> Specifies to reuse a work directory. It is recommended to use this parameter when you want to reuse the previous extraction work directory during the net-by-net rail analysis flow.
<code>-snap_layer_for_current_taps <i>file_name</i></code>	
	Default: None Specifies the layer file name for adaptive current modeling. It snaps current taps from lower via layers to higher via layers for better performance. This feature is supported for EM view of both macro PGV and hierarchical PGV. The current taps from lower layers will be processed based on the specified tile size, and redistributed equally across all the vias within the tile at the higher via layers. The default tile size is 5umx5um (25 um²) for macro PGV, and 25umx25um (625 um²) for hierarchical PGV. For example, when running full chip analysis, you already have detailed analysis on the sub-blocks or hierarchical blocks so you might consider speeding up the analysis by merging the current taps from lower layer to spread it across higher via layers. The current drawn from within the PGV will still remain the same so the accuracy at the top level will not be affected. The format of the file is: #Instance Based <pre>INST <inst_name> <via_layer_name> <tile_size></pre> #Cell Based <pre>CELL <cell_name> <via_layer_name> <tile_size></pre> <code>via_layer_name</code> is the layer name used in the LEF file. <code>tile_size</code> is optional.
<code>-static_multi_mode_analysis {true false}</code>	

	Default: <code>false</code> Specifies to analyze the worst-case static IR drop across multiple functional modes of operation for macros. When this parameter is specified, Rail Analysis will use the multi-mode static current file (<code>set_power_analysis_mode - static_multimode_scenario_file</code>) to iterate over multiple scenarios for all macro instances, and output the worst-case IR and EM report. The generated report identifies the worst Differential Instance Voltage (DIV) and the scenario that produces worst DIV across all instances. This parameter generates the following report in the Reports folder: <code>State_dir/Reports/powerNetName_gndNetName.worst_scenario_div.rpt</code> - this file reports each instance's worst scenario. It lists the instances based on worst DIV in the ascending order to determine the worst among all worst instance DIVs. The following is a snippet from the worst scenario report: <pre>#INST_NAME SCENARIO_ID DIV block19/ICG4/ICG 12 -0.107583 block19/ICG3/ICG 12 -0.107557 ...</pre> Here, <code>SCENARIO_ID</code> is the worst time in the current file.
<code>-static_trigger_file filename</code>	
	Default: <code>None</code> Specifies a file containing user-defined xPGV current scaling information on a per PG net basis for static analysis. This parameter is used for scaling of currents stored in xPGV. You must provide scaling for all PG nets associated with the xPGV in the trigger file. The parameter allows you to use the same xPGV in different designs, irrespective of the voltage at which the xPGV is generated. The format of the user-specified trigger file for a net is: <pre><INSTANCE CELL> inst_name/cell_name time scale_factor mode ref net</pre> Instance-based scale factors get higher precedence than the cell- based scale factors. Example of a Trigger File: <pre>INSTANCE inst_C1 0 3 IPPGV::DEFAULT::super_filter 0 VDDD1 INSTANCE inst_C1 0 2 IPPGV::DEFAULT::super_filter 0 VDDC1 INSTANCE inst_C1 0 3 IPPGV::DEFAULT::super_filter 0 VSSA1</pre> <code>-enable_xpgv_scaling</code> and <code>-static_trigger_file</code> are mutually exclusive.
<code>-suppress_message { message_id + }</code>	
	Default: No messages are suppressed. The messages with the specified message ids will not be output.

`-switchbit_file_for_eiv_calculation filename`

Default: None

Specifies the file that includes the cell or instance based switch-bit information. This parameter allows you to override the switch-bit in a PGV, or specify the switch-bit for a specific cell or instance.

Note: The switch-bit information is defined for memory cells to distinguish between the pre-charge and functional mode (read/write) operation. During pre-charge, the memory cells may have large IRdrops due to large demand current in a short period of time. However, these cells can tolerate the large IRdrops because word-lines are shut-off. To accurately measure the effect of IRdrop on the operation of a memory, the instance voltage should be calculated during the functional mode. The switch-bit signal is used by the software to calculate the instance voltage when memory is in the functional mode.

The format of the specified file is:

```
CELL | INST <cell_name | instance_name>
<MODE_NAME> <Tmin1> <Tmax1> <Tmin2> <Tmax2>
```

where,

- *cell_name/instance_name* must have a PGV with the dynamic current waveform.
- *MODE_NAME* must match one of the mode names in the PGV for the cell.
- *Tmin* and *Tmax* is the window when the switch-bit is ON, and when EIV will be calculated. The default time unit is ns.

The Rail Analysis engine reads this switch-bit information, and reports the elapsed and worst instance voltage drop (IVD) when the switch-bit is 1. Rail Analysis will generate two IVD values for a given instance in the EIV report:

- Worst IVD across simulation (*ELAPSE_EIV*)
- Worst IVD in switching window (*WIN_EIV*)

`-temp_directory_name directory`

Default: None

Specifies a directory to use for temporary files.

You can also use the global environment variable *TMPDIR* to specify an alternative location for the temporary data. If both are specified, the directory specified using the `-temp_directory_name` parameter will take precedence.

Following is an example for setting the *TMPDIR* environment variable:

```
setenv TMPDIR ./temp_directory
```

`-temperature value`

	Default: 25C Specifies the temperature in degree celcius during rail analysis. The default temperature is 25 degree celsius. This is an optional parameter.
<code>-topcell_placement {X Y}</code>	
	Default: None Specifies to place the top DEF block at a given X,Y location (in micron) with respect to virtual top (0,0). The following is the usage model: <pre>set_rail_analysis_mode -topcell_placement { X Y } -topcell_orientation { N S W E FN FS FE FW }</pre> For GUI display, the same parameters also work for read_def: <pre>read_def -topcell_placement { X Y } -topcell_orientation { N S W E FN FS FE FW }</pre>
<code>-topcell_orientation { N S W E FN FS FE FW }</code>	
	Default: None Specifies the orientation of the top DEF block.
<code>-tsv_subckt_modelfile_list filename</code>	
	Default: None Specifies the name of the external Through Silicon Via (TSV) subckt model files to be used for 3D-IC extraction. You can specify a list of files separated by space. Each file contains the subckt model information for TSV. The specified file has the Spice format. A sample file is shown below. <pre>.subckt TSV top bottom R1 top n2 0.03075 R2 n2 bottom 0.03075 C1 n2 0 28f .ends</pre>
<code>-unconnectedcell_ignore_file filename</code>	

	Default: None Specifies the name of a file containing the list of cells and instances to be ignored while reporting connectivity issues in the .rpt files, log file, and GUI. This parameter allows you to filter out the cells/instances that are known to have connectivity related issues and are not to be fixed. The format of the file is: <pre>CELL/INST Cell Name/Instance Name Pin Name</pre> Here: • <code>CELL</code> and <code>INST</code> are keywords • Pin names are not mandatory. If pin names are not given, the corresponding cell/instance is ignored. Example: <pre>CELL AND => ignore all AND cell(s) INST INST1 => ignore instance INST1 CELL BLOCK2* => ignore all blocks with the name starting with BLOCK2 CELL * => ignore all cells INST * => ignore all instances CELL AN1 VDD* => ignore AN1 cells's VDD* pins INST INST2 {VDD1 VDD2} => ignore multiple pins of instance INST2 INST INS* VDD => ignore VDD pin of all instances with the name starting with INS</pre>
<pre>-unconnected_die_pkg_pins {ignore error edit}</pre>	
	Default: ignore Specifies whether or not to ignore unconnected pins between die and package (if any), and consequently if the simulation should be paused, stopped, or continued. This option enables the manual connection between package and die through the <i>MCP Editor</i> utility if any disconnects are present.
<pre>-unified_power_switch_flow {true false}</pre>	
	Specifies to enable the unified analysis of the always-on and switch nets. When set to <code>true</code>, the power-up and always-on analyses are performed in a single flow, wherein the switch nets and always-on nets are simulated together resulting in improved performance and accuracy. Use Model: • Always-on Flow: <code>set_rail_analysis_mode -unified_power_switch_flow true</code> • Power-up Flow: <code>set_rail_analysis_mode -unified_power_switch_flow true and -powering_up_rails <powerup_rail_list></code>
<pre>-use_early_view_list filename</pre>	

	Default: None Specify cells in the file for which you want to use Early power-grid views. The file format accepts cell name in each line. You can specify wildcards (*, ?, -) in cell names.
<code>-use_em_view_list filename</code>	
	Default: None Specify cells in the file for which you want to use EM power-grid views. The file format accepts cell name in each line. You can specify wildcards (*, ?, -) in cell names.
<code>-use_ir_view_list filename</code>	
	Default: None Specify cells in the file for which you want to use IR power-grid views. The file format accepts cell name in each line. You can specify wildcards (*, ?, -) in cell names.
<code>-use_rms_delta_t {true false}</code>	
	Default: <code>false</code> When this parameter is set to <code>true</code>, the <code>delta_T</code> used to calculate <code>seb_temperature</code> would be based on the RMS delta T (joule heat only). If this parameter is set to <code>false</code>, the <code>delta_T</code> used to calculate <code>seb_temperature</code> would be based on the sum of self-heat delta T and RMS delta T.
<code>-verbosity {true false}</code>	
	Default: <code>false</code> Specifies to get side logs for extraction and rail analysis. The default value of this parameter is <code>false</code>.
<code>-verify_logical_connectivity {true false}</code>	
	Default: <code>false</code> Specifies to report the list power and ground pins of each cell instance that do not have logical connectivity. The list of pins with missing logical connectivity are saved in a report file called <code>cell.unconnected_pins</code> under the extraction work directory.
<code>-vsrc_search_distance value</code>	
	Default: 50 Specifies the search distance in microns for adding voltage sources at power pin locations.
<code>-watch_location_waveform { {layerName xCoord yCoord}+ }</code>	

	Default: None Specifies to perform node based voltage probing for rail analysis. You can use this parameter to observe voltage waveform at a given location, specified by x,y coordinates and the metal layer. The unit of x,y coordinates is um, and the layer name should be a LEF/DEF layer.
<code>-what_if_shapes_file filename</code>	
	Default: None Specifies to take the what-if shape specification from a file. This file is the file created with the <code>-save</code> parameter of the <code>create_what_if_shape</code> command. Note: If both the <code>-import_what_if_shapes</code> and <code>-what_if_shapes_file</code> parameters are specified, union of two sets of what-if shapes is considered for analysis.
<code>-work_directory_name directory</code>	
	Default: None Specifies the work directory location for extraction data. When you specify this parameter, the software generates extraction temporary files in the specified directory.
<code>-write_demand_current_region_file filename</code>	
	Default: None Specifies a file that contains regions for custom demand current generation. This parameter allows to compute the total current within each given region, and sum up all tap currents that are outside the specified regions as a global port. The format of the region file is: <pre><REGION_PORT_LABEL> <XL YL XR YR> <NET></pre> here, • <code>REGION_PORT_LABEL</code> specifies a name for the region. • <code>XL YL XR YR</code> specifies the enclosing region coordinates in microns. • <code>NET</code> specifies the power/ground net name for analysis. Example file: <pre>A 0 0 100 100 VDD_AO</pre> When specified, one demand current waveform "<code>tran_<netname>_region.ptiavg</code>" will be generated per net in the rail output directory at: • For DP: <code>./Reports/Waveforms/tran_<netname>_region.ptiavg</code> • For XP: <code>./reports/rail/waveforms/tran_<netname>_region.ptiavg</code>
<code>-xpgv_config_file filename</code>	

	Default: None Specifies block details, such as, block name for which xPGV is generated, via cut layer name that indicates to preserve the layers about the cut layer, and block LEF file names. This parameter is specified during block-level analysis. This parameter is not supported with the <code>analyze_resistance</code> command. Configuration File Format: <pre>CELL <block_name> CUT_LAYER <cut_via_layer_name> Lef <tech_lef_name block_lef_name> END</pre> Note: This parameter is not supported with the <code>analyze_resistance</code> command.
<code>-xp_cpu_per_job_power value</code>	
	Default: If this parameter is not specified, the value will be determined automatically based on the number of host resources. Specifies the number of CPUs per power analysis job in the XP mode (<code>-enable_xp</code>). The specified value must be greater than 0. This value should not exceed the value specified with <code>set_multi_cpu_usage -cpuPerRemoteHost</code>.
<code>-xp_cpu_per_job_simulation value</code>	
	Default: If this parameter is not specified, the value will be determined automatically based on the number of host resources. Specifies the number of CPUs per simulation job in the XP mode (<code>-enable_xp</code>). The specified value must be greater than 0. This value should not exceed the value specified with <code>set_multi_cpu_usage -cpuPerRemoteHost</code>.
<code>-xp_host_allocation_method {on_demand at_startup}</code>	
	Default: <code>on_demand</code> Specifies the method for remote host allocation during distributed processing. The possible arguments are: <code>at_startup</code> – Specifies to wait for all available remote hosts before starting rail analysis. This parameter allocates all available remote hosts specified with <code>set_multi_cpu_usage at startup</code>. <code>on_demand</code> - Allocates the available remote hosts based on demand.
<code>-xp_purge {full}</code>	

	Default: None Specifies to clean up disk after run completion to delete temporary directories created for each analysis step (data generated in the <code>voltus_data</code> directory). If this parameter is specified, the reuse parameters (<code>-reuse_state_directory</code> and <code>-xp_reuse_extraction_directory</code>) will not work for the current state directory in the future runs. This parameter is applicable only in the XP mode (<code>-enable_xp</code>).
<code>-xp_resume {true false}</code>	
	Default: false Specifies to resume from the failed step, and skip the previously successful run steps. This parameter is applicable only in the XP mode (<code>-enable_xp</code>).
<code>-xp_reuse_extraction_directory directory_name</code>	
	Default: None Specifies the path to the state directory that contains the extraction power-grid database. This parameter allows you to skip extraction and reuse the extraction data from a previous run in the XP mode.
<code>-xp_reuse_hpt_directory state_directory</code>	
	Default: None Specifies to reuse the hierarchical path tracing (HPT) data from the HPT directory of a previous run with the modified extraction options if there are no DEF changes in the component section or logical connectivity. However, you can still reuse the HPT data using this parameter if the DEF changes are related to routing shape.
<code>-xp_reuse_parse_def_directory directory_name</code>	
	Default: None Specifies to reuse the data of the parsed-DEF file from the specified directory. This parameter significantly reduces the runtime required for reading the large DEF files. Example: <pre>set_rail_analysis_mode -xp_reuse_parse_def_directory /DefHierReport/rail_dynamic/core_125C_dynamic_1</pre>
<code>-xp_simulation_cpu_timeout value</code>	
	Default: 7200 Specifies the time limit to either acquire the number of CPUs requested at the start of the run or acquire the number of CPUs specified with the <code>-xp_simulation_min_cpu</code> parameter in the XP mode. When this parameter is specified, the tool will wait for the given time limit to acquire the CPUs to run simulation before exiting. The unit for timeout is seconds. If a timeout value is not specified, the tool will wait for 7200 seconds (two hours) to acquire the CPUs.

`-xp_simulation_min_cpu value`

Default: If this parameter is not specified, the tool performs simulation with the available number of CPUs.

Specifies the minimum number of CPUs required for the rail simulation stage in the XP mode. The specified value must be greater than 0.

When this parameter is specified, the tool waits to acquire the specified number of CPUs before performing simulation. This parameter can be used if you want to perform simulation with the minimum number of CPUs required to proceed with simulation instead of waiting for all the CPUs needed during the run.

Examples

- The following command sets the rail analysis mode to static and xd accuracy, along with some other parameters:

```
set_rail_analysis_mode \
-method static \
-accuracy xd \
-power_grid_library fast_library/fast_allcells.cl \
-vsrc_search_distance 50
```

- The following command is an example of setting up rail analysis in the XP mode:

```
#resume previously failed XP run
set_rail_analysis_mode -enable_xp true -xp_resume true

set_rail_analysis_mode -method dynamic -accuracy hd \
...
-enable_xp true \
-xp_host_allocation_method at_startup \
-xp_reuse_extraction_directory dynamic_rail/core_125C_dynamic_1 \
-xp_cpu_per_job_power 2 \
-xp_cpu_per_job_simulation 8 \
-xp_purge full
```

- The following is an example of net-based early rail static analysis on an unplaced or partial placed design where virtual via/followpin would be created. The power would be distributed to placed instances on the basis of area. For unplaced instances, current region would be created on the M1 layer.

```
set_pg_nets -net vdd -voltage 0.998 -threshold 0.99301
set_power_pads -net vdd -format xy -file vdd.pp
set_power_data -format area -bias_voltage 0.998 -power 2.15
set_rail_analysis_mode -method era_static -accuracy xd -extraction_tech_file ./qrcTechFile -
era_current_distribution_layer M1 -era_current_distribution unplaced -era_skip_virtual_via_by_type
none -era_insert_virtual_followpin extended
analyze_rail -type net vdd
```

- The following is an example of domain-based early rail static analysis with virtual via/followpin. Here, user-specified

instance current files are used.

```
set_pg_nets -net VSS -voltage 0 -threshold 0.18
set_pg_nets -net VDDm -voltage 1 -threshold 0.856
set_pg_nets -net VDD -voltage 0.84 -threshold 0.756
set_power_pads -net VDD -format xy -file vdd.pp
set_power_pads -net VDDm -format xy -file vddm.pp
set_power_pads -net VSS -format xy -file vss.pp
set_power_data -format current {instance_current_files/static_VSS.ptiavg
instance_current_files/static_VDD.ptiavg instance_current_files/static_VDDm.ptiavg
instance_current_files /static_VDDlu.ptiavg instance_current_files/static_VDDau.ptiavg}
set_rail_analysis_mode -method era_static -accuracy xd -extraction_tech_file typical.tch -
era_power_gate_file pwr gates_port.txt -era_skip_virtual_via_by_type none -era_insert_virtual_followpin
extended
set_rail_analysis_domain -name PD -pwrnets {VDD VDDm} -gndnets VSS
analyze_rail -type domain PD
```

- The following is an example of net-based early rail analysis with user specified ASCII instance and current region file. Here, all virtual vias and followpin creation have been skipped.

```
set_pg_nets -net vss -voltage 0.0 -threshold 0.01
set_power_pads -net vss -format xy -file vss.pp
set_power_data -format ascii -bias_voltage 1 -power 1.5 instance_power_file.txt
set_power_data -bias_voltage 1 -power 1.5
set_rail_analysis_mode -method era_static -accuracy xd -extraction_tech_file ./qrcTechFile -
era_current_region_file ./cur_region_file -era_skip_virtual_via_on_layers MI M2
analyze_rail -type net vdd
```

- The following is an example of static early rail analysis with current distribution for unplaced instances by total die area:

```
set_rail_analysis_mode -power_grid_library fast_library/fast_allcells.cl -extraction_tech_file
typical.tch -accuracy hd -ignore_shorts true -method era_static -enable_rlrp_analysis true -
vsrsrc_search_distance 50 -era_current_distribution unplaced -era_current_distribution_layer Metall -
era_current_distribution_unplaced_area diearea -era_techlib_generation false -
era_check_wires_for_generated_current_regions true
```

- Let us consider a design with cell type A that has the M1 and M2 PG pin shapes, and cell type B that has the M3 and M4 PG pin shapes.

The following command calculates the reff values of the interface nodes on the M2 pin shapes for the cell type A and on the M4 pin shapes for the cell type B, and reports the worst (biggest) value among them. Similarly, it calculates the rlrp values of the interface nodes on the M1 pin shapes for the cell type A and on the M3 pin shapes for the cell type B, and reports the best (smallest) value among them.

```
set_rail_analysis_mode \
    -reff_pin_report_layer top \
    -reff_pin_report_method worst \
    -enable_rlrp_analysis true \
    -rlrp_pin_report_layer bottom \
    -rlrp_pin_report_method best \
```

...

analyze_resistance

- The following command reads the switch-bit information in the specified memory cells:

```
set_rail_analysis_mode \
  -power_grid_library "${Libgen_Golden_Pgv}" \
  -accuracy hd \
  -ignore_shorts true \
  -method dynamic \
  -enable_rlrp_analysis true \
  -eiv_eval_window switching \
  -limit_number_of_steps false \
  -watch_location_waveform u0 \
  -switchbit_file_for_eiv_calculation sw.list
```

The following is a snippet of the sw.list file:

```
CELL MEM128Kx32
READ 0.1ns 0.13ns
WRITE 0.1ns 0.15ns
CELL MEM64Kx32
READWRITE 0.1ns 0.2ns 0.3ns 0.4ns
```

The following is a snippet of the report:

```
INST_NAME WIN_EIV PWR_WIN_IV GND_WIN_IV WIN_TIME ELAPSE_EIV PWR_ELAPSE_IV GND_ELAPSE_IV ELAPSE_TIME
CELL_NAME
*"NA" means the instance is disconnected from the net or the instance does not have timing/switching
window
BEGIN
- external/MULT8_reg_3_6 0.75841 0.85220 0.09379 48.470 0.75841 0.85220 0.09379 48.470 SDFQXL
```

- The following command specifies the instance-based PWL current file in the dynamic ERA flow:

```
set_rail_analysis_mode -method era_dynamic -accuracy hd -vsrc_search_distance 500 -
era_instance_dynamic_current_file ./inst_cell_pwl_file -power_grid_library ../../techonly.cl
```

The following is a snippet of the PWL file:

```
instance ram/ram3 net VDD pwl (0ns 0mA 0.1ns 1mA 0.2nA 0.5mA) intrinsic_cap 10pf loading_cap 60pf
instance ram/ram2 net VDD pwl (0ns 0mA 0.1ns 1mA 0.2nA 0.5mA) intrinsic_cap 10pf loading_cap 60pf
...
```

- The following command specifies different temperature values to the layers M1, M2, M3, VIA1, VIA2, and VIA3 for average current limit analysis:

```
set_rail_analysis_mode
...
  -em_temperature 110 \ #temperature for the remaining layers
  -em_temperature_layer_list {M1 125 M2 115 M3 105 VIA1 115 VIA2 105 VIA3 120} #layer-based
temperature
```

The log file will display the following info message:

```
Layer specific EM temperature will be used for AVG check for layer M1(125C), M2(115C), M3(105C),
VIA1(115C), VIA2(105C), VIA3(120C).
```

The header of the power EM report displays the layer-based temperature, as shown below:

```
# Temperature: 110.0 (C)
```

```
# Jmax_factor:
# .....
#   M2   0.981000 @ 115.0 (C)
#   .....
```

- The following command specifies to perform unified power switch flow:

```
set_rail_analysis_mode -method dynamic -accuracy hd \
.....
-powering_up_rails "VDD_sw1 VDD_sw2" \
-enable_xp true \
-unified_power_switch_flow true
```

Related Topics

- [File Formats](#)
- [Glossary](#)
- [Static Power, IRdrop and EM Analysis](#)
- [Dynamic Power and IRDrop Analysis](#)

set_voltus_insight_mode

```
set_voltus_insight_mode
[-help]
[-all_scenario_ranking {wiv | tiv}]
-config filename
[-db_mapping filename]
-domain name
[-disable_layer filename]
[-groundREFF_threshold value]
[-groundRLRP_threshold value]
[-hier_block_instance names]
[-hierarchical {true | false}]
[-initialize]
[-input_insight_db database_path]
[-instance_mapping filename]
[-ir_effort {high|medium|low}]
[-master_list {list_of_master_blocks}]
[-master_mapping filename]
[-nParallel value]
[-output_insight_db database_path]
[-path_group {all | reg2reg}]
[-powerREFF_threshold value]
[-powerRLRP_threshold value]
[-scenario_default list]
[-scenario_ranking {wiv | tiv | powerrlrp | powerreff | groundrlrp | groundreff}]
[-skip]
[-timing_effort {high|medium|low}]
[-timing_protection_config filename]
[-violation_input filename]
[-watch_box filename]
[-watch_inst filename]
```

Specifies the user-defined settings related to the Voltus InsightAI flow. The command allows you to configure flows for various scenarios, such as differing EIV method, EIV evaluation window, and thresholds for each instance type. Using this command, you can tailor your configurations precisely, ensuring optimal performance for every scenario.

Parameters

-all_scenario_ranking {wiv tiv}	Specifies that all scenarios are ranked either based on the total number of violations (TIV) or worst violations (WIV).
-help	Outputs a brief description that includes the type and default information for each <code>set_voltus_insight_mode</code> parameter. For a detailed description of the command and all of its parameters, use the man command <code>man set_voltus_insight_mode</code> .

<code>-config filename</code>	
	(Mandatory) Specifies the name of the configuration file that includes details about the PG network definition, EIV method, EIV evaluation window, list of scenarios, and their setting.
<code>-db_mapping filename</code>	
	Specifies the name of the file that contains the mapping between a scenario and its corresponding rail directories. Each scenario can be mapped to multiple rail directories. Following is a snippet of the rail database mapping file (db_mapping.txt): <pre>SCENARIO_1 {RailDB1 RailDB2} SCENARIO_2 {RailDB1}</pre>
<code>-domain name</code>	
	(Mandatory) Specifies the name of the power domain.
<code>-disable_layer filename</code>	
	Specifies the name of the file that contains the list of layers to be disabled or excluded during pre-route and post-route RPG. The format of the file is: <pre>Layer1 Layer2 Layer3</pre>
<code>-groundREFF_threshold value</code>	
	Specifies that the violation report contains only instances that have values less than the specified ground REFF threshold.
<code>-groundRLRP_threshold value</code>	
	Specifies that the violation report contains only instances that have values less than the specified ground RLRP threshold.
<code>-hier_block_instance names</code>	
	Specifies the list of instances of the master block that are to be repaired or reported. This parameter is specified with the <code>-hierarchical</code> parameter. When the instances are specified using this parameter, only the specified instances are repaired or reported.
<code>-hierarchical {true false}</code>	

	Specifies to run the hierarchical repair flows (reinforce_pg, IR-aware placement, and ECO). When this parameter is set to <code>true</code>, you must specify at least one state directory in the configuration file. Default: <code>false</code>
<code>-initialize</code>	Saves the current power settings before running the Insight repair commands using internal power/rail values. After InsightAI-based analysis, this parameter enables restoration of the original power settings.
<code>-input_insight_db database_path</code>	
	Specifies the path of the input Voltus InsightAI database. After generating the output database using the <code>-output_insight_db</code> parameter (in the <output db>/master directory), the input database specified with <code>-input_insight_db</code> will be one of the master from the output database generation. You can then run <code>reinforce_pg</code> without loading the entire rail database, enabling you to fix issues without reloading the rail result. Example: <u>1st Step: Output Database Generation</u> <pre>set_voltus_insight_mode -config <insight_config> -db_mapping <db_mapping_file> -output_insight_db <output_db></pre> The database is generated under the directory <output_db> <u>2nd Step: Insight Database Fixing</u> <pre>set_voltus_insight_mode -config -input_insight_db <input_db> reinforce_pg -auto_ir_fix postroute</pre>
<code>-instance_mapping filename</code>	
	Specifies the name of the file that contains the mapping between a scenario and its corresponding instance files. Each scenario has its own set of instance files. Each line in an instance file defines either an instance name or a block instance name. Following is a snippet of the instance mapping file (<code>instance_mapping.txt</code>): <pre>SCENARIO_1 {instance1.txt instance2.txt} SCENARIO_2 {instance1.txt}</pre> The content of the 2 instance files is shown below: <pre>instance1.txt ----- Inst_A* Inst_B* instance2.txt ----- Inst*</pre>
<code>-ir_effort {high medium low}</code>	

	Specifies the effort level for running IR repair. The effort level for IR repair can be set to <code>high</code>, <code>medium</code>, or <code>low</code>. By default, the level is set to <code>high</code>. If there is an impact on Quality of Results (QOR), you may choose to lower the effort level. Default: <code>high</code>  To achieve optimal results, a balance between IR and timing efforts is crucial. The default setting is high IR and low timing effort, which generally provides the best IR fix rate. However, if there is a degradation in timing, you should increase the timing effort to safeguard it. If the timing continues to degrade even with a high timing effort, then you should reduce the IR effort.
<code>-master_list {list_of_master_blocks}</code>	
	Specifies the list of master blocks to be reported for IR repair.
<code>-master_mapping filename</code>	
	Specifies the name of the file that contains the mapping between a scenario and its corresponding master files. Each scenario has its own set of master files. Each line in a master file defines one master block. Following is a snippet of the master mapping file (<code>master_mapping.txt</code>): <pre>SCENARIO_1 {master1.txt master2.txt} SCENARIO_2 {master1.txt}</pre> The content of the 2 master files is shown below: <pre>master1.txt ----- Top master2.txt ----- Super_filter</pre>
<code>-nParallel value</code>	
	Specifies the number of scenarios or databases that are to be analyzed.
<code>-output_insight_db database_path</code>	

	Specifies the path of the output Voltus InsightAI database. After generating the output database using this parameter (in the <output db>/master directory), the input database specified with <code>-input_insight_db</code> will be one of the master from the output database generation. You can then run <code>reinforce_pg</code> without loading the entire rail database, enabling you to fix issues without reloading the rail result. Example: <u>1st Step: Output Database Generation</u> <pre>set_voltus_insight_mode -config <insight_config> -db_mapping <db_mapping_file> -output_insight_db <output_db></pre> The database is generated under the directory <output_db> <u>2nd Step: Insight Database Fixing</u> <pre>set_voltus_insight_mode -config -input_insight_db <input_db> reinforce_pg -auto_ir_fix postroute</pre>
<code>-path_group {all reg2reg}</code>	
	Specifies which timing paths to protect during <code>reinforce_pg</code>. • ALL: Protects all timing paths • reg2reg: Protects only register-to-register paths
<code>-powerREFF_threshold value</code>	
	Specifies that the violation report contains only instances that have values less than the specified power REFF threshold.
<code>-powerRLRP_threshold value</code>	
	Specifies that the violation report contains only instances that have values less than the specified ground RLRP threshold.
<code>-scenario_default list</code>	
	Specifies the list of default scenarios or databases.
<code>-scenario_ranking {wiv tiv powerrlrp powerreff groundrlrp groundreff}</code>	
	Specifies that the scenario ranking is done for a single scenario. When specified, the ranking is done based on the given parameter.
<code>-skip</code>	Skips user-defined instances specified with <code>-watch_box</code> and <code>-watch_instances</code>.
<code>-timing_effort {high medium low}</code>	

	Specifies the effort level the software uses for timing optimization. The effort level for timing can be set to <code>high</code>, <code>medium</code>, or <code>low</code>. By default, the level is set to <code>low</code> to ensure best IR fix. If timing degrades, you may choose to increase the effort level. ⚠ To achieve optimal results, a balance between IR and timing efforts is crucial. The default setting is high IR and low timing effort, which generally provides the best IR fix rate. However, if there is a degradation in timing, you should increase the timing effort to safeguard it. If the timing continues to degrade even with a high timing effort, then you should reduce the IR effort.
<code>-timing_protection_config filename</code>	
	Defines timing degradation thresholds per timing view using a configuration file. A template of the configuration file for a design can be generated with <code>get_voltus_insight_mode -timing_protection_config</code> after loading the design database in Innovus. The following is a snippet from the configuration file: <pre>Global 0.0001 View1 0.0002</pre> This sets a global threshold of 0.0001 ps, with a specific threshold of 0.0002 ps for View1.
<code>-violation_input filename</code>	
	Enables repair operations based on violation specifications that align with an external signoff tool. This feature allows Innovus to replicate violation magnitudes for individual instances, ensuring consistent repair behavior between the external signoff tool and Innovus while eliminating the need for global threshold adjustments. You can specify a text file containing a per-instance violation specification in the following format: <pre>instance_name threshold(mV) violating_amount (mV) Example: inst_1 100 35 inst_2 80 20</pre> For instance 1 (<code>inst_1</code>), an external signoff tool detected a 35mV violation against a 100mV threshold. This violation data allows Innovus to perform repairs using the same violation magnitude values reported by the external tool, ensuring consistency between the tools.
<code>-watch_box filename</code>	
	Specifies a file containing the list of regions, wherein only the instances inside these regions will be reported. You can write multiple regions in the file. The format of the file is: <pre>{{x1,y1}{x2,y2}} {{x3,y3}{x4,y4}}</pre>
<code>-watch_inst filename</code>	

	Specifies a file containing the list of instances to be reported for IR repair. You can write multiple instances in the file. The format of the file is: <pre>instance_1 Instance_2</pre>
--	---

Examples

- The following command runs 2 scenarios with different eiv method, eiv_eval_window, and thresholds for each instance type (clock and data):

```
set_voltus_insight_mode -config insight.config -domain ALL -db_mapping db_mapping.txt
```

Following is a snippet from the insight.config file:

```
#### PG Network Definition ####Nets VDD VSSDomain_1 ALL VDD VSS#### Global Options
####eiv_eval_window {}eiv_method {}#### Scenario Definition ####SCENARIO_DEFAULT: SCENARIO_1
SCENARIO_2#### Scenarion Setting ####SCENARIO_1 eiv_eval_window elapse eiv_method Worst Domain
Domain_1 threshold_data 0.4 threshold_clock 0.4 Net VSS threshold_data 0.09
threshold_clock 0.09 Net VDD threshold_data 0.81 threshold_clock 0.81SCENARIO_2
eiv_eval_window switching eiv_method WorstAvg Domain Domain_1 threshold_data 0.4
threshold_clock 0.4 Net VSS threshold_data 0.09 threshold_clock 0.09 Net VDD
threshold_data 0.81 threshold_clock 0.81
```

Related Topics

- [*"Voltus InsightAI" in Innovus User Guide*](#)
- `get_voltus_insight_mode`

set_voltage_regulator_module

```
set_voltage_regulator_module
-help
-component_list {VRM_DEF_component_list}
-ground_pin_mapping {{spice_pin_name[DEF_net_name]}+}
-input_pwr_pin_mapping {{spice_pin_name[DEF_net_name]}+}
-name vrm_name
-netlist vrm_spice_netlist_file_name
-output_pwr_pin_mapping {{spice_pin_name[DEF_net_name]}+}
-reset
-subckt_name vrm_subckt_name
```

Analyzes voltage regulator effects. Use this command to enable the Vreg flow.

You must specify this command before running the `analyze_rail` command.

Parameters

-help	Prints out the command usage.
-component_list {VRM_DEF_component_list}	
	Specifies the names of the DEF instances which comprise the VRM. The component name must be the instance name used for the specified VRM. If this parameter is not specified, it is assumed that VRM is represented as a single MACRO on the die.
-ground_pin_mapping {{spice_pin_name[DEF_net_name]}+}	
	Specifies the mapping between the ground pin name in the SPICE netlist and the corresponding DEF net name for the same power pin.
-input_pwr_pin_mapping {{spice_pin_name[DEF_net_name]}+}	
	Specifies the mapping between the input power pin name in the SPICE netlist and the corresponding DEF net name for the same power pin.
-name vrm_name	
	Specifies the VRM name assigned to a voltage regulator. You need to specify this parameter for each voltage regulator in the design.
-netlist vrm_spice_netlist_file_name	
	Specifies the VRM netlist file in the SPICE format that is encapsulated in a single sub-circuit. This should also include SPICE models and testbench for each voltage regulator.
-output_pwr_pin_mapping {{spice_pin_name[DEF_net_name]}+}	

	Specifies the mapping between the output power pin name in the SPICE netlist and the corresponding DEF net name for the same power pin.
<code>-reset</code>	
	Resets all specified parameters back to default values.
<code>-subckt_name vrm_subckt_name</code>	
	Specifies the subcircuit that corresponds to the VRM. This parameter is specified when the specified SPICE netlist has multiple subcircuit definitions. If this optional parameter is not specified, the first subcircuit found in the netlist file will be used.

Examples

- The following command performs Vreg analysis for the VRM `vrm1`:

```
set_voltage_regulator_module -name vrm1 -netlist ./TEST.spi -input_pwr_pin_mapping {{VCC VCC}} -
output_pwr_pin_mapping {{VDD VDD}} -ground_pin_mapping {{ VSS VSS }} -subckt_name LDO_MODEL -
component_list { vmesh_reg_1 }
```

Related Topics

- "[Static Power, IRdrop and EM Analysis](#)" and "[Dynamic Power and IRDrop Analysis](#)" in the *Voltus User Guide*

view_dynamic_movie

```
view_dynamic_movie
[-movies_directory dir]
[-type [ir|tc] ]
```

Provides the ability to view movies created during dynamic analysis. Movies are sequences of analysis plots displayed over a specific simulation time.

Parameters

<code>-movies_directory dir</code>		
	Specifies the movie directory that was generated during dynamic rail analysis (that is, <code>state_directory/movie_analysis</code>)	
<code>-type [ir tc]</code>		
	Specifies the movie type.	
	<code>ir</code>	IRdrop
	<code>tc</code>	Tap current

Examples

- The below command views the dynamic movie of type `ir` in the specified state directory:

```
view_dynamic_movie -type ir -movies_directory \
dynamic_rail/TDSP_25C_dynamic_1/VDD_TDSPCore_R/movie_analysis
```

Related Topics

- "[Dynamic Power and IRDrop Analysis](#)" in the *Voltus User Guide*

view_dynamic_waveform

```
view_dynamic_waveform
-type {rail_current voltage power_current all current profile}
[-clear_hier]
[-composite_waveform_name [hierarchy_name | clock_name]]
[-composite_waveform_type [hierarchy | clock | clock_with_seq
    | total_current]]
[-effective_voltage_waveform]
[-free_data]
[-hier_block_file filename]
[-instance_name instance_name]
[-power_db [powermeter.db]]
[-waveform_files {file1 file2 ... fileN}]
```

Provides the ability to view dynamic current or voltage waveforms generated during analysis.

Parameters

-clear_hier	Specifies to clear the hierarchical block data that was loaded using the -hier_block_file parameter.
-composite_waveform_name [hierarchy_name clock_name]	
	Only available for -type current. If hierarchy_name is specified, plots total current waveform for the specified hierarchy. If clock_name is specified, plots current waveform for the selected or all clocks. If -composite_waveform_type clock_with_seq is specified, plots total current waveform for all sequential cells. If -composite_waveform_type total_current option is specified, plots total current waveform for all instances in the current file.
-composite_waveform_type [hierarchy clock clock_with_seq total_current]	
	You can specify to view a waveform for a specific hierarchy, a clock, clock with sequential cells, or total current.
-effective_voltage_waveform	
	Plots the effective voltage waveform (i.e. VDD - VSS). You must specify the voltage for two nets to generate the effective waveform.

<code>-free_data</code>	Frees the memory but leaves the menu entries.
<code>-hier_block_file filename</code>	
	Specifies to analyze the multiple (top/block-level) current (.ptiavg) files of a hierarchical design in the SimVision waveform viewer interface. This parameter can be used to easily find an instance of a specific block when there are multiple blocks with the same instance name. <code>-hier_block_file</code> specifies a text file that contains the list of blocks. For example, if the block name <code>BLOCK_A</code> is specified in the text file, you can match the instance named <code>BLOCK_A/Instance_A</code> and <code>Instance_A</code> in the <code>BLOCK_A</code>'s .ptiavg file. When specifying multiple block names in the text file, one block name should be given per line. The following is a snippet from the <code>block_hier</code> file: <pre>u_top/vcore3 u_top/vcore2 u_top/vcore1</pre>
<code>-instance_name instance_name</code>	
	Plot the waveform for the specified instance.
<code>-power_db [powermeter.db]</code>	
	Specifies the power database. Must be specified if <code>-composite_waveform_type</code> is <code>clock</code> or <code>clock_with_seq</code>.
<code>-type {rail_current voltage power_current all}</code>	
	(Mandatory) Specifies the waveform type to be displayed. This parameter allows you load current waveforms, voltage waveforms, and profiling databases (.trn). The possible options of this parameter are: <code>rail_current</code>: specifies to load the current waveform files generated during rail analysis. The waveform instance names have the prefix <code>TC_</code>. <code>voltage</code>: specifies to load the voltage waveform files generated during rail analysis. The waveform instance names have the suffix <code>:ir</code>. <code>power_current</code>: specifies to load the current waveform files generated during power analysis. The waveform instance names do not have a suffix or prefix. <code>all</code>: specifies to load all waveform types. You can specify one or more waveform types. For viewing a profiling database, you can specify any of the waveform types. <i>Default:</i> <code>all</code>

```
-waveform_files {file1 file2 ... fileN}
```

Specifies one or more current or voltage waveform files generated with a .ptiavg extension, or VCD profiling files with a .trn extension.

Examples

- The following command views dynamic composite total current waveforms from the file `vddm_vddm.ptipeak`:

```
view_dynamic_waveform -type current -composite_waveform_type total_current \
    -waveform_files vddm_vddm.ptipeak
```

- The following command views the dynamic current waveform for instance `inst1`:

```
view_dynamic_waveform \
    -type current \
    -waveform_files dynamic_vss.ptiavg \
    -instance_name inst1
```

- The following command is for viewing the total current waveform:

```
view_dynamic_waveform \
    -type current \
    -waveform_files dynamic_vss.ptiavg \
    -composite_waveform_type total_current
```

- The following command plots a clock domain composite waveform:

```
view_dynamic_waveform \
    -type current \
    -waveform_files dynamic_vss.ptiavg \
    -power_db power.db \
    -composite_waveform_type clock \
    -composite_waveform_name clk
```

- The following command plots a hierarchical composite waveform:

```
view_dynamic_waveform \
    -type current \
    -waveform_files dynamic_vss.ptiavg \
    -composite_waveform_type hierarchy \
    -composite_waveform_name ethernet_mac_2
```

- The following command specifies to load multiple block-level current files:

```
view_dynamic_waveform -hier_block_file block_hier -waveform_files {PTIAVG1 PTIAVG2 PTIAVG3 ...}
```

- The following command specifies to load the rail current and voltage waveform types for the specified instance:

```
view_dynamic_waveform -type { rail_current voltage} -instance_name I1_1 \
    -waveform_files { PD_25C_dynamic_1/Reports/Waveforms/tran_VSS.ptiavg s/pin.trn}
```

Related Topics

- "[Dynamic Power and IRDrop Analysis](#)" in the *Voltus User Guide*

view_package_results

```
view_package_results
[-help]
[-result_file <filename>]
-type <package_analysis_type>
[-workspace <workspacename>]
```

Specifies to view the package IR drop analysis results in the PowerDC GUI which will be invoked from within Voltus/Innovus.

Parameters

-help	Outputs the command usage and a brief description about the command parameters.
-result_file <filename>	
	Specifies the name of the .xml result file.
-type <package_analysis_type>	
	Specifies the type of package analysis. Currently, the software supports only the <code>dc</code> package analysis type.
-workspace <workspace_name>	
	Specifies the workspace name for package analysis (powerDC). The file name extension is <code>.pdcx</code> .

Example

- The following example specifies to view the package analysis results:

```
view_package_results -type dc
```

Advanced Timing Tcl Scripting Commands

- [get_activity](#)
- [get_cells](#)
- [get_lib_cells](#)
- [get_nets](#)
- [get_pins](#)
- [get_ports](#)
- [get_power](#)

get_activity

```
get_activity
[-help]
[-instance inst_name]
[-list_of_nets_based_on_driver {primary_inputs | sequential | combinational | memory |
ICG | black_box}]
[-list_of_nets_based_on_source_of_activity_info
{activity_file|set_switching_activity|propagated|default}]
[-net netname]
[-outfile filename]
[-pin pinname]
[-port portname]
[-report_average_switching_activity]
[-report_cell_group_activity_summary_report]
[-summary]
[-summary]
[-tcl_list]
```

Determines the source of activity for the specified nets/pins/ports. The power analysis engine has various sources from where activities can be annotated from, such as switching activity

specification (input/sequential/global/clock gate), clock definitions/constants through SDC/TWF, activity files (VCD/TCF/SAF), user defined activity and/or propagated values.

Parameters

<code>-help</code>	Outputs a brief description that includes the type and default information for each <code>get_activity</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <code>man get_activity</code>
<code>-instance inst_name</code>	
	Specifies the top/hierarchy instance or the leaf level instance name for which average activity is to be reported.
<code>-list_of_nets_based_on_driver {primary_inputs sequential combinational memory ICG black_box}</code>	
	Specifies to output the list of nets based on the specified net driver. You can specify one or more net drivers at a time, as shown below: <pre>get_activity -list_of_nets_based_on_driver primary_inputs combinational sequential ICG memory black_box</pre> To specify multiple drivers, you can use <code>/</code> (means 'or') or <code>&</code> (means 'and') between two drivers, without a space.
<code>-list_of_nets_based_on_source_of_activity_info {activity_file set_switching_activity propagated default}</code>	
	Specifies to output the list of nets based on the specified activity source information.
<code>-net netname</code>	
	Specifies the name of the net. Note: You can use the <code>get_nets</code> command to determine the activity source for a collection of nets.
<code>-outfile netname</code>	

	Specifies the output file into which the activity sources are to be written. The format of the report is: <pre><net_name/pin_name> <activity_source> <duty_cycle> <transition_density></pre> For user-specified activity, the transition density is calculated as = switching activity * fastest clock associated with the pin/net. Following is a snippet of the output: <pre>n_14 user_defined_activity 0.3 3.125e+07</pre> Here, 0.3 is the duty cycle, and 3.125e+07 is the transition density. The transition density of 3.125e+07 for the net is calculated by: <pre>.25 (user-specified switching activity) * 1/8e-9 (8ns clock)</pre> For more information, refer to the "Static Power Calculation Overview" section of the "Static Power, IRdrop and EM Analysis" chapter in <i>Voltus User Guide</i>.
<code>-pin pinname</code>	
	Specifies the name of the pin. Note: You can use the <code>get_pins</code> command to determine the activity source for a collection of pins.
<code>-port portname</code>	
	Specifies the name of the port. Note: You can use the <code>get_ports</code> command to determine the activity source for a collection of ports.
<code>-report_average_switching_activity</code>	
	Specifies to report the average activity of the given instance/hierarchy/top-level.
<code>-report_cell_group_activity_summary_report</code>	
	Specifies to give a summary of cell group activity.

-summary	Generates a detailed activity annotation report of the specified nets, pins, and ports. This report contains the following information: • Primary Inputs • Sequential Outputs • Memory/Macro Outputs • Tristate Outputs Note: Use the <code>get_activity -summary</code> command to report activity annotation summary without printing information for nets/pins/ports.
- tcl_list	Produces the report in the Tcl list format instead of a tabular format. The <code>-tcl_list</code> and <code>-outfile</code> parameters are mutually exclusive; you cannot specify them together.

Examples

- The following example returns the activity sources of all nets that match the specified pattern and writes the output information to a file named `nets.rpt`:

```
get_activity -net [get_nets n*] -outfile nets.rpt
```

The following is the example output of the `nets.rpt` file:

```
int3 activity_file 0.5 5e+07
int2 sequential_activity 0.5 2.5e+07
int1 user_defined_activity 0.5 1e+07
nt3 global_activity 0.5 5e+07
nt3 input_activity 0.5 5e+07
int5 clock_gate_activity 0.5 2.5e+07
int1 user_defined_activity 0.5 1e+07out
sequential_activity 0.5 2.5e+07
clk sdc/twf 0.5 5e+08
c_int2 propagation 0.5 5e+08
```

- The following example returns the activity source of the pin `i_4/A1` and writes the output information to a power report file named `pin.rpt`:

```
get_activity -pin i_4/A1 -outfile pin.rpt
```

- The following command report the average activity of all nets:

```
get_activity -report_average_switching_activity
```

The following is the example output:

```
Avg Switching activity of all nets 4.727e+17
```

- The following command gives a summary of cell group activity:

```
get_activity -report_cell_group_activity_summary_report
```

The following is the example output:

Group Activity Summary Report:

Source ->	activity file	set_switching_activity
propagated	default	Total
All nets	2	1
2	5	10
Break-up based on net driver:		
Primary Input	2 (40%)	0 (0%)
0 (0%)	3 (60%)	5
Sequential	0 (0%)	1 (33.33%)
0 (0%)	2 (66.67%)	3
Combinational	0 (0%)	0 (0%)
2 (100%)	0 (0%)	2
Memory	0 (0%)	0 (0%)
0 (0%)	0 (0%)	0
ICG	0 (0%)	0 (0%)
0 (0%)	0 (0%)	0
Black Box	0 (0%)	0 (0%)
0 (0%)	0 (0%)	0

- The following command gives the list of nets based on primary inputs:

```
get_activity -list_of_nets_based_on_driver primary_inputs
```

The following is the example output:

```
D2 D1 CP2 CP1 dummy0
```

- The following command gives the list of nets based on primary inputs or sequential driver:

```
get_activity -list_of_nets_based_on_driver primary_inputs|sequential
```

The following is the example output:

```
Q2 Q1 D2 D1 CP2 CP1 n1 dummy0
```

- The following command gives the list of nets based on primary inputs or sequential driver for the instance `ff1b`:

```
get_activity -list_of_nets_based_on_driver primary_inputs|sequential -instance ff1b
```

- The following command gives the list of nets based on the propagated source for the instance `i2`:

```
get_activity -list_of_nets_based_on_source_of_activity_info propagated -instance i2
```

- The following command gives a summary of cell group activity for the instance i2:

```
get_activity -instance i2 -report_cell_group_activity_summary_report
```

- The following command gives the list of nets based on the default activity:

```
get_activity -list_of_nets_based_on_source_of_activity default
```

The following is the example output:

```
rst shutdown isolate coef_ld start bypass test_mode se sdi Xn_in[3] Xn_in[2]
Xn_in[1] Xn_in[0]
Yn_in[15] Yn_in[14] Yn_in[13] Yn_in[12] Yn_in[11] Yn_in[10] Yn_in[9] Yn_in[8]
Yn_in[7] Yn_in[6]
```

- The following command gives the list of nets based on the activity file:

```
get_activity -list_of_nets_based_on_source_of_activity activity_file
```

The following is the example output:

```
ref_clk test_clk pll_clk
```

- The following command gives the list of nets based on the switching activity:

```
get_activity -list_of_nets_based_on_source_of_activity set_switching_activity
```

The following is the example output:

```
rst ref_clk shutdown isolate coef_ld start bypass test_clk test_mode se sdi sdo
Xn_in[3] Xn_in[2]
Xn_in[1] Xn_in[0] Yn_in[15] Yn_in[14] Yn_in[13]
```

get_cells

```
get_cells
[-help]
[-filter expr]
[-hierarchical]
[-hsc char]
[-leaf]
[-nocase]
[-quiet]
{patterns | -of_objects object_list}
[-regexp]
```

Creates a collection of instances in the current design whose name matches the supplied pattern list. Assign this collection to a variable or pass it as an argument in another command.

Parameters

-help	Outputs the command usage and a brief description about the command parameters.	
-filter <i>expr</i>	Filters the collection with the specified expression. For any cells that match the pattern, the software evaluates the expression based on the attribute of the cell.	
- hierarchical	Specifies that the patterns should be matched hierarchically. If the flat name of an instance at a particular hierarchical level matches the patterns, it is included in the result. For example, if there is top level hierarchical instance <code>Top</code> and it contains another sub-instance called <code>FF1</code> , then the following command can be used to get the instance <code>Top/FF1</code> (since the given pattern will be matched with instance flat name <code>FF1</code>): <code>get_cells -hier FF*</code> However, the following command will display an error: <code>get_cells -hier Top/FF*</code>	

<code>-hsc char</code>	Specifies the hierarchical delimiter for patterns. For example, if the hierarchical delimiter is @, the <code>sub/I1@I2</code> pattern matches instance <code>I2</code> within the hierarchy limited at <code>sub/I1</code> .
<code>-leaf</code>	Returns leaf cells only. When the <code>-leaf</code> parameter is not specified, the <code>get_cells</code> command returns the hierarchical instances present on a net along with the flat instances at the same hierarchical level as the specified object. Note: The <code>-leaf</code> parameter can only be specified with the <code>-of_objects</code> parameter.
<code>-nocase</code>	Specifies that pattern matching is not case sensitive. Note: You must specify <code>-regexp</code> in order to use this parameter.
<code>-of_objects object_list</code>	
	Creates a collection of all cells associated with the specified pins or nets. You can specify a list of pins or a list of nets, but you cannot specify a list that contains both pins and nets. You can specify patterns for the pin or net names, a collection of pins or nets, or a combination of patterns and a collection.
<code>patterns</code>	Specifies patterns for instance names. Patterns include the <code>*</code> and <code>?</code> wildcard characters and collections of instances. The <code>get_cells</code> command returns a collection of instances that match the patterns. If no such instance is found, an empty collection is returned.
<code>-quiet</code>	Suppresses all error and warning messages generated when the <code>get_cells</code> command is run.

<code>-regexp</code>	Treats the specified patterns as a regular expression patterns. When this parameter is specified, the given patterns <code>start</code> and <code>end</code> are assumed to be matching with the start and end part of the complete object name. For example, the following can be used to retrieve the hierarchical instance <code>Top/block1/block2/FF1</code>: <pre>get_cells Top/**/*.FF1 get_cells -regexp Top/.*/FF1</pre> However, the following command will display an error : <pre>get_cells Top/*/FF1</pre> If this parameter is specified with <code>-hier</code> parameter, the given pattern will be matched with the complete hierarchical name of objects instead of only the flat name and all the successful matches are included in the result. For example, if there is a top level hierarchical instance <code>Top</code> and it contains another sub-instance called <code>FF1</code>, then either of these can be used to retrieve <code>Top/FF1</code> instance: <pre>get_cells -hier FF1 get_cells -hier -regexp Top/FF1</pre> However, the following commands will display an error : <pre>get_cells -hier Top/FF1 get_cells -hier -regexp FF1</pre>
----------------------	---

Example

- The following command searches for pins within the hierarchy limited at `B`:

```
get_cells -hsc @ B@*
```
- The following command searches for cells with set pattern and filter:

```
get_cells "o*" -filter "@ref_lib_cell_name != FD2 && @is_black_box==false"
```

Related Information

[get_nets](#)

`get_pins`

`get_ports`

get_lib_cells

```
get_lib_cells
[-help]
[-filter expr ]
[-nocase]
[-quiet]
<pattern_list>
[-of_objects object_list]
[-regexp]
```

Creates a collection of library cells from the loaded libraries whose name matches the supplied pattern list. Assign these library cells to a variable or pass them to another command. If no objects match the criteria, an empty collection is returned.

Parameters

-help	Prints out the command usage.
-filter <i>expr</i>	Filters the collection with the specified expression. For any library cells that match the pattern, the software evaluates the expression based on the attribute of the library cell.
-nocase	Specifies that pattern matching is not case sensitive. Note: You must specify <code>-regexp</code> in order to use this parameter.
-of_objects <i>object_list</i>	
	Creates a collection of all library cells associated with the specified library pins or cells. You can specify a list of pins or a list of cells, but you cannot specify a list that contains both pins and cells. You can specify patterns for the pin or cell names, a collection of pins or cells, or a combination of patterns and a collection.

<code>pattern_list</code>	Returns only those libraries in the collection whose name matches one or more of the patterns specified in the <code>pattern_list</code> argument. The characters <code>?</code> and <code>*</code> are wild cards. The <code>?</code> character matches any one character while the <code>*</code> character matches zero or more characters. If a library matches multiple patterns, it appears multiple times in the collection. A pattern can have only one hierarchical separator character. The left part of the pattern is the library name pattern and the right part is the cell name pattern. If there is no separator character, the pattern is compared to cell names in all the libraries. A pattern can have only one hierarchical separator character, dividing the pattern into one or two parts. If the hierarchical separator is <code>/</code>, the alternatives are: • <code>cell_name_pattern</code> Selects all cells whose name matches the <code>cell_name_pattern</code> from all libraries. • <code>lib_name_pattern / cell_name_pattern</code> Selects all cells whose name matches the <code>cell_name_pattern</code> from all libraries whose name matches the <code>lib_name_pattern</code>.
<code>-quiet</code>	Suppresses all error and warning messages generated when the <code>get_lib_cells</code> command is run.
<code>-regex</code>	Treats the specified patterns as a regular expression patterns. <i>Default:</i> Treats the specified pattern as a wild card

Examples

- The following command prints all cells for all libraries:
`get_lib_cells *`
- The following command prints a collection with all cells in the `lca500kv` lib library:
`get_lib_cells lca500kv/*`
- The following command prints a collection with all cells whose names starts with `AN` in all libraries starting with `lca`:

```
get_lib_cells lca*/AN*
```

- The following command disables delay arcs on the EF11L050032A library cell. The `set_disable_cell_timing` command effects all instantiations of the specified cell, and lets you disable delay arcs from and to internal pins.

```
set_disable_timing -from WACLK -to RBO1 [get_lib_cells {EF11L050032A}]
```

Related Information

[get_cells](#)

get_nets

```
get_nets
[-filter expr ]
[-hierarchical]
[-hsc char ]
[-nocase]
[-quiet]
<patterns>
[-of_objects object_list]
[-regexp]
```

Creates a collection of nets in the current design whose name matches the supplied pattern list. Assign this collection to a variable or pass it as an argument to an another command. If no such net is found, an empty collection is returned.

When you use the `get_nets` command to specify hierarchical nets, the software matches the specified pattern and returns the top level net connected to it.

Parameters

<code>-filter <i>expr</i></code>	Filters the collection with the specified expression. For any nets that match the pattern, the software evaluates the expression based on the attribute of the net.
<code>- hierarchical</code>	Specifies that the patterns should match hierarchically. If the hierarchical name of a net at any hierarchical level matches the patterns, it is included in the result.
<code>-hsc <i>char</i></code>	Specifies the hierarchical delimiter for patterns.
<code>-nocase</code>	Specifies that pattern matching is not case sensitive. Note: You must specify <code>-regexp</code> in order to use this parameter.
<code>-of_objects <i>object_list</i></code>	

	Creates a collection of all nets associated with the specified cells, pins, or ports. You can specify a list of cells, a list of pins, or a list of ports, but you cannot specify a list that contains cells, pins, and ports together. You can specify patterns for the cell, pin, or port names, a collection of cells, pins, or ports, or a combination of patterns and a collection.
<i>patterns</i>	Specifies patterns for net names. Patterns include the * and ? wildcard characters and collections of nets.
-quiet	Suppresses all error and warning messages generated when the <code>get_nets</code> command is run.
-regexp	Treats the specified patterns as a regular expression patterns. <i>Default:</i> Treats the specified pattern as a wild card

Examples

- The following command returns all nets within the hierarchy limited at A:

```
get_nets -hsc @ A@*
```

- The following command returns all patterns that match hierarchy and identifies nets to be excluded from optimization.

```
get_nets * -hierarchical -filter "@dont_touch== true"
```

Related Information

[get_pins](#)

get_pins

```
get_pins
[-help]
[-filter expr]
[-hierarchical]
[-hsc char]
[-leaf]
[-nocase]
[-quiet]
<patterns>
[-of_objects object_list]
[-regexp]
```

Creates a collection of instance pins whose name matches the supplied pattern list. Assign this collection to a variable or pass it as an argument to an another command. If no such pin is found, an empty collection is returned.

Parameters

-help	Prints out the command usage.
-filter <i>expr</i>	Filters the collection with the specified expression. For any pins that match the pattern, the software evaluates the expression based on the attribute of the pin. If the expression is set to true, the pin is included in the result.
-hierarchical	Specifies that patterns should match hierarchically. If the hierarchical name of a pin at any hierarchical level matches the patterns, it is included in the result.
-hsc <i>char</i>	Specifies the hierarchical delimiter for patterns.

<code>-leaf</code>	Returns leaf pins only. When the <code>-leaf</code> parameter is not used, the <code>get_pins</code> command returns the hierarchical pins present on a net along with the flat pins at the same hierarchical level as the specified object. Note: The <code>-leaf</code> parameter can only be specified with the <code>-of_objects</code> parameter. To enable this feature, you need to set the <code>timing_get_pins_of_nets_compatibility</code> timing global variable to <code>true</code>.
<code>-nocase</code>	Specifies that pattern matching is not case sensitive. Note: You must specify <code>-regexp</code> in order to use this parameter.
<code>-of_objects object_list</code>	
	Creates a collection of all pins associated with the specified cells or nets. You can specify a list of cells or a list of nets, but you cannot specify a list that contains both cells and nets. You can specify patterns for the cell or net names, a collection of cells or nets, or a combination of patterns and a collection.
<code>patterns</code>	Specifies patterns for pin names. Patterns include the <code>*</code> and <code>?</code> wildcard characters and collections of pins.
<code>-quiet</code>	Suppresses all error and warning messages generated when the <code>get_pins</code> command is run.
<code>-regexp</code>	Treats the specified patterns as a regular expression patterns. <i>Default:</i> Treats the specified pattern as a wild card

Examples

- The following command returns all instance pins within the hierarchy limited at A:

```
get_pins -hsc @ A@*
```

- The following command filters the collection for `pin_direction` pin attribute.

```
get_pins "in*" -filter "@pin_direction == in"
```

Related Information

[get_ports](#)

get_ports

```
get_ports
[-help]
[-filter expr]
[-nocase]
[-quiet]
{patterns | -of_objects object_list}
[-regex]
```

Creates a collection of ports whose name matches the supplied pattern list. Assign this collection to a variable or pass it as an argument to another command.

Note: The `get_ports` command always creates a collection of top level ports irrespective of the current instance set using the `current_instance` command.

Parameters

<code>-filter <i>expr</i></code>	
	Filters the collection with the specified expression. For any ports that match the pattern, the software evaluates the expression based on the attribute of the port. If the expression is set to true, the port is included in the result.
<code>-nocase</code>	Specifies that pattern matching is not case sensitive. Note: You must specify <code>-regex</code> in order to use this parameter.
<code>-of_objects <i>object_list</i></code>	
	Creates a collection of all ports associated with the specified nets. You can specify patterns for the net names, a collection of nets, or a combination of patterns and a collection.
<code><i>patterns</i></code>	Specifies patterns for port names. Patterns include the <code>*</code> and <code>?</code> wildcard characters and collections of ports. The command returns a collection of ports that match the patterns. If no such port is found, an empty collection is returned.

<code>-quiet</code>	Suppresses all error and warning messages generated when the <code>get_ports</code> command is run.
<code>-regex</code>	Treats the specified patterns as a regular expression patterns. <i>Default:</i> Treats the specified pattern as a wild card

Example

- The following example returns all ports beginning with the name `inb`:

```
get_ports inb*
```
- The following command filters the collection for `port_direction` port attribute.

```
get_ports "mode*" -filter {port_direction == in}
```

Related Information

[get_pins](#)

get_power

```
get_power
[-help]
[-attribute attributes_list]
[-include_unit]
[-nets nets_list | -pins pins_list | -instances instances_list]
[-outfile filename]
[-pg_net {pg_net_name_list | all}]
[-tcl_list ]
```

Queries various power related properties of design objects like nets, pins, and instances. Using this command, you can retrieve power attributes, such as internal power, switching power, leakage power, total power, and toggle rate. This command is very useful for writing custom scripts.

Note: This command outputs toggle rates as N.A. if the clock domain information for a given net/pin/instance is not found.

Use this command after running the `report_power` or `restore_power_database` command.

Note: The `-nets`, `-pins`, and `-instances` parameters are mutually exclusive. You must specify only one of the parameters.

Parameters

<code>-attribute <i>attributes_list</i></code>	
	Specifies a list of power attributes. If an attribute is not available for the specified design object, the software returns its value as NA. See List of Power Attributes for a mapping of the list of available attributes with the design objects. This is a required parameter.
<code>-help</code>	Outputs a brief description that includes the type and default information for each <code>get_power</code> parameter. For a detailed description of the command and all of its parameters, use the man command: <code>man get_power</code>
<code>-include_unit</code>	Specifies to include the units for the power attributes.

<code>-instances <i>instances_list</i></code>	
	Specifies a list of instance names. The <code>-nets</code>, <code>-pins</code>, and <code>-instances</code> parameters are mutually exclusive. You must specify one, and only one, of the parameters. Note: You can use the <code>get_cells</code> command to determine the power attributes for a collection of instances.
<code>-nets <i>nets_list</i></code>	
	Specifies a list of net names. The <code>-nets</code>, <code>-pins</code>, and <code>-instances</code> parameters are mutually exclusive. You must specify one, and only one, of the parameters. Note: You can use the <code>get_nets</code> command to determine the power attributes for a collection of nets.
<code>-outfile <i>filename</i></code>	
	Specifies the output file into which the power related properties are to be written. The <code>-tcl_list</code> and <code>-outfile</code> parameters are mutually exclusive; you cannot specify them together.
<code>-pg_net {<i>pg_net_name_list</i> all}</code>	
	Specifies to query various power-related properties of a specified power net for an instance. This parameter supports the argument <code>all</code> to query properties of all power nets connected to the instance. You must use the <code>-instances</code> parameter in conjunction with the <code>-pg_net</code> parameter. The <code>-pg_net</code> parameter supports the following attributes: <code>switching_power</code>, <code>internal_power</code>, <code>leakage_power</code>, <code>total_power</code>, and <code>dynamic_power</code>.
<code>-pins <i>pins_list</i></code>	

	Specifies a list of pin names. The <code>-nets</code>, <code>-pins</code>, and <code>-instances</code> parameters are mutually exclusive. You must specify one, and only one, of the parameters. Note: You can use the <code>get_pins</code> command to determine the power attributes for a collection of pins.
<code>-tcl_list</code>	Produces the report in Tcl list format instead of a tabular format. The <code>-tcl_list</code> and <code>-outfile</code> parameters are mutually exclusive; you cannot specify them together.

The following power attributes can be queried for the design objects:

List of Power Attributes with Attribute Description

- `switching_power (net, inst)`: switching power of a net or instance
- `internal_power (inst)`: internal power of an instance
- `dynamic_power (inst)`: switching and internal power of an instance
- `leakage_power (inst)`: leakage power of an instance
- `total_power (inst)`: total power of an instance
- `toggle_rate (net, pin, inst)`: Toggle rate - The frequency at which the state of net, pin, or instance changes between logic levels during the duration of the VCD/TCF
- `duty_cycle (net, pin)`: Static probability of the signal to stay high
- `ref_clock (net, pin, inst)`: Reference clock name (Fastest clock for multi clock domains)
- `ref_clock_period (net, pin, inst)`: Reference clock period
- `transition_density (net, pin)`: $\text{toggle_rate}/\text{ref_clock_period}$ or $\text{toggle_rate}*\text{ref_clock_frequency}$
- `loading_cap (net, pin, inst)`: Loading capacitance
- `voltage (net, pin, inst)`: Operating voltage for a net or pin. For MSMV instance, this attribute can be used to get breakdown of domain specific power.
- `rise_slew (net, pin)`: Rise slew
- `fall_slew (net, pin)`: Fall slew
- `glitch_rate (net, pin)`: Glitch rate - Number of glitches per duration. Applies only to the vector-driven flow.
- `glitch_power (net, pin, inst)`: Glitch power. Applies only to the vector-driven flow.
- `activity_source (net, pin)`: Source of activity annotation. The options are: propagated, user_annotated, and constant.
- `pin_direction (pin)`: [input | output | inout]
- `cell_type (inst)`: Lists cell type as memory, standard cell, IO, sequential, and so on
- `all_attributes (net, pin, inst)`: Lists all related power attributes for the specified design object.

Examples

- The following command returns the specified power attributes for the net `clock1`:

```
get_power -nets clock1 -attribute {switching_power internal_power toggle_rate}
```

The following is the sample output:

```
clock1 0.04 NA 2
```

- The following command returns the specified power attributes for all nets that match the specified pattern `master*`:

```
get_power -nets [get_nets master*] -attribute {switching_power toggle_rate  
activity_source duty_cycle} -include_unit
```

The following is the sample output:

```
masterin[1] 0.001mW 0.2 user_annotated 0.5  
masterin[2] 0.002mW 0.2 user_annotated 0.5  
masterin[3] 0.003mW 0.1 propagated 0.5  
masterin[4] 0.005mW 1 constant 1
```

- The following command returns power-related properties of all power nets connected to the instance `clk_buf_1`:

```
get_power -pg_net all -instances clk_buf_1 -attribute {switching_power  
internal_power leakage_power}
```

The following is the sample output:

```
clk_buf_1 VDD 0.002 0.001 0.000001  
clk_buf_1 VDDL 0.0001 0.0001 0.0000001
```

Signal ElectroMigration Commands

- [create_cell_signal_em_model](#)
- [create_top_scope](#)
- [get_signal_em_analysis_mode](#)
- [report_design_rule](#)
- [report_freq_violation](#)
- [set_max_cap_per_freq](#)
- [set_max_tran_per_freq](#)
- [set_signal_em_analysis_mode](#)
- [verify_AC_limit](#)

create_cell_signal_em_model

```
create_cell_signal_em_model
[-help]
[-cell cell_name]
[-dir directory_name]
[-check_contents]
[-def filename]
[-def_options { list_of_options }]
[-include_def]
[-lef list_of_LEF_files]
[-log_dir directory_name]
[-overwrite]
[-reuse_existing ]
[-power_net_voltages voltage]
[-primary_inputs_only]
[-sdc list_of_SDC_files]
[-spef list_of_SPEF_files]
[-spef_options { list_of_options }]
[-tech_file filename]
[-tech_file_map mapping_file]
[-use_verilog]
[-verilog list_of_Verilog_files]
```

Specifies to create the cell boundary model for the Top Scope Signal EM analysis flow. After the boundary model is generated, it is automatically loaded to the higher-level module.

You can specify the `create_cell_signal_em_model` command before or after running the `set_top_module` command. The details of the two use models are given below:

- If the `create_cell_signal_em_model` command is specified before `set_top_module`, the command will generate the boundary model for a sub-cell and load it. For this use model, the `-cell`, `-dir`, `-spef`, and `-verilog` parameters must be specified.

- If the `create_cell_signal_em_model` command is specified after `set_top_module`, the command will generate the model for the top cell for use in a subsequent top-level run. In this case, the command is issued in a block-level run, and the SPEF should be loaded prior to the `create_cell_signal_em_model` command. For this use model, the `-dir` parameter must be specified. Since the design is already loaded, the relevant information is obtained from the current session.

Parameters

<code>-help</code>	Outputs a brief description that includes the type and default information for each <code>create_cell_signal_em_model</code> parameter. For a detailed description of the command and all of its parameters, use the man command <code>man create_cell_signal_em_model</code>.
<code>-cell cell_name</code>	
	Specifies the name of the cell that is being modeled. The <code>-cell</code> parameter takes a single cell name. If the <code>-cell</code> parameter is used with the command after specifying the <code>set_top_module</code> command, the cell name must be the top cell name.
<code>-def filename</code>	
	Specifies to read the list of DEF files during the boundary model generation session so that the physical information is present when generating the model. When this parameter is specified, the Tech LEF file must be specified with the <code>-lef</code> parameter. Note: The <code>-def</code> and <code>-include_def</code> options must be used together. These parameters are specified if you want the boundary model to include a DEF that corresponds to the boundary model logic. When the boundary model is loaded into the top-level session, this DEF will override any user-supplied DEF that was specified to the <code>read_def</code> command for the model block. If you do not specify a DEF for the boundary model block, then the boundary model DEF will be added to the list of DEFs loaded by <code>read_def</code>.
<code>-def_options { list_of_options }</code>	
	Specifies additional options that are to be used with the <code>read_def</code> command run in the boundary model generation session. The options specified with the <code>-def_options</code> parameter are appended to the <code>read_def</code> command. The <code>-def_options</code> parameter must be specified with the <code>-def</code> parameter. For example, if you want to preserve the wire segments in the user-supplied DEF that was specified with the <code>read_def</code> command for the model block, use the following command: <pre>create_cell_signal_em_model ... -def files -def_options {-preserve_shape}</pre>
<code>-check_contents</code>	
	Check for required files in the BM directory.
<code>-dir directory_name</code>	
	Reads the list of specified DEF files during the boundary model generation session so that the boundary model physical information is available.

<code>-include_def</code>	Specifies to generate the boundary DEF file for the netlist in the model. When specified, the boundary model directory will contain a subset of the input DEF for just the parts of the netlist that are included in the model netlist. Note: The <code>-def</code> and <code>-include_def</code> options must be used together. These parameters are specified if you want the boundary model to include a subset DEF that corresponds to the boundary model logic. When the boundary model is loaded into the top-level session, the subset DEF will override any user-supplied DEF which was specified to the <code>read_def</code> command for the model block. If you do not specify a subset DEF for the boundary model block, then the boundary model DEF will be added to the list of DEFs loaded by <code>read_def</code>.
<code>-lef list_of_LEF_files</code>	
	Specifies the list of LEF files to be loaded. A technology LEF with the layer definition is required if layer information is to be preserved in the SPEF generated for the model.
<code>-log_dir directory_name</code>	
	Specifies the directory where the log files for the boundary model generation run will be saved. If the directory does not exist, a new directory will be created. This directory contains the <code>voltus_sigem.log</code> , <code>voltus_sigem.logv</code> , and <code>voltus_sigem.cmd</code> files.
<code>-overwrite</code>	
	Allows overwrite of the existing directory.
<code>-power_net_voltages voltage</code>	
	Specifies the voltage values for the power nets. The format of the <code>-power_net_voltages</code> argument is a list of net names and voltage values with the <code>@</code> character separating the power net name from the voltage. Example: {VDD1@1.0 VDD2@0.9}
<code>-primary_inputs_only</code>	
	Include only the primary input nets from the input side.
<code>-reuse_existing</code>	
	Specifies to load an existing boundary model directory.
<code>-sdc list_of_SDC_files</code>	
	Specifies the list of SDC files to be loaded. If SDC files are specified, then the fanin to the clock definition pins will be included in the generated model.
<code>-spef list_of_SPEF_files</code>	
	Specifies the list of SPEF files to be loaded.
<code>-spef_options { list_of_options }</code>	
	Specifies additional options that are to be used with the <code>read_spef</code> command run in the boundary model generation session. The options specified with the <code>-spef_options</code> parameter are appended to the <code>read_spef</code> command. For example, if the boundary model is to be generated from a session that uses a view definition, the <code>-spef_options</code> parameter can be used to specify the <code>rc_corner</code>, as shown in the following command: <code>create_cell_signal_em_model ... -spef files -spef_options {-rc_corner corner1}</code> The <code>-spef_options</code> parameter must be specified with the <code>-spef</code> parameter.
<code>-tech_file filename</code>	

	Specifies the extraction technology file to be used with the RC corner. The technology file is required if layer information is to be preserved in the SPEF generated for the model.
<code>-tech_file_map mapping_file</code>	
	Specifies the mapping file that includes the mapping between the SPEF layers to the LEF layers. The mapping file is required if layer information is to be preserved in the SPEF generated for the model. The format of the mapping file is: <pre># <LEF/DEF Layer Name> #<SPEF Layer Name> M1 METAL_1 \ VIA1 VIA_1 \ M2 METAL_2 \</pre>
<code>-use_verilog</code>	
	Specifies to generate boundary models that use Verilog for the model netlist. When this parameter is not specified, the models are generated using a non-readable binary database. The <code>-use_verilog</code> parameter can be specified if you want to debug the models and prefer a readable format for the model netlist.
<code>-verilog list_of_Verilog_files</code>	
	Specifies the list of Verilog files to be loaded.

Examples

- The following is the command for creating a boundary model and loading it into the current session:

```
create_cell_signal_em_model -dir bm_dir -cell cella -verilog cella.v -spef cella.spef.gz -overwrite -lef tech.lef allcells.lef -tech_file 65lp_typical.tch -tech_file_map leflayermap
```
- The following is the command for creating a boundary model after running `set_top_module` in a block-level session:

```
create_cell_signal_em_model -dir bm_dir -overwriteT
```
- The following command can be used prior to `init_design` (or `set_top_module`) to import physical design data:

```
create_cell_signal_em_model -dir bm_dir -overwrite -verilog block.v -spef block.spef -def block.def -include_def
```

Note: It is not possible to import physical design data after `init_design` (or `set_top_module`) as the data has already been loaded. A warning will be issued but will not prevent boundary model generation.
- The following command generates a boundary model, and saves the log files produced by the boundary model generation sub-process into a different directory:

```
create_cell_signal_em_model -dir bm_dir -overwrite -verilog block.v -spef block.spef -log_dir ./bm_logs
```

Related Topics

- "Signal ElectroMigration" in the *Voltus User Guide*

create_top_scope

```
create_top_scope
[-help]
[-cell_list list]
[-fanout_limit value]
[-ignore_nets netname]
[-include_pins pin_names]
[-include_side_loads]
[-inst_list list]
[-summary]
[-thread_count value]
```

Specifies to to perform abstraction of the top-level hierarchical instances for analysis of only the top-level logic, resulting in faster turnaround time. The command allows you to perform abstraction of all the hierarchical instances in the top-level, or specify the top-level hierarchical instances that should be abstracted. When a hierarchical instance is abstracted, all the internal logic is ignored while retaining the interface paths.

The `create_top_scope` command must be specified before the `read_sdc` and `set_top_module` commands.

Parameters

- help	Outputs a brief description that includes the type and default information for each <code>create_top_scope</code> parameter. For a detailed description of the command and all of its parameters, use the man command <code>man create_top_scope</code> .
-cell_list list	
	Specifies to include the interface paths of only the specified cells.
-fanout_limit value	
	Specifies to unmark the boundary nets with fanout greater than the given limit. The default limit is 1000.
-ignore_nets netname	
	Skips unmarking the fanin/fanout of the specified boundary nets.
-include_side_loads	
	Specifies to include all side receivers in the scope.
-inst_list list	
	Specifies to include the interface paths of only the specified instances.
-include_pins pin_names	
	Specifies a list of user-defined pins which should be included that may not be on the boundary paths in the Top Scope Signal EM Analysis flow. The top scope will include the full fanin cone to all the specified pins as part of the analysis. This parameter allows you to handle scenarios where there are some clock constraints defined on pins in sub-blocks which are not part of the boundary logic. In such a case, the constraint would fail and display an error message, and any constraints for clocks generated from the failing clock would fail as well. With this parameter, you can include all the root pins for the clock constraints as part of the top scope, thus resulting in no errors.

-summary	
	Specifies to print statistics about the included instances or nets.
-thread_count <i>value</i>	
	Specifies the number of threads to be used.

Examples

- The following command specifies the top-level hierarchical instances that should be abstracted:

```
create_top_scope -summary -inst_list {InstanceName1, InstanceName2..}
```

get_signal_em_analysis_mode

```
get_signal_em_analysis_mode
[-help]
[-avgRecovery]
[-current_scale_factor]
[-current_scale_table]
[-default_freq_for_unconstrained_nets]
[-delta_T]
[-effort_level]
[-em_limit_scale_factor]
[-em_limit_scale_table]
[-em_res_width]
[-em_temperature]
[-em_threshold]
[-error]
[-extraction_tech_file]
[-forceHoldView]
[-handle_pin_obs_via]
[-ict_em_models]
[-Ipeak_Td_method]
[-lifetime]
[-method]
[-minPeakDutyRatio]
[-minPeakFreq]
[-net | -selected ]
[-net_file]
[-report [-detailed]]
[-report_db]
[-set_current_file]
[-skip_net]
[-skip_net_file]
[-toggle]
[-top_scope_ignore_block_internal_nets_on_boundary_path]
[-use_db_freq]
[-useQrcTech]
[-view ]
```

Displays the following information about a [set_signal_em_analysis_mode](#) parameter in the system log file and in the system console:

- Parameter name
- Current value
- Type (Boolean, string, and so on)
- Whether the current value was set by user

If you do not specify a parameter, the software displays information for all of the `set_signal_em_analysis_mode` parameters.

Parameters

<code>-help</code>	Outputs a brief description that includes type and default information for each <code>get_signal_em_analysis_mode</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <code>man set_signal_em_analysis_mode</code> .
<code>parameter_names</code>	Displays information for the specified parameters. You can specify one or more parameters. See set_signal_em_analysis_mode for description of the parameters you can specify.
<code>-quiet</code>	Displays the current settings for the specified parameters in Tcl list format only. If you specify <code>-quiet</code> without any parameters, the software displays the current settings of all <code>set_signal_em_analysis_mode</code> parameters in the Tcl list format.

Examples

The following command displays the current setting of the `-use_db_freq` parameter:

```
get_signal_em_analysis_mode -use_db_freq
```

The software displays the following information:

```
[DEV]voltus 8> get_signal_em_analysis_mode -use_db_freq
-use_db_freq false                # bool, default=false
false
```

report_design_rule

```
report_design_rule
[-byDriver | -byNet | -byPin]
[-excNetFile fileName]
[-max | -min]
[-outfile fileName]
[-selNetFile fileName]
[-tran | -cap | -fanout]
[> filename]
[>> filename]
```

Reports the following design rule violations:

- Nets that exceed the maximum capacitance constraints in the timing library and timing constraints file.
- Nets that exceed the transition constraints in the timing constraints file.
- Pins that exceed the maximum fanout constraints in the timing library and timing constraints file.

Parameters

-byDriver -byNet -byPin	
	Specifies whether the violations are reported by driver, net or pin. <i>Default:</i> Violations are reported by net.
-excNetFile excNetFileName	
	Specifies the file that contains the hierarchical net names that are excluded from the capacitance violation report.
-max -min	
	Reports only maximum capacitance violations, or only minimum capacitance violations. <i>Default:</i> -max
-outfile fileName	
	Specifies the name of the file to which the results are written.
-selNetFile selNetFileName	
	Specifies the file that contains the hierarchical net names for the report. Only these net names are considered for inclusion in the capacitance violation report.
-tran -cap -fanout	
	Specifies whether to report transition, capacitance, or fanout constraints violations. <i>Default:</i> Reports all design rule violations.

<code>>filename</code>	Redirects report into the outfile. <i>Default:</i> stdout
<code>>> filename</code>	Redirects report into the outfile. <i>Default:</i> stdout

Example

The following command reports the design rule violations in the report file maxcap.report:

```
report_design_rule -outfile maxCap.report
```

report_freq_violation

```
report_freq_violation
[-help]
[-detailed]
[-em_type {avg peak rms all}]
[-excInstFile excInstFileName]
[-outfile fileName]
[-reportFormat integer]
[-selInstFile selInstFileName]
[-slew {min | max}]
[-sort {name | freq | minDiff | maxDiff | minFreq | maxFreq}]
[-tableScale float]
[-view viewName]
```

Runs electromigration (EM) analysis based on the frequency table defined in the Liberty library, and generates a frequency violation report.

The `report_freq_violation` command uses default activity (1.0 or 100% activity on data network and 200% on clock network) or user-defined activity (TCF/VCD) to compute effective frequency on each instance pin. It then determines frequency violation at the output pin from the electromigration templates defined in Liberty, as shown in the following figure:

```
library (name) {
    em_lut_template(EM_Tran) {
        variable_1: input_transition_time;
        index_1("transition_list");
    }
    em_lut_template(EM_Cap) {
        variable_1: total_output_net_capacitance;
        index_1("capacitance_list");
    }
    em_lut_template(EM_Tran_Cap) {
        variable_1: input_transition_time;
        variable_2: total_output_net_capacitance;
        index_1("transition_list");
        index_2("capacitance_list");
    }
}
```

The frequency limit on the output pin is derived based on the slew on the related input pin and output load.

The activity for the design can be specified as follows:

- `set_default_switching_activity`
- `propagate_activity`
- `write_tcf`
- `read_activity_file -format TCF -set_net_freq true <TCF file>`

Parameters

<code>-detailed</code>	Generates a detailed report containing information for all instances, including those without violations.
<code>-em_type {avg peak rms all}</code>	

	Specifies to report frequency violations for the rms/peak/avg current waveform in the frequency violation report. When this parameter is specified, the software reads the <code>em_max_toggle_rate</code> table in the Liberty library for the specified EM current type (rms/peak/avg). You can specify <code>all</code> to perform all three types of analyses.	
<code>-excInstFile</code> <i>excInstFileName</i>		
	Specifies a file that lists the names of instances to exclude from the frequency violation report.	
<code>-help</code>	Outputs a brief description that includes type and default information for each <code>report_freq_violation</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <code>man report_freq_violation.</code>	
<code>-outfile</code> <i>fileName</i>		
	Specifies the report file (default: <code>freqViolations.rpt</code>) for the violation data.	
<code>-reportFormat</code> <i>integer</i>		
	Specifies a format of the violation report. The report details vary according to the selected report format. You can specify report formats 0, 1, or 2. <i>Default: 0</i>	
<code>-selInstFile</code> <i>selInstFileName</i>		
	Specifies a file that lists the names of instances to include in the frequency violation report.	
<code>-slew {min max}</code>		
	Specifies the minimum or maximum slew value for the report. <i>Default: min</i>	
<code>-sort {name freq minDiff maxDiff minFreq maxFreq}</code>		
	Sorts the instance-based frequency data by <code>name</code> , <code>freq</code> , <code>minDiff</code> , <code>maxDiff</code> , <code>minFreq</code> , and <code>maxFreq</code> . By default, the violation report is not sorted.	
	<code>name</code>	Name of a pin or arc.
	<code>freq</code>	Frequency value of a pin or arc.
	<code>min_freq</code>	Minimum frequency value of a pin or arc as defined in the frequency table.
	<code>max_freq</code>	Maximum frequency value of a pin or arc as defined in the frequency table.
	<code>min_diff</code>	Difference between <code>freq</code> and <code>minFreq</code> of a pin or arc.
	<code>max_diff</code>	Difference between <code>freq</code> and <code>minFreq</code> of a pin or arc.

- tableScale float	Specifies the scale factor for EM table value. <i>Default:</i> 1.0
-view viewName	Specifies the view for which to generate the frequency violation report. Use this parameter when working with multi-mode multi-corner (MMMC) designs.

Example

The following command reports instances that violate frequency constraints and lists them in the `freqViol.rpt` report file:

```
report_freq_violation -outfile freqViol.rpt
```

Instance File Format

The instance file is a text file that contains a list of instances. Each instance name appears in one line in the file. The comments begin with "#". Leading or ending spaces in instance names are ignored.

Report Format

The report formats are described below.

Format 0

The following command generates a report providing details of `minFreq`, `Freq`, `Tran`, `Cap`, `Remark`, and `Arc/Pin` and directs the output to report file `summaryViol.rpt`:

```
report_freq_violation -outfile summaryViol.rpt -report_format 0
```

For example,

minFreq	Freq	Tran	Cap	Remark	Arc/Pin
5.985	1.009	0.0356	0.003103		tt0/data/r3/add_27/U1_1_6/Y

Format 1

When `reportFreqViolation -reportFormat` is set to 1, the report provides details as shown in the following example:

Status	Remark	freq	minFreq	maxFreq	Arc/Pin
[P]	*	1.009	5.985	5.995	tt0/data/r3/add_27/U1_1_6/Y

Format 2

A detailed report is generated when `reportFreqViolation -reportFormat` is set to 2, as shown in the following example:

Status	Remark	freq	minFreq	maxFreq	minTran	maxTran	minCap	max_cap	Arc/Pin
[P]	*	1.009	5.985	5.995	3.560e-02	3.790e-02	3.103e-03		
		3.103e-03							tt0/data/r3/add_27/U1_1_6/Y

Where:

Status	Arc or pin status - F (fail), V (violation), P (pass), or ? (unknown).
--------	--

Voltus IC Power Integrity Solution Text Command Reference
Signal ElectroMigration Commands

Remark	Indicates whether the pin is a clock.
freq	Working frequency of the arc or pin.
minFreq	Minimum frequency value taken from the frequency table.
maxFreq	Maximum frequency value taken from the frequency table.
minTran	Minimum slew value.
maxTran	Maximum slew value.
minCap	Minimum capacitance load.
maxCap	Maximum capacitance load.
Arc/Pin	Name of the arc or pin.

set_max_cap_per_freq

```
set_max_cap_per_freq
[-help]
[-cap {cap1 cap2 ...}]
[-force]
[-freq {freq1 freq2 ...}]
[-lib libName]
-pins {cell pin}
```

Adds a maximum capacitance per frequency table to the existing `max_capacitance` statement in the timing library.

Within-cell, DC electromigration limits vary with varying frequency of transition. The software manages DC electromigration using a maximum capacitance limit based on switching frequency. This is driven by `maxCapPerFreq` data in the liberty. The violations can be reported using the `report_design_rule` command in Voltus. These constraints can be set for cell pins manually using this command.

Timing analysis uses this table to detect violations, and `optDesign` in Innovus can be used to repair violations.

Parameters

-help	Prints out the command usage.
-cap {cap1cap2...}	
	Specifies a list of capacitance values that must be greater than or equal to 0. You must specify the same number of <code>-freq</code> and <code>-cap</code> values. The software derives the maximum capacitance from a linear interpolation (or extrapolation) for a given frequency value using two frequency values in the table. For interpolation, the software uses the closest value above the frequency, and the closest value below the frequency. For extrapolation, the software uses the two closest values to the frequency. <i>Type:</i> Float, specified in units of pF
-force	Computes the maximum capacitance from the interpolated <code>-freq</code> and <code>-cap</code> values only and ignores any previously defined <code>max_capacitance</code> values for these pins in the library. <i>Default:</i> Computes the maximum capacitance using the worst case (minimum value) between the library <code>max_capacitance</code> values and the interpolated <code>-cap</code> values.
-freq {freq1freq2...}	
	Specifies a list of at least two frequency values that must be greater than or equal to 0. You must specify the same number of <code>-freq</code> and <code>-cap</code> values, and the frequency values must be specified in ascending order. <i>Type:</i> Float, specified in units of Hz
-lib libName	

	Specifies the library name inside liberty. For example, library (<code>slow1vt</code>) inside <code>slow.lib</code>. The library name should be specified as <code>slow1vt</code>. Modifies only the matching cell-pins in the specified library. <i>Default:</i> Modifies the matching cell-pins in all libraries
<code>-pins {cellpin}</code>	
	Adds the new capacitance constraints to the specified set of cells and pins. You can specify output or inout pins. The asterisk (*) wildcard character can be used in a cell or pin name. If a table associated with the pin already exists, it is replaced with the new table. For analysis and optimization, the software uses the most restrictive (the lowest capacitance) value from either the table or the <code>.lib max_capacitance</code>. If you also specify the <code>-force</code> parameter, the <code>max_capacitance</code> value is ignored. The final value can be further modified using <code>.sdc set_max_capacitance</code> statements.

Example

Assume all output pins of a library have a `max_capacitance` of 3.5 pF from the library file.

- The following command sets the frequency dependent `max_capacitance` to 3.0 pF at a frequency of 100 Mhz, and to 1.5 pF at a frequency of 200 Mhz, for all output or inout pins for all cells in the library named `fast`:

```
set_max_cap_per_freq -lib fast -freq {100e6 200e6} -cap {3.0 1.5} -pins {* *}
```

As a result, for a net at 300 Mhz, an `and2` cell output pin has an extrapolated `max_capacitance` value of 0 pF (which means the cell cannot be used at that frequency). An extrapolation to 50 Mhz results in a `max_capacitance` value of 3.75 pF. However, the 3.75 pF value is greater than the existing `.lib max_capacitance` limit of 3.5 pF; therefore timing analysis uses the 3.5 pF value.

- The following command applies the same constraints to all libraries with cell names matching `dff*` that have a pin named `Qbar`:

```
set_max_cap_per_freq -force -freq {100e6 200e6} -cap {3.0 1.5} -pins {dff* Qbar}
```

The original `.lib max_capacitance` limit is ignored because the `-force` parameter is specified; therefore extrapolation to 50 Mhz results in a final `max_capacitance` constraint value of 3.75 pF.

set_max_tran_per_freq

```
set_max_tran_per_freq
[-help]
[-force]
-freq {freq1 freq2 ...}
[-lib libName]
-pins {cell pin}
-tran {tran1 tran2 ...}
```

Adds a maximum transition per frequency table to the existing `max_transition` statement in the timing library.

Within-cell, DC electromigration limits vary with varying frequency of transition. The software manages DC electromigration using a maximum transition limit based on switching frequency. The violations can be reported using the `report_design_rule` command in Voltus. These constraints can be set for cell pins manually using this command.

Timing analysis uses this table to detect violations, and `optDesign` in Innovus can be used to repair violations.

Use this command any time after loading the timing library files (that is, after importing or restoring the design).

Parameters

<code>-help</code>	Prints out the command usage.
<code>-force</code>	Computes the maximum transition from the interpolated <code>-freq</code> and <code>-tran</code> values only and ignores any previously defined <code>max_transition</code> values for these pins in the library. Default: Uses the worst case (minimum value) between the <code>.lib</code> <code>max_transition</code> values and the interpolated <code>-tran</code> values.
<code>-freq {freq1freq2...}</code>	
	Specifies a list of at least two frequency values that must be greater than or equal to 0. You must specify the same number of <code>-freq</code> and <code>-tran</code> values, and the frequency values must be specified in ascending order. Type: Float, specified in units of Hz
<code>-lib libName</code>	
	Specifies the library name inside liberty. For example, library (<code>slow1vt</code>) inside <code>slow.lib</code>. The library name should be specified as <code>slow1vt</code>. Modifies only the matching cell-pins in the specified library. Default: Modifies the matching cell-pins in all libraries.
<code>-pins {cellpin}</code>	

	Adds the new transition constraints to the specified set of cells and pins. You can specify input, output, or inout pins. The asterisk (*) wildcard character can be used in a cell or pin name. If a table associated with the pin already exists, it is replaced with the new table. For analysis and optimization, the software uses the most restrictive (the lowest transition) value from either the table or the <code>.lib max_transition</code>. If you also specify the <code>-force</code> parameter, the <code>max_transition</code> value is ignored. The final value can be further modified using <code>.sdc set_max_transition</code> statements.
<code>-tran {tran1tran2...}</code>	
	Specifies a list of transition values that must be greater than or equal to 0. You must specify the same number of <code>-freq</code> and <code>-tran</code> values. The software derives the maximum transition limit from a linear interpolation (or extrapolation) for a given frequency value using two frequency values in the table. For interpolation, the software uses the closest value above the frequency, and the closest value below the frequency. For extrapolation, the software uses the two closest values to the frequency. <i>Type:</i> Float, specified in units of ns

Examples

Assume all output pins of a library have a `max_transition` value of 0.9 ns from the library file.

- The following command sets the frequency dependent `max_transition` to 1.0 ns at a frequency of 100 Mhz, and to 0.7 ns at a frequency of 200 Mhz, for pins matching `in*` of cells matching `*` (that is, all cells) in the library named `slow`:

```
set_max_tran_per_freq -lib slow -freq {100e6 200e6} -tran {1.0 0.7} -pins {* in*}
```

As a result, at 300 Mhz, an `and2` cell input pin named `in1` has an extrapolated `max_transition` value of 0.4ns. An extrapolation to 50 Mhz results in a `max_transition` value of 1.15 ns. However, the 1.15 ns value is greater than the existing `.lib max_transition` value of 0.9 ns; therefore timing analysis uses the 0.9 value.

- The following command applies the same constraints to all libraries with cell names matching `dff*` that have a pin named `clk`:

```
set_max_tran_per_freq -force -freq {100e6 200e6} -tran {1.0 0.7} -pins {dff* clk}
```

The original `.lib max_transition` value is ignored because the `-force` parameter is specified; therefore an extrapolation to 50 Mhz results in a final `max_transition` limit of 1.15 ns.

set_signal_em_analysis_mode

```
set_signal_em_analysis_mode
[-help]
[-additional_individual_violation_report {true | false}]
[-avgRecovery em_recover]
[-check_thermal_aware_em {true | false}]
[-current_scale_factor {{avg value} {rms value} {peak value}}]
[-current_scale_table current_scale_table_file]
[-default_freq_for_unconstrained_nets freq_in_Hz]
[-delta_T value]
[-delta_T_layer_list {{<LEF_layer_name> <delta_T_in_C>}+}]
[-detailed {true|false}]
[-effort_level {low | medium | high}]
[-em_cdf_percentage value]
[-em_limit_scale_factor {{avg value} {rms value} {peak value}}]
[-em_limit_scale_table em_limit_scale_table_file]
[-em_res_width {drawn silicon}]
[-em_temperature value]
[-em_temperature_layer_list {{<LEF_layer_name> <em_temperature_in_C>}+}]
[-em_threshold <value>]
[-enable_seb {true | false}]
[-env_temperature temperature]
[-error value]
[-extraction_tech_file filename]
[-forceHoldView]
[-handle_pin_obs_via]
[-ict_em_models file]
[-IpeakTd_method {effective_width_from_integration effective_half_peak_width max_equivalent_dc_peak
sum_half_peak_width}]
[-ircx_models {RC.ircx EMIR.ircx}|{RC.ircx}]
[-lifetime value]
[-lifetime_rating_factor_file filename]
[-method {rms peak avg}]
[-minPeakDutyRatio value]
[-minPeakFreq value]
[-min_net_freq_file filename]
[-net netNames | -selected ]
[-net_file list_file]
[-net_freq_file filename]
[-read_detail_delta_temperature_file filename]
[-report filename [-detailed]]
[-report_db]
[-report_unannotated_shapes {true | false}]
[-reset]
[-seb_lifetime value]
[-seb_table table_filename]
[-seb_temperature temperature]
[-set_current_file filename]
[-simulation_net list_of_nets]
[-simulation_net_file filename]
[-skip_category_mode {0 | 1 | 2}]
[-skip_net_file list_file]
[-skip_net net_name_list]
[-toggle value]
```

```
[-top_scope_ignore_block_internal_nets_on_boundary_path]
[-useQrcTech]
[-use_rms_delta_t {true | false}]
[-view viewName]
```

Sets global parameters for signal electromigration. Use this command to specify the required `verify_AC_limit` mode settings before running any commands that perform signal electromigration. The parameters specified with the `set_signal_em_analysis_mode` command are then used automatically whenever signal electromigration takes place.

Use the `get_signal_em_analysis_mode` command to return the current settings for the `set_signal_em_analysis_mode` command.

Note: Use the `-method` parameter to specify the waveform calculation method to be used (`rms`, `peak`, or `avg`).

Parameters

<code>-help</code>	Outputs a brief description that includes type and default information for each <code>set_signal_em_analysis_mode</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <pre>man set_signal_em_analysis_mode</pre>
<code>-additional_individual_violation_report {true false}</code>	
	Specifies to generate separate violation reports for each current waveform calculation method (<code>rms</code>, <code>peak</code>, or <code>avg</code>). If this parameter is not specified, a single violation report file is generated using the <code>-method {RMS AVG peak}</code> parameter. This parameter gives you the flexibility of generating either a single collated report or individual reports for each analysis method. Example: <pre>set_signal_em_analysis_mode \ -method {rms peak avg} \ -report context1.rpt \ ... -additional_individual_violation_report true</pre> This command will generate 3 different reports, namely <code>context1.rpt_avg</code>, <code>context1.rpt_peak</code>, and <code>context1.rpt_rms</code>. If there are no EM violations for a method, the tool will not generate the corresponding separate report. In this example, if there are violations for <code>avg</code> and <code>rms</code> but none for <code>peak</code>, then it will generate only two reports, <code>context1.rpt_avg</code> and <code>context1.rpt_rms</code>. Default: <code>false</code>
<code>-avgRecovery em_recover</code>	
	Specifies the recovery factor used for calculating I_{avg} current density with recovery. By default, the recovery factor (per layer) is read from the QRC techfile. If you specify <code>-avgRecovery em_recover</code>, it overrides the QRC tech <code>em_recover</code> factor for all layers used in I_{avg} limits. The minimum <code>-avgRecovery</code> value is <code>-1</code> and the maximum is <code>1</code>. Note: If you do not specify <code>-avgRecovery</code> and the <code>em_recover</code> factor is not specified in the QRC techfile as well, the tool gives an error that I_{avg} cannot be computed.
<code>-check_thermal_aware_em {true false}</code>	

	Specifies to use the thermal map for EM checks. The thermal map file is specified using the <code>-read_thermal_map</code> parameter. The Rail Analysis engine uses the temperature specified in the thermal map file for EM analysis. EM limit has a strong temperature dependency. For a chip that has substantial temperature variation across the chip, the Rail Analysis engine can use the temperature specified in the thermal map file to perform accurate EM analysis. <i>Default:</i> <code>false</code>													
<code>-current_scale_factor</code> <code>{{avg value} {rms value} {peak value}}</code>														
	Scales the signal EM current by the respective factor for avg, rms, or peak current specified by <code>{avg value} {rms value} {peak value}</code>. The value specified for this parameter must be <code>> 0</code> and <code><= 10</code>.													
<code>-current_scale_table</code> <code>current_scale_table_file</code>														
	Specifies the name of the scale table file used for layer-based scaling of the signal EM current. The scale table contains the scale factor for avg, rms, or peak current for each layer. This parameter is mutually exclusive with <code>-current_scale_factor</code>. The <code>-current_scale_table</code> parameter takes precedence over <code>-current_scale_factor</code>. The format of the table is given below: <table><thead><tr><th>layer</th><th>avg</th><th>rms</th><th>peak</th></tr></thead><tbody><tr><td>layer1</td><td>scalefactor1</td><td>scalefactor2</td><td>scalefactor3</td></tr><tr><td>layer2</td><td>scalefactor1</td><td>scalefactor2</td><td>scalefactor3</td></tr></tbody></table> Here, - layer name is the LEF layer name that could be a metal/via layer. - avg, rms, peak correspond to the respective scale factor for avg, rms, or peak current. The scale factor value must be <code>> 0</code> and <code><= 10</code>.	layer	avg	rms	peak	layer1	scalefactor1	scalefactor2	scalefactor3	layer2	scalefactor1	scalefactor2	scalefactor3	
layer	avg	rms	peak											
layer1	scalefactor1	scalefactor2	scalefactor3											
layer2	scalefactor1	scalefactor2	scalefactor3											
<code>-default_freq_for_unconstrained_nets</code> <code>freq_in_Hz</code>														
	Specifies the frequency for EM calculation when a net has no defined frequency or has a defined frequency of 0 Hz in the design. <i>Default:</i> 1MHz													
<code>-delta_T</code> <code>temp</code>														
	Specifies the maximum temperature rise permitted in units of Celsius. This value is normally used in the QRC tech file for the RMS current limit equation. For example: $em_jmax_ac_rms = EQU \sqrt{12.66 * delta_T * (w - 0.003)^2 * (w - 0.003 + 0.264) / (w - 0.003 + 0.0443)}$ <i>Default:</i> 5													
<code>-delta_T_layer_list</code> <code>{{<LEF_layer_name> <delta_T_in_C>}+}</code>														
	Specifies the delta temperature for specific LEF layers during signal EM RMS current limit analysis.													
<code>-detailed</code>	Generates a detailed report containing information for all signal nets, including those without violations.													
<code>-effort_level</code> <code>{low medium high}</code>														

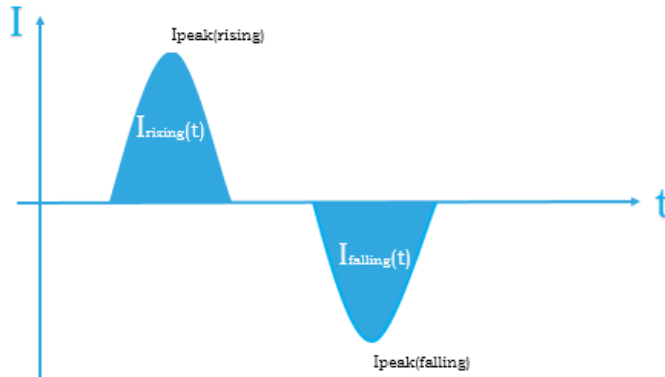
	Specifies the method used for current calculation of nets in the Signal ElectroMigration analysis flow. The two methods are: • non-simulation mode - use the capacitance ratio to distribute the current down the stream from the driver to the receiver. • simulation mode - simulates resistance and capacitance to get the true current on each segment. This mode is more accurate but requires a longer run time. This parameter has the following three arguments: • low: enables the non-simulation mode for all nets where the current distribution is based on the capacitance ratio. • medium: enables the simulation mode for the nets with physical loops, and the non-simulation mode for the non-loop nets. • high: enables the simulation mode for all nets. The detailed report (<code>set_signal_em_analysis_mode -report <filename> -detailed</code>) shows "N/A" in the Cap column for nets in the simulation mode. The default value for <code>-effort_level</code> is <code>low</code>. The <code>medium</code> and <code>high</code> levels require a VTS-AA license. Note: For the simulation mode with effort level <code>medium</code> or <code>high</code>: • If you use <code>extract_rc</code>, run <code>set_extract_rc_mode -qrcCmdType partial -qrcCmdFile qrc.user.cmd</code>, where <code>qrc.user.cmd</code> must include the following CCL commands: <pre>extraction_setup -analysis em -cluster_vias_in_em true output_db -type spef -subtype "EXTENDED"</pre> Refer to the example below: <pre>output_db \ -type spef -busbit_delimiter "[" -escape_special_character false \ -hierarchy_delimiter "/" -output_incomplete_nets true \ -output_unrouted_nets true -pin_delimiter ":" -subtype "EXTENDED" \ -disable_subnodes false -add_explicit_vias true \ -include_parasitic_res_width true -include_parasitic_res_width_drawn true</pre> • If you run standalone Quantus to generate the required SPEF file in the extended format separately, then instead of <code>extract_rc</code>, use the following command for signal EM analysis: <pre>read_spef -starN -extended spef_file</pre>
<code>-em_cdf_percentage value</code>	
	Calculates the Cumulative Distribution Factor (CDF) derating factor based on the EM rules specified in the ICT EM file, given by the <code>-ict_em_models</code> parameter. This parameter is used to calculate the fail rate and reliability of a design. The value of the CDF percentage should be between 0 and 100. The default value is <code>1e-9</code>. The use model of this parameter is: <pre>set_signal_em_analysis_mode \ -method {rms peak avg} \ ... -em_cdf_percentage 5</pre> When the CDF related rule exists in the ICT EM file, the software uses the default value <code>1e-09</code> for <code>-em_cdf_percentage</code> if it is not specified.
<code>-em_limit_scale_factor {{avg value} {rms value} {peak value}}</code>	

	Scales the calculated signal EM limits by the respective factor for avg, rms or peak current limit specified by {avg value} {rms value} {peak value}. The software compares the current through the net against the scaled EM limit to report violations in signal EM. The value specified for this parameter must be > 0 and <= 10.													
<code>-em_limit_scale_table em_limit_scale_table_file</code>														
	Specifies the name of the scale table file used for layer-based scaling of the signal EM limit. The scale table contains the scale factor for avg, rms, or peak current for each layer. This parameter is mutually exclusive with <code>-em_limit_scale_factor</code>. The <code>-em_limit_scale_table</code> parameter takes precedence over <code>-em_limit_scale_factor</code>. The format of the table is given below: <table><tr><th>layer</th><th>avg</th><th>rms</th><th>peak</th></tr><tr><td>layer1</td><td>scalefactor1</td><td>scalefactor2</td><td>scalefactor3</td></tr><tr><td>layer2</td><td>scalefactor1</td><td>scalefactor2</td><td>scalefactor3</td></tr></table> Here, - layer name is the LEF layer name that could be a metal/via layer. - avg, rms, peak correspond to the respective scale factor for avg, rms, or peak current. The scale factor value must be > 0 and <= 10.	layer	avg	rms	peak	layer1	scalefactor1	scalefactor2	scalefactor3	layer2	scalefactor1	scalefactor2	scalefactor3	
layer	avg	rms	peak											
layer1	scalefactor1	scalefactor2	scalefactor3											
layer2	scalefactor1	scalefactor2	scalefactor3											
<code>-em_res_width {drawn silicon}</code>														
	Specifies width used to get EM limit. The legal values are: - drawn - silicon Default: drawn													
<code>-em_temperature value</code>														
	Specifies the temperature expected for the device and is used to look up the temperature scaling factor. By default, no scaling is done. The <i>value</i> is a float, in units of degrees Celsius.													
<code>-em_temperature_layer_list {{<LEF_layer_name> <em_temperature_in_C>}+}</code>														
	Sets a different temperature for specific LEF layers during signal EM AVG current limit analysis.													
<code>-em_threshold value</code>														
	Specifies to report violations in signal EM above the threshold value. The value is the ratio of signal current (avg/peak/rms) to the respective EM limit. Default is 1.0.													
<code>-enable_seb {true false}</code>														
	Specifies to enable the Statistical ElectroMigration Budgeting (SEB) analysis. Default: false													
<code>-env_temperature <temperature in °C></code>														
	Specifies the ambient temperature used to calculate S_{DC} for SEB analysis. Default: 25													

Voltus IC Power Integrity Solution Text Command Reference
Signal ElectroMigration Commands

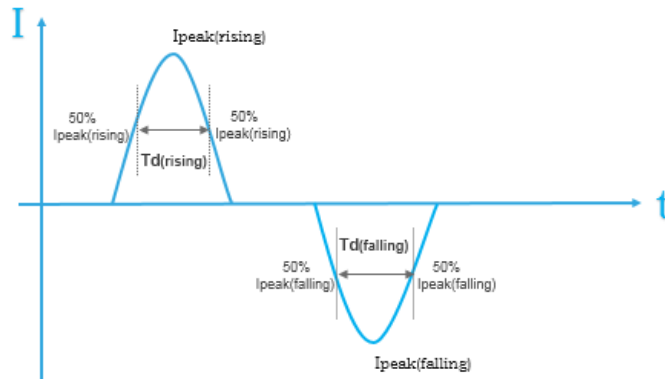
<code>-error value</code>		
	Controls the number of errors to be included in the signal EM violation report file. If you do not specify this parameter, the top 10000 signal nets with violation will be included in the violation report. This parameter can be used to control the size of the violation report file. Default: 10000	
<code>-extraction_tech_file filename</code>		
	Specifies an extraction technology file for the Signal ElectroMigration analysis flow. If this parameter is not specified, then by default the extraction technology file will be taken from the analysis view (current RC corner). This parameter must be used when the technology file is not specified through the analysis view (<code>create_rc_corner -qx_tech_file</code>). If you specify both <code>create_rc_corner -qx_tech_file</code> and <code>set_signal_em_analysis_mode -extraction_tech_file</code>, then the technology file specified with the <code>-extraction_tech_file</code> parameter will take precedence. In addition, if a technology file is not specified using either of these two command parameters, the software displays a warning message for the missing technology file, and will not read the EM models specified through the <code>-ict_em_models</code> parameter.	
<code>-forceHoldView</code>		
	Forces <code>set_signal_em_analysis_mode</code> to use the hold view for current calculation even during setup fixing.	
<code>-handle_pin_obs_via</code>		
	Specifies to consider the vias defined in the <code>OBS</code> (obstructions/blockages) section of LEF in the Signal ElectroMigration flow. By default, the software does not consider via in obstruction. This parameter does not honor the via obstructions with "SPACING 0".	
<code>-ict_em_models file</code>		
	Specifies the ICT-EM file which only contains the EM rules and excludes the RC extraction data in ICT. The software will use the EM rules in the specified ICT-EM file to calculate EM limits for signal net EM analysis even if there are embedded EM rules in the Quantus QRC tech file.	
<code>-Ipeak_Td_method {effective_width_from_integration effective_half_peak_width max_equivalent_dc_peak sum_half_peak_width}</code>		
	Specifies the Td calculation method for Ipeak check. The legal values are depicted below.	

effective_width_from_integration



- $$T_d = \frac{\int_0^{T_{SW}} |I(t)| dt}{I_{\text{peak_measured}}}$$
 - T_{SW} = Period of one switching cycle (rising + falling)
 - $I_{\text{peak_measured}} = \text{abs.max}(I_{\text{peak}}(\text{rising}), I_{\text{peak}}(\text{falling}))$

effective_half_peak_width

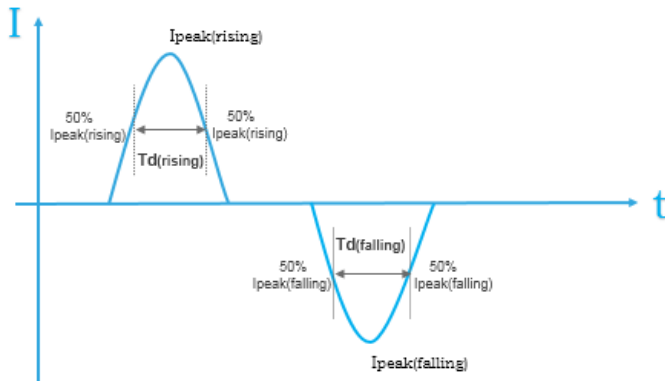


- When $I_{\text{peak}}(\text{rising}) \geq I_{\text{peak}}(\text{falling})$

$$T_d = T_d(\text{rising}) + \left(\frac{I_{\text{peak}}(\text{falling})}{I_{\text{peak}}(\text{rising})} \right)^2 * T_d(\text{falling})$$
- When $I_{\text{peak}}(\text{rising}) < I_{\text{peak}}(\text{falling})$

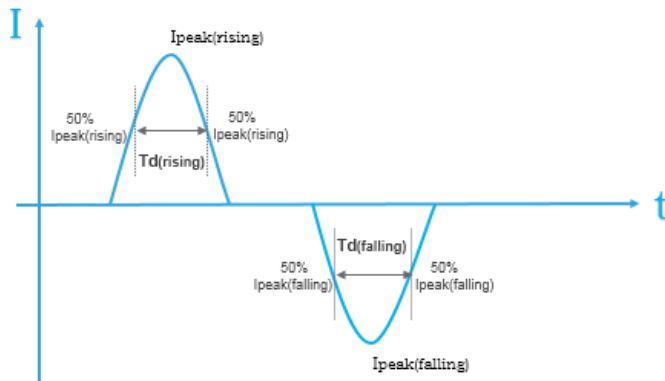
$$T_d = T_d(\text{falling}) + \left(\frac{I_{\text{peak}}(\text{rising})}{I_{\text{peak}}(\text{falling})} \right)^2 * T_d(\text{rising})$$

max_equivalent_dc_peak



- $R(\text{rising}) = T_d(\text{rising}) / (T/2)$
- $R(\text{falling}) = T_d(\text{falling}) / (T/2)$
- Choose $\max(I_{\text{peak(rising)}} \cdot \sqrt{R(\text{rising})}, I_{\text{peak(falling)}} \cdot \sqrt{R(\text{falling})})$ to compare against $I_{\text{peak_dc_limit}}$
- $I_{\text{peak_ac_limit}} = I_{\text{peak_dc_limit}} / \sqrt{R}$
- When $I_{\text{peak(rising)}} \cdot \sqrt{R(\text{rising})} \geq I_{\text{peak(falling)}} \cdot \sqrt{R(\text{falling})}$
 $T_d = 2 \cdot T_d(\text{rising})$
- When $I_{\text{peak(rising)}} \cdot \sqrt{R(\text{rising})} < I_{\text{peak(falling)}} \cdot \sqrt{R(\text{falling})}$
 $T_d = 2 \cdot T_d(\text{falling})$

sum_half_peak_width



- $T_d = T_d(\text{rising}) + T_d(\text{falling})$

Default: effective_width_from_integration

	<code>-ircx_models {RC.ircx EMIR.ircx} {RC.ircx}</code>	
	Specifies an encrypted or unencrypted RC/EM technology (.ircx) file. The foundry-specific IRCX file, which contains the process information for extraction and EM modeling, can be used when the ICT-EM file is not available for electromigration analysis. You can either specify two IRCX files, one file containing the RC process data (RC.ircx) and the other file containing EM rules (EM.ircx), or a single IRCX file that includes both the RC and EM sections (RC.ircx). A single IRCX file is used for the mature nodes, and separate RC and EM IRCX files are used for the advanced process nodes.	
	<code>-lifetime value</code>	
	Specifies the total hours of operation expected for the device, and is used to look up the lifetime scaling factor. <i>value</i> is used to find the scaling factor by interpolating from a piece-wise-linear table in the LEF (see ACCURRENTDENSITY HOURSPWL table) or the QRC tech file (em_jmax_ac model with jmax_lifetime table). If there is no scaling factor table, no scaling is done. <i>value</i> is a float, in units of hours.	
	<code>-lifetime_rating_factor_file filename</code>	
	Specifies a lifetime rating factor file that contains jmax_life values for each layer. These values are defined based on temperature and lifetime pairs. When you provide this file, EM analysis will automatically select the appropriate jmax_life factor for each layer according to the specific temperature-lifetime combinations defined within the file. The lifetime and EM temperature can be specified using the <code>-lifetime</code> and <code>-em_temperature</code> parameters. Following is a snippet of the lifetime rating factor file: <pre>*Rating factor of M0/V0/M1/V1/M2/V3/M3...TEMPERATURE 90 95 100 105 110 ... LifeTime -----0.5 2.578 2.242 2.024 1.774 1.547 1 2.520 2.191 1.979 1.734 1.512 2 2.430 2.113 1.908 1.672 1.4583 2.382 2.072 1.871 1.639 1.430...----- -----</pre>	
	<code>-method {rms peak avg}</code>	
	Specifies the current waveform calculation method. You can specify one or more of the following options: <code>avg</code>: Checks I_{avg} limits and reports any average current limit violations in the design. <code>peak</code>: Checks I_{peak} limits and reports any peak current limit violations in the design. <code>rms</code>: Checks I_{rms} limits and reports any RMS current limit violations in the design. Default: rms Notes - If you specify <code>peak</code> or <code>avg</code> but AAE is not enabled, the tool displays an error message. - When you run <code>set_signal_em_analysis_mode -method rms</code>, the command uses AC limit rules in LEF even if the EM qrcTechfile is provided. Use <code>set_signal_em_analysis_mode -method rms -useQrcTech</code> to check rms EM limits from qrcTechfile.	
	<code>-minPeakDutyRatio duty_ratio</code>	

	Sets the minimum duty ratio value for I_{peak} calculation. If the calculated duty ratio of a net is lower than the specified value, the specified value will be used to calculate I_{peak}. The minimum value for <i>duty_ratio</i> is 0.0 and the maximum is 1.0. Default: 0.05 The following determines the priority and precedence of the <code>-minPeakDutyRatio</code> parameter: - When there are explicit duty ratio (r) dependencies on EM rules, the user-specified <code>minPeakDutyRatio</code> value will overwrite the software-calculated duty ratio (r), if the calculated duty ratio is smaller than the user-specified <code>minPeakDutyRatio</code> value. The <code>minPeakDutyRatio</code> value will then be applied to the EM rule selection to find the corresponding EM rule with duty ratio (r) dependency for EM limit calculation. - When there are no explicit duty ratio (r) dependencies on EM rules, the user-specified <code>minPeakDutyRatio</code> value will overwrite the tool-calculated duty ratio (r), if the calculated duty ratio is smaller than the user-specified <code>minPeakDutyRatio</code> value, and the software will calculate the EM limit based on the user-specified <code>minPeakDutyRatio</code>. If you do not specify a value to <code>minPeakDutyRatio</code>, the default value of 0.05 will be applied.	
<code>-minPeakFreq freq_in_Hz</code>		
	Ignores I_{peak} current limit calculation for signal nets that have an effective frequency below the specified value (in Hertz). Default: 1e+06 Hz (1 MHz)	
<code>-min_net_freq_file filename</code>		
	Specifies a file that contains the user-specified minimum frequency for a net. The following cases are applicable: Case 1: Net frequency is specified using both <code>-min_net_freq_file</code> and <code>-net_freq_file</code> parameters: The frequency of the net will be maximum of the following: frequency specified using the <code>-net_freq_file</code> parameter, frequency specified using the <code>-min_net_freq_file</code> parameter, and default frequency for the unconstrained net. Case 2: Net frequency is specified using the <code>-min_net_freq_file</code> parameter, without specifying the <code>-net_freq_file</code> parameter: The frequency of the net will be maximum of the following: Tempus result, frequency specified using the <code>-min_net_freq_file</code> parameter, and default frequency for the unconstrained net. File Format: <pre><net name> <freq in Hz></pre> File Format Example: <pre>IN_1 1000000000</pre>	
<code>-net {netNames} -selected</code>		

	Specifies whether to check named or selected nets. If you name only one net, you do not need the braces around the net name. If you specify more than one net, you must separate the net names with a space. For example, <code>set_signal_em_analysis_mode -nets {clk0 clk1}</code>. The <code>-selected</code> parameter analyzes the net specified with the <code>selectNet {netName} [command]</code> command, and/or by selecting a net in the main GUI window. The following are examples of this parameter: Example 1: <pre>selectNet {netA} set_signal_em_analysis_mode -selected</pre> Example 2: <pre>selectNet [get_ccopt_clock_tree_nets] set_signal_em_analysis_mode -selected</pre> Default: If you do not specify <code>-nets</code> or <code>-selected</code>, <code>set_signal_em_analysis_mode</code> verifies all nets.	
<code>-net_file list_file</code>		
	Specifies the net from file to check the signalEM analysis. The format is one net name per line, with comments using <code>#</code> as the first character. For example: <pre>#list of nets to skip net_a net_b net_c</pre>	
<code>-net_freq_file filename</code>		
	Specifies a file containing the user-specified frequency for a list of nets. This parameter allows you to overwrite the frequency for specific nets if the frequency set by <code>-toggle</code> needs to be modified. The format of the file is: <pre><net name> <freq in Hz></pre> Example: <pre>IN_1 1000000000 n_I2_1 2700000000 OUT_1 1200000000</pre>	
<code>-read_detail_delta_temperature_file filename</code>		
	Specifies to read the detailed delta temperature file generated by the <code>analyze_self_heat</code> command (self-heat analysis).	
<code>-report filename</code>		
	Specifies the report file for the violation data. The report file includes name of the net, I_{rms} , interpolated <code>ACCURRENTDENSITY</code> limit (I_{limit}), layer, width, X and Y location, rise and fall time, effective frequency for the net (F_{eff}), Cnet and Vdd for each net.	
<code>-report_db</code>		
	Specifies to write the signal EM data into a database file (<code>filename.db</code>) after performing signal EM analysis. This file contains the information required for loading and displaying signal EM data in GUI. <code>filename</code> is the name specified using the <code>-report</code> parameter.	
<code>-report_unannotated_shapes {true false}</code>		

	Provides visibility into partial net coverage within SPEF files. When this parameter is set to <code>true</code>, the <code><report_name>.rpt.unannotated_shapes</code> is generated. This report identifies and lists nets that contain shapes without resistance annotations, or unanalyzed/ partially analyzed segments. The feature helps to detect incomplete parasitic extraction coverage and improve accuracy during validation and signoff workflows. The reporting of partial net coverage works only when <code>-effort_level</code> is set to <code>high</code>. Use Model: <pre>set_signal_em_analysis_mode -effort_level high -report_unannotated_shapes true</pre> Report Format: <pre>NET: <net_name> <- Specifies the name of the analyzed simulation nets with unannotated shapes LAYER (x1 y1 x2 y2) <- Layer name and shape coordinates</pre> Sample Report: <pre>NET: Net_A LAYER (x1 y1 x2 y2) M2 (30.2850 12.0800 37.6150 12.2200) M2 (30.2000 12.0650 30.2850 12.2350)</pre>  This parameter is applicable only when running in the DP mode; it is not supported in the XP mode.	
<code>-reset</code>	Resets all the specified parameters back to the default values.	
<code>-seb_lifetime value</code>		
	Specifies the life time of product in hours. <i>Default</i> : 43800 hours (that is, 5 years)	
<code>-seb_table table_filename</code>		
	Specifies the file name of the Failures in Time (FIT) table. This file is provided by the foundry. The file has the following columns: - Layer - MTF - Sigma - Activate energy - Current density exponent	
<code>-seb_temperature temperature</code>		

	Specifies the temperature used to update the median time to failure (MTF) with respect to the ambient temperature. • If this temperature is specified, it would be used globally for all the resistors. • If this temperature is not specified, the <code>seb_temperature</code> may take the following possible values depending on the specific circumstances: ◦ If detailed delta temperature file is not used for SEB, the <code>seb_temperature</code> always takes the same value as that used for EM analysis. ◦ If detailed delta temperature file is used for SEB, the <code>seb_temperature</code> is calculated by the following equation: $\text{seb_temperature} = \text{env_temperature} + \text{delta_T}$ Where <code>delta_T</code> is the maximum temperature rise, including joule heat and coupling heat from FEOL, in the resistor's bounding box with zero extension.	
-set_current_file <i>filename</i>		
	Specifies the current of a macro's output pins for EM calculation, in case the macro does not have the respective timing library. When the macro output pin current is set by the current file, the value calculated (if any) from AAE is overwritten. The format of the current file is: <pre>{CELL <i>cell_name</i> INSTANCE <i>instance_name</i>} PIN <i>pin_name</i> RMS <i>rms_current</i> PEAK <i>peak_current</i> AVG <i>avg_current</i> Td <value> NetFreq <value></pre> The unit of <code>Td</code> is Nanosecond and <code>NetFreq</code> is GHz. Following are few examples of the current file format: <pre>CELL cell_macro_A PIN Q1 RMS 0.01 PEAK 0.1 AVG 0.01 CELL cell_macro_B PIN Q1 RMS 0.01 PEAK 0.1 AVG 0.01 INSTANCE inst_macro_1 PIN Q2 RMS 0.01 PEAK 0.1 AVG 0.01 INSTANCE inst_macro_2 PIN Q2 RMS 0.01 PEAK 0.1 AVG 0.01</pre>	
-simulation_net <i>list_of_nets</i>		
	Forces the specified nets to use the simulation mode.	
-simulation_net_file <i>filename</i>		
	Forces the nets in the specified file to use the simulation mode.	
-skip_category_mode {0 1 2}		

Specifies to skip nets that belong to categories mapping to the specified mode. This parameter allows you to specify three modes, namely 0, 1 (default), and 2.

The following table lists the common categories for all modes, and the categories specific to each mode:

Common Categories	0 Skip Category Mode	1 Skip Category Mode	2 Skip Category Mode
• Non top scope nets • Internal nets on boundary path • P/G nets • Constant nets • User-defined skipped nets • Nets below effective frequency threshold • Physically broken nets • Nets without loading capacitance annotation • Nets with SPEF back-annotation failures during simulation • Nets failed in analysis	• Slave nets by Verilog Assign • Nets (non-constant) without frequency annotation • Nets without physical shape for output pin • Nets without wire shape connection to output pin • Nets with zero pin current	• Equivalent nets by Verilog Assign • Nets without driver • Nets without receiver	• Nets without pins • Equivalent nets by Verilog Assign • Nets without driver • Nets without receiver

For example, if you set `set_signal_em_analysis_mode -skip_category_mode` to 1, the report file lists all categories listed in the *Common Categories* and *1 Skip Category Mode* columns.

Following is a snippet of the report file for mode 1:

```

Total Nets in Design:          33451
Total Selected Nets in Design:    4
Total Analyzed Nets:            0
Total Skipped Nets:             4
  Non top scope nets:                0
  Internal nets on boundary path:    0
  Equivalent nets by Verilog Assign:  0
  P/G nets:                        0
  Constant nets:                    2
  User-defined skipped nets:        0
  Nets below effective frequency threshold:  0
  Nets without driver:              1
  Nets without receiver:            1
  Physically broken nets:           0
  Nets without loading capacitance annotation:  0
  Nets with SPEF back-annotation failures during simulation:  0
  Nets failed in analysis:          0

```

`-skip_net net_name_list`

Specifies the net to be skipped in the signalEM analysis.

Refer to the example below:

```
{net_a net_b net_c}
```

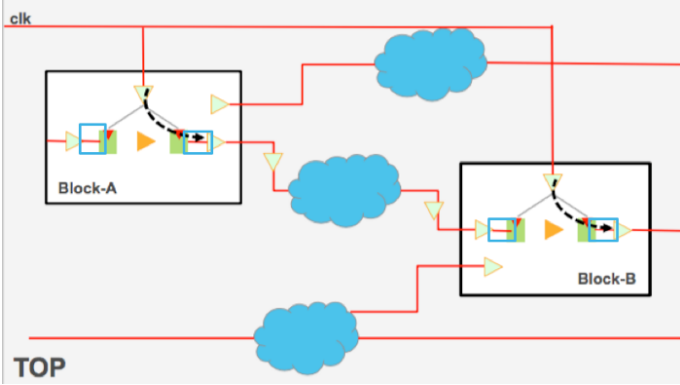
`-skip_net_file list_file`

Specifies the net file to be skipped in the signalEM analysis.

The format is one net name per line, with comments using # as the first char. For example:

```
#list of nets to skip
net_a
net_b
net_c
```

`-toggle value`

	Specifies the switching factor for signal nets. A value of 1.0 means that if a flip-flop is clocked by a 20 Mhz clock, it changes state on 100% of the clock cycles. Therefore, the signal has 20 million transitions, which is half the number of transitions of the input clock. As a result, the final switching frequency of the signal net is 10 Mhz. <i>Default:</i> 1.0 <i>Value Range:</i> 0.0 to 1.0	
<code>-top_scope_ignore_block_internal_nets_on_boundary_path {true false}</code>		
	Ignores the internal nets from the first instance to the first flop on the boundary path for the signal EM checking. The following figure represents that when this parameter is used, the internal nets (highlighted in the blue rectangles) are ignored. However, all the nets highlighted through the red lines are reported in the TOP scope (create_top_scope) flow: 	
<code>-useQrcTech</code>	Forces I_{rms} checks to use the QRC tech file instead of the technology LEF file. If either I_{peak} or I_{avg} is also checked, all checks will use the QRC tech file, including I_{rms}. This parameter is enabled by default. Note: The <code>-method avg</code> and <code>-method peak</code> analysis will only support AC current density rules from QRC tech file. If either <code>avg</code> or <code>peak</code> is also specified, then all checks will use the QRC tech file equations, including RMS.	
<code>-use_rms_delta_t {true false}</code>		
	When this parameter is set to <code>true</code>, the <code>delta_T</code> used to calculate <code>seb_temperature</code> would be based on the RMS delta T (joule heat only). If this parameter is set to <code>false</code>, the <code>delta_T</code> used to calculate <code>seb_temperature</code> would be based on the sum of self-heat delta T and RMS delta T. <i>Default:</i> false	
<code>-view viewName</code>		
	Specifies the name of the view.	

Example

- The following command specifies to calculate current waveform using the rms, avg, and peak methods, and save violations in a report named sigem.rpt

```
set_signal_em_analysis_mode \  
  
    -method {rms avg peak} \  
    -detailed true \  
    -report sigem.rpt \  
    -avgRecovery 0 \  
    -useQrcTech true \  
    -ict_em_models ../ictfile.ict \  
    -em_threshold 0 \  
  
verify_AC_limit
```

- The following is an example of the -extraction_tech_file parameter:

```
set_signal_em_analysis_mode -method {avg rms peak} -detailed \  
-ict_em_models ../EMdata.ict \  
-extraction_tech_file qrc_file -em_temperature 105 \  
-avgRecovery 0.5 -report AC_signalEM.tcf.rpt -error 100000000
```

- The following command specifies different temperature values to the layers M1, M2, M3, VIA1, VIA2, and VIA3 for average current limit analysis:

```
set_signal_em_analysis_mode  
...  
  
-em_temperature 110 \ #temperature for the remaining layers  
-em_temperature_layer_list {M1 125 M2 115 M3 105 VIA1 115 VIA2 105 VIA3 120} #layer-based temperature
```

The log file will display the following info message:

Layer specific EM temperature will be used for AVG check for layer M1(125C), M2(115C), M3(105C), VIA1(115C), VIA2(105C), VIA3(120C).

verify_AC_limit

verify_AC_limit
[-help]

Checks for the following types of AC current violations on signal nets:

- Root-mean-square (RMS) current limit violations (I_{rms} violations)
- Peak current limit violations (I_{peak} violations)

In addition, verify_AC_limit can be used for average current limit (I_{avg}) analysis.

AC current limit violations are sometimes also referred as wire self-heat violations. DRC manuals have these current limits to avoid over-heating a signal-net wire with too much AC current. To prevent wire self-heating or AC signal electromigration, signal interconnects should be analyzed for their AC current carrying capacity and measured against the AC current limits specified by foundry.

Parameters

- help	Outputs a brief description that includes type and default information for each verify_AC_limit parameter. For a detailed description of the command and all of its parameters, use the man command: man verify_AC_limit
-----------	--

Self-Heating Effect Analysis Commands

- [analyze_self_heat](#)
- [analyze_joule_heat](#)
- [set_self_heat_analysis_mode](#)

analyze_self_heat

```
analyze_self_heat  
[-help]  
[-domain rail_domain_name]  
[-output_directory dir_name]  
[-parameter_file filename]
```

Runs Self-Heating Effect (SHE) analysis on a single die or multiple dies. This command must be specified after defining the SHE properties using the `set_self_heat_analysis_mode` command.

The `analyze_self_heat` command will use the static power data to compute delta T FEOL for all instances in the design. This command:

- computes the RMS currents on signal wires
- computes the RMS currents on power network wires
- generates Detail Delta Temperature File (ddt), Instance Delta Temperature File (idt), and Tile Delta Temperature File (tdt)

 Self-heat analysis (`analyze_self_heat`) should be performed for the entire design, that is, you must use domain analysis with all PG nets (`-domain ALL`).

Parameters

- help	Outputs a brief description that includes the type and default information for each <code>analyze_self_heat</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command <code>man analyze_self_heat</code>.
<code>-domain rail_domain_name</code>	
	Specifies the domain name of the rail.
<code>-output_directory dir_name</code>	
	Specifies the directory name for storing the self-heat analysis results. When you specify this parameter, the following result files are saved in the specified directory: • Rail analysis output directory • Detail delta temperature file • Instance delta temperature file • Tile delta temperature file If <code>-output_directory</code> is not specified, by default the self-heat analysis results are stored in the current working directory.
<code>-parameter_file filename</code>	

Specifies the self-heating effect (SHE) analysis parameter file name (param.sh) required for SHE calculations. This file, provided by the foundry, contains essential alpha and beta coupling factors that the tool uses for coupledT calculations when modeling heat transfer from devices to metal layers.

Sample File:

```

HEADER {
FILE := Thermal_Coupling.ircx
}

BETA_C1_OD := 0.0055
BETA_C2_OD := -0.00065
BETA_C3_OD := 0.85000

* SELFHEAT_TABLE 17 5 (LAYER:V, ALPHA_CONNECT_OD:V, ALPHA_OVERLAP_OD:V,
ALPHA_HIGHR:V, BETA_HIGHR:V)
FIELD          ALPHA_CONNECT_OD    ALPHA_OVERLAP_OD    ALPHA_HIGHR
BETA_HIGHR
LAYER
AP              0.90                0.30                0.35
0.85
M15             .90                 0.30                0.35
0.85
M14             0.90                0.30                0.35
0.85

```

Examples

- The following command is an example of `analyze_self_heat` command in the multi-die analysis flow:

```
set_self_heat_analysis_mode \  
-die_instance_name die0 \  
-alpha_parameters {{M1 0.74 0.85} {M2 0.62 0.85} {M3 0.53 0.85} ....} \  
-beta_parameters {0.00423 -0.00045 0.81534}\  
-instance_power_file ./dynamic_power.rpt \  
-cell_thermal_resistance_file TRF.txt \  
-instance_delta_temperature_file sh_itm.txt \  
-tile_delta_temperature_file sh_ttm.txt \  
-tiles {10 10} \  
-detail_delta_temperature_file power_detail_die0.rpt  
  
set_self_heat_analysis_mode \  
-die_instance_name die1 \  
-alpha_parameters {{M1 0.74 0.85} {M2 0.62 0.85} {M3 0.53 0.85} ...} \  
-beta_parameters {0.00423 -0.00045 0.81534}\  
-instance_power_file ./dynamic_power.rpt \  
-cell_thermal_resistance_file TRF.txt \  
-instance_delta_temperature_file sh_itm.txt \  
-tile_delta_temperature_file sh_ttm.txt \  
-tiles {10 10} \  
-detail_delta_temperature_file power_detail_die1.rpt  
  
set_self_heat_analysis_mode \  
-die_instance_name die2 \  
-alpha_parameters {{M1 0.74 0.85} {M2 0.62 0.85} {M3 0.53 0.85} ...} \  
-beta_parameters {0.00423 -0.00045 0.81534}\  
-instance_power_file ./dynamic_power.rpt \  
-cell_thermal_resistance_file TRF.txt \  
-instance_delta_temperature_file sh_itm.txt \  
-tile_delta_temperature_file sh_ttm.txt \  
-tiles {10 10} \  
-detail_delta_temperature_file power_detail_die2.rpt  
  
analyze_self_heat -domain core
```

Related Topics

- "Self-Heating Effect Analysis" in the *Voltus User Guide*

analyze_joule_heat

```
analyze_joule_heat
[-help]
[-detail_delta_temperature_file filename]
[-detail_delta_temperature_region {x1 y1 x2 y2}]
[-domain rail_domain_name]
[-net pg_net_name]
[-output_directory dir_name]
[-report_conn_pin_wire filename]
[-scale_rms_limit value]
[-tile_delta_temperature_file filename]
[-tiles {n_tile_in_x m_tile_in_y}]
```

Specifies to perform joule heat analysis. This command will use the dynamic RMS current to compute delta-T RMS on the power and signal wires for all segments in the design. The `analyze_joule_heat` command is similar to the `analyze_self_heat` command, except that the `analyze_self_heat` command computes both delta-T FEOL (FEOL self heat) and delta-T RMS (BEOL self heat) whereas the `analyze_joule_heat` command computes only delta-T RMS.

 Joule heat analysis (`analyze_joule_heat`) should be performed for the entire design, that is, you must use domain analysis with all PG nets (`-domain ALL`). The `-net <pgnet>` parameter should be used only for preliminary checking.

Parameters

- help	Outputs a brief description that includes the type and default information for each <code>analyze_joule_heat</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command <code>man analyze_joule_heat</code> .
-detail_delta_temperature_file filename	
	Specifies the name of the generated BEOL detailed delta-temperature file.
-detail_delta_temperature_region {x1 y1 x2 y2}	

	Specifies the region coordinates for generating detailed delta temperature file.
<code>-domain rail_domain_name</code>	
	Specifies the domain name of the rail.
<code>-net pg_net_name</code>	
	Specifies a net for joule heat analysis.
<code>-output_directory dir_name</code>	
	Specifies the directory name for storing the joule heat analysis results. When you specify this parameter, the following result files are saved in the specified directory: • Rail analysis output directory • Detail delta temperature file If <code>-output_directory</code> is not specified, by default the joule heat analysis results are stored in the current working directory.
<code>-report_conn_pin_wire filename</code>	
	Reports results on wires connecting to cell pins.
<code>-scale_rms_limit value</code>	
	Specifies the scale factor for the root-mean-square (rms) limit relaxation factor during Electromigration current limit analysis.
<code>-tile_delta_temperature_file filename</code>	
	Specifies the name of the generated BEOL tile-based delta-temperature file.
<code>-tiles {n_tile_in_x m_tile_in_y}</code>	
	Specifies the number of tiles in the x and y directions in the tile-based delta temperature file.

Examples

- The following command is an example of `analyze_joule_heat` command:

```
analyze_joule_heat -domain ALL -tile_delta_temperature_file joule_ttm.txt -tiles  
{10 10} -detail_delta_temperature_file joule_heat_detail.rpt -report_conn_pin_wire  
joule_heat.rpt
```

Related Topics

- "Self-Heating Effect Analysis" in the *Voltus User Guide*

set_self_heat_analysis_mode

```
set_self_heat_analysis_mode
[-help]
[-alpha_parameters {{<layer_name> <  $\alpha_{overlapping}$  > <  $\alpha_{connecting}$  >}+}]
[-beta_parameters {C1 C2 C3}]
[-cell_thermal_resistance_file filename]
[-detail_delta_temperature_file filename]
[-detail_delta_temperature_region {x1 y1 x2 y2}]
[-die_instance_input_signal_em filename]
[-die_instance_name dieinstname]
[-instance_delta_temperature_file filename]
[-instance_power_file filename]
[-parameter_file filename]
[-report_conn_pin_wire filename]
[-scale_rms_limit value]
[-tile_delta_temperature_file filename]
[-tiles {n_tile_in_x m_tile_in_y}]
```

Specifies the self-heating related options for each die instance. The command supports both single-die and multi-die flows.

Parameters

- help	Outputs a brief description that includes the type and default information for each <code>set_self_heat_analysis_mode</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command <code>man set_self_heat_analysis_mode</code>.
-alpha_parameters {{<layer_name> < $\alpha_{overlapping}$ > < $\alpha_{connecting}$ >}+}	
	Specifies the alpha parameters for all layers. It is the metal layer effect.
-beta_parameters {C1 C2 C3}	
	Specifies the thermal coupling effect coefficient Value.
-cell_thermal_resistance_file <i>filename</i>	

	Specifies the name of the cell's equivalent thermal resistance file. The specified file contains the list of cells and their corresponding thermal resistance value. The unit of thermal resistance is in °C/W (temperature rise per unit rate of heat dissipation). The format of the file is: <cell_name> <thermal_resistance_in_C/W> To get instance deltaT, use half of the instance power multiplied by the cell thermal resistance.
	<code>-detail_delta_temperature_file filename</code>
	Specifies the name of the generated BEOL detailed delta-temperature file.
	<code>-detail_delta_temperature_region {x1 y1 x2 y2}</code>
	Specifies the region coordinates for generating detailed delta temperature file.
	<code>-die_instance_input_signal_em filename</code>
	Specifies the setup Tcl script for each die instance.
	<code>-die_instance_name dieinstname</code>
	Specifies the die instance name for multi-die analysis. This option is only available in multi-die analysis (<code>set_rail_analysis_mode -die_mode multi-die</code>).
	<code>-instance_delta_temperature_file filename</code>
	Specifies the name of the generated FEOL instance-based delta-temperature file.
	<code>-instance_power_file filename</code>
	Specifies a file for instances and their corresponding static power. It is a two column format of <instance_name> and <total_power>.
	<code>-parameter_file filename</code>

Specifies the self-heating effect (SHE) analysis parameter file name (param.sh) required for SHE calculations. This file, provided by the foundry, contains essential alpha and beta coupling factors that the tool uses for coupledT calculations when modeling heat transfer from devices to metal layers.

Sample File:

```

HEADER {
FILE := Thermal_Coupling.ircx
}

BETA_C1_OD := 0.0055
BETA_C2_OD := -0.00065
BETA_C3_OD := 0.85000

* SELFHEAT_TABLE 17 5 (LAYER:V, ALPHA_CONNECT_OD:V, ALPHA_OVERLAP_OD:V,
ALPHA_HIGHR:V, BETA_HIGHR:V)
FIELD          ALPHA_CONNECT_OD    ALPHA_OVERLAP_OD    ALPHA_HIGHR
BETA_HIGHR
LAYER
AP              0.90              0.30              0.35
0.85
M15             .90              0.30              0.35
0.85
M14             0.90              0.30              0.35
0.85

```

`-report_conn_pin_wire filename`

Reports results on wires connecting to cell pins.

`-scale_rms_limit value`

Specifies the scale factor for the root-mean-square (rms) limit relaxation factor during Electromigration current limit analysis.

`-tile_delta_temperature_file filename`

Specifies the name of the generated FEOL/BEOL tile-based delta-temperature file.

`-tiles {n_tile_in_x m_tile_in_y}`

Specifies the number of tiles in the x and y directions in the tile-based delta temperature file.

Examples

- The following command is an example of `set_self_heat_analysis_mode` command:

```
set_self_heat_analysis_mode \  
-die_instance_name die0 \  
-alpha_parameters {{M1 0.74 0.85} {M2 0.62 0.85} {M3 0.53 0.85} {M4 0.42 0.85} {M5  
0.32 0.85} {M6 0.26 0.85} {M7 0.26 0.85} {M8 0.26 0.85} {M9 0.26 0.85} {M10 0.26  
0.85} {M11 0.26 0.85} {M12 0.26 0.85}} \  
-beta_parameters {0.00423 -0.00045 0.81534}\  
-instance_power_file ./dynamic_power.rpt \  
-cell_thermal_resistance_file TRF.txt \  
-instance_delta_temperature_file sh_itm.txt \  
-tile_delta_temperature_file sh_ttm.txt \  
-tiles {10 10} \  
-detail_delta_temperature_file power_detail_die0.rpt  
  
set_self_heat_analysis_mode \  
-die_instance_name die1 \  
-alpha_parameters {{M1 0.74 0.85} {M2 0.62 0.85} {M3 0.53 0.85} {M4 0.42 0.85} {M5  
0.32 0.85} {M6 0.26 0.85} {M7 0.26 0.85} {M8 0.26 0.85} {M9 0.26 0.85} {M10 0.26  
0.85} {M11 0.26 0.85} {M12 0.26 0.85}} \  
-beta_parameters {0.00423 -0.00045 0.81534}\  
-instance_power_file ./dynamic_power.rpt \  
-cell_thermal_resistance_file TRF.txt \  
-instance_delta_temperature_file sh_itm.txt \  
-tile_delta_temperature_file sh_ttm.txt \  
-tiles {10 10} \  
-detail_delta_temperature_file power_detail_die1.rpt  
  
set_self_heat_analysis_mode \  
-die_instance_name die2 \  
-alpha_parameters {{M1 0.74 0.85} {M2 0.62 0.85} {M3 0.53 0.85} {M4 0.42 0.85} {M5  
0.32 0.85} {M6 0.26 0.85} {M7 0.26 0.85} {M8 0.26 0.85} {M9 0.26 0.85} {M10 0.26  
0.85} {M11 0.26 0.85} {M12 0.26 0.85}} \  
-beta_parameters {0.00423 -0.00045 0.81534}\  
-instance_power_file ./dynamic_power.rpt \  
-cell_thermal_resistance_file TRF.txt \  
-instance_delta_temperature_file sh_itm.txt \  
-tile_delta_temperature_file sh_ttm.txt \  
-tiles {10 10} \  
-detail_delta_temperature_file power_detail_die2.rpt
```

Related Topics

- "Self-Heating Effect Analysis" in the *Voltus User Guide*

Global Commands

- [exit](#)
- [help](#)
- [man](#)
- [set_analysis_view](#)
- [voltus](#)

exit

`exit`

Exits Voltus.

Parameters

None

help

```
help  
[-help]  
[-k keyword| -cmd commandName| -var][ [pattern]
```

Displays one of the following types of information based on the specified argument:

- Syntax of a command or global variable.
- Summary of the message text based on the specified message ID.
- List of commands based on keywords.
- List of commands based on search string.

Command Syntax

To display the syntax of a command or a variable, use the following command:

```
help commandName
```

The command syntax output is written in the log file and displayed in the Voltus console.

Message Summary

To display a summary of the message text for a particular message ID, use the following command:

```
help msg_id
```

Shows the summary of message text for the message ID specified.

Keyword Searches

To perform a keyword search, use the following syntax:

```
help -k keyword
```

Shows a list of commands with the specified keyword.

Partial String Searches

To perform a partial string search, use the following syntax:

```
help *string*
```

Shows all commands containing the specified string. The search results vary depending on whether you use wildcards (*) in the beginning, end, or middle of the string.

Parameters

<code>-help</code>	Prints the command usage.
<code>-cmd</code> <i>commandName</i>	Displays help for the specified command name. If you do not specify a command name, all officially supported Voltus commands are displayed.
<code>-k</code> <i>keyword</i>	Searches for the specified keyword and displays all commands associated with that keyword.
<i>msg_id</i>	Specifies command pattern, message id or keyword.
<i>pattern</i>	Instructs the Voltus software to display all commands that contain the specified series of characters. The search results vary depending on whether you use wildcards (*) in the beginning, end, or middle of the string.
<code>-var</code>	Performs variable search only.

Command Order

You can use this command after starting the Voltus software.

Example

- The following command prints the command syntax for the `set_power_analysis_mode` command:

```
help set_power_analysis_mode
```

- The following command shows the message summary for the `ENCSYC-139` message ID:

```
help ENCSYC-139
```

- The following command displays all commands that include "analysis" as a keyword:

```
help -k analysis
```

- The following commands show how wildcards can be used in string searches:
Use wildcards at the beginning of the string to get all commands that end with the specified

string:

```
voltus 18> help *power
```

Multiple commands found:

```
report_power  
set_power
```

Use wildcards in the middle of the string when you are not sure what comes in the middle of the string:

```
voltus 22> help set*mode
```

Multiple commands found:

```
set_analysis_mode  
set_delay_cal_mode  
set_eco_mode  
set_ilm_mode  
set_import_mode  
set_mode  
set_noise_run_mode  
set_power_analysis_mode  
set_pg_library_mode  
set_rail_analysis_mode
```

Use wildcards at the end of the string to get all commands that begin with the specified string:

```
voltus 23> help add*
```

Multiple commands found:

```
add_inst  
add_net  
add_receiver_stage_spice  
add_repeater  
add_thermal_layer  
add_thermal_package_model  
add_to_collection
```

Use wildcards at the beginning and end of the string to get a list of all the commands that contain the string:

```
voltus 24> help *rail*
```

Multiple commands found:

```
analyze_rail  
set_advanced_rail_options  
set_dynamic_rail_simulation
```

set_rail_analysis_domain
set_rail_analysis_mode

man

man
command_name
msg_id

Displays information for a command from the *Voltus Text Command Reference*. If no manual page is located, man prints an error message.

The *Voltus Text Command Reference* online help directories are located in the following directory:

<install_dir>/<platform>/share/voltus/man

You can access *Voltus Text Command Reference* and other *Voltus* documentation by clicking on *Help* menu option in the main *Voltus* window. You can also access documentation by typing the following UNIX command at a UNIX prompt:

install_dir/tools/bin/cdnshelp &

Both actions open the *cdnshelp: Library* window. From this window you can open or search for topics in the *Voltus Text Command Reference*, the *Voltus User Guide*, and other relevant Cadence documentation.

Parameter

<i>command_name</i>	Indicates the name of the text command for which the manual information is required.
<i>msg_id</i>	Specifies a message ID in the format like 'ABC-123'. The output will show the log file message and any extended help description that can include likely causes, possible fixes, and possible workarounds for this msg_id.

Example

- The following command returns the manual description of the command

```
set_rail_analysis_mode:  
man set_rail_analysis_mode
```


set_analysis_view

```
set_analysis_view
[-help]
[-dynamic string]
[-leakage string]
[-drv string]
-setup {defaultSetupView setupView2 ...}
-hold {defaultHoldView holdView2 ...}
[-update_timing]
```

Defines the analysis views to use for setup and hold analysis and optimization. These "active" views represent the different design variations that will be analyzed. Active views can be changed throughout the flow to utilize different subsets of views. Voltus/Tempus can handle the views concurrently or sequentially, depending on their individual capabilities. Libraries and data are loaded into the system, as required to support the selected set of active views.

You must define at least one setup and one hold analysis view.

Note: The order in which you specify views with the `-setup` and `-hold` parameters is important. By default, the first views defined in the `-setup` and `-hold` lists are the default views. If you use the `set_analysis_view` command to change the default views, the existing timing, delay calculation, and RC data is reset.

Parameters

-help	Prints out the command usage.
-drv <i>string</i>	

	Selects specific views for DRV analysis, e.g., max_transition and max_capacitance checking. When this parameter is specified, additional analysis views are specified for DRV analysis, so the timing system efficiently computes the required data for these checks. This methodology is efficient with respect to memory and performance, because all the views as full Setup views are not included. The following guidelines should be followed when selecting DRV views: • Any setup views that require DRV analysis should not be specified as DRV views. The setup views are automatically considered for DRV. • If a subset of hold-only views require DRV analysis, they should be specified as DRV views as well. The timing system will augment the timing information normally computed for Hold views with additional data required for the DRV analysis. • Additional views that are not active setup or hold views can be specified as DRV only views. The system will efficiently compute the required data for performing accurate DRV checks.
<code>-dynamic <i>string</i></code>	
	Sets a view as dynamic view. Delay calculation is required to provide transition times. Delay calculation should not perform SI analysis if the delay corner is only required for the dynamic power. The timer is not required to propagate required times or perform slack computations.
<code>-leakage <i>string</i></code>	
	Sets a view as leakage view. The timing libraries must be loaded, if not already in use. The Timer constant propagation is required, whereas delay calculation is not required.
<code>-hold {defaultHoldView holdView2 ...}</code>	
	Sets the active views for hold analysis.
<code>-setup {defaultSetupView setupView2 ...}</code>	
	Sets the active views for setup analysis.

`-update_timing`

Reloads the configuration described by the MMMC view and mode objects. When the timing global `timing_defer_mmmc_object_updates` is set to `true`, this parameter is required to update the MMMC timing environment.

The `-update_timing` parameter may be specified without any `-setup` or `-hold` arguments. In this case, the data associated with the current set of setup and/or hold views will be loaded. If no active setup or hold views exist, then using this parameter alone may display an error.

The `-update_timing` parameter may also be used in conjunction with the `-setup` and `-hold` options. In this case, the specified setup/hold view configuration will be loaded with the updated MMMC object information.

Examples

- The following command sets `missionWCCOM` and `mission2WCCOM` as the active views for setup analysis, and `missionBCCOM` and `testBCCOM` as the active views for hold analysis:

```
set_analysis_view -setup {missionWCCOM mission2WCCOM} \
```

```
-hold {missionBCCOM testBCCOM}
```

voltus

```
voltus
[-abort_on_error]
[-batch]
[-cds_lib_file in_file]
[-common_ui]
[-cpus integer]
[-disable_user_startup]
[-execute list_of_Tcl_commands]
[-files in_file_list]
[-help]
[-lic_multi_cpu lic_list]
[-lic_options lic_list]
[-lic_startup lic_list]
[-lic_startup_options lic_list]
[-lic_startup_extra lic_list]
[-log prefix]
[-no_logv]
[-no_gui]
[-overwrite]
[-version]
[-wait minutes]
```

Launches a Voltus session and starts one of the following products:

- Voltus IC Power Integrity Solution L
- Voltus IC Power Integrity Solution XL

To start a Voltus session, type the `voltus` command with the appropriate parameters on the UNIX/Linux command line. If you type a command without parameters, the software starts in the GUI mode and creates a log file and a command file. The system attempts to check out the license with the most functionality, then the license with the next most functionality, and so on.

Parameters

<code>-help</code>	Prints out the command usage. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <code>man voltus</code>
<code>-abort_on_error</code>	Exits if an error is detected at any point. If any Tcl command returns <code>TCL_ERROR</code> to the top level, or any <code>Error (XXX) : ...</code> message occurs, the process will exit.
<code>-batch</code>	Runs without interactive prompt. After all the options, including <code>-execute</code> and <code>-file</code> , are processed, the job exits.
<code>- cds_lib_file in_file</code>	Loads the specified file instead of the default <code>cds.lib</code> file in the current directory to obtain the names and locations of the OpenAccess libraries. The <code>cds.lib</code> file is used to list the OpenAccess library names, and their physical locations.
<code>-common_ui</code>	Launches the Common UI interface. This changes the Tcl commands to the Common UI interface described in the Voltus Common UI Text Command Reference Manual manual.
<code>-cpus integer</code>	Specifies the number of CPUs to be used for multi-threading on current host. This calls the appropriate Tcl command to set the local CPU count. This happens before any <code>-execute</code> or <code>-file</code> commands are processed. This may require extra multi-CPU licenses to be checked out (see <code>-lic_multi_cpu</code>).
<code>- disable_user_startup</code>	Reads only the initialization file in the installation hierarchy. Does not read the initialization files in the home or current directory. This option suppresses the default loading from the home and current directory, which can be used to guarantee there are no local over-rides to the various project settings. For more information, see "Setting Preferences" in the "Getting Started" chapter of the <i>Voltus User Guide</i> for init files that are loaded by default.
<code>-execute list_of_Tcl_commands</code>	Specifies the Tcl commands to be executed as a quoted string at startup.
<code>-files in_file_list</code>	Specifies one or more Tcl files to be sourced at startup.

<pre>-lic_multi_cpu lic_list</pre>	Controls which licenses are to be used for higher multi-CPU usage in priority order from left to right. The allowed license names are: • <code>vtcmp</code>: Voltus IC Power Integrity Solution MP • <code>vtcxl</code>: Voltus IC Power Integrity Solution XL
<pre>-lic_options lic_list</pre>	Specifies the order that extra option licenses should be checked-out if needed. In some cases more than one option license can be used to enable a given feature. You can control which options are allowed to be checked-out, and in what priority (from left to right) when required during the run. The allowed license names are: • <code>vtcaa</code>: Voltus IC Power Integrity Solution Advanced Analysis GXL Option • <code>vtcesd</code>: Voltus IC Power Integrity Solution - ESD (Electrostatic Discharge Analysis) Option • <code>vtcxm</code>: Voltus IC Power Integrity Solution - XM (Extreme Modeling) Option • <code>tpcxl</code>: Tempus Timing Signoff Solution XL • <code>tpcl</code>: Tempus Timing Signoff Solution L • <code>tpctso</code>: Tempus Timing Signoff Solution TSO • <code>tpcpi</code>: Tempus Timing Signoff Solution - PI (Power Integrity) Option • <code>invs_ms</code>: Innovus Mixed Signal Option • <code>vtsgaabsp</code>: Voltus GAA and Back Side Power Option • <code>vtc_vt</code>: Voltus InsightAI Technology Option Option licenses can be checked-out dynamically when a specific Tcl command requires it. This may cause the Tcl script to stall and wait for a license, or to exit if the wait period expires. If you specify more than one product option, begin and end the list with double quotation marks or braces. For example, <code>-lic_options "vtcaa vtcesd"</code> will only allow these two options to be checked-out dynamically, and in the given priority order. Note: For information, see "Product and Licensing Information".

<pre>-lic_startup lic_list</pre>	Requests one of these licenses at startup in priority order from left to right. The supported licenses are: <code>vtssl</code>: Voltus IC Power Integrity Solution XL <code>vtsl</code>: Voltus IC Power Integrity Solution L <code>vdsp</code>: Virtuoso Digital Signoff Power Solution For example, <code>-lic_startup "vtssl vtsl"</code> will try to check-out <code>vtssl</code> first and then <code>vtsl</code>. The check-outs will be attempted repeatedly until it succeeds or the specified wait (<code>-wait</code>) period expires.
<pre>-lic_startup_options lic_list</pre>	Specifies additional option licenses needed at startup. If not provided, they will be checked out when needed during the run. The allowed license names are: • <code>vt saa</code>: Voltus IC Power Integrity Solution Advanced Analysis GXL Option • <code>vt sesd</code>: Voltus IC Power Integrity Solution - ESD (Electrostatic Discharge Analysis) Option • <code>vt sxm</code>: Voltus IC Power Integrity Solution - XM (Extreme Modeling) Option • <code>tp sxl</code>: Tempus Timing Signoff Solution XL • <code>tp sl</code>: Tempus Timing Signoff Solution L • <code>tp tso</code>: Tempus Timing Signoff Solution TSO • <code>tp spi</code>: Tempus Timing Signoff Solution - PI (Power Integrity) Option • <code>inv s_ms</code>: Innovus Mixed Signal Option • <code>vt sgaabsp</code>: Voltus GAA and Back Side Power Option • <code>vt s_vi</code>: Voltus InsightAI Technology Option For example, <code>-lic_startup_options "vt saa, vt sesd"</code> will try to check-out both <code>vt saa</code> and <code>vt sesd</code> when Voltus starts up. The check-outs will be attempted repeatedly until it succeeds or the <code>-wait</code> period expires.

`-lic_startup_extra`
`lic_list`

Checks out the specified Innovus license to enable saving of physical changes. By default, Voltus cannot save physical changes to the database, but if you check out the correct extra Innovus license at startup, you can save the physical changes. The supported licenses are:

- `invs` - Innovus Implementation System
- `invsb` - Innovus Implementation System Basic
- `vdixl` - Virtuoso® Digital Implementation XL

Note: The `vdixl` license can only be used with the Voltus base license `vdsp`.

<code>-log prefix</code>	Specifies a name for the log file. By default, the software saves log files in the run directory and increments them, for example, <code>voltus.log</code>, <code>voltus.log1</code>, <code>voltus .log2</code>, <code>voltus .log3</code>, and so on. Default: <code>voltus.log</code> Specifies the prefix for the <code>.log</code>, <code>.cmd</code>, and <code>.logv</code> files. If three arguments are given, such as <code>-log "a b c"</code>, then the three names are used exactly without any extension added. So, <code>-log "mylog mycmd mylogv"</code> will result in <code>mylog mycmd mylogv</code> files. The default for <i>prefix</i> is <code>voltus</code>. So if <code>-log</code> is not specified, <code>voltus.log</code>, <code>voltus.cmd</code>, and <code>voltus.logv</code> files will be created. If you specify <code>-log test</code>, the tool creates <code>test.log</code>, <code>test.cmd</code> and <code>test.logv</code> files. The <code>.log</code> file contains the normal logging output, the <code>.cmd</code> file contains the Tcl commands that were executed, and the <code>.logv</code> file contains a super-set of the <code>.log</code> output plus "verbose" debugging output that is needed only for detailed debugging. If the <code>.log</code> file already exists, an integer is added to the end of all the files to make them unique. This means if <code>-log test</code> is specified, the tool will create <code>test.log test.cmd test.logv</code> for the first run, <code>test.log1 test.cmd1 test.logv1</code> for the next run, and so on. Only the existence of <code>.log</code> is checked, the existence of the <code>.cmd</code> or <code>.logv</code> files is not checked. The <code>-overwrite</code> option can be used to disable this behavior and overwrite an existing <code>.log</code> file. If the prefix has a period in it, the last extension is stripped off for the <code>.cmd</code> and <code>.logv</code> usage. For example, <code>-log out.log</code> will result in <code>out.log out.cmd out.logv</code> files, and <code>-log out.a.log</code> will result in <code>out.a.log out.a.cmd out.a.logv</code> files. You can disable any given file by using <code>/dev/null</code>. For example, <code>-log "my.log /dev/null /dev/null"</code> will only create <code>my.log</code>. Default: <code>voltus.log</code> Note: If you provide the log file name under a directory path and the directory path does not exist, Voltus will create the directory path and save the log file under the directory.
---------------------------------	--

<code>-no_gui</code>	Disables GUI and Tk support. GUI will not be available later in the session.
<code>-no_logv</code>	Specifies that verbose log file will not be created.
<code>-overwrite</code>	Overwrites an existing .log file (and associated .cmd and .logv files) instead of adding an integer to make the files unique.
<code>-version</code>	Returns the program version information.
<code>-wait <i>minutes</i></code>	Specifies the number of minutes the system waits for a license to become available before exiting. <i>Default:</i> 0 (no wait time) <i>Value Range:</i> 0 to 10,000

Setting Up the Run-Time Environment

If `install_dir` is the location of your Voltus installation, you should set up your run-time environment like this:

- Add the `install_dir/bin` directory to your path. The `bin` directory has links to all the public executables in the install hierarchy.
- If you want the legacy Voltus man pages to be available from the Unix `man` command, you can add `install_dir/share/voltus/man` to your `MANPATH` envvar.
- If you want the Tcl man pages to be available from the Unix `man` command, you can add `install_dir/share/tcltools/man` to your `MANPATH` envvar.

For example, you might add this to your startup shell script:

```
set install_dir = /tools/voltus16.2/lnx86
set path = ($install_dir/bin $path)
setenv MANPATH $install_dir/share/voltus/man:$install_dir/share/tcltools/man:$MANPATH
```

Note: When Voltus launches, it automatically adds either the legacy or the Common UI man pages (depending on the `-common_ui` option) and the Tcl man pages to the beginning of the current `MANPATH` inside Voltus. Therefore, from within Voltus, the `man` command will see both sets of man pages before any other man pages.

Temporary File Locations

Each Voltus session creates its own temporary directory to store temporary files at the beginning of the run.

By default, `tmp_dir` is created in `/tmp`. If the Unix envvar `TMPDIR` is set, then the `tmp_dir` directory is created inside `$TMPDIR`.

The name of the `tmp_dir` directory will look like:

```
voltus_temp_[pid]_[hostname]_[user]_xxxxxx
```

Where `_xxxxxx` is a string added to make the directory unique. For example:

```
voltus_temp_10233_farm254_bob_nfp9ez
```

The temporary directory is automatically removed on exit or if the run terminated with a catchable signal (e.g. `SIGSEGV`).

See also:

- [Product and Licensing Information](#)
- [Getting Started](#)

General Commands

- [get_metric](#)
- [save_testcase](#)
- [set_delay_cal_mode](#)
- [set_message](#)
- [set_message_limit](#)
- [source](#)
- [write_flow_template](#)

get_metric

```
get_metric  
[-help]  
[-branch list_of_patterns]  
[-exclude_inherited]  
[-id id_name]  
[-names label_name]  
pattern_name  
[-pending]  
[-raw]
```

Returns predefined metrics directly after the associated command is run without any need for log parsing. Metrics can be written to a log or printed as TCL output. The metrics are automatically saved/restored with design data (*designName.metric*). Metrics can accumulate across multiple sessions and keep being appended to the `.metric` file.

By default, the `get_metric` command returns all metrics last computed from the commands generating the metrics. Currently, `get_metric` supports the following command: `report_power`, `analyze_rail`, `verify_AC_limit`, and `report_freq_violation` commands. The `get_metric` command is applicable only for static power analysis.

Parameters

<code>-help</code>	Prints a brief description that includes type and default information for each <code>get_metric</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <code>man get_metric</code>.
<code>-branch list_of_patterns</code>	
	Specifies list of patterns to match snapshot branches. This will limit all output to metrics stored in snapshots whose branch matches the specified branch. This requires any inheritance to be re-run to exclude inherited values from non-matching snapshots.
<code>-exclude_inherited</code>	
	Specifies not to return any inherited value.
<code>-id id_name</code>	
	Name of id (if <code>read_metric</code> was used). Default is current database.
<code>-names label_name</code>	
	Name of label to get metrics from (wildcards supported); no label reports current metrics.
<code>pattern_name</code>	Returns the value of specified metric names. Note: When specifying metric names, you can use wildcards.
<code>-pending</code>	Returns metrics from the pending (not yet created) snapshot.
<code>-raw</code>	Returns metrics as raw value (without normalization to defined units).

Example

- The following command returns metrics for the `analyze_rail` command:

```
get_metric -pending
```

The final output is as follows:

```
rail.averagedemandcurrent.net:VSS 0
rail.averagesupplycurrent.net:VSS 0
rail.gridcap.net:VSS 0
rail.intrinsiccap.net:VSS 0
rail.ir.dynamic.avg.net:VSS 0.0119396
rail.ir.dynamic.max.net:VSS 0.0442943
rail.ir.dynamic.min.net:VSS 0
rail.ir.dynamic.violations.net:VSS 0
rail.loadingcap.net:VSS 0
rail.peakdemandcurrent.net:VSS 0
rail.peaksupplycurrent.net:VSS 0
rail.referencevoltage.net:VSS 0
rail.rj.avg.net:VSS NA
rail.rj.max.net:VSS NA
rail.rj.min.net:VSS NA
rail.rj.violations.net:VSS 0
rail.thresholdvoltage.net:VSS 0.05
rail.totalcap.net:VSS 0
rail.worstircycle.net:VSS {3.550 nS}
```

save_testcase

```
save_testcase
[-dir directoryName]
[-foundationFlow]
[-gzip]
[-name testcaseName]
[-merge]
[-noTimingGraph]
[-oaUsedLibraryOnly]
[-overwrite]
[-rc]
```

Saves your design as a standalone testcase that includes both the design and the library files. All database references to files in the generated testcase are self-contained, such that the database can be loaded into Voltus on a different network.

You can use this command after loading a design.

Note: Exit Voltus after using the command. This is to avoid a possibility where `save_design` may create a configuration file with a timing constraints file pointing to the testcase directory. Given that the testcase directory is considered temporary, and may be gzipped or even deleted, a pointer to its data is not desirable. Exiting Voltus and re-loading a previously saved database will avoid this possibility.

Parameters

<code>-name <i>testcaseName</i></code>	
	Specifies the name of the testcase to be created. The path to the saved testcase location is: <i>directoryName/testcaseName</i> If a previously generated testcase exists at the specified location, the older testcase will be renamed before the new testcase is saved. <i>Default:</i> testcase
<code>-dir <i>directoryName</i></code>	

	Specifies the directory in which to save the testcase. Both, absolute and relative directory paths, may be used. <i>Default:</i> Current working directory
<code>-foundationFlow</code>	Specifies to copy foundation flow files.
<code>-rc</code>	Specifies that RC extraction data should be saved. <i>Default:</i> RC parasitics files are not saved. Note: This option works with these limitations: • The <code>-rc</code> parameter does not have save RC extraction data if the design is not extracted or if you extract the data using the default extraction engine. • The previously saved RC data can only be restored on systems that share the same platform and architecture. For example, if you saved the RC data on a Linux platform you can not restore it on a IBM AIX platform. Similarly, RC data saved on a 64-bit system cannot be restored on a 32-bit system.
<code>-merge</code>	Specifies that a single script should be generated containing both shell executable and Voltus TCL commands. <i>Default:</i> Two script files are generated. A shell executable file and a Voltus TCL script.
<code>-oaUsedLibraryOnly</code>	Saves only the libraries listed in the <code>init_oa_tech_lib</code>, <code>init_oa_ref_lib</code>, and <code>init_oa_search_lib</code> open access designs, which are referenced through the on-demand processing. On specifying this parameter, a new <code>cds.lib</code> file is created with only the specified libraries. When this parameter is not used, all the open access libraries are saved in the original <code>cds.lib</code> file. Note: This parameter works only when the <code>init_design_netlisttype</code> is OA, <code>init_oa_ref_lib</code> is not null, and <code>init_lef_file</code> is null.
<code>-noTimingGraph</code>	Skips saving of timing graph by default. This saves time by preventing the timing update to occur in case of designs where no graph is generated in the session.

<code>-overwrite</code>	Specifies that existing files should be overwritten. <i>Default:</i> Existing testcase files are renamed before new testcase is saved.
<code>-gzip</code>	Specifies that the generated self contained design should be stored in a gzip compressed tar file. The generated test case directory and files are deleted after gzip compression is complete. <i>Default:</i> If you do not specify this parameter, the generated test case is not gzipped.

Example

- The following command creates a self contained design in the current working directory under the testcase directory.
`save_testcase`
- The following command creates a gzip compressed tar file `/tmp/testcase.tar.gz` containing a self contained testcase:

```
save_testcase -dir /tmp -gzip
```

set_delay_cal_mode

```
set_delay_cal_mode
[-help]
[-accuracy_level level]
[-accurate_receiver_out_load {true | false}]
[-advanced_pincap_mode {true | false}]
[-combine_mmmc {none | early_late}]
[-disable_randomcell_in_socv {true|false}]
[-early_irdrop_data_type {best_average | worst | best | average | worst_average}]
[-enable_corner_based_beol {true|false}]
[-enable_high_fanout {true | false}]
[-enable_quiet_receivers_for_hold {true | false}]
[-equivalent_waveform_model {none | no_propagation | propagation}]
[-equivalent_waveform_model_for_timing_check {true|false}]
[-ewm_type {moments | simulation}]
[-honorSlewPropConstraint {true | false}]
[-irdrop_data_type {best_average | worst | best | average | worst_average}]
[-irDrop_window_based {none | late | early | both}]
[-late_irdrop_data_type {best_average | worst | best | average | worst_average}]
[-library_interpolation_mode {linear | non_linear}]
[-ptr_effort <value>]
[-reportOutBound {true | false}]
[-reset]
[-SIAware {true | false}]
[-skip_slew_merge_from_disabled_path {true|false}]
[-slewOutBoundLimitHigh value]
[-slewOutBoundLimitLow value]
[-socv_accuracy_mode {low | medium | high | ultra}]
[-socv_lvf_mode {moments | early_late}]
[-socv_machine_learning_level {0|1|3}]
[-socv_use_lvf_tables {all | delay | slew | constraint}]
[-wire_variation_correlation_mode {statistical|binomial}]
[-wire_variation_rc_correlation <value>]
```

Sets the global parameters or delay calculation.

By default, the software uses linear interpolation for timing check delay calculation.

Parameters

<pre>-help</pre>	Outputs a brief description that includes type and default information for each <code>set_delay_cal_mode</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man set_delay_cal_mode</code> command.
<pre>-accuracy_level level</pre>	Sets the accuracy level for delay calculations. <i>Default:</i> 0, <i>Min:</i> 0, <i>Max:</i> 4
<pre>-accurate_receiver_out_load {true false}</pre>	When set to <code>true</code>, Voltus uses distributed interconnect model as receiver output load to compute accurate miller effect. In this mode, Voltus uses a reduced interconnect model for receivers instead of lumped capacitance as the receiver load. This improves computation of the back miller effect and the delay accuracy. When set to <code>false</code>, Voltus uses lumped capacitance as the receiver's load. <i>Default:</i> <code>false</code>
<pre>-advanced_pincap_mode {true false}</pre>	Simulates the current stage and receiver stages together, such that the back miller effect is modeled dynamically. When set to <code>true</code>, the receivers are no longer modeled as simple non-linear current source models. This results in increasing the delay calculation accuracy significantly as the modeling back miller effect improves. When set to <code>false</code>, the receiver current sources are computed using ECSM/CCS capacitance tables with the estimated input slew. <i>Default:</i> <code>false</code>

<pre>-combine_mmmc {none early_late}</pre>	Specifies whether delay calculation runs are to be combined for delay calculation simulations. This parameter provides user control for improving the runtime of AAE for all analysis types by combining runs for early and late simulations into a single simulation. The views with common delay corners may also be combined into a single delay calculation simulation for added runtime improvement. When <code>none</code> is specified, the software does not combine any delay calculation runs. When <code>early_late</code> is specified, the software combines early and late simulations of a single corner into one delay calculation simulation. Default: <code>early_late</code>
<pre>-early_irdrop_data_type {best_average worst best average worst_average}</pre>	
	Allows you to set the early IR drop EIV values for different EIV voltages. <i>Default:</i> <code>worst</code>
<pre>-enable_high_fanout {true false}</pre>	When this parameter is set to <code>true</code>, the default net delay will be annotated on high fanout nets. A net will be treated as a high fanout net when the fanout is more than the specified limit. The default value of this command is <code>1000</code> fanouts. The default net delay is also specified using this command. The default net delay is <code>1ns</code>. The cell delay will use the lumped capacitance that is specified using the <code>delaycal_default_net_load</code> global variable for the table lookup. The default net load is <code>0.5pF</code>. The driver slew will use the default value of <code>0ps</code> and there will be no slew degradation along the interconnect. The default value of this parameter is <code>false</code>.

<pre>-enable_quiet_receivers_for_hold {true false}</pre>	When set to <code>true</code>, Voltus forces some receivers as static (“quiet”). To compute the worst-case delays for early analysis, the software assumes that the receivers are not switching. This yields more conservative early delays. When set to <code>false</code>, the software assumes receivers may switch. <i>Default:</i> <code>false</code>
<pre>-equivalent_waveform_model {none no_propagation propagation}</pre>	Controls the equivalent waveform model to be used. You can specify one of the following options: • none: When this option is specified, the waveform shape beyond slew measurement points does not impact delays. For example, <pre>set_delay_cal_mode -equivalent_waveform_model none</pre> This mode is equivalent to setting the following options in the previous releases: <pre>set_delay_cal_mode - equivalent_waveform_model_type none set_delay_cal_mode - equivalent_waveform_model_propagation false</pre> • no_propagation: In this mode, the impact of receiver waveform shape on the next stage delay is estimated and adjusted the net delay of the current stage. For example, <pre>set_delay_cal_mode -equivalent_waveform_model no_propagation set_delay_cal_mode -ewm_type moments</pre> This mode is equivalent to setting the following options in the previous releases: <pre>set_delay_cal_mode - equivalent_waveform_model_type ecdsm</pre>

	<pre>set_delay_cal_mode - equivalent_waveform_model_propagation false</pre> propagation: When specified, the impact of input waveform shape is considered during delay calculation and the waveforms are propagated across each stage of the entire path. For example, <pre>set_delay_cal_mode -equivalent_waveform_model propagation set_delay_cal_mode -ewm_type moments</pre> This mode is equivalent to setting the following options in the previous releases: <pre>set_delay_cal_mode - equivalent_waveform_model_type ecsm set_delay_cal_mode - equivalent_waveform_model_propagation true</pre>
<pre>-equivalent_waveform_model_for_timing_check {true false}</pre>	
	Models the waveform distortion effect for timing check value look ups at both the data and clock pins. The model uses simulations to convert the distorted waveforms into equivalent waveforms (pre-driver waveforms in the libraries) by matching the pin CCB output waveforms. This feature requires the CCSN library. <i>Default:</i> false

<pre>-ewm_type {moments simulation}</pre>	Specifies the equivalent waveform model (EWM) to be used: The options are described below: • moments: Enables moment-based equivalent waveform model. • simulation: Enables simulation-based equivalent waveform model. The simulation-based EWM is required for predicting delay in noisy waveforms. The simulation option requires noise models (ECSMN/CCSN). If noise models are not available, then the software defaults to moments-based EWM. It is recommended to use simulation-based EWM as it is more accurate than moment-based EWM. Default: moments
<pre>-honorSlewPropConstraint {true false}</pre>	Determines whether to propagate the slew from the disabled timing arcs to the output pin. When this parameter is set to false, the slew from the disabled timing arc affects the delay of the next stage. When set to true, the timing constraints defined using commands, such as set_disable_timing and set_case_analysis impact the slew propagation, and the slew from the disabled timing arcs is not propagated. Default : true
<pre>-irdrop_data_type {best_average worst best average worst_average}</pre>	
	Allows you to set the early/late IR drop EIV values for different EIV voltages. <i>Default:</i> worst
<pre>-irDrop_window_based {none late early both}</pre>	
	Searches window-based EIV values from the EIV database. <i>Default:</i> both

<code>-late_irdrop_data_type {best_average worst best average worst_average}</code>	Allows you to set the late IR drop EIV values for different EIV voltages. <i>Default:</i> <code>worst</code>
<code>[-library_interpolation_mode {linear non_linear}]</code>	Controls library scaling mode - <code>linear</code> or <code>non_linear</code> - for the interpolation flow. <i>Default:</i> <code>linear</code>
<code>-ptr_effort</code>	Specifies the PTR analysis effort. This value should be between 0 and 1, where 0 - lowest accuracy and runtime, and 1 - highest accuracy and runtime. <i>Default:</i> <code>0.5</code>
<code>-reportOutBound { true false }</code>	Generates a report that contains a list of index values (input transition) in the delay tables that are beyond the index range. The index values detected by this parameter can cause inaccurate delay values to be calculated. The report generated by this parameter can be used for debugging. When this parameter is set to <code>true</code>, extrapolation information will be dumped into the following two files: • <code><design-name>.dc.outbound.slew.early.<view-name>.txt</code> • <code><design-name>.dc.outbound.slew.late.<view-name>.txt</code> The software writes a warning message in the log files noting that there are input transition values that are out of the index range of the timing table. The warning message also provides the file name for viewing the detail report. When set to <code>true</code>, the <code>-reportOutBound</code> parameter will generate reports that can be controlled using <code>set_outbound_report</code> command. <i>Default:</i> <code>false</code>

<code>-reset</code>	Resets the parameters to their default values. The <code>-reset</code> parameter must be specified before the parameters for which you want to reset the values. If you specify the <code>-reset</code> parameter by itself, the software resets all the <code>set_delay_cal_mode</code> parameters to their default values.
<code>-SIAware {true false}</code>	Enables SIAware delay calculation that also includes cross-talk induced delays. When set to <code>true</code>, this parameter enables signal integrity (SI) analysis. When set to <code>false</code>, this parameter disables SI analysis. <i>Default:</i> <code>false</code>
<code>-skip_slew_merge_from_disabled_path</code>	Avoids merging slews from unconstrained, segmented paths and black-box output pin. When set to <code>true</code>, the software does not allow merging transitions from disabled paths. <i>Default:</i> <code>false</code>
<code>-slewOutBoundLimitHigh value</code>	Limits the maximum slew used in the analysis to the specified value. <i>Default:</i> <code>1.3</code> <i>Min:</i> <code>1.000000</code> <i>Max:</i> <code>340282346638528859811704183484516925440.000000</code>
<code>-slewOutBoundLimitLow value</code>	Limits the minimum slew used in the analysis to the specified value. <i>Default:</i> <code>0.5</code> <i>Min:</i> <code>0.001000</code> <i>Max:</i> <code>1.000000</code>
<code>-socv_accuracy_mode {low medium high ultra}</code>	

	Allows the ability to control levels of accuracy. This parameter specifies level-based accuracy effort for delays and slews. You can use one of the following options: • <code>low</code>: Ignores correlation between slews and delays. This is the default setting. • <code>medium</code>: Models correlation between immediate slew and delay. • <code>high</code>: Considers skewness of non-gaussian distribution into account for computing quantiles. The skewness is local to one stage only. • <code>ultra</code>: Considers skewness of non-gaussian distribution, and also propagates skewness to the next stage in PBA analysis.
<code>-socv_lvf_mode {moments early_late}</code>	Controls interpretation of LVF data. <i>Default:</i> <code>moments</code>
<code>-socv_machine_learning_level {0 1}</code>	Enables the SOCV machine learning mode. For accurate SOCV analysis, the software creates several statistical models, which are not available with standard LVF library, such as correlation between cells delay and output slew, conversion of moments from late/early sigma tables, and effects of waveform distortion on sigma and mean shift. When the <code>-socv_machine_learning_level</code> parameter is set to 1, the software enables several machine learning algorithms to generate such models internally. <i>Default:</i> 0
<code>-socv_use_lvf_tables {all delay slew constraint}</code>	
	Controls the type of variations that are considered during analysis. <i>Default:</i> <code>all</code>

<code>-wire_variation_correlation_mode</code>	Specifies wire variation inter-layer correlation. When set to <code>statistical</code>, all the layers are independent of each other <i>Default:</i> <code>statistical</code>
<code>-wire_variation_rc_correlation</code>	Specifies same layer R and C correlation for wire variation. When set to 1, the R and C sensitivities are considered as fully correlated. <i>Default:</i> 1

Examples

- The following command enables waveform propagation along the path for equivalent waveform model:

```
set_delay_cal_mode -equivalent_waveform_model propagation
```

- The following command specifies that the software should not combine any delay calculation runs:

```
set_delay_cal_mode -combine_mmmc none
```

- If you have set the value of the `-SIAware` parameter to `true`, use the following command to reset the value of the parameter to the default value, which is `false`, to disable SI analysis during delay calculation:

```
set_delay_cal_mode -reset -SIAware
```

Now when you retrieve the value of the `parameter` using the `get_delay_cal_mode` command, the software returns the following:

```
get_delay_cal_mode -SIAware
```

```
-SIAware false # enums={true false}, default=false  
false
```

set_message

```
set_message
[-help]
[-id list_of_msgIDs]
[-severity {warn error info} | -limit <number> | -no_limit | -suppress | -unsuppress | -
on_error {msg_only exit stop_script} | -stop_script_limit limitNumber]
```

Specifies to change the message severity levels, `INFO`, `WARNING` or `ERROR`, at the beginning of the message. This command allows you to control the messages that are displayed for a particular design. For example, if there are some warnings that are critical for the design, you can change the message severity to `ERROR`. Similarly, if there are messages that you want to ignore, you can change the message severity to `INFO`.

Parameters

-help	Outputs a brief description that includes type and default information for each <code>set_message</code> parameter. For a detailed description of the command and all of its parameters, use the man command: <code>man set_message</code> .
<i>-id list_of_msgIDs</i>	
	Specifies the list of message IDs that require a change in the message severity level. If you do not specify a message ID, the change in severity level is applied to all message IDs.
<i>-limit number</i>	
	Sets the message limit for the selected messages. The limit must be greater than zero.
<i>-no_limit</i>	Removes any existing limit on the selected messages. Suppressed messages are not unsuppressed.
<i>-on_error {msg_only exit stop_script}</i>	

	Specifies an extra action for error messages (not for warn or info messages). If no message is specified, the action applies to all error messages. <code>msg_only</code>: Prints error message only, and the command proceeds as it normally would. This is the default. <code>exit</code>: Exits immediately and returns to Linux. <code>stop_script</code>: Waits for the current Tcl command to finish, then stops executing any Tcl script, and returns to the interactive Tcl prompt. Note, if a message severity is changed from error to either warn or info, it will no longer trigger this extra action.
<code>-severity {warn error info}</code>	
	Specifies the new severity level for the message ID being modified.
<code>-stop_script_limit limitNumber</code>	Sets the message limit for a public command. If you set <code>-stop_script_limit</code> for a command as 10 and message for the command is triggered for the 10th time, the tool will wait for the current Tcl command to finish, then stop executing any Tcl script, and return to the interactive Tcl prompt. The default value of <code>-stop_script_limit</code> is -1, which is unlimited.
<code>-suppress</code>	Suppresses the message.
<code>-unsuppress</code>	Unsuppresses the message.

Examples

- The following command changes the message ENCLF-81 from `ERROR` to `WARN`:
`set_message -id ENCLF-81 -severity warn`
- The following command returns all messages to their original severity:
`set_message -severity reset`
- The following command resets the specified message to its original severity:
`set_message -id ENCLF-18 -severity reset`

set_message_limit

```
set_message_limit  
[-help]  
numberToLimit  
[prefix [ID1, ID2, ...]]
```

Limits error and warning messages to a specified number per unique ID. Any occurrences beyond the limit are suppressed. The default limit is 20 per message ID. The default limit is applicable only for ERROR* and WARNING* messages; there is no default limit for verify*, check*, and INFO* messages.

Parameters

-help	Outputs a brief description that includes type and default information for each <code>set_message_limit</code> parameter. For a detailed description of the command and all of its parameters, use the man command: <code>man set_message_limit</code> .
<i>numberToLimit</i>	Specifies the maximum number of messages to be displayed for each unique ID. The value of <i>numberToLimit</i> should be greater than zero.
<i>prefix</i> [ID1, ID2...]	Specifies prefix-based message limit for specific warnings.

Example

- The following command sets a limit of 10 messages per unique ID for all error and warning messages:

```
set_message_limit 10
```

- The following command sets a limit of 2 for the `TCLCMD-917` error message.

```
set_message_limit 2 TCLCMD-917
```

After 2 occurrences of `TCLCMD-917`, which is the same as the limit set, the system announces that the limit has been reached.

```
[DEV]voltus 4> set_propagated_clock CLK1
```



```
**ERROR: (TCLCMD-917): Cannot find 'ports, pins, or clocks' that match 'CLK1'
```

```
[DEV]voltus 5> set_propagated_clock CLK1
```

```
**ERROR: (TCLCMD-917): Cannot find 'ports, pins, or clocks' that match 'CLK1'
```

```
[DEV]voltus 6> set_propagated_clock CLK1
```

```
Message <TCLCMD-917> has exceeded the message display limit of '2'.
```

```
setMessageLimit/set_message_limit sets the limit.
```

```
unsetMessageLimit/unset_message_limit can be used to reset this.
```

source

```
source  
[-help]  
filename  
[-quiet | -verbose]  
[-quiet | -echo ]
```

Reads a file and executes the commands in the file (scripts).

Parameters

-help	Describes the command usage.
-echo	Enables verbose source. If -echo is specified, the tool will ignore it and continue with the source instead of giving an error. Note: This is optional.
<i>filename</i>	Specifies the name of the file to source.
-quiet	Forces source to be normal Tcl source command. Note: This is optional.
-verbose	Forces source to be in verbose mode. Note: This is optional.

write_flow_template

```
write_flow_template
[-help]
[-directory directory_name]
[-tempus]
```

Copies the Voltus Foundation flows templates into the directory you specify, or to the current directory if you do not specify one with the `-directory` parameter.

Parameters

-help	Outputs a brief description that includes type and default information for each <code>write_flow_template</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <code>man write_flow_template</code> .
<code>-directory <i>directory_name</i></code>	
	Specifies the directory in which to save the templates.
-tempus	Writes the Tempus flow template.

Example

- The following command copies the Voltus foundation flows templates into the `voltusFF` directory:
`write_flow_template -directory voltusFF`

General Global Variables

Using General Globals

This chapter describes the general global variables that can be set to change the default behavior in Voltus. You can set general global variables as follows:

```
set_var variable_name value
```

For example:

```
set_var load_netlist_ignore_undefined_cell 1
```

load_netlist_ignore_undefined_cell

```
load_netlist_ignore_undefined_cell { 1 | 0 }
```

Creates a dummy .lib file and a user-defined noise model (UDN) for cells that are not present in the loaded .lib or cdB. The dummy .lib file is used for timing analysis, and the dummy UDN file is used for signal integrity analysis. To generate the dummy .lib and UDN, set this variable to 1.

To set this variable, use the `set_var` command.

Datatype: boolean

Default: 0

timing_library_infer_cap_range_from_ecsm_receiver_model

```
timing_library_infer_cap_range_from_ecsm_receiver_model {true | false}
```

Infers the rise or fall of NLDM pin capacitance range from the ECSM capacitance range while calculating the total loading capacitance during switching power computation. If the global is set to `true` and the capacitance range is not defined in the library, the rise/fall capacitance range is inferred from the ECSM receiver capacitance models. The capacitance range values are derived either from the pin-level ECSM cap models or the timing-level ECSM cap models.

When this global is set to `false`, the rise/fall capacitance range values are not inferred, and the library values are honored.

To set this global variable, use the `set_global` command.

Default: `false`

timing_library_read_ccs_power_data

```
timing_library_read_ccs_power_data {0 | 1}
```

Specifies to use the Composite Current Source (CCS) unified power library format suitable for power optimization, power analysis, and rail analysis. It also allows you to capture a supply current waveform for each power or ground pin.

When this global is set to `1`, the CCS Power construct from the Liberty library is loaded and used in power analysis.

To set this global variable, use the `set_global` command.

Default: `0`

enc_tcl_error_info

```
enc_tcl_error_info {true | false}
```

Type: Boolean

Default: false

When set to `true`, `enc_tcl_error_info` prints the error information if there is an issue in the Tcl script.

Legacy EPS Options

- [Extractor Options](#)
- [Library Generator Options](#)
- [Library Simulator Options](#)
- [Power Analysis Options](#)
- [Rail Analysis Options](#)

Extractor Options

This section lists the advanced extraction commands and variables that can be included in Voltus using `set_rail_analysis_mode -extractor_include file` during rail analysis, and `set_advanced_pg_library_mode -extraction_command_file file` during library characterization. They are listed in alphabetical order.

- [Extractor Commands](#)
- [black_macros](#)
- [cell black](#)
- [cell ignore](#)
- [cell_ignore_file](#)
- [cell map](#)
- [cluster distance](#)
- [density_check_rectangle](#)
- [extraction_type](#)
- [ignore_macros](#)
- [ignore_pin](#)
- [include](#)
- [include_via_res_table_file](#)
- [label net](#)
- [label port](#)
- [layer](#)

- `layer bias`
- `layer gds`
- `layer map`
- `pin ignore`
- `promote_ports`
- `Extractor Variables`
- `density_check_method`
- `dump_options`
- `enable_erosion_effects`
- `enable_metal_fill_effects`
- `enable_poly_routing`
- `enable_r_function_width`
- `enable_toplevel_diff_extract`
- `exclude_blockagewire`
- `gds_unit`
- `ignore_shorts`
- `layout_scale`
- `load_optimized_views`
- `log_file`
- `max_error_messages`
- `max_resistor_length`
- `message_detail_level`
- `metal_fill_type`
- `oa_analysis_point`
- `output_cache_directory_name`
- `output_directory_name`
- `output_drawn_geometry`
- `output_file_name`
- `output_incomplete_nets`
- `output_oa`
- `remove_dangling_resistors`

- [tech_file](#)
- [top_level_names_only](#)
- [use_def_over_library](#)
- [use_layer_bias_in_cmd_and_process_bias_in_tech](#)
- [use_manhattan_ports](#)
- [wee_type](#)

Extractor Commands

black_macros

Syntax

```
black_macros file filename
```

Description

Note: This is equivalent to the command `cell black` - which is now the preferred command.

The `black_macros` command excludes the contents of selected macros from the gray data during graybox extraction. Use this command when you want zx to ignore the effect of large megacells that reside under areas of the routing layer that contain no routing.

Arguments

<code>file</code> <i>filename</i>	Specifies the name of the file that lists the macros to consider as black boxes during graybox extraction. In the file, specify one macro per line.
--------------------------------------	---

Examples

Consider a file called `black.box` whose contents are the following. Each line is an instance of the macro to be ignored.

```
P20
```

```
P30
```

```
P60
```

P70

In the following example, zx ignores the contents of all instances of the P20, P30, P60, and P70 macros listed in the `black.box` file during graybox extraction:

```
black_macros file black.box
```

cell black

Syntax

```
cell black cell_name
```

Description

The `cell black` command is used to ignore gray data information from the named cell. Functionally equivalent to the QXC V3.x command [black_macros](#).

Arguments

<code>cell_name</code>	Specifies the name of the cell.
------------------------	---------------------------------

Examples

The following example specifies that zx is to ignore the gray data information for all instances of the `NAND2X' and `iobufx' cells.

```
cell black NAND2X
cell black iobufx
```

cell ignore

Syntax

```
cell ignore cell_name
```

Description

The `cell ignore` command is used to exclude all references to the named cell from the design database. Functionally equivalent to the QXC 3.x command [ignore_macros](#).

Arguments

<i>cell_name</i>	Specifies the name of the cell.
------------------	---------------------------------

Example

The following example specifies that zx is to exclude all references to the `AND3X' cell:

```
cell ignore AND3X
```

cell_ignore_file

Syntax

```
cell_ignore_file filename
```

Description

Excludes all references to the named cells in the specified file, from the design database

Arguments

<i>filename</i>	Specifies the filename containing the list of cells to be ignored.
-----------------	--

cell map

Syntax

```
cell map from_cell to_cell
```

Description

Used to resolve conflicts between names in the design database and names used in the library. One usage is to force design cell names to correspond to library names - this is a more common problem in GDS2 designs where uppercase cell names do not match lowercase library names. Another usage is to change a design cell that is not intended to be mapped to a library view.

Arguments

<i>from_cell</i>	Specifies the source cell name.
<i>to_cell</i>	Specifies the target cell name.

Examples

```
cell map NAND2X nand2x
```

```
cell map iobuf iobufx
```

cluster distance

Syntax

```
cluster distance layer_name distance max_dimension
```

Description

Specifies how to cluster or re-cluster via arrays. Since the clustering approach reduces the RC elements by clustering vias into one node, it can boost the IR drop analysis runtime in dynamic mode.

Arguments

<i>layer_name</i>	Specifies the name of the via layer.
<i>distance</i>	Specifies that all via geometries on the specified via layer within <i>distance</i> , in microns, of other via geometries will be clustered together into a multi-cut via.

<code>max_dimension</code>	Specifies that if the resulting cluster exceeds the <code>max_dimension</code> in either number of rows or columns of vias it will be split in half repeatedly until the maximum number of rows or columns, per cluster, is less than or equal to the <code>max_dimension</code> .
----------------------------	--

Examples

This example specifies that all via1 geometries within 1 micron distance from each other will be clustered. The maximum cluster size in either x or y direction is 4 vias.

```
cluster distance via1 1.0 4
```

density_check_rectangle

Syntax

```
density_check_rectangle x1 y1 x2 y2
```

Description

The `density_check_rectangle` command checks wire density in a specified area of the chip.

Note: This command must be used with the variable `density_check_method`, set to `rectangle`.

Arguments

<code>x1</code> <code>y1</code> <code>x2</code> <code>y2</code>	The two sets of <code>x</code> and <code>y</code> coordinates represent the leftmost, bottommost point of the area to be checked and the rightmost, topmost point, in microns.
--	--

Examples

The following example checks the wire density in a rectangular area defined by 0,0 and 500, 500:

```
setvar density_check_method rectangle
```

```
density_check_rectangle 0 0 500 500
```

extraction_type

Syntax

```
extraction_type [all|special|regular|mustjoin| file filename |pattern pattern ] [ extraction_type |
capacitance_type ]
```

Description

The `extraction_type` command controls the type of extraction performed on the nets and selects the nets on which to perform the extraction. By **default**, Fire & Ice QX extracts all regular nets as a decoupled RC network.

Arguments

all	Extracts all nets.
file <i>filename</i>	Extracts all nets listed in the <i>filename</i> file. Note: If you select this option, you should first turn off the default value by setting the <code>all</code> option to <code>none</code>, as in the following example, unless you want to extract in two different modes, such as coupled and decoupled: <pre>extraction_type all none</pre> <pre>extraction_type file gamma decoupled_rc</pre>
pattern <i>pattern</i>	Selects all nets whose names contain the pattern specified by <i>pattern</i> . <i>pattern</i> uses the DEF regular expression syntax, which follows:
*	Matches any sequence of 0 or more characters. For example, <code>*net*</code> matches <code>123netxyz</code>, <code>netvdd</code>, and <code>10ce463pnet</code>.
%	Matches any single character. For example, <code>bus%</code> matches <code>bus1</code>, <code>bus2</code>, <code>bus3</code>, and so forth. Similarly, <code>clock%test</code> matches <code>clock7test</code>, <code>clock8test</code>, and <code>clock9test</code>, but not <code>clock10test</code>.
-	Matches the whole sequence of characters up to and including the next period ("."). If there is no "." in the string, it matches to the end of the string. For example, <code>--bus3</code> matches <code>bus3</code> in the following expression: <code>block1.cell12.bus3</code>

<code>extraction_type</code>	Specifies the type of RC extraction. The type of RC extraction can be one of the following:
------------------------------	---

<code>r_only</code>	Extracts only the resistance.
<code>decoupled_rc</code>	Extracts the decoupled resistance and capacitance network. This option is the default .
<code>none</code>	Performs no extraction on the defined set of nets.

<code>capacitance_type</code>	Specifies the type of capacitance extracted. The type of capacitance extraction can be one of the following:
-------------------------------	--

<code>total_c</code>	Extracts the total capacitance.
<code>none</code>	Extracts no capacitance.

Note: There is no limit to the number of `extraction_type` commands that you can define in a single command file, so you can use them repeatedly to fully describe the nets to extract and completely control the extraction. Later specifications override earlier ones if they select the same nets.

Note: By **default**, zx extracts all regular nets as a decoupled RC network. Reduction of the output is controlled by the `reduce_output` variable.

Note: zx detects power and ground nets from the DEF file by their `+USE POWER` and `+USE GROUND` attributes. It excludes these nets from the extraction unless you explicitly request that they be extracted by using their name or a matching pattern with the `extraction_type` command.

Examples

Example 1

The following example performs resistance-only extraction on all nets:

```
extraction_type all r_only
```

Example 2

This example extracts the decoupled resistance and capacitance on all regular nets:

```
extraction_type regular decoupled_rc
```

Example 3

This example performs no extraction on the nets listed in the `exclude.nets` file:

```
extraction_type file exclude.nets none
```

Example 4

This example extracts decoupled resistance and capacitance from all nets containing the `clk*` pattern:

```
extraction_type pattern clk*
```

Example 5

This example shows that If the `file` option is selected, the default value must first be turned off by setting the `all` option to `none`:

```
extraction_type all none
```

```
extraction_type file gamma decoupled_rc
```

Example 6

The following example shows how to extract only a design's signal nets by excluding the extraction of the power nets:

```
extraction_type all decoupled_rc
```

```
extraction_type pattern *VDD none
```

ignore_macros

Syntax

```
ignore_macros file filename
```

Description

Note: This is equivalent to the command [cell ignore](#), which is now the preferred command.

The `ignore_macros` command excludes certain macros, also known as cells, from the `zx` resistance and capacitance extraction. If a net refers to a pin of an excluded macro, `zx` excludes that pin as well. It does not generate warnings or errors pertaining to excluded macros.

Arguments

<i>filename</i>	Specifies the name of the file that lists the macros to ignore during extraction. In the file, specify one macro per line.
-----------------	--

Examples

The following example ignores all instances of the `logo_cell`, `top_right`, `top_left`, `bottom_right`, and `bottom_left` macros, which are listed in the `ignore.cells` file:

```
ignore_macros file ignore.cells
```

ignore_pin

Syntax

```
ignore_pin pin_name
```

Description

Note: This is equivalent to the command `pin ignore`, which is now the preferred command.

The `ignore_pin` command allows you to specify certain instance ports, such as the power and ground ports, which are not extracted for timing when `zx` generates the CeltIC NDC database.

Arguments

<i>pin_name</i>	Specifies the pin name adhering to the DEF regular-expression rules for pin names. You can specify more pins by repeating the <code>ignore_pin</code> command. See example 2 below.
-----------------	---

Examples

Example 1

In the following example, zx ignores pins with a pin name of VDD during generation of the CeltIC NDC database:

```
ignore_pin VD
```

Example 2

In the following example, multiple pins are specified to be ignored:

```
ignore_pin VDD
```

```
ignore_pin VSS
```

```
ignore_pin GND
```

include

Syntax

```
include filename
```

Description

The `include` command imports a designated command file to use as part of the main command file.

Arguments

<i>filename</i>	Name of the command file to be included.
-----------------	--

Examples

The following example imports the `mercury3.cmd` command file into the main zx command file:

```
include /users/eng/test/mercury3.cmd
```

include_via_res_table_file

Syntax

```
include_via_res_table_file filename
```

Description

The `include_via_res_table_file` is used for defining different resistances for different sized vias.

Arguments

<i>filename</i>	Name of the file that defines the resistances for various sized vias.
-----------------	---

File syntax:

```
via pro_layer = " via_name " <ext_layer = " layer_name "> < res_value> [ area [ res_value area ] ...]
```

where:

<i>pro_layer via_name</i>	Refers to the via name specified in the ict file.
<i>ext_layer layer_name</i>	Refers to the extraction layer, and is optional.
<i>res_value</i>	Defines the resistance value in ohms.
<i>area</i>	Defines the area. Units are in 1.0e-06 square microns

`Ext_layer` refers to your def layer name and it is optional. `Pro_layer` must be specified first (after via command), before `ext_layer` and area settings.

For via area entries not specified in the table, ZX does as follows.

- Use linear interpolation, if the via area is falling in between area values specified.
- Use boundary values, if the area value is out of range.

Examples

The following example defines the `via_res_table_file` file as `via_res`:

```
include_via_Res_table_file /users/eng/test/via_res
```

The file would look something like this:

```
via pro_layer = "via2" ext_layer = "via23" 1.0 8100 0.75 14400
```

label net

Syntax

```
label net label x y layer_name <cell_name>
```

Description

The `label net` command is used to annotate (transfer) a net name to a geometry on the given layer enclosing the given x,y coordinates. If the cell name is given, the x,y coordinates are in reference to the coordinates of the cell.

Note: This takes precedence over GDS text annotation.

Arguments

<i>label</i>	Specifies the net names.
<i>layer_name</i>	Specifies the name of the layer.
<i>x y</i>	Specifies the enclosing coordinates specified in microns.
<i>cell_name</i>	Specifies an optional cell-name

Examples

The following example specifies that zx is to annotate the IN1 label to the 500 500 coordinates of the metal1 layer:

```
label net IN1 500 500 metal1
```

label port

Syntax

```
label port label x y layer_name <cell_name>
```

Description

The `label port` command is used to create a pin object with the label name on the given layer at the x,y location. If the cell name is given, the x,y coordinates are in reference to the coordinates of the cell.

Arguments

<i>label</i>	Specifies the net name.
<i>layer_name</i>	Specifies the name of the layer.
<i>x y</i>	Specifies the enclosing coordinates specified in microns.
<i>cell_name</i>	Specifies an optional cell name

Examples

The following example specifies that `zx` is to create a pin object with a `XYZ1` label on the `metal2` layer at the 7500 8500 coordinates:

```
label port XYZ1 7500 8500 metal2
```

layer

Syntax

```
layer layer_name [<icecaps icecaps_layername >|<gds layerID datatype >]
```

Description

Note: This is equivalent to the new commands "`layer map`", and "`layer gds`", which are now the preferred commands.

The `layer` command maps the layer names in the LEF technology file to the layer names in the IceCaps technology file or the GDSII file used to import metal fill. You only need to use this command if the layer names specified in the IceCaps technology file or the GDSII file are different from those used in the LEF file.

Arguments

<i>layer_name</i>	Specifies the name of the layer in the LEF file.
<i>icecaps</i> <i>icecaps_layername</i>	Maps between a layer in the LEF file and a layer in the IceCaps technology file or the RCgen technology file.
<i>gds</i>	Maps between a layer in the LEF file and the GDSII file.
<i>layerID</i>	Specifies the layer number of the GDSII layer.
<i>datatype</i>	Specifies the number of the subclass for the GDSII layer. If you do not specify a data type, <code>zx</code> uses all data types for that GDSII layer.

Examples

The following example maps the `METAL1` layer in the LEF file to layer 4 in the GDSII file:

```
layer METAL1 gds 4
```

layer bias

Syntax

`layer bias layer bias_value [r_only|c_only|both]`

Description

The `layer bias` command sets a bias value, in microns, by which to undersize all wires on a defined layer. It must be in the range of -1 to 1. This value is applied to each edge of the wire, decreasing the width by two times the bias value. The biasing changes only the width of the wire, not the length. You can select a different bias value for resistance extraction and capacitance extraction.

Arguments

<i>layer</i>	Specifies the name of the layer containing the wires to be biased.
<i>bias_value</i>	Specifies the amount, in microns, by which to bias the wires. A positive value undersizes the width of the wire.
<i>r_only</i>	Applies the bias value only during resistance extraction.
<i>c_only</i>	Applies the bias value only during capacitance extraction.
<i>both</i>	Applies the bias value during both resistance and capacitance extraction.

Examples

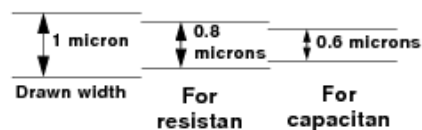
The following example decreases each edge of the wires in metal5 by one tenth of a micron during resistance extraction, and by two tenths of a micron during capacitance extraction:

```
layer bias m5 0.1 r_only
```

```
layer bias m5 0.2 c_only
```

The results will resemble the following diagram:

Figure C-1 **Layer Biasing**



layer gds

Syntax

```
layer gds layer_name gds_layer_num < gds_datatype_num < gds_datatype_range >> [port | text | net |
fill | slot]
```

Description

This command associates a *layer_name* with the integer layer value *gds_layer_num* found in the GDS2 data. This layer name comes from the extraction technology file.

All layers in GDS2 format, to be used for extraction, must be defined with the `layer gds` command. Multiple commands may use the same *layer_name* in order to combine data in GDS2 with different layer or datatype fields into a single extraction layer.

The GDS2 format defines legal values for both layers and datatypes as 0 to 255 inclusive. Any combination of `layer gds` statements may be used with these values for layer and datatype. Omission of any datatype information implies that all datatypes for that layer are used. Single values and ranges for datatypes may also be specified for each `layer gds` statement.

Arguments

<i>layer_name</i>	Specifies the name of the layer
<i>gds_layer_num</i>	Specifies the GDS layer number
<i>gds_datatype_num</i>	Specifies an optional single gds2 datatype number, or the first datatype number for when a range is specified
<i>gds_datatype_range</i>	Specifies a gds2 datatype range, specified as a third number.
port	Keyword used to include GDS2 text objects (all texttypes) as port definitions for the layer
net	Keyword used to transfer text from GDS2 text objects (all texttypes) as a net name annotation to the geometry enclosing the text location. There is no difference between the <i>port</i> and <i>net</i> keywords - they are duplicated in order to get consistent terminology with the <code>label net</code> command.
text	Keyword indicating that all GDS2 text on the given layer is to be read (including all texttypes) and interpreted as labels
fill	Keyword indicating that geometries with the given layer/datatype values are to be considered to be explicit fill data

slot	Keyword indicating that geometries with the given layer/datatype values are to be considered as explicit slotting data
------	--

Examples

Example 1

The extraction technology file contains information for a layer called MET1. The information for this layer is read from the GDS2 file on integer layer 8. The corresponding command file would contain the following:

```
layer gds MET1 8
```

This single statement would be sufficient to get data from this layer in the GDS2 to the extractor on the MET1 layer. The data used for this layer includes all polygons and paths (of all path types). If the technology file was changed to use METAL1 as the layer name instead of MET1, then in the command file you could either change the `layer gds` command or add an additional `layer map` command:

```
layer gds MET1 8
```

```
layer map MET1 METAL1
```

The `layer map` command can also be used to associate DEF layer names to technology layer names when they do not match.

Example 2

Any number of `layer gds` commands may be used. Multiple statements with the same `layer_name` have the effect of combining data.

```
layer gds MET1 8
```

```
layer gds MET1 14
```

This example combines data from GDS2 layers 8 and 14 into a single extraction layer named MET1.

Example 3

To specify a range of datatypes for a particular layer, you enter the beginning and ending datatype numbers.

```
layer gds MET1 8 2 7
```

This command associates GDS layer number 8 with the layer name MET1, for all data types numbered 2 to 7.

layer map

Syntax

```
layer map layer_name tch_layer_name
```

Description

The `layer map` command is used to map a *layer_name* to a layer defined in the technology file. Layers with names that match technology layer names are mapped implicitly. Design layers may be excluded from extraction by mapping the *layer_name* to a name not defined in the technology file.

Arguments

<i>layer_name</i>	Name of the layer
<i>tch_layer_name</i>	Name of the layer defined in the technology file

Examples

```
layer map METAL_8 dummy_METAL_8
```

Note: This example will discard data on METAL_8 from extraction.

pin ignore

Syntax

```
pin ignore pin_name
```

Description

The named pin, such as a power or ground pin, is to be excluded from being written to the CeltIC NDC database.

Arguments

<i>pin_name</i>	Specifies the pin to be excluded from being written to the CeltIC NDC database.
-----------------	---

Example

The following example specifies that the PIN1 pin is not to be written to the CeltIC NDC database:

```
pin ignore PIN1
```

promote_ports

Syntax

```
promote_ports [none | all | cell cell_name pin pin_name | inst inst_name pin pin_name ]
```

Description

The `promote_ports` command is used to control the use of port geometries during extraction.

Arguments

none	The default value. Also, this is the default if no <code>promote_ports</code> command is found. This means that all port geometry will be subtracted from interconnect geometry and will not be used to generate parasitic values. This option will ensure that no cell parasitic is included in the interconnect model, but it could cause errors with respect to feedthru resistance or coupling capacitance to ports.
all	The all option merges all of the port geometries into the interconnect. This provides more accurate feedthru resistance and coupling capacitance, but could duplicate some parasitic effects included in the library model.
cell	The cell <i>cell_name</i> pin <i>pin_name</i> specification is used to selectively include the port geometry of a specific pin from all instances of a cell.
inst	The inst <i>inst_name</i> pin <i>pin_name</i> specification is used to selectively include the port geometry of a specific pin from a single instance.
<i>pin_name</i>	Specifies the specific cell or instance pin.

Examples

The first example below merges all port geometries into the interconnect: The second example promotes the port geometry of pin FT1, of cell R64. And the third example promotes pin Z, of instance IOA.

- `promote_ports all`

- `promote_ports` cell R64 pin FT1
- `promote_ports` inst IOA pin Z

Extractor Variables

`density_check_method`

Syntax

```
setvar density_check_method [diearea|database|perlayer|rectangle|none]
```

Description

The `density_check_method` variable selects the method that zx uses to check the wire density of the chip.

Arguments

<code>diearea</code>	Checks the wire density in the area defined by the <code>diearea</code> keyword in the DEF file. It checks the area on all layers. It checks the area by dividing it into a tile size defined by the technology file, usually 50 to 100 microns on a side. This option is recommended for full-chip checking.
<code>database</code>	Checks the wire density in the area defined by the layout database. This area is defined by a leftmost, bottommost point to a rightmost, topmost point. It checks the area on all layers. It checks the area by dividing it into a tile size defined by the technology file, usually 50 to 100 microns on a side. This option is recommended for block-level checking. The default is <code>database</code> .
<code>perlayer</code>	Checks the wire density in the area defined by the extent of routing on each area of a layer. This option is recommended for validating layout accuracy only.
<code>rectangle</code>	Checks the wire density in a rectangular region defined by the command density_check_rectangle .
<code>none</code>	Uses the default method, which is <code>database</code> .

Examples

The following example checks the wire density in an area defined by the `diearea` keyword in the DEF file:

```
setvar density_check_method diearea
```

dump_options

Syntax

```
setvar dump_options [true|false]
```

Description

The `dump_options` variable saves the current zx variable settings in the log file (see [log_file](#)).

Arguments

<code>true</code>	Saves the variable settings to a log file.
<code>false</code>	Does not save the settings to a log file. The default is <code>false</code>

Examples

The following example saves the current variable settings to the log file:

```
setvar dump_options true
```

enable_erosion_effects

Syntax

```
setvar enable_erosion_effects [r_only|c_only|rc|none]
```

Description

The `enable_erosion_effects` variable determines whether to include the effects of erosion on resistance, capacitance, or both in the extraction results.

Arguments

<code>r_only</code>	Extracts only resistance effects due to erosion.
<code>c_only</code>	Extracts only capacitance effects due to erosion.
<code>rc</code>	Extracts resistance and capacitance effects due to erosion. The default is <code>rc</code> .
<code>none</code>	Does not extract resistance or capacitance effects due to erosion.

Examples

In the following example, `zx` includes the effects of erosion on capacitance in the extraction results:

```
setvar enable_erosion_effects c_only
```

enable_metal_fill_effects

Syntax

```
setvar enable_metal_fill_effects [true|false]
```

Description

The `enable_metal_fill_effects` variable determines whether `zx` includes in the extraction results the effects of the metal fill between wires on a layer that was deposited according to the specifications in the DEF file.

Note: `zx` issues a warning if the DEF file does not contain specifications for metal fill.

Arguments

<code>true</code>	When you set this variable to <code>true</code> , <code>zx</code> includes the effects of metal fill between the wires. The default is <code>true</code> .
	If you set the variable <code>enable_metal_fill_effects</code> to <code>true</code> , you must use the variable <code>metal_fill_type</code> to specify the type of metal fill to use, either grounded, which has a fixed electrical charge, or floating, which has no fixed electrical charge. Typically, grounded metal has a higher capacitance.
<code>false</code>	When you set this variable to <code>false</code> , <code>zx</code> does not include the effects of metal fill between the wires.

Examples

In this example, zx does not include the effects of metal fill between the wires in the extraction results:

```
setvar enable_metal_fill_effects false
```

enable_poly_routing

Syntax

```
setvar enable_poly_routing [true|false]
```

Description

The `enable_poly_routing` variable enables zx to extract parasitic data for the layer type POLYCID.

Arguments

true	When you set this variable to <code>true</code> , zx extracts parasitic data for the layer type POLYCID.
false	When you set this variable to <code>false</code> , zx will not extract parasitic data for the layer type POLYCID. The default is false.

Examples

The following example specifies that zx should extract parasitic data for the layer type POLYCID.

```
setvar enable_poly_routing true
```

enable_r_function_width

Syntax

```
setvar enable_r_function_width [true|false]
```

Description

The `enable_r_function_width` enables zx to establish resistivity values based on wire width, versus establishing resistivity based on a fixed value.

Arguments

<code>true</code>	When you set this variable to <code>true</code> , the tool establishes resistivity values based on wire width.
<code>false</code>	When you set this variable to <code>false</code> , the tool does not establish resistivity values based on wire width.

Examples

The following example specifies that zx establishes the resistivity values based on wire width.

```
setvar enable_r_function_width true
```

`enable_toplevel_diff_extract`

Syntax

```
setvar enable_toplevel_diff_extract true [true|false]
```

Description

The `enable_toplevel_diff_extract` variable extracts ports defined on the diffusion layer in power-grid view library. This variable is used during body bias analysis to extract connectivity in the diffusion layer..

Arguments

<code>true</code>	When you set this variable to <code>true</code> , the tool extracts ports defined on the diffusion layer in power-grid view library.
<code>false</code>	When you set this variable to <code>false</code> , the tool does not extract the ports defined on the diffusion layer.

Examples

The following example specifies that zx extracts ports defined on the diffusion layer in power-grid view library.

```
setvar enable_toplevel_diff_extract true
```

exclude_blockagewire

Syntax

```
setvar exclude_blockagewire [true|false]
```

Description

The `exclude_blockagewire` variable determines whether zx ignores, for parasitic extraction, any wires in the DEF input file which have SHAPE BLOCKAGE in the SpecialNets section.

Arguments

<code>true</code>	When you set this variable to <code>true</code> , zx ignores, for parasitic extraction, any wires in the DEF input file which have a SHAPE BLOCKAGE statement. The default is <code>true</code> .
<code>false</code>	When you set this variable to <code>false</code> , zx does not ignore wires that have a SHAPE BLOCKAGE statement.

Examples

The following example specifies that zx should not ignore wires that have a SHAPE BLOCAGE statement specified in the DEF input file.:

```
setvar exclude_blockagewire false
```

gds_unit

Syntax

```
setvar gds_unit value
```

Description

The `gds_unit` variable specifies what conversion factor `zx` is to use to convert GDS database units - used for geometric dimensions, coordinates, etc. - to microns.

Arguments

<i>value</i>	Specifies the conversion factor <code>zx</code> is to use to convert GDS database units to 1 micron. It is an integer value within the range of 1 through 1000000 (inclusive). The default is 1000.
--------------	--

Examples

The following example converts 10,000 GDS database units to 1 micron:

```
setvar gds_unit 10000
```

Ignore_shorts

Syntax

```
setvar ignore_shorts [true|false]
```

Description

The `ignore_shorts` variable enables `zx` to either flag or ignore detected shorted nets, as in previous releases. The **default setting** is `true` for DEF input, and `false` for other input. Setting it to `true` will make `zx` behave the same as in Version 3.x.

Arguments

<code>true</code>	When set to <code>true</code> , if shorts are detected, they will be ignored and <code>zx</code> continues with some degree of inaccuracy.
<code>false</code>	When set to <code>false</code> , shorts will be flagged and <code>zx</code> will continue but with those nets shorted.

Examples

The following example specifies that zx is to ignore any detected shorts:

```
setvar ignore_shorts true
```

layout_scale

Syntax

```
setvar layout_scale scale_factor
```

Description

The `layout_scale` variable scales the dimensions in the layout by the scale factor that you specify. Using this option enables zx to correctly extract designs that have been sized by a scale factor.

Arguments

<i>scale_factor</i>	Specifies the scale factor by which zx scales the layout. This factor can range from 0.01 to 5. The default is 1.0, indicating that no scaling is performed.
---------------------	---

Examples

This example scales the layout of the design by 25 percent:

```
setvar layout_scale .75
```

load_optimized_views

Syntax

```
setvar load_optimized_views [ true | false ]
```

Description

The `load_optimized_views` variable specifies that zx load the optimized version of the power-grid views from the cell library. The `load_optimized_views` variable is used in power-grid analysis flow.

Optimized means that small notches in a VDD port geometry are basically smoothed out. In effect, the extraction for power-grid analysis would typically run faster because it has to deal with less geometry. The side effects would some time cause disconnects or opens typically in tie-hi and/or tie-low types of connection. The default in power-grid analysis flow is true. If optimized view does not exist in the cell library, QX will load the non-optimized one.

Note: The optimization, which is done in LibGen, is only performed for cells which are smaller than the setting of the `stdCellSizeLimit` LibGen variable.

Arguments

<code>true</code>	When you set this variable to <code>true</code> , zx loads the optimized version of the power-grid views from the cell library. The default is true in a power-grid analysis flow. If an optimized view does not exist in the cell library, zx will load the non-optimized one.
<code>false</code>	When you set this variable to <code>false</code> , zx does not load the optimized views

Examples

The following example causes zx to load the optimized views:

```
setvar load_optimized_views true
```

log_file

Syntax

```
setvar log_file filename
```

Description

The `log_file` variable specifies the name of the log file generated by zx. (see also [dump_options](#))

Arguments

<code>filename</code>	Log file name. The default log file name is <code>zx.log</code> .
-----------------------	--

Examples

The following syntax specifies that `apollo.log` be the name of the zx log file:

```
setvar log_file apollo.log
```

max_error_messages

Syntax

```
setvar max_error_messages number
```

Description

The `max_error_messages` variable sets the maximum number of error messages issued by zx during a session.

Arguments

number	<i>number</i> must be an integer in the range of 1 through 2,147,483,647, inclusive. The default is 100.
--------	--

Examples

The following example sets the maximum number of error messages at 250:

```
setvar max_error_messages 250
```

max_resistor_length

Syntax

```
setvar max_resistor_length number
```


Description

The `max_resistor_length` variable specifies the maximum resistor length, in microns. To maintain accuracy, zx may need to fracture large resistances into multiple smaller resistances.

Arguments

<i>number</i>	<i>number</i> can be from 10 to 800 microns, inclusive. The default is 50 microns. Smaller values increase the accuracy of the capacitance extraction, decrease capacity, and increase run times.
---------------	--

Examples

This example sets the maximum resistor length at 600 microns:

```
setvar max_resistor_length 600
```

message_detail_level

Syntax

```
setvar message_detail_level number
```

Description

The `message_detail_level` variable sets the maximum number of warning messages that zx issues during a session.

Arguments

<i>number</i>	<i>number</i> must be an integer from 1 through 2147483647, inclusive. The default is 10.
---------------	--

Examples

The following example sets the maximum number of warning messages at 75:

```
setvar message_detail_level 75
```

metal_fill_type

Syntax

```
setvar metal_fill_type [grounded|floating]
```

Description

The `metal_fill_type` variable specifies the type of metal fill to use, either grounded, which has a fixed potential, or floating, which has a fixed electrical charge. Typically, grounded metal has a higher capacitance.

Note: The variable `enable_metal_fill_effects` must be set to `true`, its default, for this variable, `metal_fill_type` to have any effect. If `enable_metal_fill_effects` is set to `false`, zx ignores metal fill if it is present in the DEF file or GDSII file.

❗ **Do not set this variable to `grounded` only to obtain higher capacitance for downstream tools.**

Arguments

grounded	When you set this variable to <code>grounded</code> , zx models the metal fill as grounded.
floating	When you set this variable to <code>floating</code> , the tool models the metal fill as floating. The default is <code>floating</code> .

Examples

This example models the metal fill as grounded:

```
setvar metal_fill_type grounded
```

oa_analysis_point

Syntax

```
setvar oa_analysis_point analysis_point_name
```

Description

The `oa_analysis_point` variable specifies the name of an analysis point (process corner) that the parasitics are written to in the OpenAccess (OA) database as an `oaOpPoint`. The temperature of the analysis point stored in the `oaOpPoint` will be set if it is defined either in the command file or in the technology file

`oa_analysis_point` can be used for dumping multiple parasitic network generated from different zx runs in the same OA database.

Arguments

<code>analysis_point_name</code>	Name of the analysis point (process corner) that the parasitics are written to in the OA database as an <code>oaOpPoint</code> . If this variable is not specified the default name will be 'default'.
----------------------------------	---

Examples

The following example specifies the use of the name 'oa_corner30' as the analysis point name to be written to the OA database:

```
setvar oa_analysis_point oa_corner30
```

output_cache_directory_name

Syntax

```
setvar output_cache_directory_name cache_dirname
```

Description

The `output_cache_directory_name` variable specifies the name of a directory where all zx temporary or cache files are written for both single- and multiple-processor runs. The files inside this directory are deleted when the zx run is finished, but the directory itself will remain.

Arguments

<code>cache_dirname</code>	Name of the directory in which to store the temporary or cache files. The default value is the current working directory. Note: zx will create the specified directory if it does not already exist.
----------------------------	---

Examples

The following example specifies a directory called `../egr/design451/temp_files`:

```
setvar output_cache_directory_name ../egr/design451/temp_files
```

output_directory_name

Syntax

```
setvar output_directory_name dirname
```

Description

The `output_directory_name` variable specifies the name of the directory in which to store the output files.

Note: See also, the variable `output_cache_directory_name`

Arguments

<i>dirname</i>	Name of the directory in which to store the output files. The default value is the name of the current directory. Note: The specified directory will be created if it does not already exist.
----------------	--

Examples

This example specifies that the output files be placed in a directory called `rcdb`:

```
setvar output_directory_name /users/eng/rcdb
```

The next example places the output files in a directory called `sc5`:

```
setvar output_directory_name ../../sc5
```

output_drawn_geometry

Syntax

```
setvar output_drawn_geometry [ true | false ]
```

Description

The `output_drawn_geometry` variable determines whether drawn resistor widths and lengths should output (not scaled).

Arguments

[true false]	A value of true causes the drawn resistor lengths and widths to be output (not scaled). The default is false.
---------------------	--

Examples

This example specifies that resistor widths and lengths are output as drawn.

```
setvar output_drawn_geometry true
```

output_file_name

Syntax

```
setvar output_file_name filename
```

Description

The `output_file_name` variable specifies prefix for the name of the generated targeted output files (for example, VoltageStorm pgdb files, lumped/total c files, or def netlist files).

Arguments

<i>filename</i>	Name of the targeted output files generated by the tool. No extension is required.
-----------------	--

Example

This example sets gnu as the name of the output files:

```
setvar output_file_name gnu
```

output_incomplete_nets

Syntax

```
setvar output_incomplete_nets [true|false]
```

Description

The `output_incomplete_nets` variable determines whether zx includes the parasitics of any open nets (for example partially routed or incomplete networks) in the parasitic databases for the analysis tool.

Arguments

<code>true</code>	When you set this variable to <code>true</code> , zx includes open nets in the parasitic databases
<code>false</code>	When you set this variable to <code>false</code> , the tool does not include open nets. The default is <code>false</code> .

Note: When incomplete nets are not written out (`output_incomplete_nets false`), coupling to those nets will be decoupled.

Example

The following example includes open nets in the parasitic databases:

```
setvar output_incomplete_nets true
```

output_oa

Syntax

```
setvar output_oa [true|false]
```

Description

The `output_oa` variable controls whether or not zx writes the parasitics back to the OpenAccess (OA) database which was read from as input.

Note: In order for zx to process an OA database, it is required that the design input only come from an OA database.

Note: If an OA output is specified, but the OA input is hierarchical, a flattened SPEF file containing the extraction

data will be created instead.

Arguments

true	When set to true, zx will write the parasitics to the OA database. This is the default if OA input was specified.
false	When set to false, zx does not write the parasitics to the OA database.

Example

The following example specifies that zx is to write the parasitics back into the OA database.

```
setvar output_oa true
```

remove_dangling_resistors

Syntax

```
setvar remove_dangling_resistors [true|false]
```

Description

The `remove_dangling_resistors` variable determines whether zx removes floating metals--that is, metals that have no outputs or inputs--from the RC extraction. The purpose of this variable is to get rid of the resistors that are only connected on one end. It only has an impact on resistance and not capacitance.

Note: This variable was previously named `remove_floating_metals` .

Note: Setting this variable to false will generate version 3.3.2 comparable networks.

Arguments

true	Removes floating metals from the RC extraction. The default is true.
false	Retains floating metals from the RC extraction.

Examples

This example removes floating metals from the RC extraction:

```
setvar remove_dangling_resistors true
```

tech_file

Syntax

```
setvar tech_file filename
```

Description

The `tech_file` variable specifies the name of the technology file to be used by zx. This file is generated by RCgen, either by you or by the foundry.

Note: LibGen can also place a copy of this technology file in the cell library directory. zx will then use that file if the `tech_file` variable is not set in the zx command file.

Arguments

<i>filename</i>	Name of the technology file. There is no default file name.
-----------------	---

Examples

The following line indicates that `chip731.tch` is the name of the technology file to be submitted to the tool:

```
setvar tech_file ./lib3.cl/chip731.tch
```

top_level_names_only

Syntax

```
setvar top_level_names_only [true/false]
```

Description

The `top_level_names_only` variable is used if you want to preserve only the top level net names of a hierarchical design, and ignore any lower level names. This is only applicable to GDSII data.

Arguments

true	Set this variable to true to preserve only the top level net names.
false	This preserves the lower level net names also.

Examples

The following example preserves only the top level net names:

```
setvar top_level_names_only true
```

use_def_over_library

Syntax

```
setvar use_def_over_library [true/false]
```

Description

The `use_def_over_library` variable determines if ZX will use the DEFdata instead of .cl.

Arguments

true	Set this variable to <code>true</code> to specify to use DEF data instead of .cl. The default is <code>true</code> .
false	Set this variable to <code>false</code> to specify to use .cl data.

Examples

The following example specifies that DEF data should be used instead of .cl:

```
setvar use_def_over_library true
```

use_layer_bias_in_cmd_and_process_bias_in_tech

Syntax

```
setvar use_layer_bias_in_cmd_and_process_bias_in_tech [true/false]
```

Description

The `use_layer_bias_in_cmd_and_process_bias_in_tech` variable determines whether zx applies the layer biasing in the technology file, or in both the command file and the technology file to the wires in the design.

Arguments

<code>true</code>	When you set this variable to <code>true</code> , zx applies the layer biasing in the command file and the technology file to the wires.
<code>false</code>	When you set it to <code>false</code> , zx applies the layer biasing in the technology file only. The default is <code>false</code> .

Examples

The following example applies layer biasing in the technology file to the wires in the design:

```
setvar use_layer_bias_in_cmd_and_process_bias_in_tech false
```

use_manhattan_ports

Syntax

```
setvar use_manhattan_ports [true|false]
```

Description

The `use_manhattan_ports` variable determines if zx will promote and convert the non-manh macro port geometry to manhattan geometry.

Arguments

<code>true</code>	When you set this variable to <code>true</code> , zx will promote and convert the non-manh macro port geometry to manhattan geometry. It will give a better correlation to the pure manhattan top level design without any non-manh routing.
<code>false</code>	When you set it to <code>false</code> , zx will not promote and convert the non-manh macro port geometry to manhattan geometry . The default is <code>false</code> .

Examples

The following example specifies that zx will promote and convert the non-manh macro port geometry to manhattan geometry:

```
setvar use_manhattan_ports true
```

wee_type

Syntax

```
setvar wee_type [r_only|c_only|rc|none]
```

Description

The `wee_type` variable determines whether to include the effects of wire-edge enlargement on resistance, capacitance, or both in the extraction results.

Arguments

<code>r_only</code>	Extracts only resistance effects due to wire-edge enlargement.
<code>c_only</code>	Extracts only capacitance effects due to wire-edge enlargement.
<code>rc</code>	Extracts resistance and capacitance effects due to wire-edge enlargement. The default is <code>rc</code> .
<code>none</code>	Does not extract resistance or capacitance effects due to wire-edge enlargement.

Example

In the following example, zx extracts only resistance effects due to wire-edge enlargement:

```
setvar wee_type r_only
```

Library Generator Options

This section lists the advanced Library Generation commands and variables that can be included in Voltus using the `set_advanced_pg_library_mode -libgen_command_file` command parameter. They are listed in alphabetical order.

- [Library Generator Commands](#)
- [cell_decap_area](#)
- [cell_decap_area_file](#)
- [cell_ground_names](#)
- [cell_ignore_list](#)
- [cell_ignore_list_file](#)
- [cell_pin_net_map](#)
- [cell_pin_net_map_file](#)
- [cell_power_names](#)
- [cell_supply_names_file](#)
- [current_region](#)
- [current_region_default](#)
- [current_region_file](#)
- [cell_decap](#)
- [derived_layer_list](#)
- [discard_detailed_powerview_cell_list](#)
- [discard_detailed_powerview_cell_list_file](#)
- [foreign_name_map](#)
- [foreign_name_map_file](#)
- [lef_layer_ignore_list](#)
- [lef_layer_ignore_list_file](#)
- [li](#)
- [pgv_header_map](#)
- [pgv_header_map_file](#)
- [poly_command](#)
- [port_power_gate_file](#)
- [port_view_current_file](#)

- [scale_current_region](#)
- [scale_current_region_default](#)
- [thunder_model_table_directory](#)
- [well](#)
- [well_cap_area](#)
- [well_cap_area_file](#)
- [well_cap](#)
- [Library Generator Variables](#)
- [assume_foreigns](#)
- [assume_foreigns_mode](#)
- [case_sensitivity_default](#)
- [cell_accura_data_file](#)
- [cmd_data_case_sensitive](#)
- [collapsed_view_current_threshold](#)
- [create_dummy_tech_file](#)
- [database_precision](#)
- [debug_skip_parasitic_extraction](#)
- [debug_skip_thunder](#)
- [debug_skip_xtc](#)
- [default_ground_voltage](#)
- [default_lef_min_spacing](#)
- [default_lef_min_width](#)
- [default_power_voltage](#)
- [default_via_resistance](#)
- [disable_lef_parser_escape_check](#)
- [do_optimized_views](#)
- [do_reextract_in_parallel](#)
- [emulate_lef_parsing](#)
- [gds_cell_extension](#)
- [gds_extractor](#)
- [gds_extractor_command_file](#)

- `gds_port_explosion_depth`
- `gds_to_lef_gds_extractor_command_file`
- `generated_gds_file`
- `halt_on_missing_foreign_data`
- `halt_on_undefined_lef_layer`
- `ignore_lef_use`
- `ignore_missing_via_definition`
- `keep_debug_files`
- `keep_netlist_in_library`
- `layout_scale_factor`
- `libgen_executable`
- `maxerrors`
- `max_itfnode_dist`
- `max_resistor_length`
- `output_tool_performance_data`
- `output_wlt`
- `parasitic_extractor`
- `parasitic_extractor_command_files_directory`
- `port_file`
- `prpgv_x_offset`
- `prpgv_y_offset`
- `std_cell_size_limit`
- `stripe_step_size`
- `suppress_regular_gds_layer`
- `tech_file`
- `temp_directory_name`
- `temperature`
- `thunder_command_file`
- `thunder_executable`
- `use_current_data_from_pgv`
- `use_lef_only_itfnodes`

- [use_lef_technology](#)
- [use_old_lef_parser](#)
- [use_pwrnet_ipeak](#)
- [xhc_stripe_directory_name](#)

Library Generator Commands

cell_decap_area

```
cell_decap_area { cell_name_list } cap_value <unit>
```

This command specifies the amount of area based decoupling capacitance in a cell. This command is used in generating dynamic port or detailed power-grid views. Similar to "cell_decap" command, if only a port view is created for a cell, the capacitance specified by the "cell_decap_area" command is evenly distributed over the taps in the port view. For detailed views this is different than using the "cell_decap" command, in that the capacitance value assigned by using this command will not be used to scale the capacitance value calculated, since it is assumed that the value calculated by Thunder is more accurate.

The capacitance specified will be used for both the POWER and GROUND nets. If there are multiple POWER nets or GROUND nets, this value will be evenly distributed among those nets.

Note: If user wants to specify a decap value for a GENERIC supply net, one must specify it explicitly onto the net. If user specifies the decap value for the cell it will be applied on only the POWER and GROUND net. If the GENERIC supply net existed in the cell, LibGen will issue an INFO message to indicate that the value will not be used for the GENERIC supply net. If the user specifies the decap value for the cell, and only a generic supply net exists in the cell, LibGen will apply that value onto the generic supply net and issue a warning message.

Parameters

<i>cell_name_list</i>	Lists the cell names used to assign a capacitance value. The cell name supports wildcard characters (e.g. * ?).
<i>cap_value</i>	Specifies the amount of area based decoupling capacitance. Default unit is femtofarads per square micron.
<i>unit</i>	For specifying an optional parameter used to specify the unit of length. LibGen will use square "unit" as the area unit internally. Default unit is micron.

Example

- The following example specifies a decoupling capacitance of 5 femtofarads per square micron for cells starting with letters NTZ:

```
cell_decap_area NTZ* 5
```

cell_decap_area_file

```
cell_decap_area_file filename
```

This command specifies the name of a file containing a list of cells and the area based decoupling capacitance value for each.

Parameters

<i>filename</i>	The name of the area based decoupling capacitance file.
-----------------	---

Note: If user wants to specify a decap value for a GENERIC supply net, one must specify it explicitly onto the net. If user specifies the decap value for the cell it will be applied on only the POWER and GROUND net. If the GENERIC supply net existed in the cell, LibGen will issue an INFO message to indicate that the value will not be used for the GENERIC supply net. If the user specifies the decap value for the cell, and only a generic supply net exists in the cell, LibGen will apply that value onto the generic supply net and issue a warning message.

File Format

Decoupling capacitance file format

The lines of the area based decoupling capacitance file are as follows:

- `CAP_UNIT unit_in_farads`
The `UNIT` definition is optional. If no `UNIT` statement is present at the beginning of the file, all capacitance values not followed by a unit specification are assumed to be in femtofarads.
- `LENGTH_UNIT unit_in_meters`
The `LENGTH_UNIT` is optional. Default length unit is microns
- `CELL cell_name cap_value`
`cell_name` specifies the name of the cell. `cap_value` specifies the amount of area based decoupling capacitance. The default unit is femtofarads per square micron. The cellname can contain wildcard characters(e.g. * ?).

All capacitance values can be followed by a unit specification. The following units are recognized: millifarads (mF), microfarads (μ F), nanofarads (nF), picofarads (pF), femtofarads (fF). If specified this will take priority over the `CAP_UNIT` specification.

Examples

- The following example specifies a file named `decap_cell_list` that contains a list of cells and the area based decoupling capacitance for each.

```
cell_decap_area_file decap_cell_list
```

- The following examples illustrate the contents of the area based decoupling capacitance file. This line in the decoupling capacitance file is optional. It assumes that the default units are femtofarads.

```
CAP_UNIT          1.0e-15
```

In the next part of the file, the area based decoupling capacitance for Cell_A is 100 femtofarads per square micron.

```
CELL              Cell_A          100.00
```

The capacitance specified will be used for both the POWER and GROUND nets. If there are multiple POWER nets or GROUND nets, this value will be evenly distributed among those nets.

cell_ground_names

```
cell_ground_names [cell [cellname+]] nets [ground_net_name+] value [supply_value] <bulk>
```

This command specifies the net names to be used as the ground nets for the named cells and the nominal voltage of those nets. Use this command when any of the `generate*_powerview` variables are set to `true`, that is, when LibGen generates power-grid views.

When you specify the names of cells with `cell_ground_names`, you must also specify them with `cell_power_names`. See "[cell_power_names](#)" for information on this command.

Note: LibGen only takes nets listed by the `cell_ground_names` or `cell_power_names` command as supply nets and treats all other nets as signal nets. This is done even if those nets are listed by the `generic_ground_names` or `generic_power_names` commands. If the cell is not in the `cell_ground_names` or `cell_power_names` list, LibGen will use LEF and `generic_ground_names` or `generic_power_names` settings.

Parameters

cell {cellname+}	Identifies the cell or cells to which the ground net supplies power. No wildcards are allowed for the specification of cell names.
nets {net_name+}	Identifies the ground net or nets that supply power to the named cells. No wildcards are allowed for the specification of net names

<i>value value</i>	Gives the nominal voltage of the ground nets. If you specify the ground net names to use for cells, you must also specify the power net names for those same cells. Refer to " cell_power_names " for more information on the <code>cell_power_names</code> command.
<i>bulk</i>	Indicates that all nets specified by the <code>cell_power_names</code> command be considered bulk nets. No current will be distributed to these nets.

Examples

- The next example specifies that the `gnd` ground net supplies a nominal voltage of 0.0 volts to the `block_a` cell:

```
cell_ground_names cell {block_a} net {gnd} value 0.0
```

- The next example indicates that the `gnd` and `gnd1` nets supply a nominal voltage of 0.0 volts to the `na` cell:

```
cell_ground_names cell {na} net {gnd gnd1} value 0.0
```

- The next example indicates that the `gnd_bulk` net is a bulk net:

```
cell_ground_names cell {na} net {gnd_bulk} value 0.0 bulk
```

cell_ignore_list

```
cell_ignore_list {cellname+}
```

This command lists the cells that LibGen is not to process. LibGen processes any cell specified by the `cell_list` or `cell_list_file` command, unless it is specified by the `cell_ignore_list` or `cell_ignore_list_file` command.

Parameters

<i>Cellname</i>	Specifies the names of the cells that LibGen is not to process.
-----------------	---

Example

- In the following example, LibGen will process all cells given by the `cell_list` or `cell_list_file` command, except the MUX21X05, MUX22, and MUX01FT3 cells:

```
cell_ignore_list {MUX21X05 MUX22 MUX01FT3}
```

- An example, if you wanted to process all cells in the design except one named `mem_blk1`, you would put the following lines in the LibGen command file:

```
cell_list {*}
```

```
cell_ignore_list {mem_blk1}
```

cell_ignore_list_file

```
cell_ignore_list_file filename
```

This command specifies the name of a file containing a list of cells that LibGen is not to process.

Parameters

<i>filename</i>	The name of a file containing the list of cells that LibGen is to ignore
-----------------	--

Example

- The following line specifies a file named `ignore_cells.dat`, which contains a list of the cells that LibGen is to ignore:

```
cell_ignore_list_file ignore_cells.dat
```

cell_pin_net_map

```
cell_pin_net_map {cellname+} net_name {pin_name+}
```

This command specifies the mapping between a cell's LEF pin names and the internal names of the net to which those pins are attached. It is used when multiple pins are attached to a single net.

Parameters

<i>cellname</i>	Specifies the name of the cell or cells containing multiple pins attached to a single net. This name can include the * and ? wildcard characters.
<i>net_name</i>	Specifies the internal name of the net to which multiple pins are attached.
<i>pin_name</i>	Specifies the LEF name of the pins attached to a single net.

Examples

- The following example maps the LEF pin names `vdd1`, `vdd2`, and `vdd3` to the internal net name `vdd` in the `block_a` cell:

```
cell_pin_net_map {block_a} vdd {vdd1 vdd2 vdd3}
```

- The next example maps the names of any LEF pins starting with `vdd` to the internal net name `vdd` in any cell whose name starts with `block`:

```
cell_pin_net_map {block*} vdd {vdd*}
```

cell_pin_net_map_file

```
cell_pin_net_map_file filename
```

This command specifies the name of a file containing the mappings between a cell's LEF pin names and the internal names of the net to which those pins are attached.

Parameters

<i>filename</i>	The name of the file containing the mappings. Its format is one net mapping per line.
-----------------	---

File Format

Each line is as follows for the cell pin net map file:

```
cellname net_name pin_name
```

Cellname and *pin_name* can include the `*` and `?` wildcard characters.

Example

- The following example specifies the `pin_map.txt` file as the mapping file:

```
cell_pin_net_map_file pin_map.txt
```

cell_power_names

```
cell_power_names [cell [cellname+]] nets [power_net_name+] value supply_value <bulk>
```

This command specifies the net names to be used as the power nets for the named cells and the nominal voltage of those nets. Use this command when any of the `generate_*_powerview` variables are set to `true`, that is, when LibGen generates power-grid views.

When you specify the names of cells with `cell_power_names`, you must also specify them with `cell_ground_names`. See "[cell_ground_names](#)" for information on this command.

Note: LibGen only takes nets listed by the `cell_ground_names` or `cell_power_names` command as supply nets and treats all other nets as signal nets. This is done even if those nets are listed by the `generic_ground_names` or

`generic_power_names` commands. If the cell is not in the `cell_ground_names` or `cell_power_names` list, LibGen will use LEF and `generic_ground_names` or `generic_power_names` settings.

Parameters

<code>cell</code> <i>cellname</i>	Identifies the cell or cells to which the power net supplies power. No wildcards are allowed for the specification of cell names.
<code>nets</code> <i>[power_net_name+]</i>	Identifies the power net or nets that supply power to the named cells. No wildcards are allowed for the specification of net names.
<code>value</code> <i>supply_value</i>	Gives the nominal voltage of the power net. If you specify the power net names to use for cells, you must also specify the ground net names for those same cells. Refer to " cell_ground_names " for more information on the <code>cell_ground_names</code> command.
<i>bulk</i>	Indicates that all nets specified by the <code>cell_power_names</code> command be considered bulk nets. No current will be distributed to these nets.

Example

- The following example specifies that the VCC net is the power net for the nand2 cell and that its nominal voltage is 1.0:

```
cell_power_names cell {nand2} net {vcc} value 1.0
```

- The next example indicates that the VCC and VCC1 nets supply a nominal voltage of 1.0 volts to the `na` cell:

```
cell_power_names cell {na} net {vcc vcc2} value 1.0
```

- The next examples indicates that the `pwr_bulk` and `grd_bulk` are a bulk nets:

```
cell_power_names cell {na} net {pwr_bulk} value 1.0 bulk
```

```
cell_power_names cell {na} net {gnd_bulk} value 0.0 bulk
```

cell_supply_names_file

```
cell_supply_names_file filename
```

This command specifies the name of the file containing the names of the power and ground nets and their nominal supply values for one or more cells. This file is in ASCII format. Use this command when any of the `generate_*_powerview` variables are set to `true`, that is, when LibGen generates detailed power-grid views.

Parameters

<i>filename</i>	The name of the file containing the names of the power and ground nets and their supply values.
-----------------	---

File Format

The format of the cell supply names file is as follows.

```
CELLS {cellname+}
```

```
POWER value {net_name+}
```

```
GROUND value {net_name+}
```

```
GENERIC {net_name+}
```

- Use the `CELLS` keyword, then specify a list of cells that the next `POWER` and `GROUND` statements refer to.
- Use the `POWER` keyword, then specify a voltage value and a list of net names to be used as the power connections for the previously specified cells.
- Use the `GROUND` keyword, then specify a voltage value and a list of net names to be used as the ground connections for the previously specified cells.
- Use the `GENERIC` keyword, then specify a list of net names to be used as the generic supply connections which could be connected to either power or ground for the previously specified cells.

The file can contain multiple `CELL`, `POWER`, `GROUND`, and `GENERIC` statements. No wildcards are allowed for the specification of cell names and net names. LibGen ignores blank lines and lines beginning with the pound sign (#).

Note: If user wants to specify a decap value for a `GENERIC` supply net, one must specify it explicitly onto the net. If user specifies the decap value for the cell it will be applied on only the `POWER` and `GROUND` net. If the `GENERIC` supply net existed in the cell, LibGen will issue an `INFO` message to indicate that the value will not be used for the `GENERIC` supply net. If the user specifies the decap value for the cell, and only a generic supply net exists in the cell, LibGen will apply that value onto the generic supply net and issue a warning message.

Examples

- The following example identifies the `pwrnets.file` as the file containing the names of the power and ground nets:

```
cell_supply_names_file pwrnets.file
```

current_region

```
current_region <PROP [FLAT | HIER]> cell_name net_name layer_name x1 y1 x2 y2 current_value
<cap_value>
```

```
current_region cell_name net_name layer_name current_value <cap_value>
```

This command allows the user to specify an amount of current to be distributed over an area of the power grid.

The input from the `current_region_file` command and the `current_region` command will be combined together. See also [current_region_file](#). If there are any conflicts between the commands, the first one specified will be used and a warning will be generated.

The `current_region` and `current_region file` will override any values calculated by Thunder/Makedpm. See [makedpm](#) chapter of this manual for a discussion on makedpm. The `current_region_default` will only be used for those cells which are not specified by `current_region`, `current_region_file`, or by Thunder/Makedpm.

Parameters

PROP HIER]	[FLAT	If the keyword FLAT is applied, LibGen will not perform hierarchical tracing for the given net in this cell. If the keyword HIER is presented, LibGen will perform hierarchical tracing for the given net in this cell. This provides the ability for LibGen to automatically specify a current value to all instances of a cell in the design hierarchy. The assumption is that all of these instances of the cell will consume the same power. This is used with Quick Detailed View generation. The default is HIER.
<i>cell_name</i>		Specifies the name of the cell. Wildcards can be used in the <i>cell_name</i> .
<i>net_name</i>		Specifies the name of the net. Wildcards can be used in the <i>net_name</i> .

<i>layer_name</i>	Specifies the name of a via or contact layer. Libgen will create a tap current on the conducting layer below this via or contact layer. The tap will be created on the circuit node attached to the via or contact. The value of the current drawn by the tap will be 1/n of the total current assigned to the current region, where n is the number of taps within the current region. The number of taps within a current region is the same as the number of vias or contacts on the specified layer. The <i>layer_name</i> must be a library layer name. If the layer could not be found, LibGen will issue an error message and exit out. The user must provide a valid layer name for LibGen to run. If the user specified a valid layer, but LibGen does not find any via in the assigned current region, LibGen will issue a warning message, but the process will still keep going and exit successfully. In this case, user may get a 0 current value for the specified cell. LibGen will try to attach a current tap onto the conduct layer directly underneath it (which could be regular metal layer, poly layer or diffusion layer). If the conduct layer directly underneath can not be found, LibGen will search for the first metal layer above this via layer (here, the poly layer will not be considered as metal layer), and assign the current tap onto this layer. If no XTC command file is provided, LibGen requires to use VIA_1 or higher as current region layer since contact layer is part of device forming layers. A LibGen automatically generated XTC command file is not able to handle device forming layers very well because of lack of information. Also, use VIA_1 or higher to reduce the resistor/capacitance count, which will improve the overall performance.
<i>x1 y1 x2 y2</i>	Bounding box coordinates for the region to distribute the current. Values are assumed to be in GDSII coordinate units and will be interpreted according to the scale and precision settings contained in the GDSII file that contains the cell. The default unit of <i>x1 y1 x2 y2</i> is in database units.
<i>current_value</i>	Amount of current that will get distributed.
<i>cap_value</i>	Optional capacitance value. Default is zero.

Second form of syntax can be used only when there is only one current region specified for a cell. In this case Libgen can automatically use the bounding box of the cell for the region.

Example

The following line specifies a cell called `an4_80`, a net `grd`, and layer `CONT`. The cell will have 1.0uA of current and 2.0fF of capacitance.

current_region an4_80 gnd CONT 25500 0 28900 2000 1.0uA 2.0fF

current_region_default

current_region_default *layer_name* *current_value* <*cap_value*>

This command provides a default way to create current regions for cells without explicitly listing all the cell and net names by using the `current_region` or `current_region_file` commands. The settings provided by this command are only used for cells which do not have more specific information provided by a `current_region` or `current_region_file` commands. The `current_region_default` will also not override any values calculated by Thunder/Makedpm. See `makedpm` chapter of this manual for a discussion on makedpm.

This command should only be used when the `device_recognition_required` variable is set to `false`.

Parameters

<i>layer_name</i>	Specifies the name of a via or contact layer. The <code>layer_name</code> should be the name of the lowest via or contact layer in the design data. For each power net of any cell which does not have specific <code>current_region</code> data defined, LibGen will place a current tap with the given values at the bottom node of each resistor on the specified layer. Note that this is different than the <code>current_region</code> command, where each current tap received 1/n of the value, based on the number of taps being created for the cell. It is mandatory to specify this parameter when you define multiple contact layers, to avoid any errors due to layer conflict.
<i>current_value</i>	Amount of current that will get applied to each tap. The current value will be inverted when applied to ground nets in order to keep the polarity correct. This is optional. It is not needed in lvs-neutral flow because libgen uses thunder calculated current. Not needed in regular spice netlist flow because it uses thunder calculated current and distributes it evenly.
<i>cap_value</i>	Optional capacitance value. Default is zero. If supplied, the full capacitance value will be applied to each current tap created.

Example

- The following line specifies a `current_region_default` for layer `CONT`. Each tap will have 1.0uA of current and 2.0fF of capacitance.

```
current_region_default CONT 1.0uA 2.0fF
```

current_region_file

```
current_region_file file-name
```

This command allows the user to specify in a file a list of regions and the amount of current to be distributed within them for the power grid.

The input from the `current_region_file` command and the `current_region` command will be combined together. See also [current_region](#). If there are any conflicts between the commands, the first one specified will be used and a warning will be generated.

The `current_region` and `current_region file` will override any values calculated by `Thunder/Makedpm`. See `makedpm` chapter of this manual for a discussion on `makedpm`. The `current_region_default` will only be used for those cells which are not specified by `current_region`, `current_region_file`, or by `Thunder/Makedpm`.

Parameters

<i>file-name</i>	Specifies the name of the file that includes a list of current regions.
------------------	---

Example

- The following line specifies a `current_region_file` called `adder_currents`.

```
current_region_file adder_currents
```

- The following shows the `current_region_file` format

```
UNIT CURRENT default_unit_in_amps
```

```
UNIT CAPACITANCE default_unit_in_farads
```

```
CELL cell_name
```

```
<PROP [FLAT | HIER]>
```

```
NET net_name
```

LAYER *layer_name*

[REGION *x1 y1 x2 y2 current_value <cap_value>*

| REGION *current_value <cap_value>*]

- UNIT

Defines the default scale to be used with the current and capacitance values read from the current region file. These scales apply to any value that is not followed by an explicit unit specification. Valid current values are: A (Amps), mA (miliAmps), uA (microAmps), and nA (nanoAmps). Valid capacitance values are: F(Farads), mF (miliFarads), uF (microfarads), nF (nanofarads), pF (picoFarads), and fF (femtoFarads).

- CELL *cell_name*

This command specifies the name of the cell. All following NET, LAYER, and REGION commands will apply to this CELL until another CELL specification is encountered. Wildcards can be used in the *cell_name*.

- <PROP [FLAT | HIER]>

If the keyword FLAT is applied, LibGen will not perform hierarchical tracing for the given net in this cell.

If the keyword HIER is presented, LibGen will perform hierarchical tracing for the given net in this cell. This provides the ability for LibGen to automatically specify a current value to all instances of a cell in the design hierarchy. The assumption is that all of these instances of the cell will consume the same power.

The default is HIER.

- NET *net_name*

This command specifies that all the LAYER and REGION commands following it apply to the *net_name*. This stays in effect until another NET or CELL specification is encountered. Wildcards can be used in the *net_name*.

- LAYER *layer_name*

This command specifies that all the REGION commands following it apply to the *layer_name*. This stays in effect until another LAYER, NET, or CELL command is encountered.

The *layer_name* must be a library layer name. If the layer could not be found, LibGen will issue an error message and exit out. The user must provide a valid layer name for LibGen to run.

If the user specified a valid layer, but LibGen does not find any via in the assigned current region, LibGen will issue a warning message, but the process will still keep going and exit successfully. In this case, user may get a 0 current value for the specified cell.

- REGION *x1 y1 x2 y2 current_value <cap_value>*

This command specifies that current values and optionally the capacitance value to be associated with the rectangular region specified by the bounding box (*x1 y1 x2 y2*).

Current taps will be created at the locations of the vias or contacts specified by the most recent LAYER command. These current taps will be attached to the layer that is below the specified via or contact layer. Coordinate values are assumed to be in GDSII coordinate units and will be interpreted according to the scale and precision settings contained in the GDSII file that contains the cell. The value of the current drawn by each current tap will be 1/n of the value specified for the current region, where n is the number of taps within the current region.

The second variation of the REGION command without the bounding box is valid only if there is only one current region. In this case LibGen can automatically use the bounding box of the cell for the region.

See also [current_region](#) for a related command.

- Example of current_region_file.

```
UNIT CURRENT uA
```

```
UNIT CAPACITANCE fF
```

```
CELL an4_80
```

```
NET vcc
```

```
LAYER CONT
```

```
REGION 0 14000 28900 17000 3.0 5.0
```

```
NET gnd
```

```
LAYER CONT
```

```
REGION 1.5e-2mA
```

cell_decap

```
cell_decap cellname cap_value
```

This command specifies the amount of decoupling capacitance in a cell. This command is used in generating dynamic port or detailed power-grid views.

If only a port view is created for a cell, the capacitance specified by the `cell_decap` command is evenly distributed over the taps in the port view. If a detailed view is created for this cell, the capacitance values of the taps calculated

during the detailed view generation are scaled up or down as necessary so that the total capacitance of all the taps in the cell matches the value specified by this command. For a related command see [cell_decap_area](#).

Parameters

<i>cellname</i>	Specifies the name of the cell.
<i>cap_value</i>	Specifies the amount of decoupling capacitance associated with the cell, in femtofarads.

Note: If user wants to specify a decap value for a GENERIC supply net, one must specify it explicitly onto the net. If user specifies the decap value for the cell it will be applied on only the POWER and GROUND net. If the GENERIC supply net existed in the cell, LibGen will issue an INFO message to indicate that the value will not be used for the GENERIC supply net. If the user specifies the decap value for the cell, and only a generic supply net exists in the cell, LibGen will apply that value onto the generic supply net and issue a warning message.

Example

- The following example specifies a decoupling capacitance of 5 picofarads for the NTZ3802 cell:

```
cell_decap NTZ3802 5pF
```

derived_layer_list

```
derived_layer_list {layer_name+}
```

This command specifies to LibGen the names of any extra derived layers, which are layers generated in XTC as a result of logical operations that XTC performs on the GDS layers. It is not necessary to use this command under normal circumstances, because LibGen automatically generates this data from the layer maps and the file specified by the `gds_extractor_command_file` variable. Its purpose is to assist in debugging.

Parameters

<i>layer_name</i>	Specifies the names of the extra derived layers.
-------------------	--

Example

- The following line specifies that the DIFF, NIMP, and NDIFF layers are extra derived layers:

```
derived_layer_list {DIFF NIMP NDIFF}
```

discard_detailed_powerview_cell_list

```
discard_detailed_powerview_cell_list {cellname+}
```

This command specifies a list of cells for which to delete the detailed power-grid view from the library. A detailed power-grid view is only deleted if an abstract power-grid view, reduced power-grid view, or both are generated for the same cell, whether or not the cell is included in the list. This command is useful if you are concerned about the amount of disk space required for the power-grid view libraries. However, deleting the detailed power-grid views limits your ability to perform electromigration analysis when you use this library. You cannot perform electromigration analysis on cells that do not have detailed power-grid views.

Parameters

<i>cellname</i>	The name of a cell or a list of cells for which to delete detailed views from the library after an abstract view, reduced view, or both have been generated for the same cell.
-----------------	--

Example

- In the following example, LibGen generates a detailed power-grid view library, then deletes the cell1 and cell2 cells:

```
discard_detailed_powerview_cell_list {cell1 cell2}
```

discard_detailed_powerview_cell_list_file

```
discard_detailed_powerview_cell_list_file filename
```

This command specifies the name of a file containing the list of cells for which to delete the detailed power-grid view from the library. A detailed power-grid view is only deleted if an abstract power-grid view, reduced power-grid view, or both are generated for the same cell, whether or not the cell is included in the list. This command is useful if you are concerned about the amount of disk space required for the power-grid view libraries. However, deleting the detailed power-grid views limits your ability to perform electromigration analysis when you use this library. You cannot perform electromigration analysis on cells that do not have detailed power-grid views.

Parameters

<i>filename</i>	The name of the file containing a single-column list of cells for which to delete detailed views from the library after an abstract view, reduced view, or both have been generated for the same cell.
-----------------	--

Example

- In the following example, LibGen generates a detailed power-grid view library, then deletes the cells listed in the `dv_delete.list` file:

```
discard_detailed_powerview_cell_list_file dv_delete.list
```

foreign_name_map

```
foreign_name_map LEF_cellname GDS_cellname
```

This command allows you to map the foreign cell names specified in the LEF file to the corresponding cell names in the GDS file. This command is only necessary if the foreign names in the LEF file are different from the cell names in the GDS file.

Parameters

<i>LEF_cellname</i>	Specifies the name of the cell in the LEF file.
<i>GDS_cellname</i>	Specifies the name of the cell in the GDS file.

Example

- The following example maps the foreign XAND2 cell in the LEF file to the AND2 cell in the GDS file:

```
foreign_name_map XAND2 AND2
```

foreign_name_map_file

```
foreign_name_map_file filename
```

This command specifies the name of a file containing the foreign cell names in the LEF file and their corresponding cell names in the GDS file. This command is only necessary if the foreign names are different from the cell names in the GDS file.

Parameters

<i>filename</i>	The name of the file containing the list of LEF cell names and their associated GDS cell names.
-----------------	---

Example

- This line indicates that `lef2gds.map` is the name of the file containing foreign LEF cell names and their corresponding GDS cell names:

```
foreign_name_map_file lef2gds.map
```

lef_layer_ignore_list

```
lef_layer_ignore_list {lef_layername+}
```

This command specifies the names of the LEF layers that LibGen is to ignore when it creates the cell library.

Parameters

<code>{lef_layername+}</code>	The names of the LEF layers that LibGen is to ignore.
-------------------------------	---

Example

- As a result of the following example, LibGen ignores the LEF `metal1` and `metal3` layers when it creates the cell library:

```
lef_layer_ignore_list {metal1 metal3}
```

lef_layer_ignore_list_file

```
lef_layer_ignore_list_file {filename}
```

This command specifies the name of a file containing a list of the LEF layers that LibGen is to ignore when it creates the cell library. Each line in the file must consist of a single LEF layer name.

Parameters

<code>filename</code>	The name of the file containing a list of the LEF layers that LibGen is to ignore when it creates the cell library.
-----------------------	---

Example

- The following line indicates that a file named `nolef.txt` contains a list of the LEF layer names that LibGen should ignore:

```
lef_layer_ignore_list_file {nolef.txt}
```

li

```
li layer [gds layernum<datatype> <port> <text_only|no_text>|lefdef lefdef_layername | pgdb
pgdb_layer_name]
```

```
li layer lib second_library_name second_library_layer
```

This command has two functions:

- The `li` command maps between the layer name for local interconnect layers used in the LibGen command file and one of the following:
 - The associated layer's GDSII layer and data type numbers
 - The associated layer's LEF/DEF layer name

You can use this command a number of times for each layer name to assign a different GDSII layer number or LEF/DEF layer name to a cell library layer. Specify LEF/DEF and GDSII layers in separate commands.

This command automatically connects text and ports.

- It maps technology file local interconnect layers when there is a conflict in layer names between two libraries for the same process.

Parameters

First Syntax Variation

<i>layer</i>	Specifies the name of the layer in the cell library.
<i>gds</i>	Maps between the layer specified by <i>layer</i> and a GDSII layer.
<i>layernum</i>	Specifies the layer number of the GDSII layer.
<i>datatype</i>	Specifies the number of the subclass for the GDSII layer. It is optional. If you do not specify a datatype, LibGen uses all datatypes for that GDS layer.
<i>port</i>	Converts text labels in this layer to ports.

<code>text_only</code>	Includes only the text in the named layer and ignores all geometries.
<code>no_text</code>	
<code>lefdef</code>	Maps between the layer specified by <i>layer</i> and a LEF/DEF layer.
<code>lefdef_layername</code>	Specifies the name of the LEF/DEF layer.
<code>pgdb_layer_name</code>	Specifies the name of the pgdb layer.

Second Syntax Variation

<code>layer</code>	Specifies the name of the layer in the first cell library.
<code>second_library_name</code>	Specifies the name of second library.
<code>second_library_layer</code>	Specifies the name of the second library layer.

Example

- The following line maps between the `LI` layer in the LibGen command file and GDSII layer 49:
`li LI gds 49`
- The next example would map the `local_1` layer of `blocks.cl` library to the `LI_1` layer present in the RCgen techfile provided as input to the LibGen run. It would also map the `li_1` layer from the `stdcells.cl` library to the `LI_1` layer.

```
li LI_1 lib blocks.cl local_1
```

```
li LI_1 lib stdcells.cl li_1
```

pgv_header_map

```
pgv_header_map cell_name net_name header_name
```

This command specifies how LibGen uses data generated from VoltageStorm, VoltageStorm PE, or Lightning to create a detailed power-grid view. It uses the data in the power-grid database referenced by the header name to create a detailed power-grid view for the specified net of the specified cell. The header files used in this command are created by the `generate_view` command in VoltageStorm and VoltageStorm PE or by the `prepare_view_generation` command in Lightning. These header files typically end with the `.vhdr` extension.

Parameters

<i>cell_name</i>	Specifies the name of the LEF cell input to LibGen.
<i>net_name</i>	Specifies the name of the net generated in the power-grid view.
<i>header_name</i>	Specifies the name of the VoltageStorm, VoltageStorm PE, or Lightning header file referencing the name of the power-grid database to use in generating the power-grid view.

Example

- The following example specifies that LibGen use the data in the CELL2 cell and in the power-grid database referenced by the header file named `work/VDD_CELL2.vhdr` to create a power-grid view. The VDD net will be generated in this power-grid view.

```
pgv_header_map CELL2 VDD work/VDD_CELL2.vhdr
```

pgv_header_map_file

```
pgv_header_map_file {filename}
```

This command is similar to the `pgv_header_map` command, but it reads the data for cell, net, and header from a given file.

Each line of this file should contain a cell name, a net name, and a header file name.

Parameters

<i>filename</i>	The name of the file containing the cell name, net name, and header file name required to create a power-grid view.
-----------------	---

Example

- The following command specifies a header map file named `NFD4Q_pgv`:

```
pgv_header_map_file NFD4Q_pgv
```

This file might contain these lines:

```
CELL2 VDD work/VDD_CELL2.vhdr
```

```
CELL2 VSS work/VSS_CELL2.vhdr
```

```
CELL3 VSS work/VDD_CELL3.vhdr
```

```
CELL3 GND work/GND_CELL3.vhdr
```

poly_command

```
poly layer [gds layernum<datatype> <port> <text_only|no_text>|lefdef lefdef_layername | pgdb  
pgdb_layer_name]
```

```
poly layer lib second_library_name second_library_layer
```

This command has two functions:

- The `poly` command maps between the layer name for polycide layers used in the LibGen command file and one of the following:
 - The associated layer's GDSII layer and datatype numbers
 - The associated layer's LEF/DEF layer nameYou can use this command a number of times for each layer name to assign a different GDSII layer number or LEF/DEF layer name to a cell library layer. Specify LEF/DEF and GDSII layers in separate commands.

This command automatically connects text and ports.

- It maps technology file poly layers when there is a conflict in layer names between two libraries for the same process.

Parameters

First Syntax Variation

layer	Specifies the name of the layer in the cell library.
<i>gds</i>	Maps between the layer specified by <i>layer</i> and a GDSII layer.
<i>layernum</i>	Specifies the layer number of the GDSII layer.
<i>datatype</i>	Is an optional number of the subclass for the GDSII layer. If you do not specify a datatype, LibGen uses all datatypes for that GDS layer.
<i>port</i>	Converts text labels in this layer to ports.
<i>text_only</i>	Includes only the text in the named layer and ignores all geometries.
no_text	
lefdef	Maps between layer specified by <i>layer</i> and a LEF/DEF layer.
<i>lefdef_layername</i>	Specifies the name of the LEF/DEF layer
<i>pgdb_layer_name</i>	Specifies the name of the pgdb layer. See examples and the end of this section

Second Syntax Variation

<i>layer</i>	Specifies the name of the layer in the first cell library.
<i>second_library_name</i>	Specifies the name of second library.
<i>second_library_layer</i>	Specifies the name of the second library layer.

Example

- The following line maps between the `POLYCIDE` layer in the LibGen command file and GDSII layer 46:

```
poly POLYCIDE gds 46
```

The first line of the following example maps the POLY01 layer in LibGen command file to the LEF/DEF layer POLY_1. The second line maps the POLY01 layer in LibGen command to pgdb poly1 layer. This is an example where some cells were done using VoltageStorm Cell-Level (lefdef) while others were done using VoltageStorm Transistor-Level (pgdb) and both poly names needed to be mapped.

```
poly POLY01 lefdef POLY_1
```

```
poly POLY01 pgdb poly1
```

- The next example would map the `poly_1` layer of blocks.cl library to the `POLY_1` layer present in the RCgen

techfile provided as input to the LibGen run. It would also map the P_1 layer from the stdcells.cl library to the POLY_1 layer.

```
metal POLY_1 lib blocks.cl poly_1
```

```
metal POLY_1 lib stdcells.cl P_1
```

port_power_gate_file

```
port_power_gate_file filename
```

This command specifies a file that contains all the information that LibGen needs to create a port power gate view. This includes the names of the power gate cells, the names of the pins in the power gate cells, the on-resistance of the power gate, and the saturation current of the power gate.

Parameters

filename	Specifies the name of the text file that defines the port power gate information.
----------	---

File Format

The following is the format for a port power gate file.

Simplest form

```
cell cellname supply unswitched_pin_name switched switched_pin_name ron r_value <idsat idsat_value>  
<ileak ileak_value>
```

For more complex power gate cells that have multiple switched pins, the cell name is specified first on a line by itself and then the data for the individual power gates is specified.

```
cell cellname  
  
supply pin1 switched pin2 ron r_value idsat idsat_value ileak leak_value  
  
...
```

All resistance values are in ohms and current values in milliamps if not specified.

Example

- The following example specifies the port power gate file called `powergates.txt`:

```
port_power_gate_file powergates.txt
```

- Example of port power gate file contents:

```
cell hs65_ls_sw3l60 supply vddalways switched vddswitch ron 3.5
```

```
cell pgl
```

```
supply vdd switched vdd2 ron 3.5 idsat 15ma ileak 0.1ma
```

```
...
```

port_view_current_file

```
port_view_current_file filename
```

This command specifies the name of the port view current file, which contains current distribution information that LibGen can use when it generates port power-grid views. In the floorplanning stage of the design, you may need to distribute current unevenly among the ports. The standard LibGen flow for generating port power-grid views generates a port power-grid view based only on LEF data and assigns each interface node the same unit tap current value. When you know roughly how the current is going to be distributed, you can use a port view current file to specify the current values to assign to the pins to more accurately reflect the current distribution. LibGen distributes the currents automatically when it generates the port power-grid views.

Parameters

<i>filename</i>	Specifies the name of the port view current file.
-----------------	---

File Format

The port view current file is in the following format:

```
CELL cell_name1
```

```
PIN pin_name
```

```
lef_layer_name x y current_value/current_distribution_ratio
```

```
CELL cell_name2
```

```
PIN pin_name
```

```
lef_layer_name x y current_value/current_distribution_ratio
```

...

- *cell_name*
Specifies the name of the macro found in the LEF file.
- *pin_name*
Specifies the name of pin to which the current is being distributed.
- *lef_layer_name*
Specifies the name of the LEF layer on which the current should be placed.
- *x, y*
Are the coordinates within the specific port area (partition). These values are specified in microns and use the same coordinate system as the LEF port data.
- *current_value* or *current_distribution_ratio* (based on the use model)
Specifies the current value in amperes, or the current distribution ratio (current distributed on different partitions of a LEF port). This value is a floating-point number. You can specify the current value in milliamperes (m), microamperes (μ), or nanoamperes (n).

Lines beginning with the pound sign (#) in the port view current file are comment lines.

Examples

- The following examples specifies a port view current file called `NA310port_pgv.txt`:
`port_view_current_file NA310port_pgv.txt`
- As an example of the port view current file, suppose the VDD rail of the AN2 cell has three sets of connected port geometries (partitions) on the upper, middle, and lower portions of a layout. You can create a port view current file such as the following to specify the current distribution:

```
#Current distribution data for cell AN2
```

```
CELL      AN2
PIN       VDD
M1        0.0    10.0    1m
M2        0.0     5.0    2u
M3        0.0     0.0    1.5n
```

After LibGen generates the interface nodes, it assigns 1 milliampere to the interface nodes on first partition, 2 microamperes to the second partition, and 1.5 nanoamperes to the third partition. In each partition, the current value is divided evenly among the interface nodes. For pins in which no ports were specified as having specific current values, all ports are assigned the default current.

- The following is an example of a port view current file with 2 cells:

```
CELL CELL1
PIN VDD
```



```
M0 0.0285 0.24 1.09222e-05
PIN VDD
M0 0.0285 0.72 1.06755e-05
PIN VDD
M0 0.0285 1.68 0.000178628

CELL CELL2
PIN VDD
M0 0.0285 0.24 1.09222e-05
PIN VDD
M0 0.0285 0.72 1.06755e-05
PIN VDD
M0 0.0285 1.68 0.000178628
```

scale_current_region

`scale_current_region minx miny maxx maxy scalevalue`

Provides a current scaling factor for a specific region.

Parameters

<i>minx miny maxx maxy</i>	Specifies the coordinates of the current region that the scale factor will be applied to. Units are in Angstroms
<i>scalevalue</i>	Specifies a scale factor. The factor will be applied to the specified region . If one wants to specify a quiet region (one with zero current) then the scale factor should be set to 0. Scaling is applied to all the nodes falling in the region and currents reported by thunder are multiplied by the scale factor. The scale factor can be 0 or any positive number You can also specify a default scaling factor using the scale_current_region_default command.

Examples

- The below command specifies a scale factor of 2 for the region defined by the coordinates provided.

```
scale_current_region 0 0 70000 70000 2
```

scale_current_region_default

`scale_current_region_default scalevalue`

.Specifies a current scaling factor for everything that is not defined by [scale_current_region](#) command.

Parameters

<code>scalevalue</code>	Specifies a global scale factor. If global scaling is used, region specific scaling would override global scaling for that region.
-------------------------	--

Examples

- The below command specifies a scale factor of 3 for the any region not defined the `scale_current_region` command.

```
scale_current_region_default 3
```

thunder_model_table_directory

`thunder_model_table_directory directory_name`

This command specifies the location of the set of Thunder model tables that LibGen uses to generate the current data for the cells in the library.

Parameters

<code>directory_name</code>	The name of the directory containing the Thunder model tables
-----------------------------	---

Example

- The following example identifies the location of the Thunder model tables:

```
thunder_model_table_directory ../tables/typical_1.2
```

```
thunder_model_table_directory ../tables/typical_2.5
```

well

```
well layer [gds layernum <datatype> <port> <text_only|no_text>|lefdef lefdef_layername | pgdb
pgdb_layer_name]
```

```
well layer lib second_library_name second_library_layer
```

This command has two functions:

- The `well` command maps between the layer name for well layers used in the LibGen command file and either:
 - The associated layer's GDSII layer and datatype numbers
 - The associated layer's LEF/DEF layer name
 You can use this command a number of times for each layer name to assign a different GDSII layer number or LEF/DEF layer name to a cell library layer. Specify LEF/DEF and GDSII layers in separate commands.

This command automatically connects text and ports.

- It maps technology file well layers when there is a conflict in layer names between two libraries for the same process.

Parameters

First Syntax Variation

<code>layer</code>	Specifies the name of the layer in the cell library.
<code>gds</code>	Maps between the layer specified by <code>layer</code> and a GDSII layer.
<code>layernum</code>	Specifies the layer number of the GDSII layer.
<code>datatype</code>	Is an optional number of the subclass for the GDSII layer. If you do not specify a datatype, LibGen uses all datatypes for that GDS layer.
<code>port</code>	Converts text labels in this layer to ports.
<code>text_only</code>	Includes only the text in the named layer and ignores all geometries.
<code>no_text</code>	
<code>lefdef</code>	Maps between layer specified by <code>layer</code> and a LEF/DEF layer.
<code>lefdef_layername</code>	Specifies the name of the LEF/DEF layer

<i>pgdb_layer_name</i>	Specifies the name of the pgdb layer. See examples and the end of this section
------------------------	--

Second Syntax Variation

<i>layer</i>	Specifies the name of the layer in the first cell library.
<i>second_library_name</i>	Specifies the name of second library.
<i>second_library_layer</i>	Specifies the name of the second library layer.

Example

- The following line maps between the `N_WELL` layer in the LibGen command file and GDSII layer 42:
`well N_WELL gds 42`
- The next example would map the `well_1` layer of `blocks.cl` library to the `WELL_1` layer present in the RCgen techfile provided as input to the LibGen run. It would also map the `w_1` layer from the `stdcells.cl` library to the `WELL_1` layer.

```
well WELL_1 lib blocks.cl well_1
```

```
well WELL_1 lib stdcells.cl w_1
```

well_cap_area

```
well_cap_area { cell_name_list } cap_value <unit>
```

Specifies the amount of area based well capacitance in a cell. This command is used in generating dynamic port or detailed power-grid views. Just like the [well_cap](#) command, if only a port view is created for a cell, the capacitance specified by the `well_cap_area` command is evenly distributed and added to the decap taps in the port view. The capacitance specified will be used for both the POWER and GROUND nets. If there are multiple POWER nets or GROUND nets, this value will be evenly distributed among those nets. For a related command see [well_cap_area_file](#).

Parameters

<pre>{ cell_name_list }</pre>	Specifies a list of cells.
---	----------------------------

cap_value
<unit>

Specifies the amount of well capacitance associated with the cell, in femtofarads per square micron. All capacitance values can be followed by a unit specification. The following units are recognized: millifarads (mF), microfarads (μ F), nanofarads (nF), picofarads (pF), femtofarads (fF).

Example

- The following example specifies a well capacitance of 1 picofarads per square micron for the NTZ3802 cell:

```
well_cap_area NTZ3802 1pF
```

well_cap_area_file

well_cap_area_file filename

This command specifies the name of a file containing a list of cells and the area based well capacitance. For a related command see [well_cap_area](#).

Parameters

filename

The name of the well cap area file, which is described in below.

File Format

The well capacitance area file format:

The lines of the well capacitance area file can be of one of the following formats:

- `CAP_UNIT unit_in_farads`

The `CAP_UNIT` definition is optional. If no `CAP_UNIT` statement is present at the beginning of the file, all capacitance values not followed by a unit specification are assumed to be in femtofarads.

- `LENGTH_UNIT unit_in_meters`

The `LENGTH_UNIT` definition is optional. If no `LENGTH_UNIT` statement is present at the beginning of the file, all length values not followed by a unit specification are assumed to be in microns.

- `CELL cell_name cap_value`

cell_name specifies the name of the cell. *cap_value* specifies the amount of area based well capacitance.

The default unit is femtofarads per square micron. The cell name can contain wildcard characters (e.g. * ?).

All capacitance values can be followed by a unit specification. The following units are recognized: millifarads (mF), microfarads (μ F), nanofarads (nF), picofarads (pF), femtofarads (fF). If units are specified, they will take

priority over the `CAP_UNIT` specification.

Examples

- The following example specifies a file named `wellcapsareacelllist` that contains a list of cells and the area based well capacitance for each.

```
well_cap_area_file wellcapsareacelllist
```

- The following example shows the contents of an area based well capacitance file:

```
CAP_UNIT 1.0e-15
```

```
CELL Cell_A 100.00
```

The first line in the well capacitance file is optional. It assumes that the default units are femtofarads. The second line specifies the area based well capacitance for `Cell_A` to be 100 femtofarads per square micron. The capacitance specified will be used for both the POWER and GROUND nets. If there are multiple POWER nets or GROUND nets, this value will be evenly distributed among those nets.

well_cap

```
well_cap cellname cap_value
```

This command specifies the amount of well capacitance in a cell. This command is used in generating dynamic port or detailed power-grid views.

In both the port and detailed views, the capacitance specified by the `well_cap` command is evenly distributed over the taps and appended onto the existign C1 for each tap. If a detailed view is created for this cell, the capacitance values of the taps calculated during the detailed view generation are scaled up or down as necessary so that the total capacitance of all the taps in the cell matches the value specified by this command.

Parameters

<i>cellname</i>	Specifies the name of the cell.
<i>cap_value</i>	Specifies the amount of well capacitance associated with the cell, in femtofarads. All capacitance values can be followed by a unit specification. The following units are recognized: millifarads (mF), microfarads (μF), nanofarads (nF), picofarads (pF), femtofarads (fF)

Example

- The following example specifies a well capacitance of 1 picofarads for the NTZ3802 cell:

```
well_cap NTZ3802 1pF
```

Library Generator Variables

assume_foreigns

```
setvar assume_foreigns [true|false]
```

This variable enables LibGen to process LEF files in which `FOREIGN` statements are not defined as part of the macro definition. When generating detailed power-grid views, LibGen assumes that `FOREIGN` statements are defined as part of the macro definition in the LEF file. When this variable is set to `true`, LibGen processes the LEF file by using the name defined by the `MACRO` statement rather than requiring a `FOREIGN` statement.

The `assume_foreigns` variable ensures that the `FOREIGN` statement is placed at 0,0. Therefore, if the `MACRO` statement has an offset, you must provide a `FOREIGN` statement that contains an offset. This variable works for offsets that are 0,0 or the inverse of the origin. See [assume_foreigns_mode](#) for setting it to the inverse.

Parameters

[true false]	When set to <code>true</code>, LibGen searches the GDS file for the name defined by the <code>MACRO</code> statement in the LEF file. If set to <code>false</code>, LibGen cannot process the LEF file and issues an error message. The default is <code>false</code>.
--------------	---

Example

- The following example allows LibGen to process a LEF file for which `FOREIGN` statements are not defined as a part of the macro definition:

```
setvar assume_foreigns true
```

assume_foreigns_mode

```
setvar assume_foreigns_mode [0|1]
```

This variable determines how LibGen processes cells for which `FOREIGN` statements are assumed.

Parameters

[1 0]	If set to 0, LibGen assumes that the cell for which <code>FOREIGN</code> statements are assumed is placed at (0,0). When you set it to 1, LibGen assumes that the <code>FOREIGN</code> cell is placed at the inverse of the macro origin. The default is 1.
-------	---

When you set this variable to 0, LibGen assumes that the cell for which `FOREIGN` statements are assumed is placed at (0,0). When you set it to 1, LibGen assumes that the `FOREIGN` cell is placed at the inverse of the macro origin. The default is 1.

Example

- The following example specifies that `FOREIGN` statements are to be assumed for cells with no `FOREIGN` statements and that the `FOREIGN` cells should be placed at the inverse of the origin:

```
setvar assume_foreigns true
```

```
setvar assume_foreigns_mode 1
```

- In the next example, `FOREIGN` statements are to be assumed for cells with no `FOREIGN` statements, and the `FOREIGN` cells should be placed at (0,0):

```
setvar assume_foreigns true
```

```
setvar assume_foreigns_mode 0
```

case_sensitivity_default

```
setvar case_sensitivity_default [true|false]
```

This variable determines whether LibGen interprets the data in the LEF file as case-sensitive or case-insensitive if the first LEF file does not include a `NAMESCASESENSITIVE` statement. Normally, LibGen reads the case-sensitivity setting for data such as the layer names, cell names, pin names, and so forth from the `NAMESCASESENSITIVE` statement in the first LEF file that it processes.

Parameters

<code>[true false]</code>	When this variable is set to <code>true</code>, LibGen interprets the data in the LEF file as case-sensitive, if the file does not contain a <code>NAMESCASESENSITIVE</code> statement. When this variable is set to <code>false</code>, LibGen interprets the data in the LEF file as case-insensitive, if the file does not contain a <code>NAMESCASESENSITIVE</code> statement. The default is <code>true</code>.
---------------------------	---

Example

- In the following example, LibGen handles the data in the LEF file as case-sensitive since the file does not contain a `NAMESCASESENSITIVE` statement:

```
setvar case_sensitivity_default true
```

cell_accura_data_file

```
setvar cell_accura_data_file filename
```

This variable specifies the name of a file containing data about the electrical characteristics of cells. Thunder requires this data to generate currents during power-grid view generation.

Parameters

<i>Filename</i>	The name of the file containing the electrical characteristics data.
-----------------	--

Example

- The file containing the electrical data about the cell is called `cell_char.file` in the following example:

```
setvar cell_accura_data_file cell_char.file
```

cmd_data_case_sensitive

```
setvar cmd_data_case_sensitive [true|false]
```

This variable determines whether LibGen handles data in the LibGen command file as case-sensitive or case-insensitive.

Since this variable directly affects the way that LibGen interprets the data in the command file, it is position-dependent. Place it as the first line in the command file if you want LibGen to handle the command data as case-

insensitive.

Note: Command and variable names are always case-insensitive. The `cmd_data_case_sensitive` variable only applies to the data specified by commands or variables. Currently this only applies to:

```
generic_power_names
generic_ground_names
cell_power_names
cell_ground_names
generic_bulk_power_names
generic_bulk_ground_names
cell_decap
cell_decap_file
source_location_file
pgv_header_map
pgv_header_map_file
```

Parameters

[true false]	When this variable is set to <code>true</code>, LibGen interprets the data in the command file as case-sensitive. If it is set to <code>false</code>, LibGen interprets the data as case-insensitive. The default is <code>true</code>.
--------------	--

Example

- In the following example, LibGen interprets the data in the LibGen command file as case-insensitive:

```
setvar cmd_data_case_sensitive false
```

collapsed_view_current_threshold

```
setvar collapsed_view_current_threshold threshold
```

This variable provides guidance to LibGen when creating collapsed powerviews.

Parameters

<code>threshold</code>	Defines a value that will be used for determining which current taps get discarded, when creating a collapsed powergrid view. Valid values are between 0 and 0.2 (0 and 20%). The default is 0.05 (5%).
------------------------	--

Note: This collapsed power grid view was created for tool performance reasons. Current designs in many cases contain an embedded memory for boosting performance. The bit cells of the memory in general contribute to creation of a lot of current taps, however, they do not contribute to the IR much, but will affect the tool performance. This collapsed view suppresses those current taps which have a very small current value thus improving both memory usage and performance of the tool.

Example

- In the following example sets the current threshold to 0.1 when generating a collapsed powerview:

```
setvar collapsed_view_current_threshold 0.1
```

create_dummy_tech_file

```
setvar create_dummy_tech_file [true|false]
```

This variable generates a technology file using LEF technology data. You must set this variable when a technology file is unavailable for generating port power-grid views.

This variable is reserved for specialized flows and at no time should be used to generate graybox data for capacitance extraction or to generate detailed power-grid view libraries.

Parameters

<code>[true false]</code>	If this variable is set to <code>true</code>, LibGen generates a technology file for the library layers listed in the LEF layer map section in the LibGen command file. Setting this variable to <code>false</code> requires that a technology file be specified by using the <code>tech_file</code> variable. For a complete definition of the <code>tech_file</code> variable, refer to "tech_file". The default is <code>false</code>.
---------------------------	--

Example

- The following example indicates that a technology file is to be generated by using the LEF layers mapped from the LibGen command file and the technology data from the layer definition section in the LEF file:

```
setvar create_dummy_tech_file true
```

database_precision

```
setvar database_precision number
```

This variable specifies the number of internal database units corresponding to one micron. All coordinates in the cell libraries are stored in 32-bit integers, and the setting of this variable affects the minimum feature size that can be resolved. The default setting of this variable is 10000, which means that 1 micron (1 LEF unit) corresponds to 10000 units in the cell library. Each cell library unit therefore represents 1e-10 meter, or one angstrom.

Some common valid values for the database precision setting, other than the default value of 10000, would be 1000 or 100. The magnitude of the maximum coordinate value in the LEF data multiplied by the factor specified by `database_precision` must be less than 2,147,483,647. For example, if a maximum LEF coordinate is 248304, a `database_precision` factor of 1000 would be appropriate, because $248304 \times 1000 = 248,304,000$, which is less than the maximum value of 2,147,483,647.

Under normal circumstances, you do not need to change the `database_precision` setting from the default setting. However, for certain applications or if you are designing in lambda units, where 1 LEF unit != 1 micron, you may want to set this variable to another value to prevent internal overflow. It is recommended that you reduce the setting from its default value by factors of 10 as necessary to get the data to fit into the appropriate range, but no further.

If you change the setting of `database_precision` to a non-default value, you must place the variable near the beginning of the LibGen command file, before any other commands that input coordinate or layout data, such as the `source_location_file` command or the `port_view_current_file` command.

Parameters

<i>Number</i>	The data base precision. It must be a positive number greater than 0
---------------	---

Example

- The following example sets the number of internal database units corresponding to one micron at 1000:

```
setvar database_precision 1000
```

debug_skip_parasitic_extraction

```
setvar debug_skip_parasitic_extraction [true|false]
```

This variable specifies that the parasitic extraction step should be skipped. This forces the skipping of the parasitic extraction step when rerunning LibGen. See other `debug_*` commands for skipping other processing steps.

Parameters

[true false]	When this variable is set to <code>true</code> , the parasitic extraction step is skipped. The default is <code>false</code> .
--------------	---

Example

- The following line specifies that the parasitic extraction step should be skipped.

```
setvar debug_skip_parasitic_extraction true
```

debug_skip_thunder

```
setvar debug_skip_thunder [true|false]
```

This variable specifies that the thunder step should be skipped. This forces the skipping of the thunder step when rerunning LibGen. See other `debug_*` commands for skipping other processing steps.

Parameters

[true false]	When this variable is set to <code>true</code> the thunder step is skipped. The default is <code>false</code> .
--------------	--

Example

- The following line specifies that the thunder step should be skipped.

```
setvar debug_skip_thunder true
```

debug_skip_xtc

```
setvar debug_skip_xtc [true|false]
```

This variable specifies that the xtc step should be skipped. This forces the skipping of the xtc step when rerunning

LibGen. See other `debug_*` commands for skipping other processing steps.

Parameters

<code>[true false]</code>	When this variable is set to <code>true</code> , the xtc step is skipped. The default is <code>false</code> .
---------------------------	--

Example

- The following line specifies that the xtc step should be skipped.

```
setvar debug_xtc_fire true
```

default_ground_voltage

```
setvar default_ground_voltage value
```

This variable specifies the default return voltage value used for ground nets during power-grid view generation.

Parameters

<code>value</code>	This variable sets the ground value, in volts. The default is 0.0 volts
--------------------	--

Example

- The following example sets the default value for the ground nets to 0.0 volts:

```
setvar default_ground_voltage 0.0
```

default_lef_min_spacing

```
setvar default_lef_min_spacing value
```

This variable specifies the default spacing value used for all metal layers for which a spacing rule is not defined in the LEF file.

This variable is valid only when you set the `create_dummy_tech_file` variable to `true`.

This variable is reserved for specialized flows and at no time should be used to generate graybox data for capacitance extraction or to generate detailed power-grid view libraries.

Parameters

<i>value</i>	This variable sets the default spacing value, in microns. The default spacing is 0.18 microns.
--------------	---

Example

- The following example indicates that for all metal layers that do not have a spacing rule specified in the LEF file, the `default_lef_min_spacing` used will be 0.20 microns:

```
setvar default_lef_min_spacing 0.20
```

default_lef_min_width

```
setvar default_lef_min_width value
```

This variable specifies the default width value that is used for all metal layers that do not have a width value defined in the LEF file.

This variable is valid only when you set the `create_dummy_tech_file` variable to `true`.

This variable is reserved for specialized flows and at no time should be used to generate graybox data for capacitance extraction or to generate detailed power-grid view libraries.

Parameters

<i>value</i>	Specifies the default width value, in microns, to use for metal layers that do not have a value specified in the LEF file. The default width is 0.18 microns.
--------------	--

Example

- The following example indicates that for all metal layers that do not have a width specified in the LEF file, the default width value used will be 0.19 microns:

```
setvar default_lef_min_width 0.19
```

default_power_voltage

```
setvar default_power_voltage value
```

This variable specifies the default power voltage value used for power nets during power-grid view generation.

Parameters

<i>value</i>	Specifies the default power value, in volts. The default value is 1.2 volts
--------------	--

Example

- The following example sets the default value for the power nets to 1.0 volts:

```
setvar default_power_voltage 1.0
```

default_via_resistance

```
setvar default_via_resistance value
```

This variable specifies a default resistance value to use for LEF vias for which no resistance is specified in the LEF file.

Parameters

<i>value</i>	Specifies a default via resistance value, a floating-point number, in ohms. The default value is 5 ohms
--------------	--

Example

- The following example specifies a default resistance value of 4.5 ohms for LEF vias:

```
setvar default_via_resistance 4.5
```

disable_lef_parser_escape_check

```
setvar disable_lef_parser_escape_check [true|false]
```

This variable determines whether LibGen retains the escape characters (\) in the names in the LEF file when it reads this file.

Parameters

[true false]	When this variable is set to <code>true</code>, LibGen retains the escape characters in the LEF file names when it reads the file. When this variable is set to <code>false</code>, LibGen removes these escape characters. The default is <code>true</code>.
--------------	--

Example

- As a result of the following example, LibGen removes the escape characters from the LEF file names:

```
setvar disable_lef_parser_escape_check false
```

do_optimized_views

```
setvar do_optimized_views [true|false]
```

This variable determines whether LibGen optimizes the views of standard cells.

Parameters

[true false]	When this variable is set to <code>true</code>, LibGen optimizes the views of standard cells. When this variable is set to <code>false</code>, LibGen does not optimize these views. The default is <code>true</code>.
--------------	---

Example

- In the following example, LibGen does not optimize the standard cell views:

```
setvar do_optimized_views false
```

do_rextract_in_parallel

```
setvar do_rextract_in_parallel [true|false]
```

This variable determines whether LibGen performs a resistance extraction in parallel over the network by using the

parallel script or performs the extraction on the local machine.

Parameters

[true false]	When this variable is set to <code>true</code>, LibGen invokes the parallel script to run the resistance extraction. When this variable is set to <code>false</code>, LibGen extracts all cells on a local machine. The default is <code>false</code>.
--------------	---

Example

- This line specifies that LibGen perform the resistance extraction in parallel over the network:

```
setvar do_reextract_in_parallel true
```

emulate_lef_parsing

```
setvar emulate_lef_parsing [true|false]
```

This variable determines the algorithm that LibGen uses to create port geometries in the cell library.

Parameters

[true false]	When this variable is set to <code>true</code>, LibGen calculates the port geometries internally rather than invoking XTC to calculate them. When this variable is set to <code>false</code>, LibGen invokes XTC to calculate the port geometries. The default is <code>true</code>.
--------------	---

Example

- In the following example, LibGen calculates the port geometries internally:

```
setvar emulate_lef_parsing true
```

gds_cell_extension

```
setvar gds_cell_extension extension
```

This variable overrides the extension appended to GDS cell names when they are stored in the library.

Parameters

<i>extension</i>	This variable overrides the default extension that is appended to GDS cell names when they are stored in the library. If you do not specify an extension, LibGen uses a default of <code>_sg</code>. Normally, you should not have to set this variable.
------------------	--

Example

- The following example appends an extension of `_gds` to the GDS cell names:

```
setvar gds_cell_extension _gds
```

gds_extractor

```
setvar gds_extractor executable_name
```

This variable identifies the executable used as a GDS extractor for LibGen.

Parameters

<i>executable_name</i>	This variable identifies the executable used as a GDS extractor for LibGen. If it is not specified, LibGen defaults to XTC.
------------------------	---

Example

- The following example sets `/home/bin/xtc` as the GDS extractor executable:

```
setvar gds_extractor /home/bin/xtc
```

gds_extractor_command_file

```
setvar gds_extractor_command_file filename
```

This variable specifies the name of the command file to be submitted to the GDS extractor. This file includes the layer-map and device-forming commands required by XTC. LibGen verifies that this file is valid when checking the LibGen command file.

Parameters

<i>filename</i>	The name of the command file for the GDS extractor.
-----------------	---

Example

- In the following example, `xtc.cmd` is the command file submitted to the GDS extractor:

```
setvar gds_extractor_command_file xtc.cmd
```

`gds_port_explosion_depth`

```
setvar gds_port_explosion_depth depth
```

This variable determines how deeply LibGen traverses the structural GDS hierarchy to calculate the port geometries.

Parameters

<i>depth</i>	The level of structural GDS hierarchy that LibGen traverses to calculate the port geometries. The default is 0.
--------------	--

When `emulate_lef_parsing` is set to `true`, and you specify a hierarchy depth with `gds_port_explosion_depth`, LibGen explodes the port geometries through the hierarchy. For example, suppose that the VDD net port geometries are defined in the annotated Assura GDS file for structure A, as well as for structure B. B is instantiated in A as a structural reference. When you set `gds_port_explosion_depth` to 1, only the port geometries in A will be part of the VDD net in the cell library. If you set this variable to 2, the port geometries in B will also go into the cell library with transformation taken into account. If you set this variable to 0, LibGen traverses every leaf of this hierarchy to calculate the port geometries. There is no negative effect if *depth* is larger than the maximum depth of a design.

If `emulate_lef_parsing` is set to `false`, the `gds_port_explosion_depth` variable has no effect.

Example

- In the following example, LibGen descends to a maximum of three levels in the GDS hierarchy, starting from the top structure, while computing the port geometries:

```
setvar emulate_lef_parsing true

setvar gds_port_explosion_depth 3
```

gds_to_lef_gds_extractor_command_file

```
setvar gds_to_lef_gds_extractor_command_file filename
```

This variable specifies the name of the command file to use when you run a Synopsys LEF/GDSII flow. It is optional.

Parameters

<i>filename</i>	The name of the command file used to run the Synopsys LEF/GDSII flow.
-----------------	---

The Synopsys LEF/GDSII flow processes data in two passes. The first pass generates LEF output for any cell that does not already have a LEF description. The second pass generates the netlist and stripes to be used for the analysis.

Because of this two-pass process, you may need two separate XTC command files to reduce the total run time.

LibGen uses the optional command file specified by the `gds_to_lef_gds_extractor_command_file` variable during the first pass to reduce run time when generating LEF from GDS. For example, if the GDS file does not contain the complete port geometry at the top level, you would set the XTC `extract_flat` variable to `true`. If you continued to use the XTC command file with the `extract_flat` variable set to `true` during the second pass, you would unnecessarily increase the total run time, which could be significant.

If you do not specify a command file for this variable and must generate LEF, LibGen uses the command file specified by the `gds_extractor_command_file` variable for both passes.

Example

- The following example specifies a command file named `transform_lef.cmd` for the Synopsys LEF/GDSII flow:

```
setvar gds_to_lef_gds_extractor_command_file transform_lef.cmd
```

generated_gds_file

```
setvar generated_gds_file <filename>
```

This variable overrides the name of the GDS file that LibGen generates when creating holding cells.

Parameters

<i>filename</i>	The name of the GDS file. This file will be placed in the current working directory. If you do not specify a file name, LibGen uses the default value of <code>LibgenGeneratedForeigns.gds</code>. Normally, you should not have to set this variable and a file called <code>LibgenGeneratedForeigns.gds</code> will be placed in the <code>library_name_temp</code> directory.
-----------------	--

Example

- The following example specifies a GDS file named `lx_cell.gds` which will be placed in the current working directory:

```
setvar generated_gds_file lx_cell.gds
```

halt_on_missing_foreign_data

```
setvar halt_on_missing_foreign_data [true|false]
```

This variable aborts processing if LibGen encounters a `FOREIGN` statement referencing a cell that is not included in the GDSII files submitted as input.

Parameters

[true false]	If this variable is set to <code>true</code>, LibGen stops processing if it finds a reference to a GDSII cell that is not identified in the GDSII input files. If this variable is set to <code>false</code>, LibGen continues processing but issues warning messages. The default is <code>false</code>.
--------------	--

Example

- The next line causes LibGen to stop processing if it finds a reference to a GDSII cell that is not identified in the GDSII input files:

```
setvar halt_on_missing_foreign_data true
```

halt_on_undefined_lef_layer

```
setvar halt_on_undefined_lef_layer [true|false]
```

This variable controls how LibGen behaves while it parses the LEF file. If a layer referenced in the LEF file is not defined in the layer map in the LibGen command file and this variable is set to `true`, LibGen aborts processing. If this variable is set to `false`, LibGen issues a warning about the undefined layer and continues processing.

Parameters

[true false]	If this variable is set to <code>true</code>, LibGen stops processing if it finds an undefined LEF layer in the layer map section of the command file. If this variable is set to <code>false</code>, LibGen continues processing but issues a warning message. The default is <code>false</code>.
--------------	---

Example

- In the following example, LibGen stops processing if it finds an undefined LEF layer in the layer-map section of the command file:

```
setvar halt_on_undefined_lef_layer true
```

ignore_lef_use

```
setvar ignore_lef_use [true|false]
```

This variable directs LibGen to ignore the `USE` statements in the LEF files. This feature is useful in situations where the data in the LEF file is not correct, and signal pins are defined with `USE POWER` or `USE GROUND`.

Parameters

[true false]	If this variable is set to <code>true</code>, LibGen ignores the <code>USE</code> statements in the LEF files. When you set it to <code>false</code>, it processes the <code>USE</code> statements. The default is <code>false</code>.
--------------	---

Example

- As a result of the following example, LibGen ignores the `USE` statements in the LEF files:

```
setvar ignore_lef_use true
```

ignore_missing_via_definition

```
setvar ignore_missing_via_definition [true|false]
```

LibGen currently does checking for layer definition on lef, lefdef_layermap and the tech file. If a layer is missing in tech file LibGen will report an error message like the following:

Error:

Following LEF layers used to form via V0A were missing in layer map:

```
POLY, C00
```

The via will be ignored, which may affect following library generation or

zx run if this via is referenced in LEF or DEF file(s).

This variable can be set to prevent this error.

Parameters

[true false]	If this variable is set to <code>true</code>, LibGen will not generate an error message if a layer is missing in the tech file. When this variable is set to <code>false</code> and a layer is missing in the tech file an error will be generated. The default is <code>false</code>.
--------------	---

Example

- As a result of the following example, LibGen ignores missing layer information in the tech file and does not generate an error:

```
setvar ignore_missing_via_definition true
```

keep_debug_files

```
setvar keep_debug_files [true|false]
```

This variable retains all temporary files that LibGen generates during library creation to aid in debugging problems.

Parameters

[true false]	When this variable is set to <code>true</code> , LibGen retains the temporary files. When it is set to <code>false</code> , LibGen discards the temporary files. The default is <code>false</code> .
--------------	--

Example

- In the following example, LibGen keeps the temporary files that it generates during library creation:

```
setvar keep_debug_files true
```

keep_netlist_in_library

```
setvar keep_netlist_in_library [true|false]
```

This variable specifies whether or not the netlist will be retained as part of the library.

Parameters

[true false]	When this variable is set to <code>true</code> , LibGen retains the netlist as part of the library. When this variable is set to <code>false</code> , LibGen discards the netlist as part of the library. For those users who are not using a power-up flow, this variable can be set to <code>false</code> and added to a LibGen command file to save the space taken up by the netlist. The default is <code>true</code> .
--------------	--

Example

- In the following example, LibGen does not keep the netlist as part of the library:

```
setvar keep_netlist_in_library
```

layout_scale_factor

```
setvar layout_scale_factor value
```

This variable is a floating point number used to control the actual scale factor, when layout scaling is performed. If the scale factor has been specified in the technology file, LibGen will issue a warning message and use the one in the technology file to overwrite the user specified value. This variable is used in conjunction with the variable [output_wlt](#).

Parameters

<i>factor</i>	The factor that will be used for scaling the design. It is a floating point number.
---------------	---

Example

- The following example set the layout scale to 0.8 (80%):

```
setvar layout_scale_factor 0.8
```

libgen_executable

```
setvar libgen_executable executable_name
```

This variable specifies the path to the LibGen executable.

Parameters

<i>executable_name</i>	The name of the LibGen executable that will be invoked. The default value is <code>libgen</code> , which assumes that the <code>libgen</code> executable can be found on your path.
------------------------	--

Example

- The following example specifies a LibGen executable called `/home/bin/libgen`:

```
setvar libgen_executable /home/bin/libgen
```

maxerrors

```
setvar maxerrors number
```

This variable sets the maximum number of errors that LibGen can encounter before it aborts with a fatal error.

Parameters

<i>number</i>	Specifies the maximum number of errors that LibGen can encounter before it aborts with a fatal error. It can be an integer in the range of 1 through 10,000, inclusive. The default is 1000.
---------------	---

Example

- The following example sets the maximum number of errors to 500:

```
setvar maxerrors 500
```

max_itfnode_dist

```
setvar max_itfnode_dist number
```

This variable sets the maximum distance between interface nodes when they get generated in LibGen.

Parameters

<i>number</i>	The maximum distance between interface nodes in microns. The default is 50 microns.
---------------	--

LibGen creates interface nodes on port geometries in LEF and ZX, uses them for top level connectivity tracing and also to connect the cell/block onto top level routing. The interface node generation is purely based on port geometries (tap points are placed one per port). For ports greater than `max_itfnode_dist`, they are placed at `max_itfnode_dist` apart. If the port pin geometry in LEF is bigger than the setting of the `max_itfnode_dist` variable, at least two nodes will get generated per port geometry.

Example

- The following example sets the distance between interface nodes to 10 microns:

```
setvar max_itfnode_dist 10.0
```

max_resistor_length

```
setvar max_resistor_length value
```

This variable specifies the length of the longest resistor that LibGen generates during resistance extraction.

Parameters

<i>value</i>	Specifies the length of the longest resistor that LibGen generates during resistance extraction. The default value is 50 microns.
--------------	--

Example

- The following example specifies a maximum resistor length of 40 microns:

```
setvar max_resistor_length 40
```

output_tool_performance_data

```
setvar output_tool_performance_data [true|false]
```

This variable determines if output performance data of intermediate tools are output to log file.

Parameters

[true false]	When this variable is set to <code>true</code> , resource usage data (performance data) of intermediate tools (e.g. XTC, ZX, Thunder) called by LibGen is logged. The default is <code>false</code> .
--------------	--

Example

- In the following example, LibGen outputs performance data of intermediate tools to the log file:

```
setvar output_tool_performance_data true
```

output_wlt

```
setvar output_wlt [drawn |silicon]
```

This variable is used to control whether the resistor width and length should be reported as drawn data or as the actual silicon value. This variable is used in conjunction with the variable [layout_scale_factor](#).

Parameters

[drawn] silicon]	When this variable is set to <code>drawn</code>, the drawn geometry will be output for L and W. If this variable is set to <code>silicon</code> the silicon geometry will be output for width and length. The default is <code>drawn</code>.
---------------------	---

Example

- In the following example, LibGen outputs silicon W and L geometry values:

```
setvar output_drawn_geometry silicon
```

parasitic_extractor

```
setvar parasitic_extractor path_to_executable
```

This variable selects the executable that LibGen invokes to perform parasitic extraction.

Parameters

Example

<code>path_to_executable</code>	The path to the executable that LibGen invokes to perform parasitic extraction. If you do not set this variable, LibGen uses the <code>zx</code> executable (Fire & Ice ZX gate-level extractor) to perform the extraction.
---------------------------------	---

- In the following example LibGen invokes `fire` as the executable to the parasitic extractor:

```
setvar use_old_parasitic_extractor true
```

```
setvar parasitic_extractor /sev32/bin/fire
```

parasitic_extractor_command_files_directory

```
setvar parasitic_extractor_command_files_directory directory_name
```

This variable specifies the name of the directory where LibGen generates the command files for use during the parasitic extraction.

Parameters

<i>directory_name</i>	The name of the directory where LibGen generates the command files for use during the parasitic extraction. The default value is <code>ExtractorCmdFiles</code> in the <code>libname_temp</code> directory. There is usually no reason to override this default value.
-----------------------	---

Example

- In the following example, the command files are located in the `/users/eng/rextract/cmdfiles` directory:

```
setvar parasitic_extractor_command_files_directory /users/eng/rextract/cmdfiles
```

port_file

```
setvar port_file filename
```

This variable specifies the name of a port file that LibGen uses in generating LEF pin data from GDS. If a port file is specified in a Synopsys LEF/GDSII flow, it is passed to XTC unaltered. If it is specified in a mixed LEF/GDS flow, LibGen modifies the data read from the port file to create a modified port file that matches the impostor cell names and passes all necessary data to XTC so that the LEF file can be created properly.

Parameters

<i>filename</i>	Specifies the name of the port file, which contains a set of ports.
-----------------	---

Example

- The following line specifies a port file named `lef_pins.dat`:

```
setvar port_file lef_pins.dat
```

prpgv_x_offset

```
setvar prpgv_x_offset value
```

This variable specifies the offset in the X direction, between the power-grid database data and the LEF file data, in microns. This offset is used to align these two sets of data when the hierarchical power-grid view is generated. Often the DEF file input to VoltageStorm or VoltageStorm PE is specified in a centered coordinate system, and the LEF file for the cell is specified in a first-quadrant coordinate system (that is, its lower left corner is at 0, 0). VoltageStorm PE passes these two variables to LibGen. When LibGen loads power-grid view data, it adds the values of `prpgv_x_offset` and `prpgv_y_offset` to the location of the nodes to place them in the LEF coordinate system.

Parameters

<code>value</code>	The <code>prpgv_x_offset</code> variable specifies the offset in the X direction, between the power-grid database data and the LEF file data, in microns. The default value is 0 microns.
--------------------	--

Example

- Given these specifications in the DEF file for a cell:

```
UNITS DISTANCE MICRONS 100
```

```
DIEAREA ( -5000 -5000 ) ( 5000 5000 )
```

and this data in the LEF MACRO for the same cell:

```
ORIGIN ( 0 0 )
```

```
SIZE 100 BY 100
```

the appropriate setting of these variables would be as follows:

```
setvar prpgv_x_offset 50
```

```
setvar prpgv_y_offset 50
```

These settings would shift the power-grid data extracted from the DEF file 50 microns to the right and 50 microns up to align with the LEF macro data.

prpgv_y_offset

```
setvar prpgv_y_offset value
```

This variable specifies the offset in the Y direction, between the power-grid database data and the LEF file data, in microns. This offset is used to align these two sets of data when the hierarchical power-grid view is generated. Often the DEF file input to VoltageStorm or VoltageStorm PE is specified in a centered coordinate system, and the LEF file for the cell is specified in a first-quadrant coordinate system (that is, its lower left corner is at 0, 0). VoltageStorm PE passes these two variables to LibGen. When LibGen loads power-grid view data, it adds the values of `prpgv_x_offset` and `prpgv_y_offset` to the location of the nodes to place them in the LEF coordinate system.

Parameters

<code>value</code>	The <code>prpgv_y_offset</code> variable specifies the offset in the Y direction, between the power-grid database data and the LEF file data, in microns. The default value is 0 microns.
--------------------	--

Example

- Given these specifications in the DEF file for a cell:

```
UNITS DISTANCE MICRONS 100
```

```
DIEAREA ( -5000 -5000 ) ( 5000 5000 )
```

and this data in the LEF MACRO for the same cell:

```
ORIGIN ( 0 0 )
```

```
SIZE 100 BY 100
```

the appropriate setting of these variables would be as follows:

```
setvar prpgv_x_offset 50
```

```
setvar prpgv_y_offset 50
```

These settings would shift the power-grid data extracted from the DEF file 50 microns to the right and 50 microns up to align with the LEF macro data.

std_cell_size_limit

```
setvar std_cell_size_limit value
```

This variable specifies a value, in microns, that determines whether LibGen optimizes the ports of a cell. By default, LibGen optimizes the ports of any cell whose X or Y dimension is less than the value specified. This process reduces the number of geometries and interface nodes on the ports, which can improve the performance of the extraction tools using the data.

Parameters

<i>value</i>	Specifies a value in microns, that determines whether LibGen optimizes the ports of a cell. The default is 10 microns.
--------------	---

Example

- As a result of the following line, LibGen optimizes the ports of any cell whose X or Y dimension is less than 5 microns:

```
setvar std_cell_size_limit 5
```

stripe_step_size

```
setvar stripe_step_size number
```

This variable specifies the number of stripes to run together when LibGen performs resistance extraction using the parallel script. The parallel resistance extraction may fail in some cases if there are large objects that straddle this number of stripes. If that happens, increase the number of stripes.

Parameters

<i>number</i>	The number of stripes to process together. It is an integer. The default is 3.
---------------	---

Example

- The following example sets the number of stripes to 7:

```
setvar stripe_step_size 7
```

suppress_regular_gds_layer

```
setvar suppress_regular_gds_layer [true|false]
```

This variable determines whether the layer map that LibGen passes to XTC includes the regular GDS layers.

Parameters

[true false]	When this variable is set to <code>true</code> and the <code>emulate_lef_parsing</code> variable is set to <code>false</code>, LibGen ignores all the regular GDS layers and only passes the special layers to XTC to generate the pseudo-LEF file. If this variable is set to <code>false</code>, XTC processes all the regular GDS geometries, and all regular GDS geometries that are connected to the special layer port geometries are considered part of the ports. If <code>emulate_lef_parsing</code> is set to <code>true</code>, the <code>suppress_regular_gds_layer</code> variable has no effect. The default is <code>false</code>.
--------------	--

Example

- In the following example, LibGen passes all the GDS layers in the layer map file to XTC:

```
setvar emulate_lef_parsing false
```

```
setvar suppress_regular_gds_layer false
```

tech_file

```
setvar tech_file filename
```

This variable specifies the name of the technology file generated by RCgen. This file is copied into the cell library for future reference. It is required input for power-grid view generation and is recommended if you create cell libraries for extraction-for-timing flows.

Note: RCgen is the preferred tool for technology file generation. However, technology files generated by IceCaps

can still be used for backward compatibility. RCgen must be used for ZX4.1 and VSPE 4.1.

Parameters

<i>filename</i>	The name of the technology file. There is no default file name.
-----------------	--

Example

- The following example gives the path name of the `tetra.tch` technology file:

```
setvar tech_file /users/lxd/pwrvw/tetra.tch
```

temp_directory_name

```
setvar temp_directory_name directory_name
```

This variable specifies the name and location of the temporary directory, where LibGen places the temporary library files that it creates. Normally the name of this directory is `libname_temp`, where `libname` is the name of the library created. This variable can be useful if the disk space on your machine is running low and you want to put the temporary files at a location other than the default location.

Parameters

<i>directory_name</i>	The name of the temporary directory where LibGen places the temporary library files.
-----------------------	--

Example

- In the following example, LibGen places the temporary library files that it creates in the `/usr/src5/vs320` directory:

```
setvar temp_directory_name /usr/src5/vs320
```

temperature

```
setvar temperature value_in_celsius
```

Specifies temperature value in degree Celsius.

Example

```
setvar temperature 30
```

thunder_command_file

```
setvar thunder_command_file filename
```

This variable specifies the name of a file that passes additional user commands to Thunder during detailed power-grid view generation. These commands may be needed, for example, when conflicts with the hierarchy separator occur and you need to re-define the hierarchy separator used by Thunder.

Parameters

<i>filename</i>	The name of the file containing the additional Thunder commands. There is no default name.
-----------------	---

Example

- The following example specifies a Thunder command file named `libgen2thunder.txt`:

```
setvar thunder_command_file libgen2thunder.txt
```

thunder_executable

```
setvar thunder_executable executable_name
```

This variable specifies the program used for current estimation during power-grid view generation.

Parameters

<i>executable_name</i>	Specifies the program used for current estimation during power-grid view generation. The default Thunder executable is <code>thunder.tty</code> .
------------------------	--

Example

- The following example specifies `../thunder` as the executable:

```
setvar thunder_executable ../thunder
```

use_current_data_from_pgv

```
use_current_data_from_pgv [true | false]
```

Specifies that current data for vstorm2 will come from power-grid view.

[true false]	If set to true current data will be used from power-grid view.
----------------	--

Example

- The following example specifies the current data will come from the power-grid view:

```
setvar use_current_data_from_pgv true
```

use_lef_only_itfnodes

```
setvar use_lef_only_itfnodes [true|false]
```

Interface nodes are nodes which connect the top level routing with the cell internal resistor network. This variable controls the algorithm used for interface nodes generation.

Parameters

<i>executable_name</i>	If this variable is <code>true</code>, interface nodes are generated using only the port geometry data from the LEF files. This is done before extraction has been done. If this variable is <code>false</code>, interface nodes are generated after extraction has been completed. Libgen decides on interface nodes based on the resistor network combining the information from the LEF and GDS files. <i>Default:</i> <code>true</code>
------------------------	--

Example

- The following example specifies that interface nodes are generated after extraction has been completed.:

```
setvar use_lef_only_itfnodes false
```

use_lef_technology

```
setvar use_lef_technology [cell_library_name|LEF_file_name]
```

This variable takes the technology data from one of the libraries in the list of libraries to link or merge other than the first library or from a separate LEF file. Normally LibGen obtains the technology data from the first library in the list of libraries to link or merge. If you use a separate LEF file, you must set up a correct layer map in the LibGen command file corresponding to this LEF file.

This variable is optional when linking or merging libraries with the `input_type link_library` or `input_type merge_library` commands, respectively.

Parameters

<code>cell_library_name</code>	Specifies the name of the cell library from which to take the technology data.
<code>LEF_file_name</code>	Specifies the name of the LEF file from which to take the technology data.

Example

- The following example specifies that LibGen take the technology data from the pgvu_360 cell library:

```
setvar use_lef_technology pgvu_360
```

use_old_lef_parser

```
setvar use_old_lef_parser [true|false]
```

This variable determines whether LibGen uses the default parser or a parser that can read LEF files created before version 5.6. If the default parser issues warning messages when you try to process a LEF file created before version 5.6, you can invoke another parser to read the LEF file.

Parameters

<code>[true false]</code>	If this variable is set to <code>true</code>, LibGen uses a parser that can read a LEF file created before version 5.6. When this variable is set to <code>false</code>, it uses a parser that cannot read a LEF file created before version 5.6. The default is <code>false</code>.
---------------------------	---

Example

- In this example, LibGen uses a parser that can read a LEF file created before version 5.6:

```
setvar use_old_lef_parser true
```

use_pwrnet_ipeak

```
setvar use_pwrnet_ipeak [true|false]
```

This variable is for setting ipeak based current distribution in detailed views.

Parameters

[true false]	If this variable is set to <code>true</code>, LibGen uses ipeak based current distribution in detailed views. When this variable is set to <code>false</code>, LibGen creates lavg detailed views using Accura. The default is <code>false</code>.
--------------	---

Example

- In this example, LibGen uses ipeak based current distribution in detailed views:

```
setvar use_pwrnet_ipeak true
```

xtc_stripe_directory_name

```
setvar xtc_stripe_directory_name directory_name
```

This variable specifies the name of the directory where XTC will create the stripes to be used during the resistance extraction.

Parameters

<i>directory_name</i>	Specifies the name of the directory where XTC will create the stripes to be used during the resistance extraction. The default directory is <code>stripes</code>. There is usually no reason to override this setting.
-----------------------	--

Example

- This example specifies a directory called `../test/xtc/rex`:
`xtc_stripe_directory_name ../test/xtc/rex`

Library Simulator Options

This section lists the Library Simulator commands and variables that can be included in Voltus using the `set_advanced_pg_library_mode -thunder_command_file` command parameter. They are listed in alphabetical order.

- [Library Simulator Commands](#)
- [activity](#)
- [activity cycle_time](#)
- [activity default](#)
- [activity filed](#)
- [activity filen](#)
- [activity report](#)
- [activity reset](#)
- [activity scale](#)
- [activity scaled](#)
- [activity scalen](#)
- [activity setd](#)

- [activity setn](#)
- [activity vdd_range](#)
- [check](#)
- [check off, check on](#)
- [check tgloff, check tglon](#)
- [check time](#)
- [devc](#)
- [devc table](#)
- [devc tran](#)
- [pwrmeter](#)
 - [Format of the Clock File](#)
 - [Format of the Frequency-Activity File](#)
 - [Format of the Average-Power File](#)
- [pwrmeter plot](#)
- [pwrnet](#)
- [pwrnet clear](#)
- [pwrnet ipeak](#)
- [pwrnet off, pwrnet on](#)
- [pwrnet report](#)
- [pwrnet specify](#)
- [pwrnet tally](#)
- [pwrnet tallyint](#)
- [pwrnet tgloff, pwrnet tglon](#)
- [pwrnet time](#)
- [region](#)
- [region params](#)
- [region setd](#)
- [region setn](#)
- [setenv](#)
- [stats](#)
- [tglcnt](#)

- [tglcnt check](#)
- [tglcnt clear](#)
- [tglcnt off, tglcnt on](#)
- [tglcnt params](#)
- [tglcnt report](#)
- [tglcnt tally](#)
- [vcd](#)
- [VCD File](#)
- [VCD Values](#)
- [VCD Command Extensions](#)
- [Signal Declaration File](#)
- [Library Simulator Variables](#)
- [allowMosAsSubckt](#)
- [analysisReportSize](#)
- [busNotation](#)
- [caseSensitive](#)
- [cmdBreakOnError](#)
- [CoupledCapOnExtractedNetlist](#)
- [DCInductorResistance](#)
- [defaultTRiseFall](#)
- [errorIfDevicesBelowMinW_L](#)
- [forwardDiodeIpeakCurrent](#)
- [gatePowerNetConnection](#)
- [groundName](#)
- [hierarchySeparator](#)
- [isDetailedDynamicView](#)
- [keepTopCell](#)
- [logLimit](#)
- [minimumCapacitance](#)
- [minimumInductance](#)
- [minimumNodeCapacitance](#)

- [minimumResistance](#)
- [netlistInlineComment](#)
- [netlistTopCell](#)
- [pwrnetSpecifyBjtRatio](#)
- [pwrnetSpecifyCapCurrent](#)
- [pwrnetSpecifyDiodeRatio](#)
- [pwrnetSpecifyResistorRatio](#)
- [quitOnCmdError](#)

Library Simulator Commands

activity

This section discusses the `activity` commands:

- `activity cycle_time`
- `activity default`
- `activity filed`
- `activity filen`
- `activity report`
- `activity reset`
- `activity scale`
- `activity scaled`
- `activity scalen`
- `activity setd`
- `activity setn`
- `activity vdd_range`

Use `activity` to specify activity levels for regions (gates) and to generate average current information reports for analysis. Library Simulator uses activity levels in conjunction with cycle times, VDD to VSS range, and capacitance information to determine the average power-grid *tapcurrents* of transistors. Library Simulator applies the specification of activity levels for nodes or devices to the entire region that contains the node or device. If multiple activities are defined at the same node or device, or different activities are defined at nodes or devices in a region, the activity of the region is set to the highest activity of any node specified in the region.

activity cycle_time

The `activity cycle_time` command sets the base cycle time of the clocks in the design.

Note: If the design uses multiple clocks with different frequencies, you can use the `activity scale` command to scale the activity level on gates operating at the different frequencies. (See "[activity scale](#)".)

Syntax

```
activity cycle_time value
```

Value specifies the value, in seconds, to which the simulator sets the base cycle time of the clocks in the design. The default is 5 ns.

Example

The following line sets the base cycle time of the design clocks to 0.3 ns:

```
activity cycle_time 0.3ns
```

activity default

The `activity default` command sets the default activity level for all regions to the specified value. It sets the default activity levels for regions for which the specific activity information is unknown.

The `activity default` command overwrites activity levels that may have been set by other commands, so you should use this command before using other detailed activity commands.

Syntax

```
activity default value
```

Value specifies the value to which the simulator sets the default activity level for all regions. A typical value might be in the range of 0.2-0.5. The default is 0.2, and the range should be 0.0 to 1.0.

Example

The following line sets the default activity level for all regions to 0.03:

```
activity default .03
```

activity filed

The `activity filed` command loads device activity levels from files that have been computed by external tools such as a Verilog or VHDL simulator. A common source of these activity files is a derivation of toggle-count data from high-level simulators.

This command sets the activity levels for the devices specified in the file.

Syntax

```
activity filed filename
```

Filename specifies the name of the file containing the device for which the simulator sets the activity level. The contents of the file have the following format:

```
nodename activity_level
```

Activity_level is the average toggle ratio of the node (or devices attached to the node) over one clock period. The value is usually between 0 and 1. If a net toggles more often or you want to take glitches into account, you can specify an activity level larger than 1.

Here are some examples of the activity-level format:

```
d1 0.05
```

```
d2 0.07
```

Example

The following line sets the activity level for the regions based on the device activity level provided in the `device_activity.dat` file:

```
activity filed device_activity.dat
```

activity filen

The `activity filen` command loads node activity levels from files that have been computed by external tools such as a Verilog or VHDL simulator. A common source of these activity files is a derivation of toggle-count data from gate-level simulators. This command sets the activity levels for the nodes specified in the file.

Syntax

```
activity filen filename
```

Filename specifies the name of the file containing the node for which the simulator sets the activity level. This file has the following format:

```
nodename activity_level
```

Activity_level is the average toggle ratio of the node (or devices attached to the node) over one clock period. The

value is usually between 0 and 1. If a net toggles more often or you want to take glitches into account, you can specify an activity level larger than 1.

Here are some examples of the activity-level format:

```
net1 0.05
```

```
net2 0.07
```

Example

The following line sets the activity level of the regions on the basis of the activity level of the nodes provided in the `node_activity.dat` file:

```
activity filen node_activity.dat
```

activity report

The `activity report` command generates the power-grid tap data for the specified voltage source node name. Library Simulator considers the specified activity levels, cycle time, and VDD range in conjunction with the total regional capacitances to generate the output file, which is compatible with Lightning (VST). The name of the output file is `vsrc_name.iavg`.

Library Simulator uses the width-length ratio for all devices connected to a specified voltage source node in a specific region to determine how the average current is distributed between multiple devices in one region. If a region contains two devices, one with a width-length ratio of 2 and a second with a width-length ratio of 1, the first device is assigned two-thirds of the average current consumed by this region, and the second device is assigned one-third of the average current.

Syntax

```
activity report <format=[binary|ascii]> vsrc_name
```

This syntax contains the following parameters:

- `format`
Specifies the format of the output report file. Valid options are binary or ASCII. The default is binary, because you use binary output files from the simulator as input to Lightning (VST) for power-grid analysis. It is also more efficient to use the binary file.
- `vsrc_name`
Specifies the name of the voltage source.

Example

The following line generates the power-grid tap data for the VDD voltage source:

```
activity report vdd
```

activity reset

The activity reset command resets the activity levels of all regions in the circuit to the default activity level set by the activity default command. (See the [activity default](#).)

Syntax

```
activity reset
```

This command has no arguments.

activity scale

The activity scale command scales all activities in the circuit by the given value. This multiplicative factor enables you to add conservatism to the circuit as a whole by scaling all activity levels by a fixed amount, such as 1.1.

You also can use the scaling factor to relax the activity level by specifying a scale value smaller than 1. The activity scale is global; it scales the activity levels for all nodes and devices.

Syntax

```
activity scale value
```

Value specifies the multiplicative factor by which the simulator scales all activities in the circuit. This value must be positive.

Example

The following line scales all activities in the circuit by a factor of 1.2:

```
activity scale 1.2
```

activity scaled

The activity scaled command scales the activity levels for the regions containing the specified devices by the given value.

Syntax

```
activity scaled value [device|dgroup]+
```

This syntax contains the following parameters:

- *value*

Specifies a floating-point number larger than 0.

- *device|dgroup*

Specifies the name of a device or predefined group of devices. Use the `group device` command to establish device groups.

Example

The following line scales all activities in the region containing the `dev_grp7` device group by 1.1:

```
activity scaled 1.1 dev_grp7
```

activity scalen

The `activity scalen` command scales the activity levels for the regions containing the specified nodes by the given value.

Syntax

```
activity scalen value [ngroup|node] +
```

This syntax contains the following parameters:

- *value*
Specifies a floating-point number larger than 0.
- *ngroup*
Specifies the name of a predefined group of nodes. You establish node groups by using the `group node` command.
- *node*
Specifies the node name.

Example

The following line scales all activities in the region containing the `node5` node by 1.2:

```
activity scalen 1.2 node5
```

activity setd

The `activity setd` command sets the activity levels for the regions containing the specified devices to the given value. It enables you to set the activity level of specific regions when you know the exact activity.

Syntax

```
activity setd value [device|dgroup]+
```

This syntax contains the following parameters:

- *value*
Specifies a floating-point number, usually between 0 and 1, inclusive, but it can be larger.
- *device|dgroup*
Specifies the name of a device or predefined group of devices. Use the `group device` command to establish device groups.

Example

The following line sets the activity level for the regions containing devices specified in the `dev_grp2` group to 0.03:

```
activity setd 0.03 dev_grp2
```

activity setn

The `activity setn` command sets the activity levels for the regions containing the specified nodes to the given value. It enables you to set the activity level of specific regions when you know the exact activity.

Syntax

```
activity setn value [ngroup|node]+
```

This syntax contains the following parameters:

- *value*
Specifies the value to which the simulator sets the region.
- *ngroup*
Specifies the name of a predefined group of nodes. You establish node groups using the `group node` command.
- *node*
Specifies the node name.

Example

The following line sets the activity level for the region containing `node3` to 0.027:

```
activity setn 0.027 node3
```

activity vdd_range

The `activity vdd_range` command sets the base voltage range of the gates in the design. This is the voltage range of VDD-VSS.

Note: If the design uses multiple voltage ranges, you can use the `activity scale` command to scale the activity level on gates operating at different voltage ranges. (See "[activity scale](#)".)

Syntax

```
activity vdd_range value
```

Value specifies the value to which the simulator sets the base voltage range. The default is 3.3 V.

Example

The following line sets the base voltage range to 3.3 V:

```
activity vdd_range 3.3
```

check

The `check` command determines when Library Simulator performs node and device checks. The simulator uses these checks to print error messages whenever a node or device exhibits some behavior outside the bounds specified for the node or device. For example, you might want to know whether a node's rise time exceeds a specified limit, or if the node experiences any glitches during simulation.

The `check` command includes the following:

- `check off`
- `check on`
- `check tgloff`
- `check tglon`
- `check time`

check off, check on

The `check on` command activates (and `check off` deactivates) all checks. The default is `check on`.

Syntax

```
check [on|off]
```

This command has no arguments.

check tgloff, check tglon

The check tglon command activates (and check tgloff deactivates) all checks in the simulator when any specified node toggles high or low, depending on which switch you specify, for count number of times. If you do not specify the high or low switch, the simulator counts all toggles. If you do not specify count, the simulator activates (or deactivates) all checks each time a specified node toggles.

The optional toggle-timing parameter definitions provide the logic voltage level for the toggle time. If you do not give a switching timing parameter name, the program uses the default provided by the parameter set. You build toggle-count parameter sets by using the tglcntparams command. (See "[tglcnt params](#)".)

Syntax

```
check [tgloff|tglon] <params=tglcnt_param> <label="label"> <count=n> <high|low> [ngroup|node]+
```

This syntax contains the following parameters:

- `params`
Finds the `vt` threshold voltage from a specific set of parameters. You build toggle-count parameter sets by using the `tglcnt params` command. (See "[tglcnt params](#)".)

If you omit this argument, the simulator uses default toggle-count parameters. You can find the defaults in the description of `tglcnt params`.
- `tglcnt_param`
Specifies the name of the parameter set to measure against.
- `label`
Labels each message generated by this command.
- `"label"`
Specifies the label text. Enclose this string in double quotation marks.
- `count`
Counts node toggles.
- `n`
Specifies the number of node toggles to count before activating or deactivating checks.
- `high`
Counts the number of times that a node toggles high.
- `low`
Counts the number of times that a node toggles low.
- `ngroup`
Specifies the name of a predefined group of nodes. You establish node groups by using the `group node` command.

- *node*

Specifies the node name.

Example

The following line turns on all checks on the third rising toggle of the clock node:

```
check tglon count=3 high clock
```

check time

The check time command activates all performance monitor types between the specified times.

Syntax

```
check time [ON=time|OFF=time]+
```

Time specifies the number of units of time. Include the unit identifier, which can be one of the following:

- *s* (seconds)
- *ms* (milliseconds)
- μ *s* (microseconds)
- *ns* (nanoseconds)
- *ps* (picoseconds)
- *fs* (femtoseconds)

The default is *ns*.

Example

The following line activates performance monitors between 1.0 ns and 10.0 ns:

```
check time on=1.0ns off=10.0ns
```

devc

The *devc* command reports device capacitance information.

The *devc* commands are the following:

- *devc table*
- *devc tran*

devc table

The devc table command reports the capacitances associated with the specified devices in tabular form.

Note: Library Simulator currently reports only capacitances associated with MOSFET transistors.

Syntax

```
devc table <<append> file = filename>
```

```
<vd = {v_start, <v_end, v_step>}>
```

```
<vg = {v_start, <v_end, v_step>}>
```

```
<vs = {v_start, <v_end, v_step>}>
```

```
<vb = {v_start, <v_end, v_step>}> [device|dgroup]+
```

Note: The braces ({ }) enclose the voltage parameters for each node voltage.

This syntax contains the following parameters:

- *append*
Appends the report to the specified file.
- *file=filename*
Specifies the name of the file to which the simulator appends the table.
- *vd*
Specifies the voltage for the drain node. The default value is 0.0 volts.
- *vg*
Specifies the voltage for the gate node. The default value is 0.0 volts.
- *vs*
Specifies the voltage for the source node. The default value is 0.0 volts.
- *vb*
Specifies the voltage for the bulk node. The default value is 0.0 volts.
- *v_start*
Specifies the starting voltage value for a specific node used in generating the table.
- *v_end*
Specifies the end voltage value for a specific node to use in generating the table. If *v_end* is missing, the simulator uses a DC voltage value of *v_start*.
- *v_step*

Specifies the voltage step at which table data is reported. If `v_step` is missing, the simulator uses a DC voltage value of `v_start`.

- `device|dgroup`

Specifies the name of a device or predefined group of devices. Use the `group device` command to establish device groups.

Example

The following line reports device capacitances for `mos10` and `mos15` in the `mos.cap` file:

```
devc table file = mos.cap vd = {3.3v} vg = {0.0v, 3.3v, 0.3v} mos10 mos15
```

In this example, the simulator reports the capacitances for a DC voltage of 3.3 V at the drain node. The gate voltage increases from 0.0 V to 3.3 V in steps of 0.3 V. The source and bulk nodes are at 0.0 V by default.

devc tran

The `devc tran` command includes the capacitances of specified devices in the dynamic waveform output file during dynamic analysis.

Note: Library Simulator currently reports only capacitances associated with MOSFET transistors.

Syntax

```
devc tran [device|dgroup]+
```

`Device|dgroup` specifies the name of a device or predefined group of devices. Use the `group device` command to establish device groups.

Example

The following line displays the dynamic device capacitances for the `mos20` MOSFET:

```
devc tran mos20
```

In this example, the simulator reports the capacitances between the gate, drain, source, and bulk nodes of the `mos20` transistor during dynamic analysis.

pwrmeter

The `pwrmeter` command generates the average tap currents for all devices connected to a power grid or the average power consumption for all instances connected to a voltage source. It is based on a proprietary technology called Accura, which provides activity-based static analysis, current distribution within a cell based on charging and discharging paths, probabilistic calculation of switching activities, recognition of clock domains, and special handling of memory blocks. You can apply this static analysis at the block or the chip level.

Note: The `pwrmeter` command in the simulator is different from Cadence's PowerMeter tool.

Confusion may arise between PowerMeter and `pwrmeter` because VoltageStorm PE invokes both PowerMeter and LibGen, which in turn invokes the simulator when it creates the detailed power-grid views. In addition, VoltageStorm Transistor-Level PGS invokes the simulator in its flow.

The `pwrmeter` command is transistor-based and relies on the simulator's built-in gate-level recognition (regions) and clock-tracing algorithms. It uses the power equation $I = CAVF$, where:

- C is the capacitance, which is a combination of the device capacitance calculated from the CV tables generated by Tablegen and optionally the capacitance loading read in from the Ice database.
- A is the activity that the Accura technology calculates using a propagation algorithm applied to the gate-level recognition. The Accura technology is the same as that in the PowerMeter tool in the simulator version 4.0.2 or later. In releases prior to 4.0.2, the simulator used a different technology.
- V is the supply voltage.
- F is the highest input clock frequency to which to set the output signal for gates that Accura does not recognize.

Once the Accura algorithm calculates the current on each node, it analyzes the region driving the current to generate the current breakdown for each device within the region. It calculates the basic switching current and leakage at each node. It does not compute state-dependent leakage or short-circuit (crowbar) current. In summary, the `pwrmeter` command takes into account capacitance loading, SPICE models, and clocks and activity.

PowerMeter is the next stage in the development of the Accura technology. It uses different input data than the simulator. By using a library, which can have comprehensive power information, as well as functional behavior (complete gate recognition), it takes into account input pin slew rates, capacitance loading on pins and wires, clocks and activity, and the probability of signals being 1. PowerMeter allows state-dependent leakage. Like the `pwrmeter` command, it uses a propagation algorithm to calculate the switching activity, but it also propagates the probability of a signal being a 1, enabling it to calculate state-dependent leakage. Using the input pin slew rates, it can also calculate the short-circuit (crowbar) current that is part of the internal power and is captured in the cell's power tables. These tables are based on input slew rates and output capacitance loading.

Syntax

```
pwrmeter [cell|device] FREQ=value
```

```
<CLK_FILE=filename>+
```

```
<FA=value> <FA_FILE=filename>+
```

<AVG_PWR=*value*><AVG_PWR_FILE=*filename*>+

<VCD_FILE=*filename*>+

<OUTFILE=*filename*>*vsrc_name*

This syntax contains the following parameters:

- *cell*
Reports instance-based power data.
- *device*
Reports device-based current data.
- *FREQ=value*
Specifies the clock frequency for the entire chip used to compute the average currents.
- *CLK_FILE=filename*
Specifies the name of the file containing the clock nodes, subcircuit definitions, and subcircuit instance names and their frequencies. You can specify multiple file names. For the format of this file, see ["Format of the Clock File"](#).
- *FA=value*
Specifies the default nodal activity to be used instead of Accura. This value is used at each node unless it is overridden by a value in the file specified by *FA_FILE*. It is given as an activity factor. A value of 0.20 would be 20% of the default frequency.
- *FA_FILE=filename*
Specifies the name of the file containing nodes and their activity. See *FA=value* above for how values are specified. You can specify multiple file names. For the format of this file, see ["Format of the Frequency-Activity File"](#).
- *AVG_PWR=value*
Specifies the total average power dissipation of the circuit connected to the voltage source.
- *AVG_PWR_FILE=filename*
Specifies the name of two files:
 - The *filename.cavg* file, which contain the subcircuit definitions, subcircuit instance names, and average power consumption. You can specify multiple file names. For the format of this file, see ["Format of the Average-Power File"](#).
 - The *filename.cravg* file, which contains all the device current that you specified.
You must use both these files in Lightning (VST). First, load the *.cavg* file and scale it, if necessary, then load the *.cravg* file.
- *VCD_FILE=filename*
Specifies the name of the VCD file input to Accura.

- `OUTFILE=filename`
Specifies the name of the output file. The default is the name of the voltage source. The `.cavg` extension is appended to the file name.
- `vsrc_name`
Specifies the name of the power-grid voltage source.

Example

The following example sets the clock frequency at 10 MHz for the VDD voltage source:

```
pwrmeter device freq=10mhz vdd
```

The next example sets the clock frequency to 10 MHz, specifies the name of the clock file and the output file, and sets the total average power dissipation of the devices connected to `vdd2` at 1.0 watt:

```
pwrmeter device freq=10mhz CLK_FILE=clk.frq AVG_POWER=1.0w OUTFILE=test vdd2
```

Format of the Clock File

The clock file has the following format:

```
file      :      rows EOF                      //a file consists of rows
rows      :      ε                            //an empty line
          |      comment                      //a comment
          |      line                        //a specification line

comment   :      ^[\t]*\*.*+EOL                //a line starting with a *
line      :      CLK name freq_value           //a clock node specification
          |      INST name freq_value          //a subcircuit instance specification
          |      INST name freq_value activity //a subcircuit instance with activity
          |      CELL name freq_value          //a subcircuit definition specification
          |      CELL name freq_value activity //a subcircuit definition with activity
```

`CLK`, `INST`, and `CELL` are keywords.

- If you use the `CLK` keyword, the node name is treated as the root of a clock running at the given frequency.
- If you use the `INST` keyword, all input and output nodes--that is, node names on the instance card--are set to toggle at the specified frequency. The activity of the nodes internal to the subcircuit are computed by using Accura. If you give an optional activity value, Accura is disabled and the activities of all the nodes are set to the specified value; all nodes toggle at the time represented by the product of the specified frequency and the activity.

- If you use the `CELL` keyword, all instances of the cell are processed instead of a single instance.

`CLK` has precedence over `INST`, and `INST` overrides `CELL` information to resolve redundant specifications.

Format of the Frequency-Activity File

The frequency-activity file has the following format:

```
file           :rows EOF                //a file consists of rows of information
rows           :ε                        //an empty line
               |comment                  //a comment line
               |line                     //a specification line
comment        :^[\\t]*\\*.* EOL          //a line starting with * is a comment
line           :name FA_value            //a node with transitional frequency
```

- *Name* can be the name of any node in the circuit.
- *FA_value* is given as an activity factor. A value of 0.20 would be 20% of the default frequency.

Format of the Average-Power File

The average-power file has the following format:

```
file           :rows EOF                //a file consists of rows of information
rows           :ε                        //an empty line
               |comment                  //a comment line
               |line                     //a specification line
comment        :^[\\t]*\\*.*.powerFA_value //a subcircuit instance with average power
               |CELL name power_value    //a subcircuit definition with average power
```

The `CELL` keyword specifies that the name following it is a subcircuit definition and that all instances of the cell dissipate the same average power. If the file does not contain the `CELL` keyword, the name is an instance of a subcircuit dissipating the specified average power.

pwrmeter plot

The `pwrmeter plot` command plots the current data generated by the `pwrmeter` command.

Note: You can only execute the `pwrmeter plot` command after the `pwrmeter` command. If you perform any other analysis between these two commands, the data becomes invalid.

Syntax

```
pwrmeter plot [avgi|freq|activity|primary]
```

This syntax contains the following parameters:

- `avgi`
Plots the average device current calculated.
- `frequency`
Plots the frequency calculated for each node.
- `activity`
Plots the activity calculated for each node.
- `primary`
Plots the location of the primary input node. Primary input nodes are the following:
 - Nodes connected to DC voltage sources. They are drawn on data layer 7 with a default color of dark olive green. These nodes are assumed to operate at 0 frequency.
 - Nodes connected to pwl voltage sources. They are drawn on data layer 4 with a default color of gold.
 - Floating nodes considered as primary inputs. They are drawn on layer 1 with a default color of red.

Example

The following line plots the average device current calculated by the `pwrmeter` command:

```
pwrmeter plot avgi
```

pwrnet

The `pwrnet` command generates power-grid tap current data for Lightning (VST), the power-grid analysis tool. Power-grid taps consist of all transistors connected to the power node.

Typical power analysis involves reporting the total power consumed or current delivered by voltage sources in SPICE-type circuit analysis. This information is the first step in power analysis. The `pwrnet` command provides the next step in power analysis: reporting where power is consumed in the circuit so that you can perform region-wide power and power-grid analyses using the data.

The `pwrnet` command includes the following commands:

- `pwrnet clear`
- `pwrnet ipeak`
- `pwrnet ioff`
- `pwrnet ion`
- `pwrnet off`

- `pwrnet on`
- `pwrnet report`
- `pwrnet specify`
- `pwrnet tally`
- `pwrnet tallyint`
- `pwrnet tglloff`
- `pwrnet tglon`
- `pwrnet time`

pwrnet clear

The `pwrnet clear` command clears all power net tally data.

Syntax

```
pwrnet clear
```

This command has no arguments.

pwrnet ipeak

The `pwrnet ipeak` command prints the maximum current through transistors that are connected to a power net.

This command performs a device current evaluation by using `|vgs|=vdd*vgs_fact` and `|vds|=vdd*vds_fact` to obtain the peak IDS of the transistor. Library Simulator then reports `ids * ipeak_fact` in the output file as the peak current of the transistor.

The format of the output file is a list of transistor names followed by the scaled peak current of the transistor.

Syntax

```
pwrnet ipeak <file=filename> <format=[ascii|binary]> <vds_fact=n> <vgs_fact=n> <ipeak_fact=n>  
vsrc_name
```

This syntax contains the following parameters:

- `format`
Specifies the output report file format. Valid options are binary or ASCII. The default is binary, because you use binary output files from the simulator as input to Lightning (VST) for power-grid analysis.
- `file`
Specifies an output file other than the default of `vsrc_name.ipeak`.

If you use the `pwrnet specify` command to provide a power specification before using the `pwrnet ipeak` command, the simulator splits the output file into two files:

- `vsrc_name.aipeak`
Contains the current data for all the devices or the calculated total power consumption for blocks. This information is obtained by using the `pwrnet specify` command.
- `vsrc_name.ipeak`
Contains the current data for the rest of the devices.

In Lightning (VST), load the `.ipeak` file first, and scale the current data. Lightning (VST) then loads the `.aipeak` file.

- `filename`
Specifies the name of the file to which the simulator prints the information.
- `vds_fact`
Specifies a multiplication factor used with the drain-to-source voltage. The default is 1.
- `vgs_fact`
Specifies a multiplication factor used with the gate-to-source voltage. The default is 1.
- `ipeak_fact`
Specifies a multiplication factor used with the peak current through the device. The default is 1.
- `vsrc_name`
Specifies the name of the voltage source: VDD or another source name.

Example

The following line prints the worst-case current through all the devices connected to VDD to a file named `vdd.ipeak`:

```
pwrnet ipeak vdd
```

pwrnet off, pwrnet on

The `pwrnet on` command activates (and `pwrnet off` deactivates) the storing and tallying of power-net device current data during dynamic analysis. The default is `pwrnet on`.

Syntax

```
pwrnet [off|on]
```

This command has no arguments.

pwrnet report

The `pwrnet report` command generates a report containing the RMS, maximum, and average results of individual device current tallies. Initiate tallies by using the `pwrnet tally` command. (See "[pwrnet tally](#)".) Library Simulator reports device currents only if the current is greater than 1.0e^{-20} amperes.

Syntax

```
pwrnet report vsrc_name
```

Vsrc_name specifies the name of the voltage source for which you previously performed the `pwrnet tally` command, and the prefix for the `.rms`, `.max`, and `.avg` output files.

Example

The following line prints tallied current RMS, maximum, and average information to files named `vdd.rms`, `vdd.max`, and `vdd.avg`, respectively:

```
pwrnet report vdd
```

pwrnet specify

The `pwrnet specify` command calculates the total power consumption for blocks or the current for individual devices. This current value is used during static power-grid analysis. You can use the `pwrnet specify` command to pre-assign a current consumption number for both analog and digital circuitry.

When you use the `pwrnet specify` command to provide a power specification before using the `pwrnet ipeak` command, the simulator splits the output into two files:

- *vsrc_name.aipeak*, which contains the current data for all the devices or the calculated total power consumption for blocks. This information is obtained by using the `pwrnet specify` command.
- *vsrc_name.ipeak*, which contains the current data for the rest of the devices.

In Lightning (VST), load the `.ipeak` file first, and scale the current data. Lightning (VST) then loads the `.aipeak` file.

Syntax

```
pwrnet specify [group_file = filename]
```

or

```
pwrnet specify [DEV = device_name| [[DEF = subcircuit_definition|
```

```
INST = subcircuit_instance]
```

```
<bjt_ratio = value> <res_ratio = value>
```

```
<diode_ratio = value>]]
```

```
[POWER = value|CURRENT = value]
```

This syntax contains the following parameters:

- `DEV = device_name`
Specifies the name of the device for which the power or current value is specified.
- `DEF = subcircuit_definition`
Specifies the name of the definition of a subcircuit for which the power or current value is specified. The specified values apply to all instances of the subcircuit.
- `INST = subcircuit_instance`
Specifies the name of an instance of a subcircuit for which the power or current value is specified.
- `group_file = filename`
Specifies the name of the file containing analog circuits described as device groups. The format of the group file is as follows:

```
list_of_groups:group

|

list_of_groups

group:group_name group_attributes

{

list_of_layout_devices

}

EOF
```

This syntax contains the following parameters:

- `group_name`
Specifies the name for a group of extracted devices corresponding to a schematic netlist.
- `group_attributes:[POWER = value|CURRENT = value]`
Specifies the power dissipation or the drawn current of the subcircuit.
 - `POWER = value`
Specifies the power dissipation of the subcircuit.

- `CURRENT = value`

Specifies the drawn current of the subcircuit.

- `list_of_layout_devices`

Specifies the names of the layout devices. The name of each device must be on a separate line.

- `comment`

Indicates a comment. Any line beginning with an asterisk (*) is a comment and will be ignored.

See the "Examples" section for examples of this syntax.

- `bjt_ratio = value`

Specifies the ratio of total current drawn by all bipolar junction transistors (BJTs) in the subcircuit. You can only specify this value with the `DEF` or `INST` option. If you do not specify a value, the simulator uses the default value set by the `PwrnetSpecifyBjtRatio` environment variable.

- `res_ratio = value`

Specifies the ratio of the total current drawn by all resistors in the subcircuit. You can only specify this value with the `DEF` or `INST` option. If you do not specify a value, the simulator uses the default value set by the `PwrnetResRatio` environment variable.

- `diode_ratio = value`

Specifies the ratio of total current drawn by all diodes in the subcircuit connected to a power grid. You can only specify this value with the `DEF` or `INST` option. If you do not specify a value, the simulator uses the default value set by the `PwrnetDiodeRatio` environment variable.

- `POWER = value`

Specifies the power consumption for the specified subcircuit definition, instance, or device.

- `CURRENT = value`

Specifies the current drawn by the specified subcircuit definition, instance, or device.

Example

In the following example, a device named `m1` draws a current of 0.2 μ A:

```
pwrnet specify DEV = m1 current = 0.2u
```

In the next example, a subcircuit block named `x1` dissipates 0.1 mW of power. The specified power value is divided among all the devices connected to voltage source nodes in the block.

```
pwrnet specify INST = x1 power = 0.1m
```

In the example that follows, all subcircuit instances of the `nand2` cell draw 0.1 μ A of current:

```
pwrnet specify DEF = nand2 current = 0.1u bjt_ratio = 0.8
```

The cell contains BJT devices, and `bjt_ratio` is defined as 80 percent of the total (0.1 μ A) current. This percentage is divided among the BJTs, and the remaining 20 percent is divided among the rest of the devices connected to the voltage supply node. Similarly, if the device contains resistors or diodes, the percentage of the total current defined

by `res_ratio` or `diode_ratio` is divided among the resistors or diodes. The percentage of the total current remaining is then divided among the other devices, for example, MOSFETs.

Finally, in this example, the simulator reads a group file named `file.grp`:

```
pwrnet specify group_file = file.grp
```

The contents of `file.grp` are as follows:

```
group1 current = 0.2u
{
    m1
    m2
}
group2 power = 0.1u
{
    m3
    m4
}
```

The group file contains two device groups. In `group1`, the specified devices, `m1` and `m2`, both collectively draw 0.2 μA of current. In `group2`, the two devices dissipate 0.1 μW of power. The distribution of current in individual devices is computed as described earlier in this section. If the same device appears in multiple groups, the device current computed in the last group is used.

pwrnet tally

The `pwrnet` tally command tallies the average, maximum, and RMS currents of all transistors connected to the specified voltage source for the current dynamic simulation. The `pwrnet report` command creates a report containing these tallies.

Syntax

```
pwrnet tally vsrc_name
```

`vsrc_name` specifies the name of the voltage source.

Example

The following line tallies current drawn by all the transistors connected to the `vdd` voltage source:

```
pwrnet tally vdd
```

pwrnet tallyint

The `pwrnet tallyint` command reports the dynamic current of all devices connected to the specified voltage source during dynamic simulation. The current information is used by Lightning (VST) for dynamic analysis of the power grid connected to the voltage source.

Use `pwrnet tallyint` before running a dynamic simulation. Library Simulator calculates peak, RMS, and average current for each device during each interval, and averages these measurements over as many periods as fit into the duration of the simulation.

Library Simulator stores the data obtained in three files:

- The first file, `filename.ptimax`, contains the peak dynamic current of all devices connected to the voltage source.
- The second file, `filename.ptirms`, contains the RMS current of all devices connected to the voltage source.
- The third file, `filename.ptiavg`, contains the average current.

In all cases, the default file name is the name of the voltage source. You can specify another file by using the `file` option.

Syntax

```
pwrnet tallyint <file=filename> <method=[0|1|2] [0|1|2] [0|1|2]> intervals=n period=time vsrc_name
```

This syntax contains the following parameters:

- `filename`
Specifies the output file name.
- `method`
Specifies the method that the simulator uses in performing `tallyint` calculations. It is a three-position variable, with each position containing a 0 or a 1:
 - The first position specifies the method of peak current calculation.
 - The second position specifies the method of average current calculation.
 - The third position specifies the method of RMS current calculation.
In each position:
 - 0
Reports the maximum out of each interval over all periods.
 - 1
Reports the average out of each interval over all periods.
 - 2
Does not calculate peak, average, and RMS current.

The default is `000`.

- `intervals`
Specifies how many intervals the simulator tallies. Library Simulator divides each period into *n* number of intervals; calculates peak, RMS, and average current for each device during each interval; and averages these measurements over the periods of the simulation.
- `period`
Specifies the portion of the dynamic simulation for which the simulator tallies the current of each device. Library Simulator divides the duration of the simulation by *time* to derive as many periods as it takes to cover the entire simulation. Each period is in seconds.
- `time`
Specifies the number of units of time. Include the unit identifier:
 - `s` (seconds)
 - `ms` (milliseconds)
 - `µs` (microseconds)
 - `ns` (nanoseconds)
 - `ps` (picoseconds)
 - `fs` (femtoseconds)The default is ns.
- `vsrc_name`
Specifies the voltage source connected to the power grid.

Example

The following line tallies the currents for each of the devices connected to the `vdd` voltage source. Library Simulator tallies for 100 intervals over a period of 10 ns, reporting the maximum average and RMS current over all periods. The simulator calculates the average current for each interval as the average over all periods, and calculates the maximum and RMS current for each interval as the maximum current in every interval over all periods. The current information generated by the `tallyint` command is used by Lightning (VST) to perform power-grid analysis.

```
pwrnet tallyint file=vdd method=010 intervals=100 period=10ns vdd
```

pwrnet tgloff, pwrnet tglon

The `pwrnet tglon` command activates (and `pwrnet tgloff` deactivates) all power-net tallying during dynamic simulation. Library Simulator performs power-net tallying when any specified node toggles high or low, depending on your specification, for `count` number of times. If you do not specify the `high` or `low` switch, the simulator counts all toggles. If you do not specify `count`, the simulator activates (or deactivates) all power-net tallying each time a node toggles.

Syntax

```
pwrnet [tgloff|tglon] <params=tglcnt_param> <count=n> <high|low> [ngroup|node]+
```

This syntax contains the following parameters:

- `params`
Measured power-net information against a specific set of parameters. You build toggle-count parameter sets by using the `tglcnt params` command. (See the "[tglcnt params](#)".)
- `tglcnt_param`
If you omit this argument, the simulator uses default toggle-count parameters. You can find the defaults in the description of `tglcnt params`.
Specifies the name of the parameter set from which the simulator determines `vt`.
- `count`
Counts node toggles.
- `n`
Specifies the number of node toggles that the simulator registers before activating or deactivating power-net tallying and output.
- `high`
Counts the number of times that a node toggles high.
- `low`
Counts the number of times that a node toggles low.
- `ngroup`
Specifies the name of a predefined group of nodes for which the simulator registers toggling. Use the `group nodes` command to establish node groups.
- `node`
Specifies the name of a node for which the simulator registers toggling.

Example

The following line turns power-net checking off at the fourth high toggle of the node named clock:

```
pwrnet tgloff params=ttl_prm count=4 high clock
```

pwrnet time

The `pwrnet time` command turns power-net checking on or off during the specified time interval. The default is on.

Syntax

```
pwrnet time [ON=time|OFF=time]+
```

Time specifies the number of units of time. Include the unit identifier:

- `s` (seconds)
- `ms` (milliseconds)
- `μs` (microseconds)
- `ns` (nanoseconds)
- `ps` (picoseconds)
- `fs` (femtoseconds)

The default is `ns`.

Example

The following line turns power-net checking on 30 ns into a simulation:

```
pwrnet time ON=30ns
```

region

The `region` command works with region tolerances.

The `region` commands are the following:

- `region params`
- `region setd`
- `region setn`

region params

The `region params` command defines a set of region tolerances. You can control tolerances on a region-by-region basis. This command enables you to define or change tolerance parameters. You then can use the `region setn` and `region setd` commands to assign these parameters to specific regions.

Syntax

```
region params [default|param_name] [rtol=voltage |  
atol=voltage | vstep=voltage | iter_max=n | minstep=stepsize] +
```

This syntax contains the following parameters:

- *default*
Specifies the default parameter set. Specify *default* to change the default values for the duration of this session.
- *param_name*
Specifies the name of the parameter set containing a specific value for one or more parameters.
- *rtol=voltage*
Specifies the relative region voltage tolerance. This tolerance is the relative voltage tolerance in the region linear relaxation convergence calculation. The default is set by the `DefaultRegionRelativeTolerance` .simplexrc environment variable.
- *atol=voltage*
Specifies the absolute region voltage tolerance. This tolerance is the absolute voltage tolerance in the region linear relaxation convergence calculation. The default is set by the `DefaultRegionAbsoluteTolerance` .simplexrc environment variable.
- *vstep=voltage*
Specifies the voltage step, which determines how often a region and its fanouts are processed with respect to the voltage changes in the region. Smaller voltage steps yield higher accuracy in region behavior at the cost of higher processing time. Small voltage steps are recommended for regions containing nodes exhibiting smaller voltage fluctuations, such as analog nodes and sense amplifiers. The default is set by the `DefaultRegionEventThreshold` .simplexrc environment variable.
- *iter_max=n*
Specifies the maximum number of iterations, which is the maximum number of iterations allowed in region linear relaxation calculation. The default is set by the `DefaultRegionIterationLimit` .simplexrc environment variable.
- *minstep=stepsize*
Specifies the minimum time step size used to evaluate the region. The default is set by the `DefaultRegionMinTimeStep` .simplexrc environment variable.

Example

The following line resets two default region parameters, *vstep* and *iter_max*, for the duration of the current session:

```
region params default vstep=0.5v iter_max=50
```

region setd

The region setd command sets region tolerances for regions containing the specified devices.

Syntax

```
region setd <params=region_param> [device|dgroup]+
```

This syntax contains the following parameters:

- *params*
Applies specified region tolerances to the regions confining the specified devices.

If you omit this argument, the simulator uses default region parameters. You can find the defaults in "[region params](#)".
- *region_param*
Specifies the name of the region parameter set to use in region processing. You build region parameter sets by using the `region params` command.
- *device|dgroup*
Specifies the name of a device or predefined group of devices. Use the `group device` command to establish device groups.

Example

The following line sets regions containing the r1 and mos2 devices to the tolerances defined by the `rgn#1` parameter set:

```
region setd params=rgn#1 r1 mos2
```

region setn

The region setn command sets region tolerances for regions owning the specified nodes.

Syntax

```
region setn <params=region_param> [ngroup|node]+
```

This syntax contains the following parameters:

- *params*
Applies specified region tolerances to the region containing the specified nodes.

If you omit this argument, the simulator uses default region parameters. You can find the defaults in the description of `region params`. (See "[region params](#)".)

- *region_param*
Specifies the name of the region parameter set that the program uses in region processing. Use the `region params` command to build region parameter sets.
- *ngroup*
Specifies the name of a predefined group of nodes. Use the `group nodes` command to establish node groups.
- *node*
Specifies the name of a node.

Example

The following line sets regions containing all nodes with names that start with "node" and end with one additional character to the parameters defined by the `rgn#1` parameter set:

```
region setn params=rgn#1 node?
```

setenv

The `setenv` command sets any `.simplexrc` environment variable to a new value.

Syntax

```
setenv env_var value
```

This syntax contains the following parameter:

- *env_var*
Specifies any `.simplexrc` environment variable.
- *value*
Specifies an appropriate new value for that environment variable.

Example

The following line sets the `GroundName` environment variable to `VSS`:

```
setenv GroundName vss
```


stats

The stats command observes the status of a Library Simulator session. The resulting report contains information about the circuit elements, memory usage, and the time taken by various operations.

Syntax

```
stats
```

This command has no arguments.

The typical output of a `stats` report follows:

The circuit netlist contained:

```
855 circuit nodes
2762 MOS transistors
10 voltage sources
```

Loading the circuit required:

```
2.18 user, 0.05 system, and 3.28 real seconds
```

Partitioning the circuit required:

```
0.03 user, 0.00 system, and 0.05 real seconds
```

The DC solution required:

```
0.35 user, 0.00 system, and 0.35 real seconds
```

Dynamic analysis required:

```
0.19 user, 0.01 system, and 0.30 real seconds
```

Total memory utilization:

```
maximum heap memory usage: 2804k
current heap memory usage: 2740k
```

The program memory utilization:

```
maximum memory usage: 895k
memory allocated for nodes: 127k + 63k
memory allocated for subcircuits: 63k
memory allocated for devices: 255k
memory allocated for names: 63k + 127k
```

```
memory allocated for snames: 0k + 0k
```

In this example, the simulator reports the number of circuit nodes and elements and the memory used by the circuit along with the circuit loading, partitioning, and analysis times.

tglcnt

The `tglcnt` command works with toggle count information.

A toggle is either rising past the logic voltage level or falling past the logic voltage level. You can define the logic voltage level by using the optional `tglcnt params` command. (See "[tglcnt params](#)".)

The `tglcnt` commands are the following:

- `tglcnt check`
- `tglcnt clear`
- `tglcnt off`
- `tglcnt on`
- `tglcnt params`
- `tglcnt report`
- `tglcnt tally`

tglcnt check

The `tglcnt check` command checks the toggle count using the parameters specified in the `tglcnt_param` parameter set. Use this command to detect nodes that toggle excessively in an interval of time. Excessive toggling is associated with circuit hazards.

Syntax

```
tglcnt check <params=tglcnt_param> <label="label"> [ngroup|node]+
```

This syntax contains the following parameters:

- `params`
Measures toggle information by using a specific set of parameters. Use the `tglcnt params` command to build toggle parameter sets. (See "[tglcnt params](#)".)

If you omit this argument, the simulator uses default toggle parameters. You can find the defaults in the description of `tglcnt params`.
- `tglcnt_param`

Specifies the name of the parameter set that the simulator measures with.

- `label`

Labels each message generated by this command.

- `"label"`

Specifies the label text. Enclose this string in double quotation marks.

- `ngroup`

Specifies the name of a predefined group of nodes. You establish node groups by using the `group node` command.

- `node`

Specifies the name of a node.

Example

The following line checks toggle frequency for `node2` and `node3` by using the parameters in the `tglprm2` parameter set:

```
tglcnt check params=tglprm2 node2 node3
```

tglcnt clear

The `tglcnt clear` command clears toggle count information from tallies.

Syntax

```
tglcnt clear
```

This command has no arguments.

tglcnt off, tglcnt on

The `tglcnt on` command activates (and `tglcnt off` deactivates) the tallying and checking of toggling in the specified circuit. The default is `tglcnt on`.

Syntax

```
tglcnt [off|on]
```

This command has no arguments.

tgcnt params

The `tgcnt params` command specifies the voltage level used to define the toggle time of the specified node.

Use this command to define one or many timing sets. Because this command supports multiple node sensitivities, each timing set can be applied to any number of nodes.

Syntax

```
tgcnt params [default|tgcnt_param] <vt=voltage> <interval=time> <int_count=n>
```

This syntax contains the following parameters:

- `default`
Specifies the default parameter set. Specify `default` to change the default values for the duration of this session.
- `tgcnt_param`
Specifies the name of the parameter set containing a specific value for one or more parameters.
- `vt=voltage`
Specifies the intermediate-logic threshold. The default is set by the `DefaultParamVt` .simplexrc environment variable.
- `interval`
Specifies the duration of time during which the simulator counts toggles using the `int_count` option. The default is 0, resulting in every toggle counted.
- `time`
Specifies the number of units of time. Include the unit identifier:
 - `s` (seconds)
 - `ms` (milliseconds)
 - `μs` (microseconds)
 - `ns` (nanoseconds)
 - `ps` (picoseconds)
 - `fs` (femtoseconds)The default is `ns`.
- `int_count`
Specifies the number of toggles that the simulator counts within the interval specified by `interval`. The default is 1. Library Simulator takes a specified action when the toggle count is equal to or greater than the specified number within an interval.

Example

The following line defines a trigger condition as two toggles within 1 ns:

```
tglcnt params default interval=1ns int_count= 2
```

tglcnt report

The tglcnt report command reports the results of the individual node toggle count.

Syntax

```
tglcnt report <<append> filename>
```

This syntax contains the following parameters:

- `append`
Appends the report to the specified file.
- `filename`
Specifies the name of the file to which the simulator appends the report.

Example

The following line reports the results of the individual node toggle count:

```
tglcnt report
```

tglcnt tally

The tglcnt tally command counts the number of toggles at a specified node.

If you do not specify `params`, the simulator uses the default, `vt`, which is set by the `DefaultParamVt` `.simplexrc` environment variable.

Syntax

```
tglcnt tally <params=tglcnt_param> <label="label"> [ngroup|node]+
```

This syntax contains the following parameters:

- `params`
Measures toggle tally information by using a specific set of parameters. Use the `tglcnt params` command to build toggle tally parameter sets. (See ["tglcnt params"](#).)

If you omit this argument, the simulator uses default timing tally parameters. You can find the defaults in the

description of `tglcnt` params.

- `tglcnt_param`
Specifies the name of the parameter set that the simulator uses to measure.
- `label`
Labels each message generated by this command.
- `"label"`
Specifies the label text. Enclose this string in double quotation marks.
- `ngroup`
Specifies the name of a predefined group of nodes. Use the `group nodes` command to establish node groups.
- `node`
Specifies the name of a node.

Example

The following line tallies toggles for `node1` and `node2` using the parameters specified in the `tgl_param1` parameter set: `tglcnt tally params=tgl_param1 node1 node2`

vcd

The `vcd` command controls input toggling to the simulator by using data specified in VCD (value change dump) format. Library Simulator reads the VCD file and, by default, drives the nodes listed in that file with the values listed in it.

In addition, the `vcd` command enables you to use the data from the VCD file to compare logic-level simulation with results from the simulator.

Syntax

```
vcd <signal=signal_filename> <action=action_filename> <scope=scope_id> <reset> vcd_filename
```

This syntax contains the following parameters:

- `signal_filename`
Specifies the name of the signal declaration file, which defines directions for the signals placed in the VCD files. If you do not specify a signal declaration file, the simulator considers all signals in the VCD file to be of type `input` and therefore all values in the VCD file to be stimuli.

See ["Signal Declaration File"](#) for complete information on signal declaration files.

- `action_filename`
Specifies the name of the action file. This file defines the following:

- Times and events at which the simulator checks
- Times and events used to toggle the state of bidirectional signals
- Nodesets
- Parameters

If you do not specify an action file, the simulator does not check.

See ["Action File"](#) for complete information on action files.

- *scope_id*

Defines the scope of the command. This parameter acts as a selector for the signal declaration and action file. If you do not specify a scope, the default scope is the entire VCD file.

- *reset*

Executes the `reset` command before executing the `vcd` command.

- *vcd_filename*

Specifies the name of the VCD file. See ["VCD File"](#) for information on the VCD file.

Example

The following line uses `dump1` as the VCD input and resets the circuit before driving the node:

```
vcd dump1 reset
```

VCD File

The VCD file is the standard Verilog VCD file described in OVI documents and IEEE 1364. Library Simulator uses the following conventions to translate VCD values into input stimuli.

VCD Values

Within the VCD file, these VCD values have the following meanings during driving:

- 0
Forces the associated node to logic low.
- 1
Forces the associated node to logic high.
- X
Forces the associated node to logic unknown.
- Z
Releases the associated node.

VCD Command Extensions

The following extensions of the VCD format are supported. You can use these command extensions in the comment section of a VCD file to specify node characteristics; however, Simplex recommends that you specify node sets and parameters in the action file.

Use these extensions to specify properties such as input voltages and rise and fall times of nodes listed in the VCD file.

- `*scope`

Sets the scope of the `nodeset` command by using the `scope` command. *Scope_string* is prepended to the nodes listed in the `nodeset` command. The syntax is as follows:

```
*scope scope_string *end
```

- `*nodeset`

Groups nodes in the VCD file into node sets. You can then set properties of the nodes in the node set by using the `params` command on the node set. The syntax is as follows:

```
*nodeset set_number [node_name]+ *end
```

- `*params`

Assigns parameters to sets of nodes by using the `params` command. The syntax is as follows:

```
*params set_number <vin0=voltage> <vin1=voltage> <vinx=voltage><tir=time> <tif=time> *end
```

Example VCD File

Following is a sample VCD file named `vcd`. It is the same as that shown in ["stats"](#) but is in VCD format.

```
$date
Mon Oct 31 15:09:59 1996

$end

$version
Cadence 1.0

$end

$timescale
1ns

$end

$scope module TOP $end
```



```
$var wire 1 ! A[7] $end
$var wire 1 " A[6] $end
$var wire 1 # A[5] $end
$var wire 1 $ A[4] $end
$var wire 4 % A[3:0] $end

$upscope $end

$comment

*scope TOP *end
*nodeset 1 A[7:0] *end
*params 1 vin0=0v vin1=5v vinx=2.5v tir=1ns tif=1ns *end

$end

$enddefinitions $end

#0
1!
1"
1#
1$
b1111 %
#50
0!
0"
0#
0$
b0000 %
#50
1$
b0001 %
```

```
#50
1#
0$
b0010 %
#50
1$
b0011 %
#50
1"
0#
0$
b0100 %
```

Signal Declaration File

The signal declaration file defines directions for the signals placed in the VCD files. The signal declaration file contains blocks of the following form.

Syntax

```
keyword list_of_signal_names;
```

This syntax contains the following parameters:

- *keyword*
Specifies the signal direction. It can be any of the following:
 - *input*
Specifies the events that the simulator uses as stimuli.
 - *output*
Specifies the signal level that the simulator checks against.
 - *inout*
Specifies bidirectional signals. Library Simulator considers these of type input at the beginning of the simulation. You must explicitly toggle these signals into a different state by using commands in the action file.
 - *wire*
Specifies the internal node that the simulator ignores.

- *list_of_signal_names*

Lists the simple signal identifiers. Separate multiple names with commas.

Example

The following line checks the signal levels in the `out[1]`, `out[2]`, and `out[3]` signals:

```
output out[1], out[2], out[3];
```

Action File

The action file defines the times and events at which the simulator performs checking, executes commands, and toggles the state of the bidirectional signals. The action file contains the following kinds of statements.

Nodeset Statements

The `nodeset` statement defines a set of signals for which a specific parameter set is applied.

Syntax

```
nodeset set_number [{<[signal_name]>+}|keyword];
```

Note: Braces (`{ }`) enclose the signal names. You must enter the braces.

This syntax contains the following parameters:

- *set_number*
Specifies the number of the set that you are defining with this `nodeset` statement.
- *signal_name*
Identifies a simple signal. Separate multiple names with commas. If you do not specify a signal name, the simulator checks all signals of type output and bidirectional that are toggled into the output state.

The signal name must be defined in the VCD file.

- *keyword*
Specifies the signal direction. Library Simulator applies the parameter set to all signals of this signal type. It can be any of the following:
 - *input*
Specifies the events that the simulator uses as stimuli.
 - *output*
Specifies the signal level that the simulator checks against.

- `inout`
Specifies bidirectional signals. Library Simulator considers these of type input at the beginning of the simulation. You must explicitly toggle these signals into a different state by using commands in the action file.
- `wire`
Specifies the internal node that the simulator ignores.

Note: All `nodeset` and `parameter` statements must be set before the program's first check or the `assign` statement.

Examples

Following are three examples of the `nodeset` statement.

Example 1:

```
nodeset 1 { clk, ii_a0_dbl_rd_r_a1 };  
  
params 1 vin0=0.0 vin1=3.3 tir=0.1ns tif=0.1ns vout1=(-0.5,+0.5) vouth=(3.0,3.6) vth=(1.6,1.7);
```

All signals that are listed between the brackets are grouped into the node set labeled `1`. The label is used as the identifier for the following `params` statement, defining the voltage ranges for stimuli and checking the identified signals.

Example 2:

```
nodeset 1 input;  
  
params 1 vin0=0v vin1=2.25v vinx=1.125v tir=0.1ns tif=0.1ns;  
  
nodeset 2 output;  
  
params 2 voutL=(-0.2v, 0.6v) voutH=(1.65v, 2.45v) vth=(0.6v, 1.65v);
```

The second example uses keyword identifiers. All signals declared of type input are labeled with the node set identifier `1`. The signal direction can be either defined in the VCD file or declared in the signal file. The node set identifier specifies the stimuli waveform parameter. The second set groups all signals declared as output signals in the VCD file into node set `2` and defines the voltage ranges for checking during the simulation.

Example 3:

```
nodeset 1 { } ;  
  
params 1 vin0=0v vin1=1.8v vout1=(-0.2v,0.2v) vouth=(1.6v,2.0v) tir=80ps tif=80ps;
```

An empty signal list for the `nodeset` statements groups into one node set all signals of type output, as well as all bidirectional signals that are toggled into output mode. The node set label defines the voltage ranges for validation.

Library Simulator Variables

allowMosAsSubckt

Syntax

```
setenv allowMosAsSubckt [true | false]
```

Used for for netlist based detailed power-grid view generation. If set to `true` subckt is supported.

analysisReportSize

The `analysisReportSize` variable limits the number of lines in the simulator analysis reports.

Syntax

```
setenv analysisReportSize number
```

The default number of lines is 200.

Example

The following example sets the maximum number of lines in the the simulator analysis reports at 100:

```
setenv analysisReportSize 100
```

busNotation

The `busNotation` variable specifies the bus notation.

Syntax

```
setenv busNotation notation
```

The default notation is `[ : ]`.

Example

The following example specifies bus notation of `< : >`:

```
setenv busNotation < : >
```

caseSensitive

The `caseSensitive` variable determines whether the simulator is case-sensitive or -insensitive.

Syntax

```
setenv caseSensitive [true|false]
```

When you set this variable to `true`, the simulator is case-sensitive; when you set it to `false`, it is case-insensitive. The default is `false`.

Example

Library Simulator treats nodes as case-sensitive when the `caseSensitive` variable is set as follows:

```
setenv caseSensitive true
```

cmdBreakOnError

The `cmdBreakOnError` variable specifies whether the simulator stops processing when it encounters a command-line error.

Syntax

```
setenv cmdBreakOnError [true|false]
```

When you set this variable to `true`, the simulator stops processing when it encounters a command-line error; when you set it to `false`, the simulator continues processing. The default is `true`.

Example

In the following case, the simulator continues processing when it encounters a command-line error:

```
setenv cmdBreakOnError false
```

CoupledCapOnExtractedNetlist

Performs coupled cap analysis directly on the specified netlist, which in turn captures the effect of parasitics as well.

Syntax

```
setenv CoupledCapOnExtractedNetlist {true | false}
```

The default value is `true`.

DCInductorResistance

The `DCInductorResistance` variable specifies the small resistance value that the simulator uses to model all inductors during static analysis.

Syntax

```
setenv DCInductorResistance value
```

The default value is 1.0e-3 ohms.

Example

This example sets a value of 1.0e-2 ohms by which to model all inductors during static analysis:

```
setenv DCInductorResistance 1.0e-2
```

defaultTRiseFall

The `defaultTRiseFall` variable defines the default value for the rising and falling time for signals used as stimuli by the `params` command.

Syntax

```
setenv defaultTRiseFall value
```

The default value is 5e-10 seconds.

Example

This example specifies a default rise and fall time of 6e-10 seconds:

```
setenv defaultTRiseFall 6e-10
```

errorIfDeviceIsBelowMinW_L

The `errorIfDeviceIsBelowMinW_L` variable determines whether the simulator terminates and issues an error message or continues processing and issues a warning if it encounters a device whose length or width is below the minimum length or width defined in the table models.

Syntax

```
setenv errorIfDeviceIsBelowMinW_L [true|false]
```

When you set this variable to `true`, the simulator terminates and issues an error message when it encounters a device whose length or width is below the minimum length or width defined in the table models. When you set it to `false`, the simulator issues a warning message and continues processing. The default is `true`.

Example

In the following example, the simulator issues a warning message and continues processing when it encounters a device whose length or width is below the minimum length or width defined in the table models:

```
errorIfDevicesBelowMinW_L false
```

forwardDiodeIpeakCurrent

The `forwardDiodeIpeakCurrent` variable defines the current to apply for forward-biased parasitic MOS diodes.

Syntax

```
setenv forwardDiodeIpeakCurrent value
```

The default is 1.0e+6 amperes.

Example

As a result of this example, the simulator applies a current of 2e+6 amperes to forward-biased parasitic MOS diodes:

```
setenv forwardDiodeIpeakCurrent 2e+6
```

gatePowerNetConnection

The `gatePowerNetconnection` variable determines if devices, whose only gate node is tied to the power supply, should also be reported in the tap file. If set to `true` all such devices will be reported as tap devices.

Syntax

```
setenv gatePowerNetConnection [true|false]
```

The default is `true`

groundName

The `groundName` variable establishes the alternate name (besides 0, gnd, and ground) used to define the circuit ground node. This variable is used by every `load` command.

Syntax

```
setenv groundName nodename
```

There is no default node name.

Example

This line specifies a ground node name of VSS:

```
setenv groundName vss
```

hierarchySeparator

The `hierarchySeparator` variable establishes the hierarchical delimiter.

Syntax

```
setenv hierarchySeparator character
```

The default character is a period (.).

Example

This example specifies a pipe (|) as the hierarchical delimiter:

```
setenv hierarchySeparator |
```

isDetailedDynamicView

Enables to write out all devices (including the zero cap and current) into the current region file whenever detailed dynamic views are created by LibGen with currents from UltraSim.

Syntax

```
setenv isDetailedDynamicView {true | false}
```

Library Simulator writes all the devices (even with zero cap and current) to the current region file.

Default: false

keepTopCell

The `keepTopCell` variable determines whether the simulator removes the top cell instantiation when writing out the current data, if the top level is made up of a single instance.

Syntax

```
setenv keepTopCell [true|false]
```

When you set this variable to `true`, the simulator retains the top cell instantiation when writing out the current data. When you set it to `false`, it removes the top cell instantiation. The default is `false`.

Example

In the following example, the simulator keeps the top cell instantiation when writing the current data:

```
setenv keepTopCell true
```

logLimit

The `logLimit` variable controls the number of error messages (of each type) to be displayed.

Syntax

```
setenv logLimit value
```

The default *value* is 0, which causes no suppression of error messages.

Example

The following line specifies a limit of 10 error messages:

```
setenv LogLimit 10
```

minimumCapacitance

The `minimumCapacitance` variable specifies the minimum capacitance value allowed in the simulator. Library Simulator issues a warning when the capacitance is less than this value.

Syntax

```
setenv minimumCapacitance value
```

The default value is 0 farads.

Example

The minimum capacitance is 0.05 farads in this example:

```
setenv minimumCapacitance 0.05
```

minimumInductance

The `minimumInductance` variable specifies the minimum inductance value allowed in the simulator. Library Simulator issues a warning when the inductance is less than this value.

Syntax

```
setenv minimumInductance value
```

The default value is 1e-18 henrys.

Example

This example sets a minimum inductance value of 1e-20 henrys:

```
setenv minimumInductance 1e-20
```

minimumNodeCapacitance

The `minimumNodeCapacitance` variable specifies the minimum total node capacitance allowed in the simulator. Library Simulator changes the node capacitance by this minimum value if the node capacitance is less than the specified value.

Syntax

```
setenv minimumNodeCapacitance value
```

The default value is 1e-18 farads.

Example

This line specifies a minimum node capacitance of 2e-18 farads:

```
setenv minimumNodeCapacitance 2e-18
```

minimumResistance

The `minimumResistance` variable specifies the minimum resistance allowed in the simulator.

Syntax

```
setenv minimumResistance value
```

The default value is 1e-03 ohms.

netlistInlineComment

The `netlistInlineComment` variable specifies the comment character used for comments on input cards in the netlist.

Syntax

```
setenv netlistInlineComment character
```

The default character is \$.

Example

This line specifies the pound sign (#) as the comment character:

```
setenv netlistInlineComment #
```

netlistTopCell

The `netlistTopCell` variable specifies the top cell name in the netlist that must be defined in the command file during power calculation if the same netlist is defined inside a sub-circuit block.

Syntax

```
setenv netlistTopCell cellname
```

Example

```
setenv netlistTopCell STH_ADDDF42_GY2_2
```

pwrnetSpecifyBjtRatio

The `pwrnetSpecifyBjtRatio` variable represents the ratio of the total current or power dissipated by all BJT devices in a subcircuit. It is used with the `pwrnet specify` command.

Syntax

```
setenv pwrnetSpecifyBjtRatio value
```

The default value is 0.8.

Example

This example sets the BJT ratio to 0.5:

```
setenv pwrnetSpecifyBjtRatio 0.5
```

pwrnetSpecifyCapCurrent

The `pwrnetSpecifyCapCurrent` variable can be used to allow specifying capacitor current in the `pwrnet specify` command. To allow specifying capacitor current set this value to `true` before loading the circuit.

Syntax

```
setenv pwrnetSpecifyCapCurrent [true|false]
```

The default value is `false`.

pwrnetSpecifyDiodeRatio

The `pwrnetSpecifyDiodeRatio` variable represents the ratio of the total current or power dissipated by all diodes in a subcircuit. It is used with the `pwrnet specify` command.

Syntax

```
setenv pwrnetSpecifyDiodeRatio value
```

The default value is 0.

Example

The following example specifies a ratio of 0.02:

```
setenv pwrnetSpecifyDiodeRatio 0.02
```

pwrnetSpecifyResistorRatio

The `pwrnetSpecifyResistorRatio` variable represents the ratio of the total current or power dissipated by all resistors in a subcircuit. It is used with the `pwrnet specify` command.

Syntax

```
setenv pwrnetSpecifyResistorRatio value
```

The default value is 0.05.

Example

This line sets the ratio at 1.0:

```
setenv pwrnetSpecifyResistorRatio 1.0
```

quitOnCmdError

The `quitOnCmdError` variable determines whether the simulator terminates and issues an error message if an error occurs during the execution of a command defined in a command file.

Syntax

```
setenv quitOnCmdError [true|false]
```

When you set this variable to `true`, the simulator aborts the execution when it encounters an error; when you set it to `false`, the simulator continues processing. The default is `false`.

Example

In the following case, the simulator stops execution when it encounters an error:

```
<img class="editor-inline-macro" src=
```

Power Analysis Options

This section lists the PowerMeter options (Power Objects) that do not have an equivalent option in Voltus. PowerMeter uses Power Objects, which indicate the kind of power, the type of method used to calculate the instance-based power consumption, the default for the frequency and voltage, the interconnect parasitic used, the clock specification for the design, and many others. These Power Objects can be included in Voltus using the `set_power_include_file` command.

- [calCellsToUseWaveEst](#)
- [calCellsToUseWaveEstFile](#)
- [calPinTransitionTime](#)
- [calRailExclude](#)
- [calSwitchingCells](#)
- [rDefaultNetCap](#)
- [rDefaultPinCap](#)
- [rDynamicSwitchAllBuffers](#)
- [rDynamicSwitchAllClockBuffers](#)

- [rDynamicSwitchAllFlops](#)
- [rEnhancedBlackboxMode](#)
- [rEnhancedBlackboxAvg](#)
- [rEnhancedBlackboxMax](#)
- [rIgnoreBidirCap](#)
- [rUseFastestClockDomain](#)
- [rUseLibAdjustTransitionTime](#)
- [rUseUserDefinedCurrent](#)

calCellsToUseWaveEst

pwrMetObj calCellsToUseWaveEst *cell_name*

Allows the user to force triangular estimation on certain cells, even when the ECSM information for those cells is available.

Parameters

<i>pwrMetObj</i>	Specifies the name of the power object. It can be any TCL string.
<i>cell_name</i>	The name of the cell that will use triangular waveform estimates rather than the enhanced ECSM information in the .lib file.

Example

- The following example causes PowerMeter to ignore the ECSM extension information for the xor cells:
chip_Pwr calCellsToUseWaveEst xor

calCellsToUseWaveEstFile

pwrMetObj calCellsToUseWaveEstFile *file_name*

Allows the user to force triangular estimation on certain cells, even when the ECSM information for those cells is available.

Parameters

<i>pwrMetObj</i>	Specifies the name of the power object. It can be any TCL string.
<i>file_name</i>	Specifies the file that includes the names of cells that will use triangular waveform estimates rather than the enhanced ECSM information in the .lib file.

Example

- The following example causes PowerMeter to ignore the ECSM extension information for the cells listed in `triangular_file.txt`:
`chip_Pwr calCellsToUseWaveEstFile triangular_file.txt`

calPinTransitionTime

pwrMetObj calPinTransitionTime pin_transition_time_file

You can also specify the name of a file that contains the transition time for specific pins or instances in the design.

Parameters

<i>pwrMetObj</i>	Specifies the name of the power object. It can be any TCL string.
<i>pin_transition_ time_file</i>	Specifies the name of the file containing the transition time for the pins or instances in the design. See "Example" for the format of this file and an example of it.

Note: There are three ways that PowerMeter gets transition times (slews): 1) from TWF, 2) a transition time file (*pin_transition_time_file*), or 3) a default transition time (`rDefaultTransitionTime`). Other than TWF where thresholds are given, the slew thresholds for pin transition time file and default transition time will be assumed to be 0 to 100 by default. If `rUseLibAdjustTransitionTime` switch is on, the slew thresholds will be assumed to be the ones defined in the first liberty (.lib) file.

Note: Any negative transition time read from command file or other input file will generate a fatal error.

Example

- The following example specifies a pin transition-time file named `mydesign.slew`:
ChipPwr calPinTransitionTime mydesign.slew
- Here is an example of the pin transition-time file:
VERSION "1.0"

CREATION "12"

CREATOR "NOIDEA"

PROGRAM "power.tcl"

DIVIDERCHAR "_"

DELIMITERCHAR "/"

DESIGN "PG22E2_audclk"

UNITS "TIME 1ps"

InstanceName/PinName Max-Rise Max-Fall Min-Rise Min-Fall

SLOPES ;

- Logic0/input 0.000000 0.000000 0.000000 0.000000

- BITCLK 0.519086 0.350941 0.519086 0.350941

- U123/A 0.519086 0.350941 0.519086 0.350941

- sm2288_clkdiv_110_clk_reg/Q 0.100453 0.062533 0.100453 0.062533

- sm2288_clkdiv_111_U70/A 0.291355 0.098510 0.291355 0.098510

- sm2288_clkdiv_111_U70/B 0.092222 0.070174 0.092222 0.070174

- sm2288_clkdiv_111_U73/A 0.101057 0.073799 0.101057 0.073799

END ;

The format of the pin transition-time file is as follows:

instance_name/pin_name max_rise max_fall min_rise min_fall

This syntax contains the following parameters:

- *instance_name*
Specifies the name of the instance to which the pin is attached.
- *pin_name*
Specifies the name of the pin.
- *max_rise*
Specifies the value of the maximum rise time.
- *max_fall*
Specifies the value of the maximum fall time.
- *min_rise*
Specifies the value of the minimum rise time.
- *min_fall*
Specifies the value of the minimum fall time.

A separator character is required between the instance and the pin names. If no separator character appears in a pin name, the pin is a top pin.

You must also enter in this file a delimiter character in quotation marks--for example, "/"--and the unit for the rise and fall time. If your design is hierarchical, you must specify a divider character to indicate the levels of hierarchy. You can optionally enter the following:

- The version number of the program that created the file
- The date that the file was created
- The creator of the file
- The name of the program that created the file
- The design name

LOADS ;

- plltsmc_audclk/OE 0.073629 NA

- sm2288_clkdiv_111_U76/C 0.010614 NA

```
- sm2288_clkdiv_111_U76/A 0.005532 NA
- sm2288_clkdiv_111_U76/B 0.006345 NA

END ;
```

PowerMeter uses only the slew-rate information from the pin transition-time file. It only takes slew information specified on input pins. Information shown above for `sm228_clkdiv_110_clk_reg/Q` will be ignored. It also does not take information from the `LOADS` section of the file.

You can generate a pin transition-time file using stand alone Tempus Timing Signoff Solution or in PrimeTime. In Tempus Timing Signoff Solution, use the `-Powermeter` option of the `report_slew` command.

See *Tempus Timing Signoff Solution* manuals for more detail.

In PrimeTime, follow these steps to output the transition times to a file:

1. Perform a delay or timing calculation before outputting the transition times.
 - Set the correct search path, link the appropriate libraries, and so forth.
 - Invoke PrimeTime by typing the following:
 - `pt_shell`
 - Use the `read_verilog` command to read in the Verilog netlist file.
 - Specify the top-level design name with the `current_design` and `link_design` commands.
 - Read in the DSPF or SPEF file by using the `read_parasitics` command.
 - Source the Cadence-provided script called `pt_dump_slopes_loads.tcl` by using the following command:

```
source pt_dump_slopes_loads.tcl
```

This TCL script is available in the toolbox provided with the PowerMeter software.

- Output the transition times for the nets in the design by issuing the following command:

```
pt_dump_slopes_loads top_level_design > filename.log
```

The name of the resulting pin transition-time file is `top_level_design.slopes`.

- You can submit this file to PowerMeter by using this TCL syntax:

```
pwrMetObj calPinTransitionTime top_level_design.slopes
```

calRailExclude

```
PwrMetObj calRailExclude rail_name
```

Sometimes PowerMeter may guess that a particular net is a power rail when it is not. For these cases the `calRailExclude` power object may be used.

Parameters

<i>pwrMetObj</i>	Specifies the name of the power object. It can be any TCL string.
<i>rail_name</i>	The name of the net that is not a rail.

Example

- This example specifies that `vdd_dummy` is not a rail.
`chipPwr calRailExclude vdd_dummy`

calSwitchingCells

pwrMetObj calSwitchingCells filename

When you use the vectorless timing-windows-based method of dynamic power-consumption calculation, you can switch all cell types in the design from 0 to 1 or 1 to 0 to observe the impact of simultaneously switching outputs on them. This feature is useful when you want to focus on a particular problem involving different cell types.

You can specify the name of a file that contains the names of all cell types to be switched from 0 to 1 or 1 to 0 and the minimum loading capacitance for each. The instance will be selected for power calculation only when its loading capacitance exceeds the minimum capacitance. See "[rDynamicSwitchAllClockBuffers](#)" for the format and an example of this file.

Parameters

<i>pwrMetObj</i>	Specifies the name of the power object. It can be any TCL string.
<i>filename</i>	Specifies the name of the file containing the optional minimum loading capacitance value for each cell type in the design. The instance will be selected for power calculation only when its loading capacitance exceeds the minimum capacitance. See " rDynamicSwitchAllClockBuffers " for information on the format of this file and an example of it.

File Format

The switching cells file is an optional input to PowerMeter for dynamic analysis. It is an ASCII file. Each line is in the following format:

```
cellname <min_loading_cap_value>
```

Example

- The following example specifies a switching cells file called `switch_599`:

```
ChipPwr calSwitchingCells switch_599
```

- Switching cells file example

```
cakbuf1 20fF
```

```
ff1    15fF
```

```
drv1
```

The instance will be selected for power calculation only when its loading capacitance exceeds the minimum capacitance.

rDefaultNetCap

```
pwrMetObj rDefaultNetCap capacitance
```

Specifies a default net capacitance value if no parasitic capacitance is available.

Parameters

<i>pwrMetObj</i>	Specifies the name of the power object. It can be any TCL string.
<i>capacitance</i>	Specifies the default net capacitance. Default is 0.

Example

- The following example specifies a default net capacitance of 0.5 picofarads:

```
ChipPwr rDefaultNetCap 0.5e-12
```

rDefaultPinCap

pwrMetObj rDefaultPinCap capacitance

Specifies the default pin capacitance.

Parameters

<i>pwrMetObj</i>	Specifies the name of the power object. It can be any TCL string.
<i>capitance</i>	Specifies the default pin capacitance. Default is 0.

Example

- The following example specifies a default pin capacitance of 10 femtofarads:
ChipPwr rDefaultPinCap 0.01e-12

rDynamicSwitchAllBuffers

pwrMetObj rDynamicSwitchAllBuffers threshold_loading_capacitance

When you use the vectorless timing-windows-based method of dynamic power-consumption calculation, you can direct PowerMeter to switch all buffer cells that have a loading capacitance greater than the specified value and observe the impact.

When this debugging command is issued, only these specified cells are switched during dynamic analysis. rDynamicSwitchAllBuffers only schedules the buffer cells in the design giving user the visibility of the buffers as they switch. Scheduling is still done using the timing window of the buffer pins in TWF.

Parameters

<i>pwrMetObj</i>	Specifies the name of the power object. It can be any TCL string.
<i>threshold_loading_capacitance</i>	Specifies a loading capacitance, in farads, for the buffers, above which PowerMeter switches them from 0 to 1 or 1 to 0. If the threshold value is less than 0.0, the switching feature is considered off.

Example

- In the following example, PowerMeter switches all buffers whose loading capacitance is greater than 30 femtofarads:
ChipPwr rDynamicSwitchAllBuffers 30fF

rDynamicSwitchAllClockBuffers

pwrMetObj rDynamicSwitchAllClockBuffers [0|1]

When you use the vectorless timing-windows-based method of dynamic power-consumption calculation, you can switch all clock buffers in the design and observe the impact.

When this debugging command is issued, only these specified cells are switched during dynamic analysis. rDynamicSwitchAllClockBuffers only schedules the clock buffers in the design giving user the visibility of the clock network switching. Scheduling is still done using the timing window of the clock buffer pins in TWF.

Parameters

<i>pwrMetObj</i>	Specifies the name of the power object. It can be any TCL string.
[0 1]	If set to 1, turns off the switching of the clock buffers If set to 0, turns on the switching of the clock buffers. This command has no default setting.

Example

- The following example directs PowerMeter to switch the clock buffers:
ChipPwr rDynamicSwitchAllClockBuffers 1

rDynamicSwitchAllFlops

pwrMetObj rDynamicSwitchAllFlops threshold_loading_capacitance

When you use the vectorless timing-windows-based method of dynamic power-consumption calculation, you can direct PowerMeter to switch all flip-flops and latches that have a loading capacitance greater than the specified value and observe the impact.

When this debugging command is issued, only these specified cells are switched during dynamic analysis. rDynamicSwitchAllFlops only schedules the flip-flops and latches in the design giving user the visibility of the flip-

flop and latches as they switch. Scheduling is still done using the timing window of the flip-flop and latch pins in TWF.

Parameters

<i>pwrMetObj</i>	Specifies the name of the power object. It can be any TCL string.
<i>threshold_loading_capacitance</i>	Specifies a loading capacitance, in farads, for the flip-flops and latches, above which PowerMeter switches them from 0 to 1 or 1 to 0. If the threshold value is less than 0.0, the switching feature is considered off. This command has no default setting

Example

- In the following example, PowerMeter switches all flip-flops and latches whose loading capacitance is greater than 20 femtofarads:
ChipPwr rDynamicSwitchAllFlops 20fF

rEnhancedBlackboxMode

```
pwrMetObj rEnhancedBlackboxMode [0|1]
```

To specify whether blackbox propagation takes clock pins and timing arcs into account.

Definitions:

The following definitions are needed for the output activity descriptions in the syntax section below.

- Blackbox
A blackbox is any instance that has no functional information. For PowerMeter there are two types of blackbox instances:
 - An instance with only a port view.
 - An instance with a .lib that lacks functional data.
- Designated clock pin
A pin on an instance that is designated as a clock because of a .lib clock tag (e.g. clock: true)

- Propagated clock pin
A pin on an instance (with or without a .lib) that has no clock tag, but has a clock network propagated to it.
- Related pin
An input pin that has a timing arc related to an output pin.

Parameters

<i>pwrMetObj</i>	Specifies the name of the power object. It can be any TCL string.
[0 1]	If set to 0, blackbox propagation does not take clock pins and timing arcs into account. If set to 1, see behavior described below. Default is 0.

If `rEnhancedBlackboxMode` is set to 1, the output activity will be as follows:

- Cell with port view only
 - With a propagated clock pin the output activity will be:
min (1/2 propagated clock pin activity, highest non-clock input activity)
 - Without a propagated clock pin the output activity will be:
same as input with highest activity
- Cell with .lib and timing arcs but no functional information.
 - With a designated clock pin output activity will be:
min (1/2 designated clock pin activity, highest non-clock related-pin input activity)
 - Without a designated clock pin output activity will be:
 - With a propagated clock pin:
min (1/2 propagated clock pin activity, highest non-clock related-pin input activity)
 - Without a propagated clock, output activity will be the same as the related pin with the highest activity.
- Cell with .lib but with no timing arcs or functional information.
 - With a designated clock pin output activity will be:
min (1/2 designated clock pin activity, highest non-clock input activity).
 - Without propagated clock pin, output activity will be same as the input with highest activity.

If a cell has multiple designated clock pins, the one with the highest activity will be considered to be the clock pin and any other clock pins will be treated as normal inputs. Propagated clock pins have lower priority and will be treated as normal inputs as well.

If a cell has no designated clock pins but multiple propagated clock pins, the propagated clock pin with the highest

activity will take precedence. Other propagated clock pins will be treated as normal inputs.

Example

- The following example specifies that clock pins and timing arcs are taken into account for blackbox propagation:
`chip_Pwr rEnhancedBlackboxMode 1`

rEnhancedBlackboxAvg

`pwrMetObj rEnhancedBlackboxAvg [0 | 1]`

Specifies to use the average toggle rate of related inputs for enhanced blackbox propagation.

Parameters

<code>pwrMetObj</code>	Specifies the name of the power object. It can be any TCL string.
<code>[0 1]</code>	• When you set this to 1, PowerMeter uses the average toggle rate of related inputs for enhanced blackbox propagation. • When you set this to 0, this functionality is not considered. <i>Default: 0</i>

rEnhancedBlackboxMax

`pwrMetObj rEnhancedBlackboxMax [0 | 1]`

Specifies to use the maximum toggle rate of related inputs for enhanced blackbox propagation.

Parameters

<code>pwrMetObj</code>	Specifies the name of the power object. It can be any TCL string.
------------------------	---

[0 1]	• When you set this to 1, PowerMeter uses the maximum toggle rate of related inputs for enhanced blackbox propagation. • When you set this to 0, this functionality is not considered. <i>Default: 0</i>
-----------	---

rIgnoreBidirCap

`pwrMetObj rIgnoreBidirCap [ 0 | 1 ]`

Ignores the bidirectional pin capacitances.

Parameters

<code>pwrMetObj</code>	Specifies the name of the power object. It can be any TCL string.
[0 1]	A value of 1 ignores the capacitance of bidirectional pins in switching power calculation (internal power calculation remains unaffected). <i>Default: 0</i>

rUseFastestClockDomain

`pwrMetObj rUseFastestClockDomain [ 0 | 1 ]`

When using clock domain-based assignments (`ChipPwr rNoClockDomains 0`) and if a net is assigned to multiple clock domains, the `rUseFastestClockDomain` setting will determine if the highest or lowest frequency will be used.

Parameters

<code>pwrMetObj</code>	Specifies the name of the power object. It can be any TCL string.
------------------------	---

[0 1]	With a value of 0, PowerMeter will use the lowest frequency for clock domain-based activity assignment. With a value of 1, PowerMeter will use the highest frequency for clock domain-based activity assignment. The default is 1.
-----------	--

This command will be ignored if `rNoClockDomains` is set to 1.

Note: When the activity of an input signal is unknown (i.e. no input activity from a file such as VCD or TCF), it will use the product of the default input activity and the frequency of the fastest (or slowest) clock associated with any pin connected to the net.

Example

- The following example specifies that PowerMeter will use the highest frequency for domain-based activity.
ChipPwr rUseFastestClockDomain 1

rUseLibAdjustTransitionTime

`pwrMetObj rUseLibAdjustTransitionTime [0|1]`

Provides a way of telling PowerMeter to use the transition threshold from the first .lib.

Parameters

<code>pwrMetObj</code>	Specifies the name of the power object. It can be any TCL string.
[0 1]	When set to 0, PowerMeter will treat slew thresholds as 0-100%. When set to 1, slews specified by <code>rDefaultTransitionTime</code> and pin transition time file will use thresholds specified in the first .lib. The default is 0.

Note: For TWF flow, the voltage thresholds given in TWF will be used irrespective of the value of this switch.

Example

- The following example specifies that the transition threshold should be 0 - 100:
chip_Pwr rUseLibAdjustTransitionTime 0

rUseUserDefinedCurrent

pwrMetObj `rUseUserDefinedCurrent` [0|1]

Determines whether PowerMeter uses Enhanced Port or Quick Detailed Views or not.

Parameters

<i>pwrMetObj</i>	Specifies the name of the power object. It can be any TCL string.
[0 1]	If set to 0 enhanced port views are ignored. If set to 1, enhanced port or quick detailed views take precedence over all other views (including .lib). The default is 0.

Example

- The following example tells PowerMeter not to use Enhanced Port views:
ChipPwr `rUseUserDefinedCurrent` 0

Rail Analysis Options

This section lists the advanced Rail Analysis commands and variables that can be included in Voltus IC Power Integrity Solution using the `set_advanced_rail_options` command. They are listed in alphabetical order.

- [Rail Analysis Commands](#)
- [label](#)
- [layer](#)
- [layer_bias](#)
- [oa_library](#)
- [oa_view_name](#)
- [print](#)
- [printvar](#)

- reuse_state
- suppress_message
- viacurrent
- write
- Rail Analysis Variable
- auto_scale_adjacent_via_layers
- cell_area_capacitance
- create_label_from_vsc
- decap_exclude_static_violation_area
- decap_window_size
- DEF_merge_supply_shorts
- em_part_check_current
- em_part_current_threshold
- error_on_op_analysis
- extractor
- extractor_include
- halt_on_net_with_no_cell_connections
- hierarchy_char
- internal_nodes_visible
- internal_resistors_visible
- libgen_executable
- libgen_include
- lifetime
- max_error_messages
- max_warning_messages
- merge_util
- pin_char
- precision_digits
- prpgv_x_offset
- prpgv_y_offset
- reff_ignore_disconnected_pins

- [use_average_power_for_op](#)
- [use_capacitor_adjustment](#)
- [use_current_data_from_pgv](#)
- [use_existing_rcdb](#)
- [use_gds_annotate_attributes](#)
- [use_port_voltage_for_iv](#)
- [write_bbv_pwl](#)
- [write_decap_filler_voltage](#)

Rail Analysis Commands

label

`label [port|net] label_string x y layer_name`

This command specifies location of text labels to the extractor. These labels are used to identify specific nets, port locations, or both.

Parameter

<code>port</code>	Indicates that the label identifies a port.
<code>net</code>	Indicates that the label identifies a specific net. The net must have a geometrical shape at the specified location.
<code>label_string</code>	Specifies the text of the label.
<code>x, y</code>	Specifies the location of the text label, in microns.
<code>layer_name</code>	Specifies the name of the layer on which the text label resides. This name must be the same as that in the technology file generated by RCgen.

Examples

- The following example indicates that a text label called `vdd` will identify a port located at 8.0, 0.2 on the `metal_4` layer:

```
label port vdd 8.0 0.2 METAL_4
```
- In the next example, a text label called `vdd` identifies a net with a geometrical shape located at 8.5, 5.8 on the `metal_1` layer:

```
label net vdd 8.5 5.8 METAL_1
```

layer

```
layer map def_layer_name tch_layer_name
```

```
layer gds gds_layer_name gds_layer_number <gds_data_type> <[port|text]>
```

This command defines layer mapping in order to create physical layers from multiple GDS layers. The layer commands are forwarded to extraction and influence only the top-level extraction.

Parameter

```
map def_layer_name tch_layer_name
```

Maps an input layer name to a target layer name. This command is used to map the layer name used in the input DEF file to a layer name in the technology file.

- *def_layer_name*
Specifies the name of the layer in the DEF file.
- *tch_layer_name*
Specifies the name of the layer in the technology file.

<pre>gds gds_layer_name gds_layer_number <gds_data_type> <[port text]></pre>	Maps the layer name and the layer type in the GDS file to a physical layer. • <i>gds_layer_name</i> Specifies the layer name in the GDS file. • <i>gds_layer_number</i> Specifies the layer number in the GDS file. It must be an integer between 0 and 255. • <i>gds_data_type</i> Specifies an optional number of the subclass for the GDSII layer. It must be an integer between 0 and 255. • <i>[port text]</i> Determines whether the geometrical shapes on the given layer are used to generate ports or the layer is used to attach text to a specific physical layer.
--	---

Examples

- The following example maps the DEF layer name, METAL1, to the layer name in the technology file, METAL_1:

```
layer map METAL1 METAL_1
```

- The next example specifies the layer name, the layer type, and the data type in the GDS file:

```
layer gds METAL1 16 0
```

- The last example specifies the layer name, the layer type, and the data type in the GDS file and indicates that the layer be used to attach text to a specific physical layer:

```
layer gds METAL1 16 0 text
```

layer_bias

```
layer_bias layer value
```

This command sets a bias value, in microns, by which to undersize all wires on a defined layer. It must be in the range of -1 to 1. A negative value will make the wire larger. This value is applied to each edge of the wire, decreasing the width by two times the bias value. You can select a different bias value for resistance extraction and capacitance extraction.

Parameter

<i>layer</i>	Specifies the name of the layer containing the wires to be biased.
<i>value</i>	Specifies the amount, in microns, by which to bias the wires. A positive value undersizes the width of the wire.

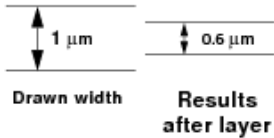
Examples

- The following example biases each edge of the wires in the `m5` layer by two tenths of a micron:

```
layer_bias m5 0.2
```

The results would resemble the diagram shown in the following figure:

Figure B-1 **Layer Biasing**



oa_library

```
oa_library library_name <cds_lib_path>
```

This command specifies the name of the OpenAccess 2.2 database generated by Cadence Innovus or Virtuoso or other applications that is to be used as input to VoltageStorm PE.

You must use the `oa_view_name` and `top_cell` commands with this command. See "[oa_view_name](#)" for information on the `oa_view_name` command.

Parameter

<i>library_name</i>	Specifies the name of the OpenAccess database.
<i>cds_lib_path</i>	Specifies the path to the <code>lib.defs</code> file, which contains the path name of the OpenAccess database. This parameter is optional. Use it if the OpenAccess database resides in a directory other than the default directory.

Examples

- The following example directs VoltageStorm PE to search for the cpu library in the `../home/cpu_pc788` directory:

```
oa_library cpu ../home/cpu_pc788/lib.defs
```

```
oa_view_name netlist
```

```
top_cell NFD2QP
```

oa_view_name

```
oa_view_name view_name
```

This command specifies the type of view to use when an OpenAccess 2.2 database is input to VoltageStorm PE.

You must use the `oa_library` and `top_cell` commands with this command. See "[oa_library](#)" for information on the `oa_library` command.

Parameter

<i>View_name</i>	Specifies the type of view. This command has no default, but <code>abstract</code> is usually used.
------------------	---

Examples

- This example specifies that the netlist view be used:

```
oa_library cpu
```

```
oa_view_name netlist
```

```
top_cell NFD2QP
```

print

```
print analysis_type filename
```

```
print iv filename <critical> [best_case | worst_case | average] <effective> <timing_window>
```

```
print pi filename instances
```

This command places in a specified file the results of filtering all power-grid data for the specified analysis type and

the specified area, layer, or selected nodes or resistors. It can also print the worst-case data for the specified analysis types, as well as the time that the worst case occurred. It is similar to the `list` command, except that it places the data in a file rather than displaying it on the screen. The filter for the analysis type must be set for the `print` command to report data.

The output data contains the following information:

- The data range
- The data value of the element or node
- The element name or node name
- A data-filtering summary

 ***The output file can be very large for large netlists.***

Parameter

<code>analysis_type</code>	Specifies one of the analysis types shown below.
<code>filename</code>	Specifies the name of the output file.

```
print iv filename
<critical>      <worst_case
| average> <effective>
<timing_window>
```

The command "print -iv *filename*" is for printing instance-based supply voltage data. This data contains instance-specific IR drop information suitable for use in Tempus.

Note: When `setvar save_voltage_waveforms true` environment variable is set before a `print iv` command, a binary file is created, not ascii instance supply voltage file. If this variable is not set before a `print iv` command, then the output will be an ascii instance supply voltage file.

The switching-window is the time interval during which non-leakage currents flow into the power and ground nets or when the gate is switching. In VCD flow the time is where a specific pin/net toggle is specified in the VCD file. PowerMeter then reads this information, computes the current waveform corresponding to the toggling and puts this waveform at the time specified in the VCD file. The switching window is the time interval of the current waveform due to the toggling. Similarly in vectorless flow, when a pin toggles, PowerMeter computes a current waveform corresponding to the switching. The difference between vectorless flow and VCD flow is that in vectorless flow the toggling time for the pins are determined by Powermeter while in VCD flow the toggling time is given by the VCD file.

`<critical>` prints the instance voltage values at critical instances (Those with a negative slack value). This is important for timing aware decap analysis.

Using `best_case` provides an option to write the best-case supply voltage per instance over the time intervals defined by `timing_window`. The best-case supply voltage for an instance is the pin voltage of the connected pin with the minimum IR -drop or ground-bounce over the `timing-window` time intervals (`timing_window` is required).

If none of the parameters `<critical>` `<best_case | worst_case | average>` `<timing_window>` is specified then the default dynamic behavior is the same as `print iv filename worst_case`. Since a timing window is not specified, the value may be too pessimistic for IR-drop aware delay calculation (i.e. Tempus).

<code>print</code> <i>pi filename instances</i>	Generates an ASCII report of all power gate cell instances. See discussion below.
---	---

Analysis Types

<code>ac</code>	Plots the available decoupling capacitance in the specified area. The available capacitance is calculated based on the available filler cells in the specified area and their matching decoupling cap cells.
<code>dd</code>	Prints the decoupling density, which is the amount of decoupling capacitance in a given area of the design.
<code>de</code>	Prints the decoupling capacitance effectiveness measured by the ratio peak IR drop divided by the average IR drop. This option applies only to dynamic analysis and generates a warning message if only static current data is available. VoltageStorm PE calculates the average IR drop over the whole transient analysis.
<code>dr</code>	Prints the decap required, which is the suggested amount of decap to add to fix IR-drop violations.
<code>er</code>	Prints electromigration risk data.
<code>fd</code>	Prints the filler-cell density, which is the number of filler cells per given area.
<code>ir</code>	Prints IR drop data
<code>jrms</code>	Prints jrms current density data based on lrms current. Only supported for dynamic.

iv	Prints instance-based supply voltage data. This data contains instance-specific IR drop information suitable for use in Tempus. In order for VoltageStorm PE to print the file of instance-based supply voltage data, the <code>print iv</code> command must be preceded by either the <code>analyze iv</code> or <code>filter iv auto</code> command. The format of this file is: <pre>INSTANCESUPPLY no_inst bus_name nom_volt [P G] instance voltage cell [power_pin [signal_pins]] ...</pre> • <code>no_inst</code> is the number of instances attached to the bus • <code>bus_name</code> is the name of the power or ground line • <code>nom_volt</code> is the nominal supply voltage • <code>P G</code> defines power or ground • <code>instance</code> is the instance name • <code>voltage</code> is the instance voltage • <code>cell</code> is the name of the cell • <code>power_pin</code> is the name of the power pin (for multi supply multi voltage cells) • <code>signal_pin</code> is the name of the signal pin (for multi supply multi voltage cells) See "<code>print iv filename <worst_case average> <effective> <timing_window></code>" described above, for additional details for <code>iv</code> option. Note: You must also set the variable <code>report_msmv_format</code> to create IR drop files for multi voltage cells.
pi	Prints the current vs. ID_{sat} and is used for seeing if too much current is flowing through power switches.
pv	Prints voltage drop across power switches. See <i>Power Gating</i> chapter of <i>VoltageStorm Cell_Level Rail Analysis User Guide</i> for details on Power Gating.
rc	Prints resistor current data.
rj	Prints DC current density data.
rv	Prints the voltage drop across resistors.
tc	Prints tap current data.
vc	Prints voltage source current data.

vu	Prints the voltage drop across the package model for both static and dynamic analysis and the worst-case IR drop for the limit and linear plots.
vv	Prints voltage across contacts and vias data.

- `print pi filename instances`

Generates an ASCII report of all power gate cell instances. Reports the instance name, cell name, Idsat threshold, total current and status. Reports the worst case voltage drop and suggests cell changes or cell additions. The report format is shown below:

Instance	Cell Type	Location	Threshold	Current	IR	Status	ChangeCell	AddCells
Head1_inst1	HEAD16	X, Y	1ma	2ma	1mV	FAILED	HEAD32	*
Head2_inst1	HEAD32	X, Y	2ma	5ma	2mV	FAILED	*	HEAD64(1)
Head3_inst1	HEAD16	X, Y	1ma	7ma	5mV	FAILED	*	HEAD64(1) HEAD32(1)
Head4_inst1	HEAD16	X, Y	1ma	0.5ma	7mV	PASS	*	*

- Threshold is the sum of Id_sat currents of the instance.
- Current is the total current across all transistors in the instance.
- IR is the worst-case IR-drop across all transistors in the instance.
- Status is FAILED if Current > Threshold otherwise it is PASSED.
- ChangeCell is the cell type available in the library with minimum Threshold (Idsat sum) and such that its Threshold is larger than the instance current in the case the instance status is FAILED.
- AddCells, if a cell for ChangeCell is not available in the library then put here the number of additional cell to add to make the total Threshold (current instance + additional cells) larger than the instance current. The format is "CellType (number of cell to add)".

Example:

```
Instance CellType Location Threshold Current IR Status ChangeCell AddCells
-----
u_switch_ring/u1_HS65_LS_SW3L60 HS65_LS_SW3L60 349200 15600 0.0104459 0.0353602 1.55524 FAILED *
HS65_LS_SW3L60 (3)
u_switch_ring/u2_HS65_LS_SW3L60 HS65_LS_SW3L60 349200 18200 0.0104459 0.0357668 1.57336 FAILED *
HS65_LS_SW3L60 (3)
u_switch_ring/u3_HS65_LS_SW3L60 HS65_LS_SW3L60 349200 20800 0.0104459 0.0360264 1.58469 FAILED *
HS65_LS_SW3L60 (3)
u_switch_ring/u361_HS65_LS_SW3L60 HS65_LS_SW3L60 468800 7800 0.0104459 0.00952458 0.420091 PASSED * *
```



```
u_switch_ring/u362_HS65_LS_SW3L60 HS65_LS_SW3L60 471400 7800 0.0104459 0.00924563 0.408337 PASSED * *
u_switch_ring/u358_HS65_LS_SW3L60 HS65_LS_SW3L60 461000 7800 0.0104459 0.0103802 0.457915 PASSED * *
u_switch_ring/u365_HS65_LS_SW3L60 HS65_LS_SW3L60 479200 7800 0.0104459 0.00872919 0.38559 PASSED * *
```

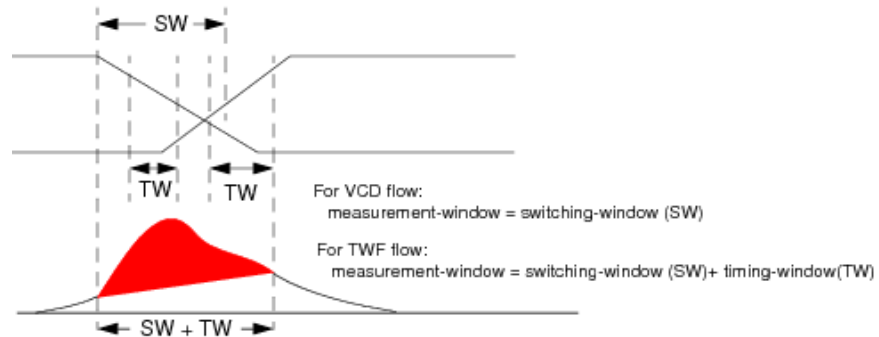


Figure B-2 Measurement Window

Examples

- The following line places all IR drop filtering data in the `ir_results` file.

```
print ir ir_results
```

- The next line places all instance-based voltage data in the `iv_results` file, whose syntax follows:

```
print iv iv_results
```

- and output file looks like

...

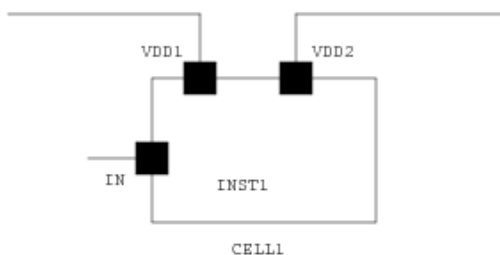
```
bic2efc__ebGntDst_reg_ECO0N01_2_L3_I1 1.754707e+00 BUFX8 ;
```

```
bic2efc__ebGntDst_reg_ECO0N01_2_L3_I12 1.756817e+00 BUFX8 ;
```

```
bic2efc__ebGntDst_reg_ECO0N01_2_L3_I11 1.756721e+00 BUFX8 ;
```

```
bic2efc__ebGntDst_reg_ECO0N01_2_L3_I21 1.756154e+00 BUFX8 ;
```

Figure B-3 Multi voltage example



In the above figure we show an instance, INST1, of a multi voltage cell, CELL1, with a VDD1 of 3 volts and VDD2 of 5 volts. The following output files will result when doing a `print iv iv_results`

VDD1 file

```
INSTANCESUPPLY 1 VDD1 3.0 P
    INST1 2.9 CELL1 PIN1;
```

VDD2 file

```
INSTANCESUPPLY 1 VDD2 5.0 P
    INST1 4.8 CELL1 PIN2 IN;
```

printvar

`printvar`

This command prints the settings of all the environment variables used in the current session of VoltageStorm PE.

Parameter

None

Example

- This example prints the setting of all the environment variables used in the current session.

```
printvar
```

reuse_state

`reuse_state state_directory`

This command directs VoltageStorm PE to use the instance-based power-consumption data output in the state directory by previous runs.

Instances in the previous run are identified by placement and orientation rather than by instance names, so power data can be reused, even if instance names have changed. For example, instance names may change when you switch from DEF-based to mixed DEF and GDS or GDS-based runs. You can only use this command if the placement between the two design iterations has not been changed.

Note: `reuse_state` is only supported for static analysis.

Parameter

<code>State_directory</code>	Specifies the name of the state directory where the information on the instance-based power-consumption data from the previous run resides.
------------------------------	---

Examples

- This example specifies that VoltageStorm PE reuse the data in the output files in the `vs2_output` directory:

```
reuse_state ../vs2_output
```

suppress_message

```
suppress_message message_id1 message_id2 ...
```

Suppress the output of the messages for the message ids that are specified.

Parameter

<code>message_id1</code> <code>message_id2 ...</code>	The messages with the specified message ids will not be output. <i>Default:</i> No messages are suppressed.
--	--

Examples

- The below command suppresses the messages with the ID `VST-1001`.

```
suppress_message VST-1001
```

viacurrent

```
viacurrent via_layer <xmin ymin xmax ymax> filename
```

This command generates a current data file to use as a power-grid macromodel.

To obtain power-grid macromodeling current data, identify a via layer that divides the power grid into upper and lower portions. VoltageStorm PE characterizes the current drawn into this via layer from the grid above it and creates the file.

When VoltageStorm PE solves the upper portion of the grid, it uses the current data in these files to macromodel the effects of the lower portion of the grid on the upper portion.

Parameter

<i>via_layer</i>	Specifies the name of a via layer.
<i>xmin, ymin, xmax, ymax</i>	Specify the coordinates of a rectangular area for which via current data is to be generated.
<i>filename</i>	Specifies the name of the output file.

Examples

- This example creates a file called `block.ivia`, which contains the current characterization data from a rectangular area defined by the coordinates 650400, 385400, 790000, and 583000 in the `via_1` layer:

```
viacurrent via_1 650400 385400 790000 583000 block.ivia
```

write

```
write [filters|models|powerpin_location_file|vs| gridchanges] filename
```

```
write block_boundary_voltage instance_name filename <-watch>
```

```
write eco filename
```

```
write [reduced_model | abstract_model]  
output_directory=output_directory_name state_directory=state_directory_name  
[power_domain_name=power_domain_name | net=net_name]  
[use_sigrity_repeat[=value]]  
[repeat=value]  
[port_grouping=[true|false]] [num_ports=number]  
[probing_node_file=probing_node_file_name]
```

This command produces ready-to-run command files that set up any of the following for all layers:

- Existing filters
- Existing model parameters
- Existing voltage sources
- Modifications performed on the power grid

Parameter

abstract_model	This parameter is equivalent to the following Voltus command parameter: <code>create_die_model -type two_port</code>. For more information, refer to create_die_model in the <i>Voltus Text Command Reference</i>.
filters	Writes the current filter settings for all analysis types using the <code>filter</code> command, where the <code>filter</code> command is written as follows: <pre>filter analysis_type filter_id min max</pre>
num_ports= <i>integer</i>	
	This parameter is equivalent to the following Voltus command parameter: <code>create_die_model -max_ports_per_net</code>. For more information, refer to create_die_model in the <i>Voltus Text Command Reference</i>.
models	Writes the current model parameters using the <code>model</code> command, where the <code>model</code> command is written as follows: <pre>model layer <parameters = values>+</pre>
output_directory= <i>output_directory_name</i>	
	This parameter is equivalent to the following Voltus command parameter: <code>create_die_model -output_directory</code>. For more information, refer to create_die_model in the <i>Voltus Text Command Reference</i>.
power_domain_name= <i>power_domain_name</i> net= <i>net_name</i>	
	This parameter is equivalent to the following Voltus command parameter: <code>create_die_model -rail_analysis_domain domain_name -net net_name</code>. For more information, refer to create_die_model in the <i>Voltus Text Command Reference</i>.
port_grouping=[true false]	

	A value of <code>true</code> enables port grouping. The default is <code>true</code>. A value of <code>false</code> disables port grouping.
<code>powerpin_location_file</code>	Writes the voltage sources in a power pin location file format.
<code>probing_node_file=filename</code>	Specifies the probe node file containing the probe point on a die. Example: <pre>write reduced_model output_directory nport_dir state_directory dom12_25C_dynamic_1 probing_node_file probe_file.txt power_domain dom12</pre>
<code>vs</code>	Writes the voltage sources using the <code>addvs</code> command, where the <code>addvs</code> command is written as follows: <pre>addvs vs_name x y layer res resistance</pre>
<code>gridchange</code>	Generates a command file that you can use to restore modifications performed on the power grid. This file includes such commands as <code>changeres</code>, <code>addres</code>, <code>addvia</code>, <code>addmvia</code>, <code>select</code>, and <code>unselect</code>.
<code>filename</code>	Specifies the name of the command file.

<pre>block_boundary_ voltage instance_name filename</pre>	Writes the voltage of the interface nodes for a specific instance to the block boundary voltage file specified by <i>filename</i>. <i>Instance_name</i> indicates the name of the instance for which to specify voltage source locations. The block boundary voltage file contains the voltage at the boundaries of a specific instance in the design. You can use it as input to the analysis of the block itself by using the <code>block_boundary_voltage_file</code> variable instead of the <code>powerpin_location_file</code> variable to specify the location and the value of the voltage at the boundaries of the block. You can use this approach for the verification of hierarchical designs. At the full-chip level, use an abstract view of a block. Use the <code>write block_boundary_voltage</code> command to capture the boundary conditions of this block after the analysis. Then, with the <code>block_boundary_voltage_file</code> variable, verify the block by using the detailed view of the contents with the same boundary conditions as for the full-chip level. The format of the block boundary voltage file is as follows: <pre>putvsrc layer_name vsrc_name voltage x y</pre>
<pre>putvsrc</pre>	Adds a voltage source to the netlist at a specified location. • <i>layer_name</i> Specifies the name of the layer to which Lightning applies the voltage source. • <i>vsrc_name</i> Specifies the name of the voltage source. • <i>voltage</i> Specifies the value of the voltage source. • <i>x, y</i> Specify the coordinates of the location at which to place the voltage source. The <i>x, y</i> coordinates are in units of 1/1000 of a micron.
<pre>layer_name</pre>	Specifies the name of the layer to which Lightning applies the voltage source.

reduced_model	This parameter is equivalent to the following Voltus command parameter: <code>create_die_model -type n_port</code>. For more information, refer to create_die_model in the <i>Voltus Text Command Reference</i>.
repeat= <i>value</i>	This parameter is equivalent to the following Voltus command parameter: <code>create_die_model -repeat</code>. For more information, refer to create_die_model in the <i>Voltus Text Command Reference</i>.
state_directory= <i>state_directory_name</i>	
	This parameter is equivalent to the following Voltus command parameter: <code>create_die_model -state_directory</code>. For more information, refer to create_die_model in the <i>Voltus Text Command Reference</i>.
use_sigrity_repeat [= <i>value</i>]	
	This parameter is equivalent to the following Voltus command parameter: <code>create_die_model -use_sigrity_repeat</code>. For more information, refer to create_die_model in the <i>Voltus Text Command Reference</i>. The following list gives the use model mapping between Voltus and VoltageStorm for this parameter: • <code>-repeat value(Voltus) => write [abstract_model reduced_model] repeat=value(VoltageStorm)</code> • <code>-repeat value -use_sigrity_repeat (Voltus) => write [abstract_model reduced_model] use_sigrity_repeat=value(VoltageStorm)</code> • <code>-use_sigrity_repeat (Voltus) => write [abstract_model reduced_model] use_sigrity_repeat (VoltageStorm - implying infinity repeat)</code> • <code>no -repeat and no -use_sigrity_repeat (Voltus) => write [abstract_model reduced_model] (VoltageStorm)</code>

Voltus IC Power Integrity Solution Text Command Reference
Legacy EPS Options

<i>vsrc_name</i>	Specifies the name of the voltage source.
<i>voltage</i>	Specifies the value of the voltage source.
<i>x, y</i>	Specify the coordinates of the location at which to place the voltage source. The <i>x, y</i> coordinates are in microns.

`-watch`

Forces the command to write additional dynamic block boundary voltage waveforms for the instance specified, in case dynamic analysis has been or will be performed.

When using the `-watch` option the setting of the `save_transient_states` variable should be taken into consideration. Below are two examples to help clarify this;

The first line of the below example tells VoltageStorm not to save the transient states. Next it specifies that the block boundary voltage waveforms are to be saved in files `inst1.bbv` and `inst2.bbv` for `inst1` and `inst2` respectively. Finally the dynamic analysis is run. By setting up the analysis in this way, less memory will be used, but at the cost of not being able to generate movies.

```
setvar save_transient_states false
...
write block_boundary_voltage inst1 inst1.bbv -
watch
write block_boundary_voltage inst2 inst2.bbv -
watch
...
analyze vdd 1.8 dynamic 1.65 ...
```

In this second example the first line tells VoltageStorm to save all the transient states. Next dynamic analysis is performed. Because all the transients states were saved, there is no need to use the `-watch` option of the `write block_boundary_voltage` command. The boundary voltage waveforms will be extracted from the saved transient states.

```
setvar save_transient_states true
...
analyze vdd 1.8 dynamic 1.65 ...
...
write block_boundary_voltage inst1 inst1.bbv
write block_boundary_voltage inst2 inst2.bbv
```

<code>eco filename</code>	Generates a single TCL script with the specified file name containing ECO changes. Following is an example of this file: <pre>putvsrc metal_2 Vbc_1 1.688599e+00 1500.720 1556.320 putvsrc metal_2 Vbc_2 1.699221e+00 1500.720 0.440 putvsrc metal_2 Vbc_3 1.690036e+00 50.000 1556.440 putvsrc metal_2 Vbc_4 1.699057e+00 50.000 0.370</pre>
---------------------------	--

Examples

- The following line creates a command file named `cmd3` using the current filter settings:
`write filters cmd3`
- The next line writes commands to restore all the resistor modifications and additions to the `changes_for_ir.cmd` file:
`write gridchanges changes_for_ir.cmd`
- This line writes the voltage of the `chip/h2/U1_T1` instance to the `U1_T1.bbv` file:
`write block_boundary_voltage chip/h2/U1_T1 U1_T1.bbv`

Rail Analysis Variable

auto_scale_adjacent_via_layers

`setvar auto_scale_adjacent_via_layers [true | false]`

This variable enables VoltageStorm PE to support auto via scaling.

[true false]	When scaling resistance on a specific metal layer, via resistance on the adjacent via layers are automatically scaled with the same scaling factor. If set to <code>false</code>, via resistances are kept un-scaled. Default: <code>true</code>
----------------------	---

Examples

The below variable specifies to perform auto via scaling.

```
setvar auto_scale_adjacent_via_layers true
```

cell_area_capacitance

```
setvar cell_area_capacitance value
```

This variable specifies the value of the area-based capacitance to apply to the static power-grid views for dynamic analysis. VoltageStorm PE applies this capacitance to all leaf cells that do not contain capacitance information. You may want to use this variable to specify capacitance by area if no decoupling capacitance was extracted by LibGen.

Parameters

Value	Specifies the area-based capacitance to apply to the static power-grid views, in femtofarads per micron squared. The default is 0 femtofarads
-------	--

Examples

- The following example specifies an area-based capacitance of 1 femtofarad per micron squared:

```
setvar cell_area_capacitance 1 fF/um2
```

create_label_from_vsc

```
setvar create_label_from_vsrc [true|false]
```

This variable determines whether VoltageStorm PE automatically generates a label that identifies a net from the GDS file. Generate net labels only if the GDS file does not contain attributes or labels identifying the nets of interest for connectivity and parasitic extraction. Incorrectly placed voltage sources may cause false shorts when you generate net labels. Setting this variable is not required for a pure DEF flow.

Parameters

[true false]	When this variable is set variable is set to <code>true</code>, VoltageStorm PE automatically generates labels that identify nets from the GDS file. If you set this variable to <code>false</code>, VoltageStorm PE does not automatically generate net labels. The default is <code>false</code>.
--------------	--

Examples

- In the following example, VoltageStorm PE creates net labels for unidentified nets from the GDS file:

```
setvar create_label_from_vsrc true
```

decap_exclude_static_violation_area

```
setvar decap_exclude_static_violation_area [true|false]
```

This variable specifies if the areas where static IR-drop violation occur are either to be excluded or to be included during `dr decap` analysis.

Parameters

[true false]	When this variable is set to <code>true</code>, VoltageStorm PE will exclude areas where static IR-drop violations occur. When this variable is set to <code>false</code>, VoltageStorm PE will not exclude areas where static IR-drop violations occur. The default is <code>true</code>.
--------------	---

Examples

- In the following example, areas where static IR-drop violations occur will be excluded when doing `dr decap` analysis:

```
setvar decap_exclude_static_violation_area true
```

decap_window_size

```
setvar decap_window_size value
```

This variable controls the size of the windows used in the vs2fe_eco.tcl ECO file for adding the decoupling capacitance.

Parameters

<i>value</i>	This variable defines the window size in microns when performing decoupling capacitance analysis (with dr option). The default is 200 microns.
--------------	--

Examples

- In the following example, VoltageStorm PE sets the window size to 100 microns for adding decoupling capacitance:

```
setvar decap_window_size 100
```

DEF_merge_supply_shorts

```
setvar DEF_merge_supply_shorts [true|false]
```

This variable determines how VoltageStorm PE handles any physical shorts that it detects in the DEF file. It can either merge the geometries of the shorted nets, treating them as electrically connected, or it can ignore them.

Parameters

[true false]	When this variable is set to <code>true</code>, VoltageStorm PE merges the geometries of the shorted nets and treats them as electrically connected. When this variable is set to <code>false</code>, VoltageStorm PE ignores the geometries of the nets not of interest. The default is <code>false</code>.
--------------	---

When you set this variable to `true`, VoltageStorm PE merges the geometries of the shorted nets and treats them as electrically connected. When you set it to `false`, VoltageStorm PE ignores the geometries of the nets not of interest. The default is `false`.

Examples

- As a result of this line, VoltageStorm PE merges the geometries of any shorted nets:

```
setvar DEF_merge_supply_shorts true
```

em_part_check_current

```
setvar em_part_check_current [true|false]
```

A clip wire (tie-high or tie-low net) is connected by a small number of vias to the power stripe. The signal pin in a cell of a clip wire usually has almost zero current, so the via of a clip wire usually has no impact on the em check.

This variable provides the ability to discard the EM effect due to a clip wire.

Parameters

[true false]	A value of <code>false</code> tells VoltageStorm to discard clip wires for em check. A value of <code>true</code> will tell VoltageStorm to look at the current to see if it should be ignored. See "em_part_current_threshold" for setting the threshold value. The default is <code>false</code>.
--------------	--

Examples

- This example tells VoltageStorm not to check the current:

```
setvar em_part_check_current false
```

em_part_current_threshold

```
setvar em_part_current_threshold threshold
```

This sets a threshold to be considered when "[em_part_check_current](#)" is set to true.

Parameters

<i>threshold</i>	The threshold. The default is 1.0e-10A.
------------------	---

Examples

- This example sets the threshold to 0.5e-10
`setvar em_part_current_threshold 0.5e-10.`

error_on_op_analysis

```
setvar error_on_op_analysis [true|false]
```

This variable determines whether VoltageStorm PE issues an error message and terminates if the average power consumption is used for a dynamic analysis and the IR drop is outside the IR-drop limit, indicating an existing weakness in the power grid. You can use this variable during optimization of the power grid to override the peak power-consumption values that VoltageStorm PE uses by default. This variable only takes effect if the `use_average_power_for_op` variable is set to `true`. See "[use_average_power_for_op](#)" for more information on the `use_average_power_for_op` variable.

Parameters

[true false]	When this variable is set to <code>true</code>, VoltageStorm PE issues an error message and terminates if the average power consumption is used for a dynamic analysis and the IR drop is outside the IR-drop limit. When this variable is set to <code>false</code>, VoltageStorm continues processing if the IR drop is outside the IR-drop limit. The default is <code>false</code>.
--------------	--

Examples

- In this example, VoltageStorm PE issues an error message and terminates when the IR drop is outside the IR-drop limit:

```
use_average_power_for_op true
```

```
setvar error_on_op_analysis true
```

extractor

```
setvar extractor executable_name
```

This variable specifies the executable to use for DEF- or GDS-based resistance extraction. In most cases, this

executable is `zx`, which is Cadence's next-generation Fire & Ice extractor.

Parameters

<code>executable_name</code>	Specifies the executable to use for DEF- or GDS-based resistance extraction. The default executable name is <code>zx</code>, located in the installation directory.
------------------------------	---

Examples

- The following example selects the Fire & Ice ZX extractor located in `../bin` as the resistance extractor:

```
setvar extractor ../bin/zx
```

extractor_include

```
setvar extractor_include filename
```

This variable specifies the name of a file containing commands to submit to the DEF or GDS extractor.

Parameters

<code>filename</code>	Specifies the name of a file containing commands to submit to the DEF or GDS extractor. This variable has no default setting.
-----------------------	---

Examples

- The following example identifies a file called `ptgn_def_cmd`, which is to be input to the DEF extractor.

```
setvar extractor_include ptgn_def_cmd
```

halt_on_net_with_no_cell_connections

```
setvar halt_on_net_with_no_cell_connections [true | false]
```

This variable specifies if an analysis of a power grid without cell connections should halt or continue.

Parameters

[true false]	If this variable is set to <code>true</code> the analysis of a power grid without cell connections will not continue. If this variable is set to <code>false</code> the analysis of a power grid without cell connections will continue. This is useful to analyze feed-through power grids. The default value is <code>true</code>.
--------------	---

Examples

- This line causes VoltageStorm to halt on nets with no cell connections.

```
setvar halt_on_net_with_no_cell_connections true
```

hierarchy_char

```
setvar hierarchy_char character
```

This variable specifies the separator between the net names in a hierarchical block.

Parameters

<i>character</i>	Specifies the separator between the net names in a hierarchical block. The default is a slash (/).
------------------	--

Examples

- The following example sets the net name separator to be a pipe slash:

```
setvar hierarchy_char |
```

An example of a resulting net name is `top|net23|net23_51`.

internal_nodes_visible

```
setvar internal_nodes_visible [true|false]
```

This variable determines whether the internal cell power-grid nodes are visible during analysis when the analysis type is `ir` or `tc`.

Parameters

[true false]	A value of <code>true</code> renders the internal cell and macro power-grid nodes visible and checks their voltage. A value of <code>false</code> does not make them visible or check them. The default is <code>true</code>.
--------------	--

Examples

- The example that follows makes the internal cell and macro power-grid nodes invisible and does not check their voltage levels:

```
setvar internal_nodes_visible false
```

internal_resistors_visible

```
setvar internal_resistors_visible [true|false]
```

This variable determines whether the internal cell power-grid resistors are visible and checks their current density and electromigration risk when the analysis type is `rj`, `er`, `rc`, or `vv`.

Parameters

[true false]	A value of <code>true</code> renders the internal cell and macro power-grid resistors visible and checks their current density and electromigration risk. A value of <code>false</code> does not check them or make them visible. The default is <code>true</code>.
--------------	--

Examples

- The following example makes the internal cell and macro power-grid resistors visible and checks their current density and electromigration risk:

```
setvar internal_resistors_visible true
```

libgen_executable

```
setvar libgen_executable executable_name
```

This variable specifies the name of the LibGen executable. LibGen is the library generator used by VoltageStorm PE when it builds a power-grid view library during execution of the `generate_view` command.

Parameters

<i>executable_name</i>	The name of the LibGen executable to use. The default executable name is <code>libgen</code>
------------------------	--

Examples

- The following line specifies a LibGen executable of `libgen`:

```
setvar libgen_executable ../exec/libgen
```

libgen_include

```
setvar libgen_include filename
```

This variable specifies the name of a file containing any extra LibGen commands to pass to LibGen when the `generate_view` command in VoltageStorm PE invokes LibGen to create a power-grid view.

Parameters

<i>filename</i>	The name of the file containing the LibGen commands.
-----------------	--

Examples

- As a result of the following example, VoltageStorm PE passes the commands in the `pgv_u1236.txt` file to LibGen when the `generate_view` command is used to invoke LibGen:
`setvar libgen_include pgv_u1236.txt`

lifetime

```
setvar lifetime value
```

This variable specifies the expected lifetime of the design in years. It is useful in assessing electromigration risk.

Parameters

<i>value</i>	Specifies the expected lifetime in years. The default value is 10 years.
--------------	---

Examples

- The following example indicates that the expected lifetime of the design is 15 years:
`setvar lifetime 15`

max_error_messages

```
setvar max_error_messages number
```

This variable sets the maximum number of error messages issued by VoltageStorm PE during the execution of a single command.

Parameters

<i>number</i>	This variable sets the maximum number of error messages issued by VoltageStorm PE during the execution of a single command. It must be an integer in the range of 1 through 10,000, inclusive. If an error occurs, VoltageStorm PE reports the total number of errors found after it executes a command The default is 100.
---------------	---

Examples

- The following example sets the maximum number of error messages at 500:

```
setvar max_error_messages 500
```

max_warning_messages

```
setvar max_warning_messages number
```

This variable sets the maximum number of warning messages that VoltageStorm PE issues during the execution of a single command.

Parameters

<i>number</i>	This variable sets the maximum number of warning messages issued by VoltageStorm PE during the execution of a single command. It must be an integer from 1 through 10,000, inclusive. The default is 100.
---------------	---

Examples

- The following example sets the maximum number of warning messages at 50:

```
setvar max_warning_messages 50
```

merge_util

```
setvar merge_util executable_name
```

This variable specifies the name of the merging utility that will generate the RC database.

Parameters

<i>executable_name</i>	This variable specifies the name of the merging utility that will generate the RC database. The default executable is <code>mergegrid</code> .
------------------------	---

Examples

- Here is an example showing MergeGrid as the name of the utility that generates the RC database:

```
setvar merge_util ../bin/mergegrid
```

pin_char

```
setvar pin_char character
```

This variable specifies a character used internally by VoltageStorm PE to separate instance names and pin names.

Parameters

<i>character</i>	This variable specifies a character used internally by VoltageStorm PE to separate instance names and pin names. The default is a colon (:). If a colon is part of any pin name or instance name, select another character to use as the internal separator. Do not use a backslash (\) as the separator character; it causes VoltageStorm PE to issue an error message.
------------------	--

Examples

- An example is a design with a pin named `vdd:` or an instance named `adder:2`. Because the pound sign character (#) is not used by any pin name or instance name in the design, the following syntax selects it as the separator character:

```
setvar pin_char #
```

precision_digits

```
setvar precision_digits number_of_digits
```

This variable specifies the number of decimal places in the instance-based supply voltage data generated by the `print iv` command. This variable enables you to control the precision of the output. If you do not set this variable, VoltageStorm PE generates the output with four digits of precision.

Parameters

<i>number_of_digits</i>	Specifies the number of decimal places in the instance-based supply voltage data generated by the <code>print iv</code> command. It is an integer. There is no default value.
-------------------------	--

Examples

- As an example, this line specifies values of six digits:

```
setvar precision_digits 6
```

The output values will appear with six digits to the right of the decimal point, like this:

```
2.996926
```

prpgv_x_offset

```
setvar prpgv_x_offset value
```

This and `prpgv_y_offset` variables specify the offset in the X and Y directions, respectively, between the power-grid database data and the LEF file data, in microns. They are used to align these two sets of data in generating the hierarchical power-grid view. Often the DEF file input to VoltageStorm PE is specified in a centered coordinate system, and the LEF file for the cell is specified in a first-quadrant coordinate system (that is, its lower left corner is at (0 0)). VoltageStorm PE passes these two variables to LibGen. When LibGen loads power-grid view data, it adds the values of `prpgv_x_offset` and `prpgv_y_offset` to the location of the nodes to place them in the LEF coordinate system.

Parameters

<i>value</i>	This variables specify the offset in the X direction, between the power-grid database data and the LEF file data, in microns. The default value f is 0 microns.
--------------	--

Examples

- Given these specifications in the DEF file for a cell:

```
UNITS DISTANCE MICRONS 100
```

```
DIEAREA ( -5000 -5000 ) ( 5000 5000 )
```

and this data in the LEF MACRO for the same cell:

```
ORIGIN ( 0 0 )
```

```
SIZE 100 BY 100
```

the appropriate setting of these variables would be as follows:

```
setvar prpgv_x_offset 50
```

```
setvar prpgv_y_offset 50
```

These settings would shift the power-grid data extracted from the DEF file 50 microns to the right and 50 microns up to align with the LEF macro data.

prpgv_y_offset

```
setvar prpgv_y_offset value
```

The `prpgv_x_offset` and `prpgv_y_offset` variables specify the offset in the X and Y directions, respectively, between the power-grid database data and the LEF file data, in microns. They are used to align these two sets of data in generating the hierarchical power-grid view. Often the DEF file input to VoltageStorm PE is specified in a centered coordinate system, and the LEF file for the cell is specified in a first-quadrant coordinate system (that is, its lower left corner is at (0 0)). VoltageStorm PE passes these two variables to LibGen. When LibGen loads power-grid view data, it adds the values of `prpgv_x_offset` and `prpgv_y_offset` to the location of the nodes to place them in the LEF coordinate system.

Parameters

<i>value</i>	This variables specify the offset in the Y direction, between the power-grid database data and the LEF file data, in microns. The default value f is 0 microns.
--------------	--

Examples

- Given these specifications in the DEF file for a cell:

```
UNITS DISTANCE MICRONS 100
```

```
DIEAREA ( -5000 -5000 ) ( 5000 5000 )
```

and this data in the LEF MACRO for the same cell:

```
ORIGIN ( 0 0 )
```

```
SIZE 100 BY 100
```

the appropriate setting of these variables would be as follows:

```
setvar prpgv_x_offset 50
```

```
setvar prpgv_y_offset 50
```

These settings would shift the power-grid data extracted from the DEF file 50 microns to the right and 50 microns up to align with the LEF macro data.

reff_ignore_disconnected_pins

```
setvar reff_ignore_disconnected_pins [true | false]
```

Ignores disconnected pins when doing effective resistance (reff) analysis. Reff analysis determines the effective resistance from voltage source to an instance pin.

[true false]	If set to <code>true</code> , disconnected pins will be ignored for reff (effective resistance) analysis. Default: <code>false</code>
----------------	--

Examples

- The below variable specifies that disconnected pins should be ignored when doing reff analysis.

```
setvar reff_ignore_disconnected_pins true
```

use_average_power_for_op

```
setvar use_average_power_for_op [true|false]
```

This variable determines whether the average power consumption is used as the operating-point value--that is, time 0--for dynamic analysis.

Parameters

[true false]	When this variable is set to <code>true</code> , VoltageStorm PE uses the average power consumption as the operating-point value for dynamic analysis. When this variable is set to <code>false</code> , VoltageStorm PE initializes itself at time 0. The default is <code>false</code> .
--------------	--

Examples

- In the following example, VoltageStorm PE uses an operating-point value of 0:

```
setvar use_average_power_for_op false
```

use_capacitor_adjustment

```
setvar use_capacitor_adjustment [true|false]
```

This variable specifies whether coupling capacitance effects between power and ground are taken into account or not.

Parameters

[true false]	When this variable is set to <code>true</code> coupling capacitance effects between power and ground are taken into account. When this variable is set to <code>false</code> coupling capacitance effects between power and ground are not taken into account. The default is <code>false</code>.
--------------	--

Examples

- In the following example, VoltageStorm PE takes into account the coupling capacitance between power and ground:

```
setvar use_capacitor adjustment true
```

use_current_data_from_pgv

```
setvar use_current_data_from_pgv [true|false]
```

This variable determines if the current data should be taken from the cell library.

Parameters

[true false]	When this variable is set to <code>true</code> and no <code>cell_power_file</code> was provided, then the current data from the cell library will be used. If a <code>cell_power_file</code> was provided, and this file contains data for the cell in question, then it should override the current value stored in the cell library. When this variable is set to <code>false</code>, then the current data from the cell library will not be used. The default is <code>false</code>.
--------------	---

Examples

- In the following example, VoltageStorm PE uses the current data from the power grid view if there is no `cell_power_file` or the data is not provided in the `cell_power_file`:

```
setvar use_current_data_from_pgv true
```

use_existing_rcdb

```
setvar use_existing_rcdb [true|false]
```

This variable allows you to use an RC database generated in a previous VoltageStorm PE run. You may want to use such a database for testing purposes, to skip the resistance extraction step, or in case the power-grid merging step failed during a VoltageStorm PE run because of missing power-grid views.

Even if this variable is set to `true`, the resistances are automatically re-extracted if the timestamp on the DEF file is later than the timestamp on the RC database, if the timestamp on the cell library is more recent than the timestamp for the RC database, or if the setting of the `pin_char` has been changed.

Parameters

.

[true false]	When this variable is set to <code>true</code> , <code>VoltageStorm</code> uses an RC database previously generated by VoltageStorm PE. If specified as <code>false</code> it generates a new RC database. The default is <code>true</code> .
--------------	---

Examples

- The following example generates a new RC database:

```
setvar use_existing_rcdb false
```

use_gds_annotate_attributes

```
setvar use_gds_annotate_attributes [true|false]
```

This variable determines whether VoltageStorm PE reads the attributes in the GDS file.

Parameters

[true false]	When this variable is set to <code>true</code>, VoltageStorm PE reads the attributes in the GDS file. When you set it to <code>false</code>, it does not read the attributes in the GDS file. The default is <code>false</code>.
--------------	---

Examples

- In the following case, VoltageStorm PE does not read the attributes in the GDS file:

```
setvar use_gds_annotate_attributes false
```

use_port_voltage_for_iv

```
setvar use_port_voltage_for_iv [true|false]
```

This variable controls whether to use port voltage to report Instance Voltage (IV) at the port for instances in the detailed view.

Parameters

[true false]	When this variable is set to <code>true</code>, VoltageStorm PE uses the port/interface node voltages to report IV at the port for instances in the detailed view. When you set it to <code>false</code>, VoltageStorm PE does not use the port voltage for IV calculation and reporting. The default is <code>false</code>.
--------------	---

write_bbv_pwl

```
setvar write_bbv_pwl [true | false]
```

This variable controls the pwl boundary waveform output.

Parameters

[true false]	When set to <code>true</code>, it writes the block boundary voltages in the pwl format, into a new *.pwl file. <i>Default:</i> <code>true</code>
--------------	--

write_decap_filler_voltage

```
setvar write_decap_filler_voltage [true | false]
```

This variable disables waveform and iv generation for filler and decap cells.

Parameters

[true false]	When set to <code>false</code>, the variable disables waveform and iv generation for filler and decap cells. <i>Default:</i> <code>false</code>
--------------	---

Voltus Utilities

- [convert_gds_to_def](#)
- [map_dies](#)

convert_gds_to_def

```
convert_gds_to_def
[-help]
[-black_box_cell_file filename]
-gds_file filename
-gds_layermap filename
[-esd_cell_bbox_file filename]
[-esd_marker_layer_file filename]
[-extract_signal_net_logic_info_only]
[-ignore_gds_inst_name]
[-inst_name_file filename]
-net_location_file filename
[-inst_name_prefix prefix]
[-output_components {ALL | NONE | CONN}]
-output_def_file filename
[-output_lef_macros filename]
-power_grid_library libraryname
-top_cell cellname
[-print_complete_short_path]
[-spef_file filename]
[-tech_gds_layermap filename]
[-use_gds_label {ALL | TOP | <tech_layer_name> | <cell_name>:<tech_layer_name> |
<cell_name>:ALL}]
[-verilog_file filename]
[-zero_pin_size]
```

A GDS2DEF utility that converts a given GDSII file to a DEF file for power/rail signoff analysis. The utility can convert both cell-level GDS and full-chip level GDS to DEF. The generated DEF contains all the geometries for the P/G routing, the specified P/G pin information, component and logical

connectivity information.

Parameters

<code>- help</code>	Outputs the command usage.
<code>-black_box_cell_file filename</code>	
	Specifies the name of a file containing a list of black box cells that the utility is to ignore. You can specify both exact string matches and wildcard searches in the specified file. For exact string matches, each cell name must reside on a separate line in the file, as shown below: <pre>CellA CellB</pre> The wildcard search feature allows you to search cell names that contain wildcard characters, such as square brackets and underscore. When you want to perform wildcard search, type the keyword <code>wildcard</code> first, followed by the cell name. For example, if you want to ignore cell names which contains CLK, add one line <code>wildcard .*CLK.*</code>.
<code>-esd_marker_layer_file filename</code>	

Specifies a file that includes a list of ESD cell/instance names and their corresponding GDS marker layers that represent the bounding box of the ESD cells. The format of the file is:

```
<cell_name> <marker_layer> [top_layer] [bottom_layer]
```

Here,

- `cell_name` is the ESD cell/instance name
- `marker_layer` could contain multiple GDS layers, separated by “;”
- The optional `top_layer` and `bottom_layer` specifies the top and bottom technology layers of the ESD cell pin shape. You can use these optional arguments to specify the top and bottom technology layers of the ESD cell pin shape. This provides you with the flexibility to specify the lowest layers to be used for creating pins in the LEF. In the following example, only the shapes between M3 and M1 layers are ESD cell pin shapes:

Marker file example:

```
ESD_diode 168;255:3 M3 M1
ESD_clamp 122 M3 M1
```

`-esd_cell_bbox_file filename`

Specifies a file that includes the bounding box for each ESD cell. The format of the file is:

```
<cell_name> <x1 y1 x2 y2> [top_layer] [bottom_layer]
```

Here,

- `cell_name` is the ESD cell/instance name
- `x1 y1 x2 y2` are the bounding box coordinates
- The optional `top_layer` and `bottom_layer` specifies the top and bottom technology layers of the ESD cell pin shape. You can use these optional arguments to specify the top and bottom technology layers of the ESD cell pin shape. This provides you with the flexibility to specify the lowest layers to be used for creating pins in the LEF. In the following example, only the shapes between AP and M11 layers are ESD cell pin shapes:

Bbox file example:

```
ESD_diode_0 1281296 860923 1296972 895445 AP M11
ESD_diode_1 1281296 898651 1296972 933173 AP M11
```

<code>-extract_signal_net_logic_info_only</code>	
	Specifies to print only the logical connectivity for the signal nets in the <code>NETS</code> section of the output DEF file. If this parameter is not specified, the software prints both the logical connectivity and routing shapes for the signal nets in the <code>SPECIALNETS</code> section of the DEF file.
<code>-gds_file filename</code>	
	Mandatory parameter. Specifies the name of the input GDS file. The top cell name in the GDSII file is the design name in the generated DEF.
<code>-gds_layermap filename</code>	
	Mandatory parameter. The layer mapping file between GDS and tech layers. All the layers defined in a given mapping file is extracted. The layers not defined in the mapping file are ignored. The format of the layer mapping file is: <pre><layer_type> <tech layer name> <gds> <gds layer#> [gds layer data type] [PORT] [TEXT_ONLY] <mask keyword> [dummy]</pre> The format of the layer mapping file to identify the die area in the output DEF is: <pre><layer> <prBoundary> <gds> <gds layer#> [gds layer data type]</pre> <u>Format Description:</u> <code><layer_type></code> specifies the type of library layer. The various layer types are: via, metal, and layer. <code><tech layer name></code> specifies the name of the library layer to map to the GDSII layer number. The library layers are technology layers which can be dumped using the Techgen <code>-techinfo</code> command. <code><gds></code> specifies the keyword for the GDS layer mapping. <code><gds layer#></code> specifies the layer number of the GDSII layer. <code>[gds layer data type]</code> specifies the optional number of the subclass for the GDSII layer. If you do not specify a datatype, the software uses all datatypes for that GDSII layer. If GDS datatype is not defined, the software considers it as a regular layout routing layer where all the shapes will be extracted. The GDS layer may have additional information/attribute assigned using the following keywords:

- **PORT:** Converts text labels in this layer to ports.
- **TEXT_ONLY:** Includes only the text in the named layer and ignores all geometries.

`<mask keyword>` keyword used to define the back-end metal layers with color masks (MASK A and Mask B).

`[dummy]` indicates that the layer is a metal fill layer. The `convert_gds_to_def` utility identifies these metal fill shapes and writes them to the `FILLS` section in the output DEF.

`<prBoundary>` specifies that the shapes on the `prBoundary` (Place-and-Route Boundaries) layer are used to determine the die area in the output DEF. This keyword can be used if you want to have the required die area in the output DEF file. If the `prBoundary` keyword is not specified, the die area is determined by the routing shapes specified in the GDS file, cell size, and the pin shapes of all the layers.

Example1:

```
metal METAL1 gds 15
via VIA1 gds 16
```

Example2:

The following is a sample layermap file with the mask keywords (`mask_a` and `mask_b`):

```
metal METAL1 gds 31 250
metal METAL1 gds 31 255 mask_a
metal METAL1 gds 31 256 mask_b
```

Here, the mask and main layers are defined using the same GDS layer number 31, but uses a different GDS data type (250, 255, and 256).

Example3:

The following is a sample layermap file with the `dummy` keyword:

```
metal METAL5 gds 35 367 dummy
```

Here, the layer with the GDS layer ID 35 and data type 367 contains the metal fill shapes only.

Example4:

The following is a sample layermap file with the `prBoundary` keyword:

```
layer prBoundary gds 62 21
```

```
-ignore_gds_inst_name
```

	Specifies to ignore the instance names from the specified GDS file, and the software will then automatically assign instance names to all the instances in the output DEF and Verilog files. This parameter can be used if instance names in the GDS file cannot be used, primarily in cases, such as duplicate instance names.
<code>-inst_name_file filename</code>	
	Specifies the file name containing instance names for the generated DEF in the following format: <pre><x coord> <y coord> <instance name></pre> The x y coordinates are in the unit of micron. The priority of getting an instance name is: 1. instance name file 2. GDS label 3. internally assigned name The <code>-inst_name_file</code> parameter allows to provide user-specified instance names. The <code>convert_gds_to_def</code> utility reads the GDS label to decide the instance name. If there are no labels, the software internally assigns instance names.
<code>-inst_name_prefix prefix</code>	
	Specifies a prefix for all the instance names in the output DEF and Verilog files.
<code>-net_location_file filename</code>	

Mandatory parameter. Specifies the name of the net location file containing a list of pin locations for nets that needed to be considered. All the geometries of the P/G nets as well as the instance pins connected to the specified P/G nets are included in the `SPECIALNETS` section of DEF. The instances connected to the P/G nets are included in the `COMPONENTS` section of DEF.

The format of the net location file is:

```
<net name> <x loc of pin> <y loc of pin> <layer> <POWER|GROUND|SIGNAL> [pin name]
```

Example:

```
VDD 60 73 METAL3 POWER
VSS 75 40 METAL2 GROUND
```

Here,

- The x/y location in the file has the unit of micron.
- The layer name is the tech layer name.
- The type of a given net can only be POWER, GROUND, or SIGNAL. If the net type is marked as `SIGNAL`, the net will be traced from the GDS file and included in the `SPECIALNETS` section with “USE SIGNAL” in the output DEF file. In addition, the signal pin connectivity is specified for the net.
- The pin name field is optional. If there is no pin name field, the pin name will be same as the net name.

If you do not provide pin locations, the utility uses the GDS text labels in either the top-level cell or cells throughout the hierarchy to get the location. In such a case, the format of net location file is:

```
<net name> <POWER|GROUND|SIGNAL>
```

Example:

```
VDD POWER
VSS GROUND
```

```
-output_components {ALL | NONE | CONN}
```

	Specifies the components to be printed in the DEF file <code>COMPONENTS</code> section during GDS to DEF conversion. The default is <code>ALL</code>. • <code>ALL</code>: Specifies to output all the instances in the DEF Components section. • <code>NONE</code>: Specifies not to print instances. • <code>CONN</code>: Specifies to output instances with logical connectivity only.
<code>-output_def_file filename</code>	
	Mandatory parameter. Specifies the name of the output DEF file. If the given DEF name is specified with “.gz”, the software will gzip the DEF file to reduce the file size.
<code>-output_lef_macros filename</code>	
	Specifies the name of the file containing a list of macros for which LEF files are to be generated. One LEF file is generated for each macro. The generated LEF pin shapes are all shapes that connect to top-level nets on all metal/via layer shapes. The cell boundary of the LEF macro is derived by the bounding box of all shapes within the GDS cell.
<code>-power_grid_library libraryname</code>	
	Mandatory parameter. Specifies the name of the merged power-grid library (.cl). You can specify only a single .cl library.
<code>-print_complete_short_path</code>	
	Specifies to print the complete shorting path when there is a short between two nets. When this parameter is specified, the entire shorting path is assigned to one of the shorted nets and the path is printed in the generated DEF file. The shapes of the shorting path will be assigned to the net that is specified first in the net location file. For example, if a tracing point of VDD is shorted with a tracing point of VSS, the software honors the net that is defined first in the net location file. If this parameter is not specified, the software assigns the shapes of the path to either of the shorted nets by default.
<code>-spef_file filename</code>	

Specifies the name of the SPEF file that can be used to include the signal nets in the generated DEF file. The input SPEF file is required to have layer annotation in the `RES` statements to indicate the pin node layers.

For the signal nets, both the `*P` and `*I` nodes are used as the starting tracing points, and the SPEF file pins (`*P`) are honored as the top-level pins and are reported in the `PINS` section in the output DEF file. For the PG nets, the `*P` nodes and the user-defined locations in the net location file are used as the starting tracing points. The SPEF file connections (`*I` statements) are honored when generating the DEF logic connectivity for both the signal and PG nets. That is, the `convert_gds_to_def` utility relies on the SPEF file for logic connection if a SPEF file is specified. The instance and pin names are from the SPEF `*I` statements. The output DEF file includes all the signal net pins from `*P` in the `PINS` section, the signal routing shapes in the `NETS` section, and the PG net routing shapes in the `SPECIALNETS` section.

In addition to the SPEF file, you need to specify the following parameters for including the signal nets in the DEF file:

- `-gds_file`
- `-net_location_file`
- `-power_grid_library`
- `-gds_layermap`
- `-output_def_file`
- `-top_cell`

Notes:

- The `-net_location_file` parameter is used to identify the power and ground nets, and therefore the remaining nets are considered as signal nets.
- The `-power_grid_library` specifies the name of the power-grid library that includes the list of standard cells or macros that should be consistent with the SPEF file. That is, the cell of the instances in the SPEF file should be defined in the PGV.

`-tech_gds_layermap filename`

	Specifies a file that contains layer mapping between GDS and tech layers. This parameter removes the requirement to convert the DEF or LEF layers to qrcTechLayer for the layer map file. The format of the layer mapping file is: <pre><layer_name> <layer_type> <gds layer#> <gds layer data type></pre> <layer_type> specifies the type of def or lef layer. <tech layer name> specifies the name of the def or lef layer to map to the GDSII layer number. <gds> specifies the keyword for the GDS layer mapping. <gds layer#> specifies the layer number of the GDSII layer. Example: <pre>m0 TEXT 55 2</pre>
	<code>-top_cell cellname</code>
	Mandatory parameter. Specifies the top cell name in the GDSII file.
	<code>-use_gds_label {ALL TOP <tech_layer_name> <cell_name>:<tech_layer_name> <cell_name>:ALL}</code>
	Specifies the criteria for selecting text labels. • ALL: specifies to honor all the text labels from GDS. • TOP: specifies to honor just the top-level text labels. • <tech_layer_name>: specifies to honor all the text labels on the specified layer. • <cellname>:<tech_layer_name>: specifies to honor all the text labels of the specified GDS cell on the specified layer. • <cellname>:ALL: specifies to honor all the text labels of the specified GDS cell. The following example specifies to use the text labels of the cell "hierblock" on layer METAL10: <pre>convert_gds_to_def -use_gds_label hierblock:METAL10</pre>
	<code>-verilog_file filename</code>

	Specifies the name of the output verilog file. This parameter allows you to use the GUI to display the rail analysis results of the GDS-only flow.
<code>-zero_pin_size</code>	
	Specifies to set the pin shape to size zero "(0 0)(0 0)" in the output DEF file. You can use this parameter when the port shape is outside the drawn shape (metal geometry) at places where the port label is defined near the edge of the geometry.

Examples

- The following command converts the specified GDS file `new.gds` to a DEF file `top.def`.

```
convert_gds_to_def -net_location_file net.loc -gds_file new.gds
-power_grid_library ../pv_library.cl -gds_layermap lmap -top_cell cella -
output_def_file top.def
```

- The following command converts the specified GDS file `chip.gds` to a DEF file `my.def`:

```
convert_gds_to_def -gds_file chip.gds.gz -power_grid_library
{pgv/techonly/techonly.cl pgv/full.cl} \
-net_location_file netlocfile -gds_layermap layermap -
output_def_file my.def -top_cell chip -spef_file chip.spef.gz
```

Here, the SPEF file `chip.spef` is specified to include the signal nets in the output DEF file.

- The following command specifies a marker layer file `esdcells` to enable ESD analysis using

```
convert_gds_to_def:
convert_gds_to_def -gds_file gw16_500_usr_test.gds -power_grid_library {
/../../TECH/techonly.cl } -net_location_file my.nlf -gds_layermap gds_layer.map \
-output_def_file my.def -top_cell gw16_500_usr -use_gds_label ALL -
esd_marker_layer_file esdcells
```

`convert_gds_to_def` utility generates a LEF file for each ESD cell specified in the `esdcells` file to enable PGV generation. The output DEF includes the user-defined nets and the ESD instances based on the specified marker layer file.

Related Topics

- "[GDS2DEF Utility](#)" in the *Voltus User Guide*

map_dies

```
map_dies
[-help]
-3dic_design_stack_file filename
[-component_config_file filename]
[-die_pad_file_pair {{<die_name1> <pad file1>} {<die_name1> <pad file2>}...}]
[-map_search_distance distance]
[-mode sanity_check]
[-output directory]
[-stacked_die_mapping filename]
```

A utility to generate the inter die data mapping file based on the design stack configuration file and die ploc files. This mapping file is required if there is a direct connection between dies. The generated mapping file defines the connection between dies (between interposer and logic die). This mapping file is then specified with the `set_multi_die_analysis_mode -stacked_die_mapping` command for multi-die analysis.

Parameters

- help	Outputs the command usage and a brief description about the command parameters.
-3dic_design_stack_file filename	
	Specifies the name of the XML file containing the design stack information. This file is from the Integrity™ 3D-IC product. It includes each die’s tier level (Z direction), die instance name, size (post shrink size if there is a scale factor), flip, rotation (counter, clockwise), placement location, and contact layer.
-component_config_file filename	

	Specifies the name of the design configuration file. This file is used to define the power pad location files, and the power and ground net names. File Format: <pre> COMPONENT IDIE Power_pad { {ldie_VDD.ploc VDD_AO} {ldie_VSS.ploc VSS} } [Power_net { VDD_AO}] [Ground_net {VSS}] END COMPONENT LDIE Power_pad { {ldie_ALL.ploc all} } [Power_net {VDD1}] [Ground_net {VSS2}] END </pre>
	<pre>-die_pad_file_pair {{<die_name1> <pad file1>} {<die_name1> <pad file2>}...}</pre>
	Specifies the mapping between die instance names and their corresponding power pad file for the interposer die and logic dies.
	<pre>-map_search_distance distance</pre>
	Specifies the legal mapping distance range between the interposer and legal dies. The unit for distance is micrometer (um).
	<pre>-mode sanity_check</pre>
	Specifies to perform sanity checks for the generated mapping file.
	<pre>-output directory</pre>

	Specifies the output directory containing the mapping results. The output folder contains the generated mapping file (<code>die_die.map</code>). The format of the 6-column mapping file is: <pre>IDIE_INST_NAME(idie_inst_name) Inet_name Ibump_name LDIE_INST_NAME(ldie_inst_name) Lnet_name Lbump_name</pre> Here, • The first 3 columns are for the interposer die instance name, net name, and bump name(vsrc name), and the last 3 columns are for the logic die instance name, net name, and bump name. • The die instance names are written in upper case that is followed by the die instance names specified in the rail TCL for each die instance name. • The file is split in two sections, POWER and GROUND.
	<pre>-stacked_die_mapping filename</pre>
	Specifies the mapping file used to define the connection between dies (between interposer and logic die). This file provides the information on bump-to-bump mapping (one bump from interposer die and the other bump from logic die) to provide logical connectivity. The format of the 6-column mapping file is: <pre>IDIE VDD_AO BUMP_PWR_1 LDIE1 VDD_AO LDIE_PWR_BUMP7 IDIE VDD_AO BUMP_PWR_2 LDIE1 VDD_AO LDIE_PWR_BUMP8 ...</pre> Here, the first 3 columns are for the interposer die instance name, net name, and bump name, and the last 3 columns are for the logic die instance name, net name, and bump name.

Output

The following is the list of reports that are generated for sanity checking:

- `die_map.summary` - This report has 2 sections:
 - Interface mapping summary - Lists all the `die_inst1/net` to `die_inst2/net` mapping pairs, and their matched bump mappings in the overlap region.

- Die connect-terminal summary - Reports matched and total terminals for each die instance/net/layer group.
- `map_diagnosis.log` - Reports a list of unmapped bumps/terminals for a die-to-die pair list and the reasons for the unmapped bumps (For example, unknown die, unknown net, undefined terminal, power to ground short, and floating terminal).
- `terminal_map.rpt` - Lists mapping pairs (`die1:net1:bump1 :: die2:net2:bump2`) that pass the mapping and alignment checks, therefore are considered as legal mapping pairs.
- `terminal_offset.rpt` - Reports all mapping pairs where offset is greater than the specified `map_search_distance`.

Examples

- The following command generates a mapping file that includes the connection between dies (between interposer die INTERPOSER and logic dies DIE0, DIE1, DIE2, DIE3):

```
map_dies \  
-output results \  
-3dic_config_file ../data/design_stack.xml \  
-die_pad_file_pair { {INTERPOSER ../data/Interposer_all.ploc} \  
                    {DIE0 ../data/Die0_all.ploc} \  
                    {DIE1 ../data/Die1_all.ploc} \  
                    {DIE2 ../data/Die2_all.ploc} \  
                    {DIE3 ../data/Die3_all.ploc} }
```

- The following command performs the sanity check for the generated mapping file:

```
map_dies \  
-mode sanity_check \  
-output Results \  
-3dic_design_stack_file ../data/design_stack.xml \  
-3dic_config_file ../data/design_stack.xml \  
-component_config_file ../data/Components.config \  
-stacked_die_mapping ../data/die2die_bump.map \  
-map_search_distance 5 \  

```

Related Topics

- "[Package Analysis](#)" in the *Voltus User Guide*

RTL-to-Gate Rule Mapping

Rule Type	RTL	Gate	Remarks
Reg Pattern	Hier/flop	Hier/flop_reg	
	Hier/flop_reg	Hier/flop_reg_reg	
	Hier/flop[1]	Hier/flop_1	
		Hier/flop_x1x	
		Hier/flop 1	flop {space} 1
		Hier/flop\[1\]	
		Hier/flop_x1	
Reg + Index	Hier/flop[1]	Hier/flop_reg_1	
		Hier/flop_regx1x	
Multi Index	Hier/flop[1][2]	Hier/flop_1_2	
		Hier/flop 1 2	
		Hier/flop\[1\]\[2\]	flop {space} 1 {space} 2
		Hier/flop\[1\]_2	
		Hier/flop_1\[2\]	
		Hier/flop_1\[2\]	
		Hier/flop.1.2	
		Hier/flop.1_2	
		Hier/flop\[1\] 2	combination of '[' and ' ' for index separation
		Hier/flop\[1\].2	
		Hier/flop/1_2	
		Hier/flop/1/2	
		Hier/flop_x1xx2x	

Voltus IC Power Integrity Solution Text Command Reference
RTL-to-Gate Rule Mapping

		Hier/flop_1_xxx_2	_xxx_ is treated as a separator
Multi Bit Flip Flop	Hier/dff_Q_reg[1]	Hier/CDN_MBIT_dff_Q_reg\[1\]_MB_dff_Q_reg\ [2\]_MB_dff_Q_reg\[3\]\Q1	
	Hier/dff_Q_reg[2]	Hier/CDN_MBIT_dff_Q_reg\[1\]_MB_dff_Q_reg\ [2\]_MB_dff_Q_reg\[3\]\Q2	
	Hier/dff_Q_reg[3]	Hier/CDN_MBIT_dff_Q_reg\[1\]_MB_dff_Q_reg\ [2\]_MB_dff_Q_reg\[3\]\Q3	
	Hier/dff_Q_reg[1]	Hier/CDN_MBIT_dff_Q_reg\[1\]____dff_Q_reg\[2\]____dff_Q_reg\ [3\]\Q1	
	Hier/dff_Q_reg[2]	Hier/CDN_MBIT_dff_Q_reg\[1\]____dff_Q_reg\[2\]____dff_Q_reg\ [3\]\Q2	
	Hier/dff_Q_reg[3]	Hier/CDN_MBIT_dff_Q_reg\[1\]____dff_Q_reg\[2\]____dff_Q_reg\ [3\]\Q3	
	Hier/Q_reg[1]	Hier.CDN_MBIT_Q_reg\[1\]_MB_Q_reg\[2\]_MB_Q_reg\[3\]\Q1	
	Hier/Q_reg[2]	Hier.CDN_MBIT_Q_reg\[1\]_MB_Q_reg\[2\]_MB_Q_reg\[3\]\Q2	
	Hier/Q_reg[3]	Hier.CDN_MBIT_Q_reg\[1\]_MB_Q_reg\[2\]_MB_Q_reg\[3\]\Q3	
	Hier/q_reg[1]	Hier/CDN_MBIT_q_reg_CDN_CPX_\[1\]_MB_\[2\]_MB_\[3\]\Q1	
	Hier/q_reg[2]	Hier/CDN_MBIT_q_reg_CDN_CPX_\[1\]_MB_\[2\]_MB_\[3\]\Q2	
	Hier/q_reg[3]	Hier/CDN_MBIT_q_reg_CDN_CPX_\[1\]_MB_\[2\]_MB_\[3\]\Q3	
	Hier/prefix_Q_reg[1]	Hier/MBFF_prefix_Q_reg_[1]_MB_Q_reg_[2]/Q1	
	Hier/prefix_Q_reg[2]	Hier/MBFF_prefix_Q_reg_[1]_MB_Q_reg_[2]/Q2	
	Hier/prefix_Q_reg[1]	Hier/MBFF_prefix_Q_reg_CDN_CPX_[1]_MB_Q_reg_[2]\Q1	
	Hier/prefix_Q_reg[2]	Hier/MBFF_prefix_Q_reg_CDN_CPX_[1]_MB_Q_reg_[2]\Q2	
	Hier/prefix_Q_reg[1]	Hier/MBFF_prefix_Q_reg_[1]____Q_reg_[2]/Q1	
	Hier/prefix_Q_reg[2]	Hier/MBFF_prefix_Q_reg_[1]____Q_reg_[2]/Q2	
	Hier/prefix_Q_reg[1]	Hier/CDN_MBIT_MBFF_prefix_Q_reg_[1]_MB_Q_reg_[2]/Q1	
	Hier/prefix_Q_reg[2]	Hier/CDN_MBIT_MBFF_prefix_Q_reg_[1]_MB_Q_reg_[2]/Q2	
	Hier/prefix_Q_reg[1]	Hier/CDN_MBIT_MBFF_prefix_Q_reg_CDN_CPX_[1]_MB_Q_reg_[2]/Q1	
	Hier/prefix_Q_reg[2]	Hier/CDN_MBIT_MBFF_prefix_Q_reg_CDN_CPX_[1]_MB_Q_reg_[2]/Q2	
	Hier/prefix_Q_reg[1]	Hier/CDN_MBIT_MBFF_prefix_Q_reg_CDN_CPX_[1]____Q_reg_[2]/Q1	
	Hier/prefix_Q_reg[2]	Hier/CDN_MBIT_MBFF_prefix_Q_reg_CDN_CPX_[1]____Q_reg_[2]/Q2	

Voltus IC Power Integrity Solution Text Command Reference
RTL-to-Gate Rule Mapping

Hierarchy Merge	A/B	A_B	
Mixed Hierarchy	A/B/C	A/B_C	
		A_B/C	
Flatten Array	A[0]	A_0_	
	A[1]	A_1_	
Flatten Double Array	A[0][1]	A_0_1	
Flatten Array with Hierarchy Merge	A[0]/B	A_0_B	
Dot Separator	A/B	A.B	
Mixed Dot Separator	A/B/C	A/B.C	
		A.B/C	
Regex			
Continuous Index/Double Index	A/B/C[0]	A/B/C[0:0]	
Multiple __	A/B/C	A__B/C	
xxx	fifoctr[1]	fifoctr_reg_xxx_1	